Programação Linux Avançada
Autores:Mark Mitchell, Jeffrey
Oldham e Alex Samuel
http://www.advancedlinuxprogramming.com/
http://www.codesourcery.com/

Advanced Linux Programming

Copyright 2001 by New Riders Publishing
FIRST EDITION: June, 2001

Todos os direitos reservados. Nenhuma parte desse livro pode ser reproduzida ou transmitida de qualquer forma ou por quaisquer meios, eletônico ou mecânico, incluindo fotocópia, gravação, ou por qualquer meio de armazenamento de informação e sistema de recuperação, exceto para a inclusão de breve citação em uma publicação.

Número International Standard Bookr: 0-7357-1043-0

Número de Cartão do Catálogo da Biblioteca do Congresso dos EUA: 00-105343 05 04 03 02 01 7 6 5 4 3 2 1

Interpretação do código de impressão: Os dois dígitos mais à direita são o ano de impressão do livro; o dígito simples mais à direita é o número de impressão do livro. Por exemplo, o código de impressão 01-1 mostra que a primeira impressão do livro ocorreu em 2001.

Composto em Bembo e MCPdigital pela New Riders Publishing. Impresso nos Estados Unidos da América.

Trademarks

Todos os temos mencionados nesse livro que são conhecidos serem trademarks ou service marks foram apropriadamente capitalizados. New Riders Publishing não pode atestar a precisão dessa informação. O uso de um termo nesse livro não deve ser considerado como afetando a validade de qualquer trademark ou service mark.

PostScript é uma marca registrada de Adobe Systems, Inc. Linux é uma marca registrada de Linus Torvalds.

Alerta e Aviso Legal

Esse livro é projetado para fornecer informação sobre Programação Avançada em Ambiente GNU/Linux. Todo esforço foi feito para tornar esse livro tão completo e preciso quanto possível, mas nenhuma garantia ou adequação etá implícita.

Essa informação é fornecida sobre uma basicamente como etá. Os autores e a New Riders Publishing não terão nenhuma dependência nem responsabilidade para com nenhuma pessoa ou entidade com relação a qualquer perda ou dano proveniente da informação contida nesse livro ou de uso dos discos ou programas que o acompanham.

Créditos

Editor
David Dwyer
Editor Associado
Al Valvano
Editor Executivo
Stephanie Wall
Editor Gerente
Gina Brown
Editor de Aquisições
Ann Quinn
Editor de Desenvolvimento
Laura Loveall
Gerente de Marketing de Produto
Stephanie Layton
Gerente de Publicidade
Susan Petro
Editor de Projeto
Caroline Wise
Editor de Cópia
Krista Hansing
Indexador Sênior
Cheryl Lenser
Coordenador de manufatura
Jim Conway
Designer de Livro
Louisa Klucznik
Designer de Capa
Brainstorm Design, Inc.
Pordução de Capa
Aren Howell
Revisor
Debra Neel
composição
Amy Parker

Sobre os Autores

Mark Mitchell recebeu o grau de bacharel em ciências da computação em Harvard em 1994 e mestrado em Stanford em 1999. Sua área de interesse está centrada em complexidade computacional e segurança computacional. Mark participou sibstancialmente no desenvolvimento da GNU Compiler Collection, e ele tem um forte interesse em qualidade de desenvolvimento de software.

Jeffrey Oldham recebeu o bacharelado do grau de artes em ciências da computação na Universidade de Rice em 1991. Após trabalhar no Center fo Research on Parallel Computation, ele obteve o doutorado em filosofia em Stanford no ano de 2000. Seu interesse de pesquisa centra-se em engenharia de algorítmos, concentrando-se em fluxo e outros algorítmos combinatoriais. Ele traba no GCC e em software de computação científica.

Alex Samuel graduado em Harvard em 1995 com um grau em física. Ele trabalhou como engenheiro de software na BBN antes de retornar a estudar física na Caltech e no Stanford Linear Accelerator Center. Alex administrou o projeto Software Carpentry e trabalha em vários outros projetos, tais como otimizações no GCC. Mark e Alex fundaram a CodeSourcery LLC juntos em 1999. Jeffrey juntou-se à copanhia em 2000. A missão da CodeSourcery é fornecer ferramentas de desenvolvimento para GNU/Linux e outros sistemas operacionais; para levar à rede de ferramentas GNU uma qualidade comercial, de acordo com os padrões de conjunto de ferrametnas de desenvolvimento; e fornecer consultoria geral e serviços de engenharia. O Web síte da CodeSourcery é http://www.codesourcery.com.

Sobre os Revisores Técnicos

Esses revisores contribuiram com seu considerável experiência de trabalho ao longo de todo o processo de desenvolvimento do Advanced Linux Programming. Quando o livro estava sendo escrito, esses dedicados profissionais revisaram todo o materia de conteúdo técnico, a organização, e o andamento. O diálogo com eles foi fundamental para garantir que o Advanced Linux Programmingse ajustasse às necessidades dos leitores por informação da mais alta qualidade técnica.

Glenn Becker tem muitas graduações, todas em teatro. Ele atualmente trabalha como produtor online para SCIFI.COM, o braço online do SCI FI channel, em New York City. Em casa ele usa o Debian GNU/Linux e é obcessivo sobre tópicos com administração de sistemas, segurança, internacionalização de software, e XML.

John Dean recebeu um BSc(Hons) da Universidade de Sheffield em 1974, em ciência pura. Como um graduado na Sheffield, John desenvolveu seu interesse em computação. Em 1986 ele recebeu um MSc do Cranfield Institute of Science and Technology em Engenharia de Controle. Enquanto trabalhava para a Roll Royce and Associates, John tornou-se envolvido no desenvolvimento de software de controle para inspeção do vapor que emana das usinas nucleares assitida por computador. Uma vez que deichou a RR&A em 1978, ele trabalhou na indústria petroquímica desenvolvendo e mantendo software de controle de processo. John worked como desenvolvedor voluntário de software para o MySQL de 1996 até maio de 2000, quando juntou-se ao MySQL como um funcionário em tempo integral. A área de responsabilidade de John é MySQL no MS Windows e desenvolvimento de uma nova GUI do cliente MySQL usando o kit de feramentas de aplicação Qt da Trolltech sobre ambos Windows e plantaforma que executa o X-11.

Agradecimentos

Apreciamos grandemente o trabalho prioneiro de Richard Stallman, sem o qual nunca teria existido o Projeto GNU, e de Linus Torvalds, sem o qual nunca teria existido o *kernel* do Linux. Incontáveis outras pessoa trabalharam sobre partes do sistema operacional GNU/Linux, e agradecemos a todos eles.

Agradecemos às faculdades de Harvard e Rice pela nosso curso superior, e Caltech e Stanford pelo nosso treinamento de graduação. Sem todos que nos ensinaram, nós nunca teríamos ousadia para ensinar outros!

W. Richard Stevens escreveu três excelentes livros sobre programação em ambiente UNIX, e nós os consultamos extensivamente. Roland McGrath, Ulrich Drepper, e muitos outros escreveram a biblioteca C GNU e sua excelente.

Robert Brazile e Sam Kendall revisaram o primeiro esboço desse livro e fizeram maravilhosas sugestões sobre ajustes e conteúdo. Nossos editores técnicos e revisores (especialmente Glenn Becker e John Dean) nos mostraram erros, fizeram sugestões, e forneceram contínuo encorajamento. Certamente, quaisquer erros que restarem não são falhas deles!

Agradecimentos a Ann Quinn, da New Riders, por se encarregar de todos os detalhes envolvidos na publicação desse livro; Laura Loveall, também da New Riders, por não nos permitir ficar muito muito atrazados para nossos compromissos; e Stephanie Wall, também da New Riders, fpor nos encorajar a escrever esse livro em primeiro lugar!

Nos Diga Qual Sua Opinião

Como leitor desse livro, você é o mais importante crítico e comentarista. Valorizamos sua opinião e desejamos conhecer o que estamos fazendo corretamene, o que podemos fazer melhor, quais áreas você gostaria de nos ver publicar, e quaisquer outras palavras de sabedoria você está disposto a colocar em nosso caminho.

Como Editora Executiva para o time de Desenvolvimento Web d New Riders Publishing, I seus comentários são bem vindos. Você pode enviar-nos um fax, um email, ou escrever-me diretamente para me permitir saber o que você gostou ou não sobre esse livro–também o que podemos fazer para tornar nossos livros melhores.

Por favor note que Eu não posso ajudar você com problemas técnicos relacionados aos tópicos desse livro, e que devido ao grande volume de correio que Eu recebo, Eu posso não ser capaz de responder a todas as mensagens.

Quando você escrever, por favor tenha certeza de incluir o título desse livro e o autor, bem como seu nome e telefone ou númeor de faz. Eu irei cuidadosamente revisar seus comentários e compartilhá-los com os autores e editores que trabalharam no livro.

Fax:	317-581-4663
Email:	Stephanie.Wall@newriders.com
Mail:	Stephanie Wall Executive Editor New Riders Publishing 201 West 103rd Street Indianapolis, IN 46290 USA

Do Tradutor

(...) Pero, con todo eso, me parece que el traducir de una lengua en otra, como no sea de las reinas de las lenguas, griega y latina, es como quien mira los tapices flamencos por el revés, que aunque se veen las figuras, son llenas de hilos que las escurecen y no se veen con la lisura y tez de la haz, y el traducir de lenguas fáciles ni arguye ingenio ni elocución, como no le arguye el que traslada ni el que copia un papel de otro papel. (...) [II, 62]

El ingenioso hidalgo Don Quijote de la Mancha
Miguel de Cervantes

Essa tradução é dedicada especialmente a um rapazinho que, na presente data, encontra-se ainda no ventre materno. Espero que todos nós possamos entregar às crianças de hoje um mundo melhor que o que nós encontramos. Melhor em todos os sentidos mas principalmente nos sentidos social, ecológico e em qualidade de vida.

Traduzido por Jorge Barros de Abreu
http://sites.google.com/site/ficmatinf
Versão - 0.23 - 17/12/2012

Da Tradução

- os códigos fontes originais dos programas podem ser encontrados no sítios citados na primeira página dessa tradução.
- em algumas páginas o latex colocou espaçamentos extras pelo fato de logo a frente encontrar-se algum objeto que não pode ser partido em duas páginas. Posteriormente pensarei sobre colocar esses objetos no final de cada capítulo, ou não, como diria nosso o Ministro Gil.
- nas listagens de programas colocou-se uma numeração com intuito de facilitar a explanação e a análise do código em condições pedagógicas.
- a tradução foi feita a partir dos originais em inglês no formato pdf e convertidos com o programa pdftotext. Isso quer dizer que alguma formatação do original foi eventualmente/inadivertidamente perdida/esquecida/omitida na conversão para o texto puro.
- o capítulo 9 precisa de mais atenção dos experts em *assembly*.
- a bibliografia foi incluída pelo tradutor.
- na tradução a expressão GNU/Linux foi usada com extensivamente e enfáticamente.
- os códigos fontes dos programas foram traduzidos mas a acentuação foi retirada por questão de compatibilidade com o pacote LaTEX listings.

Sumário

I Programação UNIX Avançada com Linux **1**

1 Iniciando **5**
1.1 Editando com *Emacs* 5
1.1.1 Abrindo um Arquivo Fonte em C ou em C++ 6
1.1.2 Formatando Automaticamente 7
1.1.3 Destaque Sintático para Palavras Importantes 7
1.2 Compilando com GCC 8
1.2.1 Compilando um Único Arquivo de Código Fonte 9
1.2.2 Linkando Arquivos Objeto 11
1.3 Automatizando com *GNU Make* 12
1.4 Depurando com o Depurador GNU (GDB) 14
1.4.1 Depurando com GNU GDB 15
1.4.2 Compilando com Informações de Depuração 15
1.4.3 Executando o *GDB* 15
1.5 Encontrando mais Informação 18
1.5.1 Páginas de Manual 18
1.5.2 Info 19
1.5.3 Arquivos de Cabeçalho 20
1.5.4 Código Fonte 20

2 Escrevendo Bom Software GNU/Linux **23**
2.1 Interação Com o Ambiente de Execução 23
2.1.1 A Lista de Argumentos 24
2.1.2 Convenções GNU/Linux de Linha de Comando 25
2.1.3 Usando *getopt_long* 26
2.1.4 E/S Padrão 30
2.1.5 Códigos de Saída de Programa 32
2.1.6 O Ambiente 32
2.1.7 Usando Arquivos Temporários 36
2.2 Fazendo Código Defensivamente 39
2.2.1 Usando *assert* 39

2.2.2 Falhas em Chamadas de Sistema 41
2.2.3 Códigos de Erro de Chamadas de Sistema 43
2.2.4 Erros e Alocação de Recursos 45
2.3 Escrevendo e Usando Bibliotecas 47
2.3.1 Agrupando Arquivos Objeto 47
2.3.2 Bibliotecas Compartilhadas 49
2.3.3 Bibliotecas Padronizadas 51
2.3.4 Dependência de uma Biblioteca 52
2.3.5 Prós e Contras 54
2.3.6 Carregamento e Descarregamento Dinâmico 55

3 Processos **57**
3.1 Visualizando Processos 57
3.1.1 Identificadores de Processos 58
3.1.2 Visualizando os Processos Ativos 58
3.1.3 Encerrando um Processo 60
3.2 Criando Processos 60
3.2.1 Usando *system* 60
3.2.2 Usando bifurcar e executar 61
3.2.3 Agendamento de Processo 64
3.3 Sinais 66
3.3.1 Encerramento de Processos 68
3.3.2 Esperando pelo Encerramento de um Processo 70
3.3.3 As Chamadas de Sistema da Família *wait* 70
3.3.4 Processos do Tipo Zumbi 71
3.3.5 Limpando Filhos de Forma Não Sincronizada 73

4 Linhas de Execução **77**
4.1 Criação de Linhas de Execução 78
4.1.1 Enviando Dados a uma Linha de Execução 80
4.1.2 Vinculando Linhas de Execução 82
4.1.3 Valores de Retorno de Linhas de Execução 84
4.1.4 Mais sobre IDs de Linhas de Execução 85
4.1.5 Atributos de Linha de Execução 86
4.2 Cancelar Linhas de Execução 88
4.2.1 Linhas de Execução Sincronas e Assincronas 89
4.2.2 Seções Críticas Incanceláveis 89
4.2.3 Quando Cancelar uma Linha de Execução 91
4.3 Área de Dados Específicos de Linha de Execução 92
4.3.1 Controladores de Limpeza 95
4.3.2 Limpeza de Linha de Execução em C++ 96

4.4 Sincronização e Seções Críticas 97
4.4.1 Condições de Corrida 98
4.4.2 Mutexes 100
4.4.3 Travas Mortas de *Mutex* 103
4.4.4 Testes de *Mutex* sem Bloqueio 105
4.4.5 Semáforos para Linhas de Execução 105
4.4.6 Variáveis Condicionais 109
4.4.7 Travas Mortas com Duas ou Mais Linhas de Execução 115
4.5 Implementação de uma Linha de Execução em GNU/Linux . . 116
4.5.1 Controlando Sinais 117
4.5.2 Chamada de Sistema *clone* 118
4.6 Processos Vs. Linhas de Execução 118

5 Comunicação Entre Processos 121
5.1 Memória Compartilhada 122
5.1.1 Comunicação Local Rápida 123
5.1.2 O Modelo de Memória 123
5.1.3 Alocação 124
5.1.4 Anexando e Desanexando 125
5.1.5 Controlando e Desalocando Memória Compartilhada . 126
5.1.6 Um programa Exemplo 127
5.1.7 Depurando 127
5.1.8 Prós e Contras 128
5.2 Semáforos de Processos 128
5.2.1 Alocação e Desalocação 129
5.2.2 Inicializando Semáforos 130
5.2.3 Operações *Wait* e *Post* 130
5.2.4 Depurando Semáforos 132
5.3 Arquivos Mapeados em Memória 132
5.3.1 Mapeando um Arquivo Comum 133
5.3.2 Programas Exemplo 134
5.3.3 Acesso Compartilhado a um Arquivo 136
5.3.4 Mapeamentos Privados 137
5.3.5 Outros Usos para Arquivos Mapeados em Memó-ria . . 137
5.4 *Pipes* 138
5.4.1 Criando *Pipes* 138
5.4.2 Comunicação Entre Processos Pai e Filho 139
5.4.3 Redirecionando os Fluxos da Entrada Padrão, da Saída Padrão e de Erro 141
5.4.4 As Funções *popen* e *pclose* 142

5.4.5 FIFOs 143
5.4.5.1 Criando um *FIFO* 144
5.4.5.2 Accessando um *FIFO* 144
5.5 *Sockets* 145
5.5.1 Conceitos de *Socket* 146
5.5.2 Chamadas de Sistema 147
5.5.3 Servidores 148
5.5.4 *Sockets* Locais 149
5.5.5 Um Exemplo Usando um *Sockets* de Escopo local . . . 150
5.5.6 *Sockets* de Domínio *Internet* 153
5.5.7 *Sockets* Casados 155

II Dominando GNU/Linux 157

6 Dispositivos 161
6.1 Tipos de Dispositivos 162
6.2 Números de Dispositivo 163
6.3 Entradas de Dispositivo 164
6.3.1 O Diretório */dev* 165
6.3.2 Acessando Dispositivos por meio de Abertura de Arquivos 166
6.4 Dispositivos de *Hardware* 167
6.5 Dispositivos Especiais 171
6.5.1 O Dispositivo */dev/null* 171
6.5.2 O Dispositivo */dev/zero* 172
6.5.3 */dev/full* 173
6.5.4 Dispositivos Geradores de *Bytes* Aleatórios 173
6.5.5 Dispositivos Dentro de Dispositivos 175
6.6 PTYs 179
6.6.1 Uma Demonstração de PTY 180
6.7 A chamada de sistema *ioctl* 181

7 O Sistema de Arquivos */proc* 183
7.1 Extraindo Informação do */proc* 184
7.2 Entradas dos Processos 186
7.2.1 /proc/self 188
7.2.2 Lista de Argumentos do Processo 189
7.2.3 Ambiente de Processo 192
7.2.4 O Executável do Processo 192
7.2.5 Descritores de Arquivo do Processo 193

7.2.6 Estatísticas de Memória do Processo 195
7.2.7 Estatísticas de Processo 196
7.3 Informações de *Hardware* 196
7.3.1 Informações sobre a CPU 196
7.3.2 Informação de Dispositivos 197
7.3.3 Informação de Barramento 197
7.3.4 Informações de Porta Serial 197
7.4 Informação do *Kernel* 198
7.4.1 Informação de versão 198
7.4.2 Nome do Host e Nome de Domínio 199
7.4.3 Utilização da Memória 199
7.5 Acionadores, Montagens, e Sistemas de Arquivos 201
7.5.1 Sistemas de Arquivo 201
7.5.2 Acionadores e Partições 201
7.5.3 Montagens 203
7.5.4 Travas 204
7.6 Estatísticas de Sistema 206

8 Chamadas de Sistema do GNU/Linux 209
8.1 Usando *strace* 211
8.2 A Chamada *access*: Testando Permissões de Arquivos 212
8.3 A Chamada de Sistema *fcntl*: Travas e Outras Operações em Arquivos 214
8.4 As Chamadas *fsync* e *fdatasync*: Descarregando para o Disco . 216
8.5 As Chamadas *getrlimit* e *setrlimit*: Limites de Recurso 218
8.6 a Chamada *getrusage*: Estatísticas de Processo 220
8.7 A Chamada *gettimeofday*: Hora Relógio Comum 221
8.8 A Família *mlock*: Travando Memória Física 222
8.9 *mprotect*: Ajustando as Permissões da Memória 224
8.10 A Chamada *nanosleep*: Temporizador de Alta Precisão 227
8.11 readlink: Lendo Links Simbólicos 228
8.12 A Chamada *sendfile*: Transferência de Dados Rápida 229
8.13 A Chamada *setitimer*: Ajustando Intervalos em Temporizadores231
8.14 A Chamada de Sistema *sysinfo*: Obtendo Estatísticas do Sistema232
8.15 A Chamada de Sistema *uname* 233

9 Código Assembly Embutido 235
9.1 Quando Usar Código em *Assembly* 236
9.2 *Assembly* Embutido Simples 237
9.2.1 Convertendo Instruções *asm* em Instruções *Assembly* . 238

9.3 Sintaxe *Assembly* Extendida 239
9.3.1 Instruções *Assembler* 239
9.3.2 Saídas 239
9.3.3 Entradas 241
9.3.4 Crítica 241
9.4 Exemplo 241
9.5 Recursos de Otimização 244
9.6 Manutensão e Recursos de Portabilidade 244

10 Segurança 245
10.1 Usuários e Grupos 246
10.1.1 O Superusuário 247
10.2 IDs de Usuário e IDs de Grupo 248
10.3 Permissões do Sistema de Arquivos 249
10.3.1 Falha de Segurança:
Sem Permissão de Execução 253
10.3.2 *Sticky Bits* 254
10.4 ID Real e ID Efetivo 255
10.4.1 Programas *Setuid* 257
10.5 Autenticando Usuários 259
10.6 Mais Falhas de Segurança 262
10.6.1 Sobrecarga no Espaço Temporário de Armazenagem . . 263
10.6.2 Condiçoes de Corrida no */tmp* 266
10.6.3 Usando *system* ou *popen* 269

11 Um Modelo de Aplicação GNU/Linux 273
11.1 Visão Geral 273
11.1.1 Ressalvas 274
11.2 Implementação 276
11.2.1 Funções Comuns 278
11.2.2 Chamando Módulos de Servidor 280
11.2.3 O Servidor 282
11.2.4 O Programa Principal 288
11.3 Modulos 291
11.3.1 Mostra a Hora do Relógio Comum 292
11.3.2 Mostra a Distribuição GNU/Linux 293
11.3.3 Mostrando o Espaço Livre do Disco 294
11.3.4 Sumarizando Processos Executando 295
11.4 Usando o Servidor 301
11.4.1 O Makefile 302
11.4.2 Gerando o Executável do Programa Server 303

11.4.3 Executando o Programa Server 303
11.5 Terminando 305

III Apêndices 307

A Outras Ferramentas de Desenvolvimento 311
A.1 Análise Estática do Programa 311
A.2 Encontrando Erros de Memória Alocada Dinâmicamente . . . 313
A.2.1 Um Programa para Testar Alocação e Desalocação de Memória 316
A.2.2 *malloc Checking* 316
A.2.3 Encontrando Vazamento de Memória Usando *mtrace* 317
A.2.4 Usando ccmalloc 318
A.2.5 *Electric Fence* 320
A.2.6 Escolhendo Entre as Diferentes Ferramentas Depuradoras de Memória 321
A.2.7 Código Fonte para o Programa de Memória Dinâmica 321
A.3 Montando Perfil 323
A.3.1 Uma Calculadora Simples 324
A.3.2 Coletando Informações de Montagem de Perfil 325
A.3.3 Mostrando Dados de Montagem de Perfil 325
A.3.4 Como *gprof* Coleta Dados 328
A.3.5 Código Fonte do Programa Calculadora 328

B E/S de Baixo Nível 333
B.1 Lendo e Escrevendo Dados 334
B.1.1 Abrindo um Arquivo 334
B.1.2 Fechando Descritores de Arquivo 337
B.1.3 Escrevendo Dados 337
B.1.4 Lendo Dados 339
B.1.5 Movendo-se ao Longo de um Arquivo 341
B.2 stat 344
B.3 Leituras e Escritas de Vetor 346
B.4 Relação de Funções de E/S da Biblioteca C GNU Padrão . . . 349
B.5 Outras Operações de Arquivo 350
B.6 Lendo o Conteúdo de um Diretório 351

C Tabela de Sinais 355

D Recursos Online **359**
D.1 Informação Geral 359
D.2 Informação Sobre *Software* GNU/Linux 359
D.3 Outros Sítios 360

E Open Publication License **361**

F GNU General Public License **365**

G Saídas Diversas do */proc* **373**
G.1 cat /proc/cpuinfo 373
G.2 Entradas de um Diretório de Processo 380
G.3 cat /proc/version 380
G.4 cat /proc/scsi/scsi 381
G.5 cat /proc/sys/dev/cdrom/info 381
G.6 cat /proc/mounts 382
G.7 cat /proc/locks 382

H Adicionais ao Capítulo 8 **385**
H.1 *strace hostname* 385
H.2 *sysctl* 386
H.3 Ano de 1970 398

I *Assembly* **401**
I.1 Alô Mundo 401
I.2 bsrl 402

J Segurança **403**
J.1 Setuid no Debian 6.0.2 403

K Anexos aos Apêndices **405**
K.1 Signal.h 405
K.2 Analizadores de Código 405

L Licença de Livre Publicação **407**

M A Licença Pública Geral do GNU - pt_BR **409**

Lista de Tabelas

2.1 Opções do Programa Exemplo 26

6.1 Lista Parcial de Dispositivos de Bloco Comuns 168
6.2 Alguns Dispostivos de Caractere Comuns 169

7.1 Caminhos Completos para os Quatro Possíveis Dispositivos IDE202

9.1 Letras de registradores para a Arquitetura *x86 Intel*. 240

A.1 Capacidades das Ferramentas de Verificação Dinâmica de Memória (X Indica Detecção, e O Indica Detecção para Alguns Casos) . 315

C.1 Sinais do GNU/Linux 356
C.2 Sinais do GNU/Linux - Continuação 357

Listagem Códigos Fonte

1.1 Arquivo Código fonte em C – main.c 9
1.2 Arquivo Código fonte em C++ – reciprocal.cpp 9
1.3 Arquivo de cabeçalho – reciprocal.hpp 9
2.1 (Arquivo arglist.c) Usando *argc* e *argv*. 25
2.2 (getopt_long.c) Usando a função *getopt_long* 29
2.3 (getopt_long.c) Continuação 30
2.4 (print-env.c) Mostrando o Ambiente de Execução 35
2.5 (client.c) Parte de um Programa Cliente de Rede 35
2.6 (temp_file.c) Usando *mkstemp* 38
2.7 (readfile.c) Liberando Recursos em Condições Inesperadas . . 46
2.8 (test.c) Área da Biblioteca 48
2.9 Um Programa Que Utiliza as Funções da Biblioteca Acima . . 48
2.10 (tifftest.c) Usando a *libtiff* 52
3.1 (print-pid.c) Mostrando o ID do Processo 58
3.2 (system.c) Usando uma chamada à função *system* 61
3.3 (fork.c) Usando *fork* para Duplicar o Processo de um Programa 62
3.4 (fork-exec.c) Usando *fork* e *exec* Juntas 64
3.5 (sigusr1.c) Usando um Controlador de Sinal 68
3.6 (zombie.c) Fazendo um Processo Zumbi 72
3.7 (sigchld.c) Limpando Processos filhos pelo manuseio de SIG-CHLD 74
4.1 (thread-create.c) Criando uma Linha de Execução 80
4.2 (thread-create2) Cria Duas Linhas de Execução 81
4.3 Função *main* revisada para thread-create2.c 83
4.4 (primes.c) Calcula Números Primos em uma Linha de Execução 85
4.5 (detached.c) Programa Esqueleto Que Cria uma Linha dde Execução Desvinculada 87
4.6 (critical-section.c) Protege uma Transação Bancária com uma Seção Crítica 91
4.7 (tsd.c) *Log* Por Linhas de Execução Implementado com Dados Específicos de Linha de Execução 94

4.8 (cleanup.c) Fragmento de Programa Demonstrando um Controlador de Limpeza de Linha de Execução 96
4.9 (cxx-exit.cpp) Implementando Saída Segura de uma Linha de Execução com Exceções de C++ 97
4.10 (job-queue1.c) Função de Linha de Execução para Processar Trabalhos Enfileirados 99
4.11 (job-queue2.c) Função de Tarefa da Fila de Trabalho, Protegida por um *Mutex* 102
4.12 (job-queue3.c) Fila de Trabalhos Controlada por um Semáforo 108
4.13 (job-queue3.c) Continuação 109
4.14 (spin-condvar.c) Uma Implementação Simples de Variável Condicional 110
4.15 (condvar.c) Controla uma Linha de Execução Usando uma Variável Condicional 114
4.16 (thread-pid) Imprime IDs de processos para Linhas de Execução116
5.1 Exercício de Memória Compartilhada 127
5.2 (sem_all_deall.c) Alocando e Desalocando um semáforo Binário 129
5.3 (sem_init.c) Inicializando um Semáforo Binário 130
5.4 (sem_pv.c) Operações *Wait* e *Post* para um Semáforo Binário 131
5.5 (mmap-write.c) Escreve um Número Aleatório para um Arquivo Mapeado em Memória 134
5.6 (mmap-read.c) Lê um Inteiro a partir de um Arquivo Mapeado em Memória, e Dobra-o 135
5.7 (pipe.c) Usando um *pipe* para Comunicar-se com um Processo Filho 140
5.8 (dup2.c) Redirecionar a Saída de um *pipe* com *dup2* 142
5.9 (popen.c) Exemplo Usando *popen* 143
5.10 (socket-server.c) Servidor de *Socket* de Escopo Local 151
5.11 (socket-client.c) Cliente de *Socket* de Escopo Local 152
5.12 (socket-inet.c) Lê de um Servidor WWW 154
6.1 (random_number.c) Função para Gerar um Número Aleatório 175
6.2 (cdrom-eject.c) Ejeta um CD-ROM/DVD 182
7.1 (clock-speed.c) Extraindo a Velocidade de *Clock* da CPU de */proc/cpuinfo* 186
7.2 (get-pid.c) Obtendo o ID de Processo de */proc/self* 189
7.3 (print-arg-list.c) Mostra na Tela a Lista de Arguentos de um Processo que está Executando 191
7.4 (print-environment.c) Mostra o Ambiente de um Processo . . . 192
7.5 (get-exe-path.c) Pega o Caminho do Programa Executando Atualmente 193
7.6 (open-and-spin.c) Abre um Arquivo para Leitura 195

7.7 (print-uptime.c) Mostra o Tempo Ligado e o Tempo Ocioso . . 206
8.1 (check-access.c) Check File Access Permissions 213
8.2 (lock-file.c) Create a Write Lock with fcntl 215
8.3 (write_journal_entry.c) Write and Sync a Journal Entry 217
8.4 (limit-cpu.c) Demonstração do Tempo Limite de Uso da CPU 219
8.5 (print-cpu-times.c) Mostra Usuário de Processo e Horas do Sistema 220
8.6 (print-time.c) Mostra a Data e a Hora 222
8.7 (mprotect.c) Detecta Acesso à Memória Usando *mprotect* . . . 226
8.8 (better_sleep.c) High-Precision Sleep Function 228
8.9 (print-symlink.c) Mostra o Alvo de um Link Simbólico 229
8.10 (copy.c) Cópia de Arquivo Usando *sendfile* 230
8.11 (itimer.c) Exemplo de Temporizador 232
8.12 (sysinfo.c) Mostra Estatísticas do Sistema 233
8.13 (print-uname.c) Mostra o número de Versão do GNU/Linux e Informação de *Hardware* 234
9.1 (bit-pos-loop.c) Encontra a Posição do *Bit* Usando um Laço . 243
9.2 (bit-pos-asm.c) Encontra a posição do *Bit* Usando *bsrl* 243
10.1 (simpleid.c) Mostra ID de usuário e ID de grupo 249
10.2 (stat-perm.c) Determina se o Proprietário do Arquivo Tem Permissão de Escrita 252
10.3 (setuid-test.c) Programa de Demonstração do *Setuid* 258
10.4 (pam.c) Exemplo de Uso do PAM 261
10.5 (temp-file.c) Cria um Arquivo Temporário 268
10.6 (grep-dictionary.c) Busca por uma Palavra no Dicionário . . . 270
11.1 (server.h) Declarações de Funções e de Variáveis 277
11.2 (common.c) Funções de Utilidade Geral 278
11.3 (common.c) Continuação 279
11.4 (module.c) Carregando e Descarregando Módulo de Servidor . 281
11.5 (server.c) Implementação do Servidor 283
11.6 (server.c) Continuação 284
11.7 (server.c) Continuação 285
11.8 (server.c) Continuação 286
11.9 (main.c) Programa Principal do Servidor e Tratamento de Linha de Comando 289
11.10 (main.c) Continuação 290
11.11 (main.c) Continuação 291
11.12 (time.c) Módulo do Servidor para Mostrar a Hora Relógio Comum 292
11.13 (issue.c) Módulo de Servidor para Mostrar Informação da Distribuição GNU/Linux 293

11.14(diskfree.c) Módulo de Servidor para Mostrar Informações Sobre Espaço Livre no Disco 294
11.15(processes.c) Módulo de Servidor para Sumarizar Processos . 296
11.16(processes.c) Continuação 297
11.17(processes.c) Continuação 298
11.18(processes.c) Continuação 299
11.19(Makefile) Arquivo de Configuração para Exemplo de Servidor 302
A.1 (hello.c) Programa Alô Mundo 312
A.2 (malloc-use.c) Exemplo de Como Testar Alocação Dinâmica de Memória 322
A.3 (malloc-use.c) Exemplo de Como Testar Alocação Dinâmica de Memória 323
A.4 (calculator.c) Programa Principal da Calculadora 329
A.5 (calculator.c) Continuação 330
A.6 (number.c) Implementação de Número Unário 330
A.7 (number.c) Continuação 331
A.8 (stack.c) Pilha do Número Unário 331
A.9 (stack.c) Continuação 332
A.10 (definitions.h) Arquivo de Cabeçalho para number.c e stack.c . 332
B.1 (create-file.c) Cria um Novo Arquivo 336
B.2 (timestamp.c) Anexa uma *Timestamp* a um Arquivo 338
B.3 (write-all.c) Escreve Tudo de uma Área Temporária de Armazenagem de Dados 339
B.4 (hexdump.c) Mostra uma Remessa de caracteres em Hexadecimal de um Arquivo 341
B.5 (lseek-huge.c) Cria Grandes Arquivos com *lseek* 343
B.6 (read-file.c) Lê um Arquivo para dentro de um Espaço Temporário de Armazenagem 346
B.7 (write-args.c) Escreve a Lista de Argumentos para um Arquivo com *writev* 348
B.8 (listdir.c) Mostra uma Listagem de Diretórios 352

Parte I

Programação UNIX Avançada com Linux

- 1 Iniciando
- 2 O Sistema de Arquivos */proc*
- 3 Processos
- 4 Linhas de Execução
- 5 Comunicação Entre Processos

Capítulo 1

Iniciando

ESSE CAPÍTULO MOSTRA COMO EXECUTAR OS PASSOS básicos requeridos para criar um programa Linux usando a linguagem C ou a linguagem C++. Em particular, esse capítulo mostra como criar e modificar código fonte C e C++, compilar esse código modificado, e depurar o resultado. Se você tem experiência em programação em ambiente Linux, você pode pular agora para o Capítulo 2, "Escrevendo Bom Software GNU/Linux" prestando cuidadosa atenção à seção 2.3, "Escrevendo e Usando Bibliotecas" para informações sobre linkagem/vinculação estática versus linkagem/vinculação dinâmica às quais você pode não conhecer ainda.

No decorrer desse livro, assumiremos que você está familiarizado com as linguagens de programação C ou C++ e as funções mais comuns da biblioteca C GNU padrão. Os exemplos de código fonte nesse livro estão em C, exceto quando for necessário demonstrar um recurso particular ou complicação de programa em C++. Também assumiremos que você conhece como executar operações básicas na linha de comando do Linux, tais como criar diretórios e copiar arquivos. Pelo fato de muitos programadores de ambiente GNU/Linux terem iniciado programação no ambiente Windows, iremos ocasionalmente mostrar semelhanças e diferenças entre Windows e GNU/Linux.

1.1 Editando com *Emacs*

Um editor é o programa que você usa para editar o código fonte. Muitos editores estão disponíveis para Linux, mas o editor mais popular e cheio de recursos é provavelmente GNU *Emacs*.

Sobre o *Emacs*:
Emacs é muito mais que um editor. *Emacs* é um programa inacreditavelmente poderoso, tanto que em CodeSourcery, *Emacs* é afetuosamente conhecido como "Um Verdadeiro Programa", ou apenas o UVP de forma curta. Você pode ler e enviar mensagens eletrônicas de dentro do *Emacs*, e você pode personalizar e extender o *Emacs* de formas muito numerosas para discorrer aqui. Você pode até mesmo navegar na web de dentro do *Emacs*!

Caso você esteja familiarizado com outro editor, você pode certamente usá-lo no lugar do *Emacs*. Note que o restante desse livro está vinculado ao uso do *Emacs*. Se você ainda não tem um editor Linux favorito, então você deve seguir adiante com o mini-tutorial fornecido aqui.

Se você gosta do *Emacs* e deseja aprender sobre seus recursos avançados, você pode considerar ler um dos muitos livros sobre *Emacs* disponíveis. Um excelente tutorial é "Learning GNU *Emacs*", escrito por Debra Cameron, Bill Rosenblatt, e Eric S. Raymond (Editora O'Reilly, 1996).

1.1.1 Abrindo um Arquivo Fonte em C ou em C++

Você pode iniciar o *Emacs* digitando *emacs* em sua janela de terminal e pressionado a tecla Enter. Quando *Emacs* tiver iniciado, você pode usar os *menus* localizados na parte superior para criar um novo arquivo fonte. Clique no menu "File", escolha "Open File", então digite o nome do arquivo que você deseja abrir no "minibuffer" localizado na parte inferior da tela.[1] Se quiser criar um arquivo fonte na linguagem C, use um nome de arquivo que termine em .c ou em .h. Se você quiser criar um arquivo fonte em C++, use um nome de arquivo que termine em .cpp, .hpp, .cxx, .hxx, .C, ou .H. Quando o arquivo estiver aberto, você pode digitar da mesma forma que faria em qualquer programa processador de texto comum. Para gravar o arquivo, escolha a entrada "Save" no menu "File". Quando você tiver encerrado a utilização do *Emacs*, você pode escolher a opção "Exit *Emacs*" no menu"File".

Se você não gosta de apontar e clicar, você pode usar teclas de atalho de teclado para automaticamente abrir arquivos, gravar arquivos, e sair do *Emacs*. Para abrir um arquivo, digite C-x C-f. (O C-x significa pressionar a tecla *ctrl* e então pressionar a tecla *x*.) Para gravar um arquivo, digite C-x C-s. Para sair do *Emacs*, apenas digite C-x C-c. Se você desejar adquirir um pouco mais de habilidade com *Emacs*, escolha a entrada "*Emacs* Tutorial" no menu "Help".O tutorial abastece você com uma quantidade grande de dicas sobre como usar *Emacs* efetivamente.

[1] Se você não está executando em um sistema X Window, você terá de pressionar F10 para acessar os *menus*.

1.1.2 Formatando Automaticamente

Se você está acostumado a programar em um Ambiente Integrado de Desenvolvimento (IDE)[2], você consequentemente estará também acostumado a ter o editor ajudando você a formatar seu código. *Emacs* pode fornecer o mesmo tipo de funcionalidade. Se você abre um arquivo de código em C ou em C++, *Emacs* automaticamente detecta que o arquivo contém código fonte, não apenas texto comum. Se você pressiona a tecla Tab em uma linha em branco, Emacs move o cursor para um ponto ajustado apropriadamente. Se você pressionar a tecla Tab em uma linha que já contém algum texto, Emacs ajusta o texto. Então, por exemplo, suponha que você tenha digitado o seguinte:

```
int main ()
{
printf ("Alo, mundo\n");
}
```

Se você pressionar a tecla Tab na linha com a chamada à função *printf*, Emacs irá reformatar seu código para parecer como mostrado abaixo:

```
int main ()
{
    printf ("Alo, mundo\n");
}
```

Note como a linha foi apropriadamente indentada.

À medida que seu uso do Emacs for acontecendo, você verá como o Emacs pode ajudar você a executar todo tipo de complicadas tarefas de formatação. Se você for ambicioso, você pode programar o Emacs para executar literalmente qualquer tipo de formatação automática que você puder imaginar. Pessoas têm usado essa facilidade de programação para implementar modos Emacs para editar todo tipo de documento, para implementar jogos[3] e para implementar interfaces para usuários acessarem bases de dados.

1.1.3 Destaque Sintático para Palavras Importantes

Adicionalmente à formatação de seu código, Emacs pode destacar palavras facilmente ao ler código em C e em C++ através da coloração de diferentes

[2]Nota do tradutor: do inglês "Integrated Development Environment". Em nosso bom português ficaria "AID".

[3]Tente executar o comando "M-x dunnet" se você desejar divertir-se com um antiquadro jogo de aventura em modo texto. Nota do tradutor: Dunnet é um jogo distribuído junto com o emacs cuja primeira versão datava dos idos de 1983.

elementos sintáticos. Por exemplo, Emacs pode atribuir a palavra chaves uma certa cor, atribuir uma segunda cor diferente da anterior a tipos de dados internos tais como *int*, e atribuir a comentários outra terceira cor diferente das duas primeiras. A utilização de cor torna muito mais fácil destacar alguns erros comum de sintaxe.

A forma mais fácil de habilitar cores é editar o arquivo ~/.emacs e inserir a seguinte sequência de caracteres:

```
(global-font-lock-mode t)
```

Grave o arquivo, saia do Emacs, e volte a ele em seguida. Agora abra um código fonte em C ou em C++ e aproveite!

Você possivelmente pode ter notado que a sequência de caracteres que você inseriu dentro do seu .emacs é semelhante a um código da linguagem de programação LISP.Isso ocorre pelo fato de ser um código LISP! Muitas partes de código do Emacs são atualmente escritas em LISP. Você pode adicionar funcionalidades ao Emacs por meio de acréscimos em código LISP.

1.2 Compilando com GCC

Um compilador converte um código fonte legível a seres humanos em um código objeto legível a computadores que pode então ser executado. Os compiladores disponíveis em sistemas linux são todos parte da coleção de compiladores GNU, comumente conhecido como GCC.[4] GCC também inclui compiladores para as linguagens C, C++, Java, Objective-C, Fortran, e Ada. Esse livro está dirigido em sua grande parte para programação em C e C++.

Suponhamos que você tenha um projeto como o da Listagem 1.2 com um arquivo de código em C++ (reciprocal.cpp) e um arquivo de código fonte em C (main.c) como o da Listagem 1.1. Esses dois arquivos são supostamente para serem compilados e então linkados juntos para produzir um programa chamado reciprocal.[5] Esse programa irá calcular o recíproco/inverso de um inteiro.

[4]Para mais informação sobre GCC, visite `http://gcc.gnu.org`.

[5]Em Windows, arquívos executáveis geralmente possuem nomes que terminam em ".exe". Programas GNU/Linux, por outro lado, geralmente não possuem extensão. Então, o equivalente Windows do programa "reciprocal" pode provavelmente ser chamado "reciprocal.exe"; a versão GNU/Linux é somente "reciprocal".

Listagem 1.1: Arquivo Código fonte em C – main.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "reciprocal.hpp"
4
5 int main (int argc, char **argv)
6 {
7   int i;
8
9   i = atoi (argv[1]);
10   printf ("The reciprocal of %d is %g\n", i, reciprocal (i));
11   return 0;
12 }
```

Listagem 1.2: Arquivo Código fonte em C++ – reciprocal.cpp

```
1 #include <cassert>
2 #include "reciprocal.hpp"
3
4 double reciprocal (int i) {
5   // A variavel i deve ser nao nula.
6   assert (i != 0);
7   return 1.0/i;
8 }
```

Existe também um arquivo de cabeçalho chamado reciprocal.hpp (veja a Listagem 1.3).

Listagem 1.3: Arquivo de cabeçalho – reciprocal.hpp

```
1 #ifdef __cplusplus
2 extern "C" {
3 #endif
4
5 extern  double reciprocal (int i);
6
7 #ifdef __cplusplus
8 }
9 #endif
```

O primeiro passo é converter o código fonte em C e em C++ em código objeto.

1.2.1 Compilando um Único Arquivo de Código Fonte

O nome do compilador C é gcc. Para compilar um código fonte em C (gerar o arquivo objeto), você usa a opção -c. Então, por exemplo, inserindo o -c no *prompt* de comando compila o arquivo de código fonte main.c:

```
% gcc -c main.c
```

O arquivo objeto resultante é chamado main.o. O compilador C++ é chamado g++. Sua operação é muito similar ao gcc; a compilação de reciprocal.cpp é realizada através do seguinte comando:

```
% g++ -c reciprocal.cpp
```

A opção -c diz ao compilador g++ para fornecer como saída um arquivo objeto somente; sem essa opção, g++ iria tentar linkar o programa para produzir um executável. Após você ter digitado esse comando, você irá ter um arquivo objeto chamado reciprocal.o.

Você irá provavelmente precisar de algumas outras opções para construir qualquer programa razoávelmente grande. A opção -I é usada para dizer ao GCC onde procurar por arquivos de cabeçalho. Por padrão, GCC olha no diretório atual e nos diretórios onde cabeçalhos para bibliotecas C GNU padrão estão instalados. Se você precisar incluir arquivos de cabeçalho localizados em algum outro lugar, você irá precisar da opção -I. Por exemplo, suponhamos que seu projeto tenha um diretório chamado "src", para arquivos fonte, e outro diretório chamado "include". Você pode compilar o arquivo reciprocal.cpp como segue abaixo para indicar que g++ deve usar o diretório "../include" adicionalmente para encontrar o arquivo de cabeçalho "reciprocal.hpp":

```
% g++ -c -I ../include reciprocal.cpp
```

Algumas vezes você irá desejar definir macros na linha de comando. Por exemplo, no código de produção, você não irá querer o trabalho adicional da checagem de declaração presente em reciprocal.cpp; a checagem só existe para ajudar a você a depurar o programa. Você desabilita a checagem definindo a macro NDEBUG. Você pode ter adicionado uma declaração explícita #define em "reciprocal.cpp", mas isso requer modificação no código fonte em si. É mais fácil simplesmente definir NDEBUG na linha de comando, como segue:

```
% g++ -c -D NDEBUG reciprocal.cpp
```

Se você tiver desejado definir NDEBUG para algum valor particular, você pode ter feito algo como:

```
% g++ -c -D NDEBUG=3 reciprocal.cpp
```

Se você estiver realmente construindo código fonte de produção, você provavelmente deseja que o GCC otimize o código de forma que ele rode tão rapidamente quanto possível.Você pode fazer isso através da utilização da opção -O2 de linha de comando. (GCC tem muitos diferentes níveis de otimização; o segundo nível é apropriado para a maioria dos programas.) Por exemplo, o comando adiante compila reciprocal.cpp com otimização habilitada:

```
% g++ -c -O2 reciprocal.cpp
```

Note que compilando com otimização pode fazer seu programa mais difícil de depurar com um depurador (veja a Seção 1.4, "Depurando com o Depurador GNU (GDB)"). Também, em certas instâncias, compilando com otimização pode revelar erros em seu programa que não apareceriam em outras situações anteriores.

Você pode enviar muitas outras opções ao compilador gcc e ao compilador g++. A melhor forma de pegar uma lista completa é ver a documentação em tempo real. Você pode fazer isso digitando o seguinte na sua linha de comando:

```
% info gcc
```

1.2.2 Linkando Arquivos Objeto

Agora que você compilou main.c e reciprocal.cpp, você irá desejar juntar os códigos objeto e gerar o executável. Você deve sempre usar o g++ para linkar um programa que contém código em C++, mesmo se esse código C++ também contenha código em C. Se seu programa contiver somente código em C, você deve usar o gcc no lugar do g++. Pelo fato de o g++ está apto a tratar ambos os arquivos em C e em C++, você deve usar g++, como segue adiante:

```
% g++ -o reciprocal main.o reciprocal.o
```

A opção -o fornece o nome do arquivo a ser gerado como saída no passo de linkagem. Agora você pode executar o *reciprocal* como segue:

```
% ./reciprocal 7
The reciprocal of 7 is 0.142857
```

Como você pode ver, g++ linkou/vinculou automaticamente a biblioteca C GNU padrão em tempo de execução contendo a implementação da função. Se você tiver precisado linkar outra biblioteca (tal como uma coleção de rotinas/códigos prontos para facilitar a criação de uma interface gráfica de usuário)[6], você pode ter especificado a biblioteca com a opção -l. Em GNU/Linux, nomes de biblioteca quase sempre começam com "lib". Por exemplo, a biblioteca "Pluggable Authentication Module" (PAM) é chamada "libpam.a". Para linkar a libpam.a, você usa um comando como o seguinte:

[6]Nota do tradutor: QT ou Gtk.

```
% g++ -o reciprocal main.o reciprocal.o -lpam
```

O compilador automaticamente adiciona o prefixo "lib" e o sufixo ".a"[7]. Da mesma forma que para os arquivos de cabeçalho, o linkador procura por bibliotecas em alguns lugares padrão, incluindo os diretórios /lib e /usr/lib onde estão localizadas as bibliotecas padrão do sistema. Se você deseja que o linkador procure em outros diretórios também, você deve usar a opção -L, que é a correspondente da opção -I discutida anteriormente. Você pode usar essa linha para instruir o linkador a procurar por bibliotecas no diretório /usr/local/lib/pam antes de procurar nos lugares usuais:

```
% g++ -o reciprocal main.o reciprocal.o -L/usr/local/lib/pam -lpam
```

Embora você não tenha a opção -I para instruir o preprocessor para procurar o diretório atual, você deve usar a opção -L para instruir o linkador a procurar no diretório atual. Dizendo mais claramente, você pode usar a seguinte linha para instruir o linkador a encontrar a biblioteca "test" no diretório atual:

```
% gcc -o app app.o -L. -ltest
```

1.3 Automatizando com *GNU Make*

Se você está acostumado a programar para o sistema operacional Windows, você está provavelmente acostumado a trabalhar com um Ambiente Integrado de Desenvolvimento (IDE).Você adiciona arquivos de código fonte a seu projeto, e então o IDE contrói seu projeto automaticamente. Embora IDEs sejam disponíveis para GNU/Linux, esse livro não vai discutí-las. Em lugar de discutir IDEs, esse livro mostra a você como usar o *GNU Make* para automaticamente recompilar seu código, que é o que a maioria dos programadores GNU/Linux atualmente fazem.

A idéia básica por trás do *make* é simples. Você diz ao *make* os alvos que você deseja construir e então fornece regras explanatória de como construir os alvos desejados. Você também especifica dependências que indicam quando um alvo em particular deve ser reconstruído.

Em nosso projeto exemplo *reciprocal*, existem três alvos óbvios: reciprocal.o, main.o, e o reciprocal executável propriamente dito. Você já tinha regras em mente para reconstruir esses alvos na forma da linha de comando fornecidas previamente. As dependências requerem um pouco de raciocínio.

[7]Nota do tradutor: a biblioteca PAM pode ser encontrada em `http://ftp.mgts.by/pub/linux/libs/pam/library/`.

Claramente, reciprocal depende de reciprocal.o e de main.o pelo fato de você não poder linkar o programa até você ter construído cada um dos arquivos objetos. Os arquivos objetos devem ser reconstruídos sempre que o correspondente arquivo fonte mudar. Se acontece mais uma modificação em reciprocal.hpp isso também deve fazer com que ambos os arquivos objetos sejam reconstruídos pelo fato de ambos os arquivos fontes incluirem o reciprocal.hpp.

Adicionalmente aos alvos óbvios, deve-se ter sempre um alvo de limpeza. Esse alvo remove todos os arquivos objetos gerados e programas de forma que você possa iniciar de forma suave. A regra para esse alvo utiliza o comando rm para remover os arquivos.

Você pode reunir toda essa informação para o *make* colocando a informação em um arquivo chamado *Makefile*. Aqui está um exemplo de conteúdo de *Makefile*:

```
reciprocal: main.o reciprocal.o
  g++ $(CFLAGS) -o reciprocal main.o reciprocal.o

main.o: main.c reciprocal.hpp
  gcc $(CFLAGS) -c main.c

reciprocal.o: reciprocal.cpp reciprocal.hpp
  g++ $(CFLAGS) -c reciprocal.cpp

clean:
rm -f *.o reciprocal
```

Você pode ver que alvos são listados do lado esquerdo, seguidos por dois pontos e então quaisquer dependência são colocadas adiante dos dois pontos. A regra para construir o referido alvo localiza-se na linha seguinte. (Ignore o $(CFLAGS) um pouco por um momento.) A linha com a regra para esse alvo deve iniciar com um caractere de tabulação, ou *make* irá se confundir. Se você editar seu *Makefile* no *Emacs*, *Emacs* irá ajudar você com a formatação. Se você tiver removido os arquivos objetos que você construiu anteriormente, e apenas digitar

```
% make
```

na linha de comando, você irá ver o seguinte:

```
% make
gcc -c main.c
```

```
g++ -c reciprocal.cpp
g++ -o reciprocal main.o reciprocal.o
```

Você pode ver que *make* contrói automaticamente os arquivos objetos e então linka-os. Se você agora modificar por algum motivo o main.c e digitar *make* novemente, você irá ver o seguinte:

```
% make
gcc -c main.c
g++ -o reciprocal main.o reciprocal.o
```

Você pode ver que *make* soube reconstruir main.o e re-linkar o programa, mas o *make* não se incomodou em recompilar reciprocal.cpp pelo fato de nenhuma das dependências para reciprocal.o ter sofrido alguma modificação. O $(CFLAGS) é uma variável do *make*. Você pode definir essa varável ou no *Makefile* mesmo ou na linha de comando. *GNU make* irá substituir o valor da variável quando executar a regra. Então, por exemplo, para recompilar com otimização habilitada, você deve fazer o seguinte:

```
% make clean
rm -f *.o reciprocal
% make CFLAGS=-O2
gcc -O2 -c main.c
g++ -O2 -c reciprocal.cpp
g++ -O2 -o reciprocal main.o reciprocal.o
```

1.4 Depurando com o Depurador GNU (GDB)

Note que o sinalizador "-O2" foi inserido no lugar de $(CFLAGS) na regra. Nessa seção, você viu somente as mais básicas capacidades do *make*. Você pode encontrar mais informações digitando:

```
% info make
```

Nas páginas *info* de manual, você irá encontrar informações sobre como fazer para manter um *Makefile* simples, como reduzir o número de regras que você precisa escrever, e como automaticamente calcular dependências. Você pode também encontrar mais informação no livro *GNU Autoconf, Automake, and Libtool* escrito por Gary V.Vaughan, Ben Elliston,Tom Tromey, e Ian Lance Taylor (New Riders Publishing, 2000). [8]

[8] Nota do tradutor: A versão eletrônica do livro pode ser encontrada em `http://sources.redhat.com/autobook/download.html`.

1.4.1 Depurando com GNU GDB

O depurador é um programa que você usa para descobrir porque seu programa não está seguindo o caminho que você pensa que ele deveria. Você fará isso muitas vezes.[9] O depurador GNU (GDB) é o depurador usado pela maioria dos programadores em ambiente Linux. Você pode usar GDB para passear através de seu código fonte, escolhendo pontos de parada, e examinar o valor de variáveis locais.

1.4.2 Compilando com Informações de Depuração

Para usar o GDB, você irá ter que compilar com as informações de depuração habilitadas. Faça isso adicionado o comutador -g na linha de comando de compilação. Se você estiver usando um *Makefile* como descrito anteriormente, você pode apenas escolher *CFLAGS* para -g quando você executar o *make*, como mostrado aqui:

```
% make CFLAGS=-g
g++    -c -o reciprocal.o reciprocal.cpp
cc -g -O2    main.c reciprocal.o   -o main
```

Quando você compila com -g, o compilador inclui informações extras nos arquivos objetos e executáveis. O depurador usa essas informações para descobrir quais endereços correspodem a determinada linha de código e em qual arquivo fonte, como mostrar os valores armazenados em variáveis locais, e assim por diante.

1.4.3 Executando o *GDB*

Você pode iniciar digitando:

```
% gdb reciprocal
```

Quando o *gdb* iniciar, você verá o *prompt* do *GDB*:

```
(gdb)
```

O primeiro passo é executar seu programa dentro do depurador. Apenas insira o comando *run* e quaisquer argumentos do programa que você está depurando. Tente executar o programa sem qualquer argumento, dessa forma [10]:

[9] ...a menos que seus programas sempre funcionem da primeira vez.

[10] Nota do tradutor: a saída foi obtida em um gdb versão 6.8 em 2009 sendo portanto uma atualização da versão disponível em 2000 que foi o ano da publicação original.

```
(gdb) run
Starting program: reciprocal

Program received signal SIGSEGV, Segmentation fault.
0xb7e7e41b in ____strtol_l_internal () from /lib/libc.so.6
```

O problema é que não existe nenhum código de verificação de entradas errôneas na função main. O programa espera um argumento, mas nesse caso o programa estava sendo executado sem argumentos. A mensagem de *SIGSEGV* indicar uma interrupção anormal do programa [11]. GDB sabe que a interrupção anormal que ocorreu agora aconteceu em uma função chamada *____strtol_l_internal*. Aquela função está na biblioteca C GNU padrão. Você pode ver a pilha usando o comando *where* [12]:

```
(gdb) where
#0  0xb7e7e41b in ____strtol_l_internal () from /lib/libc.so.6
#1  0xb7e7e180 in strtol () from /lib/libc.so.6
#2  0xb7e7b401 in atoi () from /lib/libc.so.6
#3  0x08048486 in main (argc=Cannot access memory at address 0x0
) at main.c:9
```

Você pode ver a partir dessa tela que a função main chamou a função *atoi* com um apontador *NULL*, que é a fonte de todo o problema.

Você pode subir dois níveis na pilha até encontrar a função main através do uso do comando "up":

```
(gdb) up 2
#2  0xb7e7b401 in atoi () from /lib/libc.so.6
```

Note que gdb é capaz de encontrar o código de main.c, e mostra a linha onde a chamada errônea de função ocorreu. Você pode ver os valores das variáveis usando o comando *print*:

```
(gdb) print argv[1]
No symbol "argv" in current context.
```

O que confirma que o problema é relamente um apontador *NULL* passado dentro da função *atoi*.

Você pode escolher um ponto de parada através do uso do comando *break*:

[11] Nota do tradutor: em inglês: "crash".

[12] Nota do tradutor: a saída foi obtida em um gdb versão 6.8 em 2009 sendo portanto uma atualização da versão disponível em 2000 que foi o ano da publicação original.

```
(gdb) break main
Breakpoint 1 at 0x8048475: file main.c, line 9.
```

Esse comando define um ponto de parada na primeira linha de main. [13]Agora tente executar novamente o programa com um argumento, dessa forma:

```
(gdb) run 7
The program being debugged has been started already.
Start it from the beginning? (y or n) y

Starting program: reciprocal 7

Breakpoint 1, main (argc=2, argv=0xbfa0d334) at main.c:9
9       i = atoi (argv[1]);
```

Você pode ver que o depurador alcançou o ponto de parada. Você pode dar um passo adiante da chamada à função *atoi* usando o comando *next*:

```
(gdb) next
10 printf ("The reciprocal of \%d is \%g\\n", i, reciprocal (i));
```

Se você desejar ver o que está acontecendo dentro de reciprocal, use o comando "step" como segue:

```
(gdb) step
reciprocal (i=7) at reciprocal.cpp:6
6         assert (i != 0);
Current language:  auto; currently c++
```

Você está agora no corpo da função reciprocal. Você pode perceber que é mais conveniente o uso do gdb de dentro do Emacs em lugar de usar o gdb diretamente na linha de comando. Use o comando M-x gdb para iniciar o gdb em uma janela Emacs. Se você tiver parado em um ponto de parada, Emacs automaticamente mostra o arquivo fonte apropriado. Dessa forma fica mais fácil descobrir o que está acontecendo quando você olha no arquivo completo em lugar de apenas em uma linha de texto.

[13]Algumas pessoas têm comentado que colocando um ponto de parada em main é um pouco esquisito porque de maneira geral você somente desejará fazer isso quando main já estiver quebrada.

1.5 Encontrando mais Informação

Praticamente toda distribuição GNU/Linux vem com uma grande quantidade de documentação útil. Você pode ter aprendido mais do que estamos falando aqui nesse livro por meio da leitura da documentação em sua distribuição Linux (embora isso possa provavelmente levar mais tempo). A documentação não está sempre bem organizada, de forma que a parte complicada é encontrar o que precisa. Documentação é também algumas vezes desatualizada, então tome tudo que você vier a ler como pouca informação. Se o sistema não comportar-se no caminho apontado pela página de manual e como ela diz que deve ser, por exemplo, isso pode estar ocorrendo pelo fato de a página de manual estar desatualizada. Para ajudar você a navegar, aqui está as mais úteis fontes de informação sobre programação avançada em GNU/Linux.

1.5.1 Páginas de Manual

Distribuições GNU/Linux incluem páginas de manual para os comandos mais padronizados, chamadas de sistema, e funções da biblioteca C GNU padrão. As *man pages* são divididas em seções numeradas; para programadores, as mais importantes são as seguintes:

- (1) Comandos de usuário
- (2) Chamadas de sistema
- (3) Funções da biblioteca C GNU padrão
- (8) Comandos de Sistema/administrativos

Os números denotam seções das páginas de manual. As páginas de manual do GNU/Linux vêm instaladas no seu sistema; use o comando *man* para acessá-las. Para ver uma página de manual, simplesmente chame-a escrevendo *man nome*, onde *nome* é um comando ou um nome de função. Em alguns poucos casos, o mesmo nome aparece em mais de uma seção; você pode especificar a seção explicitamente colocando o número da seção antes do nome. Por exemplo, se você digitar o seguinte, você irá receber de volta a página de manual para o comando "sleep" (na seção 1 da pagina de manual do GNU/Linux):

```
% man sleep
```

Para ver a página de manual da função de biblioteca "sleep", use o comando adiante:

```
% man 3 sleep
```

Cada página de manual inclui um sumário *on-line* do comando ou da função. O comando *whatis nome* mostra todas as páginas de manual (em todas as seções) para um comando ou função que coincidir com *nome*. Se você não tiver certeza acerca de qual comando ou função você deseja, você pode executar uma pesquisa por palavra chave sobre as linhas de sumário, usando *man -k palavrachave*.

Páginas de manual incluem uma grande quantidade de informações muito úteis e deve ser o primeiro lugar onde você vai para obter ajuda. A página de manual para um comando descreve as opções de linha de comando e argumentos, entrada e saída, códigos de erro, configuração, e coisas semelhantes. A página de manual para um chamada de sistema ou para uma função de biblioteca descreve os parâmetros e valores de retorno, listas de códigos de efeitos colaterais, e especifica quais arquivos devem ser colocados na diretiva *include* se você desejar chamar essa função.

1.5.2 Info

A documentação de sistema do tipo *Info* possuem documentação mais detalhada para muitos dos principais componentes do sistema GNU/Linux, além de muitos outros programas. Páginas *Info* são documentos no formato de hipertexto, semelhantes a páginas *Web*. Para ativar o navegador de páginas *Info* no formato texto, apenas digite *info* em uma janela de *shell*. Você irá ser presenteado com um *menu* de documentos *Info* instalado em seu sistema. (Pressione Ctrl+H para mostrar teclas de navegação em um documento *Info*.)

O conjunto de documentos *Info* que são mais úteis em nosso contexto são esses:

- gcc – O compilador gcc
- Libc – A biblioteca C GNU padrão, incluindo muitas chamadas de sistema
- Gdb – O depurador GNU
- Emacs – O editor de texto Emacs
- Info – O sistema Info propriamente dito

A maioria de todas as ferramentas padronizadas de programação em ambiente GNU/Linux (incluindo o *ld*, o linkador; *as*, o assemblador; e *gprof*, o *profiler*) são acompanhados com páginas *Info* bastante úteis. Você pode ir

diretamente a uma documento *Info* em particular especificando o nome da página *Info* na linha de comando:

```
% info libc
```

Se você fizer a maioria de sua programação no *Emacs*, você pode acessar o navegador interno de páginas *Info* digitando *M-x info* ou *C-h i*.

1.5.3 Arquivos de Cabeçalho

Você pode aprender muito sobre funções de sistema que estão disponíveis e como usá-las olhando nos arquivos de cabeçalho do sistema. Esses arquivos localizam-se em /usr/include e em /usr/include/sys. Se você estiver recebendo erros de compilação ao utilizar uma chamada de sistema, por exemplo, dê uma olhada no arquivo de cabeçalho correspondente para verificar se a assinatura da função é a mesma que a que está listada na página de manual.

Em sistemas GNU/Linux, muitos dos detalhes importantes e centrais de como as chamadas de sistema trabalham estão refletidos nos arquivos de cabeçalho nos diretórios /usr/include/bits, /usr/include/asm, e /usr/include/linux. Por exemplo, os valores numéricos dos sinais (descritos na Seção 3.3, "Sinais" no Capítulo 3, "Processos") são definidos em /usr/include/-bits/signum.h. Esses arquivos de cabeçalho são uma boa leitura para mentes inquiridoras. Não inclua-os diretamente em seus programas; sempre use os arquivos de cabeçalho em /usr/include ou como mencionado na página de manual para a função que você está usando.

1.5.4 Código Fonte

Isso é código aberto, certo? O árbitro final de como o sistema trabalha é o próprio código fonte do sistema, e afortunadamente para programadores em ambiente GNU/Linux, para os quais o código é livremente disponível. Casualmente, sua distribuição inclue o código fonte completo para o sistema completo e todos os programas incluídos nele; se não, você está autorizado nos termos da Licença Pública Geral GNU a requisitar esse código ao distribuidor. (O Código Fonte pode não estar instalado no seu disco. Veja a documentação da sua distribuição para instruções de como instalar os códigos fonte.)

O código fonte para o kernel do GNU/Linux está comumente armazenado no diretório /usr/src/linux. Se esse livro deixa você ansioso por detalher de como os processos, a memória compartilhada, e os dispositivos de sistema trabalham, você sempre pode aprender um pouco mais a partir do código

fonte. A maioria das funções de sistema descritas nesse livro estão implementadas na biblioteca C GNU padrão; verifique na documentação de sua distribição pela localização do código fonte da biblioteca C GNU padrão.

Capítulo 2

Escrevendo Bom Software GNU/Linux

ESSE CAPÍTULO ABRANGE ALGUMAS TÉCNICAS BÁSICAS QUE GRANDE PARTE dos programadores GNU/Linux utilizam. Através das orientações apresentadas adiante, você estará apto a escrever programas que trabalhem bem dentro do ambiente GNU/Linux e atenda às expectativas dos usuários GNU/Linux no que corresponde a como os programas devem trabalhar.

2.1 Interação Com o Ambiente de Execução

Quando você estudou inicialmente C ou C++, aprendeu que a função especial *main* é o ponto de entrada principal para um programa. Quando o sistema operacional executa seu programa, o referido sistema operacional fornece automaticamente certas facilidades que ajudam ao programa comunicar-se com o próprio sistema operacional e com o usuário. Você provavelmente aprendeu sobre os dois primeiros parâmetros para a função principal *main*, comumente chamados *argc* e *argv*, os quais recebem entradas para o seu programa. Você aprendeu sobre *stdout* e *stdin* (ou sobre os fluxos *cout* e *cin* na linguagem C++) que fornecem entrada e saída no console. Esses recursos são fornecidos através das linguagens C e C++, e eles interagem com o sistema GNU/Linux de certas maneiras. GNU/Linux fornece outras formas de interagir com o sistema operacional além das especificadas nesse parágrafo.

2.1.1 A Lista de Argumentos

Você executa um programa a partir de um *prompt* de *shell* através da digitação do nome do programa. Opcionalmente, você pode fornecer informações adicionais para o programa através da digitação de uma ou mais palavras após o nome do programa, separadas por espaços. Essas palavras adiconais são chamadas argumentos de linha de comando. (Você pode também incluir um argumento que contém espaços, empacotando os argumentos entre apóstrofos.) De forma mais geral, o tópico atual é referente a como a lista de argumentos do programa é passada pelo fato de essa lista não precisar ser originária de linha de comando de *shell*. No Capítulo 3, "Processos" você irá ver outro caminho para chamar um programa, no qual um programa pode especificar a lista de argumentos de outro programa diretamente. Quando um programa é chamado a partir do *shell*, a lista de argumentos contém a linha de comando completa, incluindo o nome do programa e quaisquer argumentos de linha de comando que possa ter sido fornecido. Suponhamos, por exemplo, que você chame o comando *ls* em seu *shell* para mostrar o conteúdo do diretório raíz e os correspondentes tamanhos dos arquivos com essa linha de comando:

```
% ls -s /
```

A lista de argumentos que o programa *ls* acima consta de três argumentos. O primeiro deles é o nome do programa propriamente dito, como especificado na linha de comando, *ls* a saber. O segundo e o terceiro elementos da lista de argumentos são os dois argumentos de linha de comando, o "-s" e a "/".

A função *main* de seu programa pode acessar a lista de argumentos por meio dos parâmetros da função *main argc* e *argv* (se você por acaso não utiliza esses dois argumentos, você pode simplesmente omití-los). O primeiro parâmetro, *argc*, é um inteiro que representa o número de argumentos na lista de argumentos. O segundo parâmentro, *argv*, é um vetor de apontadores de caracteres. O tamanho do vetor é *argc*, e os elementos do vetor apontam para os elementos da lista de argumentos, com cada elemento da lista terminado com o caractere nulo "/0".[1]

A utilização de argumentos de linha de comando é tão fácil quanto examinar os conteúdos de *argc* e *argv*. Se você não estiver interessado no nome do programa propriamente dito, lembre-se de ignorar o primeiro elemento.

Logo abaixo temos a Listagem 2.1 que demonstra como usar *argc* e *argv*.

[1]Nota do tradutor: ver [K & R (1989)] p. 113.

Listagem 2.1: (Arquivo arglist.c) Usando *argc* e *argv*.

```
1 #include <stdio.h>
2
3 int main (int argc, char* argv[])
4 {
5   printf ("O nome desse programa e '%s'.\n", argv[0]);
6   printf ("Esse programa foi chamado com %d argumentos.\n", argc - 1);
7
8   /* Ondes quaisquer argumentos de linha de comando sao especificados?  */
9   if (argc > 1) {
10    /* Sim, imprima-os.  */
11    int i;
12    printf ("Os argumentos sao:\n");
13    for (i = 1; i < argc; ++i)
14      printf ("  %s\n", argv[i]);
15  }
16
17  return 0;
18 }
```

2.1.2 Convenções GNU/Linux de Linha de Comando

Quase todos os programas GNU/Linux obedecem algumas convenções sobre como argumentos de linha de comando são interpretados. O argumentos que são passados a programas são de duas categorias: opções (ou sinalizadores) e outros argumentos. As opções modificam como o programa se comporta, enquanto outros argumentos fornecem entradas (por exemplo, os nomes de arquivos de entrada).

Opções chegam de duas formas:

- Opções curtas consistindo de um único hifen e um único caractere (comumente em caixa baixa ou caixa alta). Opções curtas são rápidas de digitar.

- Opções longas consistindo de dois hífens, seguidos por um nome composto em caixa baixa e caixa alta e hífens. Opções longas são fáceis de lembrar e fáceis de ler (em *scripts shell*, por exemplo).

Habitualmente, um programa fornece ambas as formas curta e longa para a maioria das opções que são suportadas pelo referido programa, a forma curta por uma questão de brevidades e a forma longa por uma questão de clareza. Por exemplo, a maioria dos programas entendem as opções -h e --help, e as tratam de forma idêntica. Normalmente, quando um programa é chamado em um *shell* sem nenhum argumento, quaisquer opções desejadas seguem o nome do programa imediatamente. Algumas opções esperam que um argumento as siga imediatamente. Muitos programas, por exemplo, interpretam a opção "–output qualquercoisa" como especificando que a saída de um programa deva ser colocada em um arquivo chamado "qualquercoisa". Após as opções, podem existir adiante delas outros argumentos de linha de comando, tipicamente arquivos de entrada ou dados de entrada.

Por exemplo, o comando "ls -s /" mostra o conteúdo do diretório raíz. A Opção "-s" modifica o comportamento padrão do *ls* instruindo o *ls* a mostrar o tamanho (em kilobytes) de cada entrada.O argumento "/ diz ao *ls* qual diretório listar. A opção *–size* é sinônima da opção "-s", de forma que o mesmo comando pode poderia ter sido digitado como *ls −−size /*.

A codificação GNU padrão lista os nomes de algumas opções de linha de comando comumente usadas. Se você planeja fornecer quaisquer opções similares a alguma dessas, é uma boa idéia usar os nomes especificados na codificação padrão. Seu programa irá se comportar mais como outros programas e irá ser mais fácil para os usuários aprenderem. Você pode visualizar o guia de Condificação GNU Padrão para opções de linha de comando digitando o seguinte em um *prompt* de comandos de um *shell* na maioria dos sistemas GNU/Linux[2]:

```
% info "(standards)User Interfaces"
```

2.1.3 Usando *getopt_long*

A passagem de opções de linha de comando é uma tarefa tediosa. Felizmente, a biblioteca C GNU padrão fornece um função que você usa em programas em C e em programas em C++ para fazer esse trabalho de alguma forma um pouco mais fácil (embora ainda assim um pouco incômoda). Essa função, *getopt_long*, recebe ambas as formas curta e longa de passagem de parâmetros. Se você for usar essa função, inclua o arquivo de cabeçalho <getopt.h>.

Suponha, por exemplo, que você está escrevendo um programa que é para aceitar as três opções mostradas na tabela 2.1.

Tabela 2.1: Opções do Programa Exemplo

Forma Curta	Forma Longa	Propósito
-h	−−help	Mostra sumário de uso e sai
-o nomearquivo	−−output nomearquivo	Especifica o nome do arquivo de saída
-v	−−verbose	Mostra mensagens detalhadas

Adicionalmente, o programa deve aceitar zero ou mais argumentos de linha de comando, que são os nomes de arquivos de entrada.

[2]Nota do tradutor: o guia de Condificação GNU Padrão também pode ser acessado via `http://www.gnu.org/prep/standards/html_node/User-Interfaces.html#User-Interfaces`.

Para usar a função *getopt_long*, você deve fornecer duas estruturas de dados. A primeira é uma sequência de caracteres contendo as opções válidas em sua forma curta, cada letra única. Uma opção que necessite de um argumento é seguida de dois pontos. Para o seu programa, a sequência de caracteres "ho:v" indica que as opções válidas são -h, -o, e -v, com a segunda dessas três opções devendo ser seguida por um argumento.

Para especificar as opções longas disponíveis, você constrói um vetor de elementos de estruturas de opções. Cada elemento corespondendo a uma opção longa e tendo quatro campos. Em circunstâncias normais, o primeiro campo é o nome da opção longa (na forma de uma seqüência de caracteres, sem os dois hífens); o segundo campo é 1 se a opção precisa de argumento, ou 0 em caso contrário; o terceiro campo é *NULL*; e o quarto é um caractere constante especificando a forma curta que é sinônimo da referida opção de forma longa. O último elemento do vetor deve ter todos os campos zerados como adiante. Você pode construir o vetor como segue:

```
const struct option long_options[] = {
{ "help", 0, NULL, 'h' },
{ "output",  1, NULL, 'o' },
{ "verbose", 0, NULL, 'v' },
{ NULL,0, NULL, 0}
};
```

Você chama a função *getopt_long*, passando a ela os argumentos *argc* e *argv* que são passados à função *main*, a sequência de caracteres descrevendo as opções curtas, e o vetor de elementos de estruturas de opções descrevendo as opções longas.

- Cada vez que você chamar *getopt_long*, a função *getopt_long* informa uma única opção, retornando a letra da forma curta para aquela opção ou -1 se nenhuma opção for encontrada.
- Tipicamente, você irá chamar *getopt_long* dentro de um laço, para processar todas as opções que o usuário tiver especificado, e você irá manusear as opções específicas usando o comando *switch*.
- Se a função *getopt_long* encontra uma opção inválida (uma opção que você não especificou como uma opção curta válida ou como uma opção longa válida), a função *getopt_long* imprime uma mensagem de erro e retorna o caractere ? (um ponto de interrogação). A grande maioria dos programas irá encerrar a execução em resposta a isso, possivelmente após mostrar informações de utilização.
- Quando se estiver manuseando uma opção que precisa de um argumento, a varável global *optarg* aponta para o texto daquele argumento.
- Após *getopt_long* terminar de manusear todas as opções, a variável global *optind* conterá o índice (dentro de *argv*) do primeiro argumento não classificado como válido.

A Listagem 2.2 mostra um exemplo de como você pode usar *getopt_long* para processar seus argumentos.

Listagem 2.2: (getopt_long.c) Usando a função *getopt_long*

```
1 #include <getopt.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 /* O nome desse programa.  */
6 const char* program_name;
7
8 /* Mostre informacao de como usar esse programa para STREAM (tipicamente
9    stdout ou stderr), e saia do programa com EXIT_CODE.  Nao
10   retorne.  */
11
12 void print_usage (FILE* stream, int exit_code)
13 {
14   fprintf (stream, "Uso:  %s opcoes [ arquivoentrada ... ]\n", program_name);
15   fprintf (stream,
16            "  -h  --help              Mostra essa informacao de uso.\n"
17            "  -o  --output filename   Escreve a saida para arquivo.\n"
18            "  -v  --verbose           Mostra mensagens detalhadas.\n");
19   exit (exit_code);
20 }
21
22 /* Ponto de entrada do programa principal.  ARGC contem o numero de elementos da
       lista de
23        argumentos; ARGV is an array of pointers to them.  */
24
25 int main (int argc, char* argv[])
26 {
27   int next_option;
28
29   /* Uma string listando letras validas de opcoes curtas.  */
30   const char* const short_options = "ho:v";
31   /* Um array descrevendo opcoes longas validas.  */
32   const struct option long_options[] = {
33     { "help",      0, NULL, 'h' },
34     { "output",    1, NULL, 'o' },
35     { "verbose",   0, NULL, 'v' },
36     { NULL,        0, NULL, 0   }   /* Requerido no fim do array.  */
37   };
38
39   /* O nome do arquivo que recebe a saida do programa, ou NULL para
40      saida padrao.  */
41   const char* output_filename = NULL;
42   /* Se mostra mensagens detalhadas.  */
43   int verbose = 0;
44
45   /* Relembrea o nome do programa, para incorporar nas mensagens.
46      O nome e armazenado em argv[0].  */
47   program_name = argv[0];
48
49   do {
50     next_option = getopt_long (argc, argv, short_options,
51                                long_options, NULL);
52     switch (next_option)
53     {
54     case 'h':   /* -h ou --help */
55       /* O usuario requisitou informacoes de uso.  Mostre-as na saida
56          padrao, e saia com codigo de saida zero (encerrado normalmente).  */
57       print_usage (stdout, 0);
58
59     case 'o':   /* -o ou --output */
60       /* Essa opcao recebe um argumento, o nome do arquivo de saida.  */
61       output_filename = optarg;
62       break;
63
64     case 'v':   /* -v ou --verbose */
65       verbose = 1;
66       break;
67
68     case '?':   /* O usuario especificou uma opcao invalida.  */
69       /* Mostre informacoes de uso para standard error, e saia com codigo de
70          saida um (indicando encerramento anormal).  */
71       print_usage (stderr, 1);
72
73     case -1:    /* Terminado com as opcoes.  */
74       break;
75
76     default:    /* Alguma coisa a mais: inexperado.  */
77       abort ();
78     }
79   }
80   while (next_option != -1);
```

Listagem 2.3: (getopt_long.c) Continuação

```
81    /* Terminado com opcoes.  OPTIND aponta para o primeiro argumento nao opcao.
82       Por propositos de demonstracao, mostre-o se a opcao verbose foi
83       especificada.  */
84    if (verbose) {
85      int i;
86      for (i = optind; i < argc; ++i)
87        printf ("Argumento: %s\n", argv[i]);
88    }
89
90    /* O programa principal e desenvolvido aqui.  */
91
92    return 0;
93  }
```

O uso de *getopt_long* pode ser visto como muito trabalho, mas escrevendo código para passar as opções de linha de comando propriamente ditas pode ser mais trabalhoso ainda. A função *getopt_long* é muito sofisticada e permite grande flexibilidade na especificação de qual tipo de opção aceitar. Todavia, é uma boa idéia adiar a adoção de recursos mais avançados e segurar-se um pouco na estrutura básica de opção descrita acima.

2.1.4 E/S Padrão

A biblioteca C GNU padrão fornece fluxos de entrada e saída padrão (*stdin* e *stdout*, respectivamente). Tanto a entrada como a saída padrão são usadas por *scanf*, *printf*, e outras funções da biblioteca C GNU padrão. Na tradição UNIX, o uso da entrada e da saída padrão é comum e habitual para programas GNU/Linux. Isso permite encadeamento de multiplos programas usando *pipes* de *shell* e redirecionamentos de entrada e saída. (Veja a página de manual para o seu *shell* preferido para aprender a sintaxe nesses casos de *pipes* e redirecionamentos.)

A biblioteca C GNU padrão também fornece *stderr*, o fluxo padrão de erro. Programas podem mostrar mensagens de erro para a saída padrão de erro em lugar de enviar para a saída padrão. Esse tipo de comportamento permite aos usuários separarem a saída normal e mensagens de erro, por exemplo, através do redirecionamento da saída padrão para um arquivo enquanto permite a impressão da saída de erro para o console. A função *fprintf* pode ser usada para imprimir para a saída padrão de erro *stderr*, por exemplo:

```
fprintf (stderr, (``Error: ...")); 
```

Esses três fluxos[3] são também accessíveis com os comandos básicos UNIX de E/S (*read*, *write*, e assim por diante) por meio dos três descritores de

[3]Nota do tradutor:stdin, stdout e stderr.

arquivo usados em *shell*. Os descritores são 0 para *stdin*, 1 para *stdout*, e 2 para *stderr*.

Quando um programa for chamado, pode ser algumas vezes útil redirecionar ambas, a saída padrão e a saída de erro, para um arquivo ou *pipe*. A sintaxe para fazer isso varia nos diversos *shells*; para *shells* do estilo *Bourne* (incluindo o *bash*, o shell padrão na maioria das distribuições GNU/Linux), dois exemplos são mostrados logo abaixo:

```
% programa > arquivo_saida.txt 2>&1
% programa 2>&1 | filtro
```

A sintaxe *2>&1* indica que o descritor 2 de arquivo (*stderr*) deve ser entregue no descritor de arquivo 1 (stdout). Note que *2>&1* deve vir após um redirecionamento de arquivo (a primeira linha exemplo logo acima) mas deve vir antes de um redirecionamento por meio de pipe (a segunda linha exemplo logo acima).

Note que *stdout* é armazenada em uma área temporária. Dados escritos para *stdout* não são enviados para o console (ou para outro dispositivo caso haja redirecionamento) imediatamente. Dados escritos para *stdout* são enviados para o console em três situações: quando a área de armazenamento temporário esteja preenchida completamente, quando o programa terminar normalmente ou quando *stdout* for fechada. Você pode explicitamente descarregar a área de armazenamento temporária através da seguinte chamada:

```
fflush (stdout);
```

Por outro lado, *stderr* não é armazenada em um local temporário; dados escritos para *stderr* vão diretamente para o console. [4]

Isso pode produzir alguns resultados surpreendentes. Por exemplo, esse laço não mosta um ponto a cada segundo; em vez disso, os pontos são armazenados em uma área temporária, e um grupo de pontos é mostrado todos de uma única vez quando o limite de armazenamento da área temporária é alcançado.

```
while (1) {
  printf (".");
  sleep (1);
}
```

[4]Em C++, a mesma distinção se mantém para *cout* e para *cerr*, respectivamente. Note que a marca *endl* descarrega um fluxo adicionalmente à impressão um caractere de nova linha; se você não quiser descarregar um fluxo (por razões de performace, por exemplo), use em substituição a *endl* uma constante de nova linha, '\n'.

No laço adiante, todavia, o ponto aparece uma vez a cada segundo:

```
while (1) {
  fprintf (stderr, ".");
  sleep (1);
}
```

2.1.5 Códigos de Saída de Programa

Quando um programa termina, ele indica sua situação de saída com um código de saída. O código de saída é um inteiro pequeno; por convenção, um código de saída zero denota execução feita com sucesso, enquanto um código de saída diferente de zero indica que um erro ocorreu. Alguns programas usam diferentes valores de códigos diferentes de zero para distinguir erros específicos. Com a maioria dos *shells*, é possível obter o código de saída do programa executado mais recentemente usando a variável especial $? (ponto de interrogação). Aqui está um exemplo no qual o comando *ls* é chamado duas vezes e seu código de saída é mostrado a cada chamada. No primeiro caso, *ls* executa corretamente e retorna o código de saída zero. No segundo caso, *ls* encontrou um erro (porque o nome de arquivo especificado na linha de comando não existe) e dessa forma retorna um código de saída diferente de zero:

```
% ls /
bincoda etc libmisc nfs proc  sbinusr
boot dev home lost+found mnt  opt root tmp var
% echo $?
0
% ls bogusfile
ls: bogusfile: No such file or directory
% echo $?
1
```

Um programa em C ou em C++ especifica seu código de saída através do retorno do código de saída devolvido pela função *main*. Existem outros métodos de fornecer códigos de saída, e códigos de saída especial são atribuídos a programas que terminam de forma diferente da esperada (por meio de um sinal). Isso será discutido adicionalmente no Capítulo 3.

2.1.6 O Ambiente

GNU/Linux fornece a cada programa sendo executado um ambiente. O ambiente é uma coleção de pares variável/valor. Ambos nome de variáveis

de ambiente e seus valores respectivos são sequências de caracteres. Por convenção, nomes de variáveis de ambiente são grafados com todas as letras em maiúscula.

Você provavelmente já está familiarizado com muitas variáveis de ambiente mais comuns. Por exemplo:

- *USER* contém seu nome de usuário.
- *HOME* contém o caminho para seu diretório de usuário.
- *PATH* contém uma lista de itens separada por ponto e vírgula dos diretórios os quais GNU/Linux busca pelo comando que você chamar.
- *DISPLAY* contém o nome e o número do display do servidor sobre o qual janelas de programas gráficos do X irão aparecer.

Seu *shell*, como qualquer outro programa, tem um ambiente. *Shells* fornecem métodos para examinar e modificar o ambiente diretamente. Para mostrar o ambiente atual em seu *shell*, chame o programa *printenv*. Vários *shells* possuem diferentes sintaxes internas para a utilização de variáveis de ambiente; o que é mostrado adiante é a sintaxe no estilo dos *shells* do tipo *Bourne*.

- O *shell* automaticamente cria uma variável *shell* para cada variável de ambiente que encontrar, de forma que você acessar valores de variáveis de ambiente usando a sintaxe *$nomedevariavel*. Por exemplo:

  ```
  % echo $USER
  samuel
  % echo $HOME
  /home/samuel
  ```

- Você pode usar o comando *export* para exportar uma variável *shell* dentro do ambiente. Por exemplo, para modificar a variável de ambiente EDITOR, você pode usar o seguinte:

  ```
  % EDITOR=emacs
  % export EDITOR
  ```

  Ou, de forma curta e rápida:

  ```
  % export EDITOR=emacs
  ```

Em um programa, você acessa uma variável de ambiente com a função *getenv* na <stdlib.h>. A função *getenv* pega um nome de variável e retorna o valor correspondente como uma sequência de caracteres, ou *NULL* se a referida variável não tiver sido definida no ambiente. Para modificar ou limpar variáveis de ambiente, use as funções *setenv* e *unsetenv*, respectivamente. Listar todas as variáveis de um ambiente é um pouco complicado. Para fazer isso, você deve acessar uma variável global especial chamada *environ*, que é definida na biblioteca C GNU padrão. Essa variável, do tipo *char***, é um vetor de apontadores terminado com o caractere *NULL* que apontam para sequências de caracteres. Cada sequência de caracteres contendo uma variável de ambiente, na forma VARIÁVEL=valor. O programa na Listagem 2.4, por exemplo, simplesmente mostra na tela todas as variáveis de ambiente através de um laço ao longo do vetor de apontadores *environ*.

Listagem 2.4: (print-env.c) Mostrando o Ambiente de Execução

```
1 #include <stdio.h>
2
3 /* A variavel ENVIRON contem o ambiente.  */
4 extern char** environ;
5
6 int main ()
7 {
8   char** var;
9   for (var = environ; *var != NULL; ++var)
10    printf ("%s\n", *var);
11  return 0;
12 }
```

Não modifique o ambiente propriamente dito; use as funções *setenv* e *unsetenv* para fazer as modificações que você precisar. Comumente, quando um novo programa é iniciado, ele herda uma cópia do ambiente do programa que o chamou (o programa de *shell*, se o referido programa tiver sido chamado de forma interativa). Dessa forma, por exemplo, programas que você executa a partir de um programa de *shell* pode examinar os valores das variáveis de ambiente que você escolheu no *shell* que o chamou.

Variáveis de ambiente são comumente usadas para indicar informações de configuração a programas. Suponha, por exemplo, que você está escrevendo um programa que se conecta a um servidor *Internet* para obter alguma informação. Você pode ter escrito o programa de forma que o nome do servidor seja especificado na linha de comando. Todavia, suponha que o nome do servidor não é alguma coisa que os usuários irão modificar muitas vezes. Você pode usar uma variável especial de ambiente digamos *SERVER_NAME* para especificar o nome do servidor; se *SERVER_NAME* não existir, um valor padrão é usado. Parte do seu programa pode parecer como mostrado na Listagem 2.5.

Listagem 2.5: (client.c) Parte de um Programa Cliente de Rede

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main ()
5 {
6   char* server_name = getenv ("SERVER_NAME");
7   if (server_name == NULL)
8     /* A variavel de ambiente SERVER_NAME nao foi ajustada.  Use o
9        padrao.  */
10    server_name = "server.my-company.com";
11
12  printf ("acessando o servidor %s\n", server_name);
13  /* Acesse o servidro aqui...  */
14
15  return 0;
16 }
```

Suponhamos que o programa acima seja chamado de *client*. Assumindo que você não tenha criado ou que não tenha sido criada anteriormente a variável *SERVER_NAME*, o valor padrão para o nome do servidor é usado:

```
% client
accessing server server.my-company.com
```

Mas é fácil especificar um servidor diferente:

```
% export SERVER_NAME=backup-server.emalgumlugar.net
% client
accessing server backup-server.emalgumlugar.net
```

2.1.7 Usando Arquivos Temporários

Algumas vezes um programa necessita criar um arquivo temporário, para armazenar grandes dados por alguns instantes ou para entregá-los a outro programa. Em sistemas GNU/Linux, arquivos temporários são armazenados no diretório /tmp. Quando fizer uso de arquivos temporários, você deve estar informado das seguintes armadilhas:

- Mais de uma instância de seu programa pode estar sendo executada simultâneamente (pelo mesmo usuário ou por diferentes usuários). As instâncias devem usar diferentes nomes de arquivos temporários de forma que eles não colidam.
- As permissões dos arquivos temporários devem ser ajustadas de tal forma que somente usuários autorizados possam alterar a execução do programa através de modificação ou substituição do arquivo temporário.
- Nomes de arquivos temporários devem ser gerados de forma imprevisível externamente; de outra forma, um atacante pode usar a espera entre a verificação de que um nome de arquivo fornecido já está sendo usado e abrir um novo arquivo temporário.

GNU/Linux fornece funções, *mkstemp* e *tmpfile*, que cuidam desses recursos para você de forma adequada (e adicionalmente muitas funções que não cuidam)[5]. Qual você irá usar depende de seu planejamento de manusear o arquivo temporário para outro programa, e de se você deseja usar E/S UNIX (*open*, *write*, e assim por diante) ou as funções de controle de fluxos da biblioteca C GNU padrão(*fopen*, *fprintf*, e assim por diante).

[5]Nota do tradutor: no *slackware* tem a *mktemp*.

Usando *mkstemp* A função *mkstemp* criará um nome de arquivo temporário de forma única a partir de um modelo de nome de arquivo, cria o arquivo propriamente dito com permissões de forma que somente o usuário atual possa acessá-lo, e abre o arquivo para leitura e escrita. O modelo de nome de arquivo é uma sequência de caracteres terminando com "XXXXXX" (seis letras X maiúsculas); *mkstemp* substitui as letras X por outros caracteres de forma que o nome de arquivo seja único. O valor de retorno é um descritor de arquivo; use a família de funções aparentadas com a função *write* para escrever no arquivo temporário. Arquivos temporários criados com *mkstemp* não são apagados automaticamente. Compete a você remover o arquivo temporário quando o referido arquivo temporário não mais for necessário. (Programadores devem ser muito cuidadosos com a limpeza de arquivos temporários; de outra forma, o sistema de arquivos /tmp irá encher eventualmente, fazendo com que o sistema fique inoperante.) Se o arquivo temporário for de uso interno somente e não for manuseado por outro programa, é uma boa idéia chamar *unlink* sobre o arquivo temporário imediatamente. A função *unlink* remove a entrada do diretório correspondente a um arquivo, mas pelo fato de arquivos em um sistema de arquivos serem contados-referenciados, o arquivos em si mesmos não são removidos até que não hajam descritores de arquivo abertos para aquele arquivo. Dessa forma, seu programa pode continuar usando o arquivo temporário, e o arquivo evolui automaticamente até que você feche o descritor do arquivo. Pelo fato de GNU/Linux fechar os descritores de arquivo quando um programa termina, o arquivo temporário irá ser removido mesmo se seu programa terminar de forma abrupta.

O par de funções na Listagem 2.6 demonstra *mkstemp*. Usadas juntas, essas duas funções tornam fácil escrever o conteúdo de uma área temporária de armazenamento na memória para um arquivo temporário (de forma que a memoria possa ser liberada ou reutilizada) e de forma que esse conteúdo armazenado possa ser trazido de volta à memória mais tarde.

Listagem 2.6: (temp_file.c) Usando *mkstemp*

```
1 #include <stdlib.h>
2 #include <unistd.h>
3
4 /* Um manipulador para um arquivo temporario criado com write_temp_file.  Nessa
5    implementacao, o arquivo temporario e apenas um descritor de arquivo.  */
6 typedef int temp_file_handle;
7
8 /* Escreva LENGTH bytes de BUFFER para um arquivo temporario.  o
9    arquivo temporario e imediatamente unlinked.  Retorna um manipulador para o
10   arquivo temporario.  */
11
12 temp_file_handle write_temp_file (char* buffer, size_t length)
13 {
14   /* Cria o filename e o file.  O XXXXXX ira ser substituido com
15      caracteres que fazem o filename unico.  */
16   char temp_filename[] = "/tmp/temp_file.XXXXXX";
17   int fd = mkstemp (temp_filename);
18   /* Unlink o arquivo imediatamente, de forma que o arquivo ira ser removido quando o
19      descritor de arquivo for fechado.  */
20   unlink (temp_filename);
21   /* Escreve o numero de bytes para o arquivo primeiramente.  */
22   write (fd, &length, sizeof (length));
23   /* Agora escreve os dados propriamente ditos.  */
24   write (fd, buffer, length);
25   /* Use o descritor de arquivo como o manipulador para o arquivo temporario.  */
26   return fd;
27 }
28
29 /* Leia o conteudo de um arquivo temporario TEMP_FILE criado com
30    write_temp_file.  O vaor de retorno e um meis recentemente alocado espaco
         temporario
31    com aquele conteudo, o qual o chamador deve desalocar com free.
32    *LENGTH e ajustado para o tamanho do conteudo, em bytes.  O
33    aruivo temporario e removido.  */
34
35 char* read_temp_file (temp_file_handle temp_file, size_t* length)
36 {
37   char* buffer;
38   /* O manipulador TEMP_FILE e um descritor de arquivo para o arquivo temporario.  */
39   int fd = temp_file;
40   /* Volte para o inicio do arquivo.  */
41   lseek (fd, 0, SEEK_SET);
42   /* Leia o tamanhos dos dados no arquivo temporario.  */
43   read (fd, length, sizeof (*length));
44   /* Aloque um espaco temporario e leia os dados.  */
45   buffer = (char*) malloc (*length);
46   read (fd, buffer, *length);
47   /* Feche o descritor de arquio, o qual ira fazer com que o arquivo temporario
48   va embora.  */
49   close (fd);
50   return buffer;
51 }
```

Usando *tmpfile* Se você está usando as funções de E/S da biblioteca C GNU padrão e não precisa passar o arquivo temporário para outro programa, você pode usar a função *tmpfile*. Essa função cria e abre um arquivo temporário, e retorna um apontador de arquivo para esse mesmo arquivo temporário. O arquivo temporário já é *unlinked*, como no exemplo anterior, de forma que será apagado automaticamente quando quando o apontador de arquivo for fechado (com *fclose*) ou quando o programa terminar.

GNU/Linux fornece muitas outras funções para a geração de arquivos temporaários e nomes de arquivos temporários, incluindo *mktemp*, *tmpnam*, e *tempnam*. Não use essas funções, apesar disso, pelo fato de elas possuírem problemas de confiabilidade e segurança já mencionados anteriormente.

2.2 Fazendo Código Defensivamente

Escrevendo programas que executam atualmente sob uso "normal" é trabalhoso; escrever programas que comportam-se de forma elegante em situações de falha é mais trabalhoso ainda. Essa seção demonstra algumas técnicas de codificação para encontrar erros facilmente e para detectar e recuperar-se de problemas durante a execução de um programa.

As amostras de código apresentadas mais adiante nesse livro omitem erros extensivos de verificação e recuperação de código pelo fato de isso eventualmente vir a obscurecer a funcionalidade básica que se deseja apresentar aquí. Todavia, o exemplo final no capítulo 11, "Um Modelo de Aplicação GNU/-Linux" retorna à demonstração de como usar essas técnicas para escrever programas robustos.

2.2.1 Usando *assert*

Um bom objetivo para se ter em mente quando criamos um código fonte de uma aplicação é que erros comuns ou mesmo erros inesperados podem fazer com que o programa falhe de forma dramática, tão facilmente quanto possível. O uso de *assert* irá ajudar você a encontrar erros facilmente no desenvolvimento e na fase de teste. Falhas que não se mostram de forma evidente passam surpreendentemente e muitas vezes desapercebidas e não se mostram até que a aplicação esteja nas mãos do usuário final.

Um dos mais simples métodos de verificar condições inesperadas é a macro *assert* da biblioteca C GNU padrão. O argumento para essa macro é uma expressão Booleana. O programa é terminado se a expressão Booleana avaliar para *false*, após mostrar uma mensagem de erro contendo o código fonte e o número da linha e o texto da expressão. A macro *assert* é muito útil para uma larga variedade de verificações de consistências internas em um dado programa. Por exemplo, use *assert* para testar a validade de argumentos de funções, para testar condições prévias e condições póstumas de chamadas a funções (e chamadas a métodos, em C++), e para testar valores de retorno.

Cada utilização de *assert* serve não somente como uma verificação em tempo de execução de uma condição, mas também como documentação sobre a operação do programa dentro do código fonte. Se seu programa contiver um *assert (condição)* que diz a alguém para ler seu código fonte pelo fato de a condição obrigatóriamente ter de ser verdadeira naquele ponto do programa, e se a condição não é verdadeira, temos aí um erro no programa. Para código de desempenho crítico, verificações tais como a utilização de *assert* podem impor uma perda muito grande de desempenho. Nesses casos,você pode compilar seu código com a macro *NDEBUG* definida, através do uso

do sinalizador *-DNDEBUG* na sua linha de comando de compilação. Com *NDEBUG* definida, aparições da macro *assert* irão ser preprocessadamente descartadas. O preprocessamento dessa forma é uma boa idéia no sentido de permitir fazer o uso de *assert* somente quando necessário por razões de performace, embora que, somente com arquivos fonte de desempenho crítico. Pelo fato de ser possível o descarte preprocessadamente da macro *assert*, garanta que qualquer expressão que você venha a usar com *assert* não tenha efeitos colaterais. Especificamente, você não deve chamar funções dentro de expressões *assert*, não deve atribuir valores a variáveis e não deve usar modificadores de operação tais como *++*.

Suponhamos, por exemplo, que você chame uma função, *fazer_algumacoisa*, repetidamente em um laço. A função *fazer_algumacoisa* retorna zero em caso de sucesso e não zero em caso de falha, mas você não espera que esse comportamento venha a falhar em seu programa. Você pode ter tentado escrever:

```
 for (i = 0; i < 100; ++i)
assert (fazer_algumacoisa () == 0);
```

Todavia, você pode encontrar que essa verificação em tempo de execução impõe uma grande perda de desempenho e decide mais tarde recompilar com *NDEBUG* definida. Essa recompilação com *NDEBUG* definida irá remover a chamada a *assert* inteiramente, de forma que a expressão nunca irá ser avaliada e *fazer_algumacoisa* nunca irá ser chamada. Você pode, ao invés do código anterior escrever o seguinte:

```
 for (i = 0; i < 100; ++i) {
int status = fazer_algumacoisa ();
assert (status == 0);
 }
```

Outra coisa para se ter em mente é que você não deve usar *assert* para testar entradas inválidas de usuário. Usuários não gostam quando aplicações simplesmente terminam abruptamente com uma mensagem de erro criptografada, mesmo em resposta a uma entrada inválida. Você deve sempre verificar entradas inválidas e produzir mensagens de erro coerentes e lógicas em resposta a uma tal entrada inválida. Use *assert* somente para verificações internas em tempo de execução.

Alguns bons lugares para usar *assert* são esses:

- Verificação contra apontadores nulos, por exemplo, como argumentos válidos a funções. A mensagem de erro gerada por *assert (pointer != NULL)*,

  ```
  Assertion 'pointer != ((void *)0)' failed.
  ```

  é mais informativa que a mensgem de erro que pode resultar se seu programa simplesmente tentar acessar um apontador nulo:

  ```
  Segmentation fault (core dumped)
  ```

- Verifique condições sobre valores de parâmetros passados a funções. Por exemplo, se uma função deve ser chamada somente com um valor positivo para o parâmetro *qualquercoisa*, use o seguinte no começo do corpo da função:

  ```
  assert (qualquercoisa > 0);
  ```

  Isso irá ajudar você a detectar uso inadequado da função, e essa prática também faz com que esteja muito claro a alguém que ao ler o código fonte da função verá que existe uma restrição sobre valores do parâmetro.

Evolua; use *assert* de forma liberal em toda a extensão de seu código.

2.2.2 Falhas em Chamadas de Sistema

A maioria de nós originalmente aprendeu como escrever programas que executam até o final ao longo de um caminho bem definido. Dividimos o programa em tarefas e sub-tarefas, e cada função completa uma tarefa através de chamadas a outras funções para executar as sub-tarefas correspondentes. Fornecendo entradas apropriadas, esperamos que uma função produza a saída correta e os efeitos corretos. As realidades das peças do computador e dos programas de computador intromete-se nesse sonho perfeito. Computadores possuem recursos limitados; peças falham; muitos programas funcionam ao mesmo tempo; usuários e programas cometem erros. Isso muitas vezes no limite entre a aplicação e o sistema operacional que essas realidades exibem por si mesmas. Portanto, quando formos usar chamadas de sistema para acessar recursos, para realizar operações de E/S, ou para outro propósito, é importante entender não somente o que ocorre quando a chamada acontece,

mas também quando e como a chamada de sistema pode falhar. Chamadas de sistema falham de muitas formas. Por exemplo:

- O sistema pode extrapolar os recursos disponíveis de *hardware* (ou o programa excede os limites de recursos impostos pelo sistema para um único programa). Por exemplo, o programa pode tentar alocar muita memória, escrever muito no disco, ou abrir muitos arquivos ao mesmo tempo.
- GNU/Linux pode bloquear uma certa chamada de sistema quando um programa tenta executar uma operação para a qual não tiver permissão. Por exemplo, um programa pode tentar escrever em um arquivo marcado como somente para leitura, acessar a memória de outro processo, ou encerrar outro programa de usuário.
- Os argumentos a uma chamada de sistema podem ser inválidos, ou devido ao usuário fornecer entradas inválidas ou devido a um erro no programa. Por exemplo, o programa pode passar a outro programa um endereço inválido de memória ou um descritor de arquivo inválido para uma chamada de sistema. Ou, um programa pode tentar abrir um diretório como um arquivo, ou pode passar o nome de um arquivo a uma chamada de sistema que espera um diretório.
- Uma chamada de sistema falha por razões externar a um programa. Isso aconteçe na maioria das vezes quando uma chamada de sistema acessa um dispositivo. O dispositivo pode estar danificado ou pode não suportar uma operação em particular, ou talvez um disco não está inserido no dispositivo de leitura e escrita em disco.
- Uma chamada de sistema pode muitas vezes ser interrompida por um evento externo, tal como a entrega de um sinal. Isso não necessariamente indica falha externa, mas ocorrer em resposta à chamada de um programa para reiniciar a chamada de sistema, se for desejável.

Em um programa bem escrito que faz uso extensivo de chamadas de sistema, a falha de chamada de sistema causa o aparecimento de mais código devotado a detectar e controlar erros e outras circunstâncias excepcionais que não o código específico dedicado ao trabalho principal do programa.

2.2.3 Códigos de Erro de Chamadas de Sistema

A maioria das chamadas de sistema retorna zero se a operação terminar corretamente, ou um valor diferente de zero caso a operação resultar em falha. (Muitas outras chamadas, apesar disso, possuem diferentes conveções de valores de retorno; por exemplo, a chamada *malloc* retorna um apontador nulo para indicar falha. Sempre leia a página de manual cuidadosamente quando for usar uma chamada de sistema.) Embora essa informação possar suficiente para determinar se o programa deva continuar a execução normalmente, a leitura da página de manual provavelmente não fornece informação suficiente para um recuperação satisfatória de erros.

A maioria das chamadas de sistema usam uma variável especial chamada *errno* para armazenar informações adicionais em caso de falha. [6] Quando uma chamada vier a falhar, o sistema ajusta *errno* para um valor indicando o que aconteceu de errado. Pelo fato de todas as chamadas de sistema usarem a mesma variável *errno* para armazenar informações de erro, você deve copiar o valor para outra variável imediatamente após ocorrer a falha na chamada. A *errno* irá ter seu valor atual apagado e preenchido com outros valores da próxima vez que você fizer uma chamada de sistema.

Valores de erro são inteiros; os valores possíveis são fornecidos pelas macros de pré-processamento, por convenção nomeadas em letras maiúsculas e iniciando com "E", por exemplo, *EACCES* e *EINVAL*. Sempre use essas macros para referir-se a valores de *errno* em lugar de valores inteiros. Inclua o cabeçalho <errno.h> se você for usar valores de *errno*.

GNU/Linux fornece uma função conveniente, *strerror*, que retorna uma descrição em forma de sequência de caracteres de um código de erro que se encontra armazenado em *errno*, adequada para usar em mensagens de erro. Inclua o arquivo de cabeçalho <string.h> caso você resolva usar a função *strerror*.

GNU/Linux também fornece *perror*, que mostra a descrição do erro diretamente para o fluxo *stderr*. Passe a *perror* uma sequência de caracteres para ser usada como prefixo a ser mostrado antes da descrição de erro, que deve habitualmente incluir o nome da função que falhou. Inclua o arquivo de cabeçalho <stdio.h> caso você resolva usar a função *perror*.

O fragmento de código adiante tenta abrir um arquivo; se a abertura falhar, o código mostra uma mensagem de erro e encerra a execução do programa. Note que a chamada *open* retorna um descritor de arquivo aberto se o operador *open* obtiver sucesso em sua tarefa, ou -1 se a operação falhar.

```
fd = open ("arquivodeentrada.txt", O_RDONLY);
```

[6] Atualmente, por razões de trabalhar de forma segura, *errno* é implementada como uma macro, mas é usada como uma variável global.

```
if (fd == -1) {
/* A abertura falhou. Mostra uma menssagem de erro e sai. */
fprintf (stderr, "erro ao abrir o arquivo: %s\n", strerror (errno));
exit (1);
 }
```

dependendo de seu programa e da natureza da chamada de sistema, a ação apropriada ao caso de falha pode ser mostrar uma mensagem de erro para cancelar uma operação, abortar o programa, tentar novamente, ou mesmo para ignorar o erro. A menção desse comportamento é importante pelo fato de ser necessário incluir código que manuseie todos os possíveis modos de falha de uma forma ou de outra.

Um possível código de erro que você deve ficar de olho, especialmente com funções de E/S, é *EINTR*. Algumas funções, tais como *read*, *select*, e *sleep*, podem precisar de um intervalo de tempo significativo para executar. Essas são consideradas funções de bloqueio pelo fato de a execução do programa ser bloqueada até que a chamada seja completada. Todavia, se o programa recebe um sinal enquanto estiver bloqueado em uma dessas chamadas, a chamada irá retornar sem completar a operação. Nesse caso, *errno* é ajustada para *EINTR*. Comumente, você irá querer chamar novamente a chamada de sistema que foi interrompida pelo sinal nesse caso.

Adiante encontra-se um fragmento de código que utiliza a chamada *chown* para mudar o dono de um arquivo fornecido pela variável *path* para o usuário especificado através de *user_id*. Se a chamada vier a falhar, o programa executa uma ação que depende do valor de *errno*. Note que quando detectamos o que é provavelmente um erro no programa nós saimos usando *abort* ou *assert*, o que causa a geração de um arquivo *core*. Esse arquivo pode ser útil para depuração após o encerramento do programa. Para outros erros irrecuperáveis, tais como condições de tentativas de acesso a áreas de memória não alocadas pelo sistema operacional ao programa em questão, saimos usando *exit* e um valor de saída não nulo em lugar de arquivo *core* pelo fato de que um arquivo *core* pode não vir a ser muito útil.

```
rval = chown (path, user_id, -1);
if (rval != 0) {
  /* Grava errno pelo fato de poder ser sobrescrito pela proxima chamada de sistema. */
  int error_code = errno;
  /* A operacao falha chown deve retornar -1 em caso de erro. */
  assert (rval == -1);
  /* Verifica o valor de errno, e executa a acao apropriada. */
  switch (error_code) {
  case EPERM:/* Permissao negada. */
  case EROFS:/* PATH esta em um sistema de arquivo somente leitura. */
  case ENAMETOOLONG: /* PATH e muito longo. */
  case ENOENT:  /* PATH nao exite. */
  case ENOTDIR: /* Um componente de PATH nao eh um diretorio. */
  case EACCES:  /* Um componente de PATH nao esta acessivel. */
 /* Algo esta errado com o arquivo. Mostre uma mensagem de erro. */
 fprintf (stderr, "erro mudando o dono de %s: %s\n",
 path, strerror (error_code));
 /* Nao encerra o programa; talvez fornecao ao usuario uma chance para
 escolher outro arquivo... */
 break;

  case EFAULT:
 /* PATH contem um endereco de memoria invalido. Isso eh provavelmente um erro.*/
```

```
 abort ();

  case ENOMEM:
 /* Executou fora da memoria do kernel. */
 fprintf (stderr, "%s\n", strerror (error_code));
 exit (1);

  default:
  /* Alguma outra coisa, inesperado, codigo de erro. Tentamos manusear todos os
  erros de codigo possiveis; se tivermos omitido algum, isso eh um erro! */
 abort ();
  };
}
```

Você pode simplesmente usar o código abaixo, que comporta-se da mesma forma se a chamada obtiver sucesso:

```
 rval = chown (path, user_id, -1);
 assert (rval == 0);
```

Mas se a chamada vier a falhar, a alternativa de código acima não faz nenhum esforço para reportar, manusear, ou para se recuperar dos erros.

Se você usa a primeira forma, a segunda forma, ou algum meio termo entre as duas vai depender da necessidade de seu sistema no tocante a detecção e recuperação de erros.

2.2.4 Erros e Alocação de Recursos

Muitas vezes, quando uma chamada de sistema falha, é mais apropriado cancelar a operação atual mas não terminar o programa porque o cancelamento simples pode tornar possível recuperar-se do erro. Uma forma de fazer isso é retornar da função em que se está no momento em que ocorreu o erro, passando um código de retorno para a função chamadora indicando o erro.

Caso você decida retornar a partir do meio de uma função, é importante garantir que quaisquer recursos que tenham sido alocados com sucesso previamente na função sejam primeiramente liberados. Esses recursos podem incluir memória, descritores de arquivo, apontadores para arquivo, arquivos temporários, objetos de sincronização, e assim por diante. De outra forma, se seu programa continuar sendo executado, os recursos alocados anteriormente à ocorrência da falha irão ser perdidos.

Considere, por exemplo, uma função que faça a leitura de um arquivo em um espaço temporário de armazenamento. A função pode seguir esses passos:

1. Alocar o espaço temporário de armazenamento.
2. Abrir o arquivo.
3. Ler a partir do arquivo na área temporária de armazenamento.
4. Fechar o arquivo.
5. Devolver o espaço temporário de armazenamento.

Se o arquivo não existir, o Passo 2 irá falhar. Um caminho de ação pode ser retornar um apontador a partir da função. Todavia, se o espaço de armazenamento temporário já tiver sido alocado no Passo 1, existe um risco de perder aquela memória. Você deve lembrar de desalocar o espaço temporário de armazenamento em algum lugar com o decorrer de qualquer fluxo de controle do qual você não venha a retornar. Se o Passo 3 vier a falhar, você não somente deve desalocar o espaço temporário de armazenamento antes de retornar, mas também deve fechar o arquivo.

A Listagem 2.7 mostra um exemplo de como você pode escrever essa função.

Listagem 2.7: (readfile.c) Liberando Recursos em Condições Inesperadas

```
1 #include <fcntl.h>
2 #include <stdlib.h>
3 #include <sys/stat.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6
7 char* read_from_file (const char* filename, size_t length)
8 {
9   char* buffer;
10  int fd;
11  ssize_t bytes_read;
12
13  /* Aloca o espaco temporario de armazenagem.  */
14  buffer = (char*) malloc (length);
15  if (buffer == NULL)
16    return NULL;
17  /* Abre o arquivo.  */
18  fd = open (filename, O_RDONLY);
19  if (fd == -1) {
20    /* abertura falhou.  Desaloque o espaco temporario de armazenagem antes de
          retornar.  */
21    free (buffer);
22    return NULL;
23  }
24  /* Leia os dados.  */
25  bytes_read = read (fd, buffer, length);
26  if (bytes_read != length) {
27    /* read falhou.  Desaloque o espaco temporario e feche fd antes de retornar.  */
28    free (buffer);
29    close (fd);
30    return NULL;
31  }
32  /* Tudo esta bem.  Feche o arquivo e retorne o conteudo do espaco temporario de
        armazenagem.  */
33  close (fd);
34  return buffer;
35 }
```

Gnu/Linux limpa a memória alocada, limpa os arquivos abertos, e libera a maioria de outros recursos quando um programa encerra, de forma que

não é necessário desalocar espaços temporários de armazenamento e fechar arquivos antes de chamar *exit*.

Você pode precisar liberar manualmente outros recursos compartilhados, todavia, tais como arquivos temporários e memória compartilhada, que podem potencialmente sobreviver ao encerramento de um programa.

2.3 Escrevendo e Usando Bibliotecas

Virtualmente todos os programas são linkados usando uma ou mais bibliotecas. Qualquer programa que usa uma função C (tais como *printf* ou *malloc*) irá ser linkado incluindo a biblioteca C GNU padrão de rotinas que atuam em tempo de execução. Se seu programa tem uma interface gráfica de usuário (GUI), seu programa será linkado incluindo bibliotecas que fazem janelas. Se seu programa usa uma base de dados, o provedor da base de dados irá fornecer a você bibliotecas que você pode usar para acessar a base de dados convenientemente. Em cada um desses casos, você deve decidir se irá linkar a biblioteca *estaticamente* ou *dinâmicamente*. Se você escolher *estaticamente*, seu programa irá ser maior e mais pesado na hora de atualizar, mas provavelmente fácil de desenvolver. Se você linkar *dinâmicamente*, seu programa irá ser menor, fácil de atualizar, mas pesado para desenvolver. Essa seção explica como linkar de ambas as formas *estaticamente* e *dinâmicamente*, examinar os reflexos dessa escolha em mais detalhes, e fornecer algumas "regras práticas de manuseio" para decidir que tipo de linkagem é melhor para você.

2.3.1 Agrupando Arquivos Objeto

Um agrupamento de arquivos objeto (ou biblioteca estática) é simplesmente vários arquivos objeto armazenados como se fossem um arquivo único. [7] Quando você fornece um agrupamento de arquivos objeto ao programa que faz linkagem, ele procura no agrupamento de arquivos objeto pelo arquivo tipo objeto que ele precisa, extrai o referido arquivo, e anexa-o ao seu programa quase da mesma forma que seria se você tivesse fornecido o referido arquivo objeto diretamente.

Você pode criar uma biblioteca estática usando o comando *ar*. Arquivos de biblioteca estática tradicionalmente usam a extensão *.a* em lugar da extensão *.o* usada por um arquivos objeto comuns. Aqui está como você pode combinar *test1.o* e *test2.o* em um arquivo único *libtest.a*:

[7] Um agrupamento de arquivos objeto é grosseiramente o equivalente ao arquivo .LIB do Windows.

```
% ar cr libtest.a test1.o test2.o
```

Os sinalizadores "cr" dizem ao *ar* para criar a biblioteca estática. [8] Agora você pode incluir essa biblioteca estática em seu programa usando a opção *-ltest* com o gcc ou com o g++, como descrito na Seção 1.2.2, "Linkando Arquivos Objeto" no Capítulo 1, "Iniciando."

Quando o programa de linkagem encontra uma biblioteca estática na linha de comando, ele procura na biblioteca estática por todas as definições de símbolo (funções ou variáveis) que são referenciadas a partir dos arquivos objeto que ele já tiver processado mas não ainda definido. Os arquivos objeto que definem aqueles símbolos são extraídos da biblioteca estática e incluídos no executável final. Pelo fato de o programa linkador procurar na biblioteca estática à medida que elas aparecem na linha de comando, faz sentido colocar a biblioteca estática no final da linha de comando. Por exemplo, suponhamos que test.c contenha o código na Listagem 2.8 e app.c contenha o código na Listagem 2.9.

Listagem 2.8: (test.c) Área da Biblioteca

```
1 int f ()
2 {
3   return 3;
4 }
```

Listagem 2.9: Um Programa Que Utiliza as Funções da Biblioteca Acima

```
1 extern int f ();
2
3 int main ()
4 {
5   return f ();
6 }
```

Agora suponhamos que *test.o* seja combinado com alguns outros arquivos objetos para produzir uma bilbioteca estática *libtest.a*. A seguinte linha de comando irá falhar:

```
% gcc -o app -L. -ltest app.o
app.o: In function 'main':
app.o(.text+0x4): undefined reference to 'f'
collect2: ld returned 1 exit status
```

[8] Você pode usar outros sinalizadores para remover um arquivo de uma biblioteca estática ou executar outras operações em uma bilioteca estática. Essas operações são raramente usadas mas estão documentadas na página de manual do *ar*.

A mensagem de erro indica que mesmo que *libtest.a* contenha uma definição de *f*, o programa de linkagem não a encontra. Isso ocorre pelo fato de que a *libtest.a* foi pesquisada quando em primeiro lugar e antes de *app.o*, e naquele ponto o programa de linkagem não viu nenhuma referência a *f*. Por outro lado, se usarmos a linha abaixo, nenhuma mensagem de erro é mostrada:

```
% gcc -o app app.o -L. -ltest
```

A razão é que a referência a *f* em *app.o* faz com que o programa de linkagem inclua o arquivo objeto *test.o* contido na biblioteca estática *libtest.a*.

2.3.2 Bibliotecas Compartilhadas

Uma biblioteca compartilhada (também conhecida como um objeto compartilhado, ou como uma biblioteca linkada dinamicamente) é similar a uma biblioteca estática no sentido de que uma biblioteca dinâmica é um agrupamento de arquivos objeto. Todavia, existem muitas diferenças importantes.A diferença mais fundamental é que quando uma biblioteca compartilhada for linkada em um programa, o executável final não conterá o código que está presente na biblioteca compartilhada. Ao invés disso, o executável meramente contém uma referência à biblioteca compartilhada. Se muitos programas no sistema forem linkados usando a mesma biblioteca compartilhada, eles irão todos referencia a referida biblioteca compartilhada, mas nenhum deles irá conter algum código da biblioteca. Dessa forma, a biblioteca é "compartilhada" por todos os programas que foram linkados fazendo referência a ela. Uma segunda diferença é que uma biblioteca compartilhada não é meramente uma coleção de arquivos objeto, entre os quais objetos o programa de linkagem escolhe aquele que é necessário para satisfazer referêcias não definidas no código principal do programa que está sendo linkado. Ao invés disso, os arquivos objetos que compões a biblioteca compartilhada estão combinados dentro de um único arquivo objeto de forma que um programa que tiver sido linkado referenciando uma biblioteca compartilhada sempre inclua todo o código presente na biblioteca, em lugar de apenas aquelas porções que forem necessárias. Para criar uma bibioteca compartilhada, você deve compilar os objetos que irão compor a biblioteca usando a opção *-fPIC* no compilador, da seguinte forma:

```
% gcc -c -fPIC test1.c
```

A opção *-fPIC*[9] diz ao compilador que você estará usando *test1.o* como parte de um objeto compartilhado.

[9] *Position-Independent Code.*

Código Independente da Posição - (PIC)
PIC habilita o suporte a código independente da posição. As funções em uma biblioteca compartilhada podem ser chamadas em diferentes endereços em diferentes programas, de forma que o código no objeto compartilhado não fica dependente do endereço (ou posição) a partir do qual é chamado. Essa consideração não tem impacto sobre você, como programador, exceto que você deve lembrar-se de usar o sinalizador *-fPIC* quando estiver compilando algum código que irá ser usado em uma biblioteca compartilhada.

Então você combina os arquivos objetos dentro de uma biblioteca compartilhada, como segue:

```
% gcc -shared -fPIC -o libtest.so test1.o test2.o
```

A opção *-shared* diz ao programa de linkagem produzir uma biblioteca compartilhada em lugar de um arquivo executável comum. As bibliotecas compartilhadas usam a extensão .so, que é usada para objeto compartilhado. Da mesma forma que nas bibliotecas estáticas, o nome sempre começa com *lib* para indicar que o arquivo é uma biblioteca.

A linkagem fazendo referência a uma biblioteca compartilhada é da mesma forma que a linkagem referenciando uma biblioteca estática. Por exemplo, a linha abaixo irá fazer a linkagem referenciando *libtest.so* se *libtest.so* estiver no diretório atual, ou em um dos diretórios de busca de bibliotecas padronizados do sistema:

```
% gcc -o app app.o -L. -ltest
```

Suponhamos agora que ambas as biblioteca *libtest.a* e *libtest.so* estejam disponíveis. Então o programa de linkagem deve uma das bibliotecas e não outras. O programa de linkagem busca cada diretório (primeiramente aqueles especificados com a opção -L, e então aqueles nos diretórios pardronizados de bibliotecas do sistema). Quando o programa de linkagem encontra um diretório que contenha qualquer uma ou *libtest.a* ou *libtest.so*, o programa de linkagem para a busca nos diretórios. Se somente uma das duas variantes estiver presente no diretório, o programa de linkagem escolhe aquela variante que foi encontrada em primeiro lugar. De outra forma, o programa de linkagem escolhe a versão compartilhada, a menos que você explicitamente instrua ao programa de linkagem para proceder de outra forma. Você pode usar a opção *-static* para exigir bibliotecas estáticas. Por exemplo, a linha de comando adiante irá usar a biblioteca estática *libtest.a*, mesmo se a biblioteca compartilhada *libtest.so* estiver também presente:

```
% gcc -static -o app app.o -L. -ltest
```

O comando *ldd* mostra as bibliotecas compartilhadas que são referenciadas dentro de um executável. Essas bibliotecas precisam estar disponíveis quando o executável for chamado. Note que o comando *ldd* irá listar uma biblioteca adicional chamada *ld-linux.so*, que é uma parte do mecanismo de linkagem dinâmica do GNU/Linux.

Usando a Variável de Ambiente LD_LIBRARY_PATH Quando você fizer a linkagem de um programa referenciando uma biblioteca compartilhada, o programa de linkagem não coloca o caminho completo da localização da biblioteca compartilhada no executável resultante. Ao invés disso, o programa de linkagem coloca apenas o nome da biblioteca compartilhada. Quando o programa for executado, o sistema busca pela biblioteca compartilhada e a torna disponível para ser usada pelo programa que precisa dela. O sistema busca somente no /lib e no /usr/lib por padrão. Se uma biblioteca compartilhada que for referenciada por seu programa executável estiver instalada fora daqueles diretórios, essa biblioteca compartilhada não irá ser encontrada, e o sistema irá se recusar a executar o programa.

Uma solução para esse problema é usar a opção *-Wl,-rpath* ao usar o programa de linkagem. Suponhamos que você use o seguinte:

```
% gcc -o app app.o -L. -ltest -Wl,-rpath,/usr/local/lib
```

Então, quando o programa *app* estiver executando, o sistema irá buscar em */usr/local/lib* por qualquer biblioteca compartilhada requerida.

Outra solução para esse problema é ajustar a variável de ambiente LD_LIBRARY_PATH na hora da execução do programa de linkagem. Da mesma forma que a variável de ambiente PATH, LD_LIBRARY_PATH é uma lista de diretórios separados por ponto e vírgula. Por exemplo, se LD_LIBRARY_PATH for "*/usr/local/lib:/opt/lib*", então */usr/local/lib* e */opt/lib* serão buscados antes dos diretórios padrão */lib* e */usr/lib*. Você deve também notar que se você tiver LD_LIBRARY_PATH, o programa de linkagem irá buscar os diretórios fornecidos lá adicionalmente aos diretórios fornecidos com a opção -L quando estiver construindo um executável.[10]

2.3.3 Bibliotecas Padronizadas

Mesmo se você não especificar qualquer bibliotecas durante a fase de linkagem, o seu programa certamente usa uma biblioteca compartilhada. Isso

[10] Você pode ver uma referência a LD_RUN_PATH em alguma documentação na *Internet*. Não acredite no que você lê; essa variável atualmente não faz nada em GNU/Linux.

acontece pelo fato de GCC automaticamente fazer a linkagem usando a biblioteca C padrão, a *libc*, mesmo sem você pedir. As funções matemáticas da biblioteca C GNU padrão não estão incluídas na *libc*; ao invés disso, as funções matemáticas constituem uma biblioteca separada, a *libm*, a qual você precisa especificar explicitamente. Por exemplo, para compilar e fazer a linkagem do programa compute.c que utiliza funções trigonométricas tais como *sin* e *cos*, você deve chamar o seguinte código:

```
% gcc -o compute compute.c -lm
```

Se escrever um programa em C++ e fizer a linkagem dele usando os comandos *c++* ou *g++*, você irá também usar a biblioteca padrão GNU C++, *libstdc++*, automaticamente.

2.3.4 Dependência de uma Biblioteca

Uma biblioteca irá muitas vezes depender de outra biblioteca . Por exemplo, muitos sistemas GNU/Linux incluem a *libtiff*, uma biblioteca que contém funções para leitura e escrita de arquivos de imagem no formato TIFF. Essa biblioteca, por sua vez, utiliza as bibliotecas *libjpeg* (rotinas de imagens no formato JPEG) e *libz* (rotinas de compressão). A Listagem 2.10 mostra um pequeno programa que usa a biblioteca *libtiff* para abrir um arquivo de imagem no formato TIFF.

Listagem 2.10: (tifftest.c) Usando a *libtiff*

```
1 #include <stdio.h>
2 #include <tiffio.h>
3
4 int main (int argc, char** argv)
5 {
6   TIFF* tiff;
7   tiff = TIFFOpen (argv[1], "r");
8   TIFFClose (tiff);
9   return 0;
10 }
```

Grave esse arquivo fonte como tifftest.c. Para compilar esse programa e fazer a linkagem referenciando a *libtiff*, especifique a opção *-ltiff* na sua linha de linkagem:

```
% gcc -o tifftest tifftest.c -ltiff
```

Por padrão, o comando acima irá selecionar a biblioteca compartilhada pela versão da *libtiff*, encontrada em */usr/lib/libtiff.so*. Pelo fato de *libtiff* utilizar *libjpeg* e *libz*, uma versão de biblioteca compartilhada dessas duas é também puxada (uma biblioteca compartilhada pode também apontar para outra biblioteca compartilhada da qual depende). Para verificar isso, use o comando *ldd*:

```
% ldd tifftest
linux-gate.so.1 =>  (0xffffe000)
        /lib/libsafe.so.2 (0xb7f58000)
libtiff.so.3 => /usr/lib/libtiff.so.3 (0xb7ee6000)
        libc.so.6 => /lib/libc.so.6 (0xb7d9a000)
libdl.so.2 => /lib/libdl.so.2 (0xb7d96000)
        libjpeg.so.62 => /usr/lib/libjpeg.so.62 (0xb7d76000)
libz.so.1 => /usr/lib/libz.so.1 (0xb7d62000)
        libm.so.6 => /lib/libm.so.6 (0xb7d3c000)
/lib/ld-linux.so.2 (0xb7f5f000)
```

Bibliotecas estáticas, por outro lado, não podem apontar para outras biblioteca. Se você decidir fazer a linkagem com a versão estática da *libtiff* especificando a opção *-static* na sua linha de comando, você irá encontrar símbolos não resolvidos:

```
% gcc -static -o tifftest tifftest.c -ltiff
/usr/lib/.../libtiff.a(tif_aux.o): In function ‘TIFFVGetFieldDefaulted’:
(.text+0x621): undefined reference to ‘pow’
/usr/lib/.../libtiff.a(tif_jpeg.o): In function ‘TIFFjpeg_data_src’:
(.text+0x189): undefined reference to ‘jpeg_resync_to_restart’
/usr/lib/.../libtiff.a(tif_jpeg.o): In function ‘TIFFjpeg_destroy’:
...
```

Para fazer a linkagem desse programa estaticamente, você deve especificar as outras duas bibliotecas explicitamente:

```
% gcc -static -o tifftest tifftest.c -ltiff -ljpeg -lz
```

Ocasionalmente, duas bibliotecas irão ser mutuamente dependentes. Em outras palavras, a primeira biblioteca estática irá referenciar símbolos na segunda biblioteca estática, e vice versa. Essa situação geralmente é proveniente de um planejamento falho, mas aparece ocasionalmente. Nesses casos, você pode repetir uma biblioteca multiplas vezes na linha de comando. O programa de linkagem irá refazer a procura na biblioteca cada vez que isso ocorrer. Por exemplo, a linha adiante irá fazer com que *libqqcoisa.a* seja procurada multiplas vezes:

```
% gcc -o app app.o -lqqcoisa -loutracoisa -lqqcoisa
```

De forma que, mesmo se *libqqcoisa.a* referencie símbolos em *liboutracoisa.a*, e vice versa, o programa irá ser linkado com sucesso.

2.3.5 Prós e Contras

Agora que você sabe tudo sobre bibliotecas estáticas e bibliotecas compartilhadas, você esté provavelmente se perguntando qual usar. Existe umas poucas consideraçoes maiores para ter em mente.

Uma grande vantagem de uma biblioteca compartilhada é que essa biblioteca compartilhada economiza espaço no sistema onde o programa estiver instalado. Se você estiver instalando 10 programas, e eles todos fazem uso da mesma biblioteca compartilhada, então você libera uma grande quantidade de espaço usando uma biblioteca compartilhada. Se você tiver usado biblioteca estática em substituição à compatilhada, a biblioteca está incluída em todos os 10 programas repetidamente. Então, usando bibliotecas compartilhadas libera espaço em disco. As bibliotecas compartilhadas também reduzem tempos cópia e libera recursos de conecção se seu programa está sendo copiado a partir da *web*. Uma vantagem relacionada às bibliotecas compartilhadas é que o usuários podem escolher entre atualizar as bibliotecas com ou sem atualizar todos os programas que dependem delas. Por exemplo, suponha que você produza uma biblioteca compartilhada que gerencia conecções HTTP. Muitos programas podem depender dessa biblioteca. Se você encontrar um erro nessa biblioteca, você pode atualizar a biblioteca. instantaneamente, todos os programas que dependerem da biblioteca irão ser corrigidos; você não terá que refazer a linkagem de todos os programas que seria o caminho adotado caso se estivesse usando a linkagem estática. As vantagem acima fariam você pensar em usar sempre a biblioteca compartilhada. Todavia, razões substanciais existem para o uso da biblioteca estática em lugar da compartilhada. O fato que uma atualização com o uso de uma biblioteca compartilhada afeta todos os programas que dependem dela pode ser uma desvantagem. Por exemplo, se você estiver desenvolvendo um programa de alta disponibilidade, você pode preferir fazer a linkagem referenciando uma biblioteca estática de forma que uma atualização de bibliotecas compartilhadas no sistema não afete seu programa. (De outra forma, usuários podem atualizar a biblioteca compartilhada, afetando seu programa que foi compilado referenciando bibliotecas compartilhadas e causarem uma parada no programa, e então chamar sua linha de suporte ao usuário, censurando você!) Se você está indo pelo caminho de não instalar suas biblioteca no */lib* ou no */usr/lib*, você deve definitivamente pensar duas vezes sobre usar uma biblioteca compartilhada. (Você não espera instalar suas bibliotecas naqueles diretórios se você não esperar que usuários que irão instalar seu software possuam privilégio de administrador.) Particularmente, a opção/artifício de compilação *-Wl,-rpath* não irá servir de nada se você não sabe onde as bibliotecas estão indo parar. E pedindo a seus usuários para ajustar a variável

de ambiente LD_LIBRARY_PATH significa uma tarefa extra para eles. Pelo fato de cada usuário ter de fazer isso individualmente, isso é uma substancial e adicional carga de responsabilidade. Você irá ter que pesar essas vantagens e desvantagens para cada programa que você vier a distribuir.

2.3.6 Carregamento e Descarregamento Dinâmico

Algumas vezes você pode desejar carregar algum código em tempo de execução sem explicitamente fazer a linkagem daquele código. Por exemplo, considere uma aplicação que suporta módulos do tipo "plug-in", tal como um navegador *Internet* . O navegador permite a desenvolvedores externos ao projeto criar acessórios para fornecer ao navegador funcionalidades adicionais. Os desenvolvedores externos criam bibliotecas compartilhadas e as colocam em uma localização conhecida pelo navegador. O navegador então automaticamente carrega o código nessas bibliotecas. Essa funcionalidade está disponível em ambiente GNU/Linux através do uso da função *dlopen*. Você já pode ter aberto uma biblioteca compartilhada chamada *libtest.so* chamando a função *dlopen* da forma abaixo:

```
dlopen ("libtest.so", RTLD_LAZY)
```

(O segundo parâmetro é um sinalizador que indica como associar símbolos na biblioteca compartilhada. Você pode consultar as páginas de manual instaladas no seu sistema sobre *dlopen* se você desejar mais informação, mas RTLD_LAZY é comumente a opção que você deseja.) Para usar funções de carregamento dinâmico, inclua o arquivo de cabeçalho <dlfcn.h> e faça a linkagem com a opção *-ldl* para selecionar a biblioteca *libdl*.

O valor de retorno dessa função é um *void ** que é usado como um controlador para a biblioteca compartilhada. Você pode passar esse valor para a função *dlsym* para obter o endereço de uma função que tiver sido chamada com a biblioteca compartilhada. Por exemplo, se *libtest.so* define uma função chamada *minha_funcao*, você pode ter chamado a *minha_funcao* como segue:

```
void* controlador = dlopen ("libtest.so", RTLD_LAZY);
void (*test)() = dlsym (controlador, "minha_funcao");
(*test)();
dlclose (controlador);
```

A função *dlsym* pode também ser usada para obter um apontador para uma variável estática na biblioteca compartilhada.

Ambas as funções *dlopen* e *dlsym* retornam *NULL* se não obtiverem sucesso. no evento descrito acima, você pode chamar a função *dlerror* (sem

parâmetros) para obter uma mensagem de erro em formato legível aos humanos descrevendo o problema.

A função *dlclose* descarrega a biblioteca compartilhada. Tecnicamente, a função *dlopen* carrega a biblioteca somente se a referida biblioteca já não tiver sido chamada anteriormente. Se a biblioteca já tiver sido chamada, *dlopen* simplesmente incrementa o contador de referência da biblioteca. Similarmente, a função *dlclose* decrementa o contador de referência e então descarrega a biblioteca somente se o contador de referência tiver alcançado o valor zero.

Se você está escrevendo um código em sua biblioteca compartilhada em C++, você irá provavelmente desejar declarar aquelas funções e variáveis que você planeja acessar a partir de algum lugar com o especificador de linkagem *extern "C"*. Por exemplos, se a função C++ *minha_funcao* estiver em uma biblioteca compartilhada e você desejar acessar essa função com a função *dlsym*, você deve declarar a *minha_funcao* como segue:

```
extern "C" void minha_funcao ();
```

Isso evita que o compilador C++ desfigure o nome da função, pelo fato de o compilador C++ poder mudar o nome da função de minha_função para um diferente, um nome mais engraçado ao olhar que expresse informações extras sobre a função. Um compilador C não irá desfigurar nomes; os nomes irão ser usados qualquer que seja o nome que você forneça para sua função ou variável.

Capítulo 3

Processos

UMA INSTÂNCIA EXECUTANDO UM PROGRAMA CHAMA-SE UM PROCESSO. Se você tem duas janelas de terminal exibindo informações em sua tela, então você está provavelmente executando o mesmo programa de terminal duas vezes – você tem dois processos de terminal. Cada janela de terminal está provavelmente executando um *shell*; cada *shell* sendo executado é um outro processo. Quando você chama um comando em um *shell*, o programa correspondente é executado em um novo processo; o processo de *shell* continua quando o processo do comando chamado se completar.

Programadores avançados muitas vezes utilizam muitos processos em cooperação em uma única aplicação para habilitar a capacidade da aplicação de executar mais de uma coisa ao mesmo tempo, para incrementar robustez da aplicação, e para fazer uso dos programas já existentes.

A maioria das funções de controle de processos descritas nesse capítulo são similares a aquelas em outros sistemas UNIX. A maioria é declarada no arquivo de cabeçalho <unistd.h>; verifique a página de manual de cada função para ter certeza.

3.1 Visualizando Processos

Sempre que você senta em seu computador para usá-lo, exitem processos em atividade. Todos os programas sendo executados usam um ou mais processos. Vamos iniciar dando uma olhada nos processos já existentes em seu computador.

3.1.1 Identificadores de Processos

Cada processo em um sistema GNU/Linux é identificado por seu único número de identificação, algumas vezes referenciado como *pid*. Identificadores de Processos são números inteiros de 16-bit que são atribuidos sequêncialmente pelo kernel GNU/Linux a cada vez que um novo processo é criado.

Todo processo tem um processo pai (exceto o processo *init*, descrito na Seção 3.3.4, "Processos do Tipo Zumbi"). Dessa forma, você pode pensar de processos em um sistema GNU/Linux como organizados em uma árvore, com o processo *init* sendo a raíz principal que originou toda a árvore. A identificação do processo pai, ou *ppid*, é simplesmente o número de identificação do processo pai. Quando fizermos referência ao número de identificação de um processo em um programa em C ou em C++, sempre usa-se a definição de tipo *pid_t*, que é feita em <sys/types.h>. Um programa pode obter o número de identificação do processo que o está executando com a chamada de sistema *getpid()*, e o programa também pode obter o número de identificação de processo do processo que o originou com a chamada de sistema *getppid()*. Por exemplo, o programa na Listagem 3.1 mostra o o número de identificação do processo que o está executando e o número de identificação do processo que o originou.

Listagem 3.1: (print-pid.c) Mostrando o ID do Processo

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main ()
5 {
6   printf ("O id do processo e %d\n", (int) getpid ());
7   printf ("O id do processo pai e %d\n", (int) getppid ());
8   return 0;
9 }
```

Observe que se você chamar esse programa muitas vezes, um ID diferente de processo será reportado a cada vez que você chamar o programa pelo fato de cada chamada estar em um novo processo. Todavia, se você chamar o programa várias vezes a partir da mesma janela de *shell*, o número de identificação do processo que o originou (isto é, a número de identificação do processo do *shell*) é o mesmo.

3.1.2 Visualizando os Processos Ativos

O comando *ps* mostra os processos que estiverem sendo executados sobre seu sistema. A versão GNU/Linux do *ps* tem muitas opções pelo fato de tentar ser compatível com as versões do *ps* de muitas outras variantes UNIXs. Essas

opções controlam quais processos são listados e qual informação sobre cada processo deverá ser mostrada.

Por padrão, chamando *ps* mostra os processos controlados pelo terminal ou janela de terminal na qual o comando *ps* for chamado. Por exemplo:

```
% ps
 PID TTY            TIME CMD
21693 pts/8      00:00:00 bash
21694 pts/8      00:00:00 ps
```

Essa chamada de *ps* mostra dois processos. O primeiro, o bash, é um *shell* executando sobre o referido terminal. O segundo é a instância de execução do programa *ps* propriamente dito. A primeira coluna, rotulada *PID*, mostra o número de identificação de cada processo listado na saída do comando.

Para uma olhada mais detalhada no que está sendo executado no seu sistema GNU/Linux, use o seguinte:

```
% ps -e -o pid,ppid,command
```

A opção *-e* instrui o *ps* a mostrar todos os processos sendo executados no sistema. A opção *-o pid,ppid,command* diz ao *ps* qual informação mostrar sobre cada processo – no caso acima, o ID do processo, o ID do processo pai, e o comando sendo executado no referido processo.

Formatos de Saída do *ps*
Com a opção *-o* fornecida ao comando *ps*, você especifica a informação sobre o processo que você deseja na saída no formato de uma lista separada por vírgulas. por exemplo, *ps -o pid,user, start_time,command* mostra o ID do processo, o nome do usuário dono do processo, o tempo decorrido desde quando o processo começou, e o comando que está executando o processo. Veja a página de manual do comando *ps* para a lista completa dos códigos de campo. Você pode usar as opções -f (lista completa), -l (lista longa), ou -j (lista de tarefas) ao invés da opção -o acima e usar esses três diferentes formatos predefinidos de listagem (completa, longa ou de tarefas).

Aqui está algumas linhas iniciais e finais de saída do comando *ps* em meu sistema. Você pode ver diferentes saídas, dependendo do que estiver sendo executado em seu sistema.

```
% ps -e -o pid,ppid,command
  PID  PPID COMMAND
    1     0 init [5]
    2     1 [kflushd]
```

```
    3     1 [kupdate]
...
21725 21693 xterm
21727 21725 bash
21728 21727 ps -e -o pid,ppid,command
```

Note que o ID do processo pai do comando *ps*, 21727, é o ID do *bash*, o *shell* a partir do qual chamou-se o *ps*. O processo pai do *bash* é por sua vez o de número 21725, o ID do processo do programa *xterm* no qual o *shell* está sendo executado.

3.1.3 Encerrando um Processo

Você pode encerrar um processo que está sendo executado com o comando *kill*. Simplesmente especificando na linha de comando o ID do processo a ser encerrado.

O comando *kill* trabalha enviando ao processo um SIGTERM, ou sinal de encerramento.[1] Isso faz com que o processo encerre, a menos que o programa em execução explicitamente controle ou mascare o sinal SIGTERM. Sinais são descritos na Seção 3.3, "Sinais."

3.2 Criando Processos

Duas técnicas são usadas para criar um novo processo. A primeira é relativamente simples mas deve ser usada de forma bem comedida e econômica pelo fato de ser ineficiente e de ter consideráveis riscos de segurança. A segunda técnica é mais complexa mas fornece grande flexibilidade, rapidez, e segurança.

3.2.1 Usando *system*

A função *system* na biblioteca C GNU padrão fornece um caminho fácil para executar um comando dentro de um programa, principalmente se o comando tiver sido digitado dentro de um *shell*. De fato, a função *system* cria um sub-processo que executa o *shell Bourne* padrão (/bin/sh) e repassa o comando àquele *shell* para execução. Por exemplo, o programa na Listagem 3.2 chama o comando *ls* para mostrar o conteúdo do diretório raíz, como se você digitasse "*ls -l /*" dentro de um *shell*.

[1] Você pode também usar o comando *kill* para enviar outros sinais a um processo. Isso é descrito na Seção 3.3.1, "Encerramento de Processos".

Listagem 3.2: (system.c) Usando uma chamada à função *system*

```
1 #include <stdlib.h>
2
3 int main ()
4 {
5   int return_value;
6   return_value = system ("ls -l /");
7   return return_value;
8 }
```

A função *system* retorna a condição de saída do comando do *shell*. Se o *shell* propriamente dito não puder ser executado, a função *system* retorna o código 127; se outro erro ocorrer, a função *system* retorna -1.

Pelo fato de a função *system* usar um *shell* para chamar seu comando, ela dá margens a recursos, limitações, e a falhas de segurança do *shell* de seu sistema. Você não pode saber seguramente sobre a disponibilidade de qualquer versão em particular do *shell Bourne*. Em muitos sistemas UNIX, /bin/sh é uma ligação simbólica para outro *shell*. Por exemplo, na maioria dos sistemas GNU/Linux, o /bin/sh aponta para o *bash* (o *Bourne-Again SHell*), e diferentes distribuições GNU/Linux utilizam diferentes versões do *bash*. Chamando um programa com privilégios de administrador com a função *system*, pode exemplo, pode ter diferentes resultados sob diferentes sistemas GNU/-Linux. Devido ao que foi aqui exposto, é preferível usar o método *fork and exec* (bifurcar e executar) para criar processos.

3.2.2 Usando bifurcar e executar

A API [2] do DOS e do Windows possuem a família *spawn* de funções. Essas funções recebem como argumento o nome de um programa para executar e criam uma nova intância de processo daquele programa. O GNU/Linux não contém uma função que faz tudo isso de uma vez só. Ao invés disso, fornece uma função, a função *fork*, que cria um processo filho que é uma cópia exata de seu processo pai. GNU/Linux fornece outro conjunto de funções, a família das funções *exec*, que faz com que um processo em particular não mais seja uma instância de um programa e ao invés disso torne-se uma instância de outro programa. Para criar um novo processo, você primeiramente deve usar a função *fork* para fazer uma cópia do processo atual que está executando seu programa. A seguir você usa a função *exec* para transformar um desses dois processos iguais em uma instância do programa que você deseja criar.

Chamando a função *fork* Quando um programa chama a função *fork*, um processo clone do processo que fez a chamada, chamado processo filho, é criado. O processo pai continua executando o programa na instrução

[2]Nota do tradutor: *Application Programming Interface*.

imediatamente após a instrução que chamou a função *fork*. O processo filho, também, executa o mesmo programa a partir da mesma posição de instrução.

Como fazer para os dois processos diferirem? Primeiramente, o processo filho é um novo processo e portanto tem um novo ID de processo, distinto do ID de seu processo pai. Um caminho para um programa distinguir se ele mesmo está em um processo pai ou em um processo filho é chamar a função *getpid* da biblioteca C GNU padrão. Todavia, a função *fork* fornece diferentes valores de retorno quando chamada a partir de um processo pai ou a partir de um processo filho – um processo "entra" na chamada a *fork*, dois processos "saem" com diferentes valores de retorno. O valor de retorno no processo pai é o ID de processo do processo filho. O valor de retorno no processo filho é zero. Pelo fato de nenhum processo mesmo ter um ID de processo com o valor zero, isso torna fácil para o programa distinguir se está sendo executado como o processo pai ou processo filho.

A Listagem 3.3 é um exemplo de utilização da função *fork* para duplicar o processo de um programa. Note que o primeiro bloco da declaração *if* é executado somente no processo pai, enquando a cláusula *else* é executada no processo filho.

Listagem 3.3: (fork.c) Usando *fork* para Duplicar o Processo de um Programa

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <unistd.h>
4
5 int main ()
6 {
7   pid_t child_pid;
8
9   printf ("o id do processo do programa principal e %d\n", (int) getpid ());
10
11   child_pid = fork ();
12   if (child_pid != 0) {
13     printf ("esse e o processo pai, com id %d\n", (int) getpid ());
14     printf ("o id do processo filho e %d\n", (int) child_pid);
15   }
16   else
17     printf ("esse e o processo filho, com id %d\n", (int) getpid ());
18
19   return 0;
20 }
```

Usando a Família *exec* As funções *exec* substituem o programa que está sendo executado em um processo por outro programa. Quando um programa chama uma função *exec*, o processo que abriga a chamada feita à função *exec* imediatamente cessa de executar o programa atual e inicia a execução de um novo programa a partir do início desse mesmo novo programa, assumindo que a chamada à função *exec* tenha sido executada com sucesso.

Dentro da família de funções *exec*, existem funções que variam de forma muito pequena na parte que se refere a compatibilidade e no que se refere à

maneira de serem chamadas.

- Funções que possuem a letra "p" em seus nomes (*execvp* e *execlp*) aceitam um nome de programa e procuram por um programa que tenha o nome recebido no atual caminho de execução; funções que não contiverem o "p" no nome devem receber o caminho completo de localização do programa a ser executado.

- Funções que possuem a letra "v" em seus nome (*execv*, *execvp*, e *execve*) aceitam a lista de argumentos para o novo programa como um vetor terminado pelo caractere *NULL* de apontadores para sequências de caractere. Funções que contiverem a letra "l"(*execl*, *execlp*, e *execle*) aceitam a lista de argumentos usando o mecanismo varargs da linguagem C. [a]

- As funções que possuem a letra "e" em seus nomes (*execve* e *execle*) aceitam um argumento adicional, um vetor de variáveis de ambiente. O argumento deve ser um vetor de apontadores para sequência de caracteres terminado pelo caractere *NULL*. Cada sequências de caractere deve ser da forma "VARIAVEL=valor".

[a] Nota do tradutor: Veja http://www.cs.utah.edu/dept/old/texinfo/glibc-manual-0.02/library_toc.html
#SEC472 e também http://gcc.gnu.org/onlinedocs/gccint/Varargs.html.

Pelo fato de a função *exec* substituir o programa chamado por outro, ela nunca retorna a menos que um erro ocorra.

A lista de argumentos passada ao programa é análoga aos argumentos de linha comando que você especifica a um programa quando você o executa a partir de um *shell*. Eles estão disponiveis através dos parâmetros *argc* e de *argv* passados à função *main*. Lembre-se, quando um programa for chamado a partir de um *shell*, o *shell* ajusta o primeiro elemento da lista de argumentos (*argv[0]*) para o nome do programa, o segundo elemento da lista de argumentos (*argv[1]*) para o primeiro argumento da linha de comando, e assim por diante. Quando você usar uma função *exec* em seu programa, você, também, deve passar o nome da função como o primeiro elemento da lista de argumentos.

Usando *fork* e *exec* Juntas Um modelo comum para executar um sub-programa dentro de um programa é primeiramente bifurcar o processo e então executar o sub-programa. Isso permite que o programa que fez a chamada continue a execução no processo pai enquanto o mesmo programa que fez a chamada é substituído pelo subprograma no processo filho.

O programa na Listagem 3.4, da mesma forma que a Listagem 3.2, mostra o conteúdo do diretório raíz usando o comando *ls*. Diferindo do exemplo anterior, de outra forma, a Listagem 3.4 chama o comando *ls* diretamente, passando ao *ls* os argumentos de linha de comando "-l" e "/" ao invés de chamar o *ls* a partir de um *shell*.

Listagem 3.4: (fork-exec.c) Usando *fork* e *exec* Juntas

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/types.h>
4 #include <unistd.h>
5
6 /* Gera um processo filho executando um programa novo.  PROGRAM e o nome
7    do programa a ser executado; o caminho ira ser procurando por esse programa.
8    ARG_LIST e um NULL-terminada lista de strings caractere a serem
9    informada como a lista de argumentos do programa.  Retorna o id de processo do
10   processo gerado.  */
11
12 int spawn (char* program, char** arg_list)
13 {
14   pid_t child_pid;
15
16   /* Duplica o processo atual.  */
17   child_pid = fork ();
18   if (child_pid != 0)
19     /* Esse e o processo pai.  */
20     return child_pid;
21   else {
22     /* Agora execute PROGRAM, buscando por ele no caminho.  */
23     execvp (program, arg_list);
24     /* A funcao execvp retorna somente se um erro ocorrer.  */
25     fprintf (stderr, "um erro ocorreu em execvp\n");
26     abort ();
27   }
28 }
29
30 int main ()
31 {
32   /* A lista de argumentos informada ao comando "ls".  */
33   char* arg_list[] = {
34     "ls",      /* argv[0], o nome do programa.  */
35     "-l",
36     "/",
37     NULL       /* A lista de argumentos deve terminar com um NULL.  */
38   };
39
40   /* Gera um processo filho rodando o comando "ls".  Ignora o
41      id de processo filho retornado.  */
42   spawn ("ls", arg_list);
43
44   printf ("terminei com o programa principal\n");
45
46   return 0;
47 }
```

3.2.3 Agendamento de Processo

GNU/Linux faz o agendamento dos processos pai e processos filho independentemente; não existe garantias de qual dos dois irá ser executado em primeiro lugar, ou quanto tempo de execução previamente irá decorrer antes de GNU/Linux interrompê-lo e liberar o ciclo de processamento para o outro processo (ou para algum outro processo do sistema que não os processos pai e filho aqui citados) ser executado. Em particular, nenhuma parte, alguma parte, ou todo o processo do comando *ls* pode executar em um processo filho

antes de o processo pai que o criou ser encerrado.[3] GNU/Linux promete que cada processo irá ser executado em algum momento – nenhum processo irá ser totalmente discriminado na distribuição dos recursos de execução.[4]

Você pode especificar que um processo é menos importante – e deve receber uma prioridades mais baixa – atribuindo a esse processo um valor alto de gentileza. Por padrão, todo processo recebe um valor de gentileza zero. Um valor de gentileza mais alto significa que o processo recebe uma menor prioridade de execução; de modo contrário, um processo com um baixo (isto é, negativo) valor de gentileza recebe mais tempo de execução.

Para executar um programa com um valor de gentileza não nulo, use o comando *nice*, especificando o valor de gentileza com a opção -n. Por exemplo, adiante mostra-se como você pode chamar o comando "sort entrada.txt > saida.txt", que corresponde a uma longa operação de ordenação, como reduzida prioridade de forma que essa operação de ordenação não torne o sistema muito lento:

```
% nice -n 10 sort input.txt > output.txt
```

Você pode usar o comando *renice* para modificar o nível de gentileza de um processo sendo executado a partir da linha de comando.

Para modificar o nível de gentileza de um processo que está em execução a partir de outro programa, use a função *nice*. O argumento dessa função é um valor de incremento, que é adicionado ao nível de gentileza do processo está executando o programa cujo nível de gentileza se deseja mudar. Lembre-se que um valor positivo aumenta o valor de gentileza e dessa forma reduz a prioridade de execução de um processo.

Note que somente um processo com privilégios de usuário *root* pode executar um ou outro processo com um valor de gentileza negativo ou reduzir o valor de gentileza de um processo que está sendo executado. Isso significa que você pode especificar valores negativos para os comando *nice* e *renice* somente quando está acessando o computador como superusuário, e somente um processo executando com privilégios de superusuário pode enviar um valor negativo para a função *nice* da glibc. Esse comportamento previne que usuários comuns consigam prioriade de execução em nome de outros usuários que não o seu próprio usando o sistema.

[3]Um método para definir a ordem de execução de dois processos é apresentado na seção 3.3.2, "Esperando pelo Encerramento de um Processo".

[4]Nota do tradutor: o autor refere-se aos algorítmos de escalonamento. Veja também `http://www.kernel.org/doc/#5.1`.

3.3 Sinais

Sinais são mecanismos usados como forma de comunicação e controle de processos em GNU/Linux. O tópico que fala de sinais é muito extenso; aqui falaremos sobre alguns sinais mais importantes e técnicas que são usadas para controlar processos.

Um sinal é uma mensagem especial enviada a um processo. Sinais são assíncronos; quando um processo recebe um sinal, o referido processo manipula o sinal imediatamente, sem encerrar a função que está processando no momento ou mesmo sem encerrar a linha de código que ele está executando no momento. Existem muitas dúzias de diferentes sinais, cada um com um significado diferente. Cada tipo de sinal é especificado através de seu número de sinal, mas em programas, você comumente se refere a um sinal através de seu nome. Em GNU/Linux, os sinais são definidos em */usr/include/bits/-signum.h*. (Você não deve incluir esse arquivo de cabeçalho diretamente em seu programa; ao invés disso, use <signal.h>.)

Quando um processo recebe um sinal, esse mesmo processo pode ter uma entre muitas respostas/comportamentos, dependendo do comportamento do sinal recebido. Para cada sinal, existe um comportamento padrão, que determina o que acontece ao processo se o programa executado no processo não especifica algum outro comportamento. Para a maioria dos tipos de sinal, um programa especifica algum comportamento – ou ignora o sinal ou chama uma função especial controladora de sinal para responder ao sinal. Se uma função controladora de sinal for usada, o programa atualmente em execução é colocado em estado de espera, a função controladora de sinal é executada, e, quando a função controladora de sinal retornar, o programa que estava sendo executado na hora da chegada do sinal é retomado pelo processo e continua do ponto onde parou.

O sistema GNU/Linux envia sinais a processos em resposta a condições específicas. Por exemplo, os sinais SIGBUS (erro de *bus*), SIGSEGV (violação de segmento de memória), e SIGFPE (exceção de ponto flutuante) podem ser enviados a um processo que tenta executar uma operação ilegal. O comportamento padrão para esses sinais é encerrar o processo e produzir um arquivo *core*.

Um processo pode também enviar um sinal a outro processo. Um uso comum desse mecanismo é encerrar outro processo enviando um sinal SIGTERM ou um sinal SIGKILL. [5]

[5] Qual a diferença? O sinal SIGTERM pergunta a um processo se ele pode terminar; o processo pode ignorar a requisição por mascaramento ou ignorar o sinal. O sinal SIGKILL sempre encerra o processo imediatamente pelo fato de o processo não poder mascarar ou ignorar o sinal SIGKILL.

Outro uso comum é enviar um comando a um programa que está sendo executado. Dois sinais "definidos pelo usuário" são reservados com esse objetivo: SIGUSR1 e SIGUSR2. O sinal SIGHUP é algumas vezes usado para esse propósito também, comumente para acordar um programa que está cochilando ou fazer com que um programa releia seus arquivos de configuração.

A função *sigaction* pode ser usada para configurar um comportamento de sinal. O primeiro parâmetro é o número do sinal. Os dois parâmetros imediatamente a seguir são apontadores para estruturas da função *sigaction*; o primeiro dos dois contém o comportamento desejado para aquele número de sinal, enquanto o segundo recebe o comportamento atualmente existente. O campo mais importante tanto na primeira como na segunda estrutura apontadas da função *sigaction* é *sa_handler*. O *sa_handler* pode receber um dos três valores abaixo:

- SIG_DFL, que especifica o comportamento padrão para o sinal.
- SIG_IGN, que especifica a possibilidade de o sinal pode ser ignorado.
- Um apontador para uma função controladora de sinal. A função deve receber um parâmetro, o número do sinal, e retornar *void* [a].

[a] Nota do tradutor:Vazio.

Pelo fato de sinais serem assíncronos, o programa principal pode estar em um estado muito frágil quando um sinal é processado e dessa forma também enquanto uma função controladora de sinal está sendo executada. Portanto, você deve evitar executar quaisquer operações de E/S ou chamar a maior parte das funções de biblioteca e de sistema a partir de controladores de sinal.

Um controlador de sinal executa o trabalho mínimo necessário para responder ao sinal, e então retornar o controle ao programa principal (ou encerrar o programa). Na maioria dos casos, a tarefa do controlador de sinal consiste simplesmente em gravar o fato de que um sinal ocorreu. O programa principal então verifica periodicamente se um sinal ocorreu e reage conforme o sinal ocorrido ou não ocorrido.

É possível que uma função controladora de sinal seja interrompida por meio da entrega de outro sinal. Embora isso seja uma ocorrência rara, se vier a ocorrer, irá ser muito difícil diagnosticar e depurar o problema. (Isso é um exemplo de uma condição de corrida, discutida no Capítulo 4, "Linhas de Execução" Seção 4.4, "Sincronização e Seções Críticas.") Portanto, você deve ser muito cuidadoso sobre o que seu programa faz em uma função controladora de sinal.

Mesmo a atribuição de um valor a uma variável global pode ser perigosa pelo fato de que a atribuição poder ser atualmente realizada em duas ou mais instruções de máquina, e um segundo sinal pode ocorrer entre essas duas instruções de máquina, abandonando a variável em um estado corrompido. Se você vier a usar uma variável global para marcar um sinal a partir de uma função controladora de sinal, essa variável deve ser do tipo especial *sig_atomic_t*. GNU/Linux garante que atribuições a variáveis desse tipo são realizadas em uma única instrução e portanto não pode ser interrompida no meio do caminho. Em GNU/Linux, *sig_atomic_t* é um *int* comum; de fato, atribuições a tipos inteiros do tamanho de *int* ou de menor tamanho, ou para apontadores, são atômicos. Se você deseja escrever um programa que seja portável para qualquer sistema UNIX padronizado, apesar do que foi aqui escrito, use o tipo *sig_atomic_t* para variáveis globais.

O esqueleto de programa na Listagem 3.5 por exemplo, utiliza uma função controladora de sinal para contar o número de vezes que o programa recebe SIGUSR1, um dos sinais reservados para uso por aplicação.

Listagem 3.5: (sigusr1.c) Usando um Controlador de Sinal

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <string.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6
7 sig_atomic_t sigusr1_count = 0;
8
9 void handler (int signal_number)
10 {
11   ++sigusr1_count;
12 }
13
14 int main ()
15 {
16   struct sigaction sa;
17   memset (&sa, 0, sizeof (sa));
18   sa.sa_handler = &handler;
19   sigaction (SIGUSR1, &sa, NULL);
20
21   /* Faz coisas demoradas e trabalhosas aqui.  */
22   /* ...  */
23
24   printf ("SIGUSR1 foi incrementada %d vezes\n", sigusr1_count);
25   return 0;
26 }
```

3.3.1 Encerramento de Processos

Normalmente, um processo encerra através de um entre dois caminhos. Ou o programa que está sendo executado chama a função *exit*, ou a fução *main* do programa retorna. Cada processo tem um código de saída: um número que o processo retorna a seu processo pai. O código de saída é o argumento passado à função *exit*, ou o valor retornado a partir da função *main*.

Um processo pode também terminar de forma abrupta, em resposta a um sinal. Por exemplo, os sinais SIGBUS, SIGSEGV, e SIGFPE mencionados

anteriormente fazem com que o processo encerre. Outros sinais são usados para encerrar um processo explicitamente. O sinal SIGINT é enviado a um processo quando o usuário tenta encerrá-lo digitando Ctrl+C em seu terminal. O sinal SIGTERM é enviado pelo comando *kill*. A disposição padrão em ambos os casos é encerrar o processo. Por meio de chamada à função *abort*, um processo envia a si mesmo o sinal SIGABRT, que encerra o processo e produz um arquivo *core*. O mais poderoso sinal para encerrar um processo é SIGKILL, que encerra um processo imediatamente e não pode ser bloqueado ou manuseado por um programa.

Qualquer desses sinais pode ser enviado usando o comando *kill* por meio da especificação de um sinalizador extra de linha de comando; por exemplo, para encerrar um processo perturbador por meio do envio de a esse processo de um SIGKILL, use o seguinte comando, onde *pid* é o número de identificação do seu processo perturbador:

```
% kill -KILL pid
```

Para enviar um sinal a partir de um programa, use a função *kill*. O primeiro parâmetro é o ID do processo alvo. O segundo parâmetro é o número do sinal; use SIGTERM para simular o comportamento padrão do comando *kill*. Por exemplo, sendo *child_pid* o ID de processo do processo filho, você pode usar a função *kill* para encerrar um processo filho a partir do processo pai por meio de um chamado à função *kill* como o seguinte:

```
kill (child_pid, SIGTERM);
```

Inclua cabeçalhos <sys/types.h> e <signal.h> caso você resolva usar a função *kill*.

Por convenção, o código de saída é usado para indicar se o programa foi executado corretamente. Um código de saída com valor zero indica execução correta, enquanto um código de saída não nulo indica que um erro ocorreu. No caso de ocorrência de erro, o valor particular retornado pode fornecer alguma indicação da natureza do erro. É uma boa idéia apegar-se a essa convenção em seus programas pelo fato de outros componentes do sistema GNU/Linux assumirem esse comportamento. Por exemplo, programas de *shells* assumem essa convenção quando você conecta multiplos programas com os operadores && (sinal lógico "e") e "||" (sinal lógico para "ou"). Portanto, você deve explicitamente retornar zero a partir de sua função *main*, a menos que um erro aconteça.

Com a maioria dos *shells*, é possível obter o código de saída da maioria dos programas para o mais recentemente programa executado usando a variável

especial $?. Segue um exemplo no qual o comando *ls* é chamado duas vezes e seu código de saída é mostrado após cada chamada. no primeiro caso, o comando *ls* executa corretamente e retorna o código de saída zero. No segundo caso, *ls* encontra um erro (pelo fato de o *nomedearquivo* especificado na linha de comando não existir) e dessa forma retorna um código de saída não nulo.

```
% ls /
bin   coda   etc    lib         misc    nfs proc       sbin   usr
boot  dev    home   lost+found  mnt     opt root       tmp    var
% echo $?
0
% ls nomedearquivo
ls: impossivel acessar nomedearquivo: Arquivo ou diretorio nao encontrado
% echo $?
1
```

Note que apesar de o tipo de dado do parâmetro da função *exit* ser *int* e a função *main* retornar um tipo de dado *int*, GNU/Linux não preserva os 32 bits completos do código de retorno. De fato, você deve usar códigos de saída somente entre zero e 127. Códigos de saída acima de 128 possuem um significado especial – quando um processo for encerrado por meio de um sinal, seus códigos de saída são 128 mais o número do sinal.

3.3.2 Esperando pelo Encerramento de um Processo

Se você tiver digitado e executado o exemplo de *fork* e *exec* na Listagem 3.4, você pode ter notado que a saída fornecida pelo programa *ls* muitas vezes aparece após o "programa principal" ter sido completado. Isso ocorre pelo fato de o processo filho, no qual *ls* estava sendo executado, é agendado independentemente do processo pai. Pelo fato de GNU/Linux ser um sistema operacional multi-tarefa, ambos os processos parecem ser executados simultâneamente, e você não pode prever se o programa *ls* irá ter uma chance de ser executado antes ou depois de o seu processo pai ser executado.

Em algumas situações, apesar disso, é desejável que o processo pai espere até que um ou mais prodessos filhos se completem. Isso pode ser realizado com a família *wait* de chamadas de sistema. Essas funções permitem a você esperar que um processo termine sua execução, e habilite o processo pai recuperar informação sobre o encerramento de seu processo filho. Existem quatro diferentes chamadas de sistema na família *wait*; você pode escolher pegar pouca ou muita informação sobre o processo encerrado, e você pode escolher se preocupar acerca de qual processo filho encerrou.

3.3.3 As Chamadas de Sistema da Família *wait*

A função mais simples da família é chamada apenas *wait*. Essa função bloqueia o processo que está fazendo a chamada até que um de seus processos

filhos encerre (ou ocorra um erro). A função *wait* retorna um código que reflete a situação atual por meio de um argumento apontador inteiro, do qual você pode extrair informação sobre como o porcesso filho terminou. Por exemplo, a macro *WEXITSTATUS* extrai o código de saída do processo filho. Você pode usar a macro *WIFEXITED* para determinar a partir da situação de saída de um processo filho se o referido processo terminou normalmente (por meio da função *exit* ou retornando a partir da função *main*) ou foi encerrado por meio de um sinal que não pode ser controlado. Nesse último caso, use a macro *WTERMSIG* para extrair a partir de sua situação de saída o número do sinal através do qual o processo em questão foi encerrado. Aqui está a função *main* de um exemplo com *fork* e com *exec* novamente. Dessa vez, o processo pai chama *wait* para esperar até que o processo filho, no qual o comando *ls* está sendo executado, termine.

```
int main ()
{
  int child_status;

    /* The argument list to pass to the "ls" command. */
    char* arg_list[] = {
        "ls",     /* argv[0], the name of the program. */
        "-l",
        "/",
        NULL      /* The argument list must end with a NULL.     */
    };

    /* Spawn a child process running the "ls" command.     Ignore the
       returned child process ID. */
    spawn ("ls", arg_list);

    /* Wait for the child process to complete. */
    wait (&child_status);
    if (WIFEXITED (child_status))
      printf ("the child process exited normally, with exit code %d\n",
WEXITSTATUS (child_status));
    else
      printf ("the child process exited abnormally\n");

    return 0;
}
```

Muitas chamadas de sistema similares estão disponíveis em GNU/Linux, que são mais flexíveis ou fornecem mais informação sobre a saída de um processo filho. A função *waitpid* pode ser usada para esperar pela saída de um processo filho específico em lugar de esperar pelo término de algum processo não específico. A função *wait3* retorna estatísticas de uso de CPU sobre o processo filho que está encerrando, e a função *wait4* permite a você especificar opções adicionais sobre quais processos aguardar.

3.3.4 Processos do Tipo Zumbi

Se um processo filho termina enquanto seu pai está chamando uma função *wait*, o processo filho desaparece e sua situação de encerramento é informada a seu processo pai por meio da chamada *wait*. Mas o que acontece quando um processo filho termina e o processo pai não está chamando a função *wait*?

O processo filho simplesmente desaparece? Não, porque a informação sobre seu encerramento - informação tal como se ele terminou normalmente ou não, e se tiver terminado normalmente, o que sua situação de saída mostra agora - pode ser perdida. Quando um processo filho termina e o processo pai não está chamando a função *wait*, ele torna-se um processo zumbi.

Um *processo zumbi* é um processo que tenha terminado mas não tenha sido limpo ainda. É da responsabilidade do processo pai limpar o sistema de sua criança zumbi. As funções *wait* fazem isso, também, de forma que não seja necessário rastrear se seu processo filho está ainda executando antes de esperar por ele. Suponhamos, por exemplo, que um programa faça um *fork* criando um processo filho, execute alguma outra computação, e então chame a função *wait*. Se o processo filho não tiver terminado nesse ponto, o processo pai irá bloquear na chamada a *wait* até que o processo filho encerre. Se o processo filho encerrar antes que o processo pai chame *wait*, o processo filho torna-se um zumbi. Quando o processo pai chama *wait*, a situação atual de encerramento do filho zumbi é extraída, o processo filho é apagado, e a chamada a *wait* retorna imediatamente.

O que acontece se o processo pai não limpa seus filhos? Eles permanecem soltos no sistemas, como processos zumbis. O programa na Listagem 3.6 cria um processo filho através de *fork*, que se encerra imediatamente e então o mesmo programa que criou o processo filho vai cochilar por um minuto, sem mesmo limpar o processo filho.

Listagem 3.6: (zombie.c) Fazendo um Processo Zumbi

```
1 #include <stdlib.h>
2 #include <sys/types.h>
3 #include <unistd.h>
4
5 int main ()
6 {
7   pid_t child_pid;
8
9   /* Cria um processo filho.  */
10  child_pid = fork ();
11  if (child_pid > 0) {
12    /* Esse e o processo pai.  Durma por um minuto.  */
13    sleep (60);
14  }
15  else {
16    /* Esse e o processo filho.  Sai imediatamente.  */
17    exit (0);
18  }
19  return 0;
20 }
```

Tente compilar esse arquivo em um executável chamado *fazer-zumbi*. Rode esse executável, e enquanto ele ainda estiver sendo executado, liste os processos no sistema usando o seguinte comando em outra janela:

```
% ps -e -o pid,ppid,stat,cmd
```

O comando acima lista o ID de processo, ID do processo pai, situação atual do processo, e linha de comando do processo. Observe que, adicionalmente ao processo pai do processo *fazer-zumbi*, existe outro processo *fazer-zumbi* listado. Esse é o processo filho; note que seu ID de processo pai está ao lado do ID de processo do processo *fazer-zumbi* principal. O processo filho é marcado como <defunct>, e seu código de situação atual é "Z", de zumbi.[6] O que acontece quando o programa principal *fazer-zumbi* termina quando o processo pai sai, sem ter chamado a função *wait*? Fica o processo zumbi continua vagando por aí? Não – tente executar o comando *ps* novamente, e notar que ambos os processos pai e filho *fazer-zumbi* se foram. Quando um programa sai, seus filhos são herdados por um processo especial, o programa *init*, o qual sempre executa com o ID de processo como sendo 1 (é o primeiro processo iniciado quando GNU/Linux passa pelo processo de inicialização). O processo *init* automaticamente limpa qualquer processo filho zumbi que ele herda.

3.3.5 Limpando Filhos de Forma Não Sincronizada

Caso você esteja usando um processo filho simplesmente para executar outro programa, funciona de forma satisfatória chamar a função *wait* imediatamente no processo pai, que irá bloquear até que o processo filho seja completado. Mas muitas vezes, você irá desejar que o processo pai continue sendo executado, como um ou mais processos filhos executando de forma sincronizada. Como pode você garantir que limpou processos filhos que já tenham completado sua tarefa de forma que você não esqueça por aí pelo sistema processo zumbis, os quais consomem recursos de sistema, com informações falsas por aí?

Uma abordagem pode ser a chamada pelo processo pai das funções *wait3* ou *wait4* periodicamente, para limpar filhos zumbis. Chamando a função *wait* com esse objetivo não funciona bem pelo fato de que, se nenhum processo filho terminar, a chamada a *wait* irá bloquear o processo pai até que algum processo filho encerre. Todavia, as funções *wait3* e *wait4* recebem um parâmetro sinalizador adicional, para o qual você pode passar o valor sinalizador *WNOHANG*. Com esse sinalizador, a função chamada executa em modo não bloqueador de processo pai – irá limpar um processo filho que terminou se existir algum, ou simplesmente retornar se não houver nenhum

[6]Nota do tradutor: em um slackware 12.2 a saída, mostrando somente as duas linhas que interessam, foi a seguinte:

PID	PPID	STAT	CMD
9152	9133	S+	./fazer-zumbi .
9153	9152	Z+	[fazer-zumbi] <defunct>

processo filho executando. O valor de retorno da chamada é o ID do processo do filho encerrado, ou zero no caso de não haver nenhum processo sendo executado.

Uma solução mais elegante é notificar o processo pai quando um filho conclui seu trabalho. Existem muitas formas de fazer isso usando os métodos discutidos no Capítulo 5, "Comunicação Entre Processos"mas afortunadamente GNU/Linux faz isso para você, usando sinais. Quando um processo filho cumpre sua tarefa, GNU/Linux envia ao processo pai o sinal *SIGCHLD*. A disposição padrão desse sinal é não fazer nada, coisa que talvez você possa não ter notado antes.

Dessa forma, um caminho fácil para limpar processos filhos é pelo manuseio de *SIGCHLD*. Certamente, durante a limpeza de processos filhos, é importante guardar sua situação atual de encerramento se essa informação for necessária, pelo fato de uma vez que o processo for limpo usando *wait*, a sua informação de encerramento não mais estará disponível. A Listagem 3.7 mostra um exemplo de programa que usa uma função controladora de *SIGCHLD* para limpar seus processos filhos. [7]

Listagem 3.7: (sigchld.c) Limpando Processos filhos pelo manuseio de SIGCHLD

```
1 #include <signal.h>
2 #include <string.h>
3 #include <sys/types.h>
4 #include <sys/wait.h>
5
6 sig_atomic_t child_exit_status;
7
8 void clean_up_child_process (int signal_number)
9 {
10   /* Limpa o processo filho.  */
11   int status;
12   wait (&status);
13   /* Armazena sua situacao de saida em uma variavel global.  */
14   child_exit_status = status;
15 }
16
17 int main ()
18 {
19   /* Manipula SIGCHLD pela chamada a clean_up_child_process.  */
20   struct sigaction sigchld_action;
21   memset (&sigchld_action, 0, sizeof (sigchld_action));
22   sigchld_action.sa_handler = &clean_up_child_process;
23   sigaction (SIGCHLD, &sigchld_action, NULL);
24
25   /* Agora faz coisas, incluindo fork sobre um processo filho.  */
26   /* ...  */
27
28   return 0;
29 }
```

[7] O código em *clean_up_child_process* pode não trabalhar corretamente se houver mais que um processo filho. O kernel do GNU/Linux irá somente chamar o controlador de sinal uma vez se dois ou mais processos filhos encerrarem quase ao mesmo tempo. Portanto, caso haja mais de um processo filho, o controlador de sinal deve repetidamente chamar por *waitpid* (ou uma das outras funções relacionada) com a opção *WNOHANG* até que *waitpid* retorne.

Note como o controlador de sinal armazena a situação de saída do processo filho em uma variável global, da qual o programa principal pode acessá-la. Pelo fato de a variável se atribuída em um controlador de sinal, ela (a variável global) é do tipo *sig_atomic_t*.

Capítulo 4

Linhas de Execução

LINHAS DE EXECUÇÃO[1],COMO PROCESSOS, SÃO UM MECANISMO PARA PERMITIR A UM PROGRAMA fazer mais de uma coisa ao mesmo tempo. Da mesma forma que acontece com processos, linhas de execução parecem executar concorrentemente; o kernel GNU/Linux agenda-as de forma não sincronizada, interrompendo cada uma dessas linhas de execução de tempos em tempos para fornecer a outros uma chance para executar.

Conceitualmente, uma linha de execução existe dentro de um processo. Linhas de execução são menores unidades de execução que processos. Quando você chama um programa, GNU/Linux cria um novo processo e esse processo cria uma linha de execução simples, que executa o programa sequencialmente. Essa linha de execução pode criar linhas de execução adicionais; todas essas linhas de execução executam o mesmo programa no mesmo processo, mas cada linha de execução pode estar executando uma parte diferente do programa em qualquer tempo fornecido.

Nós vimos como um programa pode através de um *fork* criar um processo filho. O processo filho inicialmente executa seu programa pai, na memória virtual do processo pai, com descritores de arquivo do processo pai e assim por diante copiado tudo do processo pai. O processo filho pode modificar sua memória fechar descritores de arquivo, e coisas parecidas sem afetar seu processo pai, e vice-versa.[2] Quando um programa no processo filho cria outra linha de execução, apesar disso, nada é copiado. A linha de execução criadora e a linha de execução criatura compartilham o mesmo espaço de memória, os mesmos descritores de arquivo, e outros recursos de sistema como o original. Se uma linha de execução muda o valor de uma variável, por exemplo, a outra linha de execução sequencialmente irá ver o valor modificado. Similarmente,

[1]Nota do tradutor: *Threads*.

[2]Nota do tradutor: o processo pai pode fazer vários procedimentos sem afetar o filho.

se uma linha de execução fecha um descritor de arquivo, outra linha de execução pode não ler aquele descritor ou não escrever para aquele descritor. Pelo fato de um processo e todas as suas linhas de execução poderem executar somente um programa de cada vez, se alguma linha de execução dentro de um processo chama uma das funções *exec*[3], todas as outras linhas de execução são finalizadas (o novo programa pode, certamente, criar novas linhas de execução).

GNU/Linux implementa o padrão POSIX para Interface de Programação de Aplicação (API) de linha de execução (conhecido como *pthreads*) [4]. Todas funções de linha de execução e tipos de dado são declarados no arquivo de cabeçalho <pthread.h>. As funções POSIX de linha de execução não estão incluídas na biblioteca C GNU padrão. Ao invés disso, elas estão na *libpthread*, então você deve adicionar *-lpthread* à linha de comando quando você fizer a linkagem de seu programa.

4.1 Criação de Linhas de Execução

Cada linha de execução é identificada por um ID (identificador) de linha de execução. Quando for se referir a IDs de linha de execução em programas feitos em C ou em C++, use o tipo *pthread_t*.

Sobre criação, cada linha de execução executa uma função de linha de execução. Essa função de linha de execução é apenas uma função comum e contém o código que a linha de execução deve executar. Quando a função retorna, a linha de execução encerra. Em ambiente GNU/Linux, funções de linha de execução recebem um parâmetro único, do tipo *void**, e possuem o tipo de dado retornado também *void**. O parâmetro é o argumento da linha de execução: GNU/Linux passa o valor conforme a linha de execução sem olhar para o conteúdo. Seu programa pode usar esse parâmetro para passar dados para uma nova linha de execução. Reciprocamente, seu programa pode usar o valor de retorno para passar dados a partir de uma linha de execução existente de volta ao criador da linha de execução.

A função *pthread_create* cria uma nova linha de execução. Você alimenta a *pthread_create* com o seguinte:

[3]Nota do tradutor: relembrando que a família de funções *exec* substituem o programa que está sendo executado por outro.

[4]Nota do tradutor: p-*threads* ou POSIX-*threads* ou ainda *threads* POSIX.

1. Um apontador para uma variável do tipo *pthread_t*, na qual o ID de linha de execução da nova linha de execução está armazenado.

2. Um apontador para um objeto de atributo de linha de execução. Esse apontador controla detalhes de como a linha de execução interage com o restante do programa. Se você passa um dado *NULL* como atributo de linha de execução, uma linha de execução irá ser criada com os atributos padronizados de linha de execução. Atributos de linha de execução são discutidos na Seção 4.1.5, "Atributos de Linhas de Execução."

3. Um apontador para a função de linha de execução. Esse apontador é um apontador de função comum, do seguinte tipo:

   ```
   void* (*) (void*)
   ```

4. Um valor de argumento de linha de execução do tipo *void**. Todo o resto que você enviar é simplesmente passado como argumento para a função de linha de execução quando a linha de execução inicia sua execução.

Uma chamada a *pthread_create* retorna imediatamente, e a linha de execução original continua executando as instruções imediatamente após a chamada. Enquanto isso, a nova linha de execução inicia-se executando a função de linha de execução. GNU/Linux agenda ambas as linhas de execução de forma não sincronizada, e seu programa continua independentemente da ordem relativa na qual instruções são executadas em duas linhas de execução.

O programa na Listagem 4.1 cria uma linha de execução que imprime x's continuamente para a saída de erro. Após chamar *pthread_create*, a linha de execução principal imprime o's continuamente para a saída de erro.

Listagem 4.1: (thread-create.c) Criando uma Linha de Execução

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 /* Imprime x's para stderr.  O parametro nao e usado.  Nao retorna.  */
5
6 void* print_xs (void* unused)
7 {
8   while (1)
9     fputc ('x', stderr);
10  return NULL;
11 }
12
13 /* O programa principal.  */
14
15 int main ()
16 {
17   pthread_t thread_id;
18   /* Cria uma nova linha de execucao.  A nova linha de execucao ira executar  a
        funcao
19      print_xs.  */
20   pthread_create (&thread_id, NULL, &print_xs, NULL);
21   /* Imprime o's continuamente para stderr.  */
22   while (1)
23     fputc ('o', stderr);
24   return 0;
25 }
```

Compile e faça a linkagem desse programa usando o seguinte código:

```
\% cc -o thread-create thread-create.c -lpthread
```

Tente executá-lo para ver o que ocorre. Preste atençao ao padrão imprevisível de x's e o's devido à alternância de agendamentos do Linux com relação às duas linhas de execução.

Sob circunstâncias normais, uma linha de execução encerra-se por meio de uma entre duas formas. Uma forma, como ilustrado previamente, é por meio do retorno da função de linha de execução. O valor de retorno da função de linha de execução é usado para ser o valor de retorno da linha de execução. Alternativamente, uma linha de execução pode sair explicitamente por meio de uma chamada a *pthread_exit*. Essa função pode ser chamada de dentro da função de linha de execução ou a partir de alguma outra função chamada diretamente ou indiretamente pela função de linha de execução. O argumento para *pthread_exit* é o valor de retorno da linha de execução.

4.1.1 Enviando Dados a uma Linha de Execução

O argumento de linha de execução fornece um método conveniente de enviar dados a linhas de execução. Pelo fato de o tipo de dado do argumento ser *void**, apesar disso, você não pode enviar grande quantidade de dados diretamente através do argumento. Ao invés disso, use o argumento de linha de execução para enviar um apontador para alguma estrutura ou vetor de dados. Uma técnica comumente usada é definir uma estrutura para cada

função de linha de execução, a qual contém os "parâmetros" esperados pela função de linha de execução.

Usando o argumento de linha de execução, torna-se fácil reutilizar a mesma função de linha de execução para muitas linhas de execução. Todas essas linhas de execução executam o mesmo código, mas sobre diferentes dados.

O programa na Listagem 4.2 é similar ao exemplo anterior. O referido programa cria duas novas linhas de execução, um para imprimir x's e o outro para imprimir o's. Ao invés de imprimir infinitamente, apesar disso, cada linha de execução imprime um número fixo de caracteres e então encerra-se retornando à função de linha de execução. A mesma função de linha de execução, *char_print*, é usada em ambas as linhas de execução, mas cada linha de execução é configurada diferentemente usando a estrutura *char_print_parms*.

Listagem 4.2: (thread-create2) Cria Duas Linhas de Execução

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 /* Parametros a print_function.        */
5
6 struct char_print_parms
7 {
8    /* O caractere a imprimir. */
9    char character;
10   /* O numero de vezes a imprimir o caractere acima.       */
11   int count;
12 };
13
14 /* Imprima um certo numero de caracteres para stderr, como fornecido por PARAMETERS,
15    o qual e um apontador para um struct char_print_parms. */
16
17 void* char_print (void* parameters)
18 {
19   /* Converte o cookie pointer para o tipo correto. */
20   struct char_print_parms* p = (struct char_print_parms*) parameters;
21   int i;
22
23      for (i = 0; i < p->count; ++i)
24        fputc (p->character, stderr);
25      return NULL;
26 }
27
28 /* O programa principal.     */
29
30 int main ()
31 {
32  pthread_t thread1_id;
33  pthread_t thread2_id;
34  struct char_print_parms thread1_args;
35  struct char_print_parms thread2_args;
36
37  /* Cria uma nova linha de execucao para imprimir 30,000 'x's. */
38  thread1_args.character = 'x';
39  thread1_args.count = 30000;
40  pthread_create (&thread1_id, NULL, &char_print, &thread1_args);
41
42  /* Cria uma nova linha de execucao para imprimir 20,000 o's. */
43  thread2_args.character = 'o';
44  thread2_args.count = 20000;
45  pthread_create (&thread2_id, NULL, &char_print, &thread2_args);
46
47  return 0;
48 }
```

Mas Espere! O programa na Listagem 4.2 tem um erro sério nele. A li-

nha de execução principal (que executa a função *main*) cria as estruturas do parâmetro de linha de execução (*thread1_args* e *thread2_args*) como variáveis locais, e então passa apontadores para essas estruturas destinados às linhas de execução que cria. O que fazer para prevenir o Linux do agendamento das três linhas de execução de tal forma que a linha de execução principal termine antes de qualquer das duas outras linhas de execução terem terminado? *Nada*! Mas caso isso ocorra, a memória contendo as estruturas do parâmetro da linha de execução terá sido desalocada enquanto as outras duas linhas de execução estiverem ainda acessando-a.

4.1.2 Vinculando Linhas de Execução

Uma solução é forçar *main* a esperar até que as outras duas linhas de execução tenham terminado. O que precisamos é de uma função similar à função *wait* que espere pelo fim de uma linha de execução ao invés de esperar pelo fim de um processo. A função desejada é *pthread_join*, que recebe dois argumentos: o ID de linha de execução da linha de execução pelo qual vai esperar, e um apontador para uma varíavel do tipo *void** que irá receber o valor de retorno da linha de execução terminada. Se você não quiser preocupar-se com o valor de retorno, informe *NULL* como o segundo argumento.

A Listagem 4.3 mostra a função *main* corrigida para o exemplo de falha na listagem 4.2. Nessa versão, *main* não encerra até que ambas as linhas de execução imprimindo x's e o's tenham sido completadas, então elas não mais utilizam as estruturas de argumento.

Listagem 4.3: Função *main* revisada para thread-create2.c

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 /* Parametros para print_function.  */
5
6 struct char_print_parms
7 {
8   /* O caractere a imprimir.  */
9   char character;
10  /* O numero de vezes a imprimir.  */
11  int count;
12 };
13
14 /* Mostra um numero de caracteres a stderr, como fornecido por PARAMETERS,
15    o qual e um apontador para um struct char_print_parms.  */
16
17 void* char_print (void* parameters)
18 {
19   /* Converte o ponteiro cookie para o tipo certo.  */
20   struct char_print_parms* p = (struct char_print_parms*) parameters;
21   int i;
22
23   for (i = 0; i < p->count; ++i)
24     fputc (p->character, stderr);
25   return NULL;
26 }
27
28 /* O programa principal.  */
29
30 int main ()
31 {
32   pthread_t thread1_id;
33   pthread_t thread2_id;
34   struct char_print_parms thread1_args;
35   struct char_print_parms thread2_args;
36
37   /* Cria uma nova linha de execucao para mostrar 30000 x's.  */
38   thread1_args.character = 'x';
39   thread1_args.count = 30000;
40   pthread_create (&thread1_id, NULL, &char_print, &thread1_args);
41
42   /* Cria uma nova linha de execucao para mostrar 20000 o's.  */
43   thread2_args.character = 'o';
44   thread2_args.count = 20000;
45   pthread_create (&thread2_id, NULL, &char_print, &thread2_args);
46
47   /* Garante que a primeira linha de execucao tenha terminado.  */
48   pthread_join (thread1_id, NULL);
49   /* Garante que a segunda linha de execucao tenha terminado.  */
50   pthread_join (thread2_id, NULL);
51
52   /* Agora podemos seguramente retornar.  */
53   return 0;
54 }
```

A moral da estória: garanta que qualquer dado que seja passado a uma linha de execução por referência seja mantido na memória, *mesmo que por uma linha de execução diferente*, até que você tenha certeza que a linha de execução tenha terminado com esse dado. Essa garantia é verdadeira em ambos os casos tanto para variáveis locais, que são removidas quando as linhas de execução saem do ambiente no qual foram definidas, quanto para variáveis alocadas em grupo/pilha, que você libera através de um chamado a *free* (ou usando *delete* em C++).

4.1.3 Valores de Retorno de Linhas de Execução

Se o segundo argumento que você passar a *pthread_join* for não nulo, o valor de retorno da linha de execução será colocado na localização apontada por aquele argumento. O valor de retorno da linha de execução,da mesma forma que o argumento de linha de execução, é do tipo *void**. Se você desejar devolver um dado do tipo *int* simples ou outro número pequeno, você pode fazer isso facilmente convertendo o valor para *void** e então convertendo de volta para o tipo apropriado após chamar *pthread_join*. [5] O programa na Listagem 4.4 calcula o enésimo número primo em uma linha de execução isolada. O valor de retorno dessa linha de execução isolada é o número primo desejado. A linha de execução principal, enquanto isso, está livre para executar outro código. Note que o algorítmo de divisões sucessivas usado em *compute_prime* é completamente ineficiente; consulte um livro sobre algorítmos numéricos se você precisar calcular muitos primos em seus programas.

[5] Note que esse procedimento perde a portabilidade, e cabe a você garantir que seu valor pode ser convertido seguramente para *void** e ser convertido de volta sem perder bits.

Listagem 4.4: (primes.c) Calcula Números Primos em uma Linha de Execução

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 /* Calcula sucessivos numeros primos (muito ineficientemente). Retorna o
5    enesimo numero primo, onde N e o valor apontado por *ARG. */
6
7 void* compute_prime (void* arg)
8 {
9   int candidate = 2;
10  int n = *((int*) arg);
11
12  while (1) {
13    int factor;
14    int is_prime = 1;
15
16    /* Teste de primalidade por divisoes sucessivas. */
17    for (factor = 2; factor < candidate; ++factor)
18      if (candidate % factor == 0) {
19        is_prime = 0;
20        break;
21      }
22    /* E este o numero primo que estamos procurando? */
23    if (is_prime) {
24      if (--n == 0)
25        /* Retorna o numero primo desejado como valor de retorno da linha de execucao
            . */
26        return (void*) candidate;
27    }
28    ++candidate;
29  }
30  return NULL;
31 }
32
33 int main ()
34 {
35   pthread_t thread;
36   int which_prime = 5000;
37   int prime;
38
39   /* Inicia a linha de execucao de calculo, acima do 5000-esimo numero primo. */
40   pthread_create (&thread, NULL, &compute_prime, &which_prime);
41   /* Faz algum outro trabalho aqui... */
42   /* Espera que a linha de execucao de numero primo se complete, e pega o resultado.
        */
43   pthread_join (thread, (void*) &prime);
44   /* Mostra o maior primo calculado. */
45   printf("O %d-esimo numero primo e %d.\n", which_prime, prime);
46   return 0;
47 }
```

4.1.4 Mais sobre IDs de Linhas de Execução

Ocasionalmente, é útil para uma sequência de código determinar qual linha de execução a está executando. A função *pthread_self* retorna o ID da linha de execução que a chamou. Esse ID de linha de execução pode ser comparado com outro ID de linha de execução usando a função *pthread_equal*.

Essas funções podem ser úteis para determinar se um ID de linha de execução em particular corresponde ao ID da linha de execução atual. Por exemplo, é um erro para uma linha de execução chamar *pthread_join* para vincular-se a si mesma. (Nesse caso, *pthread_join* irá retornar o código de erro *EDEADLK*.) Para verificar isso antecipadamente, você pode usar um código como o que segue:

```
if (!pthread_equal (pthread_self (), other_thread))
  pthread_join (other_thread, NULL);
```

4.1.5 Atributos de Linha de Execução

Atributos de linha de execução fornecem um mecanismo para ajuste preciso do comportamento de linhas de execução individuais. Lembrando que *pthre ad_create* aceita um argumento que é um apontador para um objeto de atributo de linha de execução. Se você informar um apontador nulo, os atributos de ancadeamento padronizados são usados para configurar a nova linha de execução. Todavia, você pode criar e personalizar um objeto de atributo de linha de execução para especificar outros valores para os atributos. [6]

Para especificar atributos personalizados de linhas de execução, você deve seguir esses passos:

1. Crie um objeto *pthread_attr_t*. O caminho mais fácil de fazer isso é simplesmente declarar uma variável automática desse tipo.

2. Chame *pthread_attr_init*, informando um apontador para esse objeto. Esse procedimento inicializa os atributos com seus valores padronizados.

3. Modifique o objeto de atributo de forma que contenha os valores de atributo desejados.

4. Informe um apontador para o objeto de atributo ao chamar *pthread_create*.

5. Chame *pthread_attr_destroy* para liberar o objeto de atributo. A variável *pthread_attr_t* propriamente dita não é desalocada. A variável *pthread_attr_t* pode ser reinicializada com *pthread_attr_init*.

Um objeto de atributo de linha de execução simples pode ser usado para muitas linhas de execução. Não é necessário manter o objeto de atributo de linha de execução por ai após as linhas de execução terem sido criadas.

Para a maioria das linha de execução de programação para criação de aplicativos em GNU/Linux, somente um atributo de linha de execução é tipicamente de interesse (os outros atributos disponíveis são primariamente para especificidades de programação em tempo real). Esse atributo é o estado de desvinculação da linha de execução. Uma linha de execução pode ser criada como uma linha de execução vinculável (o padrão) ou como uma linha de execução desvinculada. Uma linha de execução vinculável, como um processo, não tem seus recursos de sistema liberados automaticamente pelo GNU/Linux quando termina sua execução. Ao invés disso, o estado de saída

[6] Nota do tradutor: para mais detalhes sobre threads/linhas de execução veja `http://www.yolinux.com/TUTORIALS/LinuxTutorialPosixThreads.html`.

da linha de execução vagueia sem destino no sistema (semelhantemente a um processo zumbi) até que outra linha de execução chame *pthread_join* para obter seu valor de retorno. Somente então são seus recursos liberados. Uma Linha de execução desvinculada, ao cantrário, tem seus recursos de sistema automaticamete liberados quando termina sua execução. Pelo fato de uma linha de execução desvinculada ter seus recursos liberados automaticamente, outra linha de execução pode não conseguir informações sobre sua conclusão através do uso de *pthread_join* ou obter seu valor de retorno.

Para atribuir o estado desvinculado a um objeto de atributo de linha de execução, use a função *pthread_attr_setdetachstate*. O primeiro argumento é um apontador para o objeto de atributo de linha de execução, e o segundo é o estado desvinculado desejado. Pelo fato de o estado vinculável ser o padrão, é necessário chamar a função *pthread_attr_setdetachstate* somente para criar linhas de execução desvinculadas; informe *PTHREAD_CREATE_DETACHED* como o segundo argumento.

O código na Listagem 4.5 cria uma linha de execução desvinculada usando o atributo de linha de execução desvinculada para a linha de execução.

Listagem 4.5: (detached.c) Programa Esqueleto Que Cria uma Linha dde Execução Desvinculada

```
1  #include <pthread.h>
2
3  void* thread_function (void* thread_arg)
4  {
5    /* Fazer o trabalho aqui...  */
6    return NULL;
7  }
8
9  int main ()
10 {
11   pthread_attr_t attr;
12   pthread_t thread;
13
14   pthread_attr_init (&attr);
15   pthread_attr_setdetachstate (&attr, PTHREAD_CREATE_DETACHED);
16   pthread_create (&thread, &attr, &thread_function, NULL);
17   pthread_attr_destroy (&attr);
18
19   /* Fazer o trabalho aqui...  */
20
21   /* Nao precisa associar a segunda linha de execucao.  */
22   return 0;
23 }
```

Mesmo se uma linha de execução for criada com o estado vinculável, ele pode ser transformado em uma linha de execução desvinculada. Para fazer isso, chame *pthread_detach*. Uma vez que seja desvinculada, ela não pode se tornar vinculável novamente.

4.2 Cancelar Linhas de Execução

Sob circunstâncias normais, uma linha de execução encerra-se quando seu estado de saída é normal, ou pelo retorno de seu valor de retorno ou por uma chamada à função *pthread_exit*. Todavia, é possível para uma linha de execução requisitar que outra linha de execução termine. Isso é chamado cancelar uma linha de execução.

Para cancelar uma linha de execução, chame a função *pthread_cancel*, informando o ID de linha de execução da linha de execução a ser cancelada. Uma linha de execução cancelada pode mais tarde ser vinculada; de fato, você pode vincular uma linha de execução cancelada para liberar seus recursos, a menos que a linha de execução seja desvinculada (veja a Seção 4.1.5, "Atributos de Linha de Execução"). O valor de retorno de uma linha de execução cancelada é o valor especial fornecido por *PTHREAD_CANCELED*.

Muitas vezes uma linha de execução pode ter alguma parte de seu código que deva ser executada em um estilo tudo ou nada. Por exemplo, a linha de execução pode alocar alguns recursos, usá-los, e então liberar esses mesmos recursos em seguida. Se a linha de execução for cancelada no meio do código, pode não ter a oportunidade de liberar os recursos como era esperado, e dessa forma os recursos irão ser perdidos. Para contar com essa possibilidade, é possível para uma linha de execução controlar se e quando ela pode ser cancelada.

Uma linha de execução pode estar em um dos três estados abaixo com relação a cancelar linhas de execução.

- A linha de execução pode ser cancelável de forma não sincronizada. Isso que dizer que a linha de execução pode ser cancelada em qualquer ponto de sua execução.
- A linha de execução pode ser cancelável sincronizadamente. A linha de execução pode ser cancelada, mas não em algum ponto determinado de sua execução. Ou ao contrário, requisições de cancelamento são colocadas em uma região temporária de armazenamento, e a linha de execução é cancelada somente quando forem alcançados pontos específicos em sua execução.
- Uma linha de execução pode ser incancelável. Tentativas de cancelar a linha de execução são silenciosamente ignoradas.

Quando criada inicialmente, uma linha de execução é cancelável sincronizadamente.

4.2.1 Linhas de Execução Sincronas e Assincronas

Uma linha de execução cancelável assincronizadamente pode ser cancelado em qualquer ponto de sua execução. Uma linha de execução cancelável sincronizadamente, ao contrário, pode ser cancelado somente em lugares determinados de sua execução. Esses lugares são chamados pontos de cancelamento. A linha de execução irá armazenar uma requisição de cancelamento até que o ponto de cancelamento seguinte seja alcançado.

Para fazer uma linha de execução assincronizadamente cancelável, use *pthread_setcanceltype*. A função *pthread_setcanceltype* afeta linha de execução que fez o chamado. O primeiro argumento deve ser *PTHREAD_CANCEL_ASYNCHRONOUS* para tornar a linha de execução assincronizadamente cancelável, ou *PTHREAD_CANCEL_DEFERRED* para retornar a linha de execução ao estado de sincronizadamente cancelável. O segundo argumento, se não for nulo, é um apontador para uma variável que irá receber o tipo de cancelamento anterior para a linha de execução. A chamada abaixo, por exemplo, transforma a linha de execução que está fazendo a chamada em assincronizadamente cancelável.

```
pthread_setcanceltype (PTHREAD_CANCEL_ASYNCHRONOUS, NULL);
```

O que constitui um ponto de cancelamento, e onde deve ele ser colocado? O caminho mais direto para criar um ponto de cancelamento é chamar a função *pthread_testcancel*. Essa chamada faz unicamente atender um pedido de cancelamento que se encontra pendente em uma linha de execução sincronizadamente cancelável. Você deve chamar a função *pthread_testcancel* periodicamente durante computações longas em uma função de linha de execução, em pontos onde a linha de execução pode ser cancelada sem desperdiçar quaisquer recursos ou produzir outros efeitos igualmente danosos.

Certas outras funções trazem implicitamente pontos de cancelamento também. São elas listadas na página de manual da função *pthread_cancel* [7]. Note que outras funções podem usar essas funções internamente e dessa forma serem pontos de cancelamento.

4.2.2 Seções Críticas Incanceláveis

Uma linha de execução pode desabilitar o cancelamento de si mesma completamente com a função *pthread_setcancelstate*. Da mesma forma que *pthread_setcanceltype*, a função *pthread_setcancelstate* afeta a linha de execução

[7] Nota do Tradutor:se for usado o comando "man pthread_cancel" e não se encontrará a referida página de manual instalada no ubuntu 10.10 *default* mas na *Internet* existem pelo menos duas versões de man page para *pthread_cancel*.

que fizer a chamada. O primeiro argumento é *PTHREAD_CANCEL_DISABLE* para disabilitar a cancelabilidade, ou *PTHREAD_CANCEL_ENABLE* para reabilitar a cancelabilidade. O segundo argumento, se não for *NULL*, aponta para uma variável que irá receber o estado de cancelamento anterior. A chamada a seguir, por exemplo, desabilita a cancelabilidade da linha de execução na linha de execução que fizer a referida chamada.

```
pthread_setcancelstate (PTHREAD_CANCEL_DISABLE, NULL);
```

Usando a função *pthread_setcancelstate* habilita você a implementar seções críticas. Uma seção crítica é uma sequência de código que deve ser executado ou em sua totalidade ou parcialmente; em outras palavras, se uma linha de execução inicia-se executando uma seção crítica, essa linha de execução deve continuar até o final da seção crítica sem ser cancelada.

Por exemplo, suponhamos que você está escrevendo uma rotina para um programa bancário que transfere dinheiro de uma conta para outra. Para fazer isso você deve adicionar valor ao saldo em uma conta e abater o mesmo valor do saldo de outra conta. Se a linha de execução que estiver executando sua rotina for cancelada exatamente no péssimo momento entre essas duas operações, o programa pode ter um aumento espúrio do depósito total causado pela falha na conclusão da transação. Para previnir essa possibilidade, coloque as duas operações dentro de uma seção crítica.

Você pode implementar a transferência com uma função tal como a *process_transaction*, mostrada na Listagem 4.6. Essa função desabilita o cancelamento da linha de execução para iniciar uma seção crítica antes que a função modifique ou um ou outro balanço de conta.

Listagem 4.6: (critical-section.c) Protege uma Transação Bancária com uma Seção Crítica

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 /* Um array de balancos em contas, indexado por numero de conta.  */
6
7 float* account_balances;
8
9 /* Transfere DOLLARS da conta FROM_ACCT para a conta TO_ACCT.  Retorna
10    0 se a transacao obtiver sucesso, ou 1 se o balanco de FROM_ACCT for
11    muito pequeno.  */
12
13 int process_transaction (int from_acct, int to_acct, float dollars)
14 {
15   int old_cancel_state;
16
17   /* Verifica o balanco em FROM_ACCT.  */
18   if (account_balances[from_acct] < dollars)
19     return 1;
20
21   /* Comeca a secao critica.  */
22   pthread_setcancelstate (PTHREAD_CANCEL_DISABLE, &old_cancel_state);
23   /* Move o dinheiro.  */
24   account_balances[to_acct] += dollars;
25   account_balances[from_acct] -= dollars;
26   /* Fim da secao critica.  */
27   pthread_setcancelstate (old_cancel_state, NULL);
28
29   return 0;
30 }
```

Note que é importante restaurar o estado de cancelamento antigo no final da seção crítica em lugar de escolher incondicionalmente o estado *PTHREAD_CANCEL_ENABLE*. restaurando o estado antigo ao invés de usar incondicionalmente *PTHREAD_CANCEL_ENABLE* habilita você a chamar a função *process_transaction* seguramente de dentro de outra seção crítica – como no caso mostrado acima, permitindo que o estado de cancelamento seja colocado da mesma forma que se encontrava antes da sua intervenção.

4.2.3 Quando Cancelar uma Linha de Execução

Em geral, é uma boa idéia não cancelar linhas de execução para encerrar a execução de uma linha de execução, exceto em circunstâncias raras. Durante operações normais, a melhor estratégia é indicar à linha de execução que ela deve encerrar, e então esperar o término da linha de execução por seu estilo próprio e ordeiro. Iremos discutir técnicas para comunicação com linhas de execução mais tarde no atual capítulo, e no Capítulo 5, "Comunicação Entre Processos."

4.3 Área de Dados Específicos de Linha de Execução

Ao contrário dos processos, todas as linhas de execução em um programa simples compartilham o mesmo espaço de endereçamento. Isso significa que se uma linha de execução modifica uma localização na memória (por exemplo, uma variável global), a mudança é visível para todas as outras linhas de execução. Isso permite que multiplas linhas de execução operem sobre os mesmos dados sem o uso de mecanismos de comunicação entre processos (que são descritos no Capítulo 5).

Cada linha de execução tem dua própria pilha de chamadas, apesar do exposto acima. Isso permite a cada linha de execução executar código diferente e chamar e retornar de sub-rotinas no caminho usual. Como no programa de linha de execução simples, cada chamada a uma sub-rotina em cada linha de execução tem seu próprio conjunto de variáveis locais, que é armazenada na pilha para aquela linha de execução.

Algumas vezes, todavia, é desejável duplicar uma certa variável de forma que cada linha de execução tenha uma cópia separada. GNU/Linux suporta isso fornecendo cada linha de execução com uma área de dados específicos de linha de execução. As variáveis armazenadas nessa área são duplicadas para cada linha de execução, e cada linha de execução pode modificar sua cópia da variável sem afetar outras linhas de execução. Devido ao fato de todas as linhas de execução compartilharem o mesmo espaço de memória, dados específicos de linha de execução não podem ser acessados usando referências normais de variáveis. GNU/Linux fornece funções especiais para modificar e recuperar valores da área de dados específicos de linha de execução.

Você pode criar tantos dados específicos de linha de execução quantos você quiser, cada um do tipo *void**. Cada item é referenciado por uma chave. Para criar uma nova chave, e dessa forma um novo item de dado para cada linha de execução, use a função *pthread_key_create*. O primeiro argumento é um apontador para uma variável do tipo definido em *pthread_key_t*. Esse valor de chave pode ser usado por cada linha de execução para acessar sua própria cópia do correspondente item dos dados. O segundo argumento a *pthread_key_create* é uma função de limpeza. Se você informar um apontador de função aqui, GNU/Linux automaticamente chama aquela função indicada pelo apontador informado quando cada linha de execução terminar sua execução, informando o valor específico da linha de execução que corresponde áquela chave. Isso é particularmente adequado pelo fato de a função de limpeza ser chamada mesmo se a linha de execução for cancelada em algum ponto arbitrário em sua execução. Se o valor específico da linha de execução

for *NULL*, a função de limpeza da linha de execução não é chamada. Se você não precisa de uma função de limpeza, você pode informar *null* ao invés de um apontador de função.

Após você ter criado uma chave, cada linha de execução pode modificar seu valor específico correspondente para aquela chave chamando a função *pthread_setspecific*. O Primeiro argumento é a chave, e o segundo é do tipo *void** e corresponde ao valor específico da linha de execução a ser armazenado. Para recuperar algum item de dados específicos da linha de execução, chame a função *pthread_getspecific*, informando a chave como seu argumento.

Suponhamos, por exemplo, que sua aplicação distribua um trabalho entre diversas linhas de execução. Para propósitos de auditoria, cada linha de execução tem um arquivo de *log* separado, no qual mensagens de progresso, para os trabalhos executados por aquela linha de execução, são gravadas. A área especifica de dados é um lugar conveniente para armazenar o apontador para o arquivo de *log* de cada linha de execução.

A Listagem 4.7 mostra como você pode implementar isso. A função principal nesse programa exemplo cria uma chave para armazenar o apontador ao arquivo específico da linha de execução e então armazenar as informações em *thread_log_key*. Pelo fato de *thread_log_key* ser uma variável global, ela é compartilhada por todas as linhas de execução. Quando cada linha de execução inicia executando sua função de linha de execução, a linha de execução abre um arquivo de *log* e armazena o apontador de arquivo sob aquela chave. Mais tarde, qualquer dessas linhas de execução pode chamar *write_to_thread_log* para escrever uma mensagem para o arquivo de *log* específico de linha de execução. A função *write_to_thread_log* recupera o apontador de arquivo para o arquivo de *log* da linha de execução para dados específicos de linha de execução e escreve a mensagem.

Listagem 4.7: (tsd.c) *Log* Por Linhas de Execução Implementado com Dados Específicos de Linha de Execução

```
1 #include <malloc.h>
2 #include <pthread.h>
3 #include <stdio.h>
4
5 /* A chave usada para associar um apontador de arquivo de registro a cada linha de
      execucao.  */
6 static pthread_key_t thread_log_key;
7
8 /* Escreve MESSAGE no arquivo de log para a atual linha de execucao.  */
9
10 void write_to_thread_log (const char* message)
11 {
12   FILE* thread_log = (FILE*) pthread_getspecific (thread_log_key);
13   fprintf (thread_log, "%s\n", message);
14 }
15
16 /* Fecha o apontador para o arquivo de log THREAD_LOG.  */
17
18 void close_thread_log (void* thread_log)
19 {
20   fclose ((FILE*) thread_log);
21 }
22
23 void* thread_function (void* args)
24 {
25   char thread_log_filename[20];
26   FILE* thread_log;
27
28   /* Gera o nome de arquivo para esse arquivo de log de linha de execucao.  */
29   sprintf (thread_log_filename, "thread%d.log", (int) pthread_self ());
30   /* Open the log file.  */
31   thread_log = fopen (thread_log_filename, "w");
32   /* Armazena o apontador de arquivo em dados de thread-specific sob thread_log_key.
        */
33   pthread_setspecific (thread_log_key, thread_log);
34
35   write_to_thread_log ("Thread starting.");
36   /* Faz algum trabalho aqui... */
37
38   return NULL;
39 }
40
41 int main ()
42 {
43   int i;
44   pthread_t threads[5];
45
46   /* Cria uma chave para associar o apontador de arquivo de log de uma linha de
        execucao em
47      dados de thread-specific.  Use close_thread_log para limpar os apontadores
48      arquivo.  */
49   pthread_key_create (&thread_log_key, close_thread_log);
50   /* Cria linhas de execucao para fazer o trabalho.  */
51   for (i = 0; i < 5; ++i)
52     pthread_create (&(threads[i]), NULL, thread_function, NULL);
53   /* Espera por todas as linhas de execucao terminarem.  */
54   for (i = 0; i < 5; ++i)
55     pthread_join (threads[i], NULL);
56   return 0;
57 }
```

Observe que *thread_function* não precisa fechar o arquivo de *log*. Isso ocorre pelo fato de que ao ser o arquivo de *log* criado, *close_thread_log* foi especificada como a função de limpeza para aquela chave. Sempre que uma linha de execução encerra, GNU/Linux chama *close_thread_log*, informando o valor específico de linha de execução para a chave do *log* específico da linha de execução. Essa função toma o cuidado de fechar o arquivo de *log*.

4.3.1 Controladores de Limpeza

As funções de limpeza para dados específicos de linha de execução são necessárias para garantir que recursos não sejam perdidos quando a linha de execução encerrar ou for cancelada. Algumas vezes, ao longo de todo um projeto de *software*, é útil estar apto a especificar funções de limpeza sem criar novos itens de dados específicos de linha de execução que é duplicado para cada linha de execução. GNU/Linux fornece cabeçalhos de limpeza para esse propósito.

Um controlador de limpeza é simplesmente uma função que deve ser chamada quando a linha de execução termina. O controlador recebe um parâmetro único do tipo *void**, e seu valor de argumento é fornecido quando o controlador é registrado – isso facilita o uso da mesma função controladora para liberar recursos em multiplas instâncias.

Um controlador é um procedimento temporário, usado para liberar um recurso somente se a linha de execução encerrar ou for cancelada ao invés de terminar a execução de uma região particular de código. Sob circunstâncias normais, quando a linha de execução não encerra e não é cancelada, o recurso deve ser liberado explicitamente e o controlador de limpeza deve ser removido.

Para registrar um controlador de limpeza, chame a função *pthread_clean up_push*, informando um apontador para a função de limpeza e o valor do seu argumento *void**. A chamada a *pthread_cleanup_push* deve ser equilibrada por uma correspondente chamada a *pthread_cleanup_pop*, que remove o registro do maniplulador de limpeza. Por conveniência, *pthread_cleanup_pop* recebe um argumento sinalizador do tipo *int*; se o sinalizador for diferente de zero, a ação de limpeza é executada imediatamente e seu registro é removido.

O fragmento de programa na Listagem 4.8 mostra como você pode possivelmente usar um controlador de limpeza para garantir que um espaço temporário de armazenamento alocado dinamicamente seja limpo se a linha de execução terminar.

Listagem 4.8: (cleanup.c) Fragmento de Programa Demonstrando um Controlador de Limpeza de Linha de Execução

```
1 #include <malloc.h>
2 #include <pthread.h>
3
4 /* Aloca um espaco temporario de armazenagem.  */
5
6 void* allocate_buffer (size_t size)
7 {
8   return malloc (size);
9 }
10
11 /* Desaloca um espaco temporario de armazenagem passageiro.  */
12
13 void deallocate_buffer (void* buffer)
14 {
15   free (buffer);
16 }
17
18 void do_some_work ()
19 {
20   /* Aloca um espaco temporario de armazenagem.  */
21   void* temp_buffer = allocate_buffer (1024);
22   /* Registra um manipulador de limpeza para esse espaco temporario de armazenagem,
        para desaloca-lo no
23      caso da linha de execucao sair ou ser cancelada.  */
24   pthread_cleanup_push (deallocate_buffer, temp_buffer);
25
26   /* Fazer alguma coisa aqui que pode chamar pthread_exit ou pode ser
27      cancelada...  */
28
29   /* Desregistrar o manipulador de limpeza.  Uma vez que informamos um valor nao nulo
        ,
30      esse rotina aqui executa atualmente a limpeza atraves de
31      deallocate_buffer.  */
32   pthread_cleanup_pop (1);
33 }
```

Pelo fato de o argumento a *pthread_cleanup_pop* ser diferene de zero nesse caso, a função de limpeza *deallocate_buffer* é chamada automaticamente aqui e não precisa ser chamada explicitamente. Nesse único caso, pudemos ter a função da biblioteca padrão liberando diretamente como nosso controlador de limpeza ao invés de *deallocate_buffer*.

4.3.2 Limpeza de Linha de Execução em C++

Programadores em C++ estão acostumados limpar livremente empacotando ações de limpeza em objetos destrutores. Quando os objetos saem fora do escopo, ou por que um bloco é executado para completar alguma coisa ou pelo fato de uma exceção ser esquecida, C++ garante que destrutores sejam chamados para aquelas variáveis automáticas que tiverem as referidas exceções e blocos. Esse comportamento de C++ fornece um mecanismo controlador para garantir que código de limpeza seja chamado sem importar como o bloco terminou.

Se uma linha de execução chama a função *pthread_exit*, C++ não garante que destrutores sejam chamados para todas as variáveis automáticas na pilha da linha de execução. Uma maneira inteligente de recuperar essa funcionalidade é invocar a função *pthread_exit* no nível mais alto da função de linha

de execução abandonando alguma exceção especial.

O programa na Listagem 4.9 demonstra isso. Usando essa técnica, uma função indica sua intenção de encerrar a linha de execução abandonando uma *ThreadExitException* ao invés de chamar *pthread_exit* diretamente. Pelo fato de a exceção ter sido detectada na função de linha de execução de nível mais alto, todas as variáveis locais sobre a pilha da linha de execução serão destruídas como se a exceção limpasse a si mesma.

Listagem 4.9: (cxx-exit.cpp) Implementando Saída Segura de uma Linha de Execução com Exceções de C++

```
1 #include <pthread.h>
2
3 extern bool should_exit_thread_immediately ();
4
5 class ThreadExitException
6 {
7 public:
8   /* Cria uma execao sinalizando a saida da linha de execucao com RETURN_VALUE.  */
9   ThreadExitException (void* return_value)
10    : thread_return_value_ (return_value)
11  {
12  }
13
14  /* Atualmente sai da linha de execucao, usando o valor de retorno fornecido no
15     construtor.  */
16  void* DoThreadExit ()
17  {
18    pthread_exit (thread_return_value_);
19  }
20
21 private:
22  /* O valor de retorno que ira ser usado quando da saida da linha de execucao.  */
23  void* thread_return_value_;
24 };
25
26 void do_some_work ()
27 {
28   while (1) {
29     /* Faz algumas coisas uteis aqui...  */
30
31     if (should_exit_thread_immediately ())
32       throw ThreadExitException (/* valor de retorno da linha de execucao = */ NULL);
33   }
34 }
35
36 void* thread_function (void*)
37 {
38   try {
39     do_some_work ();
40   }
41   catch (ThreadExitException ex) {
42     /* Alguma funcao indicada que devemos sair da linha de execucao.  */
43     ex.DoThreadExit ();
44   }
45   return NULL;
46 }
```

4.4 Sincronização e Seções Críticas

Programar com linhas de execução é muito complicado pelo fato de que a maioria dos programas feitos usando linhas de execução serem programas que competem uns com os outros. Em particular, não existe caminho para saber quando o sistema irá agendar uma linha de execução para ser execu-

tada e quando o sistema irá executar outra linha de execução. Uma linha de execução pode ser executada pelo sistema por tempo muito longo, ou o sistema pode alternar entre diversas linhas de execução muito rapidamente. Em um sistema com múltiplos processadores, o sistema pode mesmo agendar multiplas linhas de execução para serem executadas literalmente ao mesmo tempo.

Depurar um programa que usa linha de execução é difícil pelo fato de você não poder sempre e facilmente reproduzir o comportamento que causa o problema. Você pode executar o programa e ter tudo trabalhando perfeitamente; a próxima vez que você executar o programa, ele pode cair. Não existe caminho para fazer o sistema agendar as linhas de execução exatamente da mesma maneira que foi feito anteriormente.

A mais recente causa da maioria dos erros envolvendo linhas de execução é que as linhas de execução diferentes acessando a mesma informação na memória. Como mencionado anteriormente, esse comportamento de diversas linhas de execução acessaem a mesma informação é um dos poderosos aspéctos de uma linha de execução, mas esse comportamento tambẽm pode ser perigoso. Se uma linha de execução atualiza parcialmente uma estrutura de dados quando outra linha de execução acessa a mesma estrutura de dados, vai provavelmente acontecer uma confusão. Muitas vezes, programas que usam linha de execução e possuem erros carregam um código que irá trabalhar somente se uma linha de execução recebe agendamento muitas vezes mais – ou mais cedo – que outra linha de execução. Esses erros são chamados condições de corrida; as linhas de execução estão competindo uma com a outra para modificar a mesma estrutura de dados.

4.4.1 Condições de Corrida

Suponhamos que seu programa tenha uma série de trabalhos enfileirados que são processados por muitas linhas de execução concorrentes. A fila de trabalhos é representada por uma lista linkada de objetos de estrutura de trabalho. Após cada linha de execução terminar uma operação, ela verifica a fila para ver se um trabalho adicional está disponível. Se *job_queue* for diferente de *NULL*, a linha de execução remove o trabalho do topo da lista linkada e posiciona *job_queue* no próximo trabalho da lista. A função de linha de execução que processa trabalhos na fila pode parecer-se com a Listagem 4.10.

Listagem 4.10: (job-queue1.c) Função de Linha de Execução para Processar Trabalhos Enfileirados

```
1 #include <malloc.h>
2
3 struct job {
4   /* Campo encadeado para lista encadeada.  */
5   struct job* next;
6
7   /* Outros campos descrevendo trabalho a ser feito... */
8 };
9
10 /* Uma lista encadeada de trabalhos pendentes.  */
11 struct job* job_queue;
12
13 extern void process_job (struct job*);
14
15 /* Processa trabalhos da fila ate que a lista esteja vazia.  */
16
17 void* thread_function (void* arg)
18 {
19   while (job_queue != NULL) {
20     /* Pega o proximo trabalho disponivel.  */
21     struct job* next_job = job_queue;
22     /* Remove esse trabalho da lista.  */
23     job_queue = job_queue->next;
24     /* Realiza o trabalho.  */
25     process_job (next_job);
26     /* Limpa.  */
27     free (next_job);
28   }
29   return NULL;
30 }
```

Agora suponhamos que duas linhas de execução encerrem um trabalho aproximadamente ao mesmo tempo, mas somente um trabalho reste na fila. A primeira linha de execução verifica se *job_queue* é *NULL*; encontrando que não é, a linha de execução entra no laço e armazena o apontador para o objeto de trabalho em *next_job*. Nesse ponto, o sistema GNU/Linux interrompe a primeira linha de execução e agenda a segunda. A segunda linha de execução também verifica se *job_queue* é *NULL*; e encontrando que não é, também atribui o mesmo apontador de trabalho para *next_job*. Por desafortunada coincidência, temos agora duas linhas de execução executando o mesmo trabalho.

Para piorar a situação, uma linha de execução irá deslinkar o objeto de trabalho da lista, permitindo que *job_queue* contenha *NULL*. Quando a outra linha de execução avaliar *job_queue->next*, uma falha de segmentação irá aparecer.

Esse é um exemplo de condição de corrida. Sob "afortunadas"circunstâncias, esse particular agendamento de duas linhas de execução podem nunca ocorrer, e a condição de corrida pode nunca mostrar-se. Somente em circunstâncias diferenciadas, talvez ao executar sobre um sistema muito pesado (ou sobre um novo servidor multi-processado de um importante usuário!) pode o erro mostrar-se.

Para eliminar condições de corrida, você precisa de um caminho para fazer operações atômicas. Uma operação atômica é indivisível e não pode ser

interrompida; uma vez que a operação for iniciada, não irá ser pausada ou interrompida até que se complete, e nenhuma outra operação irá tomar o seu lugar enquanto isso. Nesse exemplo em particular, você irá querer verificar *job_queue*; se não estivar vazia, remover o primeiro trabalho, tudo isso junto como uma operação atômica única.

4.4.2 Mutexes

A solução para o problema da condição de corrida da fila de trabalho é permitir que somente uma linha de execução por vez acesse a fila de linhas de execução. Assim que uma linha de execução inicia olhando na fila, nenhuma outra linha de execução deve estar apta a acessar a fila até que a primeira linha de execução tenha decidido se realiza um trabalho e, se fizer isso , tiver removido o trabalho da lista.

A implementação disso requer suporte por parte do sistema operacional. GNU/Linux fornece *mutexes*, abreviatura de trava de exclusão mútua [8]. Um *mutex* é uma trava especial que somente uma linha de execução pode travar a cada vez. Se uma linha de execução trava um *mutex* e então uma segunda linha de execução também tenta travar o mesmo *mutex*, a segunda linha de execução é bloqueada, ou colocada em espera. somente quando a primeira linha de execução destrava o *mutex* é a segunda linha de execução desbloqueada – permitindo sua execução. GNU/Linux garante que condições de corrida não ocorram em meio a linhas de execução que tentem travar um *mutex*; somente uma linha de execução irá mesmo pegar a trava, e todas as outras linhas de execução irão ser bloqueadas.

Pensando em um *mutex* como a trava de uma porta de banheiro. Quem chegar primeiro entra no banheiro e trava a porta. Se alguma outra pessoa tenta entrar no banheiro enquanto ele estiver ocupado, aquela pessoa encontra a porta fechada e irá ser forçada a esperar do lado de fora até que o ocupante apareça.

Para criar um *mutex*, crie uma variável do tipo *pthread_mutex_t* e informe um apontador para essa variável criada para a função *pthread_mutex_init*. O segundo argumento de *pthread_mutex_init* é um apontador para um objeto de atributo de *mutex*, que especifica os atributos de um *mutex*. Da mesma forma que ocorre com a função *pthread_create*, se o apontador de atributo for nulo, atributos padronizados são assumidos. A Variável *mutex* deve ser inicializada somente uma única vez. Esse fragmento de código adiante demonstra a declaração e a inicialização de uma variável *mutex*.

```
pthread_mutex_t mutex;
```

[8]Nota do tradutor:*MUTual EXclusion*.

```
pthread_mutex_init (&mutex, NULL);
```

Outra maneira mais simples de criar um *mutex* com atributos padronizados é inicializar o referido *mutex* com o valor especial *PTHREAD_MUTEX_INITIALIZER*. Nenhuma chamada adicional a *pthread_mutex_init* é necessária. Essa forma é particularmente conveniente para variáveis globais (e, em C++, membros de dados estáticos). O fragmento de código acima poderia equivalentemente ter sido escrito como segue:

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
```

Uma linha de execução pode tentar travar um *mutex* por meio de uma chamada a *pthread_mutex_lock* referindo-se ao dito *mutex*. Se o *mutex* estiver desbloqueado, ele torna-se travado e a função retorna imediatamente. Se o *mutex* estiver travado por outra linha de execução, *pthread_mutex_lock* bloqueia a execução e retorna somente quando o *mutex* for desbloqueado pela outra linha de execução. Diversas linhas de execução ao mesmo tempo podem ser bloqueadas ao tentarem usar um *mutex* travado. Quando o *mutex* for desbloqueado, somente uma das linhas de execução bloqueadas (escolhida de forma imprevisível) é desbloqueada e é permitido que a referida linha de execução trave o *mutex*; as outras linhas de execução continuam bloqueadas.

Uma chamada a *pthread_mutex_unlock* desbloqueia um *mutex*. Essa função deve sempre ser chamada a partir da mesma linha de execução que travou o *mutex*.

A listagem 4.11 mostra outra versão do exemplo de fila de trabalhos. Agora a fila é protegida por um *mutex*. Antes de acessar a fila (ou para leitura ou para escrita), cada linha de execução trava um *mutex* primeiramente. Somente quando a completa sequência de verificar a fila e remover um trabalho for completada é o mutex destravado. Isso evita a condição de corrida previamente descrita.

Listagem 4.11: (job-queue2.c) Função de Tarefa da Fila de Trabalho, Protegida por um *Mutex*

```
1 #include <malloc.h>
2 #include <pthread.h>
3
4 struct job {
5   /* Campo encadeado para lista encadeada.  */
6   struct job* next;
7
8   /* Outros campos descrevendo o trabalho a ser feito... */
9 };
10
11 /* Uma lista encadeada de trabalhos pendentes.  */
12 struct job* job_queue;
13
14 extern void process_job (struct job*);
15
16 /* Um mutex protegendo job_queue.  */
17 pthread_mutex_t job_queue_mutex = PTHREAD_MUTEX_INITIALIZER;
18
19 /* Processa trabalhos da fila ate que a fila esteja vazia.  */
20
21 void* thread_function (void* arg)
22 {
23   while (1) {
24     struct job* next_job;
25
26     /* Trava o mutex sobre o trabalho da fila.  */
27     pthread_mutex_lock (&job_queue_mutex);
28     /* Agora e seguro verificar se a fila esta vazia.  */
29     if (job_queue == NULL)
30       next_job = NULL;
31     else {
32       /* Pega o proximo trabalho disponivel.  */
33       next_job = job_queue;
34       /* Remove esse trablhoda lista.  */
35       job_queue = job_queue->next;
36     }
37     /* Desbloqueia o mutex sobre o trabalho da fila, uam vez que terminamos com a
38     fila por agora.  */
39     pthread_mutex_unlock (&job_queue_mutex);
40
41     /* Esta a fila vazia?  Se estiver, termine a linha de execucao.  */
42     if (next_job == NULL)
43       break;
44
45     /* Realiza o trabalho.  */
46     process_job (next_job);
47     /* Limpa.  */
48     free (next_job);
49   }
50   return NULL;
51 }
```

Todo o acesso a *job_queue*, o apontador de dados compartilhados, vem entre a chamada a *pthread_mutex_lock* e a chamada a *pthread_mutex_unlock*. Um objeto de trabalho, armazenado em *next_job*, é acessado de fora dessa região somente após aquele objeto de trabalho ter sido removido da fila e estar, dessa forma, inacessível a outras linhas de execução.

Note que se a fila estiver vazia (isto é, *job_queue* for *NULL*), nós não saímos fora do laço imediatamente pelo fato de termos que manter o *mutex* permanentemente travado e devemos prevenir que qualquer outra linha de execução acesse a fila de trabalhos novamente pois ela está vazia. Ao invés disso, lembramos esse fato escolhendo *next_job* para *NULL* e saimos fora do laço somente após desbloquear o *mutex*.

O uso de *mutex* para travar *job_queue* não é automático; cabe a você

adicionar o código para travar o *mutex* antes de acessar *job_queue* e também o código para destravar *job_queue* posteriormente. Por exemplo, uma função para adicionar um trabalho à fila de trabalhos pode parecer-se com isso:

```
void enqueue_job (struct job* new_job)
{
  pthread_mutex_lock (&job_queue_mutex);

        new_job->next = job_queue;
        job_queue = new_job;
        pthread_mutex_unlock (&job_queue_mutex);
   }
```

4.4.3 Travas Mortas de *Mutex*

Mutexes fornecem um mecanismo para permitir que uma linha de execução bloquei a execução de outra. Esse procedimento abre a possibilidade de uma nova classe de falhas, chamadas travas mortas. Uma trava morta ocorre quando uma ou mais linhas de execução estão presas esperando por alguma coisa que nunca irá ocorrer.

Um tipo único de trava morta ocorre quando a mesma linha de execução tenta bloquear um *mutex* duas vezes em uma linha. O comportamento nesse caso depende de qual tipo de *mutex* está sendo usado. Existem três tipos de *mutex*:

- rápido - travando um *mutex* rápido (o tipo padrão) fará com que ocorra uma trava morta. Como foi dito anteriormente, uma tentativa trava os blocos *mutex* até que o *mutex* seja desbloqueado. Mas pelo fato de a linha de execução que travou o *mutex* estar bloqueada nesse mesmo *mutex*, a trava não pode nunca ser liberada.

- recursivo - travando um *mutex* recursivo não causa uma trava morta. Um *mutex* recursivo pode seguramente ser travado várias vezes pela mesma linha de execução. O *mutex* recursivo lembra quantas vezes *pthread_mutex_lock* foi chamada sobre o mesmo *mutex* pela linha de execução que segura a trava; a linha de execução que segura a trava deve fazer o mesmo número de chamadas a *pthread_mutex_unlock* antes do *mutex* atual ser desbloqueado e outra linha de execução conseguir travar o *mutex* liberado.

- verificação de erro - GNU/Linux irá detectar e sinalizar uma trava dupla sobre um *mutex* de verificação de erro que poderia de outra forma causar uma trava morta. A segunda chamada consecutiva a *pthread_mutex_lock* retorna o código de falha *EDEADLK*.

Por padrão, um *mutex* GNU/Linux é do tipo rápido. Para criar um *mutex* de um dos outros dois tipos, primeiro crie um objeto de atributo de *mutex* declarando uma variável do tipo *pthread_mutexattr_t* e chamando *pthread_mutexattr_init* sobre um apontador para a variável do tipo *pthread_mutex attr_t*. A seguir ajuste o tipo do *mutex* chamando *pthread_mutexattr_setkind_np*; o primeiro argumento é um apontador para o objeto de atributo de *mutex*, e o segundo é *PTHREAD_MUTEX_RECURSIVE_NP* para um *mutex* recursivo, ou *PTHREAD_MUTEX_ERRORCHECK_NP* para um *mutex* de verificação de erro. Informe um apontador para esse atributo de objeto na função *pthread_mutex_init* para criar um *mutex* do tipo de verificação de erro, e então destrua o objeto de atributo com a função *pthread_mutexattr_destroy*.

A sequência de código abaixo ilustra a criação de um*mutex* de verificação de erro, por exemplo:

```
pthread_mutexattr_t attr;
pthread_mutex_t mutex;
pthread_mutexattr_init (\&attr);
pthread_mutexattr_setkind_np (\&attr, PTHREAD_MUTEX_ERRORCHECK_NP);
pthread_mutex_init (\&mutex, \&attr);
pthread_mutexattr_destroy (\&attr);
```

Como sugerido pelo sufixo "np", os *mutexes* do tipo recursivo e de verificação de erro são específicos do GNU/Linux e não são portáveis. Todavia, não é geralmente aconselhado usar esses dois tipos de *mutexes* em programas. (*Mutexes* de verificação de erro podem ser úteis quando se faz depurações, apesar disso.)

4.4.4 Testes de *Mutex* sem Bloqueio

Ocasionalmente, é útil testar se um *mutex* está travado sem sofrer bloqueio algum relativamente a esse *mutex*. Por exemplo, uma linha de execução pode precisar travar um *mutex* mas pode ter outro trabalho para fazer ao invés ser bloqueada se o *mutex* já estiver travado. Pelo fato de que *pthread_mutex_lock* não irá retornar até que o *mutex* se torne desbloqueado, alguma outra função é necessária.

GNU/Linux fornece *pthread_mutex_trylock* para esse propósito. Se você chamar *pthread_mutex_trylock* sobre um *mutex* destravado, você irá travar o *mutex* como se você tivesse chamado *called pthread_mutex_lock*, e *pthread_mut ex_trylock* irá retornar zero. Todavia, se o *mutex* já estiver bloqueado por outra linha de execução, *pthread_mutex_trylock* não irá bloquear a linha de execução atual. Ao invés disso, *pthread_mutex_trylock* irá retornar imediatamente com o código de erro *EBUSY*. A trava de *mutex* mantida pela outra linha de execução não é afetada. Você pode tentar mais tarde travar o *mutex*.

4.4.5 Semáforos para Linhas de Execução

No exemplo precedente, no qual muitas linhas de execução processam trabalhos a partir de um fila, a função de linha de execução principal das linhas de execução realiza o próximo trabalho até que nenhum trabalho seja esquecido e então termina a linha de execução. Esse esquema funciona se todos os trabalhos forem enfileirados previamente ou se novos trabalhos forem enfileirados tão rapidamente quanto as linhas de execução os processam. Todavia, se as linhas de execução trabalham muito rapidamente, a fila de trabalhos irá esvaziar e as linhas de execução encerraram. Se novos trabalhos forem mais tarde enfileirados, nenhuma linha de execução pode restar para processá-los. O que podemos apreciar ao invés do exposto acima é um mecanismo para bloquear as linhas de execução quando a fila esvaziar até que novos trabalhos estejam disponíveis.

Um semáforo fornece um método conveniente para fazer isso. Um semáforo é um contador que pode ser usado para sincronizar multiplas linhas de execução. Da mesma forma que com o *mutex*, GNU/Linux garante que a verificação ou a modificação do valor de um semáforo pode ser feito de forma segura, sem criar condições de corrida.

Cada semáforo tem um valor de contagem, que é um inteiro não negativo. Um semáforo suporta duas operações básicas:

- Uma operação *wait* decrementa o semáforo de 1. Se o valor já for zero, a operação bloqueia até que o valor do semáforo torne-se positivo (devido a ação de alguma outra linha de execução). Quando o valor do semáforo torna-se positivo, ele é decrementado de 1 e a operação de espera retorna.

- Uma operação *post* incrementa o valor do semáforo de 1. Se o semáforo era anteriormente zero e outras linhas de execução estão bloqueadas em uma operação *wait* sobre o atual semáforo, uma daquelas linhas de execução é desbloqueada e sua operação *wait* realiza-se (o que acarreta o retorno do valor do semáforo a zero).

Note que GNU/Linux fornece duas implementações de semáforos ligeiramente diferentes. A primeira que descrevemos aqui é a implementação de semáforos POSIX padrão. Use os semáforos POSIX quando comunicando-se entre linhas de execução. A outra implementação, usada para comunicação entre processos, é descrita na Seção 5.2, "Semáforos de Processos". Se você usa semáforos, inclua <semaphore.h>.

Um semáforo é representado por uma varável *sem_t*. Antes de usar a variável, você deve inicializá-la usando a função *sem_init*, informando um apontador para a variável *sem_t*. O segundo parâmetro deve ser zero [9], e o terceiro parâmetro é o valor inicial do semáforo. Se você não mais precisar de um semáforo, é bom liberar seus recursos com *sem_destroy*.

Para operações do tipo *wait*, use *sem_wait*. Para operações do tipo *post*, use *sem_post*. Uma função que não faz bloqueio do tipo *wait*, chamada *sem_trywait*, também é fornecida. A função *sem_trywait* é semelhante a *pthread_mutex_trylock* – se a operação do tipo *wait* puder ser bloqueada pelo fato de o valor do semáforo ser zero, a função retorna imediatamente, com o valor de erro *EAGAIN*, ao invés de efetuar o bloqueio.

GNU/Linux também fornece uma função para recuperar o valor atual de um semáforo, *sem_getvalue*, a qual coloca o valor em um apontador para uma variável do tipo *int* por meio de seu segundo argumento. Você não deve usar o valor do semáforo que você pegou dessa função para decidir fazer ou um *wait* ou um *post* sobre o semáforo, apesar disso. Usar o valor do semáforo pode levar a uma condição de corrida: Outra linha de execução pode mudar o valor do semáforo entre a chamada a *sem_getvalue* e a chamada a outra função de semáforo. Use as funções atômicas *post* e *wait* ao invés de usar o valor do semáforo.

Retomando para nosso exemplo de fila de trabalho, podemos usar um semáforo para contar o número de trabalhos esperando na fila. A Listagem

[9] Um valor diferente de zero pode indicar a semáforo que pode ser compartilhado por vários processos, o que não é suportado pelo GNU/Linux para esse tipo de semáforo.

4.12 controla a fila com um semáforo. A função *enqueue_job* adiciona um novo trabalho à fila.

Listagem 4.12: (job-queue3.c) Fila de Trabalhos Controlada por um Semáforo

```
1 #include <malloc.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4
5 struct job {
6   /* Campo encadeado para lista encadeada.  */
7   struct job* next;
8
9   /* Outros campos descrevendo trabalho a ser feito... */
10 };
11
12 /* Uma lista encadeada de trabalhos pendentes.  */
13 struct job* job_queue;
14
15 extern void process_job (struct job*);
16
17 /* Um mutex protegendo job_queue.  */
18 pthread_mutex_t job_queue_mutex = PTHREAD_MUTEX_INITIALIZER;
19
20 /* Um semaforo contando o numero de trabalhos na fila.  */
21 sem_t job_queue_count;
22
23 /* Execute de uma so vez a inicializacao da fila de trabalhos.  */
24
25 void initialize_job_queue ()
26 {
27   /* A fila esta inicialmente vazia.  */
28   job_queue = NULL;
29   /* Inicializa o semaforo no qual trabalhos sao contados na fila. Seu
30      valor inicial deve ser zero.  */
31   sem_init (&job_queue_count, 0, 0);
32 }
33
34 /* Processa trabalhos na fila ate que a fila esteja vazia.  */
35
36 void* thread_function (void* arg)
37 {
38   while (1) {
39     struct job* next_job;
40
41     /* Espera pelo semaforo da fila de trabalho.  Se seu valor for positivo,
42        indicando que a fila nao esta vazia, decremente o contador de
43        um.  Se a fila estiver vazia, bloqueie ate que um novo trabalho seja
             enfileirado.  */
44     sem_wait (&job_queue_count);
45
46     /* Trave o mutex sobre a fila de trabalho.  */
47     pthread_mutex_lock (&job_queue_mutex);
48     /* Devido ao semaforo, sabemos que a fila nao esta vazia.  Pegue
49        o trabalho disponivel seguinte.  */
50     next_job = job_queue;
51     /* Remove esse trabalho da lista.  */
52     job_queue = job_queue->next;
53     /* Desbloqueia o mutex sobre a fila de trabalho, uma vez que terminamos com a
54        fila por agora.  */
55     pthread_mutex_unlock (&job_queue_mutex);
56
57     /* Realizamos o trabalho.  */
58     process_job (next_job);
59     /* Limpamos.  */
60     free (next_job);
61   }
62   return NULL;
63 }
64
65 /* Adicione um novo trabalho na frente da fila de trabalho.  */
66
67 void enqueue_job (/* Informe dados especificos do trabalho aqui...  */)
68 {
69   struct job* new_job;
70
71   /* Aloque um novo objeto de trabalho.  */
72   new_job = (struct job*) malloc (sizeof (struct job));
73   /* Ajuste os outros campos da estrutura de trabalho aqui...  */
74
75   /* Trave o mutex sobre a fila de trabalho antes de acessar a fila.  */
76   pthread_mutex_lock (&job_queue_mutex);
77   /* Coloque o novo trabalho na cabeca da fila.  */
78   new_job->next = job_queue;
79   job_queue = new_job;
```

Listagem 4.13: (job-queue3.c) Continuação

```
80    /* Faca o post sobre o semaforo para indicar que outro trabalho esta disponivel.
         Se
81       linhas de execucao estiverem bloqueadas, esperando o semaforo, uma ira tornar-se
82       desbloqueada de forma que possa processar o trabalho.  */
83    sem_post (&job_queue_count);
84
85    /* Desbloqueia o mutex da fila de trabalho.  */
86    pthread_mutex_unlock (&job_queue_mutex);
87  }
```

Antes de pegar um trabalho da primeira posição da fila, cada linha de execução irá primeiramente realizar uma operação *wait* sobre o semáforo. Se o valor do semáforo for zero, indicando que a fila está vazia, a linha de execução será simplesmente bloqueada até que o valor do semáforo torne-se positivo, indicando que um trabalho foi adicionado à fila.

A função *enqueue_job* adiciona um trabalho à fila. Da mesma forma que *thread_function*, a função *enqueue_job* precisa travar o *mutex* da fila antes de modificar a fila. Após adicionar um trabalho à fila, a função *enqueue_job* efetua uma operação do tipo *post* no semáforo, indicando que um novo trabalho está disponível. Na versão mostrada na Listagem 4.12, as linhas de execução que atuam sobre os trabalhos nunca terminam; se não houverem trabalhos disponíveis em algum momento, todas as linhas de execução simplesmente bloqueiam em *sem_wait*.

4.4.6 Variáveis Condicionais

Mostramos como usar um *mutex* para proteger uma variável contra acessos simultâneos de duas linhas de execução e como usar semáforos para implementar um contador compartilhado. Uma variável condicional é uma terceiro dispositivo de sincronização que GNU/Linux fornece; com variáveis condicionais, você pode implementar condicionais mais complexas sob as quais linhas de execução realizam trabalhos.

Suponhamos que você escreva uma função que executa um laço infinitamente, fazendo algum trabalho a cada iteração. O laço da linha de execução , todavia, precisa ser controlado por um sinalizador: o laço executa somente quando o sinalizador está ativo; quando o sinalizador está desativado, o laço para.

A Listagem 4.14 mostra como você pode implementar a função suposta acima girando em um laço. Durante cada iteração do laço, a função de linha de execução verifica se o sinalizador está ativo. Pelo fato de o sinalizador ser acessado por várias linhas de execução, ele é protegido por um *mutex*. Essa implementação pode ser correta, mas não é eficiente. A função de linha de execução irá gastar recursos de CPU sempre que sinalizador estiver

dasativado, até que alguma circunstância possa fazer com que o sinalizador torne-se ativado.

Listagem 4.14: (spin-condvar.c) Uma Implementação Simples de Variável Condicional

```
1 #include <pthread.h>
2
3 extern void do_work ();
4
5 int thread_flag;
6 pthread_mutex_t thread_flag_mutex;
7
8 void initialize_flag ()
9 {
10   pthread_mutex_init (&thread_flag_mutex, NULL);
11   thread_flag = 0;
12 }
13
14 /* Chama do_work repetidamente enquanto o sinalizador da linha de execucao esta
       ajustado; de outra forma
15    laco.  */
16
17 void* thread_function (void* thread_arg)
18 {
19   while (1) {
20     int flag_is_set;
21
22     /* Protege o sinalizadro com uma trava de mutex.  */
23     pthread_mutex_lock (&thread_flag_mutex);
24     flag_is_set = thread_flag;
25     pthread_mutex_unlock (&thread_flag_mutex);
26
27     if (flag_is_set)
28       do_work ();
29     /* Caso contrario nao faz nada.  Apenas laco novamente.  */
30   }
31   return NULL;
32 }
33
34 /* Ajusta o valor do sinalizador da linha de execucao para FLAG_VALUE.  */
35
36 void set_thread_flag (int flag_value)
37 {
38   /* Portege o sinalizador com uma trava de mutex.  */
39   pthread_mutex_lock (&thread_flag_mutex);
40   thread_flag = flag_value;
41   pthread_mutex_unlock (&thread_flag_mutex);
42 }
```

Uma variável condicional capacita você a implementar uma condição sob a qual uma linha de execução realiza algum trabalho e, inversamente, a condição sob a qual a linha de execução é bloqueada. Enquanto toda linha de execução que potencialmente modifica o senso da condição usa a variável condicional propriamente, GNU/Linux garante que linhas de execução bloqueadas na condição irão ser desbloqueadas quando a condição mudar.

Da mesma forma que com um semáforo, uma linha de execução pode esperar por uma variável condicional. Se linha de execução A espera por uma variável condicional, a linha de execução A é bloqueada até que alguma outra linha de execução, uma linha de execução B, sinalize a mesma variável condicional. Diferentemente do semáforo, uma variável condicional não tem contador ou memória; a linha de execução A deve esperar pela variável condicional antes da linha de execução B sinalize essa mesma variável condicional

novamente. Se a linha de execução B sinaliza a variável condicional antes que a linha de execução A espere pela mesma variável condicional, o sinal é perdido, e a linha de execução A fica bloqueada até que alguma outra linha de execução sinalize a variável condicional novamente.

Adiante mostra-se como você poderia usar uma variável condicional para fazer a linha de execução acima de forma mais eficiente:

- O laço em *thread_function* verifica o sinalizador. Se o sinalizador está desativado, a linha de execução espera pela variável condicional.

- A função *set_thread_flag* sinaliza a variável condicional após modificar o valor do sinalizador. Por esse caminho, se o laço estiver bloqueado na variável condicional, irá ser desbloqueado e verificará a condicional novamente.

Existe um problema com isso: há uma condição de corrida entre verificar o valor do sinalizador e modificar seu valor ou esperar pela variável condicional. Suponhamos que *thread_function* verificou o sinalizador e encontrou-a desabilitada. Naquele momento, o GNU/Linux agendou uma pausa para aquela linha de execução e retomou a linha de execução principal. Por alguma coincidência, a linha de execução principal está em na função *set_thread_flag*. A função *set_thread_flag* ajusta o sinalizador e sinaliza a variável condicional. Pelo fato de nenhuma linha de execução estar esperando pela variável condicional naquele momento (lembre que *thread_function* estava pausada antes de poder esperar pela variável condicional), o sinal é perdido. Agora, quando GNU/Linux reagenda a outra linha de execução, ela inicia esperando pela variável condicional e pode acabar bloqueada para sempre.

Para resolver esse problema, precisamos de um caminho para travar o sinalizador e a variável condicional juntos com um *mutex* único. Afortunadamente, GNU/Linux fornece exatamente esse mecanismo. Cada variável condicional deve ser usada conjuntamente com um *mutex*, para prevenir esse tipo de condição de corrida. Usando esse esquema, a função de linha de execução segue os passos abaixo:

1. O laço em *thread_function* trava o *mutex* e lê o valor do sinalizador.

2. Se o sinalizador estiver ativado, o sinalizador ativado causa o desbloqueio do *mutex* e a execução da função de trabalho.

3. Se o sinalizador estiver desativado, o sinalizador desativado causa o desbloqueio atomicamente do *mutex* e a espera pela variável condicional.

A funcionalidade crítica aqui está no passo 3, no qual GNU/Linux permite a você destravar o *mutex* e esperar pela variável condicional atomicamente, sem a possibilidade de outra linha de execução interferir. Isso elimina a possibilidade que outra linha de execução possa modificar o valor da variável condicional entre o teste de *thread_function* do valor do sinalizador e a espera pela variável condicional.

Uma variável condicional é representada por uma instância de *pthread_cond_t*. Lembrando que cada variável condicional deve ser acompanhada de um *mutex*. Abaixo temos as funções que controlam variáveis condicionais:

- *pthread_cond_init* inicializa uma variável condicional. O primeiro argumento é um apontador para a instância *pthread_cond_t*. O segundo argumento, um apontador para uma objeto de atributo de variável condicional , o qual é ignorado em GNU/Linux. O *mutex* deve ser inicializado separadamente, como descrito na Seção 4.4.2, *"Mutexes"*.

- *pthread_cond_signal* sinaliza uma variável condicional. Uma linha de execução única, que é bloqueada conforme o estado da variável condicional, irá ser desbloqueada. Se nenhuma outra linha de execução estiver bloqueada conforme a variável de condição, o sinal é ignorado. O argumento é um apontador para a instância *pthread_cond_t*.

  Uma chamada similar, *pthread_cond_broadcast*, desbloqueia todos as linhas de execução que estiverem bloqueadas conforme a variável condicional, ao invés de apenas uma.

- *pthread_cond_wait* bloqueia a linha de execução que a está chamado até que a variável de condição for sinalizada. O argumento é um apontador par a instância *pthread_cond_t*. O segundo argumento é um apontador para instância de mutex *pthread_mutex_t*. Quando *pthread_cond_wait* for chamada, o *mutex* deve já estar travado por meio da linha de execução que o chamou. A função *pthread_cond_wait* atomicamente desbloqueia o *mutex* e bloqueia sob a variável de condição. Quando a variável de condição seja sinalizada e a linha de execução que chamou desbloquear, *pthread_cond_wait* automaticamente readquire uma trava sob o *mutex*.

Sempre que seu programa executar uma ação que pode modificar o senso da condição você está protegendo com a variável condicional, seu programa deve executar os passos adiante. (No nosso exemplo, a condição é o estado

do sinalizador da linha de execução, de forma que esses passos devem ser executados sempre que o sinalizador for modificado.)

1. Travar o *mutex* que acompanha a variável condicional.
2. Executar a ação que pode mudar o senso da condição (no nosso exemplo, ajustar o sinalizador).
3. Sinalizar ou transmitir a variável condicional, dependendo do comportamento desejado.
4. Desbloquear o *mutex* acompanhando a variável condicional.

A Listagem 4.15 mostra o exemplo anterior novamente, agora usando uma variável condicional para proteger o sinalizador da linha de execução. Note que na função *thread_function*, uma trava sob o *mutex* é mantida antes de verificar o valor de *thread_flag*. Aquela trava é automaticamente liberada por *pthread_cond_wait* antes de bloquear e é automaticamente readquirida posteriormente. Também note que *set_thread_flag* trava o *mutex* antes de ajustar o valor de *thread_flag* e sinalizar o *mutex*.

Listagem 4.15: (condvar.c) Controla uma Linha de Execução Usando uma Variável Condicional

```
1 #include <pthread.h>
2
3 extern void do_work ();
4
5 int thread_flag;
6 pthread_cond_t thread_flag_cv;
7 pthread_mutex_t thread_flag_mutex;
8
9 void initialize_flag ()
10 {
11   /* Inicializa o mutex e a variavel de condicao.  */
12   pthread_mutex_init (&thread_flag_mutex, NULL);
13   pthread_cond_init (&thread_flag_cv, NULL);
14   /* Inicializa o valor do sinalizador.  */
15   thread_flag = 0;
16 }
17
18 /* Chama do_work repetidamente enquanto o sinalizador da linha de execucao e ajustada
       ; bloqueia se
19    o sinalizadro esta limpo.  */
20
21 void* thread_function (void* thread_arg)
22 {
23   /* Laco infinitamente.  */
24   while (1) {
25     /* trava o mutex antes de acessar o valor do sinalizador.  */
26     pthread_mutex_lock (&thread_flag_mutex);
27     while (!thread_flag)
28       /* O sinalizador e limpo.  Espera por um sinal sobre a variavel de
29          condicao, indicando que o valor do sinalizador mudou.  Quando o
30          sinal chega e sua linha de execucao desbloqueia, laco e verificacao do
31          sinalizador novamente.  */
32       pthread_cond_wait (&thread_flag_cv, &thread_flag_mutex);
33     /* Quando tivermos aqui, sabemos que o sinalizador foi ajustado.  Destrava o
34        o mutex.  */
35     pthread_mutex_unlock (&thread_flag_mutex);
36     /* Faz algum trabalho.  */
37     do_work ();
38   }
39   return NULL;
40 }
41
42 /* Ajusta o valor do sinalizador da linha de execucao para FLAG_VALUE.  */
43
44 void set_thread_flag (int flag_value)
45 {
46   /* Trava o mutex antes de acessar o valor do sinalizador.  */
47   pthread_mutex_lock (&thread_flag_mutex);
48   /* Ajusta o valor do sinalizador, e entao o sinal no caso da thread_function estar
49      bloqueada, espere pelo sinalizador tornar-se ajustado.  Todavia,
50      thread_function nao pode atualmente verificar o sinalizador ate que o mutex
           estar
51      desbloqueado.  */
52   thread_flag = flag_value;
53   pthread_cond_signal (&thread_flag_cv);
54   /* Desbloqueia o mutex.  */
55   pthread_mutex_unlock (&thread_flag_mutex);
56 }
```

A condição protegida pela variável condicional pode ser arbitrariamente complexa. Todavia, antes de executar qualquer operação que possa mudar o senso da condição, uma trava de *mutex* deve ser requerida, e a variável condicional deve ser sinalizada depois.

Uma variável condicional pode também ser usada sem uma condição, simplesmente como um mecanismo para bloquear uma linha de execução até que outra linha de execução "acorde-a". Um sinalizador pode também ser usado para aquele propósito. A principal diferença é que um sinalizador "lembra" o chamada para acordar mesmo se nenhuma linha de execução

tiver bloqueada sobre ele naquela ocasião, enquanto uma variável condicional discarta a chamada para acordar a menos que alguma linha de execução esteja atualmente bloqueada sob essa mesam variável condicional naquela ocasião. Também, um sinalizador entrega somente um único acorde por *post*; com *pthread_cond_broadcast*, um número arbitrário e desconhecido de linhas de execução bloqueadas pode ser acordado na mesma ocasião.

4.4.7 Travas Mortas com Duas ou Mais Linhas de Execução

Travas mortas podem ocorrer quando duas (ou mais) linhas de execução estiverem bloqueadas, esperando que uma condição ocorra e que somente outra das duas (ou mais) pode fazer acontecer. Por exemplo, se uma linha de execução A está bloqueada sob uma variável condicional esperando pela linha de execução B sinalize a variável condicional, e a linha de execução B está bloqueada sob uma variável de condição esperando que a linha de execução A sinalize essa mesma variável de condição, uma trava morta ocorreu pelo fato de que nenhuma das linhas de execução envolvidas irá sinalizar para a outrar. Você deve evitar a todo custo a possibilidade de tais stuações pelo fato de elas serem bastante difíceis de detectar.

Um erro comum que causa uma trava morta envolve um problema no qual mais de uma linha de execução está tentando travar o mesmo conjunto de objetos. Por exemplo, considere um programa no qual duas diferentes linhas de execução, executando duas diferentes funções de linha de execução, precisam travar os mesmos dois *mutexes*. Suponhamos que a linha de execução A trave o *mutex* 1 e a seguir o *mutex* 2, e a linha de execução B precise travar o *mutex* 2 antes do *mutex* 1. Em um suficientemente desafortunado cenário de agendamento, GNU/Linux pode agendar a linha de execução A por um tempo suficiente para travar o *mutex* 1, e então agende a linha de execução B, que prontamente trava *mutex* 2. Agora nenhuma linha de execução pode progredir pelo fato de cada uma estar bloqueada sob um *mutex* que a outra linha de execução mantém bloqueada.

Acima temos um exemplo de um problema genérico de trava morta, que pode envolver não somente sincronização de objetos tais como *mutexes*, mas também outros recursos, tais como travas sob arquivos ou dispositivos. O problema ocorre quando multiplas linhas de execução tentam travar o mesmo conjunto de recursos em diferentes ordens. A solução é garantir que todas as linhas de execução que travam mais de um recurso façam também o travamento desses recursos na mesma ordem.

4.5 Implementação de uma Linha de Execução em GNU/Linux

A implementação de linhas de execução *POSIX* em GNU/Linux difere da implementação de linha de execução de muitos outros sistemas semelhantes ao *UNIX* em um importante caminho: no GNU/Linux, linhas de execução são implementadas como processos. Sempre que você chamar *pthread_create* para criar uma nova linha de execução, GNU/Linux cria um novo processo que executa aquela linha de execução. Todavia, esse processo não é o mesmo que o processo criado com *fork*; particularmente, o processo criado com *pthread_create* compartilha o mesmo espaço de endereço e recursos que o processo original em lugar de receber cópias.

O programa *thread-pid* mostrado na Listagem 4.16 demonstra isso. O programa cria uma linha de execução; ambas a nova linha de execução e a original chamam a função *getpid* e imprimem seus respectivos IDs de processo e então giram infinitamente.

Listagem 4.16: (thread-pid) Imprime IDs de processos para Linhas de Execução

```
1  #include <pthread.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5  void* thread_function (void* arg)
6  {
7    fprintf (stderr, "pid da linha de execucao filha eh %d\n", (int) getpid ());
8    /* Ciclo infinito.  */
9    while (1);
10   return NULL;
11 }
12
13 int main ()
14 {
15   pthread_t thread;
16   fprintf (stderr, "pid da linha de execucao principal eh %d\n", (int) getpid ());
17   pthread_create (&thread, NULL, &thread_function, NULL);
18   /* Ciclo infinito.  */
19   while (1);
20   return 0;
21 }
```

Execute o programa em segundo plano, e então chame *ps x* para mostrar seus processos executando. Lembre-se de matar o programa *thread-pid* depois – o mesmo consome muito da CPU sem fazer absolutamente nada. Aqui está como a saída do *ps x* pode parecer:

```
% cc thread-pid.c -o thread-pid -lpthread
% ./thread-pid \&
[1] 14608
main thread pid is 14608
child thread pid is 14610
```

```
\% ps x
  PID TTY      STAT   TIME COMMAND
14042 pts/9    S      0:00 bash
14608 pts/9    R      0:01 ./thread-pid
14609 pts/9   S             0:00 ./thread-pid
14610 pts/9   R             0:01 ./thread-pid
14611 pts/9   R             0:00 ps x
\% kill 14608
[1]+ Terminated       ./thread-pid
```

Notificação de Controle de Trabalho no *Shell*
As linhas iniciam-se com [1] são do *shell*. Quando você executa um programa em segundo plano, o *shell* atribui um número de trabalho para ele – nesse caso, 1 – e imprime o pid do programa. Se o trabalho em segundo plano encerra-se, o *shell* mostra esse fato da próxima vez que você chamar um comando.

Chamo a atenção para o fato de que existem três processos executando o programa *thread-pid*. O primeiro desses, com o pid 14608, é a linha de execução principal no programa; o terceiro, com pid 14610, é a linha de execução que criamos para executar *thread_function*.

O que dizer da segunda linha de execução, com pid 14609? Essa é a "linha de execução gerente" que é parte da implementação interna de linhas de execução em GNU/Linux. A linha de execução gerente é criada na primeira vez que um programa chama *pthread_create* para criar uma nova linha de execução.

4.5.1 Controlando Sinais

Suponhamos que um programa com várias linhas de execução receba um sinal. Em qual linha de execução das linhas de execução multiplas deve ser chamado o controlador para esse sinal? O comportamento da interação entre sinais e linhas de execução varia de entre os diversos sistemas operacionais semelhantes ao *UNIX*. Em GNU/Linux, o comportamento é ditado pelo fato de que as linhas de execução são implementadas como processos.

Pelo fato de cada linha de execução ser um processo separado, e pelo fato de um sinal ser entregue para um processo em particular, não existe ambiguidade sobre qual linha de execução recebe o sinal. Tipicamente, sinais enviados de fora do programa são enviados para o processo correspondente à linha de execução principal do programa. Por exemplo, se um programa executa *forks* e o processo filho faz *execs* sobre um programa com várias linhas de execução, o processo pai irá manter o ID de processo da linha de execução principal do programa do processo filho e irá usar aquele ID de

processo para enviar sinais para seu filho. Esse comportamento é geralmente uma boa convenção a seguir por você mesmo quando enviar sinais para um programa com várias linhas de execução.

Note que esse aspecto da implementação em GNU/Linux das linhas de execução é uma variância da linha de execução *POSIX* padrão. Não confie nesse comportamento em programas que são significativamente para serem portáveis.

Dentro de um programa com várias linhas de execução, é possível para uma linha de execução enviar um sinal especificamente para outra linha de execução. Use a função *pthread_kill* para fazer isso. O primeiro parâmetro é um ID de linha de execução, e seu segundo parâmetro é um número de sinal.

4.5.2 Chamada de Sistema *clone*

Embora linhas de execução em GNU/Linux criadas em um mesmo programa sejam implementadas como processos separados, eles compartilham seu espaço virtual de memória e outros recursos. Um processo filho criado com uma operação *fork*, todavia, recebe cópias desses itens. Como personalizar o processo criado?

A chamada de sistema GNU/Linux *clone* é uma forma generalizada de *fork* e de *pthread_create* que permite a quem está chamando especificar quais recursos são compartilhados entre o processo que está chamando e o processo criado recentemente. Também, *clone* requer que você especifique a região de memória para a pilha de execução que o novo processo irá usar. Embora mencionemos *clone* aqui para satisfazer a curiosidade do leitor, essa chamada de sistema não deve frequentemente ser usada em programas.

Use *fork* para criar novos processos ou *pthread_create* para criar linhas de execução.

4.6 Processos Vs. Linhas de Execução

Para alguns programas que se beneficiam da concorrência, a decisão entre usar processos ou linhas de execução pode ser difícil. Aqui estão algumas linhas guias para ajudar você a decidir qual modelo de concorrência melhor se ajusta ao seu programa:

- Todas as linhas de execução em um programa devem rodar o mesmo executável. Um processo filho, por outro lado, pode rodar um executável diferente através da função *exec*.

- Uma linha de execução errante pode prejudicar outras linhas de execução no mesmo processo pelo fato de linhas de execução compartilharem o mesmo espaço de memória virtual e outros recursos. Por exemplo, uma bárbara escrita na memória por meio de um ponteiro não inicializado em uma linha de execução pode corromper a memória visível para outra linha de execução. Um processo errante, por outro lado, não pode fazer isso pelo fato de cada processo ter uma cópia do espaço de memória do programa.

- A cópia de memória para um novo processo cria um trabalho adicional diminuindo a performace em comparação à criação de uma nova linha de execução. Todavia, a cópia é executada somente quando a memória é modificada, de forma que o penalti é minimo se o processo filho somente lê a memória.

- Linhas de Execução podem ser usadas por programas que precisam de paralelismo fino e granulado. Por exemplo, se um problema pode ser quebrado em multiplos trabalhos aproximamente identicos, linhas de execução podem ser uma boa escolha. Processos podem ser usados por programas que precisam de paralelismo rude.

- Compartilhando dados em torno de linhas de execução é trivial pelo fato de linhas de execução compartilharem a mesma memória (Todavia, grande cuidado deve ser tomado para evitar condições de corrida, como descrito anteriormente). Compartilhando dados em torno de processos requer o uso de mecanismos IPC [a], como descrito no Capítulo 5. Compartilhar dados em torno de processos pode ser incômodo mas faz multiplos processos parecer menos com navegar em erros de concorrência.

[a] Nota do tradutor:Comunicação Entre Processos.

Capítulo 5

Comunicação Entre Processos

NO CAPÍTULO 3,"PROCESSOS" FOI DISCUTIDO A CRIAÇÃO DE PROCESSOS e mostrado como um processo pode obter a situação de saída de um processo filho. Essa é a forma mais simples de comunicação entre dois processos, mas isso não significa que seja o mais poderoso. Os mecanismos do Capítulo 3 não fornecem nenhum caminhos para que o processo pai comunique-se com o processo filho a não ser através de argumentos de linha de comando e de variáveis de ambiente, nem fornece também qualquer caminho para o processo filho comunicar-se com o processo pai a não ser através da situação de saída do processo filho. Nenhum desses mecanismos fornece quaisquer meios para comunicação com o processo filho enquanto ele estiver executando, nem faz esses mecanismos permitir comunicação com um processo fora do relacionamento pai-filho.

Esse capítulo descreve meios para comunicação entre processos que contornam as limitações descritas acima. Apresentaremos vários caminho para comunicação entre pais e filhos, entre processos "desaparentados", e mesmo entre processos em diferentes máquinas.

Comunicação entre processos (IPC)[1] é a transferência de dados em meio a processos. Por exemplo, um navegador Web pode requisitar uma página Web de um servidor Web, que então envia dados no formato HTML. Essa transferência de dados comumente usa *sockets* em uma conecção semelhante às conecções telefônicas. Em outro exemplo, você pode desejar imprimir os nomes de arquivos em um diretório usando um comando tal como *ls* | *lpr*. O *shell* cria um processo *ls* e um processo *lpr* separado, conectando os dois com um *pipe*, representado pelo símbolo "|". Um *pipe* permite comunicação de mão única entre dois processos relacionados. O processo *ls* envia dados para o *pipe*, e o processo *lpr* lê dados a partir do *pipe*.

[1]Nota do tradutor:a tradução da sigla não é adequada nesse caso - CEP.

No presente capítulo, discutiremos cinco tipos de comunicação entre processos:

- Memória compartilhada - permite que processos comuniquem-se simplesmente lendo e escrevendo para uma localização de memória especificada.
- Memória mapeada - é similar à memória compartilhada, execeto que a memória mapeada está associada com um arquivo no sistema de arquivos.
- *Pipes* - permite comunicação sequêncial de um processo para um outro processo seu parente.
- *FIFO*s - são similares a *pipes*, exceto que processos não aparentados podem comunicar-se pelo fato de ao *pipe* ser fornecido um nome no sistema de arquivos.
- *Sockets* - suporta comunicação entre processos não aparentados mesmo em computadores diferentes.

Esses tipos de IPC diferem pelos seguintes critérios:

- Se a comunicação é restrita de processos aparentados (processos com um ancestral comum) com processos não aparentados compartilhando o mesmo sistema de arquivos ou com qualquer computador conectado a uma rede
- Se um processo de comunicação é limitado a somente escrita ou somente leitura de dados
- O número de processo permitidos para comunicar-se
- Se os processos de comunicação são sincronizados através de IPC – por exemplo, um processo de leitura pára até que dados estejam disponíveis para leitura

Nesse capítulo, omitiremos considerações acerca de IPC permitindo comunicações somente por um limitado número de vezes, tais como comunicação através de um valor de saída de processo filho.

5.1 Memória Compartilhada

Um dos mais simples métodos de comunicação entre processos é o uso de memória compartilhada. Memória compartilhada permite a dois ou mais

processos acessarem a mesma memória como se todos eles tivessem chamado *malloc* e tivessem obtido, como valor de retorno, apontadores para a mesma área de memória em uso atualmente. Quando um processo modifica a memória, todos os outros processos veem a modificação.

5.1.1 Comunicação Local Rápida

Memória compartilhada é a forma mais rápida de comunicação entre processos pelo fato de todos os processos compartilharem a mesma peça de memória. O acesso a essa memória compartilhada é tão rápido quanto o acesso a memória não compartilhada de processos, e não requer uma chamada de sistema ou entrada para o kernel. A comunicação usando memória compartilhada também evita cópias desnecessárias de informações.

Pelo fato de o kernel não sincronizar acessos à memória compartilhada, você deve fornecer sua própria sincronização. Por exemplo, um processo não deve ler a memória somente após dados serem escritos nela, e dois processos não devem escrever na mesma localização de memória ao mesmo tempo. Uma estratégia comum para evitar essas condições de corrida é usar-se semáforos, que serão discutidos na próxima seção. Nossos programas ilustrativos, apesar disso, mostram apenas um único processo acessando a memória, para evidenciar o mecanismo de memória compartilhada e para evitar um amontoado a amostra de código com sincronização lógica.

5.1.2 O Modelo de Memória

Para usar um segmento de memória compartilhada, um processo deve alocar o segmento. Então cada processo desejando acessar o segmento deve anexar esse mesmo segmento. Após terminar seu uso do segmento, cada processo desanexa o segmento. Em algum ponto, um processo deve desalocar o segmento.

Entendendo o modelo de memória do GNU/Linux ajuda a explicação do mecanismo de alocação e anexação. Sob GNU/Linux, cada memória virtual usada por um processo é quebrada em páginas. Cada processo mantém um mapeamento de seus endereços de memória para essas páginas de memória virtual, as quais carregam os dados atuais. Além disso cada processo tem seus próprio endereços, mapeamentos de multiplos processos podem apontar para a mesma página, permitindo compartilhameto de memória. Páginas de memória são adicionalmente discutidas na Seção 8.8, "A Família *mlock*: Travando Memória Física" do Capítulo 8, "Chamadas de Sistema do GNU/Linux."

A alocação de um novo segmento de memória compartilhada faz com que páginas de memória virtual sejam criadas. Pelo fato de todos os proces-

sos desejarem acessar o mesmo segmento compartilhado, somente um processo deve alocar um novo segmento compartilhado. A alocação de um segmento existente não cria novas páginas, mas irá retornar um identificador para as páginas existentes. Para permitir a um processo usar o segmento de memória compartilhado, um processo anexa-o, o que adiciona entradas mapeando de sua memória virtual para as páginas compartilhadas do segmento. Quando termina com o segmento, essas entradas de mapeamento são removidas. Quando nenhum processo deseja acessar esses segmentos de memória compartilhada, exatamente um processo deve desalocar as páginas de memória virtual.

Todos segmentos de memória compartilhada são alocados como multiplos inteiros do tamanho de página do sistema, que é o número de ocupado por uma página de memória. Sob sistemas GNU/Linux, o tamanho da página é 4KB, mas você pode obter esse valor chamando a função *getpagesize*.

5.1.3 Alocação

Um processo aloca um segmento de memória compartilhada usando *shmget* ("*SHared Memory GET*"). O primeiro parâmetro a *shmget* é uma chave inteira que especifica qual o segmento a ser criado. Processos não aparentados podem acessar o mesmo segmento compartilhado especificando o mesmo valor de chave inteira. Desafortunadamente, outros processos podem ter também escolhido a mesma chave fixada, o que pode levar a conflitos. Usando a constante especial *IPC_PRIVATE* como local de armazenamento da chave garante que um segmento de memória marcado como novo seja criado.

O segundo parâmetro a *shmget* especifica o número de *bytes* no segmento. Pelo fato de segmentos serem alocados usando páginas, o número de *bytes* alocados atualmente é arredondado para cima para um inteiro multiplo do tamanho da página.

O terceiro parâmetro a *shmget* é o conjunto de valores de *bits* ou de sinalizadores que especificam opções a *shmget*.

Os valores de sinalizadores incluem os seguintes:

- *IPC_CREAT* – Esse sinalizador indica que um novo segmeto deve ser criado. Permite a criação de um novo segmento na mesma hora em que especifica um valor de chave.

- *IPC_EXCL* – Esse sinalizador, que é sempre usado com IPC_CREAT, faz com que *shmget* falhe se uma chave de segmento que já exista for especificada. Portanto, *IPC_EXCL* possibilita ao processo que está chamando ter um segmento "exclusivo". Se esse sinalizador não for fornecido e a chave de um segmento existente for usada, *shmget* retorna o segmento existente ao invés de criar um novo.

- Sinalizadores de modo – Esse valor é composto de 9 bits indicando permissões garantidas ao dono, grupo e o restante do mundo para controlar o acesso ao segmento. Bits de execução são ignorados. Um caminho fácil para especificar permissões é usar constantes definidas no arquivo de cabeçalho <sys/stat.h> e documentadas na seção 2 da página de manual de *stat*[a]. Por exemplo, *S_IRUSR* e *S_IWUSR* especificam permissões de leitura e escrita para o dono do segmento de memória compartilhada, e *S_IROTH* e *S_IWOTH* especificam permissões de leitura e escrita para outros.

[a]Esses bits de permissão são os mesmos aqueles usados para arquivos. Eles são descritos na Seção 10.3, "Permissões do Sistema de Arquivos".

Por exemplo, a chamada adiante a *shmget* cria um novo segmento de memória compartilhada (ou acessa um que já existe, se *shm_key* já estiver sendo usada) que pode ser lido e escrito pelo dono mas não por outros usuários.

```
int segment\_id = shmget (shm\_key, getpagesize (), IPC\_CREAT | S\_IRUSR | S\_IWUSR);
```

Se a chamada obtiver sucesso,*shmget* retorna um identificador de segmento. Se o segmento de memória compartilhada já existir, as permissões de acesso são verificadas e uma confirmação é feita para garantir que o segmento não seja marcado para destruição.

5.1.4 Anexando e Desanexando

Para tornar o segmento de memória compartilhada disponível, um processo deve usar *shmat*, "*SHared Memory ATtach*". Informe a *shmat* o identificador de segmento de memória compartilhada *SHMID* retornado por *shmget*. O segundo argumento é um apontador que especifica onde no seu espaço de endereçamento de processo você deseja mapear a memória compartilhada; se

você especificar *NULL*, GNU/Linux irá escolher um endereço disponível. O terceiro argumento é um sinalizador, que pode incluir o seguinte:

- *SHM_RND* indica que o endereço especificado para o segundo parâmetro deve ser arredondado por baixo para um multiplo do tamanho da página de memória. Se você não especificar esse sinalizador, você deve ajustar conforme o tamanho da página o segundo argumento para *shmat* por si mesmo.
- *SHM_RDONLY* indica que o segmento irá ser somente para leitura, não para escrita.

Se a chamada obtiver sucesso, a chamada irá retornar o endereço do segmento compartilhado anexado. Processos filhos criados por chamadas a *fork* herdarão os segmentos de memória compartilhada anexados; eles podem desanexar os segmentos de memória anexados, se assim o desejarem.

Quando você tiver terminado com um segmento de memória compartilhada, o segmento deve ser liberado usando *shmdt* ("*SHared Memory DeTach*"). Informe a *shmdt* o endereço retornado por *shmat*. Se o segmento tiver sido desalocado e o processo atual for o último processo usando o segmento de memória em questão, esse segmento é removido. Chamadas a *exit* e a qualquer chamada da família *exec* automaticamente desanexam segmentos.

5.1.5 Controlando e Desalocando Memória Compartilhada

A chamada *shmctl* ("*SHared Memory ConTroL*") retorna informações sobre um segmento de memória compartilhada e pode modificar o referido segmento. O primeiro parâmetro é um identificador de segmento de memória compartilhada.

Para obter informações sobreu um segmento de memória compartilhada, informe *IPC_STAT* como o segundo argumento e um apontador para uma variável do tipo *struct* chamada *shmid_ds*.

Para remover um segmento, informe *IPC_RMID* como o segundo argumento, e informe *NULL* como o terceiro argumento. O segmento é removido quando o último processo que o tiver anexado finalmente o desanexe.

Cada segmento de memória compartilhada deve ser explicitamente desalocado usando *shmctl* quando você tiver acabado com esse mesmo segmento, para evitar violação um limite de tamanho interno ao GNU/Linux [2] com

[2]Nota do tradutor:*system-wide limit* conjunto de limites respeitado pelo *kernel* para proteger o sistema. Os limites são aplicados na quantidade de arquivos aberto por processo,

relação ao número total de segmentos de memória compartilhada. Chamadas a *exit* e *exec* desanexam segmentos de memória mas não os desalocam.

Veja a página de manual para *shmctl* para uma descrição de outras operações que você pode executar sobre segmentos de memória compartilhada.

5.1.6 Um programa Exemplo

O programa na Listagem 5.1 ilustra o uso de memória compartilhada.

Listagem 5.1: Exercício de Memória Compartilhada

```
1  #include <stdio.h>
2  #include <sys/shm.h>
3  #include <sys/stat.h>
4
5  int main ()
6  {
7    int segment_id;
8    char* shared_memory;
9    struct shmid_ds shmbuffer;
10   int segment_size;
11   const int shared_segment_size = 0x6400;
12
13   /* Aloca um segmento de mem ria compartilhada.  */
14   segment_id = shmget (IPC_PRIVATE, shared_segment_size,
15                        IPC_CREAT | IPC_EXCL | S_IRUSR | S_IWUSR);
16
17   /* Anexa o segmento de mem ria compartilhada.  */
18   shared_memory = (char*) shmat (segment_id, 0, 0);
19   printf ("mem ria compartilhada anexada no endere o %p\n", shared_memory);
20   /* Determina o tamanho do segmento.  */
21   shmctl (segment_id, IPC_STAT, &shmbuffer);
22   segment_size = shmbuffer.shm_segsz;
23   printf ("tamanho do segmento: %d\n", segment_size);
24   /* Escreve uma sequ ncia de caracteres para o segmento de mem ria compartilhada.
        */
25   sprintf (shared_memory, "Al , mundo.");
26   /* Remove a anexa  o do segmento de mem ria compartilhada.  */
27   shmdt (shared_memory);
28
29   /* Reanexa o segmento de mem ria compartilhada, em um endere o diferente.  */
30   shared_memory = (char*) shmat (segment_id, (void*) 0x5000000, 0);
31   printf ("mem ria compartilhada no endere o %p\n", shared_memory);
32   /* Mostra a sequ ncia de caracteres a partir da mem ria compartilhada.  */
33   printf ("%s\n", shared_memory);
34   /* Remove a anexa  o do segmento de mem ria compartilhada.  */
35   shmdt (shared_memory);
36
37   /* Desaloca o segmento de mem ria compartilhada.  */
38   shmctl (segment_id, IPC_RMID, 0);
39
40   return 0;
41 }
```

5.1.7 Depurando

Os comandos ipc fornecem informação sobre as facilidade da comunicação entre processos, incluindo segmentos compartilhados. Use o sinalizador -m para obter informação sobre memória compartilhada. Por exemplo, o código

no tamanho de alguma mensagem do sistema, na quantidade de arquivos em uma fila, etc. São obtidos com o comando *sysctl -a* em um slackware por exemplo.

a seguir ilustra que um segmento de memória compartilhada, cujo número é 1627649, está em uso:

```
% ipcs -m

------ Shared Memory Segments --------
key        shmid     owner     perms     bytes        nattch   status
0x00000000 1627649   user      640       25600        0
```

Se esse segmento de memória tiver sido errôneamente deixado para trás por um programa, você pode usar o comando *ipcrm* para removê-lo.

```
% ipcrm shm 1627649
```

5.1.8 Prós e Contras

Segmentos de memória compartilhada permitem comunicação bidirecional rápida envolvendo qualquer número de processos. Cada usuário pode tanto ler quanto escrever, mas um programa deve estabelecer e seguir algum protocolo para prevenir condições de corrida tais como sobrescrever informação antes que essa mesma informação seja lida. Desafortunadamente, GNU/Linux não garante estritamente acesso exclusivo mesmo se você criar um novo segmnto compartilhado com *IPC_PRIVATE*.

Também, para multiplos processos usarem um segmento compartilhado, eles devem fazer arranjos para usar a mesma chave.

5.2 Semáforos de Processos

Como se nota na seção anterior, processos devem ter acesso coordenado à memória compartilhada. Como discutimos na Seção 4.4.5, "Semáforos para Linhas de Execução" no Capítulo 4, "Linhas de Execução" semáforos são contadores que permitem sincronizar multiplas linhas de execução. GNU/Linux fornece uma implementação alternativa diferente de semáforos que pode ser usada para sincronizar processos (chamada semáforos de processo ou algumas vezes semáforos *System V*). Se máforos de processo são alocados, usados, e desalocados como segmentos de memória compartilhada. Embora um único semáforo seja suficiente para a maioria dos usos, semáforos de processo veem em conjuntos. ao longo de toda essa seção, apresentamos chamadas de sistema para semáforos de processo, mostrando como implementar semáforos binários simples usando essas chamadas de sistema.

5.2.1 Alocação e Desalocação

As chamadas *semget* e *semctl* alocam e desalocam semáforos, ambas análogas a *shmget* e *shmctl* para memória compartilhada. Chame *semget* com uma chave especificando um conjunto de semáforo, o número de semáforos no conjunto, e sinalizadores de permissão da mesma forma que para *shmget*; o valor de retorno é um identificador do conjunto de semáforo. Você pode obter o identificador de um conjunto de semáforo existente especificando o valor da chave respectiva; nesse caso, o número de semáforos pode ser zero.

Semáforos continuam a existir mesmo após todos os processos que os tiverem usado tenham terminado. O último processo a usar um conjunto de semáforo deve explicitamente remover o conjunto de forma a garantir que o sistema operacional não desperdice semáforos. Para fazer isso, chame *semctl* com o identificador de semáforo, o número de semáforos no conjunto, *IPC_RMID* como o terceiro argumento, e qualquer valor de *union semun*[3] como o quarto argumento (que é ignorado). O identificador efetivo do usuário do processo que está chamando deve coincidir com o do alocador do semáforo (ou o chamador deve ser o superusuário). Ao contrário do que ocorre com segmentos de memória compartilhada, a remoção de um conjunto de semáforo faz com que GNU/Linux o desaloque imediatamente.

A Listagem 5.2 mostra funções para alocar e desalocar um semáforo binário.

Listagem 5.2: (sem_all_deall.c) Alocando e Desalocando um semáforo Binário

```
1 #include <sys/ipc.h>
2 #include <sys/sem.h>
3 #include <sys/types.h>
4
5 /* Devemos definir union semun por nossa conta.  */
6
7 union semun {
8   int val;
9   struct semid_ds *buf;
10  unsigned short int *array;
11  struct seminfo *__buf;
12 };
13
14 /* Obt m um ID sem foro bin rio , alocando se necess rio.  */
15
16 int binary_semaphore_allocation (key_t key, int sem_flags)
17 {
18   return semget (key, 1, sem_flags);
19 }
20
21 /* Desaloca um sem foro bin rio.  Todos os usu rios devem ter terminado seu
22    uso.  Retorna -1 em caso de falha.  */
23
24 int binary_semaphore_deallocate (int semid)
25 {
26   union semun ignored_argument;
27   return semctl (semid, 1, IPC_RMID, ignored_argument);
28 }
```

[3]Nota do tradutor: definido em sem.h.

5.2.2 Inicializando Semáforos

Alocação e inicialização são duas operações distintas. Para inicializar um semáforo, use *semctl* com zero como o segundo argumento e *SETALL* como o terceiro argumento. Para quarto argumento, você deve criar um objeto *union semun* e apontar seu campo array para um array de valores inteiros curtos. Cada valor é usado para inicializar um semáforo no conjunto.

A Listagem 5.3 mostra uma função que inicializa um semáforo binário.

Listagem 5.3: (sem_init.c) Inicializando um Semáforo Binário

```
1 #include <sys/types.h>
2 #include <sys/ipc.h>
3 #include <sys/sem.h>
4
5 /* Devemos definir union semun por nossa conta.  */
6
7 union semun {
8   int val;
9   struct semid_ds *buf;
10  unsigned short int *array;
11  struct seminfo *__buf;
12 };
13
14 /* Inicializa um sem foro  bin rio com o valor de um.  */
15
16 int binary_semaphore_initialize (int semid)
17 {
18   union semun argument;
19   unsigned short values[1];
20   values[0] = 1;
21   argument.array = values;
22   return semctl (semid, 0, SETALL, argument);
23 }
```

5.2.3 Operações *Wait* e *Post*

Cada semáforo tem um valor não negativo e suporta operações *wait* e *post*. A chamada de sistema *semop* implementa ambas as operações. Seu primeiro parâmetro especifica um identificador de conjunto de semáforo. Seu segundo parâmetro é um array de elementos do tipo *struct sembuf*, que especifica as operações que você deseja executar. O terceiro parâmetro é o comprimento desse array.

Os campos de *struct sembuf* são listados aqui:

- *sem_num* é o número do semáforo no conjunto de semáforo sobre o qual a operação é executada.

- *sem_op* é um inteiro que especifica a operação do semáforo.

  Se *sem_op* for um número positivo, esse número positivo é adicionado ao valor do semáforo Imediatamente.

  Se *sem_op* for um número negativo, o valor absoluto do número negativo é subtraído do valor do semáforo. Se isso fizer com que o valor de semáforo torne-se negativo, a chamada bloqueia até que o valor de semáforo torne-se tão grande quanto o valor absoluto de *sem_op* (pelo fato de algum outro processo incrementar esse valor).

  Se *sem_op* for zero, a operação bloqueia até que o valor do semáforo torne-se zero.

- *sem_flg* é um valor de sinalizador. Especifique *IPC_NOWAIT* para prevenir a operação de bloquear; se a operação puder ter bloqueio, a chamada a *semop* falha ao invés disso. Se você especificar *SEM_UNDO*, GNU/Linux automaticamente desmancha a operação sobre o semáforo quando o processo encerra.

A Listagem 5.4 ilustra operações *wait* e *post* para um semáforo binário.

Listagem 5.4: (sem_pv.c) Operações *Wait* e *Post* para um Semáforo Binário

```
1 #include <sys/types.h>
2 #include <sys/ipc.h>
3 #include <sys/sem.h>
4
5 /* Espera por um sem foro bin rio.  Bloqueia at  que o valor do sem foro seja
6    positivo, ent o decrementa esse sem foro de uma unidade.  */
7
8 int binary_semaphore_wait (int semid)
9 {
10   struct sembuf operations[1];
11   /* Usa o primeiro (e  nico) sem foro.  */
12   operations[0].sem_num = 0;
13   /* Decrementa de 1.  */
14   operations[0].sem_op = -1;
15   /* Permite desfazer.  */
16   operations[0].sem_flg = SEM_UNDO;
17
18   return semop (semid, operations, 1);
19 }
20
21 /* Escreve em um sem foro bin rio: incrementa seu valor de um.  Esse
22    sem foro retorna imediatamente.  */
23
24 int binary_semaphore_post (int semid)
25 {
26   struct sembuf operations[1];
27   /* Use the first (and only) semaphore.  */
28   operations[0].sem_num = 0;
29   /* Increment by 1.  */
30   operations[0].sem_op = 1;
31   /* Permit undo'ing.  */
32   operations[0].sem_flg = SEM_UNDO;
33
34   return semop (semid, operations, 1);
35 }
```

Especificando o sinalizador *SEM_UNDO* permite lidar com o problema de terminar um processo enquanto esse mesmo processo tem recursos alocados através de um semáforo. Quando um processo encerra, ou voluntariamente ou involuntáriamente, o valores do semáforo são automaticamente ajustados para "desfazer" os efeitos do processo sobre o semáforo. Por exemplo, se um processo que tiver decrementado um semáforo for morto, o valor do semáforo é incrementado.

5.2.4 Depurando Semáforos

Use o comando *ipcs -s* para mostrar informação sobre conjuntos de semáforo existentes. Use o comando *ipcrm sem* para remover um conjunto de semaforo a partir da linha de comando. Por exemplo, para remover o conjunto de semáforo com o identificador 5790517, use essa linha:

```
\% ipcrm sem 5790517
```

5.3 Arquivos Mapeados em Memória

Memória mapeada permite a diferentes processos comunicarem-se por meio de um arquivo compartilhado. Embora você possa entender memória mapeada como sendo um segmento de memória compartilhada com um nome, você deve ser informado que exitem diferenças técnicas. Memória mapeada pode ser usada para comunicação entre processos ou como um caminho fácil para acessar o conteúdo de um arquivo.

Memória mapeada forma uma associação entre um arquivo e a memória de um processo. GNU/Linux quebra o arquivo em pedaços do tamanho de páginas de memória e então copia esses pedaços para dentro das páginas de memória virtual de forma que os pedaços possam se tornar disponíveis no espaço de endereçamento de um processo. Dessa forma, o processo pode ler o conteúdo do arquivo com acesso de memória comum. O processo pode também modificar o conteúdo do arquivo escrevendo para a memória. Esse processo de leitura e escrita para a memória permite acesso rápido a arquivos.

Você pode entender a memória mapeada como alocação de um espaço temporário de armazenamento para manter o conteúdo total de um arquivo, e então lendo o arquivo na área temporária de armazenamento e (se a área temporária de armazenamento for modificada) escrevendo a área temporária de armazenamento de volta para o arquivo posteriormente. GNU/Linux controla as operações de leitura e escrita para você.

Existem outros usos para arquivos mapeados em memória além do uso para comunicação entre processos. Alguns desses outros usos são discutidos na Seção 5.3.5, "Outros Usos para Arquivos Mapeados em Memória".

5.3.1 Mapeando um Arquivo Comum

Para mapear um arquivo comum para a memória de um processo, use a chamada de sistema *mmap* ("*Memory MAPped*" pronuncia-se "em-map"). O primeiro argumento é o endereço no qual você gostaria que GNU/Linux mapeasse o arquivo dentro do espaço de endereçamento do processo; o valor *NULL* permite ao GNU/Linux escolher um endereço inicial disponível. O segundo argumento é o comprimento do mapa em *bytes*. O terceiro argumento especifica a proteção sobre o intervalo de endereçamento mapeado. A proteção consiste de um "ou" bit a bit de PROT_READ, PROT_WRITE, e PROT_EXEC, correspondendo a permissão de leitura, escrita, e execução, respectivamente. O quarto argumento é um valor de sinalizador que especifica opções adicionais. O quinto argumento é um descritor de arquivo aberto para o arquivo a ser mapeado. O último argumento é o *offset* a partir do início do arquivo do qual inicia-se o mapa. Você pode mapear todo ou parte do arquivo para dentro da memória escolhendo o *offset* de início e o comprimento apropriadamente.

O valor do sinalizador é um "ou" bit a bit restrito aos seguintes:

- MAP_FIXED – Caso especifique esse sinalizador, GNU/Linux usa o endereço de sua requisição para mapear o arquivo em lugar de tratar esse endereço como uma sugestão. Esse endereço deve ser ajustado à página de memória.

- MAP_PRIVATE – Escritas para o intervalo de memória mapeado não devem ser escritos de volta ao arquivo mapeado, mas para uma cópia privada do arquivo mapeado. Nenhum outro processo vê essas escritas. Esse modo não pode ser usado com *MAP_SHARED*.

- MAP_SHARED – Escritas são imediatamente refletidas no arquivo correspondente ao invés de serem guardadas em uma área temporária na memória. Use esse modo quando estiver usando memória mapeada em IPC[a]. Esse modo não pode ser usado com *MAP_PRIVATE*.

[a]Nota do tradutor:*Inter Process Communication*.

Se a chamada de sistema *mmap* obtiver sucesso, irá retornar um apontador para o início da memória mapeada. Em caso de falha, a chamada de sistema *mmap* retorna *MAP_FAILED*.

Quando você tiver terminado com a memória mapeada, libere-a usando *munmap*. Informe a *munmap* o endereço inicial e o comprimento da região de memória mapeada. GNU/Linux automaticamente desmancha o mapeamento das regiões de memória mapeada quando um processo terminar.

5.3.2 Programas Exemplo

Vamos olhar em dois programas para ilustrar a utilização de regiões de memória mapeada para ler e escrever em arquivos. O primeiro programa, Listagem 5.5, gera um número aleatório e escreve-o em um arquivo mapeado em memória. O segundo programa, Listagem 5.6, lê o número, mostra-o, e substitui seu valor no arquivo de memória mapeada com o valor dobrado. Ambos recebem um argumento de linha de comando do arquivo a ser mapeado.

Listagem 5.5: (mmap-write.c) Escreve um Número Aleatório para um Arquivo Mapeado em Memória

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <fcntl.h>
4 #include <sys/mman.h>
5 #include <sys/stat.h>
6 #include <time.h>
7 #include <unistd.h>
8 #define FILE_LENGTH 0x100
9
10 /* Retorna um n mero  aleat rio  uniformemente  distribuido
11 no  intervalo  [low,high].  */
12
13 int random_range (unsigned const low, unsigned const high)
14 {
15   unsigned const range = high - low + 1;
16   return low + (int) (((double) range) * rand () / (RAND_MAX + 1.0));
17 }
18
19 int main (int argc, char* const argv[])
20 {
21   int fd;
22   void* file_memory;
23
24   /* Semeia o gerador de numeros  aleat rios.  */
25   srand (time (NULL));
26
27   /* Prepara um arqivo grande o suficiente para manter um inteiro sem sinal.  */
28   fd = open (argv[1], O_RDWR | O_CREAT, S_IRUSR | S_IWUSR);
29   lseek (fd, FILE_LENGTH+1, SEEK_SET);
30   write (fd, "", 1);
31   lseek (fd, 0, SEEK_SET);
32
33   /* Cria o mapeamento de mem ria.  */
34   file_memory = mmap (0, FILE_LENGTH, PROT_WRITE, MAP_SHARED, fd, 0);
35   close (fd);
36   /* Escreve um inteiro  aleat rio  para a  rea  mapeada de mem ria.  */
37   sprintf((char*) file_memory, "%d\n", random_range (-100, 100));
38   /* Libera a mem ria (desnecess ria uma vez que o programa sai).  */
39   munmap (file_memory, FILE_LENGTH);
40
41   return 0;
42 }
```

O programa *mmap-write* abre o arquivo, criando-o se ele já não existir previamente. O terceiro argumento a *open* especifica que o arquivo deve ser

aberto para leitura e escrita. Pelo fato de não sabermos o comprimento do arquivo, usamos *lseek* para garantir que o arquivo seja grande o suficiente para armazenar um inteiro e então mover de volta a posição do arquivo para seu início.

O programa mapeia o arquivo e então fecha o descritor de arquivo pelo fato de esse descritor não ser mais necessário. O programa então escreve um inteiro aleatório para a memória mapeada, e dessa forma para o arquivo, e desmapeia a memória. A chamada de sistema *munmap* é desnecessária pelo fato de que GNU/Linux deve automaticamente desmapear o arquivo ao término do programa.

Listagem 5.6: (mmap-read.c) Lê um Inteiro a partir de um Arquivo Mapeado em Memória, e Dobra-o

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <fcntl.h>
4 #include <sys/mman.h>
5 #include <sys/stat.h>
6 #include <unistd.h>
7 #define FILE_LENGTH 0x100
8
9 int main (int argc, char* const argv[])
10 {
11   int fd;
12   void* file_memory;
13   int integer;
14
15   /* Abre o arquivo.  */
16   fd = open (argv[1], O_RDWR, S_IRUSR | S_IWUSR);
17   /* Cria o mapeamento de mem ria.  */
18   file_memory = mmap (0, FILE_LENGTH, PROT_READ | PROT_WRITE,
19                       MAP_SHARED, fd, 0);
20   close (fd);
21
22   /* L  o inteiro, imprimi-o na sa da padr o, e multiplica-o por dois.  */
23   sscanf (file_memory, "%d", &integer);
24   printf ("valor: %d\n", integer);
25   sprintf ((char*) file_memory, "%d\n", 2 * integer);
26   /* Libera a memoria (desnecessaria uma vez que o programa sai).  */
27   munmap (file_memory, FILE_LENGTH);
28
29   return 0;
30 }
```

O programa *mmap-read* lê o número para fora do arquivo e então escreve o valor dobrado para o arquivo. Primeiramente, *mmap-read* abre o arquivo e mapeia-o para leitura e escrita. Pelo fato de podermos assumir que o arquivo é grande o suficiente para armazenar um inteiro sem sinal, não precisamos usar *lseek*, como no programa anterior. O programa lê e informa o valor para fora da memória usando *sscanf* e então formata e escreve o valor dobrado usando *sprintf*.

Aqui está um exemplo de execução desses dois programas exemplo. Os dois mapeiam o arquivo */tmp/integer-file*.

```
\% ./mmap-write /tmp/integer-file
\% cat /tmp/integer-file
```

```
42
\% ./mmap-read /tmp/integer-file
value: 42
\% cat /tmp/integer-file
```

Observe que o texto 42 foi escrito para o arquivo de disco sem mesmo haver uma chamada à função *write*, e foi lido de volta novamente sem haver uma chamada à função *read*. Note que esses programas amostra escrevem e leem ponteiro como uma sequência de caracteres (usando *sprintf* e *sscanf*) com propósitos didáticos somente – não existe necessidade de o conteúdo de um arquivo mapeado em memória ser texto. Você pode armazenar e recuperar binários arbitrários em um arquivo mapeado em memória.

5.3.3 Acesso Compartilhado a um Arquivo

Diferentes processos podem comunicar-se usando regiões mapeadas em memória associadas ao mesmo arquivo. Especificamente o sinalizador MAP_SHARED permite que qualquer escrita a essa regiões sejam imediatamente transferidas ao correspondente arquivo mapeado em memória e tornados visíveis a outros processos. Se você não especificar esse sinalizador, GNU/Linux pode colocar as operações de escrita em áreas temporárias de armazenamento antes de transferí-las ao arquivo mapeado.

Alternativamente, você pode forçar o GNU/Linux a esvaziar as áreas temporárias de armazenamento para o arquivo em disco chamando *msync*. Os primeiros dois parâmetros a *msync* especificam uma região de memória mapeada, da mesma forma que para *munmap*. O terceiro parâmetro pode os os seguintes valores de sinalizador:

- MS_ASYNC – A atualização é agendada mas não necessáriamente efetuada antes de a chamada retornar.
- MS_SYNC – A atualização é imediata; a chamada a *msync* bloqueia até que a atualização tenha sido finalizada. MS_SYNC e MS_ASYNC não podem ambas serem usadas simultâneamente.
- MS_INVALIDATE – Todos os outros mapeamentos são invalidados de forma que eles possam ver os valores atualizados.

Por exemplo, para descarregar a área de armazenamento temporário de um arquivo compartilhado mapeado no endereço *mem_addr* de comprimento *mem_length bytes*, chame o seguinte:

```
msync (mem_addr, mem_length, MS_SYNC | MS_INVALIDATE);
```

Da mesma forma que com segmentos de memória compartilhada, os usuários de regiões de memória mapeada devem estabelecer e seguir um protocolo para evitar condições de corrida. Por exemplo, um semáforo pode ser usado para garantir que somente um processo acesse a região de memória mapeada de cada vez. Alternativamente, você pode usar *fcntl* para colocar uma trava de leitura ou escrita no arquivo, como descrito na Seção 8.3, "A Chamada de Sistema *fcntl*: Travas e Outras Operações em Arquivos" no Capítulo 8.

5.3.4 Mapeamentos Privados

A especificação de *MAP_PRIVATE* a *mmap* cria uma região copie-na-escrita. Qualquer escrita para a região é refletida somente nessa memória do processo; outros processos que mapeiam o mesmo arquivo não irão ver as modificações. Ao invés de escrever diretamente para uma página compartilhada por todos os processos, o processo escreve para uma cópia privada dessa página. Todas as leituras e escritas subsequentes feitas pelo processo usaram essa cópia privada.

5.3.5 Outros Usos para Arquivos Mapeados em Memória

A chamada *mmap* pode ser usada para outros propósitos além da comunicação entre processos. Um uso comum é uma substituição para leitura e escrita. Por exemplo, ao invés de explicitamente ler um conteúdo de arquivo dentro da memória, um programa pode mapear o arquivo na memória e ver seu conteúdo através de leituras de memória. Para alguns programas, isso é mais conveniente e pode também executar mais rapidamente que operações explícitas de entrada e saída em arquivos.

Uma técnica avançada e poderosa usada por alguns programas é construir estruturas de dados (comumente instâncias de estruturas, por exemplo) em um arquivo mapeado em memória. Em uma chamada subsequênte, o programa mapeia aquele arquivo de volta na memória, e as estruturas de dados são restabelecidas em seu estado anterior. Note que, apesar disso, que apontadores nessas estruturas de dados irão ser inválidos a menos que eles todos apontem para dentro da mesma região mapeada de memória e a menos que cuidados sejam tomados para mapear o arquivo de volta para dentro do mesma região de endereçamento que o arquivo ocupava originalmente.

Outra técnica usada é mapear o arquivo especial de dispositivo */dev/zero* para a memória. O arquivo */dev/zero*, que é descrito na Seção 6.5.2, "O

Dispositivo /dev/zero" do Capítulo 6, "Dispositivos"comporta-se como se fosse um arquivo infinitamente longo preenchido com 0 *bytes*. Um programa que precisa uma fonte de 0 *bytes* pode *mmap* o arquivo */dev/zero*. Escritas para */dev/zero* são descartadas, de forma que a memória mapeada possa ser usada para qualquer propósito. Alocações de memória personalizadas muitas vezes mapeiam */dev/zero* para obter pedaços de memória pré-inicializados.

5.4 *Pipes*

A *pipe* é um dispositivo de comunicação que permite comunicação unidirecional. Dados escritos para a "escrita final" do *pipe* é lido de volta a partir da "leitura final". Os *Pipes* são dispositivos seriais; os dados são sempre lidos a partir do *pipe* na mesma ordem em que foram escritos. Tipicamente, um *pipe* é usado para comunicação entre duas linhas de execução em um único processo ou entre processos pai e filho.

Em um *shell*, o símbolo "|" cria um *pipe*. Por exemplo, o comando *shell* adiante faz com que o *shell* produza dois processos filhos, um para o comando "*ls*" e outros para o comando "*less*":

```
\% ls | less
```

O *shell* também cria um *pipe* conectando a saída padrão do subprocesso "*ls*" com a entrada padrão do processo "*less*". Os nomes de arquivos listados pelo "*ls*" são enviados para o "*less*" na exatamente mesma ordem como se eles tivessem sido enviados diretamente para o terminal.

A capacidade de dados do *pipe* é limitada. Se o processo escritor escreve mais rapidamente que o processo leitor pode consumir os dados, e se o *pipe* não puder armazenar mais dados, o processo escritor blioqueia até que mais capacidade torne-se disponível. Se o leitor tenta ler mas nenhum dado a ser lido está disponível, o processo leitor bloqueia até que dados tornem-se disponíveis. Dessa forma, o *pipe* automaticamente sincroniza os dois processos.

5.4.1 Criando *Pipes*

Para criar um *pipe*, chame o comando *pipe*. Forneça um array de inteiros de tamanho 2. A chamada a *pipe* armazena o descritor do arquivo de leitura na posição 0 do array e o descritor do arquivo de escrita na posição 1. Por exemplo, considere o código abaixo:

```
int pipe_fds[2];
int read_fd;
```

```
int write_fd;

pipe (pipe_fds);
read_fd = pipe_fds[0];
write_fd = pipe_fds[1];
```

Dados escritos para o descritor de arquivo *write_fd* podem ser lidos de volta a partir de *read_fd*.

5.4.2 Comunicação Entre Processos Pai e Filho

Uma chamada a *pipe* cria descritores de arquivo, os quais são válidos somente dentro do referido processo e seus filhos. Descritores de arquivo de processo não podem ser informados a processos não aparentados; todavia, quando o processo chama *fork*, descritores de arquivo são copiados para o novo processo filho. Dessa forma, *pipes* podem conectar somente com processos parentes.

No programa na Listagem 5.7, um *fork* semeia um processo filho. O filho herda os descritores de arquivo do *pipe*. O pai escreve uma sequência de caracteres para o *pipe*, e o filho lê a sequência de caracteres. O programa de amostra converte esses descritores de arquivo em fluxos *FILE** usando *fdopen*. Pelo fato de usarmos fluxos ao invés de descritores de arquivo, podemos usar funções de entrada e saída da biblioteca C GNU padrão de nível mais alto tais como *printf* e *fgets*.

Listagem 5.7: (pipe.c) Usando um *pipe* para Comunicar-se com um Processo Filho

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 /* Escreve COUNT c pias de MESSAGE para STREAM, pauspausando por um segundo
6    entre cada bloco de c pias. */
7
8 void writer (const char* message, int count, FILE* stream)
9 {
10   for (; count > 0; --count) {
11     /* Escreve a mensagem para o stream, e esvazia o fluxo imediatamente. */
12     fprintf (stream, "%s\n", message);
13     fflush (stream);
14     /* Cochila um momento. */
15     sleep (1);
16   }
17 }
18
19 /* L  sequ ncias de caractere aleat rias a partir de stream t o logo quanto
      poss vel. */
20
21 void reader (FILE* stream)
22 {
23   char buffer[1024];
24   /* L  at  que encontremos o fim do stream. fgets l  at  encontrar
25      ou um caractere de nova linha ou o fim de linha. */
26   while (!feof (stream)
27          && !ferror (stream)
28          && fgets (buffer, sizeof (buffer), stream) != NULL)
29     fputs (buffer, stdout);
30 }
31
32 int main ()
33 {
34   int fds[2];
35   pid_t pid;
36
37   /* Cria um pipe. Descritores de arquivo para os dois fins de pipe s o
38      colocados em fds. */
39   pipe (fds);
40   /* Bifurca um processo filho. */
41   pid = fork ();
42   if (pid == (pid_t) 0) {
43     FILE* stream;
44     /* Esse  o processo filho. Fecha nossa c pia do fim de escrita do
45        descritor de arquivo. */
46     close (fds[1]);
47     /* Converte o descritor de arquivo de leitura em um objeto FILE, e l
48        a partir dele. */
49     stream = fdopen (fds[0], "r");
50     reader (stream);
51     close (fds[0]);
52   }
53   else {
54     /* Esse  o processo pai. */
55     FILE* stream;
56     /* Fecha nossa c pia do read final do descritor de arquivo. */
57     close (fds[0]);
58     /* Converte o descritor de arquivo de leitura em um objeto FILE, e escreve
59        para ele. */
60     stream = fdopen (fds[1], "w");
61     writer ("Al , mundo.", 5, stream);
62     close (fds[1]);
63   }
64
65   return 0;
66 }
```

No início da *main*, a variável *fds* é declarada como sendo do tipo array inteiro de tamanho 2. A chamada a *pipe* cria um *pipe* e coloca os descritores de arquivo de leitura e de escrita naquele array. O programa então faz um *fork* no processo filho. Após o fechamento da leitura final do *pipe*, o processo

pai inicia escrevendo sequências de caractere para o *pipe*. Após o fechamento da escrita final do *pipe*, o filho lê sequências de caractere a partir do *pipe*.

Note que após a escrita na função escritora, o pai esvazia o *pipe* através de chamada a *fflush*. De outra forma, a sequência de caracteres pode não ter sido enviada imediatamente através do *pipe*.

Quando você chama o comando "*ls* | *less*", dois *forks* ocorrem: um para o processo filho "*ls*" e um para processo filho *less*. Ambos esses processos herdam o descritores de arquivo do *pipe* de forma que eles podem comunicar-se usando um *pipe*. Para ter processos não aparentados comunicando-se use um *FIFO* ao invés de *pipe*, como discutido na Seção 5.4.5, "FIFOs".

5.4.3 Redirecionando os Fluxos da Entrada Padrão, da Saída Padrão e de Erro

Frequentemente, você não irá querer criar um processo filho e escolher o final de um *pipe* bem como suas entrada padrão e sua saída padrão. Usando a chamada *dup2*, você pode equiparar um descritor de arquivo a outro. Por exemplo, para redirecionar a saída padrão de um processo para um descritor de arquivo *fd*, use a seguinte linha:

```
dup2 (fd, STDIN\_FILENO);
```

A constante simbólica *STDIN_FILENO* representa o descritor para a entrada padrão, cujo valor é 0. A chamada fecha a entrada padrão e então reabre-a com uma duplicata de *fd* de forma que os dois caminhos possam ser usados alternadamente. Descritores de arquivos equiparados compartilham a mesma posição de arquivo e o mesmo conjunto de sinalizadores de situação atual do arquivo. Dessa forma, caracteres lidos a partir de *fd* não são lidos novamente a partir da entrada padrão.

O programa na Listagem 5.8 usa *dup2* para enviar a saída de um *pipe* para o comando *sort*[4]. Após criar um *pipe*, o programa efetua um *fork*. O processo pai imprime algumas sequências de caractere para o *pipe*. O processo filho anexa o descritor de arquivo de leitura do *pipe* para sua entrada padrão usando *dup2*. O processo filho então executa o programa *sort*.

[4]O comando *sort* lê linhas de texto a partir da entrada padrão, ordena-as em ordem alfabética, e imprime-as para a saída padrão.

Listagem 5.8: (dup2.c) Redirecionar a Saída de um *pipe* com *dup2*

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <sys/wait.h>
4 #include <unistd.h>
5
6 int main ()
7 {
8   int fds[2];
9   pid_t pid;
10
11   /* Cria um pipe.  Descritores de arquivo para os dois fins do pipe sao
12      colocados na variavel fds.  */
13   pipe (fds);
14   /* Bifurca um processo filho.  */
15   pid = fork ();
16   if (pid == (pid_t) 0) {
17     /* Esse e o processo filho.  Fecha nossa copia da escrita final do
18        descritor de arquivo.  */
19     close (fds[1]);
20     /* Conecta read final do pipe com a entrada padrao.  */
21     dup2 (fds[0], STDIN_FILENO);
22     /* Substitui o processo filho com o programa "sort".  */
23     execlp ("sort", "sort", 0);
24   }
25   else {
26     /* Esse    o processo pai.  */
27     FILE* stream;
28     /* Fecha nossa c pia do read final do descritor de arquivos.  */
29     close (fds[0]);
30     /* converte o descritor de arquivos de escrita em um objeto FILE, e escreve
31        para ess objeto FILE.  */
32     stream = fdopen (fds[1], "w");
33     fprintf (stream, "Isso e um teste.\n");
34     fprintf (stream, "Alo, mundo.\n");
35     fprintf (stream, "Meu cachoro tem.\n");
36     fprintf (stream, "Esse programa    grande.\n");
37     fprintf (stream, "Um peixe, dois peixes.\n");
38     fflush (stream);
39     close (fds[1]);
40     /* Espera pelo processo filho para encerrar.  */
41     waitpid (pid, NULL, 0);
42   }
43
44   return 0;
45 }
```

5.4.4 As Funções *popen* e *pclose*

Um uso comum de *pipes* é enviar dados para ou receber dados de um programa sendo executado em um sub-processo. As funções *popen* e *pclose* facilitam esse paradigma por meio da eliminação da necessidade de chamar *pipe*, *fork*, *dup2*, *exec*, e *fdopen*.

Compare a Listagem 5.9, que utiliza *popen* e *pclose*, com o exemplo anterior (a Listagem 5.8).

Listagem 5.9: (popen.c) Exemplo Usando *popen*

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main ()
5 {
6   FILE* stream = popen ("sort", "w");
7   fprintf (stream, "Isso    um teste.\n");
8   fprintf (stream, "Al  , mundo.\n");
9   fprintf (stream, "Meu cachorro tem pulgas.\n");
10  fprintf (stream, "Esse programa    grande.\n");
11  fprintf (stream, "Um peixe, dois peixes.\n");
12  return pclose (stream);
13 }
```

A chamada a *popen* cria um processo filho executando o comando *sort*, substituindo chamadas a *pipe*, *fork*, *dup2*, e *execlp*. O segundo argumento, "w", indica que o processo que fez a chamada a *popen* espera escrever para o processo filho. O valor de retorno de *popen* é um fim de *pipe*; o outro final é conectado à entrada padrão do processo filho. Após a escrita terminar, *pclose* fecha o fluxo do processo filho, espera que o processo encerre, e retorna valor de situação atual.

O primeiro argumento a *popen* é executado como um comando *shell* em um sub-processo executando */bin/sh*. O *shell* busca pela variável de ambiente *PATH* pelo caminho usual para encontrar programas executáveis. Se o segundo argumento for "r", a função retorna o fluxo de saída padrão do processo filho de forma que o processo pai possa ler a saída. Se o segundo argumento for "w", a função retorna o fluxo de entrada padrão do processo filho de forma que o processo pai possa enviar dados. Se um erro ocorrer, *popen* retorna um apontador nulo.

Chama *pclose* para fechar um fluxo retornado por *popen*. Após fechar o fluxo especificado, *pclose* espera pelo fim do processo filho.

5.4.5 FIFOs

Um arquivo *first-in*, *first-out* (*FIFO*)[5] é um *pipe* que tem um nome no sistema de arquivos. Qualquer processo pode abrir ou fechar o *FIFO*; os processo em cada lado do *pipe* precisam ser aparentados uns aos outos. *FIFOs* são também chamados *pipes* com nomes.

Você cria um *FIFO* usando o comando *mkfifo*. Especifique o caminho do *FIFO* na linha de comando. Por exemplo, para criar um *FIFO* em */tmp/fifo* você deve fazer o seguinte:

```
\% mkfifo /tmp/fifo
\% ls -l /tmp/fifo
prw-rw-rw-    1 samuel    users           0 Jan 16 14:04 /tmp/fifo
```

[5]Nota do tradutor:Quem entrar primeiro sai também primeiro.

O primeiro caractere da saída do comando *ls* é uma letra "p", indicando que esse arquivo é atualmente um *FIFO* (*pipe* com nome). Em uma janela, leia a partir do *FIFO* usando o seguinte:

```
\% cat < /tmp/fifo
```

Em uma segunda janela, escreva para o *FIFO* fazendo o seguinte:

```
\% cat > /tmp/fifo
```

Então digite algumas linhas de texto. A cada vez que você pressionar *Enter*, a linha de texto é enviada através do *FIFO* e aparece na primeira janela. Feche o *FIFO* pressionando *Ctrl+D* na segunda janela. Remova o *FIFO* com a seguinte linha:

```
\% rm /tmp/fifo
```

5.4.5.1 Criando um *FIFO*

Criar um FIFO a partir de um programa em linguagem C use a função *mkfifo*[6]. O primeiro argumento é a localização na qual criar o *FIFO*; o segundo parâmetro especifica o dono do *pipe*, o grupo ao qual pertence o *group*, e as permissões para o resto do mundo, como discutido no Capítulo 10, "Segurança" na Seção 10.3, "Permissões do Sistema de Arquivo". Pelo fato de um *pipe* possuir obrigatóriamente um leitor e um escritor, as permissões devem incluir ambas tanto para leitura quanto para escrita. Se o *pipe* não puder ser criado (por exemplo, se um arquivo com o nome escolhido para o *pipe* já exista), *mkfifo* retorna -1. Inclua os arquivos de cabeçalho <sys/types.h> e <sys/stat.h> se você chamar a função *mkfifo*.

5.4.5.2 Accessando um *FIFO*

Acesse um *FIFO* da mesma forma que é feita com arquivos comuns. Para comunicar-se através de um *FIFO*, um programa deve abrí-lo para escrita, e outro programa deve abrí-lo para leitura. Ou ainda usando as funções de entra e saída de baixo nível (*open*, *write*, *read*, *close*, e assim por diante, como listado no apêndice B, "E/S de Baixo Nível") ou as funções de E/S da bilioteca C (*fopen*, *fprintf*, *fscanf*, *fclose*, e assim por diante) podem ser usadas.

Por exemplo, para escrever uma área temporária de armazenamento de dados para um *FIFO* usando rotinas de E/S de baixo nível, você pode usar o código abaixo:

[6]Nota do tradutor:para mais informações use o comando *shell* "*man 3 mkfifo*".

```
int fd = open (fifo_path, O_WRONLY);
write (fd, data, data_length);
close (fd);
```

Para ler uma sequência de caracteres a partir do *FIFO* usando as funções de E/S da biblioteca C GNU padrão, você pode usar o código abaixo:

```
FILE* fifo = fopen (fifo_path, "r");
fscanf (fifo, "%s", buffer);
fclose (fifo);
```

Um *FIFO* pode ter multiplos leitores ou multiplos escritores. Os *Bytes* de cada escritor são escritos automaticamente até alcançar o máximo tamanho de *PIPE_BUF* (4KB no GNU/Linux). Pedaços de escritas sumultâneas pode ser intercalados. Regras similares aplicam-se a leituras simultânea.

Differenças de *Pipes* nomeados do *Windows*

Pipes no sistemas operacionais *Win32* são muito similares a *pipes* em GNU/Linux. (Reporte-se à documentação de biblioteca do *Win32* para detalhes técnicos sobre isso.) As principais diferenças referem-se a *pipes* nomeados, os quais, para *Win32*, funcionam mais como *sockets*. Pipes nomeados em *Win32* podem conectar processos em cmputadores separados conectados via rede. Em GNU/Linux, *sockets* são usados para esse propósito. Também, *Win32* permite multiplas conecções de leitura e escrita por meio de *pipe* nomeado sem intercalação de dados, e *pipes* podem ser usados para comunicação em mão dupla.[7]

5.5 *Sockets*

Um *socket* é um dispositivo de conecção bidirecional que pode ser usado para comunicar-se com outro processo na mesma máquina ou com um processo em outras máquinas. *Sockets* são o único tipo de comunicação entre processo que discutiremos nesse capítulo que permite comunicação entre processos em dirferentes computadores . Programas de *Internet* tais como *Telnet*, *rlogin*, *FTP*, *talk*, e a *World Wide Web* usam *sockets*.

Por exemplo, você pode obter a página *WWW* de um servidor *Web* usando o programa *Telnet* pelo fato de eles ambos (servidor *WWW* e *Telnet* do cliente) usarem *sockets* para comunicações em rede.[8] Para abrir uma

[7]Note que somente *Windows NT* pode criar um *pipe* nomeado; programas *Windows 9x* pode formar somente conecções como cliente.

[8]Comumente, poderia usar *telnet* para conectar um servidor *Telnet* para acesso remoto. Mas você pode também usar o *telnet* para conectar um servidor de um tipo diferente e então digitar comentários diretamete no próprio *telnet*.

conecção com um servidor *WWW* localizado em *www.codesourcery.com*, use *telnet www.codesourcery.com 80*. A constante mágica 80 especifica uma conecção para o programa de servidor *Web* executando *www.codesourcery.com* ao invés de algum outro processo. Tente digitar "*GET /*" após a conecção ser estabelecida. O comando "*GET /*" envia uma mensagem através do *socket* para o servidro *Web*, o qual responde enviando o código fonte em na linguagem *HTML* da página inicial fechando a conecção em seguida:

```
\% telnet www.codesourcery.com 80
Trying 206.168.99.1...
Connected to merlin.codesourcery.com (206.168.99.1).
Escape character is '^]'.
GET /
<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1">
...
```

5.5.1 Conceitos de *Socket*

Quando você cria um *socket*, você deve especificar três parâmetros: o estilo da comunicação,o escopo, e o protocolo.

Um estilo de comunicação controla como o *socket* trata dados transmitidos e especifica o número de parceiros de comunicação. Quando dados são enviados através de um *socket*, esses dados são empacotados em partes menores chamadas pacotes. O estilo de comunicação determina como esses pacotes são manuseados e como eles são endereçados do emissor para o receptor.

- Estilos de conecção garantem a entrega de todos os pacotes na ordem que eles foram enviados. Se pacotes forem perdidos ou reordenados por problemas na rede, o receptor automaticamente requisita a retransmissão desses pacotes perdidos/reordenados ao emissor. Um *socket* de estilo do tipo conecção é como uma chamada telefônica: O endereço do emissor e do receptor são fixados no início da comunicação quando a conecção é estabelecida.

- Um *socket* de estilo do tipo datagrama não garante a entrega ou a ordem de chegada. Pacotes podem ser perdidos ou reordenados no caminho devido a erros de rede ou outras condições. Cada pacote deve ser rotulado com seu destino e não é garantido que seja entregue. O sistema garante somente o "melhor esforço" de forma que pacotes podem desaparecer ou chegar em uma ordem diferente daquela que foi transportado. Um estilo de transmissão do tipo datagram *socket* comporta-se mais como várias cartas colocadas na agência de correio. O emissor especifica o endereço do receptor para cada carta individualmente.

Um escopo de *socket* especifica como endereços de *socket* são escritos. Um endereço de *socket* identifica a outra extremidade de uma conecção de *socket*. Por exemplo, endereços de *socket* no "espaço de endereçamento local"são comumente nomes de arquivo comuns. No "escopo de *Internet*" um endereço de *socket* é composto do endereço *Internet* (também conhecido como um endereço de protocolo de *Internet* ou endereço IP) de uma máquina anexada à rede e um número de porta. O número de porta faz distinção no conjunto de multiplos *sockets* na mesma máquina.

Um protocolo especifica como dados são transmitidos. Alguns protocolos são *TCP/IP*, os protocolos primários usados pela *Internet*; o protocolo de rede *AppleTalk*; e o protocolo de comunicação local *UNIX*. Algumas combinações de estilos, escopo, e protocolos não são suportadas.

5.5.2 Chamadas de Sistema

Os *Sockets* são mais flexíveis que as técnicas de comunicação discutidas anteriormente. Adiante temos as chamadas de sistema relacionadas a *sockets*[9]:

- *socket* – Cria um *socket*
- *close* – Destrói um *socket*
- *connect* – Cria uma conecção entre dois *sockets*
- *bind* – Rotula um *socket* de servidor com um endereço
- *listen* – Configura um *socket* para aceitar condições
- *accept* – Aceita uma conecção e cria um novo *socket* para a conecção

Sockets são representados por descritores de arquivo.

Criando e Destruindo *Sockets*

As funções *socket* e *close* criam e destroem *sockets*, respectivamente. Quando você cria um *socket*, especifica as três escolhas de *socket*: escopo, estilo de comunicação, e protocolo. Para o parâmetro de escopo, use constantes iniciando por *PF_* (abreviatura de "*protocol families*"). Por exemplo, PF_LOCAL ou PF_UNIX especificam o escopo local, e *PF_INET* especificam escopos de *Internet* . Para o parâmetro de estilo de comunicação, use constantes iniciando com *SOCK_*. Use *SOCK_STREAM* para um *socket* de

[9]Nota do tradutor: no slackware 13.1 padrão o comando *man 2 socketcall* retorna, entre outras coisas: *accept*(2), *bind*(2), *connect*(2), *getpeername*(2), *getsockname*(2), *getsockopt*(2), *listen*(2), *recv*(2), *recvfrom*(2), *recvmsg*(2), *send*(2), *sendmsg*(2), *sendto*(2), *setsockopt*(2), *shutdown*(2), *socket*(2), *socketpair*(2).

estilo do tipo conecção, ou use *SOCK_DGRAM* para um *socket* de estilo do tipo datagrama.

O terceiro parâmetro, o protocolo, especifica o mecanismo de baixo nível para transmitir e receber dados. Cada protocolo é válido para uma combinação particular de estilo e escopo. Pelo fato de existir habitualmente um melhor protocolo para cada tal par de estilo e espaço de endereçamento, especificar 0 (zero) é comumente o protocolo correto. Se o *socket* obtiver sucesso, ele retornará um descritor de arquivo para o *socket*. Você pode ler de ou escrever para o *socket* usando *read*, *write*, e assim por diante, como com outro descritor de arquivo. Quando você tiver terminado com um *socket*, chame *close* para removê-lo.

Chamando *connect*

Para criar uma conecção entre dois *sockets*, o cliente chama *connect*, especificando o endereço de um *socket* de servidor para conectar-se. Um cliente é o processo que inicia a conecção, e um servidor é um processo esperando para aceitar conecções. O cliente chama *connect* para iniciar uma conecção de um *socket* local para o *socket* de servidor especificado pelo segundo argumento. O terceiro argumento é o comprimento, em bytes, da estrutura de endereço apontada pelo segundo argumento. O formato de endereço de *socket* difere conforme o escopo do *socket*.

Enviando Informações

Qualquer técnica para escrever para um descritor de arquivos pode ser usada para para escrever para um *socket*. Veja o Apêndice B para uma discursão sobre função de E/S de baixo nível do GNU/Linux e algumas questões envolvendo seu uso. A função *send*, que é específica para descritores de arquivo de *socket*, fornece uma alternativa pra escrever com poucas escolhas adicionais; veja a página de manual de *send* para mais informações[10].

5.5.3 Servidores

Um ciclo de vida de um servidor consiste da criação de um *socket* de estilo do tipo conecção, associação de um endereço a esse *socket*, colocação de uma chamada pra escutar e que habilita conecções para o *socket*, colocação de chamadas para aceitar conecções de entrada, e finalmente fechamento do *socket*. Dados não são lidos e escritos diretamente via *socket* do servidor; ao invés disso, a cada vez que um programa aceita uma nova conecção, GNU/Linux cria um *socket* em separado para usar na transferência de dados sobre aquela connecção. Nessa seção, introduziremos as chamadas de sistema *bind*, *listen*, e *accept*.

[10]Nota do tradutor: *man 2 send*.

Um endereço deve ser associado ao *socket* do servidor usando *bind* se for para um cliente encontrá-lo. O primeiro argumento de *bind* é o descritor de arquivo do *socket*. O segundo argumento de *bind* é um apontador para uma estrutura de endereço de *socket*; o formato desse segundo argumento depende da família de endereço do *socket*. o terceiro argumento é o comprimento da estrutura de endereço, em bytes. Quando um endereço é associado a um *socket* de estido do tipo conecção, esse *socket* de estido do tipo conecção deve chamar *listen* para indicar que esse *socket* de estido do tipo conecção é um servidor. O primeiro argumento à chamada *listen* é o descritor de arquivo do *socket*. O segundo argumento a *listen* especifica quantas conecções pendentes são enfileiradas. Se a fila estiver cheia, conecções adicionais irão ser rejeitadas. Essa rejeição de conecções não limita o número total de conecções que um servidor pode controlar; Essa rejeição de conecções limita o número de clientes tentando conectar que não tiveram ainda aceitação.

Um servidor aceita uma requisição de conecção de um cliente por meio de uma chamada à chamada de sistema *accept*. O primeiro argumento a *accept* é o descritor de arquivo do *socket*. O segundo argumento a *accept* aponta para uma estrutura de endereço de *socket*, que é preenchida com o endereço de *socket* do cliente. O terceiro argumento a *accept* é o comprimento, em bites, de uma estrutura de endereço de *socket*. O servidor pode usar o endereço do cliente para determinar se o *socket* servidor realmente deseja comunicar-se com o cliente. A chamada a *accept* cria um novo *socket* para comunicação com o cliente e retorna o correspondente descritor de arquivos. O *socket* servidor original continua a *accept* novas conecções de outros clientes. Para ler dados de um *socket* sem remover esse *socket* da fila de entrada, use *recv*. A chamada *recv* recebe os mesmos argumentos que a chamada *read*, mas adicionalmente o argumento *FLAGS*. Um sinalizador do tipo *MSG_PEEK* faz com que dados sejam lidos mas não removidos da fila de entrada.

5.5.4 *Sockets* Locais

Sockets conectando processos no mesmo computador podem usar o escopo local representado pelos sinônimos *PF_LOCAL* e *PF_UNIX*. *Sockets* conectando processos no mesmo computador são chamados *sockets* locais ou *sockets* de domínio UNIX. Seus endereços de *socket*, especificados por nomes de arquivo, são usados somente quando se cria conecções.

O nome de *socket* é especificado em *struct sockaddr_un*. Você deve escolher o campo *sun_family* para *AF_LOCAL*, indicando que o nome do *socket* só é válido no escopo local. O campo *sun_path* especifica o nome de arquivo que vai ser usado e pode ser, no máximo, do comprimento de 108 bytes. O comprimento atual de *struct sockaddr_un* deve ser calculado usando a ma-

cro *SUN_LEN*. Qualquer nome de arquivo pode ser usado, mas o processo deve ter permissão de escrita no diretório, o que permite a adição de arquivos ao diretório. Para conectar um *socket*, um processo deve ter permissão de leitura para o arquivo. Mesmo através de diferentes computadores compartilhando o mesmo sistema de arquivos, somente processos executando no mesmo computador podem comunicar-se com *sockets* de escopo local.

O único protocolo permitido para o escopo local é 0 (zero).

Pelo fato de residir no sistema de arquivos, um *socket* local é listado como um arquivo. Por exemplo, preste atenção o "s" inicial:

```
\% ls -l /tmp/socket
srwxrwx--x    1 user      group    0 Nov 13 19:18 /tmp/socket
```

Chame *unlink* para remover um *socket* local quando você tiver encerrado com o referido *socket* local.

5.5.5 Um Exemplo Usando um *Sockets* de Escopo local

Ilustraremos *sockets* com dois programas. O programa do servidor, na Listagem 5.10, cria um *socket* de escopo local e escuta à espera de conecções a esse *socket* de escopo local. Quando esse *socket* de escopo local recebe uma conecção, ele lê mensagens de texto a partir da conecção e mostra-as até que a conecção feche. Se uma das mensagens recebidas pelo *socket* do servidor for "*quit*" o programa do servidor remove o *socket* e termina. O programa *socket-server* recebe o caminho para o *socket* como seu argumetnode linha de comando.

Listagem 5.10: (socket-server.c) Servidor de *Socket* de Escopo Local

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/socket.h>
5 #include <sys/un.h>
6 #include <unistd.h>
7
8 /* L   texto de um socket e exibe-o.  Continua at   que o
9    socket feche.  Retorna um valor n o nulo se o cliente envia
10   mensagem de sa da ("quit"), retorna zero nos outros casos.  */
11
12 int server (int client_socket)
13 {
14   while (1) {
15     int length;
16     char* text;
17
18     /* Primeiro, l   o comprimento da mensagem de texto a partir do socket.  Se
19        read retorna zero, o cliente fecha a conec   o.  */
20     if (read (client_socket, &length, sizeof (length)) == 0)
21       return 0;
22     /* Aloca um espa o tempor rio de armazenamento para manter o texto.  */
23     text = (char*) malloc (length);
24     /* L   o texto propriamente dito, e mostra-o.  */
25     read (client_socket, text, length);
26     printf ("%s\n", text);
27     /* Libera o espa o temporario de armazenameto.  */
28     free (text);
29     /* Se o cliente enviar a mensagem "quit", terminamos tudo.  */
30     if (!strcmp (text, "quit"))
31       return 1;
32   }
33 }
34
35 int main (int argc, char* const argv[])
36 {
37   const char* const socket_name = argv[1];
38   int socket_fd;
39   struct sockaddr_un name;
40   int client_sent_quit_message;
41
42   /* Cria o socket.  */
43   socket_fd = socket (PF_LOCAL, SOCK_STREAM, 0);
44   /* Indica isso ao servidor.  */
45   name.sun_family = AF_LOCAL;
46   strcpy (name.sun_path, socket_name);
47   bind (socket_fd, &name, SUN_LEN (&name));
48   /* escuta esperando por conec   es.  */
49   listen (socket_fd, 5);
50
51   /* Repetidamente aceita conec   es, usando um ciclo em torno da fun   o server()
        para tratar
52      com cada cliente.  Continua at   que um cliente envia umam mensgem "quit".  */
53   do {
54     struct sockaddr_un client_name;
55     socklen_t client_name_len;
56     int client_socket_fd;
57
58     /* Aceita uma conec   o.  */
59     client_socket_fd = accept (socket_fd, &client_name, &client_name_len);
60     /* Manipula a conec   op.  */
61     client_sent_quit_message = server (client_socket_fd);
62     /* Fecha nosso fim da conec   o.  */
63     close (client_socket_fd);
64   }
65   while (!client_sent_quit_message);
66
67   /* Remove o arquivo de socket.  */
68   close (socket_fd);
69   unlink (socket_name);
70
71   return 0;
72 }
```

O programa cliente, na Listagem 5.11, conecta a um*socket* de escopo local e envia uma mensagem. O nome *path* para o *socket* e a mensagem são especificados na linha de comando.

Listagem 5.11: (socket-client.c) Cliente de *Socket* de Escopo Local

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <sys/socket.h>
4 #include <sys/un.h>
5 #include <unistd.h>
6
7 /* Escreve TEXT para o socket fornecido pelo descritor de arquivo SOCKET_FD.  */
8
9 void write_text (int socket_fd, const char* text)
10 {
11   /* Escreve o n mero de bytes na sequ ncia de caracteres, incluindo
12      o caractere de fim de sequ ncia de caracteres.  */
13   int length = strlen (text) + 1;
14   write (socket_fd, &length, sizeof (length));
15   /* escreve a sequ ncia de caracteres.  */
16   write (socket_fd, text, length);
17 }
18
19 int main (int argc, char* const argv[])
20 {
21   const char* const socket_name = argv[1];
22   const char* const message = argv[2];
23   int socket_fd;
24   struct sockaddr_un name;
25
26   /* Cria o socket.  */
27   socket_fd = socket (PF_LOCAL, SOCK_STREAM, 0);
28   /* armazena o nome do servido no endere o do socket.  */
29   name.sun_family = AF_LOCAL;
30   strcpy (name.sun_path, socket_name);
31   /* Conecta o socket.  */
32   connect (socket_fd, &name, SUN_LEN (&name));
33   /* escreve o texto na linha de comando para o socket.  */
34   write_text (socket_fd, message);
35   close (socket_fd);
36   return 0;
37 }
```

Antes de o cliente enviar uma mensagem de texto, ele envia o comprimento do texto que pretende enviar mandando *bytes* da variável inteira *length*. Da mesma forma, o servidor lê o comprimento do texto a partir do *socket* de dentro da variável inteira. Isso permite ao servidor alocar uma área temporária de armazenamento de tamanho apropriado para manter a mensagem de texto antes de lê-la a partir do *socket*.

Para tentar esse exemplo, inicie o programa servidor em uma janela. Especifique um caminho para o *socket* por exemplo, */tmp/socket*.

```
\% ./socket-server /tmp/socket
```

Em outra janela, execute o cliente umas poucas vezes, especificando o mesmo caminho de *socket* adicionando mensagens para enviar para o servidor:

```
\% ./socket-client /tmp/socket ``Hello, world."
\% ./socket-client /tmp/socket ``This is a test."
```

O programa servidor recebe e imprime as mensagens acima. Para fechar o servidor, envie a menssagem "*quit*" a partir de um cliente:

```
\% ./socket-client /tmp/socket ``quit"
```

O programa servidor termina.

5.5.6 *Sockets* de Domínio *Internet*

Sockets de domínio UNIX podem ser usados somente para comunicação entre dois processos no mesmo computador. *Sockets* de domínio *Internet* , por outro lado, podem ser usados para conectar processos em diferentes máquinas conectadas por uma rede. *Sockets* conectando processos através da *Internet* usam o escopo de *Internet* representado por *PF_INET*. Os protocolos mais comuns são TCP/IP. O protocolo *Internet* (IP), um protocolo de baixo nível, move pacotes através da *Internet*, quebrando em pedaços e remontando os pedaços, se necessário. O IP garante somente "melhor esforço" de entrega, de forma que pacotes podem desaparece ou serem reordenados durante o transporte. Todo computador participante é especificando usando um único número IP. O Protocolo de Controle de Transmissão (TCP), formando uma camada sobre o IP, fornece transporte confiável no que se refere a ordenação na conecção. Os dois protocolos juntos tornam possivel que conecções semelhantes às telefônicas sejam estabelecidas entre computadores e garante que dados se entregues de forma confiável e em ordem.

Nomes de DNS

Pelo fato de ser mais fácil lembrar nome que números, o Serviço de Nomes de Domínio (DNS) associa nomes tais como *www.codesourcery.com* a números IP únicos de computadores. DNS é implementado por meio de uma hierarquia mundial de servidores de nome, mas você não precisa entender o protocolo DNS para usar nomes de computador conectado à rede *Internet* em seus programas.

Endereços de *socket* localizados na *Internet* possuem duas partes: uma máquina e um número de porta. Essa informação é armazenada na variável *struct sockaddr_in*. Escolha o campo *sin_family* para *AF_INET* de forma a indicar que *struct sockaddr_in* é um endereço de escopo *Internet*. O campo *sin_addr* armazena o endereço *Internet* da máquina desejada como um número de IP inteiro de 32-bit. Um número de porta distingue entre diferentes *sockets* em uma mesma máquina. Pelo fato de diferentes máquinas armazenarem valores multibyte em ordem de bytes diferentes, use o comando *htons* para converter o número da porta para ordem de byte de rede. Veja a página de manual para o comando "ip" para maiores informações.[11]

Para converter para converter nomes de computador conectado à rede legíveis a humanos, ou em números na notação de ponto padronizada (tais como 10.0.0.1) ou em nomes de DNS[12] (tais como *www.codesourcery.com*) em

[11]Nota do tradutor:temos "ip" tanto na seção 7 como na seção 8 das páginas de manual.
[12]Nota do tradutor:*Domain Name service*.

números IP de 32-bit, você pode usar *gethostbyname*. A função *gethostbyname* retorna um apontador para a estrutura *struct hostent*; o campo *h_addr* contém o número IP do computador conectado à rede. Veja o programa amostra na Listagem 5.12.

A Listagem 5.12 ilustra o uso de *sockets* de domínio *Internet* . O programa obtém o página inicial do servidor Web cujo nome do computador conectado à rede é especificado na linha de comando.

Listagem 5.12: (socket-inet.c) Lê de um Servidor WWW

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <netinet/in.h>
4 #include <netdb.h>
5 #include <sys/socket.h>
6 #include <unistd.h>
7 #include <string.h>
8
9 /* Imprime o conte do da home page para o socket do servidor.
10    Retorna uma indica  o de sucesso.  */
11
12 void get_home_page (int socket_fd)
13 {
14   char buffer[10000];
15   ssize_t number_characters_read;
16
17   /* Envia o comando HTTP GET para a home page.  */
18   sprintf (buffer, "GET /\n");
19   write (socket_fd, buffer, strlen (buffer));
20   /* L  a partir do socket.  read pode n o receber todos os dados de uma
21      s  vez, ent o continua tentando at  que esgotemos os dados a serem lidos.  */
22   while (1) {
23     number_characters_read = read (socket_fd, buffer, 10000);
24     if (number_characters_read == 0)
25       return;
26     /* Escreve os dados para a sa da padr o.  */
27     fwrite (buffer, sizeof (char), number_characters_read, stdout);
28   }
29 }
30
31 int main (int argc, char* const argv[])
32 {
33   int socket_fd;
34   struct sockaddr_in name;
35   struct hostent* hostinfo;
36
37   /* Cria o socket.  */
38   socket_fd = socket (PF_INET, SOCK_STREAM, 0);
39   /* Armazena o nome do servidor no endere o do socket.  */
40   name.sin_family = AF_INET;
41   /* Converte de sequ ncia de caracteres para n meros.  */
42   hostinfo = gethostbyname (argv[1]);
43   if (hostinfo == NULL)
44     return 1;
45   else
46     name.sin_addr = *((struct in_addr *) hostinfo->h_addr);
47   /* Sevidor web usa a porta 80.  */
48   name.sin_port = htons (80);
49
50   /* Conecta-se ao servidor web  */
51   if (connect (socket_fd, &name, sizeof (struct sockaddr_in)) == -1) {
52     perror ("connect");
53     return 1;
54   }
55   /* Requisita a home page do servidor.  */
56   get_home_page (socket_fd);
57
58   return 0;
59 }
```

Esse programa recebe o nome do computador conectado à rede do servidor Web na linha de comando (não uma URL – isto é, recebe a informação

sem o "http://"). O programa chama a função *gethostbyname* para traduzir o nome do computador conectado à rede em um endereço IP numérico e então conectar um fluxo (TCP) *socket* na porta 80 daquele computador conectado à rede. Servidores Web falam o Protocolo de Transporte de Hipertexto (HTTP), de forma que o programa emita o comando *HTTP GET* e o servidor responda enviando o texto da página inicial.

Números de Porta Padronizados

Por convenção, servidores Web esperam por conecções na porta 80. A maioria dos serviços de rede *Internet* são associados a números de prota padronizados. Por exemplo, servidores Web que usam SSL esperam por conecções na porta 443, e servidores de e-mail (que usam o protocolo SMTP) esperam por conecções na porta 25. Em sistemas GNU/Linux, a associação entre nomes de protocolos, nomes de serviços e números de porta padronizados está listada no arquivo */etc/services*. A primeira coluna é o protocolo ou nome de serviço. A segunda coluna lista o número da porta e o tipo de conecção: *tcp* para serviços orientados à conecção, ou *udp* para datagramas. Se você implementar algum serviço personalizado de rede usando *sockets* de domíno *Internet*, use números de porta maiores que 1024.

Por exemplo, para recuperar a página inicial do sítio Web *www.codesourcery.com*, chame o seguinte:

```
\% ./socket-inet www.codesourcery.com
<html>
  <meta http-equiv=``Content-Type" content="text/html; charset=iso-8859-1">

...
```

5.5.7 *Sockets* Casados

Como vimos anteriormente, a função *pipe* cria dois descritores de arquivo para o início e o fim de um *pipe*. *Pipes* são limitados pelo fato de os descritores de arquivo deverem ser usados por processos aparentados e pelo fato de a comunicação ser unidirecional. A função *socketpair* cria dois descritores de arquivo para dois *sockets* conectados no mesmo computador. Esses descritpres de arquivo permitem comunicação de mão dupla entre processos aparentados.

Seus primeiros três parâmetros são os mesmo que aqueles da chamada de sistema *socket*: eles especificam o domínio, estilo de coneco, e o protocolo. O último parâmetro é um array de dois inteiros, os quais são preenchidos com as descrições de arquivo dos dois *sockets*, de maneira similar a *pipe*. Quando você chama *socketpair*, você deve especificar *PF_LOCAL* como o domínio.

Parte II

Dominando GNU/Linux

- 6 Dispositivos
- 7 O Sistema de Arquivos */proc*
- 8 Chamadas de Sistema do GNU/Linux
- 9 Código Assembly Embutido
- 10 Segurança
- 11 Um Modelo de Aplicação GNU/Linux

Capítulo 6

Dispositivos

GNU/LINUX, COMO A MAIORIA DOS SISTEMAS OPERACIONAIS, INTERAGE COM DISPOSITIVOS de *hardware* por meio de componentes de *software* modularizados chamados programas controladores de dispositivos[1]. Um programa controlador de dispositivo esconde as peculiaridades de protocolos de comunicação de dispositivos de *hardware* do systema operacional e permite ao sistema interagir o dispositivo através de uma interface padronizada.

Sob GNU/Linux, programas controladores de dispositivos são parte do *kernel* e poderm ser ou linkados estaticamente dentro do *kernel* ou chamados conforme a necessidade como módulos do *kernel*. Programas controladores de dispositivos executam como parte do *kernel* e não estão diretamente acessíveis a processos de usuário. Todavia, GNU/Linux fornece um mecanismo através do qual processos podem comunicar-se com um acionador de dispositivo – e através desse mesmo acionador de dispositivo, com um dispositivo de *hardware* – por meio de objetos semelhantes a arquivos. Esses objetos aparecem no sistema de arquivos, e programas podem abrí-los, ler a partir deles, e escrever para eles praticamente como se eles fossem arquivos normais. Usando ou operações de E/S de baixo nível do GNU/Linux (veja o Apêndix B, "E/S de Baixo Nível") ou operações de E/S da biblioteca C GNU padrão, seus programas podem comunicar-se com dispositivos de *hardware* através desse objetos semelhantes a arquivos.

GNU/Linux também fornece muitos objetos semelhantes a arquivos que comunicam-se diretamente com o *kernel* em lugar de com programas controladores de dispositivos. Esses objetos semelhantes a arquivos que comunicam-se diretamente com o *kernel* não são linkados para dispositivos de *hardware*; ao invés disso, eles fornecem vários tipos de comportamento especializado

[1]Nota do tradutor: *device drivers*.

que podem ser de uso para aplicações e programas de sitema.

Cultive a Precaução Quando Estiver Acessando Dispositivos!
A técnica nesse capítulo fornece acesso direto a programas controladores de dispositivos executando no *kernel* do GNU/Linux, e através desses acionadores de dispositivo tem-se acesso a dispositivos de *hardware* conectados ao sistema. Use essas técnicas com cuidado pelo fato de que o abuso dessas mesmas técnicas pode vir a prejudicar ou danificar o sistema GNU/Linux. Veja especialmente a barra lateral "Perigos de Dispositivos de Bloco."

6.1 Tipos de Dispositivos

Arquivos de dispositivo não são arquivos comuns – eles não representam regiões de dados sobre um sistema de arquivos localizado sobre um disco. Ao invés disso, dados lidos de um ou escritos para um arquivo de dispositivo é comunicado ao correspondente acionador de dispositivo, e do acionador de dispositivo para o dispositivo subjacente. Arquivos de dispositivos veem em dois sabores:

- Um *dispositivo de caractere* representa um dispositivo de *hardware* que lê ou escreve um fluxo serial de *bytes* de dados. Portas seriais e paralelas[a], acionadores de fita, dispositivos de terminal, e placas de som são exemplos de dispositivos de caractere.

- Um *dispositivo de bloco* representa um dispositivo de *hardware* que lê ou escreve dados em blocos de tamanho fixo. Ao contrário de um dispositivo de caractere, um dispositivo de blocos fornece acesso aleatério a dados armazenados no dispositivo. Um acionador de disco é um exemplo de dispositivo de bloco.

[a]Nota do tradutor: as "modernas" portas USB funcionam como tanto como dispositivo de bloco quanto como dispositivo de caractere, dependendo do dispositivo que estiver conectado a ela.

Programas de aplicação típicos nunca irão usar dispositivos de bloco. Enquanto um acionador de disco é representado como um dispositivo de bloco, o conteúdo de cada partição do disco tipicamente contém um sistema de arquivos, e esse sistema de arquivos é montado dentro da árvore do sistema de arquivos raíz do GNU/Linux. Somente o código do *kernel* que implementa o sistema de arquivos precisa acessar o dispositivo de bloco diretamente; programas de aplicação acessam o conteúdo do disco através de arquivos normais e diretórios.

Perigos de Dispositivos de Bloco
Dispositivos de bloco fornecem acesso direto a dados do acionador de disco. Apesar de a maioria dos sistema GNU/Linux esteja configurado para prevenir que processos de usuários comuns acessem esses dispositivos diretamente, um processo de superusuário pode inflingir danos severos através da modificação do conteúdo do disco. Por meio da escrita no dispositivo de bloco do disco, um programa pode modificar ou destuir informações de controle do sistema de arquivos e mesmo uma tabela de partição do disco e o registro principal de inicialização[a], dessa forma travar um acionador ou mesmo colocar o sistema inteiro inutilizado. Sempre acesse esses dispositivos com grande cuidado.
Aplicações algumas vezes fazem uso de dispositivos de caractere, apesar da maioria dos dispositivos ser de bloco. Discutiremos muitos dispositivos de caractere nas seções seguintes.

[a]Nota do tradutor: o *Master Boot Record* - "MBR".

6.2 Números de Dispositivo

GNU/Linux identifica dispositivos usando dois números: o número de dispositivo principal e o número de dispositivo secundário. O número de dispositivo principal especifica a qual programa controlador o dispositivo corresponde. A correspondência entre números de dispositivo principal e programas controladores é fixa e faz parte dos fontes do *kernel* do GNU/Linux. Note que o mesmo número de dispositivo principal pode corresponder a dois diferentes programas controladores, um deles é um dispositivo de caractere e outro é um dispositivo de bloco. Números de dispositivo secundário distinguem dispositivos individuais ou componenetes controlados por um único acionador. O significado de um número de dispositivo secundário depende do acionador de dispositivo.

Por exemplo, dispositivo principal no. 3 corresponde à controladora IDE primária no sistema. Uma controladora IDE pode ter dois dispositivos (disco, fita, ou acionador de CD-ROM) conectados a essa mesma controladora; o dispositivo "mestre" tem número de dispositivo secundário 0, e o dispositivo "escravo" tem número de dispositivo secundário 64. Partições individuais no dispositivo mestre (se o dispositivo suportar partições) são representados por números de dispositivo secundário 1, 2, 3, e assim por diante. Partições individuais no dispositivo escravo são representados por números de dispositivo secundário 65, 66, 67, e assim por diante.

Números de dispositivo principal são listados na documentação dos fontes do *kernel* do GNU/Linux. Em muitas distribuições GNU/Linux, essa documentação pode ser encontrada em */usr/src/linux/Documentation/de-*

vices.txt[2]. A entrada especial */proc/devices* lista números de dispositivo principal correspondendo a programas controladores de dispositivos ativos atualmente carregados dentro do *kernel*[3]. (Veja Capítulo 7, "O Sistema de Arquivos */proc*" para mais informação sobre as entradas do sistema de arquivos */proc*.)

6.3 Entradas de Dispositivo

Uma entrada de dispositivo é de muitas formas o mesmo que um arquivo regular. Você pode mover a entrada de dispositivo usando o comando "*mv*" e apagar uma entrada de dispositivo usando o comando "*rm*" . Se você tentar copiar uma entrada de dispositivo usando "*cp*" apesar disso, você irá ler *bytes* a partir do dispositivo (se o dispositivo suportar leitura) e escrever esses *bytes* para o arquivo de destino. Se você tentar sobrescrever uma entrada de dispositivo, você irá escrever *bytes* no dispositivo correspondente ao invés de sobrescrever a entrada.

Você pode criar uma entrada de dispositivo no sistema de arquivos usando o comando *mknod* (use o comando "*man 1 mknod*" para a página de manual) ou usando a chamada de sistema *mknod* (use o comando "*man 2 mknod* para acessar a página de manual correspondente). Criando uma entrada de dispositivo no sistema de arquivos não implica automaticamente que o correspondente programa controlador de dispositivo ou dispositivo de *hardware* esteja presente ou disponível; a entrada de dispositivo é meramente um acesso de comunicação com o programa controlador[4], se ele existir. Somente o processo de superusuário pode criar dispositivos de bloco e de caractere usando o comando "*mknod*" ou a chamada de sistema "*mknod*".

Para criar um dispositivo usando o comando "*mknod*" , especifique como primeiro argumento o caminho no qual a entrada irá aparecer no sistema de arquivos. Para o segundo argumento, especifique *b* para um dispositivo de bloco ou *c* para um dispositivo de caractere. Forneça os números de dispositivo principal e secundário como o terceiro e o quarto argumento, respectivamente. Por exemplo, o comando adiante cria uma entrada de dispositivo de caractere chamada *lp0* no diretório atual. O dispositivo tem número de

[2]Nota do tradutor: o slackware 13.37 padrão trás o referido arquivo no local indicado acima mas a versão mais recente que encontrada localiza-se em `ftp://ftp.kernel.org/pub/linux/docs/device-list/devices-2.6+.txt`.

[3]Nota do tradutor: o comando é "cat /proc/devices" e mostra uma saída dividida em dois grupos, os dispositivos de bloco e os dispositivos de caractere.

[4]Nota do tradutor: é um portão de embarque de aeroporto. O portão sempre está lá mas você tem que esperar pelo avião que vai usar o portão de embarque.

dispositivo principal 6 e número de dispositivo secundário 0. Esses números correspondem à primeira porta paralela no sistema GNU/Linux.

```
% mknod ./lp0 c 6 0
```

Lembrando que somente processos do superusuário podem criar dispositivos de bloco e dispositivos de caractere, de forma que você deve estar logado como *root* para usar o comando acima com sucesso.

O comando "*ls*" mostra entradas de dispositivos especificamente. Se você usar comando "*ls*" com a opção "-l" ou com a opção "-o" , o primeiro caractere de cada linha de saída especifica o tipo de entrada de dispositivo. Relembrando que o caractere "–" (um hífem) designa um arquivo normal, enquanto "d" designa um diretório. Similarment, "b" designa um dispositivo de bloco, e "c" designa um dispositivo de caractere. Para os dois últimos o comando "*ls*" mostra os números de dispositivo principal e secundário onde seria mostrado o tamanho de um arquivo comum. Por exemplo, podemos mostrar o dispositivo de caractere que acabamos de criar:

```
% ls -l lp0
crw-r-----     1 root     root      6,    0 Mar   7 17:03 lp0
```

Em um programa, você pode determinar se uma entrada de sistema de arquivos é um dispositivo de bloco ou um dispositivo de caractere e então recuperar seus números de dispositivo usando o comando "*stat*". Veja a Seção B.2, "stat" no Apêndice B, para instruções.

Para remover uma entrada de dispositivo use o comando "*rm*". O comando "*rm*" simplesmente remove a entrada de dispositivo do sistema de arquivos.

```
% rm ./lp0
```

6.3.1 O Diretório */dev*

Por convenção, um sistema GNU/Linux inclui um diretório */dev* contendo o conjunt completo das entradas de dispositivos de caractere e de dispositivos de bloco que GNU/Linux tem conhecimento. Entradas no "*/dev*" possuem nomes padronizados correspondendo aos números de dispositivo principal e secundário.

Por exemplo, o dispositivo mestre anexado à controladora IDE primária, que tem números de dispositivo principal e secundário 3 e 0, tem o nome padrão "*/dev/hda*". Se esse dispositivo suporta partições, a primeira partição do dispositivo "*/dev/hda*", que tem número de dispositivo secundário 1, tem o nome padronizado "*/dev/hda1*". Você pode verificar que isso é verdadeiro em seu sistema:

```
% ls -l /dev/hda /dev/hda1
brw-rw----    1 root     disk        3,    0 May   5   1998 /dev/hda
brw-rw----    1 root     disk        3,    1 May   5   1998 /dev/hda1
```

Similarmente, "*/dev*" tem uma entrada para o dispositivo de caractere porta paralela que usamos anteriormente:

```
% ls -l /dev/lp0
crw-rw----    1 root      daemon     6,    0 May   5   1998 /dev/lp0
```

Na maioria dos casos, você não deve usar "*mknod*" para criar suas próprias entradas de dispositivo. Use as entradas no "*/dev*" ao invés de criar entradas. Programas comuns não possuem escolha e devem usar as entradas de dispositivo pré-existentes pelo fato de eles não poderem criar suas próprias entradas de dispositivo. Tipicamente, somente administradores de sistema e desenvolvedores que trabalham com dispositivos de *hardware* especializados irão precisar criar entradas de dispositivo. A maioria das distribuições GNU/Linux incluem facilidade para ajudar administradores de sistema a criar entradas dispositivo padronizadas com os nomes corretos.

6.3.2 Acessando Dispositivos por meio de Abertura de Arquivos

Como você pode usar esses dispositivos? no caso de dispositivos de caractere, o uso pode ser bastante simples: Abra o dispositivo como se ele fosse um arquivo normal, e leia a partir do ou escreva para o dispositivo. Você pode mesmo usar comandos comuns para arquivos tais como "*cat*", ou sua sintaxe de redirecionamento de *shell*, para enviar dados ao dispositivo ou para ler dados do dispositivo.

Por exemplo, se você tiver uma impressora conectada na primeira porta paralela de seu computador, você pode imprimir arquivos enviando-os diretamente para "*/dev/lp0*".[5] Para imprimir o conteúdo de documento.txt, use o comando seguinte:

```
% cat document.txt > /dev/lp0\\
```

Você deve ter permissão de escrita para a entrada de dispositivo de forma que esse comando funcione; em muitos sistemas GNU/Linux, as permissões são escolhidas de forma que somente *root* e o *system's printer daemon (lpd)* possa escrever para o arquivo. Também, o que aparece na saída de sua impressora depende de como sua impressora interpreta o conteúdo dos dados

[5]Usuários windows irão reconhecer que esse dispositivo é similar ao arquivo mágico Windows LPT1.

que você envia. Algumas impressoras irão imprimir arquivos no formato texto plano que forem enviadas a ela,[6] enquanto outras não irão imprimí-los. Impressoras com suporte a *PostScript* irão converter e imprimir arquivo *PostScript* que você enviar para ela.

Em um programa, o envio de dados para um dispositivo muito simples. Por exemplo, o fragmento de código adiante[7] usa funções de entrada e saída de baixo nível para enviar o conteúdo de uma área temporária de armazenamento para */dev/lp0*.

```
int fd = open ("/dev/lp0", O_WRONLY);
write (fd, buffer, buffer_length);
close (fd);
```

6.4 Dispositivos de *Hardware*

Alguns dispositivos de bloco comuns são listados na Tabela 6.1[8]. Nomes de dispositivo para dispositivos similares seguem o modelo óbvio (por exemplo, a segunda partição no primeiro acionador SCSI é */dev/sda2*). Essa aparência óbvia é ocasionalmente útil para saber a quais dispositivos esses nomes de dispositivos correspondem ao examinar sistemas de arquivos montados em */proc/mounts* (veja a Seção 7.5, "Acionadores, Montagens, e Sistemas de Arquivos" no Capítulo 7, para mais sobre isso).

A Tabela 6.2 lista alguns dispositivos de caractere comuns.

Você pode acessar certos componentes de *hardware* através de mais de um dispositivo de caractere; muitas vezes, os diferentes dispositivos de caractere fornecem diferentes semânticas. Por exemplo, quando você usa o dispositivo de fita IDE */dev/ht0*, GNU/Linux automaticamente rebobina a fita no acionador quando você fecha o descritor de arquivo. Você pode usar o dispositivo */dev/nht0* para acessar o mesmo acionador de fita, exceto que GNU/Linux não irá rebobinar automaticamente a fita quando você fechar o descritor de arquivo. Você algumas vezes possivelmente pode ver programas usando */dev/cua0* e dispositivos similares; esses são antigos dispositivos para portas seriais tais como */dev/ttyS0*.

Ocasionalmente, você irá desejar escrever dados diretamente para dispositivos de caractere por exemplo:

[6]Sua impressora pode requerer caracteres explícitos de retorno de cabeça de impressão, código 13 ASCII, ao final de cada linha, e pode requerer um caractere de alimentação de página, código ASCII 12, ao final de cada página.

[7]Nota do tradutor:em linguagem C.

[8]Nota do tradutor: as duas últimas linhas da tabela foram incluídas pelo tradutor.

Tabela 6.1: Lista Parcial de Dispositivos de Bloco Comuns

Dispositivo	Nome	Principal	secundário
Primeiro acionador de disquetes	/dev/fd0	2	0
Segundo acionador de disquetes	/dev/fd1	2	1
Controladora IDE primária, dispositivo mestre	/dev/hda	3	0
Controladora IDE primária, dispositivo mestre, primeira partição	/dev/hda1	3	1
Controladora IDE primária, dispositivo secundário	/dev/hdb	3	64
Controladora IDE primária, dispositivo secundário, primeira partição	/dev/hdb1	3	65
Controladora IDE secundária, dispositivo mestre	/dev/hdc	22	0
Controladora IDE secundária, dispositivo secundário	/dev/hdd	22	64
Primeiro acionador SCSI	/dev/sda	8	0
Primeiro acionador SCSI, primeira partição	/dev/sda1	8	1
Segundo disco SCSI	/dev/sdb	8	16
Segundo acionador SCSI, primeira partição	/dev/sdb1	8	17
Primeiro acionador de CD-ROM/DVD SCSI	/dev/scd0	11	0
Segundo acionador de CD-ROM/DVD SCSI	/dev/scd1	11	1
Pendrive em porta usb	/dev/sdc	8	32
Primeira partição do pendrive acima	/dev/sdc1	8	33

Tabela 6.2: Alguns Dispostivos de Caractere Comuns

Dispositivo	Nome	Principal	secundário
Porta paralela 0	/dev/lp0 ou /dev/par0	6	0
Porta paralela 1	/dev/lp1 ou /dev/par1	6	1
Primeira porta serial	/dev/ttyS0	4	64
Segunda porta serial	/dev/ttyS1	4	65
Acionador de fita IDE	/dev/ht0	37	0
Primeiro acionador de fita SCSI	/dev/st0	9	0
Segundo acionador de fita SCSI	/dev/st0	9	1
Console do sistema	/dev/console	5	1
Primeiro terminal virtual	/dev/tty1	4	1
Segundo terminal virtual	/dev/tty2	4	2
Dispositivo de terminal do processo atual	/dev/tty	5	0
Placa de som	/dev/audio	14	4

- Um programa de terminal possivelmente pode acessar um modem diretamente através de um dispositivo de porta serial. Dados escritos para ou lidos dos dispositivos são transmitidos por meio do modem para um computador remoto.

- Um programa de *backup* de fita possivelmente pode escrever dados diretamente para um dispositivo de fita. O programa de *backup* pode implementar seu próprio formato de compressão e verificação de erro.

- Um programa pode escrever diretamente no primeiro terminal virtual[a] enviando dados para */dev/tty1*. Janelas de terminal executando em um ambiente gráfico, ou em sessões de terminal de login remoto, não estão associados a terminais virtuais; ao invés disso, essas janelas de terminal estão associadas a pseudo-terminais. Veja a seção 6.6, "PTYs" para informações sobre esses terminais.

[a]Na maioria dos sistemas GNU/Linux, você pode alternar para o primeiro terminal virtual pressionand Ctrl+Alt+F1. Use Ctrl+Alt+F2 para o segundo terminal virtual, e assim por diante.

- Algumas vezes um programa precisa acessar o dispositivo de terminal com o qual está associado.

  Por exemplo, seu programa pode precisar perguntar ao usuário por uma senha. Por razões de segurança, você pode desejar ignorar o redirecionamento da entrada padrão e da saída padrão e sempre ler a senha a partir do terminal, não importa como o usuário chame o comando. Um caminho para fazer isso é abrir */dev/tty*, que sempre corresponde ao dispositivo de terminal associado com o processo que o abriu. Escreve uma mensagem para aquele dispositivo, e lê a senha a partir de */dev/tty* também. Através do ato de ignorar a entrada e a saída padrão, evita que o usário possa fornecer ao seu programa uma senha a partir de um arquivo usando uma sintaxe do *shell* tal como:

  ```
  % secure\_program < my-password.txt
  ```

  Se você precisar autenticar usuários em seu programa. você deve aprender mais sobre o recurso PAM do GNU/Linux. Veja a seção 10.5, "Autenticando Usuários" no Capítulo 10, "Segurança" para maiores informações.

- Um programa pode emitir sons através da placa de som do sistema enviando dados de audio para o dispositivo */dev/audio*. Note que os dados de audio devem estar no formato da Sun (comumente associado com a extensão ".au"). Por exemplo, muitas distribuições GNU/Linux são acompanhadas do arquivo de som clássico */usr/share/sndconfig/sample.au*[a]. Se seu sistema inclui esse arquivo, tente tocá-lo através do seguinte comando:

  ```
  % cat /usr/share/sndconfig/sample.au > /dev/audio
  ```

  Se você está planejando usar sons em seu programa, apesar disso, você deve investigar as várias bibliotecas sonoras e serviços desponíveis para GNU/Linux. O ambiente *Gnome windowing* usa o *Enlightenment Sound Daemon* (EsounD), em http://www.tux.org/~ricdude/EsounD.html[b]. KDE usa o aRts, em http://space.twc.de/~stefan/kde/arts-mcop-doc/[c]. Se você usa um desses sistemas de som ao invés de escrever diretamente para */dev/audio*, seu programa irá cooperar melhor com outros programas que usam a placa de som do computador.

[a]Nota do tradutor: o referido arquivo não foi encontrado no slackware 13.1 padrão mas o comando "*find / -name *.au 2>/dev/null.*" encontra outro para você.

[b]Nota do tradutor:Atualmente temos o *ALSA - Advanced Linux Sound Architecture.*

[c]Nota do tradutor: http://www.arts-project.org/, aRts - analog Realtime synthesizer.

6.5 Dispositivos Especiais

GNU/Linux também fornece muitos dispositivos de caractere que não correspondem a dispositivos de *hardware*. Essas entradas todas usam o número de dispositivo principal 1, que é associado ao dispositivo de memória do *kernel* do GNU/Linux ao invés de ser associado a um acionador de dispositivo.

6.5.1 O Dispositivo */dev/null*

A entrada */dev/null*, o dispositivo nulo, é muito útil. Esse dispositivo nulo serve a dois propósitos; você está provavelmente familiarizado ao menos com o primeiro deles:

- GNU/Linux descarta quaisquer dados escritos para */dev/null*. Um artifíco comum para especificar */dev/null* como um arquivo de saída em algum contexto onde a saída é descartável.

  Por exemplo, para executar um comando e descartar sua saída padrão (sem mostrá-la ou escrevê-la em um arquivo), redirecione a saída padrão para */dev/null*:

  ```
  % verbose_command > /dev/null
  ```

- Lê de */dev/null* sempre resulta em um caractere de fim de arquivo. Por exemplo, se você abre um descritor de arquivo para */dev/null* usando a função *open* e então tenta ler a partir desse descritor de arquivo, a leitura irá ler nenhum *byte* e irá retornar 0. Se você copia a partir do */dev/null* para outro arquivo, o arquivo de destino irá ser um arquivo de tamanho zero:

  ```
  % cp /dev/null empty-file
  % ls -l empty-file
  -rw-rw----    1 samuel   samuel          0 Mar     8 00:27 empty-file
  ```

6.5.2 O Dispositivo */dev/zero*

A entrada de dispositivo */dev/zero* comporta-se como se fosse um arquivo infinitamente longo preenchido com 0 *bytes*. Tantas quantas forem as tentativas de ler *bytes* de */dev/zero*, GNU/Linux "gera" suficientes 0 *bytes*.

Para ilustrar isso, vamos executar o programa *hexdump* mostrado na Listagem B.4 na Seção B.1.4, "Lendo Dados" do Apêndice B. Esse programa mostra o conteúdo de um arquivo na forma hexadecimal.

```
% ./hexdump /dev/zero
0x000000 : 00 00 00 00    00   00   00   00   00   00   00   00   00   00   00   00
0x000010 : 00 00 00 00    00   00   00   00   00   00   00   00   00   00   00   00
0x000020 : 00 00 00 00    00   00   00   00   00   00   00   00   00   00   00   00
0x000030 : 00 00 00 00    00   00   00   00   00   00   00   00   00   00   00   00
...
```

Aperte *Ctrl+C* quando estiver convencido que a visualização irá prosseguir infinitamente.

Mapeamento de memória para */dev/zero* é uma técnica avançada de alocação de memória. Veja a Seção 5.3.5, "Outros Usos para *mmap*" no Capítulo 5, "Comunicação Entre Processos" para mais informação, e veja a barra lateral "Obtendo Página de Memória Alinhada" na Seção 8.9, "*mprotect*: Ajustando as Permissões da Memória" no Capítulo 8, "Chamadas de Sistema do GNU/Linux" para um exemplo.

6.5.3 */dev/full*

A entrada */dev/full* comporta-se como se fosse um arquivo sobre um sistema de arquivos cheio. Uma escrita para */dev/full* irá falhar e escolher *errno* para *ENOSPC*, que comumente indica que a escrita para o dispositivo não pode ser feita pelo fato de o dispositivo estar cheio.

Por exemplo, você pode tentar escrever para */dev/full* usando o comando *cp*:

```
% cp /etc/fstab /dev/full
cp: /dev/full: No space left on device
```

A entrada */dev/full* é primáriamente útil para testar como seu sistema comporta-se se esse mesmo sistema executar sem espaço no disco durante uma tentativa de escrever para um arquivo.

6.5.4 Dispositivos Geradores de *Bytes* Aleatórios

Os dispositivos especiais */dev/random* e */dev/urandom* fornecem acesso à facilidade intena do *kernel* do GNU/Linux de geração de números aleatórios.

A maioria das funções de *software* para gerar números aleatórios, tais como a função *rand* na biblioteca C GNU padrão, atualmente geram números aleatórios imperfeitos. Embora esses números satisfaçam algumas propriedades dos números aleatórios, eles são reprodutíveis: Se você iniciar com o mesmo valor semente, você irá obter a mesma sequência de números aleatórios imperfeitos todas as vezes que fizer isso. Esse comportamento é inevitável pelo fato de computadores serem intrinsecamente determinísticos e previsíveis. Para certas aplicaões, apesar disso, esse comportamento determinístico é indesejável; por exemplo, é algumas vezes possível quebrar um algorítmo criptográfico se você puder obter a sequência de números aleatórios que o referido algorítmo emprega.

Para obter melhores números aleatórios em programas de computadores é necessário uma fonte externa de aleatoriedade. O *kernel* do GNU/Linux fornece as ferramentas necessárias a uma particularmente boa fonte de aleatoriedade: você! Medindo o espaço de tempo entre suas ações de entrada, tais como pressionamentos de tecla e movimentos de *mouse*, GNU/Linux é capaz de gerar um fluxo imprevisível de números aleatórios de alta qualidade. Você pode acessar esse fluxo por meio da leitura a partir de */dev/random* e de */dev/urandom*. Os dados que você lê correspondem a um fluxo de *bytes* gerados aleatóriamente.

A diferença entre os dois dispositivos[9] mostra-se por si mesma quando exaure-se seu reservatório de aleatóriedade. Se você tenta ler um grande número de *bytes* a partir de */dev/random* mas não gera qualquer ações de entrada (você não digita, o *mouse* fica parado, ou executa ações similares), GNU/Linux bloqueia a operação de leitura. Somente ao você fornecer alguma aleatoriedade é que é possível ao GNU/Linux gerar mais alguns *bytes* aleatórios e retornar esses *bytes* aleatórios para seu programa.

Por exemplo, tente mostrar o conteúdo de */dev/random* usando o comando *od*.[10]

Cada linha de saída mostra 16 *bytes* aleatórios.

```
% od -t   x1   /dev/random
0000000   2c   9c 7a db 2e   79   3d   65   36   c2   e3   1b   52   75   1e   1a
0000020   d3   6d 1e a7 91   05   2d   4d   c3   a6   de   54   29   f4   46   04
0000040   b3   b0 8d 94 21   57   f3   90   61   dd   26   ac   94   c3   b9   3a
0000060   05   a3 02 cb 22   0a   bc   c9   45   dd   a6   59   40   22   53   d4
```

O número de linhas de saída que você vê irá variar podendo haver algumas poucas e a saída pode eventualmente pausar quando GNU/Linux esvazia seu estoque de aleatoriedade. Agora tente mover seu *mouse* ou digitar no seu teclado, e assista números aleatórios adicionais aparecerem. Para realmente melhor aleatoriedade, ponha seu gato para andar no teclado.

Uma leitura a partir de */dev/urandom*, ao contrário, nunca irá bloquear. Se GNU/Linux executa com aleatoriedade esgotada, */dev/urandom* usa um algorítmo criptográfico para gerar *bytes* aleatórios imperfeitos a partir da sequência anterior de *bytes* aleatórios. Embora esses *bytes* sejam aleatórios o suficiente para a maioria dos propósitos, eles não passam em muitos testes de aleatoriedade quanto aqueles obtidos a partir de */dev/random*.

Por exemplo, se você usar o comando seguinte, os *bytes* aleatórios irão voar para sempre, até que você mate o programa com *Ctrl+C*:

```
% od -t   x1   /dev/urandom
0000000   62   71 d6 3e af dd de 62 c0 42 78 bd 29 9c 69 49
0000020   26   3b 95 bc b9 6c 15 16 38 fd 7e 34 f0 ba ce c3
0000040   95   31 e5 2c 8d 8a dd f4 c4 3b 9b 44 2f 20 d1 54
...
```

O uso de números aleatórios de */dev/random* em um programa é fácil, também. A Listagem 6.1 mostra uma função que gera um número aleatório

[9]Nota do tradutor:*/dev/random* e */dev/urandom*.

[10]Usamos *od* aqui ao invés do programa *hexdump* mostrado na Listagem B.4, mesmo apesar dele fazer muito lindamente a mesma coisa, pelo fato de *hexdump* encerra quando esgota os dados, enquanto *od* espera por mais dados para torná-los disponíveis. A opção "*-t x1*" informa ao comando *od* para imprimir o conteúdo do arquivo em hexadecimal.

usando *bytes* lidos a partir de */dev/random*. Lembrando que */dev/random* bloqueia uma leitura até que exista suficiente aleatoriedade disponível para satisfazê-la; você pode usar */dev/urandom* ao invés de */dev/random* se execução rápida for mais importante e se você puder conviver com baixa qualidade em geração de números aleatórios.

Listagem 6.1: (random_number.c) Função para Gerar um Número Aleatório

```
1 #include <assert.h>
2 #include <sys/stat.h>
3 #include <sys/types.h>
4 #include <fcntl.h>
5 #include <unistd.h>
6
7 /* Retorna um inteiro aleat rio entre MIN e MAX, inclusive.  Obt m
8    aleatoriedade do dispositivo /dev/random.  */
9
10 int random_number (int min, int max)
11 {
12   /* Armazena um descritor de arquivo aberto para /dev/random em uma vari vel
13      est tica.  Dessa forma, n o precisamos abrir o arquivo a cada vez
14      que essa fun  o for chamada.  */
15   static int dev_random_fd = -1;
16
17   char* next_random_byte;
18   int bytes_to_read;
19   unsigned random_value;
20
21   /* Garante que MAX    maior que MIN.  */
22   assert (max > min);
23
24   /* Se essa for a primeira vez que essa fun  o    chamada, abre um
25      descritor de arquivo para /dev/random.  */
26   if (dev_random_fd == -1) {
27     dev_random_fd = open ("/dev/random", O_RDONLY);
28     assert (dev_random_fd != -1);
29   }
30
31   /* L  bytes aleat rio o suficiente para preencher uma vari vel inteira.  */
32   next_random_byte = (char*) &random_value;
33   bytes_to_read = sizeof (random_value);
34   /* Fica no ciclo at  que tenhamos lidos bytes suficientes.  Uma vez que /dev/
        random    preenchido
35      a partir das a  es geradas pelo usu rio, a leitura pode ser bloqueada, e pode
           somente
36      retornar um byte aleat rio simples de cada vez.  */
37   do {
38     int bytes_read;
39     bytes_read = read (dev_random_fd, next_random_byte, bytes_to_read);
40     bytes_to_read -= bytes_read;
41     next_random_byte += bytes_read;
42   } while (bytes_to_read > 0);
43
44   /* Calcula um n mero aleat rio no intervalo correto.  */
45   return min + (random_value % (max - min + 1));
46 }
```

6.5.5 Dispositivos Dentro de Dispositivos

Um *dispositivo dentro de um dispositivo*[11] habilita a você simular um dispositivo de bloco usando um arquivo de disco comum. Imagine um acionador de disco para o qual dados são escritos para ele e lidos dele em um arquivo chamado *imagem-disco* em lugar de escritos para e lidos de trilhas e setores

[11]Nota do tradutor:*loopback*.

de um acionador de disco físico atual ou partição de disco. (Certamente, o arquivo *imagem-disco* deve residir sobre o disco atual, o qual deve ser maior que o disco simulado.) Um dispositivo simulador habilita você usar um arquivo dessa maneira.

Dispositivos simuladores são chamados */dev/loop0*, */dev/loop1*, e assim por diante. Cada um desses dispositivos simuladores pode ser usado para simular um único dispositivo de bloco por vez. Note que somente o superusuário pode definir um dispositivo simulador.

Um dispositivo simulador pode ser usado da mesma forma que qualquer outro dispositivo de bloco. Em particular, você pode construir um sistema de arquivos sobre o dispositivo simulador e então montar aquele sistema de arquivo como você montaria o sistema de arquivos sobre um disco comum ou uma partição comum. Da mesma forma que um sistema de arquivos, que reside inteiramente dentro de um arquivo de disco comum, é chamado um sistema de arquivos virtual.

Para construir um sistema de arquivos virtual e montá-lo como um dispositivo simulado, siga os passos abaixo:

1. Crie um arquivo vazio para conter o sistema de arquivos virtual. O tamanho do arquivo irá ser o tamanho aparente do dispositivo simulado após esse mesmo dispositivo ser montado. Um caminho conveniente para construir um arquivo de um tamanho fixo é com o comando "*dd*". Esse comando copia blocos (por padrão, o tamanho de bloco é 512 *bytes* cada) de um arquivo para outro. O dispositivo */dev/zero* é uma fonte conveniente de *bytes* para serem copiados. Para construir um arquivo de 10MB chamado imagem-disco, use o comando seguinte:

```
% dd if=/dev/zero of=/tmp/disco-imagem count=20480
20480+0 records in
20480+0 records out
% ls -l /tmp/imagem-disco
-rw-rw----      1 root     root       10485760 Mar    8 01:56 /tmp/imagem-disco
```

2. O arquivo que você criou é preenchido com 0 *bytes*. Antes de você montar o referido arquivo, você deve construir um sistema de arquivos. Isso ajusta várias estruturas de controle necessárias a organizar e armazenar arquivos, e construir o diretório principal. Você pode construir qualquer tipo de sistema de arquivos que você quiser na sua imagem de disco. Para construir um sistema de arquivos *ext2* (o tipo mais comumente usado em discos GNU/Linux)[a], use o comando *mke2fs*. Pelo fato de o *mke2fs* comumente executar sobre um dispositivo de bloco, não sobre um arquivo comum, o *mke2fs* solicita uma confirmação:

```
% mke2fs -q /tmp/imagem-disco
mke2fs 1.18, 11-Nov-1999 for EXT2 FS 0.5b, 95/08/09
imagem-disco is not a block special device.
Proceed anyway? (y,n) y
```

A opção -q omite informação de sumário sobre o sistema de arquivos recentemente criado. Retire essa opção caso você desejar ver as informações de sumário. Agora imagem-disco contém um sistema de arquivos novinho em folha, como se esse sistema de arquivos tivesse sido suavemente incializado em um acionador de disco de 10MB.

[a]Nota do tradutor: o slackware vem atualmente com o *ext4* por padrão embora possa-se escolher entre outros como o próprio *ext2* e o *reiserfs*.

3. Monte o sistema de arquivos usando um dispositivo simulador. Para fazer isso, use o comando *mount*, especificando a imagem de disco como o dispositivo a ser montado. Também especifique loop=dispositivo-simulador como uma opção de montagem, usando a opção de montagem "-o" para dizer ao *mount* qual dispositivo simulador usar.

   Por exemplo, para montar nosso sistema de arquivos imagem-disco, use os comandos adiante. Lembrando, somente o superusuário pode usar um dispositivo simulador. O primeiro comando cria um diretório, /tmp/virtual-sa, a ser usado como ponto de montagem do sistema de arquivos virtual.

```
% mkdir /tmp/virtual-sa
% mount -o loop=/dev/loop0 /tmp/imagem-disco /tmp/virtual-sa
```

   Agora sua imagem de disco está montada como se fosse um acionador comum de disco de 10MB.

```
% df -h /tmp/virtual-sa
Filesystem     Size    Used Avail Use% Mounted on
/tmp/imagem-disco              9.7M     13k    9.2M      0% /tmp/virtual-sa
```

   Você pode usar essa imagem de disco como se fosse outro disco:

```
% cd /tmp/virtual-sa
% echo 'Al\^o, mundo!' > teste.txt
% ls -l
total 13
drwxr-xr-x         2 root         root             12288 Mar      8 02:00 lost+found
-rw-rw----         1 root         root 14 Mar     8 02:12 teste.txt
% cat teste.txt
Al\^o, mundo!
```

   Note que *lost+found* é um diretório que foi adicionado automaticamente pelo *mke2fs*.[a]

   Ao terminar, desmote o sistema de arquivos virtual.

```
% cd /tmp
% umount /tmp/virtual-sa
```

   Você pode apagar imagem-disco se você desejar, ou você pode montar imagem-disco mais tarde para acessar os arquivos no sistema de arquivos virtual. Você pode tambm copiar imagem-disco para outro computador e montar imagem-disco nesse mesmo outro computador o completo sistema de arquivos que você criou pois ele estará intacto.

[a] Se o sistema de arquivos for danificado, e algum dado for recuperado mas não associado a um arquivo, esse dado recuperado é colocado no *lost+found*.

Ao invés de criar um sistema de arquivos a partir do zero, você pode copiar um sistema de arquivos diretamente de um dispositivo. Por exemplo, você pode criar uma imagem do conteúdo de um CD-ROM simplesmente copiando esse mesmo CD-ROM a partir do dispositivo de CD-ROM.

Se você tiver um acionador de CD-ROM IDE, use o correspondente nome de dispositivo, tal como */dev/hda*, descrito anteriormente. Se você tiver um acionador de CD-ROM SCSI, o nome de dispositivo irá ser */dev/scd0* ou similar. Seu sistema pode também ter um link simbólico */dev/cdrom* que aponta para o dispositivo apropriado. Consulte seu arquivo */etc/fstab* para determinar qual dispositivo corresponde ao acionador de CD-ROM de seu computador.

Simplesmente copie o dispositivo para um arquivo. O arquivo resultante irá ser uma imagem de disco completa do sistema de arquivos sobre o CD-ROM no acionador por exemplo:

```
% cp /dev/cdrom /tmp/imagem-cdrom
```

Esse comando pode demorar muitos minutos, dependendo do CD-ROM que você estiver copiando e da velocidade de seu acionador de CD/DVD. O arquivo imagem resultante irá ser tão grande quanto grande for o conteúdo do CD-ROM/DVD.

Agora você pode montar essa imagem de CD-ROM/DVD sem ter o CD-ROM/DVD original no acionador. Por exemplo, para montar a imagem gravada no diretório */media/cdrom*, use a seguinte linha:

```
% mount -o loop=/dev/loop0 /tmp/imagem-cdrom /media/cdrom
```

Pelo fato de a imagem estar armazenada em um acionador de disco rígido, a referida imagem irá funcionar mais rapidamente que o acionador de disco de CD-ROM. Note que a maioria dos CD-ROMs usam o sistema de arquivos do tipo iso9660.

6.6 PTYs

Se você executar o comando *mount* sem argumentos de linha de comando, o que mostrará os sistemas de arquivos montados em seu sistema, você irá notar uma linha semelhante à seguinte [12]:

```
none on /dev/pts type devpts (rw,noexec,nosuid,gid=5,mode=0620)
```

[12]Nota do tradutor: atualizada usando ubunto 10.04.

Isso indica que um tipo especial de sistema de arquivos, *devpts*, está montado em */dev/pts*. Esse sistema de arquivos, que não está associado a nenhum dispositivo de *hardware*, é um sistema de arquivos "mágico" que é criado pelo *kernel* do GNU/Linux. Esse sistema de arquivos mágico é similar ao sistema de arquivos */proc*; veja Capítulo 7 para maiores informações sobre como esse sistema de arquivos mágico trabalha.

Da mesma forma que o diretório */dev*, o diretório */dev/pts* contém entradas correspondentes a dispositivos. Mas diferentemente do */dev*, que é um diretório comum, */dev/pts* é um diretório especial que é criado dinâmicamente pelo *kernel* do GNU/Linux. O conteúdo do diretório varia com o tempo e reflete o estado do sistema que está sendo executado.

As entradas em */dev/pts* correspondem a pseudo-terminais (ou pseudo-TTYs, ou PTYs). GNU/Linux cria um PTY para toda nova janela de terminal que você abre e mostra uma entrada correspondente em */dev/pts*. O dispositivo PTY atua como um dispositivo de terminal —— aceita entradas a partir do teclado e mostra saída no formato texto dos programas que executam nele. PTYs são numerados, e o número do PTY é o nome da correspondente entrada no /dev/pts.

Você pode mostrar o dispositivo de terminal associado a um processo usando o comando *ps*. Especifique *tty* como um dos campos de um formato personalizado com a opção "-o". Para mostrar o ID do processo, o TTY associado, e a linha de comando de cada processo compartilhando o mesmo terminal, use o comando "*ps -o pid,tty,cmd*".

6.6.1 Uma Demonstração de PTY

Por exemplo, você pode determinar o PTY associado a uma janela de terminal especificada chamando na janela respectiva o comando abaixo:

```
% ps -o pid,tty,cmd
  PID TT       CMD
28832 pts/4    bash
29287 pts/4    ps -o pid,tty,cmd
```

Essa janela particular de terminal está executando em PTY 4.

O PTY tem uma correspondente entrada em */dev/pts*:

```
% ls -l /dev/pts/4
crw--w----    1 samuel    tty      136,   4 Mar   8 02:56 /dev/pts/4
```

Note que o referido PTY é um dispositivo de caractere, e seu dono é o dono do processo para o qual ele foi criado.

Você pode ler de ou escrever para o dispositivo PTY. Se você lê do dispositivo PTY, você irá desviar a entrada do teclado que seria de outra forma enviada para o programa executando no PTY. Se você escreve para o dispositivo PTY, os dados irão aparecer naquela janela.

Tente abrir uma nova janela de terminal, e determine seu número de PTY digitando "*ps -o pid,tty,cmd*". De outra janela, escreva algum texto para o dispositivo do PTY. Por exemplo, se o número da nova janela de terminal for 7, use o comando abaixo de outra janela:

```
% echo 'Al\^o, outra janela!' > /dev/pts/7
```

A saída aparece na nova janela de terminal. Se você fecha a nova janela de terminal, a entrada 7 em */dev/pts* desaparece.

Se você usar o comando *ps* para determinar o TTY de um terminal virtual de modo texto (pressione Ctrl+Alt+F1 para mudar para o primeiro terminal virtual, por exemplo), você irá ver que esse primeiro terminal virtual está executando em um dispositivo de terminal comum ao invés de em um PTY:

```
% ps -o pid,tty,cmd
  PID TT       CMD
29325 tty1     -bash
29353 tty1     ps -o pid,tty,cmd
```

6.7 A chamada de sistema *ioctl*

A chamada de sistema *ioctl* é uma interface de propósito geral para controlar dispositivos de *hardware*. O primeiro argumento a *ioctl* é um descritor de arquivo, o qual deve ser aberto para o dispositivo que você deseja controlar. O segundo argumento é um código de requisição que indica a operação que você deseja executar. Vários códigos de requisição estão disponíveis para diferentes dispositivos. Dependendo do código de requisição, pode existir argumentos adicionais fornecendo dados a *ioctl*.

Muitos desses códigos de requisição disponíveis para vários dispositivos estão listados na página de manual de *ioctl_list*[13]. O uso de *ioctl* geralmente requer um entendimento detalhado do acionador de dispositivo correspondente ao dispositivo de *hardware* que você deseja controlar. A maioria desses dispositivos é um pouco especializado e estão além do objetivo desse livro. Todavia, iremos mostrar um exemplo para dar a você uma degustação de como *ioctl* é usada.

[13]Nota do tradutor:*man ioctl_list*.

Listagem 6.2: (cdrom-eject.c) Ejeta um CD-ROM/DVD

```
1 #include <fcntl.h>
2 #include <linux/cdrom.h>
3 #include <sys/ioctl.h>
4 #include <sys/stat.h>
5 #include <sys/types.h>
6 #include <unistd.h>
7
8 int main (int argc, char* argv[])
9 {
10   /* Abre um descritor de arquivo para o dispositivo especificado na linha de comando
        . */
11   int fd = open (argv[1], O_RDONLY);
12   /* Ejeta o CD-ROM. */
13   ioctl (fd, CDROMEJECT);
14   /* Fecha o descritor de arquivo. */
15   close (fd);
16
17   return 0;
18 }
```

A Listagem 6.2 mostra um programa curto que ejeta o disco em um acionador de CD-ROM/DVD (se o acionador suportar isso). O programa recebe um único argumento de linha de comando, o acionador de dispositivo do CD-ROM. O programa abre um descritor de arquivo para o dispositivo e chama *ioctl* com o código de requisição CDROMEJECT. Essa requisição, definida no arquivo de cabeçalho <linux/cdrom.h>, instrui o dispositivo a ejetar o disco.

Por exemplo, se seu sistema tiver um acionador de CD-ROM/DVD IDE conectado como dispositivo mestre na placa controladora IDE secundária, o dispositivo correspondente é */dev/hdc*. Para ejetar o disco do acionador, use o comando adiante:

```
% ./cdrom-eject /dev/hdc
```

Capítulo 7

O Sistema de Arquivos */proc*

CHAMAR O COMANDO *mount* SEM ARGUMENTOS – irá mostrar os sistemas de arquivo montados atualmente em seu computador GNU/Linux. Você irá ver uma linha que se parece com a seguinte[1]:

```
none on /proc type proc (rw)
```

Esse é o sistema de arquivo especial */proc*. Note que o primeiro campo, *none*, indica que esse sistema de arquivo não é associado com um dispositivo de *hardware* tal como um acionador de disco. Ao invés disso, */proc* é uma janela para dentro do *kernel* do GNU/Linux que está executando. Arquivos no sistema de arquivo */proc* não correspondem a arquivos atuais em um dispositivo físico. Ao invés disso, eles são objetos mágicos que comportam-se como arquivos mas fornecem acesso a parâmetros, estruturas de dados, e estatísticas no *kernel*. O "conteúdo" desses arquivos não são sempre blocos fixos de dados, como os conteúdos dos arquivos comuns. Ao invés disso, eles são gerados instantâneamente pelo *kernel* do GNU/Linux quando você lê de um arquivo. Você também pode mudar a configuração do *kernel* que está sendo executado escrevendo em certos arquivos no sistema de arquivo */proc*.

Vamos olhar um exemplo:

```
% ls -l /proc/version
-r--r--r--    1 root       root             0 Jan 17 18:09 /proc/version
```

Note que o tamanho do arquivo é zero; pelo fato de os conteúdos dos arquivos serem gerados pelo *kernel*, o conceito de tamanho de arquivo não é aplicável. Também, se você tentar esse comando propriamente dito, você irá notar que a hora de modificação corresponde à hora atual.

Qual o conteúdo desse arquivo? O conteúdo de */proc/version* consiste de uma sequência de caracteres descrevendo o número de versão do *kernel*

[1]Nota do tradutor: no slackware 13.1 padrão a linha é "proc on /proc type proc (rw)".

do GNU/Linux. O */proc/version* contém a informação de versão que pode ser obtida por meio da chamada de sistema *uname*, descrita no Capítulo 8,"Chamadas de Sistema do GNU/Linux" na Seção 8.15, "A chamada de Sistema *uname*" acrescentando informações adicionais tais como a versão do compilador que foi usado para compilar o *kernel*. Você pode ler de */proc/version* como você leria de qualquer outro arquivo. Por exemplo, um caminho fácil para mostrar o conteúdo do */proc/version* é com o comando *cat*[2].

```
% cat /proc/version
Linux version 2.2.14-5.0 (root@porky.devel.redhat.com) (gcc version egcs-2.91.
66 19990314/Linux (egcs-1.1.2 release)) \#1 Tue Mar 7 21:07:39 EST 2000
```

As várias entradas no sistema de arquivo */proc* são descritas extensivamente na página de manual do proc (Seção 5). Para visualizar a descrição, chame o comando:

```
% man 5 proc
```

Nesse capítulo, nós iremos descrever alguns dos recursos do sistema de arquivo */proc* os quais estão em sua maioria feitos para serem úteis a programadores de aplicações, e nós iremos fornecer exemplos de como usá-los. Alguns dos recursos do */proc* estão disponíveis para depuração, também.

Se você está interessado em exatamente como */proc* trabalha, dê uma olhada nos códigos fonte *kernel* do GNU/Linux, no diretório */usr/src/linux/fs/proc/*.

7.1 Extraindo Informação do */proc*

A maioria das entradas no */proc* fornece informações formatadas para serem legíveis a humanos, mas os fomatos são simples o suficiente para serem facilmente fornecidos a programas. Por exemplo, */proc/cpuinfo* contém informação sobre a CPU[3] do sistema (ou CPUs, para uma máquina com vários processadores). A saída é uma tabela de valores, um valor por linha, com uma descrição do valor e um dois pontos precedendo cada valor.

Por exemplo, a saída pode se parecer como segue[4]:

[2]Nota do tradutor: no slackware 13.1 temos: Linux version 2.6.33.4-smp (root@midas) (gcc version 4.4.4 (GCC)) #2 SMP Wed May 12 22:47:36 CDT 2010.

[3]Nota do tradutor: *Central Processing Unit*.

[4]Nota do tradutor: veja o Apêndice G na Seção G.1 para maiores detalhes.

```
% cat /proc/cpuinfo
processor       :0
vendor_id       : GenuineIntel
cpu family      :6
model           :5
model name      : Pentium II (Deschutes)
stepping        :2
cpu MHz         : 400.913520
cache size      : 512 KB
fdiv_bug        : no
hlt_bug         : no
sep_bug         : no
f00f_bug        : no
coma_bug        : no
fpu             : yes
fpu_exception   : yes
cpuid level     :2
wp              : yes
flags           : fpu vme de pse tsc msr pae mce cx8
apic sep mtrr pge mca cmov pat pse36 mmx fxsr
bogomips        : 399.77
```

Iremos descrever a interpretação de alguns desses campos na Seção 7.3.1, "Informações sobre a CPU".

Um caminho simples para extrair um valor dessa saída é ler o arquivo e colocá-lo em uma área de armazenamento temporário e analisá-lo em memória usando *sscanf*. A Listagem 7.1 mostra um exemplo disso. O programa inclui a função *get_cpu_clock_speed* que lê de */proc/cpuinfo* que está na memória e extrai a primeira velocidade do *clock* da CPU.

Listagem 7.1: (clock-speed.c) Extraindo a Velocidade de *Clock* da CPU de */proc/cpuinfo*

```
1 #include <stdio.h>
2 #include <string.h>
3
4 /* Retorna a velocidade do clock da CPU em MHz, como reportado por
5    /proc/cpuinfo. Em uma m quina com v rios processadores, retorna a velocidade da
6    primeira CPU. Em caso de erro retorna zero. */
7
8 float get_cpu_clock_speed ()
9 {
10   FILE* fp;
11   char buffer[1024];
12   size_t bytes_read;
13   char* match;
14   float clock_speed;
15
16   /* L  todo o conte do de /proc/cpuinfo para um espa o tempor rio de
        armazenamento. */
17   fp = fopen ("/proc/cpuinfo", "r");
18   bytes_read = fread (buffer, 1, sizeof (buffer), fp);
19   fclose (fp);
20   /* Reclama se a leitura falhar ou se o espa o tempor rio de armazenamento n o
        for grande o suficiente. */
21   if (bytes_read == 0 || bytes_read == sizeof (buffer))
22     return 0;
23   /* Acrescenta do caractere de termina  o de texto NUL. */
24   buffer[bytes_read] = '\0';
25   /* Localiza a linha que inicia-se com "cpu MHz". */
26   match = strstr (buffer, "cpu MHz");
27   if (match == NULL)
28     return 0;
29   /* Informa a linha da qual se extrai a velocidade de clock. */
30   sscanf (match, "cpu MHz : %f", &clock_speed);
31   return clock_speed;
32 }
33
34
35 int main ()
36 {
37   printf ("Velocidade de clock da CPU: %4.0f MHz\n", get_cpu_clock_speed ());
38   return 0;
39 }
```

Seja informado, todavia, que os nomes, as semânticas, e formatos de entradas no sistema de arquivo */proc* podem mudar em novas revisões de *kernel* do GNU/Linux. Se você usá-lo em um programa, você deve garantir que o comportamento do programa se desatualiza se a entrada do */proc* for retirada ou estiver formatada de forma inesperada.

7.2 Entradas dos Processos

O sistema de arquivo */proc* contém uma entrada de diretório para cada processo executando no sistema GNU/Linux. O nome de cada diretório é o ID de processo do processo correspondente[5]. Esses diretórios aparecem e desaparecem dinâmicamente à medida que processos iniciam e encerram no sistema. Cada diretório contém muitas entradas fornecendo acesso a informações sobre o precsso que está executando. Foi a partir desses diretórios de processos

[5]Em alguns sistemas UNIX, os IDs de processo são completados com zeros. No GNU/Linux, eles não são.

que o sistema de arquivos */proc* recebeu seu nome.

Cada diretório de processo contém as seguintes entradas[6]:

- *cmdline* contém a lista de argumentos para o processo. A entrada *cmdline* é descrita na Seção 7.2.2, "Lista de Argumentos do Processo".

- *cwd* é um link simbólico que aponta para o diretório atual de trabalho do processo (como escolhido, por exemplo, com a chamada *chdir*).

- *environ* contém o ambiente do processo. A entrada *environ* é descrita na Seção 7.2.3, "Ambiente de Processo".

- *exe* é um link simbólico que aponta para a imágem executável rodando no processo. A entrada *exe* é descrita na Seção 7.2.4, "Executável do Processo".

- *fd* é um subdiretório que contém entradas para os descritores abertos pelo processo. Essas entradas são descritas na Seção 7.2.5, "Descritores de Arquivo do Processo".

- *maps* mostra informação sobre arquivos mapeados dentro da área de endereçamento de memória do processo. Veja o Capítulo 5, "Comunicação Entre Processos" Seção 5.3, "Arquivos Mapeados em Memória" para detalhes de como arquivos mapeados em memória trabalham, como mapas mostram o intervalo de endereçamento no espaço de endereçamento do processo dentro do qual o arquivo é mapeado, as permissões desses endereços, o nome do arquivo, e outras informações.

  A tabela de mapeamento para cada processo mostra o executável rodando no processo, qualquer biblioteca compartilhada carregada, e outros arquivos que o processo tenha mapeado.

- *root* um link simbólico para o diretório principal desse processo. Comumente, esse link aponta para o diretório "/" o diretório raíz do sistema. O diretório principal de um processo pode ser modificado usando a chamada de sistema *chroot* ou o comando *chroot*[a].

[a] A chamada *chroot* e o comando *chroot* estão fora do escopo desse livro. Veja a página de manual do*chroot* na seção 1 para informação sobre o comando (chame *man 1 chroot*), ou a página de manual na Seção 2 (chame *man 2 chroot*) para informações sobre a chamada de sistema.

[6] Nota do tradutor: veja o Apêndice G na Seção G.2 para a listagem de outras entradas.

- *stat* contém muitas informações de stuação atual e estatística sobre o processo. Esses dados são os mesmos dados apresentados na entrada *status*, mas no formato de linha numérada, todos em uma única linha. O formato é difícil para ler mas pode ser mais adequado para informar a programas. Se você desejar usar a entrada *stat* em seus programas, veja a página de manual do *proc* a qual descreve seu conteúdo, chamando "*man 5 proc*".

- *statm* contém informação sobre a memória usada pelo processo. A entrada *statm* é descrita na Seção 7.2.6, "Estatísticas da Memória do Processo".

- *status* contém grande quantidade de informação sobe a situação atual e informação estatística sobre o processo, formatados para serem compreensíveis a humanos. A Seção 7.2.7, "Estatísticas de Processo" contém uma descrição da entrada *status*.

- *cpu* aparece somente em *kernels* GNU/Linux SMP. A entrada *cpu* contém um fracionamento de tempo de processo (usuário e sistema) pela CPU.

Note que por razões de segurança, as permissões de algumas entradas são posicionadas de forma que somente o usuário que é dono do processo ou o super-usuário) pode acessá-las.

7.2.1 /proc/self

Uma entrada adicional no sistema de arquivo */proc* torna fácil para um programa usar */proc* para encontrar informação sobre seu próprio processo. A entrada */proc/self* é um link simbólico para o diretório do */proc* correspondente ao processo atual. O objetivo do link */proc/self* depende de qual processo olha para o link simbólico */proc/self*: Cada processo vê seu próprio diretório de processo como alvo do link.

Por exemplo, o programa na Listagem 7.2 lê o alvo do link */proc/self* para determinar seu ID de processo. (Estamos fazendo isso dessa maneira para propósitos ilustrativos somente; chamando a função *getpid*, descrita no Capítulo 3, "Processos" na Seção 3.1.1, "Identificadores de Processos" está uma forma muito fácil para fazer a mesma coisa.) O programa a seguir usa a chamada de sistema *readlink*, descrita na Seção 8.11, "readlink: Lendo Links Simbólicos" para extrair o alvo do link simbólico.

Listagem 7.2: (get-pid.c) Obtendo o ID de Processo de */proc/self*

```
1 #include <stdio.h>
2 #include <sys/types.h>
3 #include <unistd.h>
4
5 /* Retorna o id de processo dos processos camados, como determinado a partir
6    do link simb lico /proc/self.  */
7
8 pid_t get_pid_from_proc_self ()
9 {
10   char target[32];
11   int pid;
12   /* L   o alvo do link simb lico.  */
13   readlink ("/proc/self", target, sizeof (target));
14   /* O alvo   um diret rio cujo nome   o id do processo.  */
15   sscanf (target, "%d", &pid);
16   return (pid_t) pid;
17 }
18
19 int main ()
20 {
21   printf ("/proc/self mostra o id de processo %d\n",
22           (int) get_pid_from_proc_self ());
23   printf ("getpid() mostra o id de processo %d\n", (int) getpid ());
24   return 0;
25 }
```

7.2.2 Lista de Argumentos do Processo

A entrada *cmdline* contém a lista de argumentos de um processo (veja o Capítulo 2,"Escrevendo Bom Software GNU/Linux" Seção 2.1.1,"A Lista de Argumentos"). Os argumentos são mostrados como uma única sequência de caracteres, com os argumentos separados por NULs. A maioria das funções de sequência de caractere esperam que toda a sequência de caracteres seja terminada por um NUL único e não irá controlar NULs embutidos dentro da sequência de caracteres, de forma que você irá ter que controlar o conteúdo especialmente.

NUL vs. NULL
NUL é um caractere com valor inteiro 0. Esse caractere é diferente do caractere *NULL*, que é um apontador com valor 0. Na linguagem C, uma sequência de caracteres é comumente terminada com um caratere NUL. Por exemplo, a sequência de caracteres "Hello, world!" ocupa 14 *bytes*[a] pelo fato de existir um NUL implícito após o ponto de exclamação indicando o final da sequência de caracteres.
NULL, por outro lado, é um valor de apontador que você pode estar certo que nunca corresponderá a um endereço de memória real em seu programa.
Em C e em C++, NUL é representado como a constante do tipo caractere '\0', ou (char) 0. A definição de *NULL* difere entre sistemas operacionais; em GNU/Linux, o caractere *NULL* é definido como ((void*)0) em C e simplesmente 0 em C++.

[a]Nota do tradutor: o espaço, a vírgula e a exclamação contam como "letras" e cada letra ocupa um *byte*. Se seu computador tiver um processador de 32 *bits* cada *byte* tem o tamanho de 32 *bits*, idem para o computador de 64 *bits*. Cada *bit* corresponde ao dígito binário e só pode assumir dois valores: 0 e 1.

Na Seção 2.1.1, mostramos um programa na Listagem 2.1 que mostrava na tela sua própria lista de argumentos. Usando as entradas de *cmdline* no sistema de arquivo */proc*, podemos implementar um programa que mostra os argumentos de outro processo. A Listagem 7.3 é o tal programa; A Listagem 7.3 mostra na tela a lista de argumentos do processo com o ID de processo especificado. Pelo fato de poderem existir muitos NULs no conteúdo da entrada *cmdline* em lugar de um único NUL no final, podemos determinar o comprimento da sequência de caracteres com a função *strlen* (que simplesmente conta o número de caracteres até encontrar um NUL). Ao invés disso, determinamos o comprimento da da leitura da entrada *cmdline*, o qual retorna o número de *bytes* que foram lidos.

Listagem 7.3: (print-arg-list.c) Mostra na Tela a Lista de Arguentos de um Processo que está Executando

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/stat.h>
5 #include <sys/types.h>
6 #include <unistd.h>
7
8 /* Mostra na tela a lista de argumentos, um argumento por linha, do processo
9    fornecido por PID.  */
10
11 void print_process_arg_list (pid_t pid)
12 {
13   int fd;
14   char filename[24];
15   char arg_list[1024];
16   size_t length;
17   char* next_arg;
18
19   /* Gera o nome do arquivo de cmdline para o processo. */
20   snprintf (filename, sizeof (filename), "/proc/%d/cmdline", (int) pid);
21   /* L   o conte do do arquivo. */
22   fd = open (filename, O_RDONLY);
23   length = read (fd, arg_list, sizeof (arg_list));
24   close (fd);
25   /* read n o coloca a termina  o de sequ ncia de caractere no espa o
        tempor rio de
26    armazenamento, de forma que isso    feito aqui. */
27   arg_list[length] = '\0';
28   /* Ciclo sobre argumentos.  Argumentos s o separados por NULs. */
29   next_arg = arg_list;
30   while (next_arg < arg_list + length) {
31     /* Mostre o argumento.  Cada argumento    NUL-terminado, ent o apenas tratamos
32        esses argumentos como uma sequ ncia de caracteres comum.  */
33     printf ("%s\n", next_arg);
34     /* Avance para o argumento seguinte.  Uma vez que cada argumento
35        NUL-terminado, strlen conta o comprimento do argumento seguinte,
36        n o da lista de argumentos completa.  */
37     next_arg += strlen (next_arg) + 1;
38   }
39 }
40
41 int main (int argc, char* argv[])
42 {
43   pid_t pid = (pid_t) atoi (argv[1]);
44   print_process_arg_list (pid);
45   return 0;
46 }
```

Por exemplo, suponhamos que o processo 372 seja o programa que trabalha em segundo plano chamado *system logger*, isto é, o *syslogd*.

```
% ps 372
   PID TTY      STAT   TIME COMMAND
   372 ?        S      0:00 syslogd -m 0
% ./print-arg-list 372
syslogd
-m
0
```

Nesse caso, *syslogd* foi chamado com os argumentos "*-m 0*".

7.2.3 Ambiente de Processo

A entrada *environ* contém uma descrição do ambiente do processo (veja a Seção 2.1.6, "O Ambiente"). Da mesma forma que a entrada *cmdline*, as variáveis de memória que descrevem o ambiente individual são separadas por NULs. O formato de cada elemento é o mesmo que o formato usado na variável de ambiente, isto é, VARIÁVEL=valor.

A listagem 7.4 mostra uma generalização do programa na Listagem 2.4 na Seção 2.1.6. Essa versão recebe um número de ID de processo em sua linha de comando e mostra o ambiente para aquele processo lendo essa informação a partir do */proc*.

Listagem 7.4: (print-environment.c) Mostra o Ambiente de um Processo

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/stat.h>
5 #include <sys/types.h>
6 #include <unistd.h>
7
8 /* Mostra o ambiente, uma vari vel de ambiente por linha,
9    do processo fornecido por PID.  */
10
11 void print_process_environment (pid_t pid)
12 {
13   int fd;
14   char filename[24];
15   char environment[8192];
16   size_t length;
17   char* next_var;
18
19   /* Gera o nome do arquivo de ambiente para o processo. */
20   snprintf (filename, sizeof (filename), "/proc/%d/environ", (int) pid);
21   /* Read the contents of the file. */
22   fd = open (filename, O_RDONLY);
23   length = read (fd, environment, sizeof (environment));
24   close (fd);
25   /* read n o NUL-termina o espa o tempor rio de armazenamento, ent o fa a isso
        aqui. */
26   environment[length] = '\0';
27
28   /* Ciclo sobre vari veis.  Vari veis s o separadas por NULs. */
29   next_var = environment;
30   while (next_var < environment + length) {
31     /* Mostre a vari vel.  Cada vari vel    NUL-terminada, ent o apenas tratamos
32        cada vari vel como uma sequ ncia de caracteres comum.  */
33     printf ("%s\n", next_var);
34     /* Avance para a vari vel seguinte.  Uma vez que cada vari vel
35        NUL-terminada, strlen conta o comprimento da vari vel seguinte,
36         n o da lista inteira de vari vels.  */
37     next_var += strlen (next_var) + 1;
38   }
39 }
40
41 int main (int argc, char* argv[])
42 {
43   pid_t pid = (pid_t) atoi (argv[1]);
44   print_process_environment (pid);
45   return 0;
46 }
```

7.2.4 O Executável do Processo

A entrada *exe* aponta para o arquivo executável sendo rodado em um processo. a Seção 2.1.1, explanamos que tipicamente o nome do programa exe-

cutável é informado como o primeiro elemento da lista de argumentos. Note, apesar disso, que isso é puramente uma convenção; um programa pode ser chamado com qualquer lista de argumentos. Usando a entrada *exe* no sistema de arquivo */proc* é um caminho mais seguro para determinar qual executável está rodando.

Uma técnica útil é extrair o caminho contendo o executável a partir do sistema de arquivo */proc*. Para muitos programas, arquivos auxiliares são instalados em diretórios com caminhos conhecidos relativamente ao executável do programa principal, de forma que é necessário determinar onde aquele executável principal atualmente está. A função *get_executable_path* na Listagem 7.5 determina o caminho do executável que está rodando no processo que está chamando examinando a link simbólico */proc/self/exe*.

Listagem 7.5: (get-exe-path.c) Pega o Caminho do Programa Executando Atualmente

```
1 #include <limits.h>
2 #include <stdio.h>
3 #include <string.h>
4 #include <unistd.h>
5
6 /* Encontra o caminho contendo o programa executavel atualmente rodando.
7    O caminho     colocado dentro da variavel BUFFER, a qual possui comprimento LEN.
8    Retorna o nmero de caracteres no caminho, ou -1 em caso de erro.  */
9
10 size_t get_executable_path (char* buffer, size_t len)
11 {
12   char* path_end;
13   /* L   o alvo de /proc/self/exe.  */
14   if (readlink ("/proc/self/exe", buffer, len) <= 0)
15     return -1;
16   /* Encontra a  ltima ocorrncia de uma barra invertida, o separador de caminho.
        */
17   path_end = strrchr (buffer, '/');
18   if (path_end == NULL)
19     return -1;
20   /* Advance to the character past the last slash.  */
21   ++path_end;
22   /* Obt  o diretrio contendo o programa por quebra do
23      caminho ap s a  ltima barra.  */
24   *path_end = '\0';
25   /* O comprimento do caminho     o nmero de caracteres do incio at  a
26       ltima barra.  */
27   return (size_t) (path_end - buffer);
28 }
29
30 int main ()
31 {
32   char path[PATH_MAX];
33   get_executable_path (path, sizeof (path));
34   printf ("esse programa est  no diretrio %s\n", path);
35   return 0;
36 }
```

7.2.5 Descritores de Arquivo do Processo

A entrada *fd* é um subdiretório que contém entradas para os descritores de arquivo abertos por um processo. Cada entrada é um link simbólico para o arquivo ou dispositivo aberto indicado pelo respectivo descritor de arquivo. Você pode escrever para ou ler desses links simbólicos; esses descritores de

arquivo escrevem para ou leem do correspondente arquivo ou dispositivo aberto no processo alvo. As entradas no subdiretório *fd* são chamadas pelos números dos descritores de arquivo.

Aqui está um artifício que você pode tentar com entradas *fd* no */proc*. Abra uma nova janela, e encontre o ID de processo do processo que está rodando o *shell* usando o comando *ps*.

```
% ps
  PID TTY          TIME CMD
 1261 pts/4    00:00:00 bash
 2455 pts/4    00:00:00 ps
```

Nesse caso, o *shell* (*bash*) está rodando no processo 1261. Agora abra uma segunda janela, e olhe o conteúdo do subdiretório *fd* para aquele processo.

```
% ls -l /proc/1261/fd
total 0
lrwx------    1 samuel     samuel          64 Jan 30 01:02 0 -> /dev/pts/4
lrwx------    1 samuel     samuel          64 Jan 30 01:02 1 -> /dev/pts/4
lrwx------    1 samuel     samuel          64 Jan 30 01:02 2 -> /dev/pts/4
```

(Pode haver outras linhas de saída correspondendo a outros descritores de arquivos abertos também.) Relembrando o que mencionamos na Seção 2.1.4, "E/S Padrão" que descritores de arquivo 0, 1, e 2 são inicializados para entrada padrão, saída padrão, e saída de erro, respectivamente. Dessa forma, por meio de escrita para */proc/1261/fd/1*, você pode escrever para o dispositivo anexado a *stdout*[7] para o processo do *shell* – nesse caso, o pseudo TTY na primeira janela. Na segunda janela, tente escrever uma mensagem para aquele arquivo:

```
% echo "Al\^o, mundo." >> /proc/1261/fd/1
```

O texto aparece na primeira janela.

Descritores de arquivo ao lado de entrada padrão, saída padrão, e saída de erro aparecem no subdiretório *fd*, também. A Listagem 7.6 mostra um programa que simplesmente abre um descritor de arquivo para um arquivo especificado na linha de comando e então entra em um laço para sempre.

[7]Nota do tradutor: saída padrão

Listagem 7.6: (open-and-spin.c) Abre um Arquivo para Leitura

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <sys/stat.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6
7 int main (int argc, char* argv[])
8 {
9   const char* const filename = argv[1];
10   int fd = open (filename, O_RDONLY);
11   printf ("no processo %d, o descritor de arquivo %d est  aberto para %s\n",
12           (int) getpid (), (int) fd, filename);
13   while (1);
14   return 0;
15 }
```

Tente executar o programa em uma janela:

```
% ./open-and-spin /etc/fstab
in process 2570, file descriptor 3 is open to /etc/fstab
```

Em outra janela, dê uma olhada no subdiretório *fd* que corresponde a esse processo no */proc*.

```
% ls -l /proc/2570/fd
total 0
lrwx------     1 samuel   samuel       64 Jan 30 01:30 0 -> /dev/pts/2
lrwx------     1 samuel   samuel       64 Jan 30 01:30 1 -> /dev/pts/2
lrwx------     1 samuel   samuel       64 Jan 30 01:30 2 -> /dev/pts/2
lr-x------     1 samuel   samuel       64 Jan 30 01:30 3 -> /etc/fstab
```

Note a entrada para o descritor de arquivo 3, apontando para o arquivo */etc/fstab* aberto sobre esse descritor.

Descritores de arquivo podem ser abertos sobre *sockets* ou sobre *pipes*, também (veja Capítulo 5 para maiores informações sobre isso). Em tal caso, o alvo do link simbólico correspondente ao descritor de arquivo irá para o estado "*socket*" ou "*pipe*" ao invés de apontar para um arquivo comum ou dispositivo.

7.2.6 Estatísticas de Memória do Processo

A entrada *statm* contém uma lista de sete números, separados por espaços. Cada número é um contador do número de páginas de memória usadas pelo processo em uma categoria em particular. As categorias, na ordem em que os números aparecem, são listadas aqui:

- O tamanho total do processo
- O tamanho do processo residente em memória física

- A memória compartilhada com outros processos, isto é, memória mapeada ambas por esse processo e ao menos um outro (tais como bibliotecas compartilhadas ou páginas de memória intocadas do tipo copie-na-escrita)[8]
- O tamanho do texto do processo, isto é, o tamanho do código do executável carregado
- O tamanho das bibliotecas compartilhadas mapeadas dentro do espaço de memória desse processo
- A memória usada por esse processo para sua pilha
- O número de páginas sujas, isto é, páginas de memória que tenham sido modificadas pelo programa

7.2.7 Estatísticas de Processo

A entrada *status* contém uma variedade de informação sobre o processo, formatada para ser compreensível por humanos. Entre essa variedade está o ID do processo e o ID do processo pai, os IDs reais e os IDs efetivos de usuário e do grupo, uso de memória, e máscaras de *bits* especificando quais sinais são capturados, ignorados, e bloqueados.

7.3 Informações de *Hardware*

Muitas das outras entradas no sistema de arquivo */proc* fornecem acesso a informações sobre o *hardware* do sistema. Embora esses sejam tipicamente de interesse a configuradores do sistema e a administradores, a informação pode ocasionalmente ser do interesse para programadores de aplicação também. Iremos mostrar algumas das entradas mais úteis aqui.

7.3.1 Informações sobre a CPU

Como mostrado anteriormente, */proc/cpuinfo* contém informações a CPU ou CPUs que estão executando o sistema GNU/Linux. O campo *Processor* lista o número do processador; esse campo é 0 para sistemas de um único processador. O fabricante, a Família da CPU, o Modelo, e os campos *Stepping* habilitam você a determinar o exato modelo e a revisão da CPU. Mais útil, os campos *Flags* mostram quais sinalizadores de CPU estão escolhidos, o

[8]Nota do tradutor: relembrando a Subseção 5.3.4 "Mapeamentos Privados".

que indica os recursos disponíveis nessa CPU. Por exemplo, "mmx" indica a disponibilidade das instruções extendidas MMX. [9]

A maioria das informações retornadas por */proc/cpuinfo* é derivada da instrução *assembly x86 cpuid*. Essa instrução é o mecanismo de baixo nível por meio do qual um programa obtém informação sobre a CPU. Para um grande entendimento da saída de */proc/cpuinfo*, veja a documentação da instrução *cpuid* no Manual do Desenvolvedor de Software da Arquitetura Intel IA-32, Volume 2: Instruction Set Reference. Esse manual etá disponível em `http://developer.intel.com/design` [10].

O último elemento, *bogomips*, é um valor específico do GNU/Linux. Esses *bogomips* são uma medida da velocidade do processador em torno de um laço restrito e sendo portanto um indicador um pouco pobre da velocidade global do processador.

7.3.2 Informação de Dispositivos

O arquivo */proc/devices* lista os números de dispositivo principal para dispositivos de bloco e de caractere disponíveis para o sistema. Veja o Capítulo 6, "Dispositivos" para informações sobre tipos de dispositivos e números de dispositivo.

7.3.3 Informação de Barramento

O diretório */proc/bus* abriga as informações dos dispositivos anexados ao sistema via placas de expansão e portas *usb* e pode também incluir dispositivos localizados na placa mãe. Os comando *hal-device*, *lspci* e o *lsusb*, fornecem as informações dos dispositivos *pci*, *pci-express* e *usb* anexados ao sistema[11].

7.3.4 Informações de Porta Serial

O arquivo */proc/tty/driver/serial* lista informações de configuração e estatísticas sobre portas seriais. Portas seriais são numeradas a partir de 0[12]. Informação de configuração sobre portas seriais podem também ser obtidas,

[9] Veja o Manual do Desenvolvedor de Software da Arquitetura Intel IA-32 para documentação sobre instruções MMX, e veja o Capítulo 9, "Código Assembly Embutido" nesse livro a título de fornecer informação sobre como usar essas e outras instruções especiais *assembly* em programas GNU/Linux.

[10] Nota do tradutor:`http://www.intel.com/Assets/PDF/manual/253668.pdf`.

[11] Nota do tradutor: esse trecho foi completamente reescrito uma vez que o arquivo */proc/pci* não existe nas versões mais recentes do *kernel*.

[12] Note que sob e Windows, portas seriais são numeradas a partir de 1, de forma que COM1 corresponde a prota serial número 0 em GNU/Linux.

bem como modificadas, usando o comando *setserial*. Todavia, */proc/tty/driver/serial* mostra estatísticas adicionais sobre cada contagem de iterrupção de porta serial.

Por exemplo, a linha a seguir de */proc/tty/driver/serial* pode descrever a porta serial 1 (que deve ser a COM2 em Windows):

```
1: uart:16550A port:2F8 irq:3 baud:9600 tx:11 rx:0
```

Isso indica que a porta serial está executando através de um *chip* tipo *16550A UART*, usa a porta de entrada e saída 0x2f8 e a IRQ 3 para comunicações, e possui a velocidade de 9,600 *baud*. Através da porta serial trafegou 11 interrupções de transmissão e 0 interrupções de recepção.

Veja a seção 6.4, "Dispositivos de *Hardware*" para informações sobre dispostivos seriais.

7.4 Informação do *Kernel*

Muitas das entradas no */proc* fornecem acesso a informações sobre a configuração e o estado do *kernel* que está sendo executado. Algumas dessas entradas estão no nível mais alto do */proc*; outras entradas encontram-se em */proc/sys/kernel*.

7.4.1 Informação de versão

O arquivo */proc/version* contém uma longa sequência de caracteres descrevendo o número de versão do *kernel* e a versão de compilação. O */proc/version* também inclui informação sobre como o *kernel* foi construído: o usuário que o compilou, a máquina na qual foi compilado, a data em que foi feita a compilação, e a versão do compilador que foi usado para fazer a compilação – por exemplo[13]:

```
%cat /proc/version
Linux version 2.2.14-5.0 (root@porky.devel.redhat.com) (gcc version
egcs-2.91.66 19990314/Linux (egcs-1.1.2 release)) \#1 Tue Mar 7
21:07:39 EST 2000
```

A saída acima indica que o sistema está executando um *release* 2.2.14 do *kernel* do GNU/Linux, que foi compilado com o EGCS release 1.1.2. (EGCS, *Experimental GNU Compiler System*, que foi o precursos do atual projeto GCC.)

Os mais importantes itens nessa saída, o nome do sistema operacional e a versão do *kernel* e a revisão do mesmo, estão disponíveis em entradas

[13]Nota do tradutor: veja um outro exemplo no Apêndice G Seção G.3.

separadas do */proc* também. As entradas do */proc* são respectivamente: */proc/sys/kernel/ostype*, */proc/sys/kernel/osrelease*, e */proc/sys/kernel/version*.

```
% cat /proc/sys/kernel/ostype
Linux
% cat /proc/sys/kernel/osrelease
2.2.14-5.0
% cat /proc/sys/kernel/version
#1 Tue Mar 7 21:07:39 EST 2000
```

7.4.2 Nome do Host e Nome de Domínio

As entradas */proc/sys/kernel/hostname* e */proc/sys/kernel/domainname* carregam o nome de host do computador e o nome de domínio, respectivamente. Essa informação é a mesma retornada pela chamada de sistema *uname*, desccrita na Seção 8.15.

7.4.3 Utilização da Memória

A entrada */proc/meminfo* contém informação sobre o uso da memória do sistema. A informação está presente para ambos memória física e espaço *swap*. Por exemplo:

```
$ cat /proc/meminfo
MemTotal:        1995692 kB
MemFree:         1341280 kB
Buffers:          120888 kB
Cached:           289868 kB
SwapCached:            0 kB
Active:           263144 kB
Inactive:         285124 kB
Active(anon):     141712 kB
Inactive(anon):       16 kB
Active(file):     121432 kB
Inactive(file):   285108 kB
Unevictable:           0 kB
Mlocked:               0 kB
HighTotal:       1187464 kB
HighFree:         729740 kB
LowTotal:         808228 kB
LowFree:          611540 kB
SwapTotal:       5277304 kB
SwapFree:        5277304 kB
Dirty:                80 kB
Writeback:             0 kB
AnonPages:        137516 kB
Mapped:            52728 kB
Shmem:              4212 kB
Slab:              46616 kB
SReclaimable:      37868 kB
SUnreclaim:         8748 kB
KernelStack:        2088 kB
PageTables:         4108 kB
NFS_Unstable:          0 kB
Bounce:                0 kB
WritebackTmp:          0 kB
CommitLimit:     6275148 kB
Committed_AS:     518892 kB
VmallocTotal:     122880 kB
VmallocUsed:       76848 kB
VmallocChunk:      34812 kB
HardwareCorrupted:     0 kB
HugePages_Total:       0
HugePages_Free:        0
HugePages_Rsvd:        0
HugePages_Surp:        0
Hugepagesize:       4096 kB
DirectMap4k:       12280 kB
DirectMap4M:      897024 kB
```

A saída acima mostra 1948MB de memória física, dos quais 1309MB estão livres, e 5153MB de espaço *swap*, todo livre. Relacionado a memória física três outros valores são mostrados:

- *Shmem* mostra o total de memória compartilhada atualmente alocada no sistema (veja a Seção 5.1, "Memória Compartilhada").

- *Buffers* mostra a memória alocada pelo GNU/Linux para *buffers* de dispositivos de bloco. Esses *buffers* são usados por acionadores de dispositivo para manter blocos de dados sendo lidos do e escritos para o disco.

- *Cached* mostra a memória alocada pelo GNU/Linux para *cache* de página. Essa memória é usada para acessos de *cache* para arquivos mapeados.

Você pode usar o comando *free* para mostrar a mesma informação de memória.

7.5 Acionadores, Montagens, e Sistemas de Arquivos

O sistema de arquivo */proc* também contém informação sobre os acionadores de disco presentes no sistema e os sistemas de arquivo montados nesse mesmo sistema.

7.5.1 Sistemas de Arquivo

A entrada */proc/filesystems* mostra os tipos de sistema de arquivo conhecidos do *kernel*. Note que essa lista não é muito útil pelo fato de não ser completa: Sistemas de arquivo podem ser carregados e descarregados dinamicamente como módulos do *kernel*. O conteúdo do */proc/filesystems* lista somente tipos de sistema de arquivo que ou são estatísticamtente linkados para dentro do *kernel* ou são atualmente carregados. Outros sistema de arquivo podem estar disponíveis no sistema como módulos mas podem não ter sido chamados ainda.

7.5.2 Acionadores e Partições

O sistema de arquivo */proc* inclui informações sobre dispositivos conectados a ambas as controladoras IDE e SCSI (se estiverem incluídas).

Em sistemas típicos, o subdiretório */proc/ide* pode conter um ou ambos dos dois subdiretórios, *ide0* e *ide1*, correspondentes à controladora IDE primária e à controladora IDE secundária no sistema[14]. A esses subdiretórios conterão adicionais subdiretórios correspondendo aos dispositivos físicos conectados às controladoras. Os diretórios de dispositivos ou controladoras podem estar ausentes se GNU/Linux não tiver reconhecido quaisquer dispositivos conectados. Os caminhos completos correspondentes aos quatro possíveis dispositivos IDE são listados na tabela 9.1.

Veja a Seção 6.4, "Dispositivos de *Hardware*" para mais informação sobre nomes de dispositivos IDE.

Cada diretório de dispositivo IDE contém muitas entradas fornecendo acesso a identificação e informação de configuração para o dispositivo. Algumas das mais úteis estão listadas aqui:

- *model* contém a sequência de caracteres com a identificação do modelo do dispositivo.

[14]Se adequadamente configurado, o *kernel*. do GNU/Linux pode suportar controladoras IDEs adicionais. Essas IDEs adicionais devem ser numeradas sequêncialmente a partir de ide2.

Tabela 7.1: Caminhos Completos para os Quatro Possíveis Dispositivos IDE

Controladora	Dispositivo	Subdiretório
Primária	Mestre	/proc/ide/ide0/hda/
Primária	Escravo	/proc/ide/ide0/hdb/
Secundária	Mestre	/proc/ide/ide1/hdc/
Secundária	Escravo	/proc/ide/ide1/hdd/

- *media* contém o tipo de mídia do dispositivo. Possíveis valores são *disk*, *cdrom*, *tape*, *floppy*, e *UNKNOWN*.

- *capacity* contém a capacidade do dispositivo, em blocos de 512-*byte*. Note que para dispositivos de CD-ROM, o valor irá ser $2^{31} - 1$, não a capacidade do disco no acionador. Note que o valor em capacidade representa a capacidade do disco físico inteiro; a capacidade dos sistemas de arquivo contidos em partições do disco irá ser menor.

Por exemplo, os comandos abaixo mostram como determinar o tipo de mídia e a identificação do dispositivo para o dispositivo mestre conectado à controladora IDE secundária. Nesse caso, verifica-se ser um acionador de CD-ROM Toshiba.

```
% cat /proc/ide/ide1/hdc/media
cdrom
% cat /proc/ide/ide1/hdc/model
TOSHIBA CD-ROM XM-6702B
```

Se dispositivos SCSI estiverem presentes no sistema, */proc/scsi/scsi* contém um sumário de seus valores de identificação. Por exemplo, o conteúdo pode se parecer com o que segue [15]:

```
% cat /proc/scsi/scsi
Attached devices:
Host: scsi0 Channel: 00 Id: 00 Lun: 00
  Vendor: QUANTUM Model: ATLAS_V__9_WLS      Rev: 0230
  Type:   Direct-Access      ANSI SCSI revision: 03
Host: scsi0 Channel: 00 Id: 04 Lun: 00
  Vendor: QUANTUM Model: QM39100TD-SW        Rev: N491
  Type:   Direct-Access      ANSI SCSI revision: 02
```

[15]Nota do tradutor: veja a Seção G.4 do Apêndice G para um exemplo adicional.

Esse computador contém uma controladora SCSI de canal simples (designada "scsi0"), na qual dois acionadores de discos da marca Quantum estão conectados, com IDs de dispositivo SCSI 0 e 4.

A entrada */proc/partitions* mostra as partições dos dispositivos de disco reconhecidos. Para cada partição, a saída inclui o número de dispositivo principal e secundário, o número de blocos de 1024-*byte*, e o nome de dispositivo correspondente a aquela partição.

A entrada */proc/sys/dev/cdrom/info* mostra informações diversar sobre a capacidade dos acionadores de CD-ROM/DVD. Os campos explicam-se a sí mesmos [16]:

```
% cat /proc/sys/dev/cdrom/info
CD-ROM information, Id: cdrom.c 2.56 1999/09/09

drive name: hdc
drive speed: 48
drive \# of slots: 0
Can close tray: 1
Can open tray: 1
Can lock tray: 1
Can change speed: 1
Can select disk: 0
Can read multisession:   1
Can read MCN: 1
Reports media changed:   1
Can play audio: 1
```

7.5.3 Montagens

O arquivo */proc/mounts* fornece um sumário dos sistemas de arquivo montados. Cada linha corresponde a um único descritor de montagem e mostra o dispositivo montado, o ponto de montagem, e outra informação. Note que */proc/mounts* contém a mesma informação que o arquivo comum */etc/mtab*, o qual é automaticamente atualizado pelo comando *mount*.

Segue addiante os elementos de um descritor de montagem:

- O primeiro elemento na linha é o dispositivo montado (veja Capítulo 6). Para sistemas de arquivo especiais tais como o sistema de arquivo */proc*, esse elemento é *none*.

[16] Nota do tradutor: veja a Seção G.5 do Apêndice G para um exemplo adicional.

- O segundo elemento é o ponto de montagem, o local no sistema de arquivo raíz no qual o conteúdo do sistema de arquivo montado aparece. Para o sistema de arquivo raíz propriamente dito, o ponto de montagem é listado com /. Para acionadores *swap*, o ponto de montagem é listado como *swap*.

- O terceiro elemento é o tipo do sistema de arquivo. Atualmente[17],a maioria dos sistemas GNU/Linux usam o sistema de arquivo ext2 para acionadores de disco, mas acionadores DOS ou Windows podem ser montados com outros tipos de sistema de arquivo, tais como *fat* ou *vfat*. A maioria dos CD-ROMs/DVDs possuem um sistema de arquivo iso9660. Veja a página de manual do comando *mount* para uma lista de tipos de sistema de arquivo.

- O quarto elemento mostra sinalizadores de montagem. esses sinalizadores de montagem são opções que foram especificadas quando o comando *mount* foi chamado. Veja a página de manual do comando *mount* para uma explanação de sinalizadores para os vários tipos de sistema de arquivo.

No */proc/mounts*, os dois últimos elementos são sempre 0 e não possuem significado.

Veja a página de manual do *fstab* para detalhes sobre o formato dos descritores de montagem [18]. GNU/Linux inclui funções para ajudar você a informar descritores de montagem; veja a página de manual para a função *getmntent* para informação de como usá-la.

7.5.4 Travas

A Seção 8.3, "A chamada de Sistema *fcntl*: Travas e Outras Operações em Arquivos" descreve como usar a chamada de sistema *fcntl* para controlar travas de leitura e escrita sobre arquivos. A entrada */proc/locks* descreve todas as travas de arquivo atualmente funcionando no sistema. Cada linha na saída corresponde a uma trava.

Para travas criadas com *fcntl*, as primeiras duas entradas na linha são *POSIX ADVISORY* [19]. A terceira entrada na linha pode ser ou WRITE ou READ, dependendo do tipo da trava. O próximo número é o ID de processo do processo mantendo a trava. Os seguintes três números, separados por dois

[17]Nota do tradutor:2001.

[18]O arquivo */etc/fstab* lista a configuração estática de montagem do sistema GNU/Linux.

[19]Nota do tradutor: veja G.7 para um exemplo adicional.

pontos, são os números de dispositivo principal e secundário do dispositivo sobre o qual o arquivo reside e o número do *inode*, que localiza o arquivo no sistema de arquivo. O restante da linha mostra valores internos ao *kernel* que geralmente não são de utilidade.

O ajuste do conteúdo de */proc/locks* dentro das informações úteis precisa um pouco de trabalho de detetive. Você pode assistir o */proc/locks* em ação, por exemplo, rodando o programa na Listagem 8.2 para criar uma trava de escrita sobre o arquivo */tmp/test-file*.

```
% touch /tmp/test-file
% ./lock-file /tmp/test-file
file /tmp/test-file
opening /tmp/test-file
locking
locked; hit enter to unlock...
```

Em outra janela, olhe o conteúdo do */proc/locks*.

```
 cat /proc/locks
1: POSIX ADVISORY WRITE 5467 08:05:181288 0 2147483647 d1b5f740 00000000
dfea7d40 00000000 00000000
```

Podem existir outras linhas de saída, também, correspondendo a travas mantidas por outros programas. Nesse caso, 5467 é o ID do processo do programa *lock-file*. Use o comando *ps* para mostrar o que esse pprocesso está rodando.

```
ps 5467
 PID TTY       STAT     TIME COMMAND
5467 pts/28    S        0:00 ./lock-file /tmp/test-file
```

O arquivo de trava, */tmp/test-file*, reside sobre o dispositivo que tem números de dispositivo principal e secundário 8 e 5, respectivamente. Esses números correspondem ao */dev/sda5*.

```
% df /tmp
Filesystem           1k-blocks        Used Available Use% Mounted on
/dev/sda5              8459764     5094292   2935736 63% /
% ls -l /dev/sda5
brw-rw----    1 root     disk          8,   5 May   5   1998 /dev/sda5
```

O arquivo */tmp/test-file* própriamente dito está no *inode* 181,288 sobre aquele dispositivo.

```
% ls --inode /tmp/test-file
 181288 /tmp/test-file
```

Veja a Seção 6.2, "Números de Dispositivo" para maiores informações sobre números de dispositivo.

7.6 Estatísticas de Sistema

Duas entradas no */proc* possuem estatísticas úteis do sistema. O arquivo */proc/loadavg* contém informação sobre a carga do sistema. Os primeiros três números represetnam o número de tarefas ativas no sistema – processos que estão atualmente executando em termos médios sobre os ultimos 1, 5, e 15 minutos. A entrada seguinte mostra o número instantâneo corrente de tarefas rodáveis – processos que estão atualmente agendados para executar mas não sendo bloqueados em uma chamada de sistema – e o número total de processos no sistema. A entrada final é o ID do processo que executou mais recentemente.

O arquivo */proc/uptime* contém contagem de tempo desde quando o sistema foi inicializado, bem como o montante do tempo desde então que o sistema tenha estado ocioso. Ambos são fornecidos como valores em ponto flutuante, em segundos[20].

```
% cat /proc/uptime
3248936.18 3072330.49
```

O programa na Listagem 7.7 extrai o tempo total de funcionamento e o tempo de ociosidade estando ligado a partir do sistema e mostra-os em unidades amigáveis.

Listagem 7.7: (print-uptime.c) Mostra o Tempo Ligado e o Tempo Ocioso

```
1 #include <stdio.h>
2
3 /* Sumariza a dura  o do tempo para sa sa da padr o.  TIME    o
4    total de tempo, em segundos, e LABEL    um r tulo descritivo curto.  */
5
6 void print_time (char* label, long time)
7 {
8   /* Constantes de convers o.  */
9   const long minute = 60;
10  const long hour = minute * 60;
11  const long day = hour * 24;
12  /* Produz a sa da.  */
13  printf ("%s: %ld dias, %ld:%02ld:%02ld\n", label, time / day,
14          (time % day) / hour, (time % hour) / minute, time % minute);
15 }
16
17 int main ()
18 {
19   FILE* fp;
20   double uptime, idle_time;
21   /* L  o tempo de funcionamento do sistema e o tempo de uso acumulado a partir de /
        proc/uptime.  */
22   fp = fopen ("/proc/uptime", "r");
23   fscanf (fp, "%lf %lf\n", &uptime, &idle_time);
24   fclose (fp);
25   /* Sumariza o tempo de funcionamento do sistema e o tempo de uso acumulado.  */
26   print_time ("uptime   ", (long) uptime);
27   print_time ("idle time", (long) idle_time);
28   return 0;
29 }
```

[20]Nota do tradutor: aqui temos uma ótima indicação de uso de um servidor por exemplo.

O comando *uptime* e a chamada de sistema *sysinfo* (veja a Seção 8.14, "A Chamada de Sistema *sysinfo*: Obtendo Estatísticas do Sistema") também pode obter o *uptime* do sistema. O comando *uptime* também mostra a carga média encontrada em */proc/loadavg*.

Capítulo 8

Chamadas de Sistema do GNU/Linux

ATÉ AGORA, APRESENTAMOS UMA VARIEDADE DE FUNÇÕES que seu programa pode chamar para executar tarefas relacionadas ao sistema, tais como informar opções de linha de comando, controlar processos, e mapeamento de memória. Se você olhar sob a tampa do compartimento do motor, irá encontrar que essas funções se encaixam em duas categorias, baseado em como elas são implementadas.

- *Função de biblioteca* que é uma função comum que reside em uma biblioteca externa ao seu programa. A maioria das funções de biblioteca que mostramos até agora estão na biblioteca C GNU padrão, a *libc*. Por exemplo, *getopt_long* e *mkstemp* são funções fornecidas na biblioteca C GNU padrão.

  Uma chamada a uma função de biblioteca é apenas como qualquer outra chamada de função. Os argumentos são colocados em registros de processador ou em uma pilha, e a execução é transferida ao início do código da função, que tipicamente reside em uma biblioteca compartilhada que foi carregada.

- *Chamada de sistema* que é implementada no *kernel* do GNU/Linux. Quando um programa faz uma chamada de sistema, os argumentos são empacotados e manipulados para o *kernel*, o qual assume a execução do programa até que a chamada se complete. Uma chamada de sistema não é uma chamada de função comum, e um procedimento especial é requerido para transferir o controle ao *kernel*. Todavia, a biblioteca C GNU padrão(a implementação da biblioteca C GNU padrão fornecida com sistemas GNU/Linux) envolve chamadas de sistema do GNU/Linux com funções de forma que você pode chamá-las facilmente. Funções de entrada e saída de baixo nível tais como *open* e *read* são exemplos de chamadas de sistema em GNU/Linux.

  O conjunto de chamadas de sistema do GNU/Linux forma a mais básica interface entre programas e o *kernel* do GNU/Linux. Cada chamada mostra uma operação básica ou capacidade básica. Algumas chamadas de sistema são mutio poderosas e podem exercer grande influência no sistema. Por exemplo, algumas chamadas de sistema habilitam você a desligar o sistema GNU/Linux ou a alocar recursos do sistema e prevenir que outros usuários o acessem. Essas chamadas possuem a restrição que somente processos executando com privilégios de superusuário (programas executando pela conta *root*) podem chamá-las. Essas chamadas falham se chamadas por um processo comum.

Note que uma função de biblioteca pode chamar uma ou mais outras funções de biblioteca ou chamadas de sistema como parte de sua implementação.

GNU/Linux atualmente fornece cerca de 200 chamadas de sistema diferentes[1]. Uma listagem de chamadas de sistema para sua versão do *kernel* do

[1]Nota do tradutor: mais de 300 nos *kernels* 2.6.

GNU/Linux encontra-se em /usr/include/asm/unistd.h [2]. Algumas dessas chamadas de sistema são de uso interno pelo sistema, e outras são usadas somente em implementação de funções de bibliotecas especializadas. Nesse capítulo, mostraremos uma seleção de chamadas de sistema que são mais sucetíveis de serem úteis a aplicações e a programadores de sistemas.

A maioria dessas chamadas de sistema estão declaradas em <unistd.h>.

8.1 Usando *strace*

Antes de iniciarmos discutindo chamadas de sistema, irá ser útil mostrar um comando com o qual você pode aprender sobre chamadas de sistema e a depurar programas que contenham chamadas de sistema. O comando *strace* rastreia a execução de outro programa, listando quaisquer chamadas de sistema que o programa faz e qualquer sinal que o programa recebe.

Para ver as chamadas de sistema e os sinais em um programa, simplesmente chame *strace*, seguido pelo programa e seus argumentos de linha de comando. Por exemplo, veja as chamadas de sistema que são chamadas pelo comando *hostname*[3], use a linha abaixo:

```
% strace hostname
```

Isso produz algumas telas de saída. Cada linha corresponde a uma única chamada de sistema. Para cada chamada, o nome da chamada de sistema é listado, seguido por seus argumentos (ou argmentos abreviados, se eles forem muito longos) e seus valores de retorno. Onde possível, *strace* convenientemente mostra nomes simbólicos ao invés de valores numéricos para argumentos e valores de retorno, e *strace* mostra os campos de estruturas informados por um apontador dentro da chamada de sistema. Note que *strace* não mostra chamadas a funções comuns.

Na saída do comando *strace hostname*, a primeira linha mostra a chamada de sistema *execve* que chama o programa *hostname*[4] [5]:

```
execve("/bin/hostname", ["hostname"], [/* 49 vars */]) = 0
```

[2]Nota do tradutor: no arquivo *unistd.h* atual tem uma condição que redireciona conforme a arquitetura seja 32 ou 64 *bits*. Se for 32 o arquivo é o *unistd_32.h*. Se for 64 o arquivo é o *unistd_64.h*.

[3]o comando *hostname* chamado sem quaisquer sinalizadores simplesmente mostra o nome de *host* do computador para a saída padrão.

[4]Em GNU/Linux, a família de funções *exec* é implementada usando a chamada de sistema *execve*.

[5]Nota do tradutor: veja no Apêndice H toda a saída na Seção H.1.

O primeiro argumento é o nome do programa a executar; o segundo é sua lista de argumentos, consistindo de somente um único elemento; e o terceiro argumento é sua lista de ambiente, a qual *strace* omite por brevidade. As seguintes 30 ou mais linhas são parte do mecanismo que carrega a biblioteca C GNU padrão a partir de um arquivo de biblioteca compartilhada.

Mais para o final estão chamadas de sistema que atualmente ajudam a fazer o programa trabalhar. A chamada de sistema *uname* é usada para obter o nome do *host* do sistema reportado pelo *kernel*,

```
uname({sys="Linux", node="computador", ...}) = 0
```

Observe que *strace* prestativamente rotula os campos (*sys* e *node*) do argumento estrutura. Essa estrutura é preenchida pela chamada de sistema – GNU/Linux ajusta o campo *sys* para o nome do sistema operacional e o campo *node* para o nome do *host* do sistema. A chamada de sistema *uname* será discutida mais detalhadamente na Seção 8.15, "A Chamada de Sistema *uname*".

Finalmente, a chamada de sistema *write* produz a saída. Relembrando que o descritor de arquivo 1 corresponde à saída padrão. O terceiro argumento é o número de caracteres a escrever, e o valor de retorno é o número de caracteres que foram atualmente escritos.

```
write(1, "computador\n", 11)= 11
```

Isso pode parecer truncado quando você executa *strace* pelo fato de a saída do programa *hostname* propriamente dita estar misturada com a saída do *strace*.

Se o programa que você está rastreando produz grande quantidade de saída, é algumas vezes mais conveniente redirecionar a saída de *strace* para dentro de um arquivo. Use a opção "-o nomearquivo" para fazer isso.

Entender toda a saída de *strace* requer familiaridade detalhada com o desenho do *kernel* do GNU/Linux e também do ambiente de execução. A maioria dessa familiaridade detalhada é de interesse limitado para programadores de aplicação. Todavia, algum entendimento é util para depurar problemas complicados ou entender como outros programas trabalham.

8.2 A Chamada *access*: Testando Permissões de Arquivos

A chamada de sistema *access* determina se o processo que a chamou tem permissão de acesso a um arquivo. A chamada de sistema *access* pode verifi-

car qualquer combinação de permissão de leitura, escrita e execução, e *access* pode também verificar a existência de um arquivo.

A chamada de sistema *access* recebe dois argmentos. O primeiro é o caminho para o arquivo a ser verificado. O segundo argumento é uma operação bit a bit do tipo entre R_OK, W_OK, e X_OK, correspondendo a permissão de leitura, escrita e execução. O valor de retorno é 0 se o processo tiver todas as permissões especificadas. Se o arquivo existe mas o processo chamador não tem as permissões especificadas, a chamada de sistema *access* retorna -1 e ajusta *errno* para *EACCES* (ou *EROFS*, se permissão de escrita for requisitada para um arquivo sobre um sistema de arquivo somente leitura).

Se o segundo argumento for F_OK, *access* simplesmente verifica pela existência do arquivo. Se o arquivo existir, o valor de retorno é 0; se o arquio não existir, o valor de retorno é -1 e *errno* é ajustada para *ENOENT*. Note que *errno* pode ao contrário ser ajustada para *EACCES* se um diretório no caminho do arquivo estiver inacessível.

O programa mostra na Listagem 8.1 usos de *access* para verificar a existência de um arquivo e para determinar permissões de leitura e escrita. Especifique o nome do arquivo a ser verificado na linha de comando.

Listagem 8.1: (check-access.c) Check File Access Permissions

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 int main (int argc, char* argv[])
6 {
7   char* path = argv[1];
8   int rval;
9
10   /* Verifica a exist ncia do arquivo.  */
11   rval = access (path, F_OK);
12   if (rval == 0)
13     printf ("%s existe\n", path);
14   else {
15     if (errno == ENOENT)
16       printf ("%s n o existe\n", path);
17     else if (errno == EACCES)
18       printf ("%s n o est  acess vel\n", path);
19     return 0;
20   }
21
22   /* Verifique o acesso de leitura.  */
23   rval = access (path, R_OK);
24   if (rval == 0)
25     printf ("%s est  leg vel\n", path);
26   else
27     printf ("%s n o est  leg vel (acesso negado)\n", path);
28
29   /* verifica o acesso de escrita.  */
30   rval = access (path, W_OK);
31   if (rval == 0)
32     printf ("%s permite escrita\n", path);
33   else if (errno == EACCES)
34     printf ("%s n o permite escrita (acesso negado)\n", path);
35   else if (errno == EROFS)
36     printf ("%s n o permite escrita (sisteam de arquivos somente leitura)\n", path);
37
38   return 0;
39 }
```

Por exemplo, para verificar as permissões de acesso para um arquivo

chamado LEIAME gravado em um CD-ROM, chame o programa da listagem 8.1 como segue:

```
% ./check-access /mnt/cdrom/LEIAME
/mnt/cdrom/LEIAME exists
/mnt/cdrom/LEIAME is readable
/mnt/cdrom/LEIAME is not writable (read-only filesystem)
```

8.3 A Chamada de Sistema *fcntl*: Travas e Outras Operações em Arquivos

A chamada de sistema *fcntl* é o ponto de acesso para muitas operações avançadas sobre descritores de arquivos. O primeiro argumetno a *fcntl* é um descritor já aberto, e o segundo é um valor que indica qual operação é para ser executada. Para algumas operações, *fcntl* recebe um argumento adicional. Iremos descrever aqui uma das mais úteis operações de *fcntl*, o travamento de um arquivo. Veja a página de manual de *fcntl* para informação sobre as outras operações que podem ser feitas sobre arquivos por *fcntl*.

A chamada de sistema *fcntl* permite a um programa colocar uma trava de leitura ou uma trava de escrita sobre um arquivo, até certo ponto análoga a travas de *mutex* discutidas no Capítulo 5, "Comunicação Entre Porcessos". Uma trava de leitura é colocada sobre um descritor que pode ser lido, e uma trava de escrita é colocada sobre um descritor de arquivo que pode ser escrito. Mais de um processo pode manter uma trava de leitura sobre o mesmo arquivo ao mesmo tempo, mas somente um processo pode manter uma trava de leitura, e o mesmo arquivo não pode ser simultâneamente travado para leitura e escrita. Note que colocando uma trava não previne atualmente outros processos de abrirem o arquivo, ler a partir dele, ou escrever para ele, a menos que esses outros processos adquiram travas com *fcntl* também.

Para colocar uma trava sobre um arquivo, primeiro crie uma variável *struct flock* com todos os seus campos zerados. Ajuste o campo *l_type* da estrutura para *F_RDLCK* para uma trava de leitura ou para *F_WRLCK* para uma trava de escrita. Então chame *fcntl*, informando um descritor de arquivo para o arquivo, o código de operação *F_SETLKW*, e um apontador para a variável *struct flock*. Se outro processo mantém uma trava que evita que uma nova trava seja adquirida, *fcntl* bloqueia até que aquela trava seja liberada.

O programa na Listagem 8.2 abre um arquivo para escrita cujo nome é fornecido pela linha de comando, e então coloca uma trava de escrita nesse

mesmo arquivo aberto. O programa espera pelo usuário pressionar *Enter* e então destrava e fecha o arquivo.

Listagem 8.2: (lock-file.c) Create a Write Lock with fcntl

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <string.h>
4 #include <unistd.h>
5
6 int main (int argc, char* argv[])
7 {
8   char* file = argv[1];
9   int fd;
10  struct flock lock;
11
12  printf ("abrindo %s\n", file);
13  /* Abreu um descritor de arquivo para o arquivo.  */
14  fd = open (file, O_WRONLY);
15  printf ("travando\n");
16  /* Inicializa a estrutura flock.  */
17  memset (&lock, 0, sizeof(lock));
18  lock.l_type = F_WRLCK;
19  /* Coloca uma trava de leitura sobre o arquivo.  */
20  fcntl (fd, F_SETLKW, &lock);
21
22  printf ("travado; precione enter para destravar... ");
23  /* Esperando que o usu rio pressione enter.  */
24  getchar ();
25
26  printf ("destravando\n");
27  /* Libera a trava.  */
28  lock.l_type = F_UNLCK;
29  fcntl (fd, F_SETLKW, &lock);
30
31  close (fd);
32  return 0;
33 }
```

Compile e rode o programa sobre um arquivo de teste – digamos, /tmp/test-file – como segue:

```
% cc -o lock-file lock-file.c
% touch /tmp/test-file
% ./lock-file /tmp/test-file
opening /tmp/test-file
locking
locked; hit Enter to unlock...
```

Agora, em outra janela, tente rodar o mesmo programa novamente sobre o mesmo arquivo.

```
% ./lock-file /tmp/test-file
opening /tmp/test-file
locking
```

Note que a segunda instância fica bloqueada enquanto tenta travar o arquivo. Volte à primeira janela e pressione *Enter*:

```
unlocking
```

O programa rodando na segunda janela imediatamente adquire a trava.

Se você prefere que *fcntl* não bloqueie se a chamada não puder pegar a trava que você requisitou, use *F_SETLK* ao invés de *F_SETLKW*. Se a trava não puder ser adquirida, *fcntl* retorna -1 imediatamente.

GNU/Linux fornece outra implementação de travamento de arquivo com a chamada *flock*. A versão *fcntl* tem uma vantagem importante: a versão *fcntl* trabalha com arquivos no sistema de arquivos NFS[6] (contanto que o servidor NFS seja razoavelmente recente e corretamente configurado). Então, se você tiver acesso a duas máquinas que ambas montem o mesmo sistema de arquivos via NFS, você pode repetir o exemplo prévio usando duas diferentes máquinas. Rode *lock-file* em uma máquina, especificando um arquivo em um sistema de arquivo NFS, e então rode o programa *lock-file* novamente em outra máquina, especificando o mesmo arquivo. NFS acorda o segundo programa quando a trava é liberada pelo primeiro programa.

8.4 As Chamadas *fsync* e *fdatasync*: Descarregando para o Disco

Na maioria dos sistemas operacionais, quando você escreve em um arquivo, os dados não são imediatamente escritos no disco. Ao invés disso, o sistema operacional oculta os dados escritos em uma área de armazenamento temporária da memória, para reduzir o número de requisições de escrita para o disco e diminuir o tempo de resposta do programa. Quando a área temporária de armazenamento enche ou alguma outra condição ocorrer (por exemplo, intervalo de tempo satisfatória para se fazer a escrita), o sistema escreve os dados ocultos para o disco todos de uma só vez.

GNU/Linux fornece essa otimização de acesso ao disco também. Normalmente, essa ocultação apresenta um grande ganho de performace. Todavia, esse compartamento pode fazer programas que dependem da integridade de gravações baseadas em disco não serem confiáveis. Se o sistema para de funcionar bruscamente – por exemplo, devido a um travamento do *kernel* ou interrupção no fornecimento de energia – quaisquer dados escritos por um programa que está na memória cache mas não tiver sido ainda escrito no disco é perdido.

Por exemplo, suponhamos que você está escrevendo um programa para efetuar uma transação que mantém um arquivo de um sistema de arquivos

[6] *Network File System (NFS)* é uma tecnologia comum de compartilhamento de arquivos em rede, comparável aos acionadores de rede e compartilhamentos do Windows.

com jornal. O arquivo de jornal do sistema de arquivos contém registros de todas as transações que tenham sido processadas de forma que se uma falha no sistema vier a ocorrer, o estado dos dados da transação pode ser reconstruído. Isso é obviamente importante para preservar a integridade do arquivo de jornal – toda vez que uma transação é processada, sua entrada de jornal deve ser enviada para o acionador de disco imediatamente.

Para ajudar você a implementar esse arquivo de jornal, GNU/Linux fornece a chamada de sistema *fsync*. A chamada de sistema *fsync* recebe um argumento, um descritor de arquivo que pode ser escrito, e descarrega para o disco quaisquer dados escritos para esse arquivo de jornal. A chamada *fsync* não retorna até que os dados tenham sido fisicamente escritos.

A função na Listagem 8.3 ilustra o uso de *fsync*. A função da listagem escreve uma entrada de linha única para um arquivo de jornal.

Listagem 8.3: (write_journal_entry.c) Write and Sync a Journal Entry

```
1 #include <fcntl.h>
2 #include <string.h>
3 #include <sys/stat.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6
7 const char* journal_filename = "journal.log";
8
9 void write_journal_entry (char* entry)
10 {
11   int fd = open (journal_filename, O_WRONLY | O_CREAT | O_APPEND, 0660);
12   write (fd, entry, strlen (entry));
13   write (fd, "\n", 1);
14   fsync (fd);
15   close (fd);
16 }
```

Outra chamada de sistema, *fdatasync* faz a mesma coisa. Todavia, embora *fsync* garanta que a hora de modificação de arquivo irá ser atualizada, *fdatasync* não garante; *fdatasync* garante somente que os dados do arquivo irão ser escritos. Isso significa que princípio, *fdatasync* pode executar mais rapidamente que *fsync* pelo fato de *fdatafsync* precisar forçar somente uma escrita ao disco ao invés de duas.

Todavia, nas versões correntes do GNU/Linux, essas duas chamadas de sistema atualmente fazem a mesma coisa, ambas atualizam a hora de modificação do arquivo[7].

A chamada de sistema *fsync* habilita você a forçar um descarregamento de área temporária de armazenamento explicitamente. Você pode também abrir um arquivo para E/S sincronizada, o que faz com que todas as escritas sejam imediatamente enviadas ao disco. Para fazer isso, especifique o sinalizador *O_SYNC* ao abrir o arquivo com a chamada de sistema *open*.

[7]Nota do tradutor:a afirmação refere-se a *kernels* 2.2. Nos *kernels* pós-2.2 *fdatasync* é mais rápida pelo fato de não atualizar a hora de modificação do arquivo.

8.5 As Chamadas *getrlimit* e *setrlimit*: Limites de Recurso

As chamadas de sistema *getrlimit* e *setrlimit* permitem a um processo ler e ajustar limites sobre recursos de sistema que o processo chamador pode consumir. Você pode estar familiarizado com o comando *shell ulimit*, o qual habilita você a restringir o uso de recurso de programas que você roda; [8] essa chamadas de sistema permite a um programa fazer isso programaticamente[9].

Para cada recurso existe dois limites, o limite inegociável e o limite negociável. O limite negociável jamais pode exceder o limite inegociável, e somente processos com privilégio de superusuário podem mudar o limite inegociável. Tipicamente, um programa de aplicação irá reduzir o limite negociável para colocar um controle sobre os recursos que usa.

As chamadas *getrlimit* e *setrlimit* recebem como argumentos um código especificando o tipo de limite de recurso e um apontador a uma variável do tipo *struct rlimit*. A chamada *getrlimit* preenche os campos dessa estrutura, enquanto a chamada *setrlimit* muda o limite basedo no conteúdo da *struct rlimit*. A estrutura *rlimit* tem dois campos: *rlim_cur* que é o limite negociável, e *rlim_max* que é o limite rígido máximo.

Alguns dos limites de recursos mais úteis que podem ser mudados são listados aqui, com seus códigos:

[8] Veja a página de manual para seu *shell* para maior informação sobre *ulimit*.

[9] Nota do tradutor: programaticamente quer dizer *a partir do* ou *no* ou *usando o* código fonte ou função de biblioteca de um programa na linguagem C.

- RLIMIT_CPU – O tempo máximo de CPU, em segundos, usado por um programa. Esse é o total de tempo que o programa está atualmente executando sobre a CPU, que não é necessariamente o mesmo que mostra em horas no relógio comum. Se o programa excede esse limite de tempo, o programa é terminado com um sinal SIGXCPU.

- RLIMIT_DATA – O total máximo de memória que um programa pode alocar para seus dados. Alocação adicional além desse limite irá falhar.

- RLIMIT_NPROC – O número máximo de processos filhos que pode ser rodados para esse usuário. Se o processo chama *fork* e muitos processos pertencentes a esse usuário estão rodando so sistema, a chamada a *fork* irá falhar.

- RLIMIT_NOFILE – O númeo máximo de descritores de arquivo que o processo pode ter aberto ao mesmo tempo.

Veja a página de manual de *setrlimit* para se informar sobre a lista completa de recursos do sistema.

Listagem 8.4: (limit-cpu.c) Demonstração do Tempo Limite de Uso da CPU

```
1 #include <sys/resource.h>
2 #include <sys/time.h>
3 #include <unistd.h>
4
5 int main ()
6 {
7   struct rlimit rl;
8
9   /* Obt m os limites atuais.  */
10   getrlimit (RLIMIT_CPU, &rl);
11   /* Set a CPU limit of one second.  */
12   rl.rlim_cur = 1;
13   setrlimit (RLIMIT_CPU, &rl);
14   /* Encarrega-se do trabalho.  */
15   while (1);
16
17   return 0;
18 }
```

Quando o programa encerra por SIGXCPU, o *shell* prestativamente mostra uma mensagem intepretando o sinal:

```
% ./limit_cpu
CPU time limit exceeded
```

8.6 a Chamada *getrusage*: Estatísticas de Processo

A chamada de sistema *getrusage* recupera estatísticas de processo a partir do *kernel*. A chamada de sistema *getrusage* pode ser usada para obter estatísticas ou para o processo atual informando RUSAGE_SELF como o primeiro argumento, ou para todos os processos filhos encerrados que foram *forkados* pelo processo chamador e seus filhos informando RUSAGE_CHILDREN. O segundo argumento a *rusage* é um apontador para uma variável do tipo *struct rusage*, a qual é preenchida com as estatísticas.

Alguns dos campos mais interessantes em *struct rusage* são listados aqui:

- ru_utime – Um campo do tipo *struct timeval* contendo o total de tempo de usário, em segundos, que o processo tenha usado. Tempo de usário é o tempo de CPU investido executando o programa do usuário, ao invés de em chamadas de sistema do *kernel*.
- ru_stime – Um campo do tipo *struct timeval* contendo o total de tempo do sistema, em segundos, que o processo tenha usado. Tempo do sistema é o tempo de CPU investido executando chamadas de sistema na conta do processo.
- ru_maxrss – O maior total de memória física ocupada pelos dados do processo de uma só vez ao longo de sua execução.

A página de manual de *getrusage* lista todos os campos disponíveis. Veja Seção 8.7, "A Chamada *gettimeofday*: Hora Relógio Comum" para informação sobre *struct timeval*.

A função na Listagem 8.5 mostra o usuário atual do processo e a hora do sistema.

Listagem 8.5: (print-cpu-times.c) Mostra Usuário de Processo e Horas do Sistema

```
1  #include <stdio.h>
2  #include <sys/resource.h>
3  #include <sys/time.h>
4  #include <unistd.h>
5
6  void print_cpu_time()
7  {
8    struct rusage usage;
9    getrusage (RUSAGE_SELF, &usage);
10   printf ("Tempo CPU: %ld.%06ld sec usu rio , %ld.%06ld sec sistema\n",
11           usage.ru_utime.tv_sec, usage.ru_utime.tv_usec,
12           usage.ru_stime.tv_sec, usage.ru_stime.tv_usec);
13 }
```

8.7 A Chamada *gettimeofday*: Hora Relógio Comum

A chamada de sistema *gettimeofday* pega a hora relógio comum do sistema. A chamada de sistema *gettimeofday* pega um apontador para uma variável *struct timeval*. Essa estrutura representa uma hora determinada, em segundos, quebrada em dois campos. O campo *tv_sec* contém o número total de segundos, e o campo *tv_usec* contém um número adicional de micro-segundos. Essa valor de variável do tipo *struct timeval* representa o número de segundos que se passaram desde o início da época UNIX, na meia noite UTC de primeiro de Janeiro de 1970[10]. A chamada *gettimeofday* também recebe um segundo argumento, que deve ser *NULL*. *Include <sys/time.h>* se você usa essa chamada de sistema.

O número de segundos na época UNIX não é usualmente um caminho conveniente de representar datas. As funções de biblioteca *localtime* e *strftime* ajudam a manipular o valor de retorno de *gettimeofday*. A função *localtime* pega um apontador ao número de segundos (o campo *tv_sec* de *struct timeval*) e retorna um apontador a um objeto *struct tm*. Essa estrutura contém campos mais úteis, os quais são preenchidos conforme o fuso horário local:

- *tm_hour*, *tm_min*, *tm_sec* – A hora do dia, em horas, minutos, e segundos.
- *tm_year*, *tm_mon*, *tm_day* – O dia, mes e ano da data.
- *tm_wday* – O dia da semana. Zero representa Domingo.
- *tm_yday* – O dia do ano.
- *tm_isdst* – Um sinalizador indicando se o horário de verão está vigorando.

A função *strftime* adicionalmente pode produzir a partir do apontador a *struct tm* uma sequência de caracteres personalizada e formatada mostrando a data e a hora. o formato é especificado de uma maneira similar a *printf*, como uma sequência com códigos embutidos indicando quais campos de hora incluir. Por exemplo, ess formato de sequência de caracteres:

```
"%Y-%m-%d %H:%M:%S"
```

especifica a data e hora na seguinte forma:

[10]Nota do tradutor: veja em H.3 um resumo histórico do ano de 1970 no Brasil e no mundo.

```
2001-01-14 13:09:42
```

Informe a *strftime* um espaço temporário de armazenamento do tipo caractere para receber a sequência de caracteres, o comprimento daquele espaço temporário de armazenamento, a sequência de caracteres que exprime o formato esperado, e um apontador a uma variável do tipo *struct tm*. Veja página de manual da *strftime* para uma lista completa de códigos que podem ser usados na sequência de caracteres indicadora de formato. Note que nem a *localtime* e nem a *strftime* manipulam a parte fracionária da hora atual mais precisa que 1 segundo (o campo *tv_usec* da *struct timeval*). Se você deseja isso em sua sequência de caracteres formatada indicadora da hora, você terá de incluir isso por sua própria conta.

Include <*time.h*> se você chama *localtime* ou *strftime*.

A função na Listagem 8.6 mostra a data atual e hora do dia, arredondando para baixo nos milésimos de segundo.

Listagem 8.6: (print-time.c) Mostra a Data e a Hora

```
1 #include <stdio.h>
2 #include <sys/time.h>
3 #include <time.h>
4 #include <unistd.h>
5
6 void print_time ()
7 {
8   struct timeval tv;
9   struct tm* ptm;
10  char time_string[40];
11  long milliseconds;
12
13  /* Obt m a data atual, e converte essa data atual par um tm struct.  */
14  gettimeofday (&tv, NULL);
15  ptm = localtime (&tv.tv_sec);
16  /* Formata a data e a hora, arredondando para baixo em um segundo.  */
17  strftime (time_string, sizeof (time_string), "%Y-%m-%d %H:%M:%S", ptm);
18  /* Calcula os milisegundos a partir dos microssegundos.  */
19  milliseconds = tv.tv_usec / 1000;
20  /* Mostra o tempo formatado, em segundos, seguido por um ponto decimal
21     e os milissegundos.  */
22  printf ("%s.%03ld\n", time_string, milliseconds);
23 }
```

8.8 A Família *mlock*: Travando Memória Física

A família *mlock* de chamadas de sistema permite a um programa travar alguma parte ou todo o seu espaço dentro da memória física. Isso evita que o GNU/Linux faça a paginação dessa memória para um espaço *swap*, mesmo se o programa não tenha acessado esse espaço em algum momento.

Um programa onde intervalo de tempo é muito importante pode travar memória física pelo fato de a defasagem de paginação de memória saindo e voltando pode ser muito longa ou muito imprevisível.Aplicações de alta

segurança podem também desejar prevenir que dados importantes sejam retirados da memória para um arquivo de *swap*, o qual pode ser recuperado por um invasor após o programa terminar.

Travar uma região de memória é tão simples quanto chamar *mlock* com um apontador para o início da região e o comprimento da região. GNU/Linux divide a memória em paginas e pode travar somente páginas inteiras de uma vez; cada página que contém parte da região de memória especificada a *mlock* é travada. A função *getpagesize* retorna o tamanho da página do sistema, o qual é 4KB no GNU/Linux x86[11].

Por exemplo, para alocar 32MB de espaço de endereço e travar esse espaço dentro da RAM, você pode usar esse código:

```
const int alloc\_size = 32 * 1024 * 1024;
char* memory = malloc (alloc\_size);
mlock (memory, alloc\_size);
```

Note que simplesmente alocando um página de memória e travando-a com *mlock* não reserva memória física para o processo que está fazendo a requisição pelo fato de as páginas poderem ser do tipo copie-na-escrita[12]. Portando, você deve escrever um valor sem importância para cada página também:

```
size_t i;
size_t page_size = getpagesize ();
for (i = 0; i < alloc\_size; i += page_size)
  memory[i] = 0;
```

A escrita para cada página força GNU/Linux a atribuit uma página de memória única, não compartilhada para o processo para aquela página.

Para destravar uma região, chame *munlock*, a qual recebe os mesmos argumentos que *mlock*.

Se você desejar todo o espaço de endereçamento de memória de seu programa travado em memória física, chame *mlockall*. Essa chamada de sistema *mlockall* recebe um único argumento sinalizador: MCL_CURRENT trava toda a memória atualmente alocada para processo que fez a chamada a *mlockall*, mas alocações futuras não são travadas; MCL_FUTURE trava todas as páginas que forem alocadas após a chamada a *mlockall*. Use

[11]Nota do tradutor: outros comandos relacionados a memória são *free*, *top vmsat*.

[12]Copie-na-escrita significa que GNU/Linux faz uma cópia privada de uma página de memória para um processo somente quando o processo escreve um valor em algm lugar dentra da página.

MCL_CURRENT|MCL_FUTURE para travar dentro da memória física ambos os tipos de alocação: as atuais e as subsequêntes.

O travamento de grandes quantidade de memória, especialmente usando *mlockall*, pode ser perigoso para todo o sistema GNU/Linux. Travamento indiscriminado da memória é um bom método de trazer seu sistema para um travamento pelo fato de outros processos que estão rodando serem forçados a competir por recursos menores de memória e *swap* rapidamente dentro da memória e voltando para a memória (isso é conhecido como *thrashingNota do tradutor: debulhamento.*). Se você trava muita memória, o sistema irá executar fora da memória inteiramente e GNU/Linux irá encerrar processos.

Por essa razão, somente processos com privilégios de superusuário podem travar memória com *mlock* ou com *mlockall*. Se um processo de usuário comum chama uma dessas funções, a chamada irá falhar, retornar -1, e ajustar *errno* para EPERM.

A chamada *munlockall* destrava toda a memória travada pelo processo atual, incluindo memória travada com *mlock* e *mlockall*.

Um meio conveniente para monitorar o uso de memória de seu programa é usar o comando *top*. Na saída de *top*, a coluna SIZE[13] mostra o tamanho do espaço de endereço virtual de cada programa (o tamanho total de seu código de programa, dados, e pilha, alguns dos quais podem ser paginadas para o espaço *swap*). A coluna RSS[14] (para *resident set size*) mostra o tamanho de memória física que cada programa atualmente ocupa. O somatório de todos os valores RES para todos os programas que estão rodando não pode exceder o tamanho da memória física de seu computador, e o somatório de todos os tamanhos de espaço de endereçamento está limitado a 2GB [15] (para versões de 32-bit do GNU/Linux).

Include <sys/mman.h> se você usa qualquer das chamadas de sistema *mlock*.

8.9 *mprotect*: Ajustando as Permissões da Memória

Na Seção 5.3, "Arquivos Mapeados em Memória" foi mostrado como usar a chamada de sistema *mmap* para mapear um arquivo para dentro da memória. Relembrando que o terceiro argumentos a *mmap* é uma operação bit-a-bit ou de sinalizadores de proteção de memória PROT_READ, PROT_WRITE,

[13]Nota do tradutor: no *kernel* 2.6 essa coluna é "VIRT".
[14]Nota do tradutor: no *kernel* 2.6 essa coluna é "RES".
[15]Nota do tradutor: verificar esse valor.

e PROT_EXEC para permissão de leitura, escrita, e execução, respectivamente, ou de PROT_NONE para nenhum acesso de memória. Se um programa tenta executar uma operação em uma localização de memória que não é permitida por essas permissões, o programa é encerrado com um sinal SIGSEGV (violação de segmento).

Após a memória ter sido mapeada, essas permissões podem ser modificadas com a chamada de sistema *mprotect*. Os argumentos a *mprotect* são um endereço de uma região de memória, o tamanho da região, e um conjunto de sinalizadores de proteção. A região de memória deve consistir de páginas completas: O endereço da região deve ser alinhado com o tamanho de página do sistema, e o comprimento da região deve ser um múltiplo do tamanho de página. Os sinalizadores de proteção para essas páginas são substituídos com o valor especificado.

Obtendo Página de Memória Alinhada

Note que regiões de memória retornadas por *malloc* são tipicamente páginas não alinhadas, mesmo se o tamanho da memória seja um múltiplo do tamanho da página. Se você deseja proteger a memória obtida a partir de *malloc*, você irá ter que alocar uma região de memória maior e encontrar uma região ajustada ao tamanho da página dentro da região alocada. Alternativamente, você pode usar a chamada de sistema *mmap* para evitar *malloc* e alocar memória ajustada ao tamanho da página diretamente do *kernel* do GNU/Linux. Veja a Seção 5.3, "Arquivos Mapeados em Memória" para detalhes.

Por exemplo, suponhamos que seu programa faça a alocação de uma página de memória mapeando /dev/zero, como descrito na Seção 5.3.5, "Outros Usos para Arquivos Mapeados em Memória". A memória é inicialmente ser alvo de ambas as operações de leitura e escrita.

```
int fd = open ("/dev/zero", O\_RDONLY);
char* memory = mmap (NULL, page\_size, PROT\_READ | PROT\_WRITE,
     MAP\_PRIVATE, fd, 0);
close (fd);
```

Mais tarde, seu programa poderá vir a tornar a memória somente para leitura chamando *mprotect*:

```
mprotect (memory, page\_size, PROT\_READ);
```

Uma técnica avançada para monitorar acessos a memória é proteger regiões de memória usando *mmap* ou *mprotect* e então controlar o sinal SIGSEGV que GNU/Linux envia ao programa quando esse mesmo programa tenta acessar aquela memória. O exemplo na Listagem 8.7 ilustra essa técnica.

Listagem 8.7: (mprotect.c) Detecta Acesso à Memória Usando *mprotect*

```
1 #include <fcntl.h>
2 #include <signal.h>
3 #include <stdio.h>
4 #include <string.h>
5 #include <sys/mman.h>
6 #include <sys/stat.h>
7 #include <sys/types.h>
8 #include <unistd.h>
9
10 static int alloc_size;
11 static char* memory;
12
13 void segv_handler (int signal_number)
14 {
15   printf ("mem ria acessada!\n");
16   mprotect (memory, alloc_size, PROT_READ | PROT_WRITE);
17 }
18
19 int main ()
20 {
21   int fd;
22   struct sigaction sa;
23
24   /* Instala segv_handler como o manipulador para SIGSEGV.  */
25   memset (&sa, 0, sizeof (sa));
26   sa.sa_handler = &segv_handler;
27   sigaction (SIGSEGV, &sa, NULL);
28
29   /* Aloca uma p gina de mem ria por meio de mapeamento para /dev/zero.  Mapeia a
        mem ria
30      como somente escrita, inicialmente.  */
31   alloc_size = getpagesize ();
32   fd = open ("/dev/zero", O_RDONLY);
33   memory = mmap (NULL, alloc_size, PROT_WRITE, MAP_PRIVATE, fd, 0);
34   close (fd);
35   /* Escreve para a p gina para obter uma c pia privada.  */
36   memory[0] = 0;
37   /* Torna a mem ria bloqueada para escrita.  */
38   mprotect (memory, alloc_size, PROT_NONE);
39
40   /* Escreve para a regi o de mem ria alocada.  */
41   memory[0] = 1;
42
43   /* Tudo feito; desmapeia a mem ria.  */
44   printf ("all done\n");
45   munmap (memory, alloc_size);
46   return 0;
47 }
```

O programa segue os seguintes passos:

1. O programa instala um controlador de sinal para SIGSEGV.

2. O programa aloca uma página de memória mapeando */dev/zero* e escrevendo um valor para a página alocada para obter uma cópia privada.

3. O programa protege a memória chamando *mprotect* com a permissão PROT_NONE.

4. Quando o programa sequêncialmente escreve para a memória, GNU/Linux envia seu SIGSEGV, o qual é manipulado por *segv_handler*. O controlador de sinal retira a proteção da memória, o que permite que o acesso à memória continue.

5. Quando o controlador de sinal cumpre sua função, o controle retorna para *main*, onde o programa desaloca a memória usando *munmap*.

8.10 A Chamada *nanosleep*: Temporizador de Alta Precisão

A chamada de sistema *nanosleep* é uma versão em alta precisão da chamada padrão UNIX *sleep*. Ao invés de temporizar um número inteiro de segundos, *nanosleep* recebe como seu argumento um apotador para um objeto *struct timespec*, o qual pode expressar hora com precisão de nanosegundos. Todavia, devido a detalhes de como o *kernel* do GNU/Linux trabalha, a atual precisão fornecida por *nanosleep* é de 10 milisegundos[16] – ainda melhor que aquela oferecida por *sleep*. Essa precisão adicional pode ser útil, por exemplo, para agendar operações frequêntes com intervalos de tempo curtos entre elas.

A estrutura *struct timespec* tem dois campos: *tv_sec*, o número de segundos inteiros, e *tv_nsec*, um número adicional de nanosegundos. O valor de *tv_nsec* deve ser menor que 109.

A chamada *nanosleep* fornece outra vantagem sobre a chamada *sleep*. Da mesma forma que *sleep*, a entrega de um sinal interrompe a execução de *nanosleep*, a qual ajusta *errno* para EINTR e retorna -1. Todavia, *nanosleep* recebe um segundo argumento, outro apontador para um objeto *struct timespec*, o qual, se não for *null*, é preenchido com o total dos tempos restantes (isto é, a diferença entre o tempo do *sleep* da requisição e o tempo do *sleep* atual). Isso torna fácil continuar a operação de temporização.

A função na Listagem 8.8 fornece uma implementação alternativa de *sleep*.

[16]Nota do tradutor: verificar esse valor.

Ao contrário da chamada de sistema comum, essa função recebe um valor em ponto flutuante para o número de segundos a temporizar e rinicia a operação de temporização se for interrompida por um sinal.

Listagem 8.8: (better_sleep.c) High-Precision Sleep Function

```
1 #include <errno.h>
2 #include <time.h>
3
4 int better_sleep (double sleep_time)
5 {
6   struct timespec tv;
7   /* Constr i o timespec a partir do n mero total de segundos ...  */
8   tv.tv_sec = (time_t) sleep_time;
9   /* ... e o restante em nanosegundos.  */
10  tv.tv_nsec = (long) ((sleep_time - tv.tv_sec) * 1e+9);
11
12  while (1)
13  {
14    /* Cochila por um tempo especificado em tv.  Se for interrompido por um
15       sinal, coloque o tempo restante que falta cochilar de volta em tv.  */
16    int rval = nanosleep (&tv, &tv);
17    if (rval == 0)
18      /* Completado o tempo total de cochilo; tudo feito.  */
19      return 0;
20    else if (errno == EINTR)
21      /* Interronpido por um sinal.  Tente novamente.  */
22      continue;
23    else
24      /* Algum outro erro; reclame de volta.  */
25      return rval;
26  }
27  return 0;
28 }
```

8.11 readlink: Lendo Links Simbólicos

A chamada de sistema *readlink* recupera o alvo de um link simbólico. A chamada de sistema *readlink* recebe três argumentos: o caminho para o link simbólico, uma área temporária de armazenamento para receber o alvo do link, e o comprimento da área temporária de armazenamento. Desafortunadamente, *readlink* não coloca NUL para encerrar o caminho do alvo que *readlink* coloca na área temporária de armazenamento. A *readlink*, todavia, retorna o número de caracteres no caminho do alvo, de forma que terminar a sequência de caracteres com NUL é simples.

Se o primeiro argumento a *readlink* apontar para um arquivo que não for um link simbólico, *readlink* ajusta *errno* para EINVAL e retorna -1.

O pequeno programa na Listagem 8.9 mostra o alvo do link simbólico especificado em sua linha de comando.

Listagem 8.9: (print-symlink.c) Mostra o Alvo de um Link Simbólico

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <unistd.h>
4
5 int main (int argc, char* argv[])
6 {
7   char target_path[256];
8   char* link_path = argv[1];
9
10   /* Tenta ler o alvo do link simb lico.  */
11   int len = readlink (link_path, target_path, sizeof (target_path) - 1);
12
13   if (len == -1) {
14     /* A chamada falhou.  */
15     if (errno == EINVAL)
16       /* N o    um link simb lico; reporta isso.  */
17       fprintf (stderr, "%s n o    um link simb lico\n", link_path);
18     else
19       /* Algum outro problema ocorreu; mostre a mensagem gen rica.  */
20       perror ("readlink");
21     return 1;
22   }
23   else {
24     /* O caminho do alvo NUL-terminate.  */
25     target_path[len] = '\0';
26     /* Mostre o caminho do alvo.  */
27     printf ("%s\n", target_path);
28     return 0;
29   }
30 }
```

Por exemplo, aqui está como você poderá vir a fazer um link simbólico e usar *print-symlink* para recupera o caminho do referido link:

```
% ln -s /usr/bin/wc meu_link
% ./print-symlink meu_link
/usr/bin/wc
```

8.12 A Chamada *sendfile*: Transferência de Dados Rápida

A chamada de sistema *sendfile* fornece um eficiente mecanismo para copiar dados de um descritor de arquivos para outro. Os descritores de arquivo podem ser abertos para arquivos em disco, *sockets*, ou outros dispositivos.

Tipicamente, para copiar de um descritor de arquivo para outro, um programa aloca uma área temporária de armazenamento de tamanho fixo, copia algum dado de um descritor de arquivo para a área temporária, escreve o conteúdo da área temporária para o outro descritor, e repete essa operação até que todos os dados tenham sido copiados. Isso é ineficiente de ambas as formas, demora e espaço, pelo fato de essa operação requerar memória adicional para o espaço de armazenamento temporário e executar um cópia extra de dados na área temporária.

Usando *sendfile*, a área de armazenamento pode ser eliminada. Chame *sendfile*, informando o descritor de arquivo no qual deve ser feita a cópia; o

descritor que deve ser lido; um apontador para uma variável do tipo *offset*; e o número de *bytes* a serem transferidos. A variável do tipo *offset* contém o *offset* no arquivo de entrada do qual a leitura deve iniciar (0 indica o início do arquivo) e é atualizado para a posição no arquivo após a transferência. O valor de retorno é o número de bytes transferidos. *Include <sys/sendfile.h>* em seu programa se ele for usar *sendfile*.

O programa na Listagem 8.10 é uma implementação simples mas extremamente eficiente de uma cópia de arquivo. Quando chamada com dois nomes de arquivo pela linha de comando, o programa da Listagem 8.10 copia o conteúdo do primeiro arquivo dentro de um arquivo nomeado pelo segundo. O programa usa *fstat* para determinar o tamanho, em *bytes*, do arquivo de orígem.

Listagem 8.10: (copy.c) Cópia de Arquivo Usando *sendfile*

```
1 #include <fcntl.h>
2 #include <stdlib.h>
3 #include <stdio.h>
4 #include <sys/sendfile.h>
5 #include <sys/stat.h>
6 #include <sys/types.h>
7 #include <unistd.h>
8
9 int main (int argc, char* argv[])
10 {
11   int read_fd;
12   int write_fd;
13   struct stat stat_buf;
14   off_t offset = 0;
15
16   /* Abre o arquivo de entrada.  */
17   read_fd = open (argv[1], O_RDONLY);
18   /* Coleta informa es do arquivo de entrada para obter seu tamanho.  */
19   fstat (read_fd, &stat_buf);
20   /* Abre o arquivo de sa da para escrita, com as mesmas permiss es
21      que o arquivo de or gem.  */
22   write_fd = open (argv[2], O_WRONLY | O_CREAT, stat_buf.st_mode);
23   /* Arremessa os bytes de um para o outro.  */
24   sendfile (write_fd, read_fd, &offset, stat_buf.st_size);
25   /* Fecha tudo.  */
26   close (read_fd);
27   close (write_fd);
28
29   return 0;
30 }
```

A chamada *sendfile* pode ser usada em muitos lugares para fazer cópias mais eficientes. Um bom exemplo é em um servidor *web* ou outro programa que trabalha em segundo plano dentro de uma rede, que oferece o conteúdo de um arquivo através de uma rede para um programa cliente. Tipicamente, uma requisição é recebida de um *socket* conectado ao computador cliente. O programa servidor abre um arquivo no disco local para recuperar os dados a serem servidos e escrever o conteúdo do arquivo para o *socket* de rede. O uso *sendfile* pode aumentar a velocidade dessa operação considerávelmente. Outros passos perecisam ser seguidos para fazer a transferência de rede tão eficiente quanto possível, tais como ajustar os parêmetros do *socket* corretamente. Todavia, esses passo estão fora do escopo desse livro.

8.13 A Chamada *setitimer*: Ajustando Intervalos em Temporizadores

A chamada de sistema *setitimer* é uma generalização da chamada *alarm*. A chamada de sistema *setitimer* agenda a entrega de um sinal em algum ponto no futuro após um intervalo fixo de tempo ter passado.

Um programa pode ajustar três diferentes tipos de temporizadores com *setitimer*:

- Se o código do temporizador for ITIMER_REAL, ao processo é enviado um sinal SIGALRM após o tempo de relógio normal especificado ter passado.

- Se o código do temporizador for ITIMER_VIRTUAL, ao processo é enviado um sinal SIGVTALRM após o processo ter executado por um tempo específico. Tempo no qual o processo não está executando (isto é, quando o *kernel* ou outro processo está rodando) não é contado.

- Se o código do temporizador for ITIMER_PROF, ao processo é enviado um sinal SIGPROF quando o tempo especificado tiver passado ou durante a execução própria do processo ou da execução de uma chamada de sistema por parte do processo.

O primeiro argumento a *setitimer* é o código do temporizador, especificando qual o tipo de temporizador a ser ajustado. O segundo argumento é um apontador a um objeto *struct itimerval* especificando o novo ajuste paa aquele temporizador. O terceiro argumento, se não for vazio, é um apontador outro objeto *struct itimerval* que recebe os antigos ajustes de temporizador.

Uma variável *struct itimerval* tem dois campos:

- *it_value* é um campo *struct timeval* que contém o tempo até a próxima ativação do temporizador e o sinal é enviado. Se esse valor for 0, o temporizador está desabilitado.

- *it_interval* é outro campo *struct timeval* contendo o valor para o qual o temporizador irá ser zerado após esse tempo passar. Se esse valor for 0, o temporizador irá ser desabilitado após esse valor expirar. Se esse valor for diferente de zero, o temporizador é ajustado para expirar repetidamente após esse intrevalo.

O tipo *struct timeval* é descrito na Seção 8.7, "A Chamada *gettimeofday*: Hora relógio Comum".

O programa na Listagem 8.11 ilustra o uso de *setitimer* para rastrear o tempo de execução de um programa. Um temporizador é configurado para expirar a cada 250 milisegundos e enviar um sinal SIGVTALRM.

Listagem 8.11: (itimer.c) Exemplo de Temporizador

```
1 #include <signal.h>
2 #include <stdio.h>
3 #include <string.h>
4 #include <sys/time.h>
5
6 void timer_handler (int signum)
7 {
8   static int count = 0;
9   printf ("cron metro zerou %d vezes\n", ++count);
10 }
11
12 int main ()
13 {
14   struct sigaction sa;
15   struct itimerval timer;
16
17   /* Instala timer_handler como o manipulador de sinal para SIGVTALRM.  */
18   memset (&sa, 0, sizeof (sa));
19   sa.sa_handler = &timer_handler;
20   sigaction (SIGVTALRM, &sa, NULL);
21
22   /* Configura oretorno a zero ap s 250 milisegundos...  */
23   timer.it_value.tv_sec = 0;
24   timer.it_value.tv_usec = 250000;
25   /* ... e a cada 250 milisegundos ap s o primeiro retorno a zero.  */
26   timer.it_interval.tv_sec = 0;
27   timer.it_interval.tv_usec = 250000;
28   /* Inicia um cron metro virtual.  Sua contagem decresce mesmo se o processo
        estiver
29      executando.  */
30   setitimer (ITIMER_VIRTUAL, &timer, NULL);
31
32   /* Encarrega-se do trabalho.  */
33   while (1);
34 }
```

8.14 A Chamada de Sistema *sysinfo*: Obtendo Estatísticas do Sistema

A chamada de sistema *sysinfo* preenche uma estrutura com estatísticas do sistema. O único argumento da chamada de sistema *sysinfo* é um apontador para uma *struct sysinfo*. Alguns dos mais interessantes campos de *struct sysinfo* que são preenchidos incluem os seguintes:

- *uptime* – Tempo decorrido desde o boot do sistema, em segundos
- *totalram* – Total de memória RAM física disponível
- *freeram* – Memória RAM livre
- *procs* – Número de processos no sistema

Veja a página de manual da *sysinfo* para uma completa descrição da *struct sysinfo*. *Include* <*linux/kernel.h*>, <*linux/sys.h*>, e <*sys/sysinfo.h*> se você usa *sysinfo*.

O programa na Listagem 8.12 mostra algumas estatísticas sobre o sistema atual.

Listagem 8.12: (sysinfo.c) Mostra Estatísticas do Sistema

```
1 #include <linux/kernel.h>
2 #include <linux/sys.h>
3 #include <stdio.h>
4 #include <sys/sysinfo.h>
5
6 int main ()
7 {
8   /* Constantes de convers o.  */
9   const long minute = 60;
10  const long hour = minute * 60;
11  const long day = hour * 24;
12  const double megabyte = 1024 * 1024;
13  /* Obt m estat sticas do sistema.  */
14  struct sysinfo si;
15  sysinfo (&si);
16  /* Sumariza valores interessantes.  */
17  printf ("tempo ligado : %ld days, %ld:%02ld:%02ld\n",
18          si.uptime / day, (si.uptime % day) / hour,
19          (si.uptime % hour) / minute, si.uptime % minute);
20  printf ("RAM total            : %5.1f MB\n", si.totalram / megabyte);
21  printf ("RAM livre            : %5.1f MB\n", si.freeram / megabyte);
22  printf ("contador de processos: %d\n", si.procs);
23
24  return 0;
25 }
```

8.15 A Chamada de Sistema *uname*

A chamada de sistema *uname* preenche uma estrutura com várias informações do sistema, incluindo o nome de rede computador e o nome de domínio, e a versão do sistema operacional que está rodando. Informe a *uname* um único argumento, um apontador para um objeto *struct utsname*. *Include* <*sys/utsname.h*> se você usa *uname*.

A chamada a *uname* preenche os campos abaixo:

- *sysname* – O nome do sistema operacional (tal como GNU/Linux).
- *release, version* – O número de *release* do *kernel* do GNU/Linux e nível da versão.
- *machine* – Alguma informação sobre a plantaforma do *hardware* rodando GNU/Linux. Para x86 GNU/Linux, a plantaforma é i386 ou i686, dependendo do processador.
- *node* – O nome de *host* não qualificado do computador.
- *__domain* – O nome de domínio do computador.

Cada um desses campos é sequência de caracteres.

O pequeno programa na Listagem 8.13 mostra o *release* do GNU/Linux e o número de versão e a informação de *hardware*.

Listagem 8.13: (print-uname.c) Mostra o número de Versão do GNU/Linux e Informação de *Hardware*

```
1 #include <stdio.h>
2 #include <sys/utsname.h>
3
4 int main ()
5 {
6   struct utsname u;
7   uname (&u);
8   printf ("%s liberado %s (vers o %s) sobre %s\n", u.sysname, u.release,
9           u.version, u.machine);
10  return 0;
11 }
```

Capítulo 9

Código Assembly Embutido

HOJE, POUCOS PROGRAMADORES USAM A LINGUAGEM *ASSEMBLY*. Linguagens de alto nível tais C e C++ rodam sobre praticamente todas as arquiteturas e retornam alta produtividade em escrita e manutenção de código. Para ocasiões em que programadores precisam usar instruções *assembly* em seus programas, a Coleção de Compiladores GNU (*GNU Compiler Collection*) permite aos programadores adicionar instruções em linguagem *assembly* independentes da arquitetura a seus programas.

As declarações em *assembly* embutido da GCC não deve ser usado indiscriminadamente. Instruções em linguagem *assembly* são dependentes da arquitetura, de forma que, por exemplo, programas usando instruções *x86* não podem ser compilados sobre computadores *PowerPC*. Para usar instruções em *assembly*, você irá precisar de uma facilidade na linguagem *assembly* para sua arquitetura. Todavia, declarações em *assembly* embutido permitem a você acessar o *hardware* diretamente e pode também retornar código de execução mais rápido.

Uma instrução *asm* permite a você inserir instruções da linguagem *assembly* dentro de seus programas C e C++. Por exemplo, essa instrução

```
asm ("fsin" : "=t" (resposta) : "0" (angulo));
```

é uma forma específica da arquitetura *x86* de codificar a seguinte declaração em C [1]:

```
resposta = sin (angulo);
```

[1] A expressão *sin (angulo)* é comumente implementada como uma chamada de função dentro da biblioteca *math*, mas se você especificar o sinalizador de otimização -O1 ou maior, o GCC é esperto o suficiente para substituir a chamada de função com uma única instrução *fsin* na linguagem *assembly*.

Observe que diferentemente das instruções em código *assembly* comum, declarações *asm* permitem a você especificar operandos de entrada e saída usando sintaxe em C[2].

Para ler mais sobre o conjunto de instruções da arquitetura *x86*, que iremos usar nesse capítulo, veja `http://developer.intel.com/design/pentiumii/manuals/`[3] e `http://www.x86-64.org/documentation.html`[4].

9.1 Quando Usar Código em *Assembly*

Embora declarações *asm* possam ser abusadas, elas permitem a seus programas acessar um *hardware* do computador diretamente, e as instruções *asm* podem produzir programas que executam rapidamente. Você pode usar as instruções *asm* quando estiver escrevendo código de sistema operacional que precisa interagir diretamente com *hardware*. Por exemplo, */usr/include/asm/io.h*[5] contém instruções em *assembly* para acessar portas de entrada/saída diretamente. O arquivo de código fonte do GNU/Linux */usr/src/linux/arch/i386/kernel/process.s* fornece outro exemplo, usando *hlt* no código do laço *idle*[6]. Veja outros arquivos de código fonte do GNU/Linux no */usr/src/linux/arch/* e */usr/src/linux/drivers/*[7].

Instruções *assembly* podem também aumentar a velocidade do laço mais interno de programas de computadores[8]. Por exemplo, Se a maior parte

[2]No slackware existem o *assembler as86* que é o mesmo *as*, o *nasm* e o *yasm*.

[3]Nota do tradutor:`http://www.intel.com/products/processor/manuals/index.htm`.

[4]Nota do tradutor. Outras fontes:
O Assembly-HOWTO que acompanha sua distribuição GNU/Linux
`http://asm.sourceforge.net/resources.html#tutorials`
`http://download.savannah.gnu.org/releases/pgubook/ProgrammingGroundUp-1-0-booksize.pdf`
`http://homepage.mac.com/randyhyde/webster.cs.ucr.edu/www.artofasm.com/index.html` `http://www.drpaulcarter.com/pcasm` `http://www.iaik.tugraz.at/content/teaching/bachelor_courses/rechnernetze_und_organisation/downloads/02_LinuxAssembly.pdf`
`http://heather.cs.ucdavis.edu/~matloff/50/LinuxAssembly.html`
`http://gcmuganda.faculty.noctrl.edu/classes/Winter10/220/gasmanual.pdf`
`http://gcmuganda.faculty.noctrl.edu/classes/Winter10/220/asm.pdf`
`http://fileadmin.cs.lth.se/cs/Education/EDA180/tools/intel.pdf`
`http://asm.sourceforge.net/` .

[5]Nota do tradutor: esse arquivo localiza-se em */usr/include/sys* no *kernel* 2.6.

[6]Nota do tradutor: nos fontes do *kernel* 2.6 temos dois arquivos ".s" o */usr/src/linux/arch/x86/kernel/asm-offsets.s* e o */usr/src/linux/kernel/bounds.s*.

[7]Nota do tradutor: veja a nota 4 para outros exemplos.

[8]Nota do tradutor: o laço mais interno é o que executa mais vezes.

do tempo de execução de um programa é calculando o seno e o cosseno dos mesmos ângulos, esse laço mais interno pode ser recodificado usando a instrução *x86 fsincos*[9]. Veja, por exemplo, */usr/include/bits/mathinline.h*, que empacota dentro de *macros* algumas sequência de *assembly* embutido que aumentam a velocidade de cálculo de funções transcedentes[10].

Você deve uar *assembly* embutido para fazer códigos mais rápidos somente em último caso. Atualmente os compiladores estão bastante sofisticados e conhecem muito sobre os detalhes dos processadores para os quais eles geram código. Portanto, compiladores podem muitas vezes mudar sequências de código que podem ser vistas como não intuitivas ou que voltam ao mesmo lugar mas que no momento executam mais rápido que outras sequências de instrução. A menos que você entenda o conjunto de instruções e os atributos de agendamento de tarefas de seu processador alvo muito bem, você está provavelmente melhor pegando o código *assembly* otimizado gerado pelo compilador para você na maioria das operações.

Ocasionalmente, uma ou duas instruções em *assembly* podem substituir muitas linhas de código de uma linguagem de alto nível. Por exemplo, a determinação da posição do *bit* mais significativo diferente de zero de um inteiro não nulo usando a linguagem de programação C requer um laço de computações em ponto flutuante. Muitas arquiteturas, incluindo a *x86*, possuem uma única instrução *assembly* (*bsr*) para calcular a posição desse *bit*. Iremos demonstrar o uso de uma dessas na Seção 9.4, "Exemplo".

9.2 *Assembly* Embutido Simples

Aqui introduziremos a sintaxe das intruções do *assembler asm* com um exemplo *x86* para deslocar um valor de 8 *bits* para a direita:

```
asm ("shrl $8, %0" : "=r" (resposta) : "r" (operando) : "cc");
```

A palavra chave *asm* é seguida por uma expressão entre parêntesis consistindo de seções separadas por dois pontos. A primeira seção contém uma instrução em *assembler* e seus operandos. Nesse exemplo, *shrl* desloca para a direita os *bits* em seu primeiro operando. Seu primeiro operando é representado por %0. Seu segundo operando é a constante imediata $8.

A segunda seção especifica as saídas. A saída de uma instrução irá ser colocada na varável C *resposta*, que deve ser um *lvalue*. A sequência de

[9]Algorítmos ou modificações em estruturas de dados podem ser mais efetivos em reduzir um tempo de execução de programa que usar instruções *assembly*.

[10]Nota do tradutor: veja [Djairo (1985)] para mais informações sobre números transcendentes.

caracteres "=r" contém um sinal de igual indicando um operando de saída e uma letra *r* indicando que a *resposta* está armazenada em um registrador.

A terceira seção especifica as entradas. O operando da varável C especifica o valor para deslocar. A sequência de caractere "r" indica que esse valor a deslocar está armazenado em um registro mas omite um sinal de igualdade pelo fato de esse operando ser um operando de entrada, não um operando de saída.

A quarta seção indica que a instrução muda o valor na condição de código de registrador *cc*.

9.2.1 Convertendo Instruções *asm* em Instruções *Assembly*

O tratamento do GCC de declarações *asm* é muito simples. O GCC produz instruções *assembly* para tratar os operandos *asm*, e o GCC substitui a declaração *asm* com a instrução que você especifica. O GCC não analiza a instrução de nenhuma forma.

Por exemplo, GCC converte esse fragmento de programa[11]

```
double foo, bar;
asm ("mycool_asm %1, %0" : "=r" (bar) : "r" (foo));
```

para essas intruções *assembly x86*:

```
          movl -8(%ebp),%edx
          movl -4(%ebp),%ecx
#APP
          mycool_asm %ecx, %edx
#NO_APP
          movl %edx,-16(%ebp)
          movl %ecx,-12(%ebp)
```

Lembrando que *foo* e *bar* cada um requer duas palavras de armazenamento de pilha sob a arquitetura *32-bit x86*. O registrador *ebp* aponta para dados na pilha.

As primeiras duas instruções copiam *foo* para os registradores EDX e ECX nos quais *mycool_asm* opera. O compilador decide usar os mesmos registradores para para armazenar a resposta, a qual é copiada para *bar* pelas duas instruções finais. O compilador escolhe os registradores apropriados, inclusive reutilizando os mesmos registradores, e copia operandos para e das localizações apropriadas automaticamente.

[11]Nota do tradutor: veja I.1 para outro exemplo.

9.3 Sintaxe *Assembly* Extendida

Nas subseções que seguem, descrevemos as regras de sintaxe para declarações *asm*. Suas seções são separadas por dois pontos.

Iremos referir-nos à seguinte declaração *asm* ilustrativa, que calcula a expressão booleana x > y:

```
asm ("fucomip %%st(1), %%st; seta %%al" :
     "=a" (result) : "u" (y), "t" (x) : "cc", "st");
```

Primeiro, *fucomip* compara seus dois operandos x e y, e armazena valores indicando o resultado dentro do registrador de código de condição "cc". Então *seta* converte esses valores em um resultado 0 ou em um resultado 1.

9.3.1 Instruções *Assembler*

A primeira seção contém as instruções *assembler*, envolvidas entre aspas duplas. O exemplo *asm* contém duas instruções *assembly*, *fucomip* e *seta*, separadas por um ponto e vírgula. Se o *assembler* não permitir ponto e vírgula, use caracteres de nova linha (\n) para separar instruções.

O compilador ignora o conteúdo dessa primeira seção, a menos que um nível de sinal de porcentagem seja removido, de forma que %% seja mudado para %. O significado de *%%st(1)* e outros termos semelhantes é dependente da arquitetura.

GCC irá reclamar se você especificar a opção "*-traditional*" ou a opção "*-ansi*" ao compilar um programa contendo declarações *asm*. Para evitar a produção desses erros, como ocorre em arquivos de cabeçalho, use a palavra chave alternativa __asm__.

9.3.2 Saídas

A segunda seção especifica os operandos das instruções de saída usando a sintaxe do C. Cada operando é especificado por um sequência de caracteres abreviada de operando seguida por uma expressão em C entre parentesis. Para operandos de saída, os quais devem ser *lvalues*, a sequência de caracteres abreviada deve começar com um sinal de igual. O compilador verifica que a expressão em C para cada operando de saída seja de fato um *lvalue*.

Letras especificando registradores para uma arquitetura em particular podem ser encontrados no código fonte do GCC, na macro REG_CLASS_FROM

Tabela 9.1: Letras de registradores para a Arquitetura *x86 Intel*.

Letra de Registrador	Registrador Que GCC Pode Usar
R	Registrador geral (EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP)
q	Registrador geral para dados (EAX, EBX, ECX, EDX)
f	Registrador de ponto flutuante
t	Primeiro registrador de nível mais alto em ponto flutuante
u	Segundo registrador de nível mais alto em ponto flutuante
a	Registrador EAX
b	Registrador EBX
c	Registrador ECX
d	Registrador EDX
x	Registrador SSE (Registrador de extensão de fluso de SIMD)
y	Registrador MMX registradores multimedia
A	Um valor de *8-byte* formado a partir de EAX e de EDX
D	Apontador de destino para operações de sequência de caracteres (EDI)
S	Apontador de orígem para operações de sequência de caracteres (ESI)

_LETTER[12]. Por exemplo, o arquivo gcc/config/i386/i386.h[13] de configuração no GCC lista as letras de registradores para a arquitetura *x86*[14]. A Tabela 9.1 sumariza isso.

Multiplos operandos em uma declaração *asm*, cada um especificado por uma constante no formato de uma sequência de caracteres abreviada e uma expressão C, são separados por vírgulas, como ilustrado no exemplo da seção de entradas *asm*. Você pode especificar até 10 operandos, denotados por %0, %1, ..., %9, em seções de saída e em seções de entrada. Se não existirem operandos de saída mas houverem operandos de entrada ou registradores de

[12]Nota do tradutor: encontra-se referência a REG_CLASS_FROM_LETTER em `/usr/lib64/gcc/x86_64-slackware-linux/4.5.2/plugin/include/defaults.h`.

[13]Nota do tradutor: `/usr/lib64/gcc/x86_64-slackware-linux/4.5.2/plugin/include/config/i386/i386.h`.

[14]Você precisará ter alguma familiaridade com a parte interna do GCC para ver algum sentido nesse arquivo.

crítica, mantenha a seção de saída vazia ou marque a seção de saída como comentário como /* saídas vazias */.

9.3.3 Entradas

A terceira seção especifica os operandos de entrada para as instruções *assembler*. A constante de sequência de caracteres não deve ter um sinal de igual, o qual indica um *lvalue*. De outra forma, a sintaxe de operandos de entrada é a mesma para operandos de saída.

Para indicar que cada registrador é ambos de "leitura de" e "escrever para" o mesmo *asm*, use uma entrada de sequência de caracteres abreviada do número de operandos de saída. Por exemplo, para indicar que um registrador de entrada é o mesmo que o primeiro número de registrador de saída, use 0. Operandos de saída são numerados da esquerda para a direita, iniciando em 0. Meramente especificando a mesma expressão em C para um operando de saída e um operando de entrada não garante que os dois valores irão ser colocados no mesmo registrador.

Essa seção de entrada pode ser omitida se não houverem operandos de entrada e a subsequênte seção de crítica estiver vazia.

9.3.4 Crítica

Se uma instrução modifica os valores de um ou mais registradores como um efeito colateral, especifique os registradores criticados na quarta seção do comando *asm*. Por exemplo, a instrução *fucomip* modifica o registrador de código de condição, o qual é denotado por *cc*. Sequências de caractere separadas representam registradores criticados com vírgulas. Se a instrução pode modificar uma localização de memória arbitrária, especifique a memória. Usando a informação de crítica, o compilador determina quais os valores que devem ser recarregados após a linha *asm* ser executada. Se você não especifica essa informação corretamente, GCC pode assumir incorretamente que registradores ainda conteem valores que tenham, de fato, sido sobrescritos, o que irá afetar a correção de seu programa.

9.4 Exemplo

A arquitetura *x86* inclui instruções que determinam as posições do menos significativo conjunto de *bit* e do mais significativo conjunto de *bit* em uma palavra. O processador pode executar essas instruções muito eficientemente.

Ao contrário, implementando a mesma operação em C requer um laço e um deslocamento de *bit*.

Por exemplo, a instrução *assembly bsrl*[15] calcula a posição do conjunto de *bits* mais significativo em seu primeiro operando, e coloca a posição (contando a partir do 0, o *bit* menos significativo) dentro do seu segundo operando. Para colocar a posição do *bit* para número dentro da posição, podemos usar essa declaração *asm*:

```
asm ("bsrl %1, %0" : "=r" (posicao) : "r" (numero));
```

Um caminho através do qual você pode implementar a mesma operação em C é usando o seguinte laço:

```
long i;
for (i = (numero >> 1), posicao = 0; i != 0; ++posicao)
  i >>= 1;
```

Para testar a velocidad relativa dessas duas versões, iremos colocá-las em um laço que calcula as posições de *bit* para um grande número de valores. A Listagem 9.1 faz isso usando o laço na implementação em C. O programa entra no laço a cada inteiro, de 1 até o valor especificado na linha de comando. Para cada valor de número, o programa calcula o *bit* mais significativo que é escolhido. A Listagem 9.2 faz a mesma coisa usando a instrução em *assembly* embutido. Note que em ambas as versões, atribuimos à posição de *bit* calculada a uma variável de resultado do tipo *volatile*. Isso é para forçar a otimização do compilador de forma que o otimizador do compilador não elimine a computação da posição do *bit* completamente; se o resultado não é usado ou não é armazenado na memória, o otimizador elimina a computação como "código morto".

[15]Nota do tradutor: veja I.2 para maiores detalhes.

Listagem 9.1: (bit-pos-loop.c) Encontra a Posição do *Bit* Usando um Laço

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main (int argc, char* argv[])
5 {
6   long max = atoi (argv[1]);
7   long number;
8   long i;
9   unsigned position;
10  volatile unsigned result;
11
12  /* Repita a opera  o para um grande conjunto de valores.  */
13  for (number = 1; number <= max; ++number) {
14    /* Repetidamente mude o n mero para a direita, at  que o resultado seja
15       zero.  Continue contando o n mero de mudan as que essa opera  o requer.
            */
16    for (i = (number >> 1), position = 0; i != 0; ++position)
17      i >>= 1;
18    /* A posi  o do conjunto de bit mais significativo    o n mero de
19       mudan as para a direita que precisamos ap s a primeira.  */
20    result = position;
21  }
22
23  return 0;
24 }
```

Listagem 9.2: (bit-pos-asm.c) Encontra a posição do *Bit* Usando *bsrl*

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main (int argc, char* argv[])
5 {
6   long max = atoi (argv[1]);
7   long number;
8   unsigned position;
9   volatile unsigned result;
10
11  /* Repita a opera  o para um grande n mero de valores.  */
12  for (number = 1; number <= max; ++number) {
13    /* Calcule a posi  o do mais significativo conjunto de bit usando a
14       instru  o assembly bsrl.  */
15    asm ("bsrl %1, %0" : "=r" (position) : "r" (number));
16    result = position;
17  }
18
19  return 0;
20 }
```

Compilaremos ambos com otimização completa:

```
% cc -O2 -o bit-pos-loop bit-pos-loop.c
% cc -O2 -o bit-pos-asm bit-pos-asm.c
```

Agora vamos executar cada um usando o comando *time* para medir o tempo de execução. Especificaremos um valor grande como argumento de linha de comando, para garantir que cada versão demore pelo menos alguns segundos para executar.

```
% time ./bit-pos-loop 250000000
19.51user 0.00system 0:20.40elapsed 95%CPU (0avgtext+0avgdata
0maxresident)k0inputs+0outputs (73major+11minor)pagefaults 0swaps
% time ./bit-pos-asm 250000000
3.19user 0.00system 0:03.32elapsed 95%CPU (0avgtext+0avgdata
0maxresident)k0inputs+0outputs (73major+11minor)pagefaults 0swaps
```

Note que a versão que usa *assembly* embutido executa com larga margem mais rapidamente (seus resultados para esse exemplo podem variar).

9.5 Recursos de Otimização

O otimizador do GCC tenta rearranjar e reescrever códigos de programas para minimizar o tempo de execução mesmo na presença de expressões *asm*. Se o otimizador determinar que um valor de saída da expressão *asm* não é usada, a instrução irá ser omitida a menos que a palavra chave *volatile* ocorra entre *asm* e seus argumentos. (Como um caso especial, GCC não irá mover um *asm* sem qualquer operandos de saída fora de um laço.) Qualquer *asm* pode ser movido em caminhos que são difíceis de prever, mesmo em saltos. O único caminho para garantir que uma instrução *assembly* em uma ordem em particular é incluir todas as instruções em um mesmo *asm*.

Usando *asms* podemos restringir a eficiência do otimizador pelo fato de o compilador não conhece a semântica dos *asms*. GCC é forçado a suposições conservadoras que podem prevenir algumas otimizações. *Caveat emptor!*[16]

9.6 Manutensão e Recursos de Portabilidade

Caso você resolva usar declarações *asm* não portáveis e dependentes da arquitetura, encapsulando essas declarações dentro de *macros* ou funções pode ajudar na manutensão e na portabilidade. Colocando todas essas macros em um arquivo e documentando-as irá ser fácil portá-las para uma arquitetura diferente, alguma coisa que ocorre com surpreendente frequência mesmo para programas "jogados-longe". Dessa forma, o programador irá precisar re-escrever somente um arquivo para a arquitetura diferente.

Por exemplo, a maioria das declarações *asm* em código fonte GNU/Linux estão agrupadas nos arquivos de cabeçalho /usr/src/linux/include/asm e em /usr/src/linux/include/asm-i386[17], e arquivos fonte em /usr/src/linux/arch/i386/ e /usr/src/linux/drivers/.

[16]Nota do tradutor: expressão latina que quer dizer "o risco é do comprador".

[17]Nota do tradutor: no slackware 13.37 64 bits temos três diretórios asm: o asm (que é um link) o asm-x86 e o asm-generic.

Capítulo 10

Segurança

GRANDE PARTE DO PODER DE UM SISTEMA GNU/LINUX VEM DE seu suporte a múltiplos usuários e a trabalhos em rede. Muitas pessoas podem usar o sistema de uma só vez, e essas pessoas podem conectar-se ao sistema a partir de localizações remotas. Desafortunadamente, com esse poder vem riscos, especialmente para sistemas conectados à *Internet*. Sob algumas circunstâncias, um "*hacker*"[1] pode conectar-se ao sistema e ler, modificar, ou remover arquivos que estão armazenados na máquina. Ou, dois usuários na mesma máquina podem ler, modificar, ou remover arquivos do outro quando eles não deveriam ter permissão para fazer isso. Quando isso acontece, a segurança do sistema foi comprometida.

O *kernel* do GNU/Linux fornece uma variedade de facilidades para garantir que esses eventos não tenham lugar. Mas para evitar violações de segurança, aplicações comuns devem ser cuidadosas também. Por exemplo, imagine que você está desenvolvendo um programa de contabilidade. Embora você possa desejar que todos os usuários sejam capazes de enviar relatórios de despesa com o sistema, você pode não desejar que todos os usuários sejam capazes de aprovar esses relatórios. Você pode desejar que usuários sejam capazes de ver suas próprias informações salariais enquanto funcionários da empresa, mas você certamente não irá desejar que eles sejam capazes de ver as informações salariais de todos os outros funcionários. Você pode desejar que gerentes sejam capazes de ver os salários de empregados de seus departamentos respectivos, mas você não irá desejar que os gerentes vejam os salários de empregados de outros departamentos.

Para fazer cumprir esses tipos de controle, você deve ser muito cuidadoso. É surpreendentemente fácil cometer um erro que permite ao usuários fazer alguma coisa que você não tinha intenção de permitir que eles fossem fazer. A

[1] Nota do tradutor: as aspas procuram ressaltar a diferença entre o significado popular e o verdadeiro significado da palavra *hacker*.

melhor abordagem é conseguir ajuda de *experts* em segurança. Não obstante, todo desenvolvedor de aplicação deveria entender o básico.

10.1 Usuários e Grupos

A cada usuário GNU/Linux é atribuído um número único, chamado ID de usuário, ou UID. Certamente quando você coloca sua senha, você usa um nome de usuário em lugar do ID[2]. O sistema converte seu nome de usuário em um ID particular de usuário, e a partir de então somente o ID de usuário passa a ser considerado.

Você pode atualmente ter mais de um nome de usuário para o mesmo ID de usuário. No que diz respeito ao sistema, os IDs de usuário, não os nomes de usuário, devem coincidir. Não existe forma de fornecer um nome de usuário mais forte que outro se eles ambos correspondem ao mesmo ID de usuário.

Você pode controlar o acesso a um arquivo ou outro recurso associando esse arquivo ou recurso a um ID de usuário em particular. Então somente o usuário correspondendo àquele ID de usuário pode acessar o recurso. Por exemplo, você pode criar um arquivo que somente você pode ler, ou um diretório no qual somente você pode criar novos arquivos. Isso é bom o suficente para muitos casos simples.

Algumas vezes, todavia, você deve compartilhar um recurso entre multiplos usuários. Por exemplo, se você é um gerente, você pode desejar criar um arquivo que qualquer gerente pode ler mas que funcionários comuns não possam. GNU/Linux não permite a você associar multiplos IDs de usuário a um arquivo, de forma que você não pode apenas criar uma lista de todas as pessoas às quais você deseja fornecer acesso e anexar a essa lista todo o arquivo.

Você pode, no entanto, criar um grupo. A cada grupo é atribuído um número único, chamado ID de grupo, ou GID. Todo grupo contém um ou mais IDs de usuário. Um único ID de usuário pode ser membro de muitos grupos, mas grupos não podem conter outros grupos; os grupos podem conter somente usuários. Como usuário, grupos possuem nomes. Também como nomes de usuários, todavia, os nomes de grupo não precisam realmente coincidir; o sistema sempre usa o ID de grupo internamente.

Continuando nosso exemplo, você pode criar um grupo de gerentes e colocar os IDs de usuário de todos os gerentes nesse grupo. Você pode então criar um arquivo que possa ser lido por qualquer um no grupo de gerentes mas

[2]Nota do tradutor: *IDentification*.

não por pessoas que estejam fora do grupo. Em geral, você pode associar somente um grupo a um recurso[3]. Não existe forma para especificar que usuários podem acessar um arquivo somente se estiver no grupo 7 ou no grupo 42, por exemplo.

Se você está curioso para ver qual é seu ID de usuário e em quais grupos você está inscrito, você pode usar o comando *id*. Por exemplo, a saída pode se parecer como segue:

```
% id
uid=501(mitchell) gid=501(mitchell) groups=501(mitchell),503(csl)
```

A primeira parte mostra a você que o ID de usuário para o usuário que executou o comando era 501. O comando também mostra qual o correspondente nome de usuário que está sendo usado entre parêntesis. O comando mostra que o ID de usuário 501 está atualmente em dois grupos: grupo 501 (chamado *mitchell*) e grupo 503 (chamado *csl*). Você está provavelmente maravilhado pelo fato de o grupo 501 aparecer duas vezes: uma vez no campo *gid* e outra vez no campo de *groups*. Iremos falar sobre isso mais tarde.

10.1.1 O Superusuário

Uma conta de usuário é muito especial[4]. Esse usuário tem ID de usuário 0 e comumente tem o nome de usuário *root*. Essa conta especial de usuário é também algumas vezes referida como a conta de superusuário. O usuário *root* pode fazer qualquer coisa: ler qualquer arquivo, remover qualquer arquivo, adicionar novos usuários, desligar o acesso à rede, e assim por diante. Grandes quantidades de operações especiais podem ser executadas somente por processos executando com privilégio de *root* – isto é, executando como usuário *root*.

O problema com esse projeto é que uma grande quantidade de programas precisa ser executado pelo *root* pelo fato de uma grande quantidade de programas precisar executar uma dessas operações especiais. Se qualquer desses programas se comporta mal, o caos pode ser o resultado. Não existe uma maneira efetiva para conter um programa quando ele está sendo executado pelo *root*; o *root* não pode fazer nada. Programas executando pelo *root* devem ser escritos muito cuidadosamente.

[3]Nota do tradutor: um arquivo não pode ter dois grupos ou dois donos.

[4]O fato de existir somente um usuário especial deu a AT&T o nome para seu sistema operacional UNIX. Ao contrário, um sistema operacional anterior que tinha diversos usuários especiais era chamado MULTICS. GNU/Linux, certamente, é muito mais compatível com UNIX.

10.2 IDs de Usuário e IDs de Grupo

Até agora, nós vinhamos falando sobre comandos sendo executados por um usuário em particular. Isso não é bem verdade pelo fato de o computador nunca realmente saber qual usuário está usando o referido programa que está sendo executado. Se o usuário *Eve* descobre o nome de usuário e a senha do usuário *Alice*, então o usuário *Eve* pode se conectar ao computador como *Alice*, e o computador irá permitir a *Eve* fazer tudo que *Alice* pode fazer. O sistema sabe somente qual o ID do usuário em uso, não qual o usuário que está digitando os comandos. Se *Alice* não pode ser confiável em manter sua senha somente consigo mesma, por exemplo, então nada que você fizer enquanto desenvolvedor de uma aplicação irá evitar *Eve* de acessar os arquivos de *Alice*. A responsabilidade pela segurança do sistema é compartilhada entre o desenvolvedor da aplicação, os usuários do sistema, e dos administradores do sistema.

Todo processo tem um ID de usuário e um ID de grupo associado. Quando você chama um comando, esse comando tipicamente executa em um processo cujos ID de usuário e ID de grupo são iguais aos seus ID de usuário e ID de grupo. Quando dizemos que um usuário executa uma operação, realmente significa que um processo com o correspondente ID de usuário executa aquela operação. Quando o processo faz uma chamada de sistema, o *kernel* decide se permite que a operação continue. O *kernel* decide examinando as permissões associadas ao recurso que o processo está tentando acessar e verificando o ID de usuário e ID de grupo associado ao processo tentando executar a ação.

Agora você sabe o que aquele campo do meio mostrado pelo comando *id* significa. Ele está mostrando o ID de grupo do processo atual. Apesar de o usuário 501 estar em multiplos grupos, o processo atual pode ter um único ID de grupo. No exemplo mostrado previamente, o atual ID de grupo é 501.

Se você tem que controlar IDs de usuário e IDs de grupo em seu programa (e você irá, se você está escrevendo programas que lidam com segurança), então você deve usar os tipos *uid_t* e *gid_t* definidos em <sys/types.h>[5]. Apesar de IDs de usuário e IDs de grupo serem essencialmente apenas inteiros, evite fazer qualquer suposições sobre quantos *bits* são usados nesses tipos ou executar operações aritméticas sobre eles. Apenas trate os tipos *uid_t* e *gid_t* como controladores opacos para a identidade de usuário e identidade de grupo.

Para pegar o ID de usuário e o ID de grupo para o processo atual, você pode usar as funções *geteuid* e *getegid*, declaradas em <unistd.h>[6]. Essas

[5]Nota do tradutor: /usr/include/sys/types.h.
[6]Nota do tradutor: /usr/include/unistd.h.

funções não recebem parâmetros, e funcionam sempre; você não deve verificar erros. A Listagem 10.1 mostra um programa simples que fornece um subconjunto de funcionalidades fornecida pelo comando *id*:

Listagem 10.1: (simpleid.c) Mostra ID de usuário e ID de grupo

```
1 #include <sys/types.h>
2 #include <unistd.h>
3 #include <stdio.h>
4
5 int main()
6 {
7   uid_t uid = geteuid ();
8   gid_t gid = getegid ();
9   printf ("uid=%d gid=%d\n", uid, gid);
10  return 0;
11 }
```

Quando esse programa é executado (pelo mesmo usuário que executou o programa *id* anteriormente) a saída é como mostrado abaixo:

```
% ./simpleid
uid=501 gid=501
```

10.3 Permissões do Sistema de Arquivos

Uma boa maneira de ver usuários e grupos em ação é olhar em permissões do sistema de arquivo. Por meio do exame de como o sistema associa permissões a cada arquivo e então vendo como o *kernel* verifica para descobrir quem está autorizado a acessar determinados arquivos, os conceitos de ID de usuário e ID de grupo devem tornar-se claros.

Cada arquivo tem exatamente um usuário proprietário e exatamente um grupo proprietário. Quando você cria um novo arquivo, o arquivo é possuído pelo usuário e pelo grupo do processo que o criou [7].

As coisas básicas que você pode fazer com arquivos, no que diz respeito ao GNU/Linux, são ler a partir de um arquivo, escrever no arquivo, e executar um arquivo. (Note que criar um arquivo e remover um arquivo não são consideradas coisas que você pode fazer com o arquivo; criar e remover são considerados coisas que você pode fazer com o diretório contendo o arquivo. Iremos falar sobre isso um pouco mais tarde.) Se você não tem permissão para ler um arquivo, GNU/Linux não irá permitir a você examinar o conteúdo do arquivo. Se você não tem permissão para escrever em um arquivo, você não pode mudar seu conteúdo. Se existe um arquivo de programa cuja permissão de execução você não tem, você não pode executar o programa.

[7] Atualmente, existem algumas raras excessões, envolvendo *sticky bits*, discutidos mais tarde na Seção 10.3.2, "*Sticky Bits*".

GNU/Linux habilita a você designar qual dessas três ações – leitura, escrita, e execução – pode ser executada pelo usuário proprietário, grupo proprietário, e todas os outros usuários. Por exemplo, você poderá dizer que o usuário proprietário pode fazer qualquer coisa que desejar com o arquivo, que qualquer um no grupo proprietário pode ler e executar o arquivo (mas não escrever no arquivo), e que ninguém mais fora os especificados anteriormente pode fazer qualquer coisa com o arquivo.

Você pode ver esses *bits* de permissão interativamente com o comando *ls* usando as opções "-l" ou "-o" e programaticamente como a chamada de sistema *stat*. Você pode ajustar os *bits* de permissão com o programa *chmod*[8] ou programaticamente com a chamada de sistema *chmod*. Para olhar as permissões sobre um arquivo chamado *hello*, use *ls -l hello*. Aqui está como a saída pode se parecer:

```
% ls -l hello
-rwxr-x---    1 samuel      csl          11734 Jan 22 16:29 hello
```

Os campos *samuel* e *csl* indicam que o usuário proprietário é *samuel* e que o grupo proprietário é *csl*.

A sequência de caracteres no início da linha indica as permissões associadas ao arquivo. O primeiro traço indica que o arquivo *hello* é um arquivo normal. Esse primeiro traço poderia ser *d* para um diretório, ou esse primeiro traço poderia ser outras letras para tipos especiais de arquivos tais como dispositivos (veja Capítulo 6, "Dispositivos") ou *pipes* nomeados (veja Capítulo 5, "Comunicação Entre Processos" Seção 5.4, "*Pipes*"). Os três caracteres seguintes mostram permissões para o usuário proprietário; eles indicam que *samuel* pode ler, escrever, e executar o arquivo. Os três caracteres seguintes mostram as permissões dos membros do grupo *csl*; esses membros possuem permissão para somente ler e executar o arquivo *hello*. Os últimos três caracteres mostram as permissões para todos os outros usuários restantes; a esses usuários restantes não é permitido fazer nada com o arquivo *hello*.

Vamos ver como isso trabalha. Primeiramente, vamos tentar acessar o arquivo como o usuário *nobody*, que não está no grupo *csl*:

```
% id
uid=99(nobody) gid=99(nobody) groups=99(nobody)
% cat hello
cat: hello: Permission denied
% echo hi > hello
```

[8]Você irá algumas vezes ver os *bits* de permissões de um arquivo sendo referenciados como o *modo* do arquivo. O nome do comando *chmod* é a forma curta para "*change mode*".

```
sh: ./hello: Permission denied
% ./hello
sh: ./hello: Permission denied
```

Não podemos ler o arquivo, o que causa a falha do comando *cat*; não podemos escrever no arquivo, o que causa a falha no comando *echo*; e não podemos executar o arquivo, o que causa a falha em "*./hello*".

As coisas ficam melhor se você estiver acessando o arquivo como *mitchell*, que é um membro do grupo *csl*:

```
% id
uid=501(mitchell) gid=501(mitchell) groups=501(mitchell),503(csl)
% cat hello
#!/bin/bash
echo "Hello, world."
% ./hello
Hello, world.
% echo hi > hello
bash: ./hello: Permission denied
```

Podemos listar o conteúdo do arquivo, e podemos executá-lo (o executável é um *script shell* simples), mas ainda não podemos escrever nele.

Se executamos o *script* como o proprietário (*samuel*), podemos até mesmo sobrescrever o arquivo:

```
% id
uid=502(samuel) gid=502(samuel) groups=502(samuel),503(csl)
% echo hi > hello
% cat hello
hi
```

Você pode mudar as permissões associadas a um arquivo somente se você for o proprietário do arquivo (ou o superusuário). Por exemplo, se você desejar permitir a todos executar o arquivo, você pode fazer isso:

```
% chmod o+x hello
% ls -l hello
-rwxr-x--x    1 samuel     csl             3 Jan 22 16:38 hello
```

Note que existe um x ao final da primeira sequência de caracteres. O *bit* $o{+}x$ significa que você deseja adicionar a permissão de execução para outras pessoas (que não o proprietário do arquivo ou os membros de seu grupo proprietário). Você pode usar $g{-}w$ ao invés de $o{+}x$, para remover a permissão de escrita para o grupo. Veja a página de manual de *chmod* na seção 1 para detalhes sobre essa sintaxe:

```
% man 1 chmod
```

Programaticamente, você pode usar a chamada de sistema *stat* para encontrar as permissões associadas a um arquivo. A Função *stat* recebe dois parâmetros: o nome do arquivo do qual você deseja encontrar a informação,

e o endereço de uma estrutura de dados que é preenchida com a infomação sobre o arquivo. Veja o Apêndice B,"E/S de Baixo Nível" Seção B.2, "stat" para uma discussão de outras informações que você pode obter com *stat*. A Listagem 10.2 mostra um exemplo de uso da *stat* para obter as permissões do arquivo.

Listagem 10.2: (stat-perm.c) Determina se o Proprietário do Arquivo Tem Permissão de Escrita

```
1 #include <stdio.h>
2 #include <sys/stat.h>
3
4 int main (int argc, char* argv[])
5 {
6   const char* const filename = argv[1];
7   struct stat buf;
8   /* Pega informa es do arquivo.  */
9   stat (filename, &buf);
10  /* Se as permiss es estiverem ajustadas de forma que o dono do arquivo possa
        escrever
11     no referido arquivo, mostre uma mensagem.  */
12  if (buf.st_mode & S_IWUSR)
13    printf ("Usu rio propriet rio pode escrever '%s'.\n", filename);
14  return 0;
15 }
```

Se você executa o programa acima sobre o nosso programa *hello*, ele vai retornar:

```
% ./stat-perm hello
Owning user can write 'hello'.
```

A constante S_IWUSR corresponde à permissão de escrita para o usuário proprietário. Existem outras constantes para todos os outros *bits* de permissão. Por exemplo, S_IRGRP é permissão de leitura para o grupo proprietário, e S_IXOTH é permissão de execução para usuários que não estão nem no usuário proprietário nem no grupo proprietário. Se você armazena permissões em uma variável, use o *typedef mode_t* para essa variável. Como a maioria das chamadas de sistema, *stat* irá retornar -1 e ajustar *errno* caso não possa obter informações sobre o arquivo.

Você pode usar a função *chmod* para mudar os *bits* de permissão de um arquivo existente. Você chama *chmod* com o nome do arquivo que você deseja mudar e os *bits* de permissão que você deseja ajustar, apresentados na forma de conjunto de *bits* ou na forma das várias constantes de permissão mencionadas previamente. Por exemplo, a linha adiante fará o *hello* legível e executável pelo seu usuário proprietário mas desabilitará todas as outras permissões associadas a *hello*:

```
chmod ("hello", S\_IRUSR | S\_IXUSR);
```

Os mesmos *bits* de permissão aplicam-se a diretórios, mas eles possuem outros significados. Se a um usuário é permitido ler um diretório, ao usuário é permitido ver a lista de arquivos que estão presentes naquele diretório. Se a um usuário é permitido escrever em um diretório, ao usuário é permitido adicionar ou remover arquivos no referido diretório. Note que um usuário pode remover arquivos de um diretório se esse mesmo usuário tem permissão de escrita no diretório, mesmo se ele não tem permissão para modificar o arquivo que ele está removendo. Se a um usuário é permitido executar um diretório, ao usuário é permitido entrar naquele diretório e acessar os arquivos dentro dele. Sem acesso de execução a um diretório, ao usuário não é permitido acessar os arquivos naquele diretório independentemente das permissões sobre os arquivos propriamente ditos.

Sumarizando, vamos revisar como o *kernel* decide permitir a um processo acessar um arquivo em particular. O *kernel* faz verificações para ver se o usuário que está acessando é o usuário proprietário, um membro do grupo proprietário, ou alguém mais. A categoria dentro da qual o usuário que está acessando falhar é usada para determinar qual o conjunto de *bits* de leitura/escrita/execução que é verificado. Então o *kernel* verifica a operação que está sendo executada contra os *bits* de permissão que se aplicam a esse usuário[9].

Existe uma importante excessão: Processos executando como *root* (aqueles com ID de usuário 0) possuem sempre permissão de acesso a qualquer arquivo, indiferentemente das permissões associadas.

10.3.1 Falha de Segurança: Sem Permissão de Execução

Aqui está o primeiro exemplo de onde a segurança encontra muitas complicações. Você pode pensar que se você desabilita a execução de um programa, então ninguém pode executa-lo. Afinal de contas, isto é o que significa deabilitar a execução. Mas um usuário malicioso pode fazer uma cópia do programa, mudar as permisses no sentido de tornar esse programa executável, e então executar a cópia! Se você acredita que os usuários não estão aptos a executar programas que não sejam executáveis mas não previne que eles copiem os programas, você tem uma falha de segurança – um meio através do qual usuários podem executar qualquer ação que você não quer que eles

[9] O *kernel* pode também proibir o acesso a um arquivo se um diretório componente no caminho do seu arquivo estiver inacessível. Por exemplo, se um processo não puder acessar o diretório */tmp/private/*, esse processo não pode ler o diretório */tmp/private/data*, mesmo se as permissões sobre esse ultimo estiverem ajustadas para permitir o acesso.

façam.

10.3.2 *Sticky Bits*

Adicionalmente a permissões para ler, escrever, e executar, existe um *bit* mágico chamado *sticky bit*[10]. Esse *bit* aplica-se somente a diretórios.

Um diretório que tem o *sticky bit* ativo permite a você apagar um arquivo somente se você for o dono do arquivo. Como mencionado previamente, você pode comumente apagar um arquivo se você tiver acesso de escrita para o diretório que o contém, mesmo se você não for o dono do arquivo. Quando o *sticky bit* está ativo, você ainda deve ter acesso de escrita ao diretório, mas você deve também ser o dono do arquivo que você quer apagar.

Uns poucos diretórios em um sistema típico GNU/Linux possuem o *sticky bit*. Por exemplo, o diretório */tmp*, no qual qualquer usuário pode colocar arquivos temporários, tem o *sticky bit* ativado. Esse diretório é especificamente disignado para ser usado por todos os usuários, de forma que o diretório deve ter permissão de escrita para todos os usuários. Mas essa permissão de escrita para todos os usuários pode não vir a ser uma boa coisa se um usuário puder vir a apagar arquivos de outro usuário, de forma que o *sticky bit* é ativado sobre aquele diretório. Então somente o usuário proprietário (ou o *root*, certamente) pode remover um arquivo.

Você pode ver se o *sticky bit* está ativo pelo fato de existir uma letra *t* no final dos *bits* de permissão quando você executa o comando *ls* sobre o diretório */tmp*:

```
% ls -ld /tmp
drwxrwxrwt   12 root       root          2048 Jan 24 17:51 /tmp
```

A constante correspondente a ser usada com as funções de linguagem C *stat* e *chmod* é S_ISVTX [11].

Se seu programa cria diretórios que comportam-se como o */tmp*, no qual muitas pessoas colocam coisas lá mas não devem estar aptas a remover arquivos de outras pessoas, então você deve ativar o *sticky bit* sobre o diretório. Você pode ativar o *sticky bit* sobre um diretório com o comando *chmod* da seguinte forma:

```
% chmod o+t directory
```

[10]Esse nome é anacronístico; *sticky bit* remonta ao tempo quando a ativação do *sticky bit* fazia com que um programa ficasse retido na memória principal mesmo quando esse programa terminasse sua execução. As páginas alocadas para o programa estavam "stuck"(coladas) na memória.

[11]Nota do tradutor: *man 2 chmod* e *man 2 stat*.

Para ajustar o *sticky bit* programáticamente, chame *chmod* com o sinalizador de modo S_ISVTX. Por exemplo, para ativar o *sticky bit* do diretório especificado em *dir_path* para as mesmas do */tmp* e fornecer leitura completa, escrita completa, e permissão de execução para todos os usuários, use essa chamada:

```
chmod (dir_path, S_IRWXU | S_IRWXG | S_IRWXO | S_ISVTX);
```

10.4 ID Real e ID Efetivo

Até agora, falamos sobre o ID de usuário e o ID de grupo associados a um processo como se existisse somente um tal ID de usuário e um tal ID de grupo. Mas, atualmente, isso não é tão simples assim.

Todo processo realmente tem dois IDs de usuário: o ID efetivo de usuário e o ID real de usuário. (Certamente, existe também um ID efetivo de grupo e um ID real de grupo. Quase tudo que é verdadeiro sobre IDs de usuário é também verdadeiro sobre IDs de grupo.) A maioria das vezes, o *kernel* verifica somente o ID efetivo de usuário. Por exemplo, se um processo tenta abrir um arquivo, o *kernel* verifica o ID efetivo de usuário para decidir se permite que o processo acesse o arquivo.

As funções *geteuid* e *getegid* descritas previamente retornam o ID efetivo de usuário e o ID efetivo de grupo. De forma análoga as funções *getuid* e *getgid* retornam o ID real de usuário e o ID real de grupo.

Se o *kernel* cuida somente do ID efetivo de usuário, não parece haver muito razão em ter distinção entre um ID real de usuário e um ID efetivo de usuário. Todavia, existe um caso muito importante no qual o ID real de usuário é importante. Se você deseja mudar o ID efetivo de usuário de um processo já existente que está executando no presente momento, o *kernel* olha para o ID real de usuário bem como o ID efetivo de usuário.

Antes de olhar para como você pode mudar o ID efetivo de usuário de um processo, vamos examinar porque você desejaria fazer a mudança do usuário efetivo olhando de volta para nosso programa de contabilidade. Suponhamos que haja um processo servidor que pode precisar olhar em qualquer arquivo no sistema, indiferentemente do usuário que o criou. Então o processo deve executar como *root* pelo fato de somente ao *root* pode ser garantido a capacidade de olhar em qualquer arquivo. Mas agora suponhamos que uma requisição venha de um usuário em particular (digamos, *mitchell*) para acessar algum arquivo. O processo servidor poderá examinar cuidadosamente as permissões associadas aos arquivos em questão e tentar decidir se ao *mitchell* deve ser permitido acessar aqueles arquivos. Mas isso pode significar

duplicar todo o processo que o *kernel* pode normalmente fazer para verificar permissões de acesso a arquivo. A implementação dessa lógica pode ser complexa, propensa a erros, e tediosa.

Uma abordagem melhor é simplesmente ceder temporariamente o ID efetivo de usuário do processo de *root* para *mitchell* e então tentar executar as operações requeridas. Se *mitchell* não tem permissão para acessar os dados, o *kernel* irá evitar que o processo faça isso e irá retornar a indicação apropriada de erro. Após todas as operações realizadas em favor de *mitchell* estarem completas, o processo pode devolver o ID efetivo de usuário para o *root*.

Programas que autenticam usuários quando eles estão usando o computador aproveitam da capacidade de mudar IDs de usuário também. Esses programas de *login* executam como *root*. Quando o usuário informa um nome de usuário e uma senha, o programa de *login* verifica o nome de usuário e a senha na base de dados de senha do sistema. Então o programa de *login* muda ambos o ID efetivo de usuário e o ID real de usuário para serem aquele do usuário. Finalmente, o programa de *login* chama a função *exec* para iniciar o *shell* do usuário, deixando o usuário executando um *shell* cujo ID efetivo de usuário e ID real de usuário são arqueles do usuário.

A função usada para mudar os IDs de usuário para um processo é a *setreuid*. (Existe, certamente, uma função correspondente *setregid* também.) Essa função recebe dois argumentos. O primeiro argumento é o ID real de usuário desejado; o segundo é o ID efetivo de usuário desejado. Por exemplo, segue adiante como você pode trocar os IDs real e efetivo de usuário:

```
setreuid (geteuid(), getuid ());
```

Óbviamente, o *kernel* não irá apenas permitir que quaisquer processos mudem seus IDs de usuário. Se a um processo for permitido mudar seu ID efetivo de usuário na hora que ele quiser e bem entender, então qualquer usuário pode facilmente fingir ser qualquer outro usuário, simplesmente mudando o ID efetivo de usuário de um de seus processos. O *kernel* irá permitir que um processo executando com um ID efetivo de usuário de valor 0 mude seus IDs de usuário como lhe aprouver. (Novamente, note quanto poder um processo executando como *root* tem! Um processo cujo ID efetivo de usuário é 0 pode fazer absolutamente qualquer coisa que lhe agradar.) Um processo comum, todavia, pode fazer somente uma das seguintes coisas[12]:

- Ajustar seu ID efetivo de usuário para ser o mesmo que seu ID real de usuário

[12]Nota do tradutor: considerando o ID real/efetivo de usuário/grupo.

- Ajustar seu ID real de usuário para ser o mesmo que seu ID efetivo de usuário
- Trocar os dois IDs de usuário

A primeira alternativa pode ser usada pelo nosso processo de contabilidade quando ele tiver terminado de acessar arquivos como *mitchell* e desejar voltar a ser *root*. A segunda alternativa pode ser usada por um programa de *login* após esse mesmo programa ter ajustado o ID efetivo de usuário para aquele do usuário se conectou agora. Ajustando o ID real de usuário garante que o usuário nunca irá estar apto a voltar a ser *root*. Trocar os dois IDs de usuário é quase um artefato histórico; programas modernos raramente usam essa funcionalidade.

Você pode informar "-1" em qualquer argumento a *setreuid* se você deseja permitir o ID de usuário somente. Existe também uma função de conveniência chamada *seteuid*. Essa função ajusta o ID efetivo de usuário, mas *seteuid* não modifica o ID real de usuário. As seguintes duas declarações fazem exatamente a mesma coisa:

```
seteuid (id);
setreuid (-1, id);
```

10.4.1 Programas *Setuid*

Usando as técnicas anteriores, você sabe como fazer um processo com privilégio de superusuário tornar-se outro usuário temporariamente e então voltar a ser *root*. Você também sabe como fazer um processo *root* levar consigo todos os seus privilégios especiais ajustando ambos seu ID real de usuário e seu ID efetivo de usuário.

Aqui está uma charada: Pode você, executando um programa como usuário comum, tornar-se *root*? Isso não parece possível, usando as técnicas anteriores, mas segue uma prova que isso pode ser feito:

```
% whoami
mitchell
% su
Password: ...
% whoami
root
```

O comando *whoami* funciona como o comando *id*, exceto que mostra somente the ID efetivo de usuário, não todas as outras informações. O comando *su* habilita você a tornar-se o superusuário se você souber a senha de *root*.

Como trabalha o *su*? Pelo fato de sabermos que o *shell* estava originalmente executando com ambos seu ID real de usuário e seu ID efetivo de usuário ajustado para *mitchell*, *setreuid* não iria nos permitir mudar qualquer ID de usuário.

A complicação é que o programa *su* é um programa *setuid*. Isso significa que quando o programa executa, o ID efetivo de usuário do processo irá ser aquele do dono do arquivo em lugar daquele do ID efetivo de usuário do processo que executou a chamada *exec*. (O ID real de usuário irá ainda ser aquele do usuário que chamou o programa.) Para criar um programa *setuid*, você usa o comando *chmod +s* na linha de comando, ou use o sinalizador S_ISUID se chamar a função *chmod* programaticamente [13].

Por exemplo, considere o programa na Listagem 10.3.

Listagem 10.3: (setuid-test.c) Programa de Demonstração do *Setuid*

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main ()
5 {
6   printf ("uid=%d euid=%d\n", (int) getuid (), (int) geteuid ());
7   return 0;
8 }
```

Agora suponhamos que esse programa é *setuid* e de propriedade do *root*. Nesse caso, a saída do *ls* irá se parecer com isso:

```
-rwsrws--x     1 root      root        11931 Jan 24 18:25 setuid-test
```

Os *bits s* indicam que o arquivo não é somente executável (como um *bit x* pode indicar) mas também *setuid* e *setgid*. Quando usamos esse programa, recebemos uma saída como segue:

```
% whoami
mitchell
% ./setuid-test
uid=501 euid=0
```

Note que o ID efetivo de usuário é ajustado para 0 quando o programa está executando.

Você pode usar o comando *chmod* com os argumentos *u+s* ou *g+s* para ajustar os *bits setuid* e *setgid* sobre um arquivo executável, rspectivamente por exemplo:

[13] Certamente, existe uma noção similar de um programa *setgid*. Quando executa, seu ID efetivo de grupo é o mesmo daquele do grupo proprietário do arquivo. A maioria dos programas *setuid* são também programas *setgid*.

```
% ls -l program
-rwxr-xr-x    1 samuel       csl              0 Jan 30 23:38 program
% chmod g+s program
% ls -l program
-rwxr-sr-x    1 samuel       csl              0 Jan 30 23:38 program
% chmod u+s program
% ls -l program
-rwsr-sr-x    1 samuel       csl              0 Jan 30 23:38 program
```

Você também pode também usar a chamada de sistema *chmod* com os sinalizadores de modo S_ISUID ou S_ISGID.

O comando *su* tem capacidade de mudar o ID efetivo de usuário através desse mecanismo. O comando *su* executa inicialmente com um ID efetivo de usuário de 0. Então o comando *su* pergunta a você por uma senha. Se a senha coincidir com a senha de *root*, o comando *su* ajusta seu ID real de usuário para ser *root* também e então inicia um novo *shell*. De outra forma, o comando *su* sai, sem a menor cerimônia deixando você como um usuário comum.

Dê uma olhada nas permissões do programa *su*:

```
% ls -l /bin/su
-rwsr-xr-x    1 root         root         14188 Mar   7   2000 /bin/su
```

Note que o programa *su* é de propriedade do *root* e que o *bit setuid* está ativo.

Note que *su* não muda agora o ID de usuário do *shell* a partir do qual ele foi executado. Ao invés disso, o comando *su* inicia um novo processo de *shell* com o novo ID de usuário. O *shell* original é bloqueado até que o novo *shell* realize todas as suas tarefas e o *su* termine [14].

10.5 Autenticando Usuários

Muitas vezes, se você tem um programa com *setuid* ativado, você não deseja disponibilizá-lo para todo mundo. Por exemplo, o programa *su* permite a você tornar-se *root* somente se você conhece a senha de *root*. O programa faz com que você prove que você está autorizado a tornar-se *root* antes de ir adiante com suas ações. Esse processo é chamado *autenticação* – o programa *su* está verificando para comprovar que você é autêntico.

Se você está administrando um sistema muito seguro, você provavelmente não deseja permitir às pessoas conectar-se apenas digitando uma senha qualquer. Usuários tendem a escrever senhas inseguras, e *chapéus pretos* [15] ten-

[14] Nota do tradutor: veja a relação de programas com setuid no debian 6.0.2 no Apêndice J "Seguraça" Seção J.1 "Setuid no Debian 6.0.2".

[15] Nota do tradutor: os *hackers* se dividem em dois grupos - os *black hats* e os *white hats*. Em http://en.wikipedia.org/wiki/Hacker_%28computer_security%29 temos que os *black*

dem a descobrí-las. Usuários tendem a usar senhas que envolvem seus aniversários, os nomes de seus animaizinhos de estimação, e assim por diante[16]. Senhas simples não são nada seguras.

Por exemplo, muitas organizações agora requerem o uso de senha especial de "uma única vez"que são geradas por cartões de identificação eletrônicos especiais que usuários mantêm consigo. A mesma senha não pode ser usada duas vezes, e você não pode pegar uma senha válida sem o cartão de identificação eletrônico sem inserir uma PIN[17]. De forma que, um atacante deve obter ambos o cartão físico e a PIN para romper a segurança. Em um ambiente realmente seguro, digitalização de retina ou outros tipos de testes biométricos são usados.

Se você está escrevendo um programa que obrigatoriamente executa autenticação, você deve permitir ao administrador do sistema usar qualquer meio de autenticação que for apropriado para aquela instalação. GNU/Linux vem com uma biblioteca muito útil que faz isso muito facilmente. Essa facilidade, chamada *Pluggable Authentication Modules*, ou PAM, torna muito fácil escrever aplicações que autenticam seus usuários da mesma forma que o administrador de sistema deseja.

É muito fácil ver como PAM trabalha olhando para uma aplicação PAM simples. A Listagem 10.4 ilustra o uso do PAM.

---

hats seriam os criminosos e os *white hats* seriam os experts em computadores. Existem muitas controvérsias sobre o significado do termo *hacker*. Não pesquisei a ligação com o nome da distribuição *Red Hat*.

[16]Verificou-se que administradores de sistema tendem a selecionar a palavra *deus* como sua senha muitas vezes mais qualquer outra palavra. (Você irá fazer isso também.) De forma que, se você mesmo precisar de acesso *root* a uma máquina e o administrador do sistema não está por perto, um pouco de inspiração divina pode ser apenas o que você precisa.

[17]Nota do tradutor: *Personal Identification Number.*. Veja `https://www.pcisecuritystandards.org/security_standards/glossary.php`.

Listagem 10.4: (pam.c) Exemplo de Uso do PAM

```
1 #include <security/pam_appl.h>
2 #include <security/pam_misc.h>
3 #include <stdio.h>
4
5 int main ()
6 {
7   pam_handle_t* pamh;
8   struct pam_conv pamc;
9
10  /* Ajusta a conversa o com o PAM. */
11  pamc.conv = &misc_conv;
12  pamc.appdata_ptr = NULL;
13  /* Inicia uma nova se o de autentica o. */
14  pam_start ("su", getenv ("USER"), &pamc, &pamh);
15  /* Autentica o usu rio. */
16  if (pam_authenticate (pamh, 0) != PAM_SUCCESS)
17    fprintf (stderr, "A autentica o falhou!\n");
18  else
19    fprintf (stderr, "Autentica o OK.\n");
20  /* Tudo feito. */
21  pam_end (pamh, 0);
22  return 0;
23 }
```

Para compilar esse programa, voce tem que linká-lo com duas bibliotecas: a biblioteca *libpam* e uma biblioteca auxiliar chamada *libpam_misc*:

```
% gcc -o pam pam.c -lpam -lpam_misc
```

Esse programa inicia-se pela construção de um objeto de conversação PAM. Esse objeto é usado pela biblioteca PAM caso seja necessário perguntar alguma informação ao usuário. A função *misc_conv* usada nesse exemplo é uma função de conversação padrão que usa o terminal para entrada e saída. Você poderá escrever sua própria função que faz surgir na tela uma caixa de diálogo, ou que usa diálogo para entrada e saída, ou que fornece ainda mais exóticos métodos de entrada e saída.

O programa então chama *pam_start*. Essa função inicializa a biblioteca PAM. O primeiro argumento é um nome de serviço. Você deve usar um nome que unicamente identifique sua aplicação. Por exemplo, se sua aplicação se chama *whizbang*, você provavelmente deve usar *whizbang* para o nome do serviço, também. Todavia, o programa provavelmente não irá trabalhar até que o administrador de sistema explicitamente configure o sistema para trabalhar com seu serviço. De forma que, nesse exemplo, usamos o serviço *su*, o qual diz que nosso programa deve autenticar usuários da mesma forma que o comando *su* faz. Você não deve usar essa técnica em um programa real. Selecionar um nome de serviço real, e ter seus *scripts* de instalação ajudam ao administrador do sistema a ajustar uma configuração correta para sua aplicação.

O segundo argumento é o nome do usuário que você deseja autenticar. Nesse exemplo, usamos o valor da variável de ambiente USER. (Normalmente, o valor dessa variável é o nome de usuário que corresponde ao ID

efetivo de usuário do processo atual, mas isso não é sempre o que ocorre.) Na maioria dos programas reais, você deve perguntar por um nome de usuário nesse momento. O terceiro argumento indica a conversação PAM, discutida anteriormente. A chamada a *pam_start* preenche o controlador fornecido como o quarto argumento. Informe esse controlador às chamadas subsequêntes a rotinas da biblioteca PAM.

A seguir, o programa chama *pam_authenticate*. O segundo argumento habilita você a informar vários sinalizadores; o valor 0 significa usar as opções padronizadas. O valor de retorno dessa função indica se a autenticação foi efetuada com sucesso.

Finalmente, o programa chama *pam_end* para limpar qualquer estruturas de dados alocadas.

Vamos assumir que uma senha válida para o usuário atual seja "senha" (uma senha excepcionalmente pobre). Então, executando esse programa com a senha correta produz o esperado:

```
% ./pam
Password: senha

Authentication OK.
```

Se você executar esse programa em um terminal, a senha provavelmente não irá aparecer na hora em que está sendo digitada; a senha é escondida para evitar que leiam sua senha por cima do seu ombro quando você a tiver digitando.

Todavia, se um *hacker* tenta usar a senha errada, a biblioteca PAM irá corretamente indicar falha:

```
% ./pam
Password: palpiteerrado

Authentication failed!
```

O básico abrangido aqui é suficiente para a maioria dos programas simples. Documentação completa sobre como PAM trabalha está disponível em */usr/doc/pam* na maioria dos sistemas GNU/Linux.

10.6 Mais Falhas de Segurança

Embora esse capítulo venha a apontar umas poucas falhas de segurança comuns, você não deve de modo algum confiar que esse livro abrange todas

as possíveis falhas de segurança. Uma grande quantidade de falhas de segurança já foram descobertas, e muitas outras estão por aí afora esperando para serem descobertas. Se você está tentando escrever código seguro, não existe seguramente substituto para se ter um especialista em segurança auditando seu código.

10.6.1 Sobrecarga no Espaço Temporário de Armazenagem

A maioria entre todos os principais programas da *Internet* que rodam em segundo plano, incluindo o *sendmail*, o *finger*, o *talk*, e outros, possuem algum ponto que foi comprometido através de um *buffer overrun*[18].

Se você está escrevendo qualquer código que irá alguma vez ser executado como *root*, você absolutamente deve estar atento para esse tipo particular de falha de segurança. Se você está escrevendo um programa que executa qualquer tipo de comunicação entre processos, você deve definitivamente estar atento para esse tipo de falha de segurança. Se você está escrevendo um programa que lê arquivos (ou possivelmente pode vir a ler arquivos) que não são de propriedade do usuário que está executando o programa, você deve estar atento a esse tipo de falha de segurança. Esse último critério aplica-se à maioria de todos os programas. Fundamentalmente, se você pretendo escrever programas para GNU/Linux, você deveria conhecer a falha provocada pela sobrecarga no espaço de armazenagem.

A idéia por trás de um ataque de sobrecarga no espaço temporário de armazenagem é induzir um programa a executar algum código que ele não tem a intenção de executar. O mecanismo usual para ter êxito nessa proeza é sobrescrever alguma porção da pilha do processo do programa. A pilha do processo do programa contém, entre outras coisas, a localização da memória na qual o programa irá transferir o controle quando a função atual retornar. Todavia, se você puder colocar o código que você deseja que seja executado dentro da memória em algum lugar e então mudar o endereço de retorno apontando para aquela peça de memória, você pode fazer com que o programa que está rodando execute qualquer coisa que você quiser. Quando o programa retornar da função que está executando, ele irá pular para o novo código e executar qualquer coisa que estiver lá, executando com os privilégios do processo atual. Claramente, se o processo atual estiver executando como *root*, isso pode ser um desastre. Se o processo estiver rodando como outro usuário, isso será um desastre "somente" para aquele usuário e alguém mais que depende do conteúdo dos arquivos de propriedade daquele usuário, e

[18]Nota do tradutor: Sobrecarga no Espaço Temporário de Armazenagem.

assim por diante.

Se o programa está executando em segundo plano e esperando por conecções de rede externas, a situação é pior. Um programa em segundo plano tipicamente executa como *root*. Se esse programa que está rodando em segundo plano como *root* contiver falhas de sobrecarga do espaço de armazenagem, alguém que pode se conectar pela rede a um computador executando o programa em segundo plano tem a possibilidade de tomar o controle do computador enviando uma sequência maligna de dados para programa em segundo plano pela rede. Um programa que não se envolve em comunicações em rede é mais seguro pelo fato de somente usuários que já estão habilitados a se conectar ao computador executando o programa estarem aptos a atacá-lo.

As versões com falha do *finger*, do *talk*, e do *sendmail* todas compartilham uma falha comum. Cada um deles usa um espaço de armazenagem de sequências de caractere de comprimento fixo, o que implica um constante limite superior sobre o tamanho da sequência de caracteres mas então permitido a clientes de rede fornecer sequências de caracteres que sobrecarrem o espaço temporário de armazenamento. Por exemplo, eles possuem código similar ao que segue:

```
#include <stdio.h>

int main ()
{
  /* Nobody in their right mind would have more than 32 characters in
     their username. Plus, I think UNIX allows only 8-character
     usernames. So, this should be plenty of space. */
  char username[32];
  /* Prompt the user for the username. */
  printf ("Enter your username: ");
  /* Read a line of input. */
  gets (username);
  /* Do other things here... */

    return 0;
}
```

A combinação do espaço temporário de armazenamento de 32 caracteres com a função *gets* permite uma sobrecarga no espaço temporário de armazenamento. A função *gets* lê entradas de usuário até encontrar um caractere de nova linha e armazena o resultado inteiro no espaço temporário de armazenamento chamado *username*. Os comentários nesse código estão corretos em que pessoas geralmente possuem nomes de usuário curtos, então nenhum usuário bem intencionado tem probabilidade de digitar mais que 32 caracteres. Mas quando você estiver escrevendo programas seguros, você deve considerar o que um atacante malicioso pode fazer. Nesse caso, o atacante pode deliberadamente digitar um nome de usuário muito longo. Variáveis locais tais como *username* estão armazenadas na pilha, de forma que extrapolando as fronteiras do array, é possível colocar *bytes* arbitrários dentro da pilha além da área reservada para a variável *username*. A *username* irá

sobrecarregar o espaço temporário de armazenamento e sobrescrever partes da pilha vizinha, permitindo o tipo de ataque descrito previamente.

Afortunadamente, é relativamente fácil evitar sobrecargas do espaço temporário de armazenagem. Ao ler sequências de caracteres, você deve sempre usar uma função, tal como a *getline*, que ou aloca dinamicamente um espaço temporário de armazenamento suficiente grande ou para a leitura da entrada se o espaço temporário estiver cheio. Por exemplo, você pode vir a usar isso[19]:

```
char *username = NULL;
size_t username_length = 0;
ssize_t characters_read = getline (&username, &username_length, stdin);
```

Essa chamada automaticamente usa *malloc* para alocar um espaço temporário de armazenagem suficientemente grande para manter a linha e retorná-la para você. Você deve se lembrar de chamar *free* para desalocar o espaço temporário de armazenagem, com certeza, para evitar desperdício de memória.

Sua vida irá ser ainda mais fácil se você usar C++ ou outra linguagem que forneça primitivas simples para leitura de entradas. Em C++, por exemplo, você pode simplesmente usar isso:

```
string username;
getline (cin, username);
```

A sequência de caracteres contida na variável *username* irá ser automaticamente desalocada também; você não tem que lembrar-se de liberar a memória usada para armazenar a variável pois isso é feito automaticamente em C++ [20].

Certamente, sobrecargas do espaço temporário de armazenagem podem ocorrer com qualquer array de tamanho definido estaticamente, não apenas com sequências de caractere. Se você deseja escrever código seguro, você nunca deve escrever dentro de uma estrutura de dados, dentro da pilha ou em algum outro lugar, sem verificar se você não está indo escrever além de sua região de memória.

[19] Se a chamada falhar, *characters_read.* irá ser -1. De outra forma, *username* irá apontar para um espaço temporário de armazenamento alocado com *malloc* medindo *username_length* caracteres.

[20] Alguns programadores acreditam que C++ é uma linguagem horrível e excessivamente complexa. Seus argumentos sobre multiplas heranças e outras tais complicações possuem alguns méritos, mas em C++ é mais fácil escrever código que evita sobrecargas do espaço temporário de armazenagem e outros problemas similares em C++ que em C.

10.6.2 Condiçoes de Corrida no */tmp*

Outro problema muito comum envolve a criação de arquivos com nomes previsíveis, tipicamente no diretório */tmp*. Suponhamos que seu programa *prog*, executando como *root*, sempre cria um arquivo temporário chamado */tmp/prog* e escreve alguma informação vital nele. Um usuário malicioso pode criar um link simbólico de */tmp/prog* para qualquer outro arquivo no sistema. Quando seu programa vai criar o arquivo, a chamada de sistema *open* irá ter sucesso. Todavia, os dados que você escreve não irão para */tmp/prog*; ao invés disso, esses dados escritos irão ser escritos em algum arquivo arbitrário da escolha do atacante.

Esse tipo de ataque é chamado explorar uma *condição de corrida*. Existe implicitamente uma corrida entre você e o atacante. Quer quer que consiga criar o arquivo primeiro vence.

Esse ataque é muitas vezes usado para destruir partes importantes do sistema de arquivos. Através da criação de *links* apropriados, o atacante pode enganar um programa executando como *root* que supostamente escreve um arquivo temporário de forma que esse programa escreva sobre um importante arquivo do sistema. Por exemplo, fazendo um *link* simbólico para */etc/passwd*, o atacante pode varrer a base de dados de senhas do sistema. Existe também formas através das quais um usuário malicioso pode obter acesso de *root* usando essa técnica.

Uma tentativa de evitar esses ataques é usar um nome aleatorizado para o arquivo. Por exemplo, você pode ler em */dev/random* alguns *bits* para usar no nome do arquivo temporário. Isso certamente torna as coisas mais difíceis para um usuário malicioso deduzir o nome do arquivo temporário, mas não torna o ataque impossível. O atacante pode apenas criar um grande número de *links* simbólicos, usando muitos nomes em potencial. Mesmo se ele tiver que tentar 10.000 vezes antes de vencer a condição de corrida, mesmo que uma única vez pode ser desastroso.

Uma outra abordagem é usar o sinalizador O_EXCL ao chamar a *open*. Esse sinalizador faz com que a *open* falhe se o arquivo já existir. Desafortunadamente, se você está usando o *Network File System* (NFS), ou se alguém que está usando seu programa puder possivelmente vir a usar o NFS, a abordagem de sinalizador não é robusta o suficiente pelo fato de O_EXCL não ser confiável quando NFS estiver em uso. Você não pode mesmo realmente saber com certeza se seu código irá ser usado sobre um sistema que use NFS, de forma que se você for altamente paranóico, não deve de modo algum usar O_EXCL.

No Capítulo 2, "Escrevendo Bom Software GNU/Linux" Seção 2.1.7, "Usando Arquivos Temporários" mostramos como usar *mkstemp* para criar

arquivos temporários. Desafortunadamente, o que *mkstemp* faz em GNU/Linux é abrir o arquivo com O_EXCL após tentar selecionar um nome que é difícil de prever. Em outras palavras, o uso do *mkstemp* é ainda inseguro se o */tmp* for montado sobre o NFS[21]. De forma que, o uso do *mkstemp* é melhor que nada, mas não é completamente seguro.

Uma abordagem que funciona é chamar *lstat* sobre o arquivo recentemente criado (*lstat* discutida na Seção B.2, "*stat*"). A função *lstat* é como a função *stat*, exceto que se o arquivo referenciado for um *link* simbólico, *lstat* informa a você sobre o *link*, não sobre o arquivo para o qual ele aponta. Se *lstat* informa a você que seu novo arquivo é um arquivo comum, não um link simbólico, e que esse novo arquivo é de sua propriedade, então deve estar tudo bem.

A Listagem 10.5 mostra uma função tenta seguramente abrir um arquivo no */tmp*. Os autores desse livro não tiveram o código fonte da função adiante auditado profissionalmente, nem são profissionais de segurança, de forma que existe uma boa chance que esse código tenha uma fraqueza, também. Não recomendamos que você use esse código sem que ele seja examinado em alguma auditoria, mas esse código deve ao menos convencer você de que escrever um código seguro é complicado. Para ajudar você a mudar de opinião, fizemos deliberadamente a interface difícil para usar em programas reais. A verificação de erro é uma parte importante da escrita de programas seguros, de forma que incluímos lógica de verificação de erros nesse exemplo.

[21] Obviamente, se você for também o administrador de sistema, você não deve montar o */tmp* sobre o NFS.

Listagem 10.5: (temp-file.c) Cria um Arquivo Temporário

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/stat.h>
5 #include <unistd.h>
6
7 /* Retorna o descritor de arquivo de um recentemente criado arquivo tempor rio.
8    O arquivo tempor rio ir  ser leg vel e escrit vel pelo ID de usu rio
9    efetivo do processo corrente mas n o ir  ser leg vel ou escrit vel
10   por ningu m mais.
11
12   retorna -1 se o arquivo tempor rio n o puder vir a ser criado.  */
13
14 int secure_temp_file ()
15 {
16   /* Esse descritor de arquivo aponta para /dev/random e permite nos pegar uma
17      boa fonte de bits aleat rios.  */
18   static int random_fd = -1;
19   /* Um inteiro aleat rio.  */
20   unsigned int random;
21   /* Um espa o tempor rio de armazenamento, usao para converter de um tipo de dado
         num rico para uma representa  o em forma de
22      sequ ncia de caracteres da aleat riedade.  Esse espa o tempor rio de
            armazenamento tem tamano fixo, significando
23      que teremos potencialmente um erro de sobrecarga do espa o tempor rio de
            armazenamento se os inteiros nessa
24      m quina tiverem *grande quantidade* de bits.  */
25   char filename[128];
26   /* O descritor de arquivo para novo arquivo tempor rio.  */
27   int fd;
28   /* Informa  o sobre o o arquivo criado recentemente.  */
29   struct stat stat_buf;
30
31   /* Se n s n o tivermos aberto /dev/random, abrimos agora.  (Isso n o
32      seguro para ser usado em linhas de execu  o.)  */
33   if (random_fd == -1) {
34     /* Abre /dev/random.  Note que estamos assumindo que /dev/random
35        realmente    uma fonte de bits aleat rios, n o um arquivo preenchido com
              zeros
36        colocados l  por um atacante.  */
37     random_fd = open ("/dev/random", O_RDONLY);
38     /* Se n o pudermos abrir /dev/random, desista.  */
39     if (random_fd == -1)
40       return -1;
41   }
42
43   /* Leia um inteiro a partir de /dev/random.  */
44   if (read (random_fd, &random, sizeof (random)) !=
45       sizeof (random))
46     return -1;
47   /* Cria um nome de arquivo usando o n mero aleat rio.  */
48   sprintf (filename, "/tmp/%u", random);
49   /* Try to open the file.  */
50   fd = open (filename,
51              /* Use O_EXECL, mesmo que O_EXECL n  trabalhe sobre NFS.  */
52              O_RDWR | O_CREAT | O_EXCL,
53              /* Garanta que ningu m mais possa ler ou escrever no arquivo.  */
54              S_IRUSR | S_IWUSR);
55   if (fd == -1)
56     return -1;
57
58   /* Chama lstat sobre o arquivo, para garantir que o arquivo n o seja
59      um link simb lico.  */
60   if (lstat (filename, &stat_buf) == -1)
61     return -1;
62   /* Se o arquivo n o for um arquivo regular, algu m tentou enganar-
63      nos.  */
64   if (!S_ISREG (stat_buf.st_mode))
65     return -1;
66   /* Se n s n o possuimos o arquivo, algu m mais pode remover o arquivo, lar o
         arquivo,
67      ou mudar o arquivo enquanto olhamos para ele.  */
68   if (stat_buf.st_uid != geteuid () || stat_buf.st_gid != getegid ())
69     return -1;
70   /* Se ouverem mais bits de permiss o ajustados sobre o arquivo,
71      suspeite de alguma coisa.  */
72   if ((stat_buf.st_mode & ~(S_IRUSR | S_IWUSR)) != 0)
73     return -1;
74
75   return fd;
76 }
```

Essa função chama *open* para criar o arquivo e então chama *lstat* umas poucas linhas depois para garantir que o arquivo não é um link simbólico. Se você está pensando cuidadosamente, você irá perceber que existe o que parece ser uma condição de corrida nesse ponto. Em particular, um atacante pode remover o arquivo e substituí-lo com um *link* simbólico no intervalo de tempo entre a chamada a *open* e a chamada a *lstat*. Isso não irá nos causar dano diretamente pelo fato de já termos um descritor de arquivo aberto para o arquivo criado recentemente, mas isso irá nos causar indicar um erro ao nosso chamador. Esse ataque não traria nenhum prejuízo direto, mas o aproveitamento dessa condição de corrida tornará impossível para o nosso programa ver seu trabalho realizado. Tal ataque é chamado ataque de negação de serviço – *denial-of-service (DoS)*.

Afortunadamente, o *sticky bit* vem para o salvamento. Pelo fato de o *sticky bit* estar ativado no */tmp*, ninguém mais pode remover arquivos daquele diretório. Certamente, *root* pode ainda remover arquivos do */tmp*, mas se o atacante tiver privilégios de *root*, não existe nada que você possa fazer para proteger seu programa.

Se você escolhe assumir uma administração de sistema competente, então o */tmp* não irá ser montado via NFS. E se o administrador do sistema for tolo o suficiente para montar o */tmp* sob NFS, então existe uma boa chance que o *sticky bit* não esteja ajustado. Então, para a maioria dos propósitos práticos, pensamos que é seguro usar *mkstemp*. Mas você deve ser informado desses recursos, e você não deve definitivamente confiar em O_EXCL trabalhar corretamente se o diretório em uso não seja o */tmp* nem você deve confiar que o *sticky bit* esteja ativado em algum outro lugar.

10.6.3 Usando *system* ou *popen*

A terceira mais comum falha de segurança que todo programador deve ter em mente envolve a utilização do *shell* para executar outros programas. Como um exemplo de brinquedo, vamos considerar um servidor dicionário. Esse programa é projetado para aceitar conecções via *Internet*. Cada cliente envia uma palavra, e o servidor diz a cada cliente se a referida palavra é uma palavra válida do Inglês. Pelo fato de todo sistema GNU/Linux vir com um lista de mais de 45.000 palavras do Inglês em */usr/dict/words*, uma forma fácil de construir esse servidor é chamar o programa *grep*, como segue:

```
% grep -x word /usr/dict/words
```

Aqui, *word* é a palavras que o usuário tem curiosidade de conhecer. O código de saída do *grep* irá dizer a você se aquela *word* aparece em

/usr/dict/words[22].

A Listagem 10.6 mostra como você pode tentar codificar a parte do servidor que chama o *grep*:

Listagem 10.6: (grep-dictionary.c) Busca por uma Palavra no Dicionário

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 /* Retorna um valor n o nulo se e somente se a WORD aparece em
5    /usr/dict/words.  */
6
7 int grep_for_word (const char* word)
8 {
9   size_t length;
10  char* buffer;
11  int exit_code;
12
13  /* Constr i a sequ ncia de caracteres 'grep -x WORD /usr/dict/words'.  Aloca a
14     sequ ncia de caracteres dinamicamente para evitar sobrecarga no espa o
          tempor rio de armazenamento.  */
15  length =
16    strlen ("grep -x ") + strlen (word) + strlen (" /usr/dict/words") + 1;
17  buffer = (char*) malloc (length);
18  sprintf (buffer, "grep -x %s /usr/dict/words", word);
19
20  /* executa o comando.  */
21  exit_code = system (buffer);
22  /* Libera o espa o tempor rio de armazenamento.  */
23  free (buffer);
24  /* Se grep retornar zero, ent o a palavra estava presente no
25     dicion rio.  */
26  return exit_code == 0;
27 }
```

Note que para realizar o cálculo do número de caracteres precisamos e então fazemos a alocação dinâmica da área temporária de armazenagem, garatindo o não aparecimento de sobrecargas do espaço temporário de armazenagem.

Desafortunadamente, o uso da função *system* (descrita no Capítulo 3, "Processos" Seção 3.2.1, "Usando *system*") é insegura. Essa função chama o *shell* padrão do sistema para executar o comando e então retorna o valor de saída. Mas o que ocorre se um *hacker* malicioso envia uma "word" que está atualmente seguida pela seguinte linha ou uma sequência de caracteres similar?

```
foo /dev/null; rm -rf /
```

Nesse caso, o servidor irá executar o comando adiante:

```
grep -x foo /dev/null; rm -rf / /usr/dict/words
```

[22]Se você não sabe nada sobre *grep*, você deve olhar nas páginas de manual. O *grep* é um programa incrivelmente útil.

Agora o problema é óbvio. O usuário transformou um comando, ostensivamente a chamada a *grep*, em dois comandos pelo fato de o *shell* tratar um ponto e vírgula como um separador de comandos. O primeiro comando é ainda uma inocente invocação do *grep*, mas o segundo comando remove todos os arquivo do sistema! Mesmo se o sistema não estiver rodando como *root*, todos os arquivos que podem ser removidos pelo usuário executando o servidor irão ser removidos. O mesmo problema pode aparecer com *popen* (descrito na Seção 5.4.4, "As Funções *popen* e *pclose*"), as quais criam um *pipe* entre o processo pai e o processo filho mas ainda usam o *shell* para executar o comando.

Existem duas forma para evitar esses problemas. Uma é usar a família de funções *exec* ao invés de *system* ou de *popen*. Essa solução evita o problema pelo fato de caracteres que o *shell* trata especialmente (tais como o ponto e vírgula no comando anterior) não são tratados especialmente quando aparecerem na lista de argumentos para uma chamada a *exec*. Certamente, você desiste da comodidade de *system* e de *popen*.

A outra alternativa é validar a sequência de caracteres para garantir que é benigna. No exemplo do servidor de dicionário, você pode garantir que a palavra fornecida contenha somente caracteres alfabéticos, usando a função *isalpha*. Se a palavra fornecida não contiver qualquer outro caractere, não existe forma de enganar o *shell* de forma que ele execute um segundo comando. Não implemente a verificação olhando para os caracteres perigosos e inesperados; a verificação é sempre mais segura explicitando a verificação dos caracteres que você sabe serem seguros em lugar de tentar antecipar todos os caracteres que podem causar complicações.

Capítulo 11

Um Modelo de Aplicação GNU/Linux

ESSE CAPÍTULO É ONDE TUDO SE JUNTA. IREMOS DESCREVER e implementar um programa GNU/Linux completo que incorpora muitas das técnicas descritas ness livro. O programa fornece informação sobre o sistema no qual está instalado por meio de uma interface *Web*.

O programa é uma demonstração completa de alguns dos métodos que descrevemos para programação em GNU/Linux e ilustra em programas curtos. Esse programa é escrito mais como código "realista", diferentemente da maioria das listagens de código que mostramos nos capítulos anteriores. O código mostrado aqui pode servir como um ponto de partida para seus próprios programas GNU/Linux.

11.1 Visão Geral

O programa exemplo é parte de um sistema para monitorar um sistema GNU/Linux que está sendo executado. O programa de monitoramento inclui os recursos adiante:

- O programa incorpora um servidor *Web* mínimo. Clientes locais ou remotos acessam informação do sistema por meio de requisições de páginas *Web* ao servidor usando o protocolo HTTP.

- O programa não disponibiliza páginas HTML estáticas. Ao invés disso, as páginas são geradas em tempo real por módulos, cada um dos quais fornece uma página sumarizando um aspécto do estado do sistema.

- Modulos não são linkados estaticamente dentro do executável do servidor. Ao invés disso, eles são carregados dinâmicamente a partir de bibliotecas compartilhadas. Módulos podem ser adicionados, removidos, ou substituídos enquanto o servidor está executando.

- O servidor atende cada conecção em um processo filho. Essa forma de atendimento habilita o servidor a mante-se respondendo mesmo quando requisições individuais demorarem um momento para completarem-se, e também protege o servidor de falhas nos módulos.

- O servidor não precisa de privilégios de superusuário para executar (bem como não executa em uma porta privilegiada). Todavia, isso limita a informação de sistema que o servidor pode coletar.

Fornecemos quatro módulos amostra que demonstram como módulos podem ser escritos. Eles adicionalmente ilustram algumas das técnicas para reunir informações do sistema mostradas anteriormente nesse livro. O módulo *time* demonstra o uso da chamada de sistema *gettimeofday*. O módulo *issue* demonstra entrada e saída de baixo nível e a chamada de sistema *sendfile*. O módulo *diskfree* demonstra o uso de *fork*, *exec*, e *dup2* por meio da execução de um comando em um processo filho. O módulo *processes* demonstra o uso do sistema de arquivo */proc* e várias chamadas de sistema.

11.1.1 Ressalvas

Esse programa tem muitos dos recursos que você iria esperar de um programa de aplicação, tais como recepção de informações pela linha de comando e tratamento de erros. Ao mesmo tempo, fizemos algumas simplificações para melhorar a legibilidade e focar em tópicos específicos do GNU/Linux discutidos nesse livro. Tenha em mente essas ressalvas ao examinar o código.

- Não tentamos fornecer uma completa implementação do HTTP. Ao invés disso, implementamos apenas o suficiente para o servidor interagir com clientes *Web*. Um programa realista ou poderia fornecer uma implemantação HTTP mais completa ou poderia interagir com uma das várias excelentes implementações de servidor *Web* [1] disponíveis ao invés de fornecer serviços HTTP diretamente.

[1] O mais popular e de código aberto servidor *Web* para GNU/Linux é o servidor *Apache*, disponível em http://www.apache.org.

- Similarmente, não objetivamos alcançar compatibilidade completa com as especificações HTML (veja `http://www.w3.org/MarkUp/`). Geramos uma saída simples em HTML que pode ser manuseada pelos navegadores populares.

- O servidor não está ajustado para alta performace ou uso mínimo de recursos. Em particular, intencionalmente omitimos alguns dos códigos de configuração de rede que você poderia esperar em um servidor *Web*. Esse tópico está fora do escopo desse livro. Veja um das muito execelentes referências sobre desenvolvimento de aplicações em rede, tais como *UNIX Network Programming*,Volume 1: *Networking APIs Sockets and XTI*, de autoria de W. Richard Stevens (Prentice Hall, 1997), para mais informação.

- Não fizemos tentativas para regular os recursos (número de processos, uso de memória, e assim por diante) consumidos pelo servidor ou seus módulos. Muitas implementações de servidores *Web* multiprocessados de conecções de serviço usam um reservatório limitado de processos em lugar de criar um novo processo filho para cada conecção.

- O servidor chama a biblioteca compartilhada para um módulo do servidor cada vez que o servidor é requisitado e então imediatamente descarrega o módulo quando a requisição tiver sido completada. Uma implementação mais eficiente poderia provavelmente selecionar os módulos mais usados e mantê-los na memória.

HTTP
O *Hypertext Transport Protocol* (HTTP)[a] é usado para comunicação entre clientes *Web* e servidores *Web*. O cliente conecta-se ao servidor por meio do estabelecimento de uma conecção a uma bem conhecida porta (usualmente a porta 80 para servidor *Web* conectados à *Internet* , mas qualquer porta pode ser usada). Requisições HTTP e cabeçalhos HTTP são compostos de texto puro. Uma vez conectado, o cliente envia uma requisição ao servidor. Uma requisição típica é *GET /page HTTP/1.0*. O método GET indica que o cliente está requisitando que o servidor envie a ele cliente uma página *Web*. O segundo elemento é o caminho para a referida página no servidor. O terceiro elemento é o protocolo e a versão do protocolo. Linhas subsequêntes possuem campos de cabeçalho, formatados similarmente a cabeçalhos de *email*, os quais possuem informações extras sobre o cliente. O cabeçalho termina com uma linha em branco. O servidor envia de volta uma resposta indicando o resultado do processamento da requisição. Uma resposta típica é *HTTP/1.0 200 OK*. O primeiro elemento é a versão do protocolo. Os seguintes dois elementos indicam o resultado; nesse caso, resultado 200 indica que a requisição foi processada com sucesso. Linhas subsequêntes posuem campos de cabeçalho, formatados similarmente a cabeçalhos de *email*. O cabeçalho termina com uma linha em branco. O servidor pode então enviar dados arbitrários para satisfazer a requisição. Tipicamente, o servidor responde a uma requisição de determinada página enviando de volta o código HTML da página *Web* requisitada. Nesse caso, os cabeçalhos de resposta irão incluir *Content-type: text/html*, indicando que o resultado é código na linguagem HTML. O código HTML segue imediatamente após o cabeçalho. Veja a especificação HTTP em `http://www.w3.org/Protocols/` para mais informação.

[a]Nota do tradutor: Protocolo de Transporte de Hipertexto.

11.2 Implementação

Todos incluindo os programas menores escritos em C requerem organização cuidadosa para preservar a modularidade e manutensibilidade do código fonte. O programa apresentado nesse capítulo é dividido em quatro arquivos fonte principais.

Cada arquivo fonte exporta funções ou variáveis que podem ser acessadas por outras partes do programa. Por simplicidade, todas as funções e as variáveis exportadas são declaradas em um único arquivo de capeçalho, *server.h* (veja a Listagem 11.1), o qual está incluído em outros arquivos. Funções que são intencionalmente para uso dentro de uma única unidade de compilação somente são declaradas como sendo do tipo *static* e não são declaradas em *server.h*.

Listagem 11.1: (server.h) Declarações de Funções e de Variáveis

```
1 #ifndef SERVER_H
2 #define SERVER_H
3
4 #include <netinet/in.h>
5 #include <sys/types.h>
6
7 /*** S mbolos definidos em common.c.***********************************/
8
9 /* O nome dese programa.  */
10 extern const char* program_name;
11
12 /* Se n o zero, mostre mensgens detalhadas.  */
13 extern int verbose;
14
15 /* como em malloc, exceto que o programa aborta se a aloca  o falhar.  */
16 extern void* xmalloc (size_t size);
17
18 /* Como em realloc, exceto que o programa aborta se a aloca  o falhar.  */
19 extern void* xrealloc (void* ptr, size_t size);
20
21 /* Como em strdup, exceto que o programa aborta se a aloca  o falhar.  */
22 extern char* xstrdup (const char* s);
23
24 /* Mostre uma mensagem de erro para uma chamada com falha em OPERATION, usnado o
       valor
25    de errno, e termine o programa.  */
26 extern void system_error (const char* operation);
27
28 /* Mostre uma mensgem de erro para falhas envolvendo CAUSE, incluindo uma
29    MESSAGE descritiva, e termine o programa.  */
30 extern void error (const char* cause, const char* message);
31
32 /* Retorne o diret rio contendo o execut vel do programa rodando.
33    O valor de retorno    um espa o tempor rio de mem ria o que o chamador deve
          desalocar
34    usando free.  essa chamada de fun  o aborta em caso de falha.  */
35 extern char* get_self_executable_directory ();
36
37
38 /*** s mbolos definidos em module.c***********************************/
39
40 /* Uma inst ncia de um m dulo de servidor carregado.  */
41 struct server_module {
42   /* O manipulador de biblioteca compartilhada correspondendo ao m dulo carregado.
         */
43   void* handle;
44   /* Um nome descrevendo o m dulo.  */
45   const char* name;
46   /* A fun  o que gera os resultados HTML para esse m dulo.  */
47   void (* generate_function) (int);
48 };
49
50 /* O diret rio a partir do qual m dulos s o carregados.  */
51 extern char* module_dir;
52
53 /* Tenta carregar um m dulo de servidor com o nome MODULE_PATH.  Se um
54    m dulo de servidor existir com esse caminho, carrega o m dulo e retorna uma
55    estrutura server_module structure representando-o.  De outra forma, retorna NULL.
         */
56 extern struct server_module* module_open (const char* module_path);
57
58 /* Fecha um m dulo de servidor e desaloca o objeto de MODULE.  */
59 extern void module_close (struct server_module* module);
60
61
62 /*** S mbolos definidos em server.c.  ***********************************/
63
64 /* Executa o servidor sobre LOCAL_ADDRESS e PORT.  */
65 extern void server_run (struct in_addr local_address, uint16_t port);
66
67
68 #endif  /* SERVER_H */
```

11.2.1 Funções Comuns

O programa *common.c* (veja a Listagem 11.2) contém funções de utilidade geral que são usadas em todo o programa.

Listagem 11.2: (common.c) Funções de Utilidade Geral

```
1 #include <errno.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 #include <unistd.h>
6
7 #include "server.h"
8
9 const char* program_name;
10
11 int verbose;
12
13 void* xmalloc (size_t size)
14 {
15   void* ptr = malloc (size);
16   /* Aborte se a aloca  o tiver falhado.  */
17   if (ptr == NULL)
18     abort ();
19   else
20     return ptr;
21 }
22
23 void* xrealloc (void* ptr, size_t size)
24 {
25   ptr = realloc (ptr, size);
26   /* Aborte se a aloca  o tiver falhado.  */
27   if (ptr == NULL)
28     abort ();
29   else
30     return ptr;
31 }
32
33 char* xstrdup (const char* s)
34 {
35   char* copy = strdup (s);
36   /* Aborte se a aloca  o tiver falhado.  */
37   if (copy == NULL)
38     abort ();
39   else
40     return copy;
41 }
42
43 void system_error (const char* operation)
44 {
45   /* Gere uma mensagem de erro para errno.  */
46   error (operation, strerror (errno));
47 }
48
49 void error (const char* cause, const char* message)
50 {
51   /* Mostre uma mensagem de erro para stderr.  */
52   fprintf (stderr, "%s: erro: (%s) %s\n", program_name, cause, message);
53   /* Termine o programa.  */
54   exit (1);
55 }
56
57 char* get_self_executable_directory ()
58 {
59   int rval;
60   char link_target[1024];
61   char* last_slash;
62   size_t result_length;
63   char* result;
64
65   /* L  o alvo do link simb lico /proc/self/exe.  */
66   rval = readlink ("/proc/self/exe", link_target, sizeof (link_target) - 1);
67   if (rval == -1)
68     /* A chamado a readlink falhou, ent o reclame.  */
69     abort ();
70   else
71     /* NUL-termina  o no alvo.  */
72     link_target[rval] = '\0';
```

Listagem 11.3: (common.c) Continuação

```
73   /* Desejamos desmembrar o nome do arquivo execut vel , para obter o
74       diret rio que o cont m .  Encontra a barra mais    direita.  */
75   last_slash = strrchr (link_target , '/');
76   if (last_slash == NULL || last_slash == link_target)
77     /* Alguma coisa estranha est  acontecendo.  */
78     abort ();
79   /* Aloca um espa o tempor rio de armazenamento para receber o caminho resultante.
         */
80   result_length = last_slash - link_target;
81   result = (char*) xmalloc (result_length + 1);
82   /* Copy the result.  */
83   strncpy (result , link_target , result_length);
84   result[result_length] = '\0';
85   return result;
86 }
```

Você pode usar essas funções em outros programas também; o conteúdo desse arquivo pode ser incluído em uma biblioteca de código comum que é compartilhada entre muitos projetos:

- *xmalloc*, *xrealloc*, e *xstrdup* são versões com verificação de erro das funções da biblioteca C GNU padrão *malloc*, *realloc*, e *strdup*, respectivamente. Ao contrário das versões padronizadas que retornam um apontador nulo se a alocação falhar, essas funções imediatamente abortam o programa quando a memória necessária for insuficiente. Detecção antecipada de falhas de alocação de memória é uma boa idéia. De outra forma, uma alocação que falhou introduz apontadores nulos em lugares inesperados dentro do programa. Pelo fato de falhas de alocação não serem fáceis de reproduzir, depurar tais problemas pode ser difícil. Falhas de alocação são de modo geral catastróficas, de forma que abortar o programa é muitas vezes uma linha de ação aceitável.

- a função *error* é para reportar um erro fatal que venha a ocorrer durante a execução do programa. A função *error* imprime uma mensagem para *stderr* e termina o programa. Para erros causados por chamadas de sistema que falharam ou por chamadas a bibliotecas que falharam, *system_error* gera parte da mensagem de erro a partir do conteúdo da variável *errno* (veja a Seção 2.2.3, "Códigos de Erro de Chamadas de Sistema" no Capítulo 2, "Escrevendo Bom Software GNU/Linux").

- *get_self_executable_directory* determina o diretório contendo o arquivo executável que vai rodar no processo atual. O caminho do diretório pode ser usado para localizar outros componentes do programa, os quais são instalados no mesmo lugar em tempo de execução. Essa função trabalha examinando o link simbólico */proc/self/exe* no sistema de arquivos */proc* (veja Seção 7.2.1, "/proc/self" no Capítulo 7, "O Sistema de Arquivos */proc*").

Adicionalmente, *common.c* define duas variáveis globais úteis:

- O valor de *program_name* é o nome do programa sendo executado, como especificado em sua lista de argumentos (veja Seção 2.1.1, "A Lista de Argumentos" no Capítulo 2). Quando o programa é chamado a partir do *shell*, esse é o caminho e nome do programa como o usuário informou.

- A variável *verbose* é diferente de zero se o programa está rodando no modo *verbose*. Nesse caso, várias partes do programa mostram mensagens de progresso em *stdout*.

11.2.2 Chamando Módulos de Servidor

O programa *module.c* (veja a Listagem 11.4) fornece a implementação do servidor de módulos carregados dinamicamente. Um servidor de módulos carregados é representado por uma instância de *struct server_module*, a qual é definida em *server.h*.

Listagem 11.4: (module.c) Carregando e Descarregando Módulo de Servidor

```
1 #include <dlfcn.h>
2 #include <stdlib.h>
3 #include <stdio.h>
4 #include <string.h>
5
6 #include "server.h"
7
8 char* module_dir;
9
10 struct server_module* module_open (const char* module_name)
11 {
12   char* module_path;
13   void* handle;
14   void (* module_generate) (int);
15   struct server_module* module;
16
17   /* Constr i o caminho completo do m dulo de biblioteca compartilhada o qual
        iremos tentar
18      carregar.  */
19   module_path =
20     (char*) xmalloc (strlen (module_dir) + strlen (module_name) + 2);
21   sprintf (module_path, "%s/%s", module_dir, module_name);
22
23   /* Tenta abrir MODULE_PATH como uma bilbioteca compartilhada.  */
24   handle = dlopen (module_path, RTLD_NOW);
25   free (module_path);
26   if (handle == NULL) {
27     /* Falha; ou o caminho n o existe, ou isso n o    uma bilbioteca
28        compartilhada.  */
29     return NULL;
30   }
31
32   /* Resolve o s mbolo module_generate a partir da bilbioteca compartilhada.  */
33   module_generate = (void (*) (int)) dlsym (handle, "module_generate");
34   /* Garante que o s mbolo foi encontrado.  */
35   if (module_generate == NULL) {
36     /* O s mbolo est  desaparecido.  apesar disso ser uma biblioteca compartilhada,
37        provavelmente n o    um m dulo de servidor.  feche e retorne indica  o de
             falha.  */
38     dlclose (handle);
39     return NULL;
40   }
41
42   /* Aloque e inicialize um objeto server_module.  */
43   module = (struct server_module*) xmalloc (sizeof (struct server_module));
44   module->handle = handle;
45   module->name = xstrdup (module_name);
46   module->generate_function = module_generate;
47   /* Retorne esse objeto, indicando sucesso.  */
48   return module;
49 }
50
51 void module_close (struct server_module* module)
52 {
53   /* Feche a biblioteca compartilhada.  */
54   dlclose (module->handle);
55   /* Desaloque o nome do m dulo.  */
56   free ((char*) module->name);
57   /* Desaloque o objeto do m dulo.  */
58   free (module);
59 }
```

Cada módulo é um arquivo de biblioteca compartilhada (veja Seção 2.3.2, "Bibliotecas Compartilhadas" no Capítulo 2) e deve definir e exportar uma função chamada *module_generate*. Essa função gera uma página *Web* HTML e a escreve no descritor de arquivo do *socket* do cliente informado como seu argumento.

O programa *module.c* contém duas funções:

- *module_open* tenta carregar um módulo de servidor com um nome for-

necido. O nome geralmente terminda com a extensão "*.so*" pelo fato de módulos de servidor serem implementados como bilbiotecas compartilhadas. A função *module_open* abre a biblioteca compartilhada com *dlopen* e resolve um símbolo chamado *module_generate* da biblioteca com *dlsym* (veja Seção 2.3.6, "Carregamento e Descarregamento Dinâmico" no Capítulo 2). Se a biblioteca não puder ser aberta, ou se *module_generate* não for um nome exportado pela biblioteca, a chamada falha e *module_open* retorna um apontador nulo. De outra forma, *module_open* aloca e retorna um objeto módulo.

- *module_close* fecha a biblioteca compartilhada correspondente ao módulo de servidor e desaloca o objeto *struct server_module*.

O programa *module.c* também define uma variável global *module_dir*. Essa variável contém o caminho do diretório no qual *module_open* tenta encontrar bibliotecas compartilhadas correspondendo aos módulos de servidor.

11.2.3 O Servidor

O programa *server.c* (veja Listagem 11.5) é a implementação do servidor HTTP mínimo.

Listagem 11.5: (server.c) Implementação do Servidor

```
1 #include <arpa/inet.h>
2 #include <assert.h>
3 #include <errno.h>
4 #include <netinet/in.h>
5 #include <signal.h>
6 #include <stdio.h>
7 #include <string.h>
8 #include <sys/types.h>
9 #include <sys/socket.h>
10 #include <sys/wait.h>
11 #include <unistd.h>
12
13 #include "server.h"
14
15 /* Resposta HTTP e cabe alho para uma requisis  o feita com sucesso.  */
16
17 static char* ok_response =
18   "HTTP/1.0 200 OK\n"
19   "Content-type: text/html\n"
20   "\n";
21
22 /* resposta HTTP, cabe alho, e corpo indicando que n o
23    entendemos a requisi  o.  */
24
25 static char* bad_request_response =
26   "HTTP/1.0 400 Bad Request\n"
27   "Content-type: text/html\n"
28   "\n"
29   "<html>\n"
30   " <body>\n"
31   "   <h1>Bad Request</h1>\n"
32   "   <p>This server did not understand your request.</p>\n"
33   " </body>\n"
34   "</html>\n";
35
36 /* Resposta HTTP, cabe alho, e modelo de corpo indicando que o
37    requisitado n o foi encontrado.  */
38
39 static char* not_found_response_template =
40   "HTTP/1.0 404 Not Found\n"
41   "Content-type: text/html\n"
42   "\n"
43   "<html>\n"
44   " <body>\n"
45   "   <h1>Not Found</h1>\n"
46   "   <p>The requested URL %s was not found on this server.</p>\n"
47   " </body>\n"
48   "</html>\n";
49
50 /* Resposta HTTP, cabe alho, e modelo de corpo indicando que o
51    m todo n o foi entendido.  */
52
53 static char* bad_method_response_template =
54   "HTTP/1.0 501 Method Not Implemented\n"
55   "Content-type: text/html\n"
56   "\n"
57   "<html>\n"
58   " <body>\n"
59   "   <h1>Method Not Implemented</h1>\n"
60   "   <p>The method %s is not implemented by this server.</p>\n"
61   " </body>\n"
62   "</html>\n";
63
64 /* Manipulador para SIGCHLD, limpar processos filhos que tiverem
65    terminado.  */
66
67 static void clean_up_child_process (int signal_number)
68 {
69   int status;
70   wait (&status);
71 }
72
73 /* Processa uma requisi  o HTTP "GET" para PAGE, e envia os resultados para o
74    descritor de arquivo CONNECTION_FD.  */
75
76 static void handle_get (int connection_fd, const char* page)
77 {
78   struct server_module* module = NULL;
```

Listagem 11.6: (server.c) Continuação

```
79    /* garanta que a p gina requisitada inicia-se com uma barra e n o
80       cont m quaisquer barras adicionais -- n o suportamos quaisquer
81       subdiret rios.  */
82    if (*page == '/' && strchr (page + 1, '/') == NULL) {
83      char module_file_name[64];
84
85      /* O nome da p gina parece parece OK.  Constr i o nome do m dulo por meio do
          acrescimo de
86         ".so" no nome da p gina.  */
87      snprintf (module_file_name, sizeof (module_file_name),
88                "%s.so", page + 1);
89      /* Tenta abrir o m dulo.  */
90      module = module_open (module_file_name);
91    }
92
93    if (module == NULL) {
94      /* Ou a requisi  o da p gina est  mal formada, ou n o podemos abrir um
95         m dulo com o nome indicado.  Em qualquer caso, retorna-se a resposta
96         HTTP 404, Not Found.  */
97      char response[1024];
98
99      /* Gera a mensagem resposta.  */
100     snprintf (response, sizeof (response), not_found_response_template, page);
101     /* Envie-a a ao cliente.  */
102     write (connection_fd, response, strlen (response));
103   }
104   else {
105     /* O m dulo requisitado n o foi carregado com sucesso.  */
106
107     /* Envie a resposta HTTP indicando sucesso, e o cabe alho HTTP
108        para uma p gina HTML.  */
109     write (connection_fd, ok_response, strlen (ok_response));
110     /* Chame o m dulo, o qual ir  gerar a sa da HTML e envi -la
111        para o descritor de arquivo do cliente.  */
112     (*module->generate_function) (connection_fd);
113     /* Encerramos com o m dulo.  */
114     module_close (module);
115   }
116 }
117
118 /* Manipula uma conec  o cliente sobre o descritor de arquivos CONNECTION_FD.  */
119
120 static void handle_connection (int connection_fd)
121 {
122   char buffer[256];
123   ssize_t bytes_read;
124
125   /* L  alguns dados a partir do cliente.  */
126   bytes_read = read (connection_fd, buffer, sizeof (buffer) - 1);
127   if (bytes_read > 0) {
128     char method[sizeof (buffer)];
129     char url[sizeof (buffer)];
130     char protocol[sizeof (buffer)];
131
132     /* Alguns dados forma lidos com sucesso.  NUL-terminado o espa o tempor rio
133        de armazenamento de forma que podemos usar opera  es sobre sequ ncias de
             caractere neles contidas.  */
134     buffer[bytes_read] = '\0';
135     /* A primeira linha que o cliente envia  a requisi  o HTTP, a qual
136        composta de um m todo, a p gina requisitada, e a vers o do
137        protocolo.  */
138     sscanf (buffer, "%s %s %s", method, url, protocol);
139     /* O cliente pode enviar v rios cabe alhos de informa  o seguindo a
140        requisi  o.  Por essa implementa  o HTTP, n o cuidamos disso.
141        Todavia, precisamos ler quaisquer dados que o cliente tenta enviar.  Mantemos
142        habilitada a leitura de dados at  que recebamos o fim do cabe alho, o qual

143        delimitado por uma linha em branco.  HTTP especifica CR/LF como delimitador
144        de linha.  */
145     while (strstr (buffer, "\r\n\r\n") == NULL)
146       bytes_read = read (connection_fd, buffer, sizeof (buffer));
147     /* Garante que a  ltima leitura n o falhe.  Se isso acontecer, existe um
148        problema com a conec  o, ent o abandone.  */
149     if (bytes_read == -1) {
150       close (connection_fd);
151       return;
152     }
153     /* Verifica o campo protocolo.  Entendemos vers es HTTP 1.0 e
154        1.1.  */
155     if (strcmp (protocol, "HTTP/1.0") && strcmp (protocol, "HTTP/1.1")) {
156       /* N o entendemos esse protocolo.  Reporte resposta ruim.  */
157       write (connection_fd, bad_request_response,
```

Listagem 11.7: (server.c) Continuação

```
158                 sizeof (bad_request_response));
159       }
160       else if (strcmp (method, "GET")) {
161         /* esse servidro somente implementa o m todo GET.  Se o cliente
162            especificou algum outro m todo, ent o reporte a falha.  */
163         char response[1024];
164
165         snprintf (response, sizeof (response),
166                   bad_method_response_template, method);
167         write (connection_fd, response, strlen (response));
168       }
169       else
170         /* Uma requisi  o v lida.  Processe-a.  */
171         handle_get (connection_fd, url);
172     }
173     else if (bytes_read == 0)
174       /* O cliente fechou a conec  o antes do envio de quaisquer dados.
175          Nothing to do.  */
176       ;
177     else
178       /* A chamada a read falhou.  */
179       system_error ("read");
180 }
181
182
183 void server_run (struct in_addr local_address, uint16_t port)
184 {
185   struct sockaddr_in socket_address;
186   int rval;
187   struct sigaction sigchld_action;
188   int server_socket;
189
190   /* Instala um manipulador para SIGCHLD que limpa processos que
191      tiverem terminado.  */
192   memset (&sigchld_action, 0, sizeof (sigchld_action));
193   sigchld_action.sa_handler = &clean_up_child_process;
194   sigaction (SIGCHLD, &sigchld_action, NULL);
195
196   /* Cria um socket TCP.  */
197   server_socket = socket (PF_INET, SOCK_STREAM, 0);
198   if (server_socket == -1)
199     system_error ("socket");
200   /* Constr i uma estrutura de endere o de socket para o endere o local sobre
201      a qual queremos escutar as conec  es.  */
202   memset (&socket_address, 0, sizeof (socket_address));
203   socket_address.sin_family = AF_INET;
204   socket_address.sin_port = port;
205   socket_address.sin_addr = local_address;
206   /* Associa o socket para aquele endere o.  */
207   rval = bind (server_socket, &socket_address, sizeof (socket_address));
208   if (rval != 0)
209     system_error ("bind");
210   /*  Instrui o socket a aceitar conec  es.  */
211   rval = listen (server_socket, 10);
212   if (rval != 0)
213     system_error ("listen");
214
215   if (verbose) {
216     /* No modo detalhado, mostra o endere o local e n mero de porta
217        que estamos escutando.  */
218     socklen_t address_length;
219
220     /* encontre o endere o local do socket.  */
221     address_length = sizeof (socket_address);
222     rval = getsockname (server_socket, &socket_address, &address_length);
223     assert (rval == 0);
224     /* Mostre uma mensagem.  O n mero de porta precisa ser convertido de
225        ordem de byte de rede (big endian) para ordem de byte de host.  */
226     printf ("servidor escutando %s:%d\n",
227             inet_ntoa (socket_address.sin_addr),
228             (int) ntohs (socket_address.sin_port));
229   }
230
231   /* ciclo infinito, manipulando conec  es.  */
232   while (1) {
233     struct sockaddr_in remote_address;
234     socklen_t address_length;
235     int connection;
236     pid_t child_pid;
```

Listagem 11.8: (server.c) Continuação

```

    /* Aceita uma conec o. Essa chamada bloqueia at que uma conec o esteja
       pronta. */
    address_length = sizeof (remote_address);
    connection = accept (server_socket, &remote_address, &address_length);
    if (connection == -1) {
      /* A chamada a accept falhou. */
      if (errno == EINTR)
        /* A chamada foi interrompida por um sinal. Tente novamente. */
        continue;
      else
        /* Algo ruim aconteceu. */
        system_error ("accept");
    }

    /* Temos uma conec o. Mostre uma mensagem se estivermos rodando no
       modo detalhado. */
    if (verbose) {
      socklen_t address_length;

      /* Pegue o endere o remoto da conec o. */
      address_length = sizeof (socket_address);
      rval = getpeername (connection, &socket_address, &address_length);
      assert (rval == 0);
      /* Mostre uma mensagem. */
      printf ("conec o aceita de %s\n",
              inet_ntoa (socket_address.sin_addr));
    }

    /* Fork um processo filho para manipular a conec o. */
    child_pid = fork ();
    if (child_pid == 0) {
      /* Esse o processo filho. Ele n o pode usar stdin ou stdout,
         ent o feche stdin e stdout. */
      close (STDIN_FILENO);
      close (STDOUT_FILENO);
      /* Tamb m esse processo filho n o deve fazer nada com a
         escuta de socket. */
      close (server_socket);
      /* Manipule uma requisi o da conec o. Temos nossa pr pria c pia
         do descritor do socket conectado. */
      handle_connection (connection);
      /* Tudo terminado; feche o socket de conec o, e termine o processo
         filho. */
      close (connection);
      exit (0);
    }
    else if (child_pid > 0) {
      /* Esse o processo pai. O processo filho manipula a
         conec o, ent o n o precisamos de nossa c pia do descritor do socket
         conectado. Feche-a. Ent o continue com o ciclo e
         aceite outra conec o. */
      close (connection);
    }
    else
      /* Chamada a fork falhou. */
      system_error ("fork");
  }
}
```

Essas são as funções em *server.c*:

- *server_run* é o ponto de entrada principal para executar o servidor. Essa função inicia o servidor e começa aceitando conecções, e não retorna a menos que um erro sério ocorra. O servidor usa um *socket* servidor de fluxo TCP (veja Seção 5.5.3, "Servidores" no Capítulo 5, "Comunicação Entre Processos"). O primeiro argumento a *server_run* especifica o endereço local no qual as conecções são aceitas. Um computador GNU/Linux pode ter multiplos endereços de rede, e cada en-

dereço pode estar associado a uma interface de rede diferente[2]. Para restringir o servidor a aceitar conecções de uma interface em particular[3], especifique o correspondente endereço de rede. Especifique o endereço local INADDR_ANY para aceitar conecções de qualquer endereço local.

O segundo argumento a *server_run* é o número de porta sobre a qual aceitar conecções. Se o número de porta já estiver sendo usada por outro serviço, ou se corresponder a uma porta privilegiada e o servidor não estiver sendo executado com privilégios de superusuário, o servidor irá falhar. O valor especial 0 instrui o GNU/Linux a selecionar uma porta livre automaticamente. Veja a página de manual para *inet* para mais informação sobre endereço de domínio *Internet* e números de porta.

O servidor controla cada conecção com os clientes em um processo filho criado com *fork* (veja a Seção 3.2.2, "Usando Bifurcar e Executar" no Capítulo 3, "Processos"). O processo principal (pai) continua aceitando novas conecções enquanto as já existentes estão sendo servidas. O processo filho chama *handle_connection* e então fecha o *socket* de conecção e sai.

- *handle_connection* processa uma única conecção de cliente, usando o descritor de arquivo do *socket* informado como seu argumento. Essa função lê dados vindos do *socket* e tenta interpretá-los como uma requisição de página HTTP.

  O servidor processa somente requisições HTTP na versão 1.0 e na versão 1.1. Quando encontra um protocolo ou versão diferente, o servidor responde enviando o código de resultado HTTP 400 e a mensagem *bad_request_response*. O servidor entende somente o método GET do HTTP. Se o cliente requisita qualquer outro método, o servidor responde enviando o código de resultado HTTP 501 e a mensagem *bad_method_response_template*.

- Se o cliente envia uma bem formada requisição GET, *handle_connection* chama *handle_get* para atendê-la. Essa função tenta chamar um módulo de servidor com um nome gerado da pagina requisitada. Por exemplo, Se o cliente requisita a página chamada *information*, *handle_get* tenta chamar um módulo de servidor chamado *information.so*. Se o módulo não pode ser chamado, *handle_get* envia ao cliente o código de resul-

[2]Seu computador pode ser configurado para incluir tais interfaces como *eth0*, uma placa *Ethernet*; *lo*, a rede (*loopback*) local; ou *ppp0*, uma conecção de rede *dial-up*.

[3]Nota do tradutor: temos também as interfaces *wireless* iniciando com *wlan0*.

tado HTTP 404 e a mensgem *not_found_response_template*. Se o cliente envia uma requisição de página que corresponde a um módulo de servidor, *handle_get* envia um cabeçalho de código de resultado 200 para o cliente, o qual indica que a requisição foi processada com sucesso e chama a função de módulo *module_generate*. Essa função gera o código fonte HTML para uma página *Web* e envia esse código fonte para o cliente *Web*.

- *server_run* instala *clean_up_child_process* como o controlador de sinal para SIGCHLD. Essa função simplesmente limpa processos filhos que terminaram (veja Seção 3.3.5, "Limpando Filhos de Forma Não Sincronizada" no Capítulo 3).

11.2.4 O Programa Principal

O programa *main.c* (veja Listagem 11.9) fornece uma função *main* para o programa *server*. A responsabilidade da função *main* é tratar opções de linha de comando, detectar e reportar erros, e configurar e executar o servidor.

Listagem 11.9: (main.c) Programa Principal do Servidor e Tratamento de Linha de Comando

```
1 #include <assert.h>
2 #include <getopt.h>
3 #include <netdb.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <sys/stat.h>
8 #include <unistd.h>
9
10 #include "server.h"
11
12 /* Descri  o de op  es longas para getopt_long.  */
13
14 static const struct option long_options[] = {
15   { "address",           1, NULL, 'a' },
16   { "help",              0, NULL, 'h' },
17   { "module-dir",        1, NULL, 'm' },
18   { "port",              1, NULL, 'p' },
19   { "verbose",           0, NULL, 'v' },
20 };
21
22 /* Descri  o de op  es curtas para getopt_long.  */
23
24 static const char* const short_options = "a:hm:p:v";
25
26 /* Texto de sumariza  o de uso.  */
27
28 static const char* const usage_template =
29   "Us: %s [ op  es ]\n"
30   "  -a, --address ADDR        Bind to local address (by default, bind\n"
31   "                              to all local addresses).\n"
32   "  -h, --help                Print this information.\n"
33   "  -m, --module-dir DIR      Load modules from specified directory\n"
34   "                              (by default, use executable directory).\n"
35   "  -p, --port PORT           Bind to specified port.\n"
36   "  -v, --verbose             Print verbose messages.\n";
37
38 /* Mostre informa  es de uso e saia.  Se IS_ERROR for n o zero, escreva para
39    stderr e use um c digo de erro de sa da.  De outra forma, escreva para stdout e
40    use um c digo de termina  o de n o erro.  N o retorne.  */
41
42 static void print_usage (int is_error)
43 {
44   fprintf (is_error ? stderr : stdout, usage_template, program_name);
45   exit (is_error ? 1 : 0);
46 }
47
48 int main (int argc, char* const argv[])
49 {
50   struct in_addr local_address;
51   uint16_t port;
52   int next_option;
53
54   /* Armazene o nome do programa, o qual iremos usar em mensagens de erro.  */
55   program_name = argv[0];
56
57   /* Ajuste os padr es para as op  es.  Associe o servidor a todos os endere os
        locais,
58      e atribua uma porta livre automaticamente.  */
59   local_address.s_addr = INADDR_ANY;
60   port = 0;
61   /* N o imprima mensagens detalhadas.  */
62   verbose = 0;
63   /* Carregue os m dulos a partir de diret rio contendo esse execut vel.  */
64   module_dir = get_self_executable_directory ();
65   assert (module_dir != NULL);
66
67   /* Informe op  es.  */
68   do {
69     next_option =
70       getopt_long (argc, argv, short_options, long_options, NULL);
71     switch (next_option) {
72     case 'a':
73       /* Usu rio especificou -a ou --address.  */
74       {
75         struct hostent* local_host_name;
```

Listagem 11.10: (main.c) Continuação

```
76          /* Olhe o nome de host do usu rio especificado.  */
77          local_host_name = gethostbyname (optarg);
78          if (local_host_name == NULL || local_host_name->h_length == 0)
79            /* N o     poss vel resolver o nome.  */
80            error (optarg, "invalid host name");
81          else
82            /* Nome de host est  OK, ent o use-o.  */
83            local_address.s_addr =
84              *((int*) (local_host_name->h_addr_list[0]));
85        }
86        break;
87
88      case 'h':
89        /* Usu rio especificou -h ou --help.  */
90        print_usage (0);
91
92      case 'm':
93        /* Usu rio especificou -m ou --module-dir.  */
94        {
95          struct stat dir_info;
96
97          /* Verifique se o m dulo existes.  */
98          if (access (optarg, F_OK) != 0)
99            error (optarg, "diret rio do m dulo nao existe");
100         /* Verifica se o m dulo est  acess vel.  */
101         if (access (optarg, R_OK | X_OK) != 0)
102           error (optarg, "diret rio do m dulo n o est  acess vel");
103         /* Garanta que   um diret rio.  */
104         if (stat (optarg, &dir_info) != 0 || !S_ISDIR (dir_info.st_mode))
105           error (optarg, "n o   um diret rio");
106         /* Parece OK, ent o use-o.  */
107         module_dir = strdup (optarg);
108       }
109       break;
110
111     case 'p':
112       /* Us rio especificou -p ou --port.  */
113       {
114         long value;
115         char* end;
116
117         value = strtol (optarg, &end, 10);
118         if (*end != '\0')
119           /* O usu rio especificou n o d gitos no n mero da porta.  */
120           print_usage (1);
121         /* O n mero de porta precisa ser convertido para ordem de byte (big endian)
122            de rede.  */
123         port = (uint16_t) htons (value);
124       }
125       break;
126
127     case 'v':
128       /* Usu rio especificou -v ou --verbose.  */
129       verbose = 1;
130       break;
131
132     case '?':
133       /* Usu rio especificou uma op  o desconhecida.  */
134       print_usage (1);
135
136     case -1:
137       /* terminado com as op  es.  */
138       break;
139
140     default:
141       abort ();
142     }
143   } while (next_option != -1);
144
145   /* Esse programa n o recebe nenhum argumento adicional.  Ser  mostrado uma
         mensagem de erro se o
146      usu rio especificar qualquer argumento adicional.  */
147   if (optind != argc)
148     print_usage (1);
149
150   /* Mostre o diret rio do m dulo, Se estivermos executando com mensagens
         detalhadas.  */
151   if (verbose)
```

Listagem 11.11: (main.c) Continuação

```
152     printf ("m dulos ir o ser carregados a partir de %s\n", module_dir);
153
154   /* execute o servidor.  */
155   server_run (local_address, port);
156
157   return 0;
158 }
```

O programa *main.c* contém essas funções:

- a função *main* chama *getopt_long* (veja Seção 2.1.3, "Usando *getopt_long*" no Capítulo 2) para tratar opções de linha de comando. A *getopt_long* fornece ambas as formas de opção longa e curta, a antiga no array *long_options* e a mais nova na sequência de caracteres *short_options*. O valor padronizado para a porta escutada pelo servidor é 0 e para um endereço local é INADDR_ANY. Esses valores padronizados podem ser sobrescritos pelas opções *−−port* (-p) e *−−address* (-a), respectivamente. Se o usuário especifica um endereço, a função *main* chama a função de biblioteca *gethostbyname*[4] para converter esse endereço fornecido pelo usuário para um endereço de *Internet* numérico.
  O valor padrão para o diretório do qual chamar módulos de servidor é o diretório contendo o executável *server*, como determinado por *get_self_executable_directory*. O usuário pode sobrescrever o valor contido em *get_self_executable_directory* com a opção *−−module-dir* (-m); a função *main* garante que o diretório especificado esteja acessível.
  Por padrão, mensgens detalhadas não são impressas. O usuário pode habilitar as mensagens detalhadas especificando a opção *−−verbose* (-v).

- Se o usuário especificar a opção *−−help* (-h) ou especificar alguma opção inválida, a função *main* chama a função *print_usage*, a qual mostra um sumário de uso e sai.

11.3 Modulos

Fornecemos quatro módulos para demonstrar o tipo de funcionalidade que você pode implementar usando essa implementação de servidor. Implementar seu próprio módulo de servidor é tão simples quanto definir uma função *module_generate* para retornar um texto apropriado na linguagem HTML.

[4]A função de biblioteca *gethostbyname* realiza a resolução de nomes usando DNS, se necessário.

11.3.1 Mostra a Hora do Relógio Comum

O módulo *time.so* (veja a Listagem 11.12) gera uma única página contendo a hora do local do relógio comum do servidor. A função *module_generate* desse módulo chama *gettimeofday* para obter a hora atual (veja Seção 8.7, "A chamada *gettimeofday*: Hora Relógio Comum" no Capítulo 8, "Chamadas de Sistema do GNU/Linux") e usa *localtime* e *strftime* para gerar uma representação em modo texto da hora solicitada. Essa representação é embutida no modelo HTML *page_template*.

Listagem 11.12: (time.c) Módulo do Servidor para Mostrar a Hora Relógio Comum

```
1  #include <assert.h>
2  #include <stdio.h>
3  #include <sys/time.h>
4  #include <time.h>
5
6  #include "server.h"
7
8  /* Um modelo para p gina HTML que esse m dulo gera.  */
9
10 static char* page_template =
11   "<html>\n"
12   " <head>\n"
13   "  <meta http-equiv=\"refresh\" content=\"5\">\n"
14   " </head>\n"
15   " <body>\n"
16   "  <p>The current time is %s.</p>\n"
17   " </body>\n"
18   "</html>\n";
19
20 void module_generate (int fd)
21 {
22   struct timeval tv;
23   struct tm* ptm;
24   char time_string[40];
25   FILE* fp;
26
27   /* Obt m a data e hora atual, e converte-a para uma tm struct.  */
28   gettimeofday (&tv, NULL);
29   ptm = localtime (&tv.tv_sec);
30   /* Formata a data e hora, despreza os d cimos e mil simos de segundo.  */
31   strftime (time_string, sizeof (time_string), "%H:%M:%S", ptm);
32
33   /* Cria um fluxo correspondendo ao descritor de arquivo de socket
34      do cliente.  */
35   fp = fdopen (fd, "w");
36   assert (fp != NULL);
37   /* Gera a sa da HTML.  */
38   fprintf (fp, page_template, time_string);
39   /* Tudo terminado; esvazie o fluxo.  */
40   fflush (fp);
41 }
```

Esse módulo usa as rotinas de E/S da biblioteca C GNU padrão por conveniência. A chamada a *gefdopen* gera um apontador de fluxo (FILE*) correspondendo ao descritor de arquivo do *socket* do cliente (veja Seção B.4, "Relação de Funções de E/S da Biblioteca C GNU Padrão" no Apêndice B, "E/S de Baixo Nível"). O módulo escreve para o apontador de fluxo usando *fprintf* e descarrega o fluxo usando *fflush* para evitar a perda de dados armazenados no espaço temporário de armazenagem quando o *socket* é fechado.

A página HTML retornada pelo módulo *time.so* inclui um elemento *<meta>* no cabeçalho da página que instrui os clientes a atualizar a página a cada 5 segundos. Dessa forma o cliente mosta a hora atual.

11.3.2 Mostra a Distribuição GNU/Linux

O módulo *issue.so* (veja Listagem 11.13) mostra informação sobre a distribuição GNU/Linux instalada no servidor. Essa informação é tradicionalmente armazenada no arquivo */etc/issue*. Esse módulo envia o conteúdo desse arquivo, envolvido em um elemento *<pre>* de uma página HTML.

Listagem 11.13: (issue.c) Módulo de Servidor para Mostrar Informação da Distribuição GNU/Linux

```
1 #include <assert.h>
2 #include <stdio.h>
3 #include <sys/time.h>
4 #include <time.h>
5
6 #include "server.h"
7
8 /* Um modelo para p gina HTML que esse m dulo gera.  */
9
10 static char* page_template =
11   "<html>\n"
12   " <head>\n"
13   "  <meta http-equiv=\"refresh\" content=\"5\">\n"
14   " </head>\n"
15   " <body>\n"
16   "  <p>The current time is %s.</p>\n"
17   " </body>\n"
18   "</html>\n";
19
20 void module_generate (int fd)
21 {
22   struct timeval tv;
23   struct tm* ptm;
24   char time_string[40];
25   FILE* fp;
26
27   /* Obt m a data e hora atual, e converte-a para uma tm struct.  */
28   gettimeofday (&tv, NULL);
29   ptm = localtime (&tv.tv_sec);
30   /* Formata a data e hora, despreza os d cimos e mil simos de segundo.  */
31   strftime (time_string, sizeof (time_string), "%H:%M:%S", ptm);
32
33   /* Cria um fluxo correspondendo ao descritor de arquivo de socket
34      do cliente.  */
35   fp = fdopen (fd, "w");
36   assert (fp != NULL);
37   /* Gera a sa da HTML.  */
38   fprintf (fp, page_template, time_string);
39   /* Tudo terminado; esvazie o fluxo.  */
40   fflush (fp);
41 }
```

O módulo primeiramente tenta abrir */etc/issue*. Se o */etc/issue* não puder ser aberto, o módulo envia uma página de erro para o cliente. De outra forma, o módulo envia o início da página HTML, contido em *page_start*. Então o módulo *issue.so* envia o conteúdo do */etc/issue* usando a chamada de sistema *sendfile* (veja a Seção 8.12, "A Chamada *sendfile*: Transferência de Dados Rápida" no Capítulo 8). Finalmente, o módulo *issue.so* envia o fim de página HTML, contido em *page_end*.

Você pode facilmente adaptar esse módulo para enviar o conteúdo de outro arquivo. Se o arquivo contiver uma página completa HTML, simplesmente omita o código que envia o conteúdo de *page_start* e *page_end*. Você pode também adaptar a implementação do sevidor principal para disponibilizar arquivos estáticos, da maneira de um servidor *Web* tradicional. O uso de *sendfile* fornece um grau extra de eficiência.

11.3.3 Mostrando o Espaço Livre do Disco

O módulo *diskfree.so* (veja a Listagem 11.14) gera uma página mostrando informação sobre o espaço livre do disco sobre os sistemas de arquivos montados no computador servidor. Essa informação gerada é simplesmente a saída de uma chamada ao comando *df -h*. Da mesma forma que o módulo *issue.so*, esse módulo empacota a saída em um elemento *<pre>* de uma página HTML.

Listagem 11.14: (diskfree.c) Módulo de Servidor para Mostrar Informações Sobre Espaço Livre no Disco

```
1 #include <assert.h>
2 #include <stdio.h>
3 #include <sys/time.h>
4 #include <time.h>
5
6 #include "server.h"
7
8 /* Um modelo para p gina HTML que esse m dulo gera.  */
9
10 static char* page_template =
11   "<html>\n"
12   " <head>\n"
13   "  <meta http-equiv=\"refresh\" content=\"5\">\n"
14   " </head>\n"
15   " <body>\n"
16   "  <p>The current time is %s.</p>\n"
17   " </body>\n"
18   "</html>\n";
19
20 void module_generate (int fd)
21 {
22   struct timeval tv;
23   struct tm* ptm;
24   char time_string[40];
25   FILE* fp;
26
27   /* Obt m a data e hora atual, e converte-a para uma tm struct.  */
28   gettimeofday (&tv, NULL);
29   ptm = localtime (&tv.tv_sec);
30   /* Formata a data e hora, despreza os d cimos e mil simos de segundo.  */
31   strftime (time_string, sizeof (time_string), "%H:%M:%S", ptm);
32
33   /* Cria um fluxo correspondendo ao descritor de arquivo de socket
34      do cliente.  */
35   fp = fdopen (fd, "w");
36   assert (fp != NULL);
37   /* Gera a sa da HTML.  */
38   fprintf (fp, page_template, time_string);
39   /* Tudo terminado; esvazie o fluxo.  */
40   fflush (fp);
41 }
```

Enquanto *issue.so* envia o conteúdo de um arquivo usando *sendfile*, esse módulo deve chamar um comando e redirecionar sua saída para o cliente.

Para fazer isso, o módulo segue esses passos:

1. Primeiramente, o módulo cria um processo filho usando *fork* (veja Seção 3.2.2, "Usando Bifurcar e Executar" no Capítulo 3).

2. O processo filho copia o descritor de arquivo do *socket* do cliente para os descritores de arquivo STDOUT_FILENO e STDERR_FILENO, os quais correspondem à saída padrão e à saída de erro (veja Seção 2.1.4, "E/S Padrão" no Capítulo 2. Os descritores de arquivo são copiados usando a chamada *dup2* (veja Seção 5.4.3, "Redirecionando os Fluxos da Entrada Padrão, da Saída Padrão e de Erro" no Capítulo 5). Toda a saída adicional do processo para qualquer desses fluxos é enviada para o *socket* do cliente.

3. O processo filho chama o comando *df* como a opção -h por meio de uma chamada a *execv* (veja Seção 3.2.2, "Usando Bifurcar e Executar" no Capítulo 3).

4. O processo pai espera pela saída do processo filho por meio de uma chamada a *waitpid* (veja Seção 3.3.3, "As Chamadas de Sistema da Família *wait*" no Capítulo 3).

Você pode facilmente adaptar esse módulo para chamar um comando diferente e redirecionar sua saída para o cliente.

11.3.4 Sumarizando Processos Executando

O módulo *processes.so* (veja Listagem 11.15) é uma implementação de módulo de servidor mais extensível. O módulo *processes.so* gera uma página contendo uma tabela que sumariza os processos atualmente executando no sistema do servidor. Cada processo é representado por uma linha na tabela que lista o PID, o nome do programa executável, o usuário proprietário e o nomes dos grupos, e o tamanho do conjunto residente.

Listagem 11.15: (processes.c) Módulo de Servidor para Sumarizar Processos

```
1 #include <assert.h>
2 #include <dirent.h>
3 #include <fcntl.h>
4 #include <grp.h>
5 #include <pwd.h>
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
9 #include <sys/stat.h>
10 #include <sys/types.h>
11 #include <sys/uio.h>
12 #include <unistd.h>
13
14 #include "server.h"
15
16 /* Ajusta *UID e *GID para o ID do usu rio propriet rio e para o ID de grupo,
       respectivamente,
17    do PID do processo.  Retorna zero em caso de successo, n o zero em caso de falha.
          */
18
19 static int get_uid_gid (pid_t pid, uid_t* uid, gid_t* gid)
20 {
21   char dir_name[64];
22   struct stat dir_info;
23   int rval;
24
25   /* Gera o nome do diret rio do processo no /proc.  */
26   snprintf (dir_name, sizeof (dir_name), "/proc/%d", (int) pid);
27   /* Obt m informa  o sobre o diret rio.  */
28   rval = stat (dir_name, &dir_info);
29   if (rval != 0)
30     /* N o posso encontr -lo; pode ser que o processo n o mais exista.  */
31     return 1;
32   /* garanta que isso seja um diret rio; qualquer outra coisa    inexperada.  */
33   assert (S_ISDIR (dir_info.st_mode));
34
35   /* Extrai os IDs que desejamos.  */
36   *uid = dir_info.st_uid;
37   *gid = dir_info.st_gid;
38   return 0;
39 }
40
41 /* Retorna o nome do UID do usu rio.  O valor de retorno    um espa o tempor rio
       de armazenamento que o
42    chamador deve alocar com free.  UID deve ser um ID de usu rio v lido.  */
43
44 static char* get_user_name (uid_t uid)
45 {
46   struct passwd* entry;
47
48   entry = getpwuid (uid);
49   if (entry == NULL)
50     system_error ("getpwuid");
51   return xstrdup (entry->pw_name);
52 }
53
54 /* Retorna o nomedo GID de grupo.  O valor de retorno    um espa o tempor rio de
       armazenamento que o
55    chamador deve alocar com free.  GID deve ser um ID de grupo v lido.  */
56
57 static char* get_group_name (gid_t gid)
58 {
59   struct group* entry;
60
61   entry = getgrgid (gid);
62   if (entry == NULL)
63     system_error ("getgrgid");
64   return xstrdup (entry->gr_name);
65 }
66
67 /* Retorna o nome do programa executando no PID do processo, ou NULL em caso
68    de erro.  O valor de retorno    um recentemente alocado espa o tempor rio de
          armazenamento
69    o qual o chamador deve desalocar com free.  */
70
71 static char* get_program_name (pid_t pid)
72 {
73   char file_name[64];
74   char status_info[256];
75   int fd;
76   int rval;
77   char* open_paren;
78   char* close_paren;
79   char* result;
```

Listagem 11.16: (processes.c) Continuação

```
80    /* Gera o nome do arquivo "stat" no diret rio no /proc
81       do processo, e abre-o.  */
82    snprintf (file_name, sizeof (file_name), "/proc/%d/stat", (int) pid);
83    fd = open (file_name, O_RDONLY);
84    if (fd == -1)
85      /* N o pode abrir o arquivo de stat para esse processo.  Pode ser que o
86         processo n o mais exista.  */
87      return NULL;
88    /* L   o conte do.  */
89    rval = read (fd, status_info, sizeof (status_info) - 1);
90    close (fd);
91    if (rval <= 0)
92      /* N o posso ler, por alguma raz o; reclame.  */
93      return NULL;
94    /* NUL-terminado o conte do do arquivo.  */
95    status_info[rval] = '\0';
96
97    /* O nome do programa    o segundo elemento do conte do do arquivo, e e'
98       envolvido por par ntesis.  Encontre as posi  es dos par ntesis
99       no conte do do arquivo.  */
100   open_paren = strchr (status_info, '(');
101   close_paren = strchr (status_info, ')');
102   if (open_paren == NULL
103       || close_paren == NULL
104       || close_paren < open_paren)
105     /* n o posso encontr -los; reclame.  */
106     return NULL;
107   /* Aloque mem ria para o resultado.  */
108   result = (char*) xmalloc (close_paren - open_paren);
109   /* Copie o nome do programa para dentro do resultado.  */
110   strncpy (result, open_paren + 1, close_paren - open_paren - 1);
111   /* strncpy n o NUL-termina o resultado, ent o fa a isso aqui.  */
112   result[close_paren - open_paren - 1] = '\0';
113   /* Tudo terminado.  */
114   return result;
115 }
116
117 /* Retorne o tamanho do conjunto residente (RSS), em kilobytes, do PID do processo.
118    Retorne -1 em caso de falha.  */
119
120 static int get_rss (pid_t pid)
121 {
122   char file_name[64];
123   int fd;
124   char mem_info[128];
125   int rval;
126   int rss;
127
128   /* Gere o nome da entrada "statm" do processo no seu diret rio
129      /proc.  */
130   snprintf (file_name, sizeof (file_name), "/proc/%d/statm", (int) pid);
131   /* Abra-o.  */
132   fd = open (file_name, O_RDONLY);
133   if (fd == -1)
134     /* N o posso abr -lo; pode ser que o processo n o mais exista.  */
135     return -1;
136   /* L   o conte do do arquivo.  */
137   rval = read (fd, mem_info, sizeof (mem_info) - 1);
138   close (fd);
139   if (rval <= 0)
140     /* N o posso ler o conte do; reclame.  */
141     return -1;
142   /* NUL-termina o conte do.  */
143   mem_info[rval] = '\0';
144   /* extrai o RSS.  esse    o segundo item.  */
145   rval = sscanf (mem_info, "%*d %d", &rss);
146   if (rval != 1)
147     /* O conte do de statm    formatado de uma forma que n o entendemos.  */
148     return -1;
149
150   /* O valor em statm est  em unidades do tamanho de p gina do sistema.
151      Converta o RSS para kilobytes.  */
152   return rss * getpagesize () / 1024;
153 }
154
155 /* Gere uma linha de tabela HTML paa o PID de processo.  O valor de retorno    um
156    apontador para um espa o tempor rio de armazenamento o qual o chamador deve
          desalocar com free, ou
157    NULL se um erro ocorrer.  */
158
159 static char* format_process_info (pid_t pid)
```

Listagem 11.17: (processes.c) Continuação

```
{
  int rval;
  uid_t uid;
  gid_t gid;
  char* user_name;
  char* group_name;
  int rss;
  char* program_name;
  size_t result_length;
  char* result;

  /* Obtém os IDs usuário e grupo do processo. */
  rval = get_uid_gid (pid, &uid, &gid);
  if (rval != 0)
    return NULL;
  /* Obtém o RSS do processo. */
  rss = get_rss (pid);
  if (rss == -1)
    return NULL;
  /* Obtém no nome de programa do processo. */
  program_name = get_program_name (pid);
  if (program_name == NULL)
    return NULL;
  /* Converte os IDs de grupo e usuário para os nomes correspondentes. */
  user_name = get_user_name (uid);
  group_name = get_group_name (gid);

  /* Calcula o comprimento da sequência de caracteres que iremos precisar para
       manter o resultado, e
     aloca a memória para mantê-la. */
  result_length = strlen (program_name)
    + strlen (user_name) + strlen (group_name) + 128;
  result = (char*) xmalloc (result_length);
  /* Formata o resultado. */
  snprintf (result, result_length,
            "<tr><td align=\"right\">%d</td><td><tt>%s</tt></td><td>%s</td>"
            "<td>%s</td><td align=\"right\">%d</td></tr>\n",
            (int) pid, program_name, user_name, group_name, rss);
  /* Limpa. */
  free (program_name);
  free (user_name);
  free (group_name);
  /* tudo terminado. */
  return result;
}

/* Fonte HTML para o início da página de listagem do processo. */

static char* page_start =
  "<html>\n"
  " <body>\n"
  "  <table cellpadding=\"4\" cellspacing=\"0\" border=\"1\">\n"
  "   <thead>\n"
  "    <tr>\n"
  "     <th>PID</th>\n"
  "     <th>Program</th>\n"
  "     <th>User</th>\n"
  "     <th>Group</th>\n"
  "     <th>RSS (KB)</th>\n"
  "    </tr>\n"
  "   </thead>\n"
  "   <tbody>\n";

/* fonte HTML para o fim da página de listagem do processo. */

static char* page_end =
  "   </tbody>\n"
  "  </table>\n"
  " </body>\n"
  "</html>\n";

void module_generate (int fd)
{
  size_t i;
  DIR* proc_listing;

  /* Ajusta um vetor estático iovec. Iremos preenchê-lo com espaços temporários
       de armazenamento que irão ser
     parte de nossa saída, ajustando-o dinâmicamente quando necessário. */

  /* O número de elementos no vetor estático que teremos usado. */
  size_t vec_length = 0;
```

Listagem 11.18: (processes.c) Continuação

```
240   /* O tamanho alocado do vetor est tico.  */
241   size_t vec_size = 16;
242   /* O vetor est tico dos elementos de iovcec.  */
243   struct iovec* vec =
244     (struct iovec*) xmalloc (vec_size * sizeof (struct iovec));
245
246   /* O primeiro espa o tempor rio de armazenamento    o fonte HTML para o in cio
        da p gina.  */
247   vec[vec_length].iov_base = page_start;
248   vec[vec_length].iov_len = strlen (page_start);
249   ++vec_length;
250
251   /* Inicia uma listagem de diret rio para /proc.  */
252   proc_listing = opendir ("/proc");
253   if (proc_listing == NULL)
254     system_error ("opendir");
255
256   /* Ciclo sobre as entradas de diret rio em /proc.  */
257   while (1) {
258     struct dirent* proc_entry;
259     const char* name;
260     pid_t pid;
261     char* process_info;
262
263     /* Pegue a entrada seguinte no /proc.  */
264     proc_entry  = readdir (proc_listing);
265     if (proc_entry == NULL)
266       /* temos que alcan ar o fim da listagem.  */
267       break;
268
269     /* Se essa entrada n o for composta puramente de d gitos, isso n o    um
270        diret rio de processo, ent o ignore-a.  */
271     name = proc_entry->d_name;
272     if (strspn (name, "0123456789") != strlen (name))
273       continue;
274     /* O nome da entrada    o ID do processo.  */
275     pid = (pid_t) atoi (name);
276     /* Gera o HTML para uma linha de tabela descrevendo esse processo.  */
277     process_info = format_process_info (pid);
278     if (process_info == NULL)
279       /* Alguma coisa deu errado.  O processo pode ter desaparecido enquanto
             est vamos
280          olhando para ele.  Use uma linha reservada ao inv s da linha montada antes.
               */
281       process_info = "<tr><td colspan=\"5\">ERROR</td></tr>";
282
283     /* Garanta que o vetor est tico iovec    grande o suficiente para manter esse
             espa o tempor rio de armazenamento
284        (adicione um a mais, uma vez que iremos adicionar um elemento extra quando
               terminarmos
285        processos de listagem).  Se n o for grande o suficiente, aumente-o para o
               dobro do seu tamanho atual.  */
286     if (vec_length == vec_size - 1) {
287       vec_size *= 2;
288       vec = xrealloc (vec, vec_size * sizeof (struct iovec));
289     }
290     /* Armazene esse espa o tempor rio como sendo o elemento seguinte do vetor
             est tico.  */
291     vec[vec_length].iov_base = process_info;
292     vec[vec_length].iov_len = strlen (process_info);
293     ++vec_length;
294   }
295
296   /* termine a opera  o de listagem de diret rios.  */
297   closedir (proc_listing);
298
299   /* Adicione um  ltimo espa o tempor rio  com o HTML que termina a p gina.  */
300   vec[vec_length].iov_base = page_end;
301   vec[vec_length].iov_len = strlen (page_end);
302   ++vec_length;
303
304   /* Sa da da p gina inteira para o cliente tudo de uma vez.  */
305   writev (fd, vec, vec_length);
306
307   /* Desaloque os espa os tempor rios que criamos.  O primeiro e o  ltimo s o
          est ticos,
308      e n o devem ser desalocados.  */
309   for (i = 1; i < vec_length - 1; ++i)
310     free (vec[i].iov_base);
311   /* Desaloque o vetor est tico iovec.  */
312   free (vec);
313 }
```

O ato de reunir dados dos processos e formatá-los como uma tabela HTML é quebrado em algumas operações mais simples:

- *get_uid_gid* extrai os IDs do proprietário e do grupo de um processo. Para fazer isso, a função chama *stat* (veja Seção B.2, "*stat*" no Apêndice B) sobre o subdiretório do processo no */proc* (veja a Seção 7.2, "Entradas dos Processos" no Capítulo 7). O usuário e o grupo que são proprietários desse diretório são idênticos ao usuário e grupo proprietários do processo.

- *get_user_name* retorna o nome de usuário correspondente a um determinado UID. Essa função simplesmente chama a função *getpwuid* da biblioteca C GNU padrão, que consulta o arquivo */etc/passwd* do sistema e retorna uma cópia do resultado. *get_group_name* retorna o nome do grupo correspondente a um determinado GID. A função *get_group_name* usa a chamada de sistema *getgrgid*.

- *get_program_name* retorna o nome do programa executando em um processo especificado. Essa informação é extraída da entrada *stat* no diretório do processo sob o */proc* (veja Seção 7.2, "Entradas de Processos" no Capítulo 7). Usamos essa entrada do */proc* em lugar de examinar o link simbólico *exe* (veja Seção 7.2.4, "O Executável do Processo" no Capítulo 7) ou a entrada *cmdline* (veja Seção 7.2.2, "Lista de Argumentos do Processo" no Capítulo 7) pelo fato de esses últimos dois estarem inacessíveis se o processo executando o programa *server* não for de propriedade do mesmo usuário que o processo sendo examinado. Também, lendo de *stat* não força o GNU/Linux a paginar o processo sob exame de volta à memória, se ocorrer de o processo ser removido da área de *swap*.

- *get_rss* retorna o tamanho do conjunto residente de um processo. Essa informação está disponível como o segundo elemento no conteúdo da entrada *statm* do processo (veja Seção 7.2.6, "Estatísticas de Memória do Processo" no Capítulo 7) no seu subdiretório do */proc*.

- *format_process_info* gera uma sequência de caracteres contendo elementos HTML para uma única linha de tabela, representando um único processo. Após chamar as funções listadas previamente para preencher a única linha, *format_process_info* aloca uma área temporária de armazenagem e gera o código HTML usando função *snprintf* da Biblioteca C GNU Padrão.

- *module_generate* gera a página HTML completa, incluindo a tabela. A saída consiste de uma sequência de caracteres contendo o início da página e a tabela (em *page_start*), uma sequência de caracteres para cada linha da tabela (gerada por *format_process_info*), e uma sequência de caracteres contendo o fim da tabela e a página (em *page_end*). A função *module_generate* determina os PIDs dos processos executando no sistema por meio de um exame ao conteúdo do */proc*. A função *module_generate* obtém uma listagem desse diretório usando *opendir* e *readdir* (veja a Seção B.6, "Lendo o Conteúdo de um Diretório" no Apêndice B). A função *module_generate* percorre o conteúdo, procurando por entradas cujos nomes são compostos inteiramente de dígitos; esses nomes compostos inteiramente de dígitos são usados para serem entradas de processos. Potencialmente um grande número de sequências de caractere devem ser escritas no *socket* do cliente uma sequência para cada início e fim de página, adicionalmente uma para cada processo. Se tivermos de escrever cada sequência de caracteres para o descritor de arquivo do *socket* do cliente com uma chamada separada a *write*, pode gerar tráfego de rede desnecessário pelo fato de cada sequência de caracteres pode ser enviada em um pacote separado de rede.
  Para otimizar a embalagem de dados em pacotes, usamos uma única chamada a *writev* ao invés de chamar *write* (veja Seção B.3, "Leituras e Escritas de Vetor" no Apêndice B). Para fazer isso, devemos construir um array de objetos *struct iovec*, *vec*. Todavia, pelo fato de não sabermos o número de processos antecipadamente, devemos iniciar com um pequeno array e expandir esse mesmo array à medida que novos processos são adicionados. A variável *vec_length* contém o número de elementos de *vec* que são usados, enquanto *vec_size* contém o tamanho alocado de *vec*. Quando *vec_length* está prestes a extrapolar *vec_size*, expandimos *vec* para duas vezes seu tamano por meio de uma chamada a *xrealloc*. Quando estamos terminando com a escrita do vetor, devemos desalocar tudo da sequências de caractere dinâmicamente alocada apontada pelo *vec*, e então desalocar o *vec* propriamente dito.

11.4 Usando o Servidor

Se estivermos planejando distribuir esse programa por meio de seu código fonte, mantendo esse código fonte sobre uma base contínua, ou portá-lo para outras plantaformas, provavelmente iremos desejar empacotar esse código fonte usando o *GNU Automake* e o *GNU Autoconf*, ou um similar sistema

de configuração automática. Tais ferramentas estão fora do escopo desse livro; para mais informações sobre as ferramentas de configuração automática, consulte o *GNU Autoconf, Automake, and Libtool* (de autoria de Vaughan, Elliston,Tromey, e Taylor, publicado pela *New Riders*, 2000).

11.4.1 O Makefile

Ao invés de usar o *Autoconf* ou uma ferramenta similar, fornecemos um único *Makefile* compatível com o *GNU Make*[5] de forma que é fácil compilar e linkar o programa *server* e seus módulos. O *Makefile* é mostrado na Listagem 11.19. Veja a página *info* do *GNU Make* para detalhes da sintaxe do arquivo[6].

Listagem 11.19: (Makefile) Arquivo de Configuração para Exemplo de Servidor

```
1 ### Configura o. ##################################################
2
3 # Arquivos fontes em C para o servidor.
4 SOURCES          = server.c module.c common.c main.c
5 # Arquivos objeto correspondentes aos arquivos fontes.
6 OBJECTS          = $(SOURCES:.c=.o)
7 # M dulo servidor de arquivos de biblioteca compartilhada.
8 MODULES          = diskfree.so issue.so processes.so time.so
9
10 ### Rules. ##########################################################
11
12 .PHONY:          all clean
13
14 # Alvo padr o: constr i tudo.
15 all:             server $(MODULES)
16
17 # Limpa os produtos da constru o.
18 clean:
19         rm -f $(OBJECTS) $(MODULES) server
20
21 # O programa servidor principal.  Link com -Wl,-export-dyanamic de forma que
22 # m dulos dinamicamente chamados possam associar s mbolos no programa.  Link em
23 # libdl, a qual cont m chamadas para carregamento din mico.
24 server:          $(OBJECTS)
25         $(CC) $(CFLAGS) -Wl,-export-dynamic -o $@ $^ -ldl
26
27 # Todos os arquivos objeto no servidor dependem de server.h.  Mas use a
28 # regra padr o para contruir arquivos objeto a partir dos arquivos fontes.
29 $(OBJECTS):      server.h
30
31 # Regra para construir bibliotecas de m dulo compartilhadas a partir dos
       correspondentes
32 # arquivos fontes.   Compile -fPIC e gere um arquivo objeto compartilhado.
33 $(MODULES): \
34 %.so:            %.c server.h
35         $(CC) $(CFLAGS) -fPIC -shared -o $@ $<
```

O *Makefile* fornece esses alvos:

- *all* (o padrão se você chama o *make* sem argumentos pelo fato de *all* ser o primeiro no *Makefile*) inclui o executável *server* e todos os módulos. Os módulos estão listados na variável MODULES.

[5] *GNU Make* vem instalado em sistemas GNU/Linux.

[6] Nota do tradutor: na linha de comando digite *info Make*.

- *clean* apaga qualquer arquivo produzido pelo *Makefile*.
- *server* linka somente o executável *server*. Os arquivos fontes listados na variável SOURCES são compilados e linkados também.
- A última regra é um modelo genérico para compilar arquivos de objetos compartilhados para os módulos do servidor para os correspondentes arquivos fonte.

Note que arquivos fonte para os módulos de servidor são compilados com a opção *-fPIC* pelo fato deles serem linkados dentro de bibliotecas compartilhadas (veja Seção 2.3.2,"Bibliotecas Compartilhadas" no Capítulo 2).

Também observe que o executável *server* é linkado com a opção de compilador *-Wl,-export-dynamic*. Com essa opção, GCC informa a opção *-export-dynamic* para o linkador, o qual cria um arquivo executável que também exporta seus símbolos externos como uma biblioteca compartilhada. Isso permite aos módulos, os quais são chamados dinâmicamente como bibliotecas compartilhadas, referencias funções de *common.c* que foram linkadas estaticamente dentro do executável *server*.

11.4.2 Gerando o Executável do Programa Server

Compilar e linkar programas é fácil. De dentro do diretório contendo os fontes, simplesmente chame *make*:

```
% make
cc -Wall   -g     -c -o server.o server.c
cc -Wall   -g     -c -o module.o module.c
cc -Wall   -g     -c -o common.o common.c
cc -Wall   -g     -c -o main.o main.c
cc -Wall   -g   -Wl,-export-dynamic -o server server.o module.o common.o main.o -ldl
cc -Wall   -g   -fPIC -shared -o diskfree.so diskfree.c
cc -Wall   -g   -fPIC -shared -o issue.so issue.c
cc -Wall   -g   -fPIC -shared -o processes.so processes.c
cc -Wall   -g   -fPIC -shared -o time.so time.c
```

Ao chamar o *make* compila-se e linka-se o programa *server* e as bibliotecas compartilhadas de módulos.

```
% ls -l server *.so
-rwxr-xr-x    1 samuel        samuel      25769   Mar   11   01:15   diskfree.so
-rwxr-xr-x    1 samuel        samuel      31184   Mar   11   01:15   issue.so
-rwxr-xr-x    1 samuel        samuel      41579   Mar   11   01:15   processes.so
-rwxr-xr-x    1 samuel        samuel      71758   Mar   11   01:15   server
-rwxr-xr-x    1 samuel        samuel      13980   Mar   11   01:15   time.so
```

11.4.3 Executando o Programa Server

Para rodar o servidor, simplesmente chame o executável *server*.

Se você não especificar o número de porta que o *server* deve escutar com a opção *−−port* (-p), GNU/Linux irá escolher uma para você; nesse caso,

especifique −−*verbose* (-v) para fazer com que o *server* mostre na tela o número de porta em uso.

Se você não especificar um endereço com a opção −−*address* (-a), o executável *server* irá estar acessível a todos os seus endereços de computadores de rede. Se seus computadores estão conectados a uma rede, isso significa que outros irão ser capazes de acessar o programa *server*, desde que eles conheçam o número de porta correto a ser usado e a página a requisitar. Por razões de segurança, é uma boa idéia especificar o endereço *localhost* até que você esteja convencido de que o programa *server* trabalha corretamente e não está liberando informação que você prefere não tornar pública. Associando ao *localhost* faz com que o servidor associe para o dispositivo de rede local (designado "lo") – somente programas executando no mesmo computador podem conectar ao programa *server*. Se você especificar um endereço diferente, esse endereço diferente deve corresponder a seu computador:

```
% ./server --address localhost --port 4000
```

O servidor está agora executando. Abrindo uma janela de navegador, e tente contactar o programa *server* nesse número de porta. Requisite uma página cujo nome coincide com um dos módulos. Por exemplo, chame o módulo *diskfree.so*, use a URL adiante:

```
http://localhost:4000/diskfree
```

Ao invés de 4000, informe o número de porta que você especificou (ou o número de prta que o GNU/Linux escolheu para você). Pressione *Ctrl+C* para interromper o programa *server* quando você tiver terminado. Se você não especificar *localhost* como o endereço do servidor, você pode também conectar-se ao programa *server* com um navegador rodando em outro computador por meio do uso do nome de *host* [7] do seu computador na URL – por exemplo:

```
http://host.domain.com:4000/diskfree
```

Se você especificou a opção −−*verbose* (-v), o programa *server* mostra na tela algumas informações durante a inicialização e mostra o endereço de *Internet* em sua forma numérica de cada cliente que se conectar a esse mesmo programa *sever*. Se você conecta via o endereço *localhost*, o endereço do cliente irá sempre ser 127.0.0.1.

[7] Nota do tradutor: para ver o nome do *host* simplesmente abra um terminal/konsole pois em algumas distribuições ele aparece entre o arroba e o cursor piscante ou use o comando *cat /etc/hostname*. No slackware temos o */etc/HOSTNAME*.

Se você fizer uma experiência e escrever seu próprio módulo de servidor, você pode colocá-lo em um diretório diferente daquele contendo os módulos atuais. Nesse caso, especifique o novo diretório onde está localizado o novo módulo com a opção −−*module-dir* (-m). O programa *server* irá olhar nesse diretório ao procurar por módulos de servidor ao invés de olhar na localização padrão.

Se você esquecer a sintaxe das opções de linha de comando, chame o programa *server* com a opção −−*help* (-h).

```
% ./server --help
Usage: ./server [ options ]
  -a, --address ADDR        Bind to local address (by default, bind to all local addresses).
  -h, --helpPrint this information.
  -m, --module-dir DIR      Load modules from specified directory
(by default, use executable directory).
  -p, --port PORT           Bind to specified port.
  -v, --verbose             Print verbose messages.
```

11.5 Terminando

Se você realmente planeja liberar esse programa para uso geral, você irá precisar escrever documentação para ele também. Muitas pessoas não percebem que escrever boa documentação é tão difícil e demorado – e tão importante – quanto escrever um bom programa. Todavia, documentação de programas é assunto para outro livro, de forma que iremos deixar você com algumas referência sobre onde aprender mais sobre como documentar programas GNU/Linux.

Você irá provavelmente desejar escrever uma página de manual para o programa *server*, por exemplo. A página de manual é o primeiro lugar onde muitos usuários irão olhar buscando informações sobre um programa. Páginas de manual são formatadas usando um sistema de formatação clássico do UNIX chamado *troff*. Para ver uma página de manual para *troff*, a qual descreve o formato de arquivos *troff*, use o seguinte comando:

```
% man troff
```

Para aprender sobre como GNU/Linux localiza páginas de manual, consulte a página de manual do comando *man* propriamente dito por meio do seguinte comando:

```
% man man
```

Você pode também desejar escrever páginas *info*, usando o sistema *GNU Info*, para o programa *server* e seus módulos. Naturalmente, documentação sobre o sistema *info* vem no formato *info*; para ver essas informações, use o seguinte comando:

```
% info info
```

Muitos programas GNU/Linux vêem com documentação no formato texto puro ou no formato HTML também.

Desejamos que o ato de programar com GNU/Linux seja feliz para você!

Parte III

Apêndices

- A Outras Ferramentas de Desenvolvimento
- B E/S de Baixo Nível
- C Tabela de Sinais
- D Online Resources
- E Open Publication License Version 1.0
- F GNU General Public License
- G Saídas Diversas do */proc*
- H Adicionais ao Capítulo 8 "Chamadas de Sistema do GNU/Linux"
- I *Assembly*
- J Segurança
- K Anexos aos Apêndices

Apêndice A

Outras Ferramentas de Desenvolvimento

O DESENVOLVIMENTO PROGRAMAS GNU/LINUX CORRETOS e rápidos requer mais que apenas entender o sistema operacional GNU/Linux e suas chamadas de sistema. Nesse apêndice, discutiremos ferramentas de desenvolvimento para encontrar erros durante a execução tais como uso ilegal de memória alocada dinâmicamente e para determinar quais partes de um programa estão consumindo mais do tempo de execução. A analize do código de um programa pode revelar alguma dessas informações; por meio do uso dessas ferramentas de execução e executando o programa dentro dessa ferramenta de execução, você pode encontrar muito mais.

A.1 Análise Estática do Programa

Alguns erros de programação podem ser detectados usando ferramentas de análise estática que analizam o código fonte do programa. Se você chama o GCC com -Wall e -pedantic, o compilador mostra alertas sobre erradas ou possivelmente errôneas construções de programação. Através da eliminação de tais construções, você irá reduzir o risco de erros de programa, e você irá encontrar essas construções errôneas facilmente ao compilar seus programas com diferentes variantes GNU/Linux e mesmo em outros sistemas operacionais.

Usando várias opções de comando, você pode fazer com que o GCC mostre alertas sobre muitos diferentes tipos de construções questionáveis de programação. A opção -Wall habilita a maioria dessas verificações. Por exemplo, o compilador irá produzir um alerta sobre um comentário que começa dentro de outro comentário, sobre um tipo incorreto de retorno especificado

pela função *main*, e sobre uma função não *void* omitindo uma declaração de retorno. Se você especificar a opção -pedantic, GCC emite alertas demandados por obediência estrita a ANSI C e ISO C++. Por exemplo, o uso da extensão*GNU asm* fará com que ocorra um alerta de uso dessa opção. Umas poucas extensões GNU, tais como o uso de palavras chave alternativas começando com __ (duas sublinhas), não irá disparar mensagens de alerta. Embora as páginas *info* do GCC censure o uso dessa opção, recomendamos que você a use mesmo assim e evite a maioria das extensões de linguagem GNU pelo fato de as extensões do GCC tenderem a mudar com o decorrer do tempo e frequêntemente interagirem pobremente com a otimização de código.

Listagem A.1: (hello.c) Programa Alô Mundo

```
1 int main ()
2 {
3   printf ("Al , mundo.\n");
4   return 0;
5 }
```

Considere compilar o programa "Alô Mundo" o programa mostrado na Listagem A.1. Apesar de o GCC compilar o programa sem reclamar, o código fonte não obedece as regras ANSI C. Se você habilitar alertas por meio de uma compilação com as opções -Wall -pedantic, o GCC revela três construções questionáveis.

```
% gcc -Wall -pedantic hello.c
hello.c:2: warning: return type defaults to 'int'
hello.c: In function 'main':
hello.c:3: warning: implicit declaration of function 'printf'
hello.c:4: warning: control reaches end of non-void function
```

Esses alertas indicam que os seguintes problemas ocorreram:

- O tipo de dado de retorno da função *main* não foi especificado.
- A função *printf* está implicitamente declarada pelo fato de <stdio.h> não estar incluída.
- A função, implicitamente declarada para retornar um tipo de dado *int*, atualmente não retorna nenhum valor.

Analizando um código fonte de programa não podemos encontrar todos os enganos de programação e ineficiências. Na seção seguintes, mostramos quatro ferramentas paa encontrar enganos no uso de memória dinâmicamente alocada. Na seção subsequênte, mostramos como analizar o tempo de execução de um programa usando o gerador de perfil *gprof*.

A.2 Encontrando Erros de Memória Alocada Dinâmicamente

Ao escrever um programa, você frequantemente não pode saber quanta memória o programa irá precisar quando estiver sendo executado. Por exemplo, uma linha lida de um arquivo na hora da execução pode ter qualquer comprimento finito. Programas em C e em C++ usam *malloc*, *free*, e suas variantes para alocar memória dinâmicamente enquanto o programa está rodando. As regras para o uso de memória dinâmica incluem as seguintes:

- O número de chamadas de alocação (chamadas a *malloc*) deve exatamente coincidir com o número de chamadas de desalocação (chamadas a *free*).
- Leituras e escritas para a memória alocada devem ocorrer dentro da memória alocada, não fora desse intervalo.
- A memória alocada não pode ser usada antes de ser alocada ou após ser desalocada.

Pelo fato de alocação e desalocação dinâmica de memória ocorrerem durante a execução, a análise estática de programas raramente encontra violações. Ao invés disso, ferramentas de verificação de memória executam o programa, coletando dados para determinar se qualquer regra foi violada. As violações que uma ferramenta pode encontrar incluem as seguintes:

- Leitura da memória antes de alocá-la
- Escrita na memória antes de alocá-la
- Leitura antes do início da memória alocada
- Escrita antes do início da memória alocada
- Leitura após o fim da memória alocada
- Escrita após o fim da memória alocada
- Leitura da memória após sua desalocação
- Escrita na memória após sua desalocação
- Falha na desalocação da memória alocada
- Desalocação da mesma memória duas vezes

- Desalocação de memória que não foi alocada

É também importante alertar sobre requisitar uma alocação com 0 *bytes*, a qual provavelmente indica erro do programador.

A Tabela A.1 indica quatro diferentes capacidades de ferramentas de diagnóstico. Desafortunadamente, nenhuma ferramenta simples diagnostica todos os usos errados de memória. Também, nenhuma ferramenta alega detectar leitura ou escrita antes de fazer a alocação de memória, mas fazendo isso irá provavelmente causar uma falha de segmentação. Desalocação de memória duas vezes irá provavelmente também causar uma falha de segmentação. Essas ferramentas diagnosticam erros que atualmente ocorrem enquanto o programa está sendo executado. Se você roda o programa sem entradas que façam com que nenhuma memória seja alocada, a ferramenta irá indicar a você que não existem erros de memória. Para testar um programa completamente, você deve executar o programa usando diferentes entradas para garantir que todo possível caminho através do programa seja seguido. Também, você pode usar somente uma ferramenta de cada vez, de forma que você irá ter que repetir o teste com muitas ferramentas para ter a melhor verificação de erro[1].

[1]Nota do tradutor: veja no Apêndice K Seção K.2 uma relação de analizadores de código.

Tabela A.1: Capacidades das Ferramentas de Verificação Dinâmica de Memória (X Indica Detecção, e O Indica Detecção para Alguns Casos)

Comportamento Errôneo	*malloc Checking*	*mtrace*	*ccmalloc*	*Eletric Fence*
Ler antes de alocar memória				
Escrever antes de alocar memória				
Ler antes do início da alocação				X
Escrever antes do início da alocação	O		O	X
Ler após o fim da alocação				X
Escrever após o fim da alocação			X	X
Ler após desalocar				X
Escrever após desalocar				X
Falha ao desalocar memória			X	
Desalocar memória duas vezes	X		X	
Desalocar memória não alocada		X	X	
Alocação de memória de tamanho zero			X	X

Nas seções que seguem, primeiro descrevemos como usar os mais facilmente usados *malloc checking* e *mtrace*, e então *ccmalloc* e *Electric Fence*.

A.2.1 Um Programa para Testar Alocação e Desalocação de Memória

Iremos usar o programa *malloc-use* na listagem Listagem A.2 para ilustrar alocação de memória, desalocação de memória, e como usar a alocação e a desalocação. Para iniciar rode o programa *malloc-use*, especificando o número máximo de regiões de memória alocada como seu único argumento de linha de comando. Por exemplo, *malloc-use* 12 cria um array A com 12 apontadores de caractere que não apontam para nada. O programa aceita cinco diferentes comandos:

- Para alocar b *bytes* apontados por meio de uma entrada de array A[i], insira a i b. O índice de array i pode ser qualquer número não negativo menor que o argumento de linha de comando. O número de *bytes* deve ser não negativo.
- Para desalocar memória no índice de array i, insira d i.
- Para ler o p-ésimo caractere da memória alocada no índice i (como em A[i][p]), insira r i p. Aqui, p pode ter um valor inteiro.
- Para escrever um caractere na p-ésima posição na memória alocada no índice i, insira w i p.
- Quando terminar, insira q.

Iremos mostrar o código do programa mais tarde, na Seção A.2.7, e ilustrar como usá-lo.

A.2.2 *malloc Checking*

As funções de alocação de memória fornecidas pela biblioteca C GNU padrão podem detectar escrita antes do início de uma alocação e desalocação da mesma alocação duas vezes. Definindo a variável de ambiente MALLOC_CHECK_ para o valor 2 faz com que um programa pare ao ser detectado tal erro. (Note que a variável de ambiente termina com uma sublinha.) Não é necessário recompilar o programa.

Ilustraremos o diagnóstico de uma escrita para a memória em uma posição apenas antes do início de uma alocação.

```
% export MALLOC_CHECK_=2
% ./malloc-use 12
Please enter a command: a 0 10
Please enter a command: w 0 -1
Please enter a command: d 0
Aborted (core dumped)
```

O comando *export* habilita *malloc checking*. Especificando o valor 2 faz com que o programa pare na hora em que um erro é detectado.

O uso de *malloc checking* é vantajoso pelo fato de o programa não precisar ser recompilado, mas sua capacidade de diagnosticar erros é limitada. Basicamente, o programa *malloc checking* verifica se a estrutura de dados do alocador não foi corrompida. Dessa forma, o programa *malloc checking* pode detectar desalocação dupla da mesma alocação. Também, a escrita apenas antes do início de uma alocação de memória pode comumente ser detectada pelo fato de o alocador armazenar o tamanho de cada alocação de memória apenas antes da região alocada. Dessa forma, escrita apenas antes da memória alocada irá corromper esse número. Desafortunadamente, verificação de consistência pode ocorrer somente quando seu programa chama rotinas de alocação, não quando seu programa acessa a memória, de forma que muitas leituras ilegais e escritas ilegais podem ocorrer antes de um erro ser detectado. No exemplo prévio, a escrita ilegal foi detectada somente quando a memória alocada foi desalocada.

A.2.3 Encontrando Vazamento de Memória Usando *mtrace*

A ferramenta *mtrace* ajuda a diagnosticar o mais comum erro quando usamos memória dinâmica: falha em corresponder, em termos numéricos, as alocações e desalocações. Existem quatro passos a serem seguidos para se usar *mtrace*, a qual está disponível com a biblioteca C GNU padrão:

1. Modifique o código fonte incluindo <mcheck.h> e chame *mtrace ()* tão logo o programa inicie, no início da *main*. A chamada a *mtrace* habilita rastreamento de alocações de memória e de desalocações de memória.

2. Especifique o nome de um arquivo para armazenar informação sobre todas as alocações e desalocações:

```
% export MALLOC_TRACE=memory.log
```

3. Execute o programa. Todas a alocações e desalocações de memória são armazenadas no arquivo de registro de transações *memory.log*.

4. Usando o comando *mtrace*, analize as alocações e as desalocações de memória para garantir que elas coincidam.

```
% mtrace my_program $MALLOC_TRACE
```

As mensagens produzidas por *mtrace* são relativamente fáceis de entender Por exemplo, para o nosso exemplo *malloc-use*, a saída seria parecida com o seguinte:

```
- 0000000000 Free 3 was never alloc'd malloc-use.c:39

Memory not freed:
-----------------
   Address     Size       Caller
0x08049d48      0xc    at malloc-use.c:30
```

Essas mensagens indicam uma tentativa na linha 39 de *malloc-use.c* para liberar memória que nunca foi alocada, e uma alocação de memória na linha 30 que nunca foi liberada. O comando *mtrace* diagnostica erros tendo a gravação de toda a alocação e desalocação de memória feita pelo programa executável no arquivo especificado pela variável de ambiente MALLOC_TRACE. O executável deve terminar normalmente para os dados serem escritos. O comando *mtrace* analiza esse arquivo e lista alocações e desalocações não coincidentes.

A.2.4 Usando ccmalloc

A biblioteca *ccmalloc* diagnostica erros de memória alocada dinâmicamente por meio da substituição de *malloc* e *free* com código de rastreio de seu uso. Se o programa termina elegantemente, *ccmalloc* produz um relatório de vazamento de memória e outros erros. A bilioteca *ccmalloc* foi escrita por Armin Bierce.

Você irá provavelmente ter que baixar e instalar a biblioteca *ccmalloc* você mesmo. Pegue a *ccmalloc* a partir de http://www.inf.ethz.ch/personal/biere /projects/ccmalloc/, [2] descompacte o código, e rode o *script configure*. exe-

[2]Nota do tradutor: o endereço ainda é válido mas deu erro no navegador - 403 *Forbidden*. Em http://www.filewatcher.com/ digite *ccmalloc-0.4.0.tar.gz* em *Filename Search* que aparecerá uma série de sítios onde o arquivo pode ser encontrado. O sítio original continua sendo citado na *Internet* mas está inaccessível no presente momento. O tamanho do arquivo que encontrei é 57.917 *bytes*.

cute a seguir o *make* e o *make install*, copie o arquivo *ccmalloc.cfg* para o diretório onde você irá executar o programa que você deseja testar, e renomeie a cópia para *.ccmalloc*. Agora você está pronto para usar a ferramenta.

Os arquivos objetos devem ser linkados com a biblioteca *ccmalloc* e com a biblioteca de linkagem dinâmica[3]. Anexe *-lccmalloc -ldl* a seu comando de linkagem, por exemplo.

```
% gcc -g -Wall -pedantic malloc-use.o -o ccmalloc-use -lccmalloc -ldl
```

Execute o programa para produzir um relatório. Por exemplo, executando nosso programa *malloc-use* para alocar mas não desalocar memória produz o seguinte relatório:

```
% ./ccmalloc-use 12
file-name=a.out does not contain valid symbols
trying to find executable in current directory ...
using symbols from 'ccmalloc-use'
(to speed up this search specify 'file ccmalloc-use'
 in the startup file '.ccmalloc')
Please enter a command: a 0 12
Please enter a command: q
.---------------.
|ccmalloc report|
=====================================================
| total # of| allocated   |  deallocated |       garbage |
+-----------+-------------+-------------+---------------+
|       60  |          48 |           12 |          bytes|
+-----------+-------------+-------------+---------------+
|allocations|            2|            1|              1|
+-----------------------------------------------------+
| number of checks: 1                                  |
| number of counts: 3                                  |
| retrieving function names for addresses ... done.    |
| reading file info from gdb ... done.                 |
| sorting by number of not reclaimed bytes ... done.   |
| number of call chains: 1                             |
| number of ignored call chains: 0                     |
| number of reported call chains: 1                    |
| number of internal call chains: 1                    |
| number of library call chains: 0                     |
=====================================================
|
*100.0% = 12 Bytes of garbage allocated in 1 allocation
|       |
|       |        0x400389cb in <???>
|       |
|       |        0x08049198 in <main>
|       |   malloc-use.c:89
|       |
|       |        0x08048fdc in <allocate>
|       |   malloc-use.c:30
|       |
|       '-----> 0x08049647 in <malloc>
|          at src/wrapper.c:284
|
'-----------------------------------------------------
```

As últimas poucas linhas indicam a sequência de chamadas de funções que alocou memória que não foi desalocada.

Para usr *ccmalloc* para diagnosticar escritas antes do início ou após o fim da região alocada, você irá ter que modificar o arquivo *.ccmalloc* no diretório atual [4]. Esse arquivo é lido quando o programa inica sua execução [5].

[3]Nota do tradutor: *man dlltool*.

[4]Nota do tradutor: onde o teste está sendo feito.

[5]Nota do tradutor: http://zentire.com/teaching/cs352_summer05/notes/ccmalloc-howto.pdf.

A.2.5 *Electric Fence*

Escrito por Bruce Perens, *Electric Fence* pára os programas que estão em execução na linha exata do código onde uma escrita ou leitura é feita fora da área de memória alocada ocorre. Essa é a única ferramenta que descobre leituras ilegais. O programa *Eletric Fence* está incluído nas maiorias da distribuições GNU/Linux, mas o código fonte pode ser encontrado em http://www.perens.com/FreeSoftware/.

Da mesma forma que ocorre com *ccmalloc*, seus arquivos objeto do programa devem ser linkados com a biblioteca do *Electric Fence* por meio da colocação no final do comando de linkagem *-lefence*, por exemplo:

```
% gcc -g -Wall -pedantic malloc-use.o -o emalloc-use -lefence
```

Como o programa executa, os usos da memória alocada são verificados à procura de correções. Uma violação faz com que ocorra uma falha de segmentação[6]:

```
% ./emalloc-use 12
  Electric Fence 2.0.5 Copyright (C) 1987-1998 Bruce Perens.
Please enter a command: a 0 12
Please enter a command: r 0 12
Segmentation fault
```

Usando um depurador, você pode determinar o contexto da ação ilegal.

Por padrão, *Electric Fence* diagnostica somente acessos após o final das alocações. Para encontrar acessos antes do início das alocações ao invés de acessos após o fim das alocações, use esse código:

```
% export EF_PROTECT_BELOW=1
```

Para encontrar acessos a memória desalocada, ajuste EF_PROTECT_FR EE para 1. Mais capacidades são descritas na página de manual *libefence*.

Electric Fence diagnostica acessos legais à memória através da armazenagem de cada alocação em ao menos duas páginas de memória. least two memory pages. O *Electric Fence* coloca a alocação no fim da primeira página; qualquer acesso além do fim da alocação, na segunda página, causará uma falha de segmentação. Se você ajustou EF_PROTECT_BELOW para 1, o *Electric Fence* coloca a alocação no início da segunda página ao invés de colocar na primeira página. Pelo fato de alocar duas páginas de memória por chamada a *malloc*, o *Electric Fence* pode usar uma enorme quantidade de memória. Use essa biblioteca para depuração somente.

[6]Nota do tradutor:
http://perens.com/FreeSoftware/ElectricFence/electric-fence_2.1.13-0.1.tar.gz.

A.2.6 Escolhendo Entre as Diferentes Ferramentas Depuradoras de Memória

Discutimos quatro ferramentas separadas e incompatíveis ferramentas para diagnosticar uso errôneo de memória dinâmica. Como faz um programador GNU/Linux para garantir que memória dinâmica seja corretamente usada? Nenhuma ferramenta garante o diagnóstico de todos os erros, mas usando qualquer uma delas aumenta a possibilidade de encontrar erros. Para facilmente encontrar todos os erros de memória alocada dinâmicamente, desenvolva separadamente o código que controla a memória dinâmica. Esse procedimento reduz o total de código no qual você deve procurar os erros. Se você está usando C++, escreva uma classe que controle todo o uso da memória dinâmica. Se você estiver usando C, minimize o nmero de funções usando alocação e desalocação. Ao testar esse código, garanta usar somente uma ferramenta de cada vez pelo fato de elas serem incompatíveis. Ao testar um programa, garanta variação na forma como o programa executa, para testar a proção mais comumente executada do código.

Qual das quatro ferramentas você deve usar? Pelo fato de falhas de correspondência entre alocações e desalocações serem o erro mais comum de alocação de memória dinâmica, use *mtrace* durante o desenvolvimento inicial. O programa está disponível em todos os sistemas GNU/Linux e tem sido bem testado. Após a garantia de que o número de alocações e desalocações coincide, use *Electric Fence* para encontrar acessos ilegais à memória. Essa pesquisa de acessos ilegais à memória irá eliminar a maioria de todos os erros de memória. Ao usar o *Electric Fence*, você irá precisar ser cuidadoso não executar muitas alocações e desalocações pelo fato de cada alocação requerer pelo menos duas páginas de memória. Usando essas duas ferramentas irá revelar a maioria dos erros de memória.

A.2.7 Código Fonte para o Programa de Memória Dinâmica

A Listagem A.2 mostra o código fonte para um programa ilustrando alocação e desalocação de memória dinâmica, e como usar a memória dinâmica. Veja a Seção A.2.1, "Um Programa para Testar Alocação e Desalocação de Memória" para uma descrição de como usar alocação e desalocação de memóra.

Listagem A.2: (malloc-use.c) Exemplo de Como Testar Alocação Dinâmica de Memória

```
1  /* Uso das fun  es de aloca  o din mica de mem ria da linguagem C.  */
2
3  /* Chame o programa usando um argumento de linha de comando especificando o
4     tamanho de um vetor est tico.  Esse vetor est tico consiste de apontadores a (
          possivelmente)
5     vetores est ticos alocados.
6
7     Quando o programa estiver executando, selecione entre os seguintes
8     comandos:
9
10    o aloque mem ria:   a < ndice> <tamanho-mem ria>
11    o desaloque mem ria:        d < ndice>
12    o leia da mem ria:  r < ndice> <posi  o -na- aloca  o >
13    o escreva na mem ria:w < ndice> <posi  o -na- aloca  o >
14    o sair:             q
15
16    O usu rio    respons vel por obedecer (ou desobedecer as regras acerca do
17    uso de mem ria din micamente.  */
18
19 #ifdef MTRACE
20 #include <mcheck.h>
21 #endif /* MTRACE */
22 #include <stdio.h>
23 #include <stdlib.h>
24 #include <assert.h>
25
26 /* Aloca mem ria com um tamanho espec fico, retornado n o zero em caso de
27    sucesso.  */
28
29 void allocate (char** array, size_t size)
30 {
31   *array = malloc (size);
32 }
33
34 /* desaloca mem ria.  */
35
36 void deallocate (char** array)
37 {
38   free ((void*) *array);
39 }
40
41 /* Leia de uma posi  o de mem ria.  */
42
43 void read_from_memory (char* array, int position)
44 {
45   volatile char character = array[position];
46 }
47
48 /* escreve para uma posi  o na mem ria.  */
49
50 void write_to_memory (char* array, int position)
51 {
52   array[position] = 'a';
53 }
54
55 int main (int argc, char* argv[])
56 {
57   char** array;
58   unsigned array_size;
59   char command[32];
60   unsigned array_index;
61   char command_letter;
62   int size_or_position;
63   int error = 0;
64
65 #ifdef MTRACE
66   mtrace ();
67 #endif /* MTRACE */
68
69   if (argc != 2) {
70     fprintf (stderr, "%s: array-size\n", argv[0]);
71     return 1;
72   }
73
74   array_size = strtoul (argv[1], 0, 0);
75   array = (char **) calloc (array_size, sizeof (char *));
76   assert (array != 0);
77
78   /* Segue os comandos do usu rio.  */
```

Listagem A.3: (malloc-use.c) Exemplo de Como Testar Alocação Dinâmica de Memória

```
79    while (!error) {
80      printf ("Please enter a command: ");
81      command_letter = getchar ();
82      assert (command_letter != EOF);
83      switch (command_letter) {
84
85      case 'a':
86        fgets (command, sizeof (command), stdin);
87        if (sscanf (command, "%u %i", &array_index, &size_or_position) == 2
88            && array_index < array_size)
89          allocate (&(array[array_index]), size_or_position);
90        else
91          error = 1;
92        break;
93
94      case 'd':
95        fgets (command, sizeof (command), stdin);
96        if (sscanf (command, "%u", &array_index) == 1
97            && array_index < array_size)
98          deallocate (&(array[array_index]));
99        else
100         error = 1;
101       break;
102
103     case 'r':
104       fgets (command, sizeof (command), stdin);
105       if (sscanf (command, "%u %i", &array_index, &size_or_position) == 2
106           && array_index < array_size)
107         read_from_memory (array[array_index], size_or_position);
108       else
109         error = 1;
110       break;
111
112     case 'w':
113       fgets (command, sizeof (command), stdin);
114       if (sscanf (command, "%u %i", &array_index, &size_or_position) == 2
115           && array_index < array_size)
116         write_to_memory (array[array_index], size_or_position);
117       else
118         error = 1;
119       break;
120
121     case 'q':
122       free ((void *) array);
123       return 0;
124
125     default:
126       error = 1;
127     }
128   }
129
130   free ((void *) array);
131   return 1;
132 }
```

A.3 Montando Perfil

Agora que seu programa está (esperançosamente) correto, voltaremos nossa atenção para aumentar a velocidade de sua execução. Usando o gerador de perfil *gprof*, você pode determinar quais funções requerem a maioria do tempo de execução. O gerador de perfil *gprof* pode ajudar você a determinar quais partes do programa otimizar ou reescrever para executar mais rapidamente. O gerador de perfil *gprof* pode também ajudar você a encontrar. Por exemplo, você pode encontrar que uma particular função é chamada muito mais vezes que você esperava.

Nessa seção, descrevemos como usar o gerador de perfil *gprof*. Reescrevendo código para executar mais rapidamente requer criatividade e cuidadosa escolha de algorítmos.

O ato de obter informação de perfil requer três passos:

1. Compilar e linkar seu programa para habilitar a geração de perfil.
2. Execute seu programa para gerar dados de perfil.
3. Use *gprof* para analizar e mostrar os dados de perfil.

Antes de ilustrarmos esses passos, introduziremos um programa grande o suficiente para tornar a geração do perfil interessante.

A.3.1 Uma Calculadora Simples

Para ilustrar a geração de um perfil, usaremos um programa de calculadora simples. Para garantir que a calculadora use um montante de tempo não trivial, usaremos números unários para os cálculos, alguma coisa qque nós definitivamente não desejamos fazer em um programa do mundo real. O código para esse programa aparece no final desse capítulo.

Um número unário é representado por tantos símbolos quantos forem seu valor. Por exemplo, o número 1 é representado por "x" 2 por "xx" e 3 por "xxx." Ao invés de usar letras x, nosso programa representa um número não negativo usando uma lista encadeada com tantos elementos quantos o valor do número. O arquivo *number.c* contém rotinas para criar o número 0, adicionar 1 a um número, subtrair 1 de um número, e adicionar, subtrair, e multiplicar números. Outra função converte uma sequência de caracteres realizando a conversão de um número decimal não negativo para um número unário, e uma função converte de número unário para *int*. A operação de adição é implementada usando repetidas adições da unidade, enquanto a subtração usa repetidas remoções de unidades. A multiplicação é definida usando adições repetidas. Os predicados unários par e ímpar cada um retorna o número unário para 1 se e somente se seu único operando for par ou ímpar, respectivamente; de outra forma eles retornam o númeo unário para 0. Os dois predicados (par e ímpar) são mutuamente exclusivos. Por exemplo, um número é par se for zero, ou se um número abaixo desse número for ímpar.

A calculadora aceita expressões com operador pós fixado do tamanho de uma linha[7] e mostra o valor de cada expressão – por exemplo:

[7]Na notação com operador pós fixado, um operador binário é colocado após seus operando ao invés de entre eles. De forma que, por exemplo, para multiplica 6 e 8, você deve usaar "6 8 x" . Para multiplicar 6 e 8 e então adicionar 5 ao resultado, você deve usar 6 8 5 +.

```
% ./calculator
Please enter a postfix expression:
2 3 +
5
Please enter a postfix expression:
2 3 + 4 -
1
```

A calculadora, definida em *calculator.c*, lê cada expressão, armazenando valores intermediários em uma pilha de números unários, definida em *stack.c*. a pilha armazena seus números unários em uma lista encadeada.

A.3.2 Coletando Informações de Montagem de Perfil

O primeiro passo na montagem do perfil de um programa é fazer o compilador incluir informações adicionais em seus executáveis para coletar informações de montagem de perfil. Para fazer isso, use o sinalizador de compilação "-pg" ao fazer ambas a compilação dos arquivos objetos e a linkagem. Por exemplo, considere esse código:

```
%   gcc   -pg   -c -o calculator.o calculator.c
%   gcc   -pg   -c -o stack.o stack.c
%   gcc   -pg   -c -o number.o number.c
%   gcc   -pg   calculator.o stack.o number.o -o calculator
```

Isso habilita a coleta de informações sobre chamadas a funções e informações de cronometragem. Para coletar informação de uso linha por linha, também especifique o sinalizador de depuração "-g" . Para contar blocos de execução básicos, tais como o número de interações em um "do-loop", use o sinalizador "-a".

O segundo passo é executar o programa. Enquanto o programa está executando, dados de montagem de perfil é coletado em um arquivo chamado *gmon.out*, somente para aquelas porções de código que forem usadas. Você deve variar as entradas ou comandos do programa para executar as seções de código qu você deseja montar o perfil. O programa deve terminar normalmente para que o arquivo de montagem de perfil seja escrito.

A.3.3 Mostrando Dados de Montagem de Perfil

Fornecido o nome de um arquivo executável, o *gprof* o arquivo *gmon.out* para mostrar informações sobre quanto tempo cada função requereu. Por

exemplo, considere os dados de montagem de perfil de "flat" para o cálculo de 1787 x 13 - 1918 usando nosso programa calculador, o qual é produzido por meio da execução de *gprof ./calculator*:

```
Flat profile:

Each sample counts as 0.01 seconds.


  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 26.07      1.76     1.76 20795463   0.00     0.00   decrement_number
 24.44      3.41     1.65     1787   0.92     1.72   add
 19.85      4.75     1.34 62413059   0.00     0.00   zerop
 15.11      5.77     1.02     1792   0.57     2.05   destroy_number
 14.37      6.74     0.97 20795463   0.00     0.00   add_one
  0.15      6.75     0.01     1788   0.01     0.01   copy_number
  0.00      6.75     0.00     1792   0.00     0.00   make_zero
  0.00      6.75     0.00       11   0.00     0.00   empty_stack
```

Calculando a função *decrement_number* e todas as funções que *decrement_number* chama requereu 26.07% do total de tempo de execução do programa. A função *decrement_number* foi chamada 20,795,463 vezes. Cada execução individual requereu 0.0 segundos – a saber, um tempo muito pequeno para se medir. A função *add* foi chamada 1,787 ezes, presumivelmente para calcular o produto. Cada chamada requereu 0.92 segundos. A função *copy_number* function was invoked only 1,788 times, while it and the functions it calls required only 0.15% do total do tempo de execução. Algumas vezes as funções *mcount* e *profil* usadas na montagem do perfil aparecem nos dados.

Adicionalmente aos dados de perfil de flat, o qual indica o tempo total gasto dentro de cada função, *gprof* produz o gráfico de chamadas mostrando o tempo gasto em cada função e seus processos filhos dentro do contexto de um encadeamento de chamada de função:

```
index % time    self  children    called     name
                                                 <spontaneous>
[1]    100.0    0.00    6.75                 main [1]
                0.00    6.75       2/2           apply_binary_function [2]
                0.00    0.00       1/1792        destroy_number [4]
                0.00    0.00       1/1           number_to_unsigned_int [10]
                0.00    0.00       3/3           string_to_number [12]
                0.00    0.00       3/5           push_stack [16]
                0.00    0.00       1/1           create_stack [18]
                0.00    0.00       1/11          empty_stack [14]
                0.00    0.00       1/5           pop_stack [15]
                0.00    0.00       1/1           clear_stack [17]
-----------------------------------------------
                0.00    6.75       2/2           main [1]
[2]    100.0    0.00    6.75       2         apply_binary_function [2]
                0.00    6.74       1/1           product [3]
                0.00    0.01       4/1792        destroy_number [4]
                0.00    0.00       1/1           subtract [11]
                0.00    0.00       4/11          empty_stack [14]
                0.00    0.00       4/5           pop_stack [15]
                0.00    0.00       2/5           push_stack [16]
-----------------------------------------------
                0.00    6.74       1/1           apply_binary_function [2]
[3]     99.8    0.00    6.74       1         product [3]
                1.02    2.65    1787/1792        destroy_number [4]
                1.65    1.43    1787/1787        add [5]
                0.00    0.00    1788/62413059     zerop [7]
                0.00    0.00       1/1792        make_zero [13]
```

O primeiro quadro mostra que a execução da função *main* e seus filhos requereu 100% dos 6.75 segundos do programa. A função *main* chamou a função *apply_binary_function* duas vezes, que por sua vez foi chamada um total de duas vezes ao longo de todo o programa. Seu chamador foi <*spontaneous*>; isso indica que o gerador de perfil não foi capaz de determinar quem chamou a função *main*. O primeiro quadro também mosta que *string_to_number* chamou *push_stack* três vezes mas foi chamada cinco vezes ao longo de todo o programa. O terceiro quadro mostra que a execução da função *product* e as funções que *product* chamou requer 99.8% do tempo total de execução do programa. A função *product* foi chamada uma vez por *apply_binary_function*.

Os dados do gráfico de chamada mostram o tempo total gasto executando uma função e seus processos filhos. Se o gráfico de chamada de função é uma árvore esse número é fácil de calcular, mas funções definidas recursivamente devem ser tratadas especialmente. Por exemplo, a função *even* chama a função *odd*, a qual chama *even*. A cada ciclo de tal chamada é fornecido seu próprio número e é mostrado individualmente nos dados do gráfico de chamada. Considere esses dados de geração de perfil obtidos na determinação de se 1787133 é par:

```
-----------------------------------------------
                0.00    0.02       1/1           main [1]
[9]     0.1     0.00    0.02       1         apply_unary_function [9]
                0.01    0.00       1/1           even <cycle 1> [13]
                0.00    0.00       1/1806        destroy_number [5]
                0.00    0.00       1/13          empty_stack [17]
                0.00    0.00       1/6           pop_stack [18]
                0.00    0.00       1/6           push_stack [19]
-----------------------------------------------
[10]    0.1     0.01    0.00       1+69693   <cycle 1 as a whole> [10]
                0.00    0.00   34847             even <cycle 1> [13]
-----------------------------------------------
                                34847             even <cycle 1> [13]
[11]    0.1     0.01    0.00   34847         odd <cycle 1> [11]
                0.00    0.00   34847/186997954     zerop [7]
                0.00    0.00       1/1806        make_zero [16]
                                34846             even <cycle 1> [13]
```

O 1+69693 no quadro [10] indica que o ciclo 1 foi chamado uma vez, enquanto as funções no ciclo foram chamadas 69,693 vezes.O ciclo chamou a função *even*. A entrada seguinte mostra que a função *odd* foi chamada 34,847 vezes pela função *even*.

Nessa seção, discutimos brevemente somente alguns recursos do *gprof*. As páginas *info* do *gprof* contém informação sobre outros recursos úteis:

- Use a opção -s para sumarizar os resultados de muitas diferentes execuções.
- Use a opção -c para identificar processos filhos que porderiam ter sido chamados mas não foram.
- Use a opção -l para mostrar informação de geração de perfil linha por linha.

- Use a opção -A para mostrar código fonte anotado com números de porcentagem de execução.

As páginas *info* também fornecem mais informação sobre a interpretação dos dados analizados.

A.3.4 Como *gprof* Coleta Dados

Quando um executável cujo perfil se quer estabelecer roda, a cada vez que uma função é chamada seu contador é também incrementado. Também, *gprof* periodicamente interrompe o executável para determinar a função que está sendo executada atualmente. Essas amostras determinam o número de vezes que uma função é executada. Pelo fato de dois cliques seguidos do relógio do GNU/Linux terem a diferença entre sí de 0.01 segundos, essas interrupções ocorrem, no máximo, a cada 0.01 segundo. Dessa forma, perfís de programas que executam muito rapidamente ou para chamadas de funções rapidamente e raramente executadas podem ser imprecisas. Para evitar essas imprecisões, rode o executável por longos períodos de tempo, ou sumarize juntos dados de perfil de muitas execuções. Leia sobre a opção -s para sumarizar dados de geração de perfil na página *info* do *gprof*.

A.3.5 Código Fonte do Programa Calculadora

A Listagem A.4 mostra um programa que calcula o valor de expressões com operadores pós-fixados.

Listagem A.4: (calculator.c) Programa Principal da Calculadora

```
1 /* Calcula usando n meros un rios.  */
2
3 /* Insira express es uma por linha usando reverse postfix notation, e.g.,
4      602 7 5 - 3 * +
5    N meros n o negativos s o inseridos usndo n o t a  o decimal.  Os
6    operadores "+", "-", e "*" s o suportados.  Os operadores un rios
7    "even" e "odd" retornam o n mero um se seu nicooperando for par
8    ou mpar, respectivamente.  Espa os em branco devem separar todas as palavras.
9    N meros negativos n o s o suportados.  */
10
11 #include <stdio.h>
12 #include <stdlib.h>
13 #include <string.h>
14 #include <ctype.h>
15 #include "definitions.h"
16
17
18 /* Aplique a fun  o bin ria com operandos obtidos a partir da pilha,
19    colocando de volta na pilha a resposta.  Retorna n o zero em caso de sucesso.  */
20
21 int apply_binary_function (number (*function) (number, number),
22                            Stack* stack)
23 {
24   number operand1, operand2;
25   if (empty_stack (*stack))
26     return 0;
27   operand2 = pop_stack (stack);
28   if (empty_stack (*stack))
29     return 0;
30   operand1 = pop_stack (stack);
31   push_stack (stack, (*function) (operand1, operand2));
32   destroy_number (operand1);
33   destroy_number (operand2);
34   return 1;
35 }
36
37 /* Aplique a fun  o bin ria com operandos obtidos a partir da pilha,
38    colocando de volta na pilha a resposta.  Retorna n o zero em caso de sucesso.  */
39
40 int apply_unary_function (number (*function) (number),
41                           Stack* stack)
42 {
43   number operand;
44   if (empty_stack (*stack))
45     return 0;
46   operand = pop_stack (stack);
47   push_stack (stack, (*function) (operand));
48   destroy_number (operand);
49   return 1;
50 }
51
52 int main ()
53 {
54   char command_line[1000];
55   char* command_to_parse;
56   char* token;
57   Stack number_stack = create_stack ();
58
59   while (1) {
60     printf ("Por favor insira uma express o postfix:\n");
61     command_to_parse = fgets (command_line, sizeof (command_line), stdin);
62     if (command_to_parse == NULL)
63       return 0;
64
65     token = strtok (command_to_parse, " \t\n");
66     command_to_parse = 0;
67     while (token != 0) {
68       if (isdigit (token[0]))
69         push_stack (&number_stack, string_to_number (token));
70       else if (((strcmp (token, "+") == 0) &&
71                 !apply_binary_function (&add, &number_stack)) ||
72                ((strcmp (token, "-") == 0) &&
73                 !apply_binary_function (&subtract, &number_stack)) ||
74                ((strcmp (token, "*") == 0) &&
```

Listagem A.5: (calculator.c) Continuação

```
75                !apply_binary_function (&product, &number_stack)) ||
76               ((strcmp (token, "even") == 0) &&
77                !apply_unary_function (&even, &number_stack)) ||
78               ((strcmp (token, "odd") == 0) &&
79                !apply_unary_function (&odd, &number_stack)))
80          return 1;
81        token = strtok (command_to_parse, " \t\n");
82      }
83      if (empty_stack (number_stack))
84        return 1;
85      else {
86        number answer = pop_stack (&number_stack);
87        printf ("%u\n", number_to_unsigned_int (answer));
88        destroy_number (answer);
89        clear_stack (&number_stack);
90      }
91    }
92
93    return 0;
94  }
```

As funções na Listagem A.6 implementam números unários usando listas encadeadas vazias.

Listagem A.6: (number.c) Implementação de Número Unário

```
1  /* Opera sobre n meros un rios. */
2
3  #include <assert.h>
4  #include <stdlib.h>
5  #include <limits.h>
6  #include "definitions.h"
7
8  /* Cria um n mero representando zero. */
9
10 number make_zero ()
11 {
12   return 0;
13 }
14
15 /* Reorna n o zero se o n mero representa zero. */
16
17 int zerop (number n)
18 {
19   return n == 0;
20 }
21
22 /* Decrementa um n mero positivo de uma unidade. */
23
24 number decrement_number (number n)
25 {
26   number answer;
27   assert (!zerop (n));
28   answer = n->one_less_;
29   free (n);
30   return answer;
31 }
32
33 /* Adiciona um a um n mero. */
34
35 number add_one (number n)
36 {
37   number answer = malloc (sizeof (struct LinkedListNumber));
38   answer->one_less_ = n;
39   return answer;
40 }
41
42 /* Destroi um n mero. */
43
44 void destroy_number (number n)
```

Listagem A.7: (number.c) Continuação

```
45 {
46   while (!zerop (n))
47     n = decrement_number (n);
48 }
49
50 /* Copia um n mero.  Essa fun  o     somente necess ria devido a aloca  o de
51    mem ria.  */
52
53 number copy_number (number n)
54 {
55   number answer = make_zero ();
56   while (!zerop (n)) {
57     answer = add_one (answer);
58     n = n->one_less_;
59   }
60   return answer;
61 }
62
63 /* Adiciona dois n meros.  */
64
65 number add (number n1, number n2)
66 {
67   number answer = copy_number (n2);
68   number addend = n1;
69   while (!zerop (addend)) {
70     answer = add_one (answer);
71     addend = addend->one_less_;
72   }
73   return answer;
74 }
75
76 /* Subtrai um n mero de outro.  */
77
78 number subtract (number n1, number n2)
79 {
80   number answer = copy_number (n1);
81   number subtrahend = n2;
82   while (!zerop (subtrahend)) {
83     assert (!zerop (answer));
84     answer = decrement_number (answer);
85     subtrahend = subtrahend->one_less_;
86   }
87   return answer;
88 }
```

As funções na Listagem A.8 implementam uma pilha de números unários usando uma lista encadeada.

Listagem A.8: (stack.c) Pilha do Número Unário

```
1 /* Fornece uma pilha de "n mero"s.  */
2
3 #include <assert.h>
4 #include <stdlib.h>
5 #include "definitions.h"
6
7 /* Crira uma pilha vazia.  */
8
9 Stack create_stack ()
10 {
11   return 0;
12 }
13
14 /* Retorna n o zero se a pilha for vazia.  */
15
16 int empty_stack (Stack stack)
17 {
18   return stack == 0;
19 }
20
21 /* Remove o n mero no topo de uma pilha n o vazia.  Se a pilha estiver
22    vazia, aborte.  */
23
24 number pop_stack (Stack* stack)
```

Listagem A.9: (stack.c) Continuação

```
25 {
26   number answer;
27   Stack rest_of_stack;
28
29   assert (!empty_stack (*stack));
30   answer = (*stack)->element_;
31   rest_of_stack = (*stack)->next_;
32   free (*stack);
33   *stack = rest_of_stack;
34   return answer;
35 }
36
37 /* Adicione um n mero ao in cio da pilha.  */
38
39 void push_stack (Stack* stack, number n)
40 {
41   Stack new_stack = malloc (sizeof (struct StackElement));
42   new_stack->element_ = n;
43   new_stack->next_ = *stack;
44   *stack = new_stack;
45 }
46
47 /* Remove todos os elementos da pilha.  */
```

Listagem A.10 contendo declarações para pilhas e números.

Listagem A.10: (definitions.h) Arquivo de Cabeçalho para number.c e stack.c

```
1 #ifndef DEFINITIONS_H
2 #define DEFINITIONS_H 1
3
4 /* Implementa um n mero usando uma lista linkada.  */
5 struct LinkedListNumber
6 {
7   struct LinkedListNumber*
8                 one_less_;
9 };
10 typedef struct LinkedListNumber* number;
11
12 /* Implementa uma pilha de n meros como uma lista linkada.  Use 0 para representar
13    uma pilha vazia.  */
14 struct StackElement
15 {
16   number        element_;
17   struct        StackElement* next_;
18 };
19 typedef struct StackElement* Stack;
20
21 /* Opera sobre a pilha de n meros.  */
22 Stack create_stack ();
23 int empty_stack (Stack stack);
24 number pop_stack (Stack* stack);
25 void push_stack (Stack* stack, number n);
26 void clear_stack (Stack* stack);
27
28 /* Opera es sobre n meros.  */
29 number make_zero ();
30 void destroy_number (number n);
31 number add (number n1, number n2);
32 number subtract (number n1, number n2);
33 number product (number n1, number n2);
34 number even (number n);
35 number odd (number n);
36 number string_to_number (char* char_number);
37 unsigned number_to_unsigned_int (number n);
38
39 #endif /* DEFINITIONS_H */
```

Apêndice B

E/S de Baixo Nível

PROGRAMADORES EM C NO GNU/LINUX POSSUEM DOIS CONJUNTOS DE FUNÇÕES DE ENTRADA/SAÍDA à sua disposição. A biblioteca C GNU padrão fornece funções de E/S: *printf*, *fopen*, e assim por diante[1]. O *kernel* do GNU/Linux por sí mesmo fornece outro conjunto de operações de E/S que trabalham em um nível mais baixo que as funções da biblioteca C GNU padrão.

Pelo fato de esse livro ser para pessoas que já sabem a linguagem C, assumiremos que você tenha encontrado e saiba como usar as funções de E/S da biblioteca C GNU padrão.

Muitas vezes existem boas razões para usar as funções de E/S de baixo nível do *kernel* do GNU/Linux. Muitas dessas funções são chamadas de sistema[2] e fornecem a maioria das opções de acesso direto às capacidades subjacentes do sistema que está disponível a programas de aplicação. De fato, as rotinas da biblioteca C GNU padrão de E/S são implementadas no topo das chamadas de sistema de E/S de baixo nível do *kernel* do GNU/Linux. O uso das chamadas de sistema é usualmente o caminho mais eficiente para executar operações de entrada e saída e é algumas vezes mais conveniente, também.

Ao longo de todo esse livro, assumimos que você está familiarizado com as chamadas descritas nesse apêndice. Você pode já estar familiarizado com elas pelo fato de elas estarem muito próximas daquelas mesmas fornecidas em outros sistemas operacionais UNIX e semelhantes ao UNIX (e na plantaforma Win32 também). Se você não está familiarizado com elas, todavia, aprenda; você irá encontrar o restante do livro mais fácil para entender se

[1] A biblioteca padrão C++ fornece *iostreams* com funcionalidades similares. A biblioteca C GNU padrão está também disponível na linguagem C++.

[2] Veja Capítulor 8, "Chamadas de Sistema do GNU/Linux" para uma explanação das diferenças entre uma chamada e uma chamada de função comum.

você familiarizar-se por conta própria com as chamadas de sistema primeiramente.

B.1 Lendo e Escrevendo Dados

A primeira função de E/S que você provavelmente encontrou quando você inicialmente aprendeu a linguagem C foi *printf*. A *printf* formata um texto na forma de sequência de caracteres e então mostra o resultado da formatação na saída padrão. A versão generalizada, a função *fprintf*, pode imprimir um texto para um fluxo outro que não a saída padrão. Um fluxo é representado por meio de um apontador do tipo *FILE**. Você obtém um apontador do tipo *FILE** através da abertura de um arquivo com a função *fopen*. Quando você tiver terminado, você pode fechar o apontador do tipo *FILE** com a função *fclose*. Adicionalmente a *fprintf*, você pode usar funções outras tais como *fputc*, *fputs*, e *fwrite* para escrever dados para o fluxo, ou *fscanf*, *fgetc*, *fgets*, e *fread* para ler dados.

Com as operações de baixo nível de E/S do GNU/Linux, você usa um controlador de função chamado de um descritor de arquivos ao invés de um apontador do tipo *FILE**. Um descritor de arquivo é um valor inteiro que se refere a uma particular instância de um arquivo aberto em um único processo. Esse descritor de arquivo pode ser aberto para leitura, para escrita, ou para ambos leitura e escrita. Um descritor de arquivos não tem que referir-se a um arquivo aberto; um descritor de arquivo pode representar uma conecção com outro componente do sistema que é capaz de enviar ou receber dados. Por exemplo, uma conecção a um dispositivo de *hardware* é representada por um descritor de arquivo (veja Capítulo 6, "Dispositivos"), como pode ser um *socket* aberto (veja Capítulo 5, "Comunicação Entre Processos" Seção 5.5, "*Sockets*") ou um fim de um *pipe* (veja Seção 5.4, "*Pipes*").

Inclua os arquivos de cabeçalho <fcntl.h>, <sys/types.h>, <sys/stat.h>, e <unistd.h> se você usa qualquer das funções de E/S de baixo nível descritas aqui.

B.1.1 Abrindo um Arquivo

Para abrir um arquivo e produzir um descritor de arquivo que pode acessar o referido arquivo, use a chamada a *open*. A chamada a *open* recebe como argumentos o nome do caminho do arquivo a ser aberto, como uma sequência de caracteres, e sinalizadores especificando como abrir o arquivo explicitado no nome do caminho. Você pode usar *open* para criar um novo arquivo; se você quiser, informe um terceiro argumento que especifica as permissões de

acesso a serem colocadas no novo arquivo.

Se o segundo argumento for O_RDONLY, o arquivo é aberto para leitura somente; um erro irá resultar se você subsequêntemente tentar escrever para o descritor de arquivo resultante. Similarmente, O_WRONLY faz com que o descritor de arquivo seja para escrita somente. Especificando O_RDWR produz um descritor de arquivo que pode ser ambos para leitura e para escrita. Note que alguns arquivos não podem ser abertos em todos os três modos. Por exemplo, as permissões sobre um arquivo já existente podem proibir um processo particular de abrir o arquivo desejado para leitura ou para escrita; um arquivo localizado em um dispositivo que pode ser lido somente tal como um acionador de CD-ROM não pode ser aberto para escrita.

Você pode especificar opções adicionais por meio da utilização de operações *bit* a *bit ou* desse valor com um ou mais sinalizadores. Esses são os valores mais comumente usados:

- Especifique O_TRUNC para truncar o arquivo aberto, se o arquivo aberto já existir previamente. Dados escritos para o descritor de arquivo irão substituir o conteúdo anterior do arquivo.

- Especifique O_APPEND para anexar ao final de um arquivo que já existe. Dados escritos para o descritor de arquivo irão ser adicionados ao final do arquivo.

- Especifique O_CREAT para criar um novo arquivo. Se o nome de arquivo que você forneceu para ser aberto não existir, um novo arquivo será criado, contanto que o diretório que for receber o novo arquivo exista e que o processo tenha permissão para criar arquivos no diretório de destino. se o arquivo já existir anteriormente, ele será aberto ao invés de criado.

- Especifique O_EXCL com O_CREAT para forçar a criação de um novo arquivo. Se o arquivo já existir, a chamada a *open* irá falhar.

Se você chama *open* com O_CREAT, forneça um adicional terceiro argumento especificando as permissões para o novo arquivo. Veja Capítulo 10, "Segurança" Seção 10.3, "Permissões do Sistema de Arquivos" para uma descrição dos *bits* de permissão e como usá-los.

Por exemplo, o programa na Listagem B.1 cria um novo arquivo com o nome de arquivo especificado na linha de comando.O programa na Listagem B.1 usa o sinalizador O_EXCL com a *open*, de forma que se o arquivo existir anteriormente, um erro irá ocorrer. O novo arquivo é fornecido com permissões de leitura e escrita para o proprietário e o grupo proprietário,

e permissão de de leitura somente para os outros usuários. (Se sua *umask* for ajustada para um valor não nulo, as permissões atuais podem ser mais restritivas.)

Umasks

Quando você cria um novo arquivo com a *open*, alguns *bits* de permissão que você especificou podem ser desabilitados. Isso ocorre pelo fato de sua *umask* ser ajustada para um valor não nulo. Uma *umask* de processo especifica *bits* que são mascarados pelas permissões do arquivo criado recentemente. As permissões atuais usadas são operações *bit* a *bit e* das permissões que você especificou a *open* e a operação de complemento *bit* a *bit* da *umask*.
Para mudar sua *umask* a partir do *shell*, use o comando *umask*, e especifique o valor numérico da máscara, na notação octal. Para mudar a *umask* para um processo que está executando no momento, use a chamada a *umask*, informando o valor de máscara desejado para ser usado por subsequêntes chamadas a *open*.
Por exemplo, chamando essa linha

```
umask (S\_IRWXO | S\_IWGRP);
```

em um programa, ou chamando esse comando

```
% umask 027
```

especifica que permissões de escrita para membros do grupo e permissões de leitura, escrita, e execução para outros irão sempre ser mascaradas nas permissões de um novo arquivo.

Listagem B.1: (create-file.c) Cria um Novo Arquivo

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <sys/stat.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6
7 int main (int argc, char* argv[])
8 {
9   /* O caminho no qual se deve criar o novo arquivo.  */
10  char* path = argv[1];
11  /* As permiss es para o novo arquivo.  */
12  mode_t mode = S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP | S_IROTH;
13
14  /* Cria o novo arquivo.  */
15  int fd = open (path, O_WRONLY | O_EXCL | O_CREAT, mode);
16  if (fd == -1) {
17    /* Um erro ocorreu.  Mostre uma mensagem de erro e reclame.  */
18    perror ("open");
19    return 1;
20  }
21
22  return 0;
23 }
```

Aqui está o programa em ação:

```
% ./create-file testfile
% ls -l testfile
-rw-rw-r--    1 samuel   users 0 Feb   1 22:47 testfile
% ./create-file testfile
open: File exists
```

Note que o tamanho de novos arquivos é 0 pelo fato de o programa não escrever qualquer dado no arquivo criado.

B.1.2 Fechando Descritores de Arquivo

Quando você tiver terminado com um descritor de arquivo, feche esse descritor com *close*. Em alguns casos, tais como o do programa na Listagem B.1, não é necessário chamar *close* explicitamente pelo fato de GNU/Linux fechar todos os descritores de arquivo abertos quando um processo termina (isso é, quando o programa termina). Certamente, uma vez que você fecha um descritor de arquivo, você não deve mais usá-lo.

Fechando um descritor de arquivo pode fazer com que GNU/Linux tome uma ação em particular, dependendo da natureza do descritor de arquivo. Por exemplo, quando você fecha um descritor de arquivo para um *socket* de rede, GNU/Linux fecha a conecção de rede entre os dois computadores que estão se comunicando através do *socket*.

GNU/Linux limita o número de descritores de arquivo abertos que um processo pode ter aberto de uma só vez. Abrir descritores de arquivo usa recursos do *kernel*, de forma que é uma boa idéia fechar descritores de arquivo quando você tiver terminado com eles. Um limite típico é 1.024 descritores de arquivo por processo. Você pode ajustar esse limite com a chamada de sistema *setrlimit*; veja Seção 8.5, "As Chamadas *getrlimit* e *setrlimit*: Limites de Recurso" para mais informação.

B.1.3 Escrevendo Dados

Escrevemos dados para um descritor de arquivo usando a chamada a *write*. Forneça o descritor de arquivo, um apontador para um espaço temporário de armazenagem de dados, e o número de *bytes* a serem escritos. O descritor de arquivo deve ser aberto para escrita. Os dados escritos para o arquivo não precisam ser uma sequência de caracteres; escreva cópias arbitrárias de *bytes* a partir do espaço temporário de armazenagem para o descritor de arquivo.

O programa na Listagem B.2 anexa a hora atual para o arquivo especificado na linha de comando. Se o arquivo não existe, ele é criado. esse

programa também utiliza as funções *time*, *localtime*, e *asctime* para obter e formatar a hora atual; veja suas respectivas páginas de manual para mais informação.

Listagem B.2: (timestamp.c) Anexa uma *Timestamp* a um Arquivo

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <string.h>
4 #include <sys/stat.h>
5 #include <sys/types.h>
6 #include <time.h>
7 #include <unistd.h>
8
9 /* Retorna uma sequ ncia de caracteres representando a data e a hora atual.  */
10
11 char* get_timestamp ()
12 {
13   time_t now = time (NULL);
14   return asctime (localtime (&now));
15 }
16
17 int main (int argc, char* argv[])
18 {
19   /* O arquivo para o qual ser  anexado ao final do mesmo a data e a hora.  */
20   char* filename = argv[1];
21   /* Pega a informa  o atualizada de data e hora.  */
22   char* timestamp = get_timestamp ();
23   /* Abre o arquivo para leitura.  Se ele existir, anexa a informa  o ao final do
        mesmo;
24      de outra forma, cria um novo arquivo.  */
25   int fd = open (filename, O_WRONLY | O_CREAT | O_APPEND, 0666);
26   /* Calcula o comprimento da sequ ncia de caracteres representando a data e a hora.
        */
27   size_t length = strlen (timestamp);
28   /* Escreve a informa  o de data e hora no arquivo.  */
29   write (fd, timestamp, length);
30   /* Tudo terminado.  */
31   close (fd);
32   return 0;
33 }
```

Aqui está como o programa *timestamp* trabalha:

```
% ./timestamp tsfile
% cat tsfile
Thu Feb 1 23:25:20 2001
% ./timestamp tsfile
% cat tsfile
Thu Feb 1 23:25:20 2001
Thu Feb 1 23:25:47 2001
```

Note que a primeira vez que nós chamamos *timestamp*, foi criado o arquivo *tsfile*, enquanto a segunda vez o *timestamp* passou a adicionar entradas ao final do *tsfile*.

A chamada a *write* retorna o número de *bytes* que foram escritos agora, ou -1 se um erro tiver ocorrido. Para certos tipos de descritores de arquivo, o número de *bytes* atualmente escritos pode ser menor que o número de *bytes* requisitados. Nesse caso, cabe a você chamar *write* novamente para

escrever o resto dos dados. A função na Listagem B.3 demonstra como você pode fazer isso. Note que para algumas aplicações, você pode ter de verificar condições especiais no meio da operação de escrita. Por exemplo, se você está escrevendo para um *socket* de rede, você irá ter que ampliar essa função para detectar se a conecção de rede foi fechada no meio da operação de escrita, e se a conecção de rede tiver sido fechada, para reativar a conecção de forma apropriada.

Listagem B.3: (write-all.c) Escreve Tudo de uma Área Temporária de Armazenagem de Dados

```
1 #include <assert.h>
2 #include <unistd.h>
3
4 /* Escreve todos os COUNT bytes de BUFFER para  o descritor de arquivo FD.
5    Retorna -1 em caso de erro, ou o n mero de bytes escritos.  */
6
7 ssize_t write_all (int fd, const void* buffer, size_t count)
8 {
9   size_t left_to_write = count;
10  while (left_to_write > 0) {
11    size_t written = write (fd, buffer, count);
12    if (written == -1)
13      /* Um erro ocorreu; reclame.  */
14      return -1;
15    else
16      /* Mantenha contagem de quanto mais precisamos escrever.  */
17      left_to_write -= written;
18  }
19  /* N o devemos escrever mais que COUNT bytes!  */
20  assert (left_to_write == 0);
21  /* O n mero de bytes escritos    exatamente COUNT.  */
22  return count;
23 }
```

B.1.4 Lendo Dados

A correspondente chamada para leitura de dados é *read*. Da mesma forma que *write*, a *read* recebe um descritor de arquivo, um aontador para um espaço temporário de armazenagem, e uma contagem. A contagem especifica quantos *bytes* são lidos do descritor de arquivo para dentro do espaço temporário de armazenagem. A chamada a *read* retorna -1 em caso de erro ou o número de *bytes* lidos atualmente. Esse número de *bytes* lidos pode ser menor que o número de *bytes* requisitados, por exemplo, se não ouverem *bytes* suficientes no arquivo.

Lendo Arquivos Texto do DOS/Windows

Após ler esse livro, somos positivos no sentido de que você irá escolher escrever todos os seus programas para GNU/Linux. Todavia, seus programas podem ocasionalmente precisar ler arquivos texto gerados por programas DOS ou Windows. É importante antecipar uma diferença importante em como arquivos texto são estruturados entre essas duas plantaformas.
Em arquivos texto GNU/Linux, cada linha é separada da seguinte com um caractere de nova linha. Uma nova linha é representada por uma constante do tipo caractere '\n', a qual tem o código ASCII 10. No Windows, todavia, linhas são separadas por uma combinação de dois caracteres: um caractere de retorno de carro (o caractere '\r,' o qual tem o código ASCII 13), seguido por um caractere de nova linha.
Alguns editores de texto GNU/Linux mostram M̂ ao final de cada linha ao mostrar um arquivo de texto do Windows – esse é o caractere de retorno de carro. Emacs mostra arquivos de texto do Windows de forma adequada mas indica-os mostrando (DOS) na linha de modo na parte inferior do espaço temporário de armazenagem. Alguns editores Windows, tais como bloco de notas, mostram todo o texto de um arquivo texto GNU/Linux em uma única linha pelo fato de eles esperarem um caractere de retorno de carro ao final de cada linha. Outros programas para ambos GNU/Linux e Windows que processam arquivos texto podem reportar erros misteriosos quando se fornece a eles como entrada um arquivo texto no formato inadequado. Se seu programa lê arquivos de texto gerados por programas Windows, você irá provavelmente desejar substituir a sequência '\r\n' por um único caractere de nova linha. Similarmente, se seu programa escreve arquivos de texto que devem ser lidos por programas Windows, substitua os caracteres isolados de nova linha por combinações '\r\n'. Você deve fazer isso se você usa as chamadas de E/S de baixo nível mostradas nesse apêndice ou se usa as funções de E/S da biblioteca C GNU padrão.

A Listagem B.4 fornece uma demonstração simples de leitura. O programa imprime na tela uma remessa de hexadecimal do conteúdo do arquivo especificado na linha de comando. Entre cada linha e a seguinte temos um deslocamento de 16 *bytes*.

Listagem B.4: (hexdump.c) Mostra uma Remessa de caracteres em Hexadecimal de um Arquivo

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <sys/stat.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6
7 int main (int argc, char* argv[])
8 {
9   unsigned char buffer[16];
10  size_t offset = 0;
11  size_t bytes_read;
12  int i;
13
14  /* Abre o arquivo para leitura.  */
15  int fd = open (argv[1], O_RDONLY);
16
17  /* Leia a partir do arquivo, um peda o de cada vez.  Continue at  que a leitura
18     "comes up short", i.e. leia menos que perguntamos.  Isso
19     indica que temos que acertar o fim de arquivo.  */
20  do {
21    /* Leia a pr xima linha no valor correto dos bytes.  */
22    bytes_read = read (fd, buffer, sizeof (buffer));
23    /* Mostre o deslocamento no arquivo, seguuidos pelos bytes propriamente ditos.
        */
24    printf ("0x%06x : ", offset);
25    for (i = 0; i < bytes_read; ++i)
26      printf ("%02x ", buffer[i]);
27    printf ("\n");
28    /* Continue a contagem de nossa posi  o no arquivo.  */
29    offset += bytes_read;
30  }
31  while (bytes_read == sizeof (buffer));
32
33  /* Tudo terminado.  */
34  close (fd);
35  return 0;
36 }
```

Aqui está *hexdump* em ação. Está mostrando uma remessa de seu próprio arquivo executável:

```
% ./hexdump hexdump
0x000000 : 7f 45 4c    46  01  01  01  00  00  00  00  00  00  00  00  00
0x000010 : 02 00 03    00  01  00  00  00  c0  83  04  08  34  00  00  00
0x000020 : e8 23 00    00  00  00  00  00  34  00  20  00  06  00  28  00
0x000030 : 1d 00 1a    00  06  00  00  00  34  00  00  00  34  80  04  08
...
```

Sua saída pode ser diferente, dependendo do compilador que você usou para compilar o *hexdump* e dos sinalizadores de compilação que você especificou.

B.1.5 Movendo-se ao Longo de um Arquivo

Um descritor de arquivo lembra sua posição em um arquivo. Como você lê de ou escreve para o descritor de arquivo, sua posição avança correspondendo do número de *bytes* que você lê ou escreve. Algumas vezes, todavia, você irá precisar mover-se ao londo de um arquivo sem ler ou escrever dados. Por exemplo, você pode desejar escrever no meio de um arquivo sem modificar o início, ou você pode desejar pular de volta ao início de um arquivo e reler sem a necessidade de reabrir o arquivo.

A chamada a *lseek* habilita você a reposicionar um descritor de arquivo em um arquivo. Informe a *lseek* o descritor de arquivo e dois argumentos adicionais especificando a nova posição.

- Se o terceiro argumento for SEEK_SET, *lseek* interpreta o segundo argumento como uma posição, em *bytes*, a partir do início do arquivo.
- Se o terceiro argumento for SEEK_CUR, *lseek* interpreta o segundo argumento como uma distância, a qual pode ser positiva ou negativa, a partir da posição atual.
- Se o terceiro argumento for SEEK_END, *lseek* interpreta o segundo argumento como uma distância a partir do final do arquivo. Um valor positivo indica uma posição para além do final do arquivo.

A chamada a *lseek* retorna a nova posição, como uma distância a partir do início do arquivo. O tipo de dado da distância é *off_t*. Se um erro ocorrer, *lseek* retorna -1. Você não pode usar *lseek* com alguns tipos de descritores de arquivo, tais como descritores de arquivo de *socket*.

Se você desejar encontrar a posição de um descritor de arquivo em um arquivo sem modificá-lo, especifique uma distância 0 a partir da posição atual por exemplo:

```
off_t position = lseek (file_descriptor, 0, SEEK_CUR);
```

GNU/Linux habilita você a usar *lseek* para posicionar um descritor de arquivo para além do final do arquivo. Normalmente, se um descritor de arquivo está posicionado no final de um arquivo e você escreve para o descritor de arquivo, GNU/Linux automaticalmente expande o arquivo para dar lugar aos novos dados. Se você posiciona um descritor de arquivo para além do final de um arquivo e então escreve, GNU/Linux primeiro expande o arquivo para acomodar a "lacuna" que você criou com a operação *lseek* e então escreve até o final do arquivo. Essa lacuna, todavia, não ocupa espaço atualmente no disco; ao invés disso, GNU/Linux apenas faz uma anotação de tamanho dessa lacuna. Se você mais tarde tentar ler a partir do arquivo anotado, no arquivo anotado irá aparecer que a lacuna está preenchida com 0 *bytes*.

Usando esse comportamento de *lseek*, torna possível criar arquivos extremamente grandes que quase não ocupam nenhum espaço no disco. O programa *lseek-huge* na Listagem B.5 faz isso. O programa *lseek-huge* recebe como argumentos de linha de comando um nome de arquivo e um tamanho de arquivo alvo, em *megabytes*. O programa abre um novo arquivo, avança além do final usando *lseek*, e então escreve um único 0 *byte* antes de fechar o arquivo.

Listagem B.5: (lseek-huge.c) Cria Grandes Arquivos com *lseek*

```
1 #include <fcntl.h>
2 #include <stdlib.h>
3 #include <sys/stat.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6
7 int main (int argc, char* argv[])
8 {
9   int zero = 0;
10  const int megabyte = 1024 * 1024;
11
12  char* filename = argv[1];
13  size_t length = (size_t) atoi (argv[2]) * megabyte;
14
15  /* Abre um novo arquivo.  */
16  int fd = open (filename, O_WRONLY | O_CREAT | O_EXCL, 0666);
17  /* Pulamos para um byte antes de onde desejamos que o nome do arquivo termine.  */
18  lseek (fd, length - 1, SEEK_SET);
19  /* Escreve um  nico byte zero.  */
20  write (fd, &zero, 1);
21  /* Tudo terminado.  */
22  close (fd);
23
24  return 0;
25 }
```

Usando *lseek-huge*, iremos fazer um arquivo de 1GB (1024MB). Note que o espaço livre no acionador de disco antes e depois da operação.

```
% df -h .
Filesystem            Size Used Avail Use% Mounted on
/dev/hda5             2.9G 2.1G 655M  76%  /
% ./lseek-huge arquivogrande 1024
% ls -l arquivogrande
-rw-r-----    1 samuel   samuel  1073741824 Feb 5 16:29 arquivogrande
% df -h .
Filesystem            Size Used Avail Use% Mounted on
/dev/hda5             2.9G 2.1G 655M  76%  /
```

Não foi consumido nenhum espaço apreciável, apesar do enorme tamanho do *arquivogrande*. Ainda, se abrirmos o *arquivogrande* e fizermos uma leitura a partir dele, ele irá aparecer como sendo preenchido com 1GB de valores 0s. Por exemplo, podemos examinar seu conteúdo com o programa *hexdump* da Listagem B.4.

```
% ./hexdump bigfile | head -10
0x000000 : 00 00 00   00    00 00 00   00   00   00   00   00   00   00   00   00
0x000010 : 00 00 00   00    00 00 00   00   00   00   00   00   00   00   00   00
0x000020 : 00 00 00   00    00 00 00   00   00   00   00   00   00   00   00   00
0x000030 : 00 00 00   00    00 00 00   00   00   00   00   00   00   00   00   00
0x000040 : 00 00 00   00    00 00 00   00   00   00   00   00   00   00   00   00
0x000050 : 00 00 00   00    00 00 00   00   00   00   00   00   00   00   00   00
...
```

Se você executar esse arquivo você mesmo, irá provavelmente desejar encerrar o programa com Ctrl+C, em lugar de assistir o programa mostrar 2^{30} vezes 0 *bytes*.

Note que essas lacunas mágicas em arquivos são um recurso especial do sistema de arquivo ext2 que é tipicamene usado em discos GNU/Linux. Se você tentar usar *lseek-huge* para criar um arquivo em algum outro tipo de sistema de arquivo, tais como o *fat* ou o *vfat* usados para montar partições

DOS e Windows, irá encontrar que o arquivo resultante ocupa atualmente todo o montante de espaço do disco[3].

GNU/Linux não permite a você voltar para antes do início de um arquivo com *lseek*.

B.2 stat

Usando *open* e *read*, você pode extrair o conteúdo de um arquivo. Mas como extrair informações sobre os arquivos? Por exemplo, chamando "ls -l" mostra, para os arquivos no diretório atual, informações como tamanho do arquivo, a hora da última modificação, permissões, e o proprietário.

A chamada a *stat* obtém essa informação sobre um arquivo. Chame *stat* com o caminho para o arquivo que lhe interessa e um apontador para uma variável do tipo *struct stat*. Se a chamada a *stat* obtiver sucesso, irá retornar o valor 0 e preencher os campos da estrutura com informações sobre aquele arquivo; de outra forma, a chamada a *stat* retorna -1.

Esses são os campos mais úteis em *struct stat*:

- *st_mode* contém as permissões de acesso do arquivo. Permissões de arquivo são explicadas na Seção 10.3, "Permissões do Sistema de Arquivos".

- Adicionalmente às permissões de acesso, o campo *st_mode* codifica o tipo do arquivo em *bits* de alta ordem . Veja o texto que segue imediatamente os itens para instruções sobre como decodificar essa informação.

- *st_uid* e *st_gid* possuem os IDs de usuário e grupo, respectivamente, aos quais o arquivo pertence. Os IDs de usuários e de grupos são descritos na Seção 10.1, "Usuários e Grupos."

- *st_size* contém o tamanho do arquivo, em *bytes*.

- *st_atime* contém a última data e hora na qual o arquivo foi acessado (ou leitura ou escrita).

- *st_mtime* contém a data e a hora da última modificação.

Essa macros verificam o valor do campo *st_mode* para mostrar que tipo de arquivo sobre o qual você chamou *stat*. Uma macro avalia para *true* se o arquivo é do tipo da macro usada.

[3]Nota do tradutor: no ext4 e no reiserfs funcionou também.

- S_ISBLK (modo) dispositivo de bloco
- S_ISCHR (modo) dispositivo de caractere
- S_ISDIR (modo) diretório
- S_ISFIFO (modo) fifo (*ipe* nomeado)
- S_ISLNK (modo) link simbólico
- S_ISREG (modo) arquivo regular
- S_ISSOCK(modo) *socket*

O campo *st_dev* contém o número de dispositivo principal e secundário do dispositivo de *hardware* sobre o qual o arquivo localiza-se.Números de dispositivo são discutidos no Capítulo 6. O número de dispositivo principal corresponde aos 8 *bits* mais para a esquerda; o número de dispositivo secundário ocupa os 8 *bits* menos significativos. O campo *st_ino* contém o número do *inode* desse arquivo. Esse número de *inode* localiza o arquivo no sistema de arquivos.

Se você chama *stat* sobre um link simbólico, *stat* segue o link e você pode obter informação sobre o aruivo para o qual o link aponta, não sobre o link simbólico propriamente dito. Isso implica que S_ISLNK nunca irá ser verdadeira par o resultado de *stat*. Use a função *lstat* se você não deseja seguir links simbólicos; essa função obtém informação sobre o link propriamente dito em lugar de obter informações sobre o alvo do link. Se você chama *lstat* sobre um arquivo que não é um link simbólico, o comportamento de *lstat* é o mesmo que o de *stat*. Chamando *stat* sobre um link quebrado (um link que aponta para um alvo inexistente ou inacessível) resulta em um erro, enquanto chamar *lstat* sobre tal link não causa erro.

Se você já tem um arquivo aberto para leitura ou escrita, chame *fstat* ao invés de chamar *stat*. A *fstat* recebe um descritor de arquivo como seu primeiro argumento ao invés de um caminho.

A Listagem B.6 mostra uma função que aloca um espaço temporário de armazenagem grande o suficiente para manter o conteúdo de um arquivo e então ler o arquivo dentro do espaço temporário de armazenagem. A função usa *fstat* para determinar o tamanho do espaço temporário de armazenagem que a função precisa para fazer a alocação e também para verificar que o arquivo é realmente um arquivo regular.

Listagem B.6: (read-file.c) Lê um Arquivo para dentro de um Espaço Temporário de Armazenagem

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <sys/stat.h>
5 #include <sys/types.h>
6 #include <unistd.h>
7
8 /* L   o conte do de FILENAME para dentro de recentemente alocado espa o
      tempor rio de armazenamento.
9    O tamanho do espa o tempor rio de armazenamento    guardado em *LENGTH.  Retorna
         o espa o tempor rio de armazenamento, o qual
10   o chamador deve liberar.  Se FILENAME n o corresponder a um arquivo
11   comum, retorna NULL.  */
12
13 char* read_file (const char* filename, size_t* length)
14 {
15   int fd;
16   struct stat file_info;
17   char* buffer;
18
19   /* Abre o arquivo.  */
20   fd = open (filename, O_RDONLY);
21
22   /* Pega informa es sobre o arquivo.  */
23   fstat (fd, &file_info);
24   *length = file_info.st_size;
25   /* Garante que o arquivo seja um arquivo comum.  */
26   if (!S_ISREG (file_info.st_mode)) {
27     /* N o   , ent o desista.  */
28     close (fd);
29     return NULL;
30   }
31
32   /* Aloca um espa o tempor rio de armazenamento grande o suficiente para manter o
        conte do do arquivo.  */
33   buffer = (char*) malloc (*length);
34   /* L   o arquivo para dentro do espa o tempor rio de armazenamento.  */
35   read (fd, buffer, *length);
36
37   /* Finalize.  */
38   close (fd);
39   return buffer;
40 }
```

B.3 Leituras e Escritas de Vetor

A chamada a *write* recebe como argumentos um apontador para o início do espaço temporário de armazenagem de dados e o comprimento do referido espaço temporário de armazenagem. A chamada a *write* escreve sobre uma região contínua de memória para o descritor de arquivo. Todavia, um programa muitas vezes irá precisar escrever muitos itens de dado, cada um residindo em uma diferente parte da memória. Para usar *write*, o programa ou irá ter de copiar os itens para uma única região de memória, a qual obviamente torna uso ineficiente dos ciclos de CPU e da memória, ou irá ter que fazer multiplas chamadas a *write*.

Para algumas aplicações, multiplas chamadas a *write* são ineficientes ou indesejáveis. Por exemplo, ao escrever para um *socket* de rede, duas chamadas a *write* podem fazer com que dois pacotes sejam enviados através da rede, onde os mesmos dados poderiam ser enviados em um pacote único se

uma chamada única para *write* fosse possível. A chamada a *writev* habilita você a escrever multiplas regiões descontínuas de memória para um descritor de arquivo em uma operação única. Essa escrita de diversas regiões é chamada escrita de vetor. O custo de usar *writev* é que você deve ajustar uma estrutura de dados especificando o início e o comprimento de cada região de memória. Essa estutura de dados é um array de elementos *struct iovec*. Cada elemento especifica uma região de memória a ser escrita; os campos *iov_base* e *iov_len* especificam o endereço do início da região e o comprimento da região, respectivamente. Se você conhece previamente quantas regiões você irá precisar, você pode simplesmente declarar uma variável do tipo *struct iovec*; se o número de regiões pode variar, você deve alocar o array dinâmicamente.

Chame *writev* informando um descritor de arquivo para o qual você vai escrever, o array do tipo *struct iovec*, e o número de elementos no array. O valor de retorno é o número total de *bytes* escritos.

O programa na Listagem B.7 escreve seus argumentos de linha de comando para um arquivo usando uma única chamada a *writev*. O primeiro argumento é o nome do arquivo; o segundo e os subsequêntes argumentos são escritos para o arquivo com o nome informado no primeiro argumento, um argumento por linha. O programa aloca um array com elementos do tipo *struct iovec* que é duas vezes mais longo que o número de argumentos que o programa está escrevendo – para cada argumento o programa escreve o texto do argumento propriamente dito bem como um caractere de nova linha. Pelo fato de não sabermos o número de argumentos previamente, o vetor é alocado usando *malloc*.

Listagem B.7: (write-args.c) Escreve a Lista de Argumentos para um Arquivo com *writev*

```
1 #include <fcntl.h>
2 #include <stdlib.h>
3 #include <sys/stat.h>
4 #include <sys/types.h>
5 #include <sys/uio.h>
6 #include <unistd.h>
7
8 int main (int argc, char* argv[])
9 {
10   int fd;
11   struct iovec* vec;
12   struct iovec* vec_next;
13   int i;
14   /* Iremos precisar de um "espa o tempor rio de armazenamento" contendo um
        caractere de nova linha.  Use uma
15      vari vel comum do tipo char para isso.  */
16   char newline = '\n';
17   /* O primeiro argumento de linha de comando    o nome do arquivo de sa da.  */
18   char* filename = argv[1];
19   /* Ignore os primeiros dois elementos da lista de argumentos.  O elemento
20      zero    o nome desse programa, e o elemento um    o nome do arquivo
21      de sa da.  */
22   argc -= 2;
23   argv += 2;
24
25   /* Aloque um vetor est tico de iovec elementos.  Iremos precisar de dois para cada
26      elemento da lista de argumentos, um para o texto propriamente dito e um para
27      um caractere de nova linha.  */
28   vec = (struct iovec*) malloc (2 * argc * sizeof (struct iovec));
29
30   /* Ciclo sobre a lista de argumentos, construindo as entradas iovec.  */
31   vec_next = vec;
32   for (i = 0; i < argc; ++i) {
33     /* O primeiro elemento    o texto do argumento propriamente dito.  */
34     vec_next->iov_base = argv[i];
35     vec_next->iov_len = strlen (argv[i]);
36     ++vec_next;
37     /* O segundo elemento    um  nico caractere de nova linha.  Nesse caso
          correto se
38        multiplos elementos do vetor est tico struct iovec apontarem para a
39        mesma regi o de mem ria contendo o referido caractere de nova linha.  */
40     vec_next->iov_base = &newline;
41     vec_next->iov_len = 1;
42     ++vec_next;
43   }
44
45   /* escreve os argumentos para um arquivo.  */
46   fd = open (filename, O_WRONLY | O_CREAT);
47   writev (fd, vec, 2 * argc);
48   close (fd);
49
50   free (vec);
51   return 0;
52 }
```

Aqui está um exemplo de execução do programa *write-args*.

```
% ./write-args outputfile "primeiro arg" "segundo arg" "terceiro arg"
% cat outputfile
primeiro arg
segundo arg
terceiro arg
```

GNU/Linux fornece uma função correspondente *readv* que lê em uma operação única multiplas regiões descontínuas de memória. Similar a *writev*, um array de elementos do tipo *struct iovec* especifica as regiões de memória nas quais os dados irão ser lidos a partir do descritor de arquivo.

B.4 Relação de Funções de E/S da Biblioteca C GNU Padrão

Mencionamos anteriormente que as funções de E/S da biblioteca C GNU padrão são implementadas sobre o topo dessas funções de E/S de baixo nível. Algumas vezes, apesar disso, é conveniente usar funções da biblioteca padrão com descritores de arquivo, ou usar funções de E/S de baixo nível sobre um fluxo de biblioteca C GNU padrão do tipo FILE*. GNU/Linux habilita você a fazer ambos.

Se você tiver aberto um arquivo usando *fopen*, você pode obter o descritor de arquivo respectivo usando a função *fileno*. A função *fileno* recebe um argumento FILE* e retorna o descritor de arquivo. Por exemplo, para abrir um arquivo com a chamada *fopen* da biblioteca C GNU padrão mas escrever para esse arquivo com *writev*, você pode usar esse código:

```
FILE* fluxo = fopen (nomearquivo, "w");
int descritor_arquivo = fileno (fluxo);
writev (descritor_arquivo, vetor, comprimento_vetor);
```

Note que *fluxo* e *descritor_arquivo* correspondem ao mesmo arquivo aberto. Se você chamar a linha adiante, você não pode mais escrever para *descritor_arquivo*:

```
fclose (fluxo);
```

Similarmente, se você chama a linha adiante, você não pode mais escrever para *fluxo*:

```
close (descritor_arquivo);
```

Indo por outro caminho, de um descritor de arquivo para um fluxo, use a função *fdopen*. A função *fdopen* constrói um apontador do tipo FILE* *fluxo* correspondendo a um descritor de arquivo. A função *fdopen* recebe um argumento que é um descritor de arquivo e um argumento do tipo seq para uência de caracteres especificando o modo no qual é para ser criado o *fluxo*. A sintaxe do argumento que especifica o modo é a mesma que a do segundo argumento a *fopen*, e esse argumento que especifica o modo deve ser compatível com o descritor de arquivo. Por exemplo, especifique um modo "r" para um descritor de arquivo de leitura ou "w" para um descritor de arquivo de escrita. Da mesma forma que *fileno*, o fluxo e o descritor de arquivo devem referir-se ao mesmo arquivo aberto, de forma que se você fechar um, você não pode subsequêntemente usar o outro.

B.5 Outras Operações de Arquivo

Umas poucas outras operações sobre aruivos e diretórios são adequadas:

- *getcwd* obtém o diretório atual de trabalho. A *getcwd* recebe dois argumentos, um do tipo *char* que especifica o espaço temporário de armazenagem e o comprimento do espaço temporário de armazenagem. A *getcwd* copia o caminho do diretório de trabalho atual paa dentro do espaço temporário de armazenagem.

- *chdir* muda o diretório de trabalho atual para o caminho fornecido como seu argumento.

- *mkdir* cria um novo diretório. Seu primeiro argumento é o caminho do novo diretório. Seu segundo argumento é as permissões de acesso a serem usadas para o novo diretório. A interpretação das permissões são as mesmas que as do terceiro argumento a *open* e são modificados pela *umask* do processo.

- *rmdir* apaga um diretório. Seu argumento é o caminho do diretório.

- *unlink* apaga um arquivo. Seu argumento é o caminho para o arquivo. Essa chamada pode também ser usada para apagar outros objetos do sistema de arquivos, tais como *pipes* nomeados (veja Seção 5.4.5, "FIFOs") ou dispositivos (veja Capítulo 6). Atualmente, *unlink* não necessáriamente apaga o conteúdo do arquivo. Como mostra seu nome, a *unlink* desmancha o link para o arquivo do diretório que o contém. O arquivo não mais é listado naquele diretório, mas se qualquer processo mantiver um descritor de arquivo aberto para o arquivo, o conteúdo do arquivo não é removido do disco. Somente quando nenhum processo tiver um descritor de arquivo aberto é que o conteúdo do arquivo é apagado. De forma que, se um processo abre um arquivo para leitura ou escrita então um segundo processo chama *unlink* sobre o arquivo e cria um novo arquivo cm o mesmo nome, o primeiro processo vê o conteúdo antigo do arquivo em lugar de o conteúdo novo (a menos que o processo antigo feche o arquivo antigo e reabra-o em seguida).

- *rename* renomeia ou move um arquivo. Seus dois argumentos são o caminho antigo e o caminho novo para o arquivo. Se os caminhos estiverem em diferentes diretórios, *rename* move o arquivo, enquanto ambos estiverem no mesmo sistema de arquivo. Você pode usar *rename* para mover diretórios ou outros objetos de sistema de arquivo também.

B.6 Lendo o Conteúdo de um Diretório

GNU/Linux fornece funções para ler conteúdos de diretórios. Ambora essas funções não sejam diretamente relacionadas com as funções de E/S de baixo nível descritas nesse apêndice, vamos mostrá-las aqui de qualquer forma pelo fato de elas serem muitas vezes úteis em programas de aplicações.

Para ler o conteúdo de um diretório, siga esses passos:

1. Chame *opendir*, informando o caminho do diretório que você deseja examinar. A chamada a *opendir* retorna um controlador DIR*, o qual você irá usar para acessar o conteúdo do diretório. Se um erro ocorrer, a chamada retorna *NULL*.

2. chame *readdir* repetidamente, informando o controlador DIR* que você obteve de *opendir*. Cada vez que você chama *readdir*, a *readdir* retorna uma instância de apontador para uma estrutura do tipo *struct dirent* correspondendo à entrada seguinte do diretório. Quando você encontra o fim do conteúdo do diretório, *readdir* retorna *NULL*. A *struct dirent* que você pegou de volta a partir de *readdir* tem um campo *d_name*, o qual contém o nome da entrada de diretório.

3. Chame *closedir*, informando o controlador DIR*, para terminar a operação de listagem de diretório.

Inclua <sys/types.h> e <dirent.h> se você usa essas funções em seu programa.

Note que se você precisa do conteúdo do diretório arrumado em uma ordem em particular, você irá ter que organizá-la você mesmo.

O programa na Listagem B.8 mostra o conteúdo de um diretório. O diretório pode ser especificado na linha de comando, mas se não for especificado, o programa usa o diretório atual de trabalho. Para cada entrada no diretório, o programa mostra o tipo da entrada e seu caminho. A função *get_file_type* usa *lstat* para determinar o tipo de uma entrada do sistema de arquivo.

Listagem B.8: (listdir.c) Mostra uma Listagem de Diretórios

```
1 #include <assert.h>
2 #include <dirent.h>
3 #include <stdio.h>
4 #include <string.h>
5 #include <sys/stat.h>
6 #include <sys/types.h>
7 #include <unistd.h>
8
9 /* Retorna uma sequ ncia de caracteres que descreve o tipo de entrada do sistema de
      arquivo armaenada em PATH.  */
10
11 const char* get_file_type (const char* path)
12 {
13   struct stat st;
14   lstat (path, &st);
15   if (S_ISLNK (st.st_mode))
16     return "link simb lico";
17   else if (S_ISDIR (st.st_mode))
18     return "diret rio";
19   else if (S_ISCHR (st.st_mode))
20     return "dispositivo de caractere";
21   else if (S_ISBLK (st.st_mode))
22     return "dispositivo de bloco";
23   else if (S_ISFIFO (st.st_mode))
24     return "fifo";
25   else if (S_ISSOCK (st.st_mode))
26     return "socket";
27   else if (S_ISREG (st.st_mode))
28     return "arquivo comum";
29   else
30     /* Inesperado.  Cada entrada deve ser um dos tipos acima.  */
31     assert (0);
32 }
33
34 int main (int argc, char* argv[])
35 {
36   char* dir_path;
37   DIR* dir;
38   struct dirent* entry;
39   char entry_path[PATH_MAX + 1];
40   size_t path_len;
41
42   if (argc >= 2)
43     /* Se um diret rio for especificado na linha de comando, use-o.  */
44     dir_path = argv[1];
45   else
46     /* De outra forma, use o diret rio atual.  */
47     dir_path = ".";
48   /* Copie o caminho do diret rio para dentro de entry_path.  */
49   strncpy (entry_path, dir_path, sizeof (entry_path));
50   path_len = strlen (dir_path);
51   /* Se o caminho do diret rio  n o terminar com uma barra, coloque-a.  */
52   if (entry_path[path_len - 1] != '/') {
53     entry_path[path_len] = '/';
54     entry_path[path_len + 1] = '\0';
55     ++path_len;
56   }
57
58   /* Inicia a opera  o de escuta do diret rio especificado na
59      linha de comando.  */
60   dir = opendir (dir_path);
61   /* Ciclo sobre todas as entradas de diret rio.  */
62   while ((entry = readdir (dir)) != NULL) {
63     const char* type;
64     /* constr i o caminho para a entrada de diret rio anexando ao final o nome
65        da entrada para o nome do caminho.  */
66     strncpy (entry_path + path_len, entry->d_name,
67              sizeof (entry_path) - path_len);
68     /* Determine o tipo da entrada.  */
69     type = get_file_type (entry_path);
70     /* Mostre o tipo e caminho da entrada.  */
71     printf ("%-18s: %s\n", type, entry_path);
72   }
73
74   /* Tudo terminado.  */
75   closedir (dir);
76   return 0;
77 }
```

Aqui está as primeiras poucas linhas de saída da listagem do diretório /dev. (Sua saída pode diferir um pouco.)

```
% ./listdir /dev
directory          :   /dev/.
directory          :   /dev/..
socket             :   /dev/log
character device   :   /dev/null
regular file       :   /dev/MAKEDEV
fifo               :   /dev/initctl
character device   :   /dev/agpgart
...
```

Para verificar isso, você pode usar o comando *ls* no mesmo diretório. Especifique o sinalizador -U para instruir o comando *ls* não ordenar entradas, e especifique o sinalizador -a para fazer com que o diretório atual (.) e o diretório pai (..) sejam incluídos.

```
% ls -lUa /dev
total 124
drwxr-xr-x    7   root     root        36864   Feb 1 15:14 .
drwxr-xr-x   22   root     root         4096   Oct 11 16:39 ..
srw-rw-rw-    1   root     root            0   Dec 18 01:31 log
crw-rw-rw-    1   root     root        1,   3   May 5 1998 null
-rwxr-xr-x    1   root     root        26689   Mar 2 2000 MAKEDEV
prw-------    1   root     root            0   Dec 11 18:37 initctl
crw-rw-r--    1   root     root       10, 175   Feb 3 2000 agpgart
...
```

O primeiro caractere de cada linha na saída do comando *ls* indica o tipo de entrada.

Apêndice C

Tabela de Sinais

A TABELA C.1 LISTA ALGUNS DOS SINAIS DO GNU/LINUX QUE VOCÊ TEM MAIS CHANCE de encontrar ou usar. Note que alguns sinais possuem múltiplas interpretações, dependendo de onde eles ocorrem.

Os nomes dos sinais listados aqui são definidos como macros de préprocessador. Para usá-los em seu programa, inclua <signal.h>. As atuais definições estão em /usr/include/sys/signum.h, o qual está incluído como parte de <signal.h>.

Para uma lista completa dos sinais, incluindo uma descrição curts de cada um e o comportamento padrão quando o sinal é entregue, consulte a página de manual para sinais na Seção 7 através da seguinte chamada:

```
% man 7 signal
```

Tabela C.1: Sinais do GNU/Linux

Nome	Descrição
SIGHUP	GNU/Linux envia a um processo esse sinal quando o processo torna-se disconectado de um terminal. Muitos programas GNU/Linux usam SIGHUP para um propósito completamente diferente: para indicar a um programa que está sendo executado que esse mesmo programa deve reler seus arquivos de configuração.
SIGINT	GNU/Linux envia a um processo esse sinal quando o usuário tenta terminar o referido programa por meio de Ctrl+C.
SIGILL	Um processo recebe esse sinal quando esse mesmo processo tenta executar uma instrução ilegal. Essa tentativa por parte do processo pode indicar que a pilha do programa está corrompida.
SIGABRT	A função *abort* faz com que o processo receba esse sinal.
SIGFPE	O processo executou uma instrução inválida em ponto flutuante. Dependendo de como a CPU está configurada, uma operação inválida em ponto flutuante pode retornar um valor não numérico especial tal como *inf* (infinito) or *NaN* (Não Númeror) ao invés de SIGFPE.
SIGKILL	Esse sinal termina um processo imediatamente e não pode ser controlado.
SIGUSR1	Esse sinal é reservado para uso em programas aplicativos.
SIGUSR2	Esse sinal é reservado para uso em programas aplicativos.
SIGSEGV	O programa tentou um acesso inválido à memória. O acesso pode ser para um endereço que é inválido no espaço virtual de endereçamento de memória do processo, ou o acesso pode ser proibido pelas permissões da memória alvo. Dereferenciar um "apontador selvagem" [1]. pode causar um SIGSEGV.

Tabela C.2: Sinais do GNU/Linux - Continuação

Nome	Descrição
SIGPIPE	O programa tentou acessar um fluxo de dados interrompido, tal como uma conecção através de *socket* que tenha sido fechada pela outra parte.
SIGALRM	A chamada de sistema *alarm* agendou a entrega desse sinal em uma hora posterior. Veja a Seção 8.13, "A Chamada *setitimer*: Ajustando Intervalos em Temporizadores" no Capítulo 8, "Chamadas de Sistema do GNU/Linux" para informações sobre *setitimer*, uma versão generalizada de *alarm*.
SIGTERM	Esse sinal requisita que o processo termine. Esse é o sinal padrão enviado pelo comando *kill*.
SIGCHLD	GNU/Linux envia a um processo esse sinal quando um processo filho termina. Veja Seção 3.3.5, "Limpando Filhos de Forna Não Sincronizada" no Capítulo 3, "Processos."
SIGXCPU	GNU/Linux envia a um processo esse sinal quando esse mesmo processo excede o limite de tempo de CPU que o referido processo pode consumir. Veja Seção 8.5, "As Chamadas *getrlimit* e *setrlimit*: Limites de Recurso" no Capítulo 8 para informação sobre limite de tempo de CPU.
SIGVTALRM	A *setitimer* agendou a entrega desse sinal em um tempo futuro. Veja Seção 8.13, "A Chamada *setitimer*: Ajustando Intervalos em Temporizadores".

Veja no Apêndice K Seção K.1 para uma lista completa (até o momento) dos sinais e seus valores respectivos.

Apêndice D

Recursos Online

ESSE APÊNDICE LISTA ALGUNS LUGARES PARA VISITAR NA INTERNET para aprender mais sobre programação para sistemas GNU/Linux.

D.1 Informação Geral

- http://www.advancedlinuxprogramming.com é a casa desse livro na *Internet* . Aqui, você pode copiar para seu computador o texto completo desse livro e o código fonte dos programas, encontrar links para outros recursos *online*, e pegar mais informação sobre programação para GNU/Linux. A mesma informação pode também ser encontrada em http://www.newriders.com.

- http://www.linuxdoc.org é a casa do *Linux Documentation Project*. Esse sítio é um rpositório para uma riqueza de documentação, lista de FAQ, HOWTOs, e outras documentações sobre sistemas GNU/Linux e *software* GNU/Linux.

D.2 Informação Sobre *Software* GNU/Linux

- http://www.gnu.org é a casa do projeto GNU. Nesse sítio, você pode copiar para o seu computador um impressionante variedade de software aplicativos livres sofisticados. Entre eles está a biblioteca C GNU padrão, a qual é parte de todo sistema GNU/Linux e contém muitas das funções descritas nesse livro. O sítio do Projeto GNU também fornece informação sobre como você pode contribuir para o desenvolvimento do sistema GNU/Linux através da escrita de código ou documentação,

através do uso de *software* livre, e através da divulgação da mensagem do *software* livre.

- http://www.kernel.org é o sítio primário para distribuição do código fontr do kernel do GNU/Linux. Para os problemasmais complicados e questões mais técnicamente detalhadas sobre como GNU/Linux trabalha, o código fonte é o melhor lugar para olhar. Veja também o diretório de documentação para explanação sobre a parte interna do *kernel*.

- http://www.linuxhq.com também distribui códigos fonte do *kernel* do GNU/Linux, *patches*, e informação relacionada.

- http://gcc.gnu.org é a casa do *GNU Compiler Collection* (GCC). GCC é o compilador primário usado em sistemas GNU/Linux, e a GCC inclui compiladores para C, C++, Objective C, Java, Chill, e Fortran.

- http://www.gnome.org e http://www.kde.org são as casas dos dois mais populares ambientes gerenciadores de janela do GNU/Linux, Gnome e KDE. Se você planeja escrever uma aplicação com um interface gráfica de usuário, você deve familiarizar-se por si mesmo com qualquer dos dois ou com ambos.

D.3 Outros Sítios

- http://developer.intel.com fornece informação sobre as arquiteturas dos processadores intel, incluindo a arquitetura x86 (IA32). Se você está desenvolvendo para x86 GNU/Linux e você usar instruções assembly enbutidas, os manuais técnicos disponíveis aqui irão ser muito úteis.

- http://www.amd.com/devconn/ fornece similar informação sobre a linha de microprocessadores da AMD e seus recursos especiais.

- http://freshmeat.net está um índice de *software open source*, geralmente para GNU/Linux. Esse sítio é um dos melhores lugares para estar informado sobre as mais novas atualizações do *software* GNU/Linux, sobre os principais componentes do sistema para aplcações mais especializadas.

- http://www.linuxsecurity.com contém informação, técnicas, e links para *software* relacionado com a segurançca em GNU/Linux. O sítio é do interesse para usuários, administradores de sistema, e desenvolvedores.

Apêndice E

Open Publication License

Version 1.0

I. Requirements on Both Unmodified and Modified Versions The Open Publication works may be reproduced and distributed in whole or in part, in any medium, physical or electronic, provided that the terms of this license are adhered to and that this license or an incorporation of it by reference (with any options elected by the author(s) and/or publisher) is displayed in the reproduction. Proper form for an incorporation by reference is as follows: Copyright ¡year¿ by ¡author's name or designee¿.This material may be distributed only subject to the terms and conditions set forth in the Open Publication License, vX.Y or later (the latest version is presently available at http://www.opencontent.org/openpub/).

The reference must be immediately followed with any options elected by the author(s) or publisher of the document (see Section VI, "License Options"). Commercial redistribution of Open Publication-licensed material is permitted. Any publication in standard (paper) book form shall require the citation of the original publisher and author.The publisher and author's names shall appear on all outer surfaces of the book. On all outer surfaces of the book, the original publisher's name shall be as large as the title of the work and cited as possessive with respect to the title.

306 Appendix E Open Publication License Version 1.0

II. Copyright The copyright to each Open Publication is owned by its author(s) or designee.

III. Scope of License The following license terms apply to all Open Publication works, unless otherwise explicitly stated in the document. Mere aggregation of Open Publication works or a portion of an Open Publication work with other works or programs on the same media shall not cause this license to apply to those other works.The aggregate work shall contain a notice specifying the inclusion of the Open Publication material and appro-

priate copyright notice. Severability. If any part of this license is found to be unenforceable in any n

jurisdiction, the remaining portions of the license remain in force. No warranty. Open Publication works are licensed and provided "as is" with- n

out warranty of any kind, express or implied, including, but not limited to, the implied warranties of merchantability and fitness for a particular purpose or a warranty of noninfringement.

IV Requirements on Modified Works . All modified versions of documents covered by this license, including translations, anthologies, compilations, and partial documents, must meet the following requirements: 1. The modified version must be labeled as such. 2. The person making the modifications must be identified, and the modifications must be dated. 3. Acknowledgement of the original author and publisher, if applicable, must be retained according to normal academic citation practices. 4. The location of the original unmodified document must be identified. 5. The original author's (or authors') name(s) may not be used to assert or imply endorsement of the resulting document without the original author's (or authors') permission.

V Good-Practice Recommendations . In addition to the requirements of this license, it is requested from and strongly rec- ommended of redistributors that: 1. If you are distributing Open Publication works on hard copy or CD-ROM, you provide email notification to the authors of your intent to redistribute at least 30 days before your manuscript or media freeze, to give the authors time to provide updated documents.This notification should describe modifications, if any, made to the document.

Open Publication Policy Appendix 307

2. All substantive modifications (including deletions) be either clearly marked up in the document or else described in an attachment to the document. 3. Finally, although it is not mandatory under this license, it is considered good form to offer a free copy of any hard copy and CD-ROM expression of an Open Publication-licensed work to its author(s).

VI. License Options The author(s) or publisher of an Open Publication-licensed document may elect cer- tain options by appending language to the reference to or copy of the license.These options are considered part of the license instance and must be included with the license (or its incorporation by reference) in derived works. A. To prohibit distribution of substantively modified versions without the explicit permission of the author(s). "Substantive modification" is defined as a change to the semantic content of the document and excludes mere changes in format or typographical corrections. To accomplish this, add the phrase "Distribution of substantively modified ver- sions of this document is prohibited without the explicit permission of the copyright holder" to the license reference or copy. B. To prohibit any

publication of this work or derivative works in whole or in part in standard (paper) book form for commercial purposes is prohibited unless prior permission is obtained from the copyright holder. To accomplish this, add the phrase "Distribution of the work or derivative of the work in any standard (paper) book form is prohibited unless prior permission is obtained from the copyright holder" to the license reference or copy.

Open Publication Policy Appendix (This is not considered part of the license.) Open Publication works are available in source format via the Open Publication home page at http://works.opencontent.org/. Open Publication authors who want to include their own license on Open Publication works may do so, as long as their terms are not more restrictive than the Open Publication license. If you have questions about the Open Publication License, please contact David Wiley, or the Open Publication Authors' List at opal@opencontent.org, via email. To subscribe to the Open Publication Authors' List, send email to opal-request@opencontent.org with the word "subscribe" in the body.

308 Appendix E Open Publication License Version 1.0

To post to the Open Publication Authors' List, send email to opal@open content.org, or simply reply to a previous post. To unsubscribe from the Open Publication Authors' List, send email to opal-request@opencontent.org with the word "unsubscribe" in the body.

Apêndice F

GNU General Public License[1]

Version 2, June 1991 Copyright 1989, 1991 Free Software Foundation, Inc. 59 Temple Place-Suite 330, Boston, MA 02111-1307, USA Everyone is permitted to copy and distribute verbatim copies of this license docu- ment, but changing it is not allowed.

Preamble The licenses for most software are designed to take away your freedom to share and change it. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change free software to make sure the software is free for all its users.This General Public License applies to most of the Free Software Foundation's software and to any other program whose authors commit to using it. (Some other Free Software Foundation software is covered by the GNU Library General Public License instead.) You can apply it to your programs, too. When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for this service if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs; and that you know you can do these things.

310 Appendix F GNU/General Public License

To protect your rights, we need to make restrictions that forbid anyone to deny you these rights or to ask you to surrender the rights.These restrictions translate to certain responsibilities for you if you distribute copies of the software, or if you modify it. For example, if you distribute copies of such a program, whether gratis or for a fee, you must give the recipients all the rights that you have.You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights. We protect your rights with two steps: (1) copyright the software, and

[1]This license can also be found online at *http://www.gnu.org/copyleft/gpl.html*.

(2) offer you this license which gives you legal permission to copy, distribute and/or modify the software. Also, for each author's protection and ours, we want to make certain that everyone understands that there is no warranty for this free software. If the software is modified by someone else and passed on, we want its recipients to know that what they have is not the original, so that any problems introduced by others will not reflect on the original authors' reputations. Finally, any free program is threatened constantly by software patents.We wish to avoid the danger that redistributors of a free program will individually obtain patent licenses, in effect making the program proprietary.To prevent this, we have made it clear that any patent must be licensed for everyone's free use or not licensed at all. The precise terms and conditions for copying, distribution and modification follow.

Terms and Conditions for Copying, Distribution and Modification 0. This License applies to any program or other work which contains a notice placed by the copyright holder saying it may be distributed under the terms of this General Public License.The "Program" below, refers to any such program or work, and a "work based on the Program" means either the Program or any derivative work under copyright law: that is to say, a work containing the Program or a portion of it, either verbatim or with modifications and/or trans- lated into another language. (Hereinafter, translation is included without limita- tion in the term "modification.") Each licensee is addressed as "you."Activities other than copying, distribution and modification are not covered by this License; they are outside its scope.The act of running the Program is not restricted, and the output from the Program is covered only if its contents constitute a work based on the Program (independent of having been made by running the Program).Whether that is true depends on what the Program does. 1. You may copy and distribute verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropri- ately publish on each copy an appropriate copyright notice and disclaimer of warranty; keep intact all the notices that refer to this License and to the absence

of any warranty; and give any other recipients of the Program a copy of this License along with the Program. You may charge a fee for the physical act of transferring a copy, and you may at your option offer warranty protection in exchange for a fee. 2. You may modify your copy or copies of the Program or any portion of it, thus forming a work based on the Program, and copy and distribute such modifica- tions or work under the terms of Section 1 above, provided that you also meet all of these conditions: a) You must cause the modified files to carry prominent notices stating n

that you changed the files and the date of any change. b) You must cause any work that you distribute or publish, that in whole n

or in part contains or is derived from the Program or any part thereof, to be licensed as a whole at no charge to all third parties under the terms of this License. c) If the modified program normally reads commands interactively when n

run, you must cause it, when started running for such interactive use in the most ordinary way, to print or display an announcement including an appropriate copyright notice and a notice that there is no warranty (or else, saying that you provide a warranty) and that users may redistribute the program under these conditions, and telling the user how to view a copy of this License. (Exception: if the Program itself is interactive but does not normally print such an announcement, your work based on the Program is not required to print an announcement.)

These requirements apply to the modified work as a whole. If identifiable sec- tions of that work are not derived from the Program, and can be reasonably considered independent and separate works in themselves, then this License, and its terms, do not apply to those sections when you distribute them as separate works. But when you distribute the same sections as part of a whole which is a work based on the Program, the distribution of the whole must be on the terms of this License, whose permissions for other licensees extend to the entire whole, and thus to each and every part regardless of who wrote it. Thus, it is not the intent of this section to claim rights or contest your rights to work written entirely by you; rather, the intent is to exercise the right to control the distribution of derivative or collective works based on the Program. In addition, mere aggregation of another work not based on the Program with the Program (or with a work based on the Program) on a volume of a storage or distribution medium does not bring the other work under the scope of this License.

312 Appendix F GNU/General Public License

3. You may copy and distribute the Program (or a work based on it, under Section 2) in object code or executable form under the terms of Sections 1 and 2 above provided that you also do one of the following: a) Accompany it with the complete corresponding machine-readable n

source code, which must be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or, b) Accompany it with a written offer, valid for at least three years, to give n

any third party, for a charge no more than your cost of physically performing source distribution, a complete machine-readable copy of the corre- sponding source code, to be distributed under the terms of Sections 1 and 2 above on a medium customarily used for software interchange; or, c) Accompany it

with the information you received as to the offer to dis- n

tribute corresponding source code. (This alternative is allowed only for noncommercial distribution and only if you received the program in object code or executable form with such an offer, in accord with Subsection b above.)

The source code for a work means the preferred form of the work for making modifications to it. For an executable work, complete source code means all the source code for all modules it contains, plus any associated interface definition files, plus the scripts used to control compilation and installation of the exe- cutable. However, as a special exception, the source code distributed need not include anything that is normally distributed (in either source or binary form) with the major components (compiler, kernel, and so on) of the operating sys- tem on which the executable runs, unless that component itself accompanies the executable. If distribution of executable or object code is made by offering access to copy from a designated place, then offering equivalent access to copy the source code from the same place counts as distribution of the source code, even though third parties are not compelled to copy the source along with the object code. 4. You may not copy, modify, sublicense, or distribute the Program except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense or distribute the Program is void, and will automatically terminate your rights under this License. However, parties who have received copies, or rights, from you under this License will not have their licenses terminated so long as such parties remain in full compliance. 5. You are not required to accept this License, since you have not signed it. However, nothing else grants you permission to modify or distribute the Program or its derivative works.These actions are prohibited by law if you do not accept this License.Therefore, by modifying or distributing the Program (or any work based on the Program), you indicate your acceptance of this License to do so, and all its terms and conditions for copying, distributing or modifying the Program or works based on it.

TERMS AND CONDITIONS FOR COPYING, DISTRIBUTION AND MODIFICATION 313

6. Each time you redistribute the Program (or any work based on the Program), the recipient automatically receives a license from the original li- censor to copy, distribute or modify the Program subject to these terms and conditions.You may not impose any further restrictions on the recipients' exercise of the rights granted herein.You are not responsible for enforcing compliance by third parties to this License. 7. If, as a consequence of a court judgment or allegation of patent infringement or for any other reason (not limited to patent issues), conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the condi- tions of this

License, they do not excuse you from the conditions of this License. If you cannot distribute so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not distribute the Program at all. For example, if a patent license would not per- mit royalty-free redistribution of the Program by all those who receive copies directly or indirectly through you, then the only way you could satisfy both it and this License would be to refrain entirely from distribution of the Program. If any portion of this section is held invalid or unenforceable under any particu- lar circumstance, the balance of the section is intended to apply and the section as a whole is intended to apply in other circumstances. It is not the purpose of this section to induce you to infringe any patents or other property right claims or to contest validity of any such claims; this section has the sole purpose of protecting the integrity of the free software distribution system, which is implemented by public license practices. Many people have made generous contributions to the wide range of software distributed through that system in reliance on consistent application of that system; it is up to the author/donor to decide if he or she is willing to distribute software through any other system and a licensee cannot impose that choice. This section is intended to make thoroughly clear what is believed to be a consequence of the rest of this License. 8. If the distribution and/or use of the Program is restricted in certain countries either by patents or by copyrighted interfaces, the original copyright holder who places the Program under this License may add an explicit geographical distribu- tion limitation excluding those countries, so that distribution is permitted only in or among countries not thus excluded. In such case, this License incorporates the limitation as if written in the body of this License. 9. The Free Software Foundation may publish revised and/or new versions of the General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

314 Appendix F GNU/General Public License

Each version is given a distinguishing version number. If the Program specifies a version number of this License which applies to it and "any later version", you have the option of following the terms and conditions either of that version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of this License, you may choose any version ever published by the Free Software Foundation. 10. If you wish to incorporate parts of the Program into other free programs whose distribution conditions are different, write to the author to ask for permission. For software which is copyrighted by the Free Software Foundation, write to the Free Software Foundation; we sometimes make exceptions for this. Our decision will be guided by the two goals of preserving the free

status of all derivatives of our free software and of promoting the sharing and reuse of soft- ware generally.

No Warranty 11. BECAUSE THE PROGRAM IS LICENSED FREE OF CHARGE,THERE IS NO WARRANTY FOR THE PROGRAM,TO THE EXTENT PERMIT- TED BY APPLICABLE LAW. EXCEPT WHEN OTHER WISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS"WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO,THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE,YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION. 12. IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MAY MODIFY AND/OR REDISTRIBUTE THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

How to Apply These Terms to Your New Programs 315

End of Terms and Conditions How to Apply These Terms to Your New Programs If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms. To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively convey the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found. one line to give the program's name and an idea of what it does. Copyright yyyy name of author This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 2 of the License, or (at your option) any later version. This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Pu-

blic License for more details. You should have received a copy of the GNU General Public License along with this program; if not, write to the Free Software Foundation, Inc., 59 Temple Place-Suite 330, Boston, MA 02111-1307, USA. Also add information on how to contact you by electronic and paper mail. If the program is interactive, make it output a short notice like this when it starts in an interactive mode: Gnomovision version 69, Copyright year name of author Gnomovision comes with ABSOLUTELY NO WARRANTY; for details type 'show w'.This is free software, and you are welcome to redistribute it under certain conditions; type 'show c' for details. The hypothetical commands 'show w' and 'show c' should show the appropriate parts of the General Public License. Of course, the commands you use may be called some- thing other than 'show w' and 'show c'; they could even be mouse-clicks or menu items whatever suits your program. You should also get your employer (if you work as a programmer) or your school, if any, to sign a "copyright disclaimer" for the program, if necessary. Here is a sample; alter the names: Yoyodyne, Inc., hereby disclaims all copyright interest in the program 'Gnomovision' (which makes passes at compilers) written by James Hacker. signature of Ty Coon, 1 April 1989 Ty Coon, President of Vice

316 Appendix F GNU/General Public License

This General Public License does not permit incorporating your program into propri- etary programs. If your program is a subroutine library, you may consider it more use- ful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Library General Public License instead of this License. FSF & GNU inquiries & questions to gnu@gnu.org. Comments on these web pages to webmasters@www.gnu.org, send other questions to gnu@gnu.org. Copyright notice above. Free Software Foundation, Inc., 59 Temple Place-Suite 330, Boston, MA 02111, USA Updated: 31 Jul 2000 jonas

Apêndice G

Saídas Diversas do */proc*

Apêndice incluído pelo tradutor.

G.1 cat /proc/cpuinfo

```
processor : 0
vendor_id : GenuineIntel
cpu family : 6
model : 42
model name : Intel(R) Core(TM) i7-2600 CPU @ 3.40GHz
stepping : 7
cpu MHz : 3392.495
cache size : 8192 KB
physical id : 0
siblings : 8
core id : 0
cpu cores : 4
apicid : 0
initial apicid : 0
fdiv_bug : no
hlt_bug : no
f00f_bug : no
coma_bug : no
fpu : yes
fpu_exception : yes
cpuid level : 13
wp : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge
```

```
mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe nx
rdtscp lm constant_tsc arch_perfmon pebs bts xtopology nonstop_tsc
perfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3 cx16
xtpr pdcm sse4_1 sse4_2 x2apic popcnt aes xsave avx lahf_lm ida arat
tpr_shadow vnmi flexpriority ept vpid
bogomips : 6784.99
clflush size : 64
cache_alignment : 64
address sizes : 36 bits physical, 48 bits virtual
power management:

processor : 1
vendor_id : GenuineIntel
cpu family : 6
model : 42
model name : Intel(R) Core(TM) i7-2600 CPU @ 3.40GHz
stepping : 7
cpu MHz : 3392.495
cache size : 8192 KB
physical id : 0
siblings : 8
core id : 1
cpu cores : 4
apicid : 2
initial apicid : 2
fdiv_bug : no
hlt_bug : no
f00f_bug : no
coma_bug : no
fpu : yes
fpu_exception : yes
cpuid level : 13
wp : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge
mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe nx
rdtscp lm constant_tsc arch_perfmon pebs bts xtopology nonstop_tsc
aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3
cx16 xtpr pdcm sse4_1 sse4_2 x2apic popcnt aes xsave avx lahf_lm ida
arat tpr_shadow vnmi flexpriority ept vpid
bogomips : 6783.71
clflush size : 64
```

```
cache_alignment : 64
address sizes : 36 bits physical, 48 bits virtual
power management:

processor : 2
vendor_id : GenuineIntel
cpu family : 6
model : 42
model name : Intel(R) Core(TM) i7-2600 CPU @ 3.40GHz
stepping : 7
cpu MHz : 3392.495
cache size : 8192 KB
physical id : 0
siblings : 8
core id : 2
cpu cores : 4
apicid : 4
initial apicid : 4
fdiv_bug : no
hlt_bug : no
f00f_bug : no
coma_bug : no
fpu : yes
fpu_exception : yes
cpuid level : 13
wp : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge
mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe nx
rdtscp lm constant_tsc arch_perfmon pebs bts xtopology nonstop_tsc
aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3
cx16 xtpr pdcm sse4_1 sse4_2 x2apic popcnt aes xsave avx lahf_lm ida
arat tpr_shadow vnmi flexpriority ept vpid
bogomips : 6783.71
clflush size : 64
cache_alignment : 64
address sizes : 36 bits physical, 48 bits virtual
power management:

processor : 3
vendor_id : GenuineIntel
cpu family : 6
```

```
model : 42
model name : Intel(R) Core(TM) i7-2600 CPU @ 3.40GHz
stepping : 7
cpu MHz : 3392.495
cache size : 8192 KB
physical id : 0
siblings : 8
core id : 3
cpu cores : 4
apicid : 6
initial apicid : 6
fdiv_bug : no
hlt_bug : no
f00f_bug : no
coma_bug : no
fpu : yes
fpu_exception : yes
cpuid level : 13
wp : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge
mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe nx
rdtscp lm constant_tsc arch_perfmon pebs bts xtopology nonstop_tsc
aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3
cx16 xtpr pdcm sse4_1 sse4_2 x2apic popcnt aes xsave avx lahf_lm ida
arat tpr_shadow vnmi flexpriority ept vpid
bogomips : 6783.72
clflush size : 64
cache_alignment : 64
address sizes : 36 bits physical, 48 bits virtual
power management:

processor : 4
vendor_id : GenuineIntel
cpu family : 6
model : 42
model name : Intel(R) Core(TM) i7-2600 CPU @ 3.40GHz
stepping : 7
cpu MHz : 3392.495
cache size : 8192 KB
physical id : 0
siblings : 8
```

```
core id : 0
cpu cores : 4
apicid : 1
initial apicid : 1
fdiv_bug : no
hlt_bug : no
f00f_bug : no
coma_bug : no
fpu : yes
fpu_exception : yes
cpuid level : 13
wp : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge
mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe nx
rdtscp lm constant_tsc arch_perfmon pebs bts xtopology nonstop_tsc
aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3
cx16 xtpr pdcm sse4_1 sse4_2 x2apic popcnt aes xsave avx lahf_lm ida
arat tpr_shadow vnmi flexpriority ept vpid
bogomips : 6783.72
clflush size : 64
cache_alignment : 64
address sizes : 36 bits physical, 48 bits virtual
power management:

processor : 5
vendor_id : GenuineIntel
cpu family : 6
model : 42
model name : Intel(R) Core(TM) i7-2600 CPU @ 3.40GHz
stepping : 7
cpu MHz : 3392.495
cache size : 8192 KB
physical id : 0
siblings : 8
core id : 1
cpu cores : 4
apicid : 3
initial apicid : 3
fdiv_bug : no
hlt_bug : no
f00f_bug : no
```

```
coma_bug : no
fpu : yes
fpu_exception : yes
cpuid level : 13
wp : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge
mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe nx
rdtscp lm constant_tsc arch_perfmon pebs bts xtopology nonstop_tsc
aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3
cx16 xtpr pdcm sse4_1 sse4_2 x2apic popcnt aes xsave avx lahf_lm ida
arat tpr_shadow vnmi flexpriority ept vpid
bogomips : 6783.71
clflush size : 64
cache_alignment : 64
address sizes : 36 bits physical, 48 bits virtual
power management:

processor : 6
vendor_id : GenuineIntel
cpu family : 6
model : 42
model name : Intel(R) Core(TM) i7-2600 CPU @ 3.40GHz
stepping : 7
cpu MHz : 3392.495
cache size : 8192 KB
physical id : 0
siblings : 8
core id : 2
cpu cores : 4
apicid : 5
initial apicid : 5
fdiv_bug : no
hlt_bug : no
f00f_bug : no
coma_bug : no
fpu : yes
fpu_exception : yes
cpuid level : 13
wp : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge
mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe nx
```

```
rdtscp lm constant_tsc arch_perfmon pebs bts xtopology nonstop_tsc
aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3
cx16 xtpr pdcm sse4_1 sse4_2 x2apic popcnt aes xsave avx lahf_lm ida
arat tpr_shadow vnmi flexpriority ept vpid
bogomips : 6783.71
clflush size : 64
cache_alignment : 64
address sizes : 36 bits physical, 48 bits virtual
power management:

processor : 7
vendor_id : GenuineIntel
cpu family : 6
model : 42
model name : Intel(R) Core(TM) i7-2600 CPU @ 3.40GHz
stepping : 7
cpu MHz : 3392.495
cache size : 8192 KB
physical id : 0
siblings : 8
core id : 3
cpu cores : 4
apicid : 7
initial apicid : 7
fdiv_bug : no
hlt_bug : no
f00f_bug : no
coma_bug : no
fpu : yes
fpu_exception : yes
cpuid level : 13
wp : yes
flags : fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge
mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe nx
rdtscp lm constant_tsc arch_perfmon pebs bts xtopology nonstop_tsc
aperfmperf pni pclmulqdq dtes64 monitor ds_cpl vmx smx est tm2 ssse3
cx16 xtpr pdcm sse4_1 sse4_2 x2apic popcnt aes xsave avx lahf_lm ida
arat tpr_shadow vnmi flexpriority ept vpid
bogomips : 6783.71
clflush size : 64
cache_alignment : 64
```

```
address sizes : 36 bits physical, 48 bits virtual
power management:
```

G.2 Entradas de um Diretório de Processo

O comando "ls -l" executado dentro do diretório de um processo, mostrando seu conteúdo:

```
total 0
dr-xr-xr-x 2 root root 0 2011-07-03 07:56 attr
-r-------- 1 root root 0 2011-07-03 07:56 auxv
--w------- 1 root root 0 2011-07-03 07:56 clear_refs
-r--r--r-- 1 root root 0 2011-07-03 07:56 cmdline
-rw-r--r-- 1 root root 0 2011-07-03 07:56 comm
-rw-r--r-- 1 root root 0 2011-07-03 07:56 coredump_filter
lrwxrwxrwx 1 root root 0 2011-07-03 07:56 cwd -> /home/usu\'ario/programa/slots/4
-r-------- 1 root root 0 2011-07-03 07:56 environ
lrwxrwxrwx 1 root root 0 2011-07-03 07:56 exe -> /home/usu\'ario/programa/projects/xxx/execut\'avel
dr-x------ 2 root root 0 2011-07-03 07:56 fd
dr-x------ 2 root root 0 2011-07-03 07:56 fdinfo
-r--r--r-- 1 root root 0 2011-07-03 07:56 io
-r-------- 1 root root 0 2011-07-03 07:56 limits
-rw-r--r-- 1 root root 0 2011-07-03 07:56 loginuid
-r--r--r-- 1 root root 0 2011-07-03 07:56 maps
-rw------- 1 root root 0 2011-07-03 07:56 mem
-r--r--r-- 1 root root 0 2011-07-03 07:56 mountinfo
-r--r--r-- 1 root root 0 2011-07-03 07:56 mounts
-r-------- 1 root root 0 2011-07-03 07:56 mountstats
dr-xr-xr-x 6 root root 0 2011-07-03 07:56 net
-rw-r--r-- 1 root root 0 2011-07-03 07:56 oom_adj
-r--r--r-- 1 root root 0 2011-07-03 07:56 oom_score
-r-------- 1 root root 0 2011-07-03 07:56 pagemap
-r-------- 1 root root 0 2011-07-03 07:56 personality
lrwxrwxrwx 1 root root 0 2011-07-03 07:56 root -> /
-rw-r--r-- 1 root root 0 2011-07-03 07:56 sched
-r--r--r-- 1 root root 0 2011-07-03 07:56 sessionid
-r--r--r-- 1 root root 0 2011-07-03 07:56 smaps
-r-------- 1 root root 0 2011-07-03 07:56 stack
-r--r--r-- 1 root root 0 2011-07-03 07:49 stat
-r--r--r-- 1 root root 0 2011-07-03 07:56 statm
-r--r--r-- 1 root root 0 2011-07-03 07:56 status
-r-------- 1 root root 0 2011-07-03 07:56 syscall
dr-xr-xr-x 3 root root 0 2011-07-03 07:56 task
-r--r--r-- 1 root root 0 2011-07-03 07:56 wchan
```

G.3 cat /proc/version

Segue um outro exemplo de "*cat /proc/version*":

```
%cat /proc/version
Linux version 2.6.32-24-generic (buildd@palmer)
(gcc version 4.4.3 (Ubuntu 4.4.3-4ubuntu5) )
#43-Ubuntu SMP Thu Sep 16 14:17:33 UTC 2010
```

A saída acima indica que o sistema está executando um release 2.6.32 do *kernel* do GNU/Linux (revisão *24-generic*), que foi compilado com um release do gcc, *GNU C Compiler*, versão 4.4.3 especialmente modificado para integrar a distribuição ubuntu. Podemos ver também que o usuário que o compilou pode ser identificado pelo código *buildd@palmer* e a versão no fuso horário UTC/GMT.

```
$ cat /proc/sys/kernel/ostype
Linux
$ cat /proc/sys/kernel/osrelease
2.6.32-24-generic
$ cat /proc/sys/kernel/version
#43-Ubuntu SMP Thu Sep 16 14:17:33 UTC 2010
```

G.4 cat /proc/scsi/scsi

Segue um outro exemplo de "*cat /proc/scsi/scsi*":

```
$ cat /proc/scsi/scsi
Attached devices:
Host: scsi2 Channel: 00 Id: 00 Lun: 00
Vendor: ATA      Model: ST3160318AS      Rev: CC44
Type:   Direct-Access                    ANSI  SCSI revision: 05
Host: scsi2 Channel: 00 Id: 01 Lun: 00
Vendor: HL-DT-ST Model: DVDRAM GH22NS40  Rev: NL02
Type:   CD-ROM                           ANSI  SCSI revision: 05
Host: scsi3 Channel: 00 Id: 00 Lun: 00
Vendor: ATA      Model: ST3160318AS      Rev: CC44
Type:   Direct-Access                    ANSI  SCSI revision: 05
Host: scsi5 Channel: 00 Id: 00 Lun: 00
Vendor: Kingston Model: DataTravelerMini Rev: PMAP
Type:   Direct-Access                    ANSI  SCSI revision: 00
```

G.5 cat /proc/sys/dev/cdrom/info

Segue um outro exemplo de "*cat /proc/sys/dev/cdrom/info*":

```
CD-ROM information, Id: cdrom.c 3.20 2003/12/17

drive name: sr0
drive speed: 48
drive # of slots: 1
Can close tray: 1
Can open tray: 1
Can lock tray: 1
Can change speed: 1
Can select disk: 0
```

```
Can read multisession: 1
Can read MCN: 1
Reports media changed: 1
Can play audio: 1
Can write CD-R: 1
Can write CD-RW: 1
Can read DVD: 1
Can write DVD-R: 1
Can write DVD-RAM: 1
Can read MRW: 1
Can write MRW: 1
Can write RAM: 1
```

G.6 cat /proc/mounts

Segue um outro exemplo de "*cat /proc/mounts*":

```
rootfs / rootfs rw 0 0
none /sys sysfs rw,nosuid,nodev,noexec,relatime 0 0
none /proc proc rw,nosuid,nodev,noexec,relatime 0 0
none /dev devtmpfs rw,relatime,size=993448k,nr_inodes=199859,mode=755 0 0
none /dev/pts devpts rw,nosuid,noexec,relatime,gid=5,mode=620,ptmxmode=000 0 0
/dev/disk/by-uuid/c5d4e917-f52d-4562-b312-7a9defab403e / ext4 rw,relatime,errors=remount-ro,barrier=1,data=ordered 0 0
none /sys/fs/fuse/connections fusectl rw,relatime 0 0
none /sys/kernel/debug debugfs rw,relatime 0 0
none /sys/kernel/security securityfs rw,relatime 0 0
none /dev/shm tmpfs rw,nosuid,nodev,relatime 0 0
none /var/run tmpfs rw,nosuid,relatime,mode=755 0 0
none /var/lock tmpfs rw,nosuid,nodev,noexec,relatime 0 0
none /lib/init/rw tmpfs rw,nosuid,relatime,mode=755 0 0
/dev/sda3 /compartilhar0 ext3 rw,relatime,errors=continue,data=ordered 0 0
rpc_pipefs /var/lib/nfs/rpc_pipefs rpc_pipefs rw,relatime 0 0
nfsd /proc/fs/nfsd nfsd rw,relatime 0 0
binfmt_misc /proc/sys/fs/binfmt_misc binfmt_misc rw,nosuid,nodev,noexec,relatime 0 0
192.168.5.120:/cp0 /cp1 nfs rw,relatime,vers=3,rsize=131072,wsize=131072,namlen=255,hard,proto=tcp,timeo=600,\
retrans=2,sec=sys,mountaddr=192.168.5.120,mountvers=3,mountproto=tcp,addr=192.168.5.120 0 0
gvfs-fuse-daemon /home/aluno/.gvfs fuse.gvfs-fuse-daemon rw,nosuid,nodev,relatime,user_id=1001,group_id=1001 0 0
/dev/sdd1 /media/KINGSTON vfat rw,nosuid,nodev,relatime,uid=1001,gid=1001,fmask=0022,dmask=0077,codepage=cp437,\
iocharset=iso8859-1,shortname=mixed,utf8,flush,errors=remount-ro 0 0
```

Gostaria de chamar a atenção para a linha que inicia-se com 192.168.5.120 que mostra uma partição remota nfs e para a linha com /dev/sdd1 que mostra um pendrive.

G.7 cat /proc/locks

Segue um outro exemplo de "*cat /proc/locks*":

```
cat /proc/locks
1: POSIX  ADVISORY  WRITE 2444 08:01:1448371 1073741824 1073742335
2: POSIX  ADVISORY  READ  2444 08:01:1448361 1073741826 1073742335
3: POSIX  ADVISORY  WRITE 2444 08:01:1448351 0 EOF
4: POSIX  ADVISORY  READ  1248 00:11:5695 4 4
5: POSIX  ADVISORY  READ  1233 08:01:915898 4 4
6: POSIX  ADVISORY  READ  1233 00:11:5709 4 4
```

```
7: POSIX  ADVISORY  READ  1233 00:11:5695 4 4
8: POSIX  ADVISORY  WRITE 1233 00:11:5700 0 0
9: POSIX  ADVISORY  WRITE 1121 08:01:915815 0 EOF
10: POSIX  ADVISORY  WRITE 1135 00:11:4957 0 EOF
11: FLOCK  ADVISORY  WRITE 1111 00:11:4934 0 EOF
12: POSIX  ADVISORY  WRITE 699 00:11:3938 0 EOF
```

Gostaria de chamara atenção para a linha 11 que mostra uma entrada diferente.

Apêndice H

Adicionais ao Capítulo 8

Apêndice incluído pelo tradutor.

H.1 *strace hostname*

```
execve("/bin/hostname", ["hostname"], [/* 53 vars */]) = 0
brk(0)                                  = 0x804b000
mmap2(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xb78d5000
access("/etc/ld.so.preload", R_OK)      = -1 ENOENT (No such file or directory)
open("/lib/tls/i686/sse2/libc.so.6", O_RDONLY) = -1 ENOENT (No such file or directory)
stat64("/lib/tls/i686/sse2", 0xbf7fec20) = -1 ENOENT (No such file or directory)
open("/lib/tls/i686/libc.so.6", O_RDONLY) = -1 ENOENT (No such file or directory)
stat64("/lib/tls/i686", 0xbf7fec20)     = -1 ENOENT (No such file or directory)
open("/lib/tls/sse2/libc.so.6", O_RDONLY) = -1 ENOENT (No such file or directory)
stat64("/lib/tls/sse2", 0xbf7fec20)     = -1 ENOENT (No such file or directory)
open("/lib/tls/libc.so.6", O_RDONLY)    = -1 ENOENT (No such file or directory)
stat64("/lib/tls", 0xbf7fec20)          = -1 ENOENT (No such file or directory)
open("/lib/i686/sse2/libc.so.6", O_RDONLY) = -1 ENOENT (No such file or directory)
stat64("/lib/i686/sse2", 0xbf7fec20)    = -1 ENOENT (No such file or directory)
open("/lib/i686/libc.so.6", O_RDONLY)   = -1 ENOENT (No such file or directory)
stat64("/lib/i686", 0xbf7fec20)         = -1 ENOENT (No such file or directory)
open("/lib/sse2/libc.so.6", O_RDONLY)   = -1 ENOENT (No such file or directory)
stat64("/lib/sse2", 0xbf7fec20)         = -1 ENOENT (No such file or directory)
open("/lib/libc.so.6", O_RDONLY)        = 3
read(3, "\177ELF\1\1\1\0\0\0\0\0\0\0\0\0\3\0\3\0\1\0\0\0\3401\1\0004\0\0\0<"..., 512) = 512
fstat64(3, {st_mode=S_IFREG|0755, st_size=1649149, ...}) = 0
mmap2(NULL, 1452296, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0xb7772000
mprotect(0xb78ce000, 4096, PROT_NONE)   = 0
mmap2(0xb78cf000, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x15c) = 0xb78cf000
mmap2(0xb78d2000, 10504, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xb78d2000
close(3)                                = 0
mmap2(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xb7771000
set_thread_area({entry_number:-1 -> 6, base_addr:0xb77716c0, limit:1048575, seg_32bit:1, contents:0,
                         read_exec_only:0, limit_in_pages:1, seg_not_present:0, useable:1}) = 0
mprotect(0xb78cf000, 8192, PROT_READ)   = 0
mprotect(0xb78f3000, 4096, PROT_READ)   = 0
brk(0)                                  = 0x804b000
brk(0x806c000)                          = 0x806c000
open("/usr/lib/locale/locale-archive", O_RDONLY|O_LARGEFILE) = -1 ENOENT (No such file or directory)
open("/usr/share/locale/locale.alias", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=2570, ...}) = 0
mmap2(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xb7770000
read(3, "# Locale name alias data base.\n# "..., 4096) = 2570
read(3, ""..., 4096)                    = 0
close(3)                                = 0
munmap(0xb7770000, 4096)                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_IDENTIFICATION", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=351, ...}) = 0
mmap2(NULL, 351, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb7770000
close(3)                                = 0
open("/usr/lib/gconv/gconv-modules.cache", O_RDONLY) = -1 ENOENT (No such file or directory)
open("/usr/lib/gconv/gconv-modules", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=56028, ...}) = 0
mmap2(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xb776f000
read(3, "# GNU libc iconv configuration.\n#"..., 4096) = 4096
read(3, "B1.002//\nalias\tJS//\t\t\tJUS_I.B1.00"..., 4096) = 4096
```

```
read(3, "59-3\t1\nmodule\tINTERNAL\t\tISO-8859-"..., 4096) = 4096
read(3, "859-14//\nalias\tISO-IR-199//\t\tISO-"..., 4096) = 4096
read(3, "CDIC-DK-NO-A//\tEBCDIC-DK-NO-A\t1\n\n"..., 4096) = 4096
read(3, "\t\tIBM281//\t\tIBM281\t\t1\n\n#\tfrom\t\t\tt"..., 4096) = 4096
read(3, "\tIBM863\t\t1\n\n#\tfrom\t\t\tto\t\t\tmodule\t"..., 4096) = 4096
read(3, "//\t\tIBM937//\nalias\tCSIBM937//\t\tIB"..., 4096) = 4096
read(3, "JAPANESE//\tEUC-JP//\nalias\tOSF0003"..., 4096) = 4096
read(3, "MACINTOSH//\t\tMACINTOSH\t1\n\n#\tfrom\t"..., 4096) = 4096
read(3, "367-BOX//\nalias\tISO_10367BOX//\t\tI"..., 4096) = 4096
read(3, "EUC-JISX0213//\t\tINTERNAL\t\tEUC-JIS"..., 4096) = 4096
read(3, "/\t\tIBM1130//\nalias\tCSIBM1130//\t\tI"..., 4096) = 4096
read(3, "\t1\n\n#\tfrom\t\t\tto\t\t\tmodule\t\tcost\nal"..., 4096) = 2780
read(3, ""..., 4096)                    = 0
close(3)                                = 0
munmap(0xb776f000, 4096)                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_MEASUREMENT", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=23, ...}) = 0
mmap2(NULL, 23, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb776f000
close(3)                                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_TELEPHONE", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=51, ...}) = 0
mmap2(NULL, 51, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb776e000
close(3)                                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_ADDRESS", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=127, ...}) = 0
mmap2(NULL, 127, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb776d000
close(3)                                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_NAME", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=62, ...}) = 0
mmap2(NULL, 62, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb776c000
close(3)                                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_PAPER", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=34, ...}) = 0
mmap2(NULL, 34, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb776b000
close(3)                                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_MESSAGES", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFDIR|0755, st_size=4096, ...}) = 0
close(3)                                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_MESSAGES/SYS_LC_MESSAGES", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=54, ...}) = 0
mmap2(NULL, 54, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb776a000
close(3)                                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_MONETARY", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=290, ...}) = 0
mmap2(NULL, 290, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb7769000
close(3)                                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_COLLATE", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=1163682, ...}) = 0
mmap2(NULL, 1163682, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb764c000
close(3)                                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_TIME", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=2358, ...}) = 0
mmap2(NULL, 2358, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb764b000
close(3)                                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_NUMERIC", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=54, ...}) = 0
mmap2(NULL, 54, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb764a000
close(3)                                = 0
open("/usr/lib/locale/pt_BR.utf8/LC_CTYPE", O_RDONLY) = 3
fstat64(3, {st_mode=S_IFREG|0644, st_size=256324, ...}) = 0
mmap2(NULL, 256324, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb760b000
close(3)                                = 0
uname({sys="Linux", node="computador", ...}) = 0
fstat64(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(136, 5), ...}) = 0
mmap2(NULL, 4096, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xb760a000
write(1, "computador\n"..., 6)              = 6
exit_group(0)                           = ?
```

H.2 *sysctl*

No slackware 13.1 a página de manual para o comando *ulimit* informa que ele está obsoleto. Resta então o comando *sysctl* e as chamadas de sistema getrlimit e setrlimit para controlar os limites do sistema. Para modificar os limites do sistema no slackware 13.1 crie o /etc/sysctl.conf e coloque nele os

limites que você deseja alterar. O formato do sysctl.conf basicamente é o mesmo apresentado na saída do sysctl -a. Exemplo de sysctl.conf:

dev.cdrom.lock = 0 kernel.sysrq = 0 net.ipv4.conf.default.rp_filter = 1 net.ipv4.conf.all.rp_filter = 1 net.ipv4.ip_forward = 1 net.ipv6.conf.all.forwarding = 1 net.ipv4.icmp_echo_ignore_broadcasts = 1 net.ipv4.icmp_echo_ignore_all = 1 net.ipv4.conf.all.accept_redirects = 0 net.ipv6.conf.all.accept_redirects = 0 net.ipv4.conf.all.send_redirects = 0 net.ipv4.conf.all.accept_source_route = 0

A saída do sysctl -a:

```
% sysctl -a
kernel.sched_child_runs_first = 0
kernel.sched_min_granularity_ns = 4000000
kernel.sched_latency_ns = 20000000
kernel.sched_wakeup_granularity_ns = 4000000
kernel.sched_shares_ratelimit = 1000000
kernel.sched_tunable_scaling = 1
kernel.sched_shares_thresh = 4
kernel.sched_migration_cost = 500000
kernel.sched_nr_migrate = 32
kernel.sched_time_avg = 1000
kernel.timer_migration = 1
kernel.sched_rt_period_us = 1000000
kernel.sched_rt_runtime_us = 950000
kernel.sched_compat_yield = 0
kernel.panic = 0
kernel.core_uses_pid = 0
kernel.core_pattern = core
kernel.core_pipe_limit = 0
kernel.tainted = 1
kernel.real-root-dev = 0
kernel.print-fatal-signals = 0
kernel.ctrl-alt-del = 0
kernel.ftrace_enabled = 1
kernel.ftrace_dump_on_oops = 0
kernel.modprobe = /sbin/modprobe
kernel.modules_disabled = 0
kernel.hotplug =
kernel.acct = 4 2 30
kernel.sysrq = 1
kernel.cad_pid = 1
kernel.threads-max = 58491
kernel.random.poolsize = 4096
kernel.random.entropy_avail = 2795
kernel.random.read_wakeup_threshold = 64
kernel.random.write_wakeup_threshold = 128
kernel.random.boot_id = eff50304-4b17-4071-9083-6a72e3ba0a1b
kernel.random.uuid = 194c4e6a-88d8-45ad-876b-305e85cadba8
kernel.overflowuid = 65534
kernel.overflowgid = 65534
kernel.pid_max = 32768
kernel.panic_on_oops = 0
kernel.printk = 3 4 1 7
kernel.printk_ratelimit = 5
kernel.printk_ratelimit_burst = 10
kernel.printk_delay = 0
kernel.ngroups_max = 65536
kernel.unknown_nmi_panic = 0
kernel.nmi_watchdog = 0
kernel.panic_on_unrecovered_nmi = 0
kernel.panic_on_io_nmi = 0
kernel.bootloader_type = 2
kernel.bootloader_version = 2
kernel.kstack_depth_to_print = 24
kernel.io_delay_type = 0
kernel.randomize_va_space = 1
kernel.acpi_video_flags = 0
kernel.softlockup_panic = 0
kernel.softlockup_thresh = 60
kernel.hung_task_panic = 0
kernel.hung_task_check_count = 32768
kernel.hung_task_timeout_secs = 120
kernel.hung_task_warnings = 10
kernel.max_lock_depth = 1024
```

```
kernel.poweroff_cmd = /sbin/poweroff
kernel.keys.maxkeys = 200
kernel.keys.maxbytes = 20000
kernel.keys.root_maxkeys = 200
kernel.keys.root_maxbytes = 20000
kernel.keys.gc_delay = 300
kernel.slow-work.min-threads = 2
kernel.slow-work.max-threads = 8
kernel.slow-work.vslow-percentage = 50
kernel.perf_event_paranoid = 1
kernel.perf_event_mlock_kb = 512
kernel.perf_event_max_sample_rate = 100000
kernel.blk_iopoll = 1
kernel.sched_domain.cpu0.domain0.min_interval = 1
kernel.sched_domain.cpu0.domain0.max_interval = 2
kernel.sched_domain.cpu0.domain0.busy_idx = 0
kernel.sched_domain.cpu0.domain0.idle_idx = 0
kernel.sched_domain.cpu0.domain0.newidle_idx = 0
kernel.sched_domain.cpu0.domain0.wake_idx = 0
kernel.sched_domain.cpu0.domain0.forkexec_idx = 0
kernel.sched_domain.cpu0.domain0.busy_factor = 64
kernel.sched_domain.cpu0.domain0.imbalance_pct = 110
kernel.sched_domain.cpu0.domain0.cache_nice_tries = 0
kernel.sched_domain.cpu0.domain0.flags = 687
kernel.sched_domain.cpu0.domain0.name = SIBLING
kernel.sched_domain.cpu0.domain1.min_interval = 1
kernel.sched_domain.cpu0.domain1.max_interval = 4
kernel.sched_domain.cpu0.domain1.busy_idx = 2
kernel.sched_domain.cpu0.domain1.idle_idx = 0
kernel.sched_domain.cpu0.domain1.newidle_idx = 0
kernel.sched_domain.cpu0.domain1.wake_idx = 0
kernel.sched_domain.cpu0.domain1.forkexec_idx = 0
kernel.sched_domain.cpu0.domain1.busy_factor = 64
kernel.sched_domain.cpu0.domain1.imbalance_pct = 125
kernel.sched_domain.cpu0.domain1.cache_nice_tries = 1
kernel.sched_domain.cpu0.domain1.flags = 4655
kernel.sched_domain.cpu0.domain1.name = MC
kernel.sched_domain.cpu1.domain0.min_interval = 1
kernel.sched_domain.cpu1.domain0.max_interval = 2
kernel.sched_domain.cpu1.domain0.busy_idx = 0
kernel.sched_domain.cpu1.domain0.idle_idx = 0
kernel.sched_domain.cpu1.domain0.newidle_idx = 0
kernel.sched_domain.cpu1.domain0.wake_idx = 0
kernel.sched_domain.cpu1.domain0.forkexec_idx = 0
kernel.sched_domain.cpu1.domain0.busy_factor = 64
kernel.sched_domain.cpu1.domain0.imbalance_pct = 110
kernel.sched_domain.cpu1.domain0.cache_nice_tries = 0
kernel.sched_domain.cpu1.domain0.flags = 687
kernel.sched_domain.cpu1.domain0.name = SIBLING
kernel.sched_domain.cpu1.domain1.min_interval = 1
kernel.sched_domain.cpu1.domain1.max_interval = 4
kernel.sched_domain.cpu1.domain1.busy_idx = 2
kernel.sched_domain.cpu1.domain1.idle_idx = 0
kernel.sched_domain.cpu1.domain1.newidle_idx = 0
kernel.sched_domain.cpu1.domain1.wake_idx = 0
kernel.sched_domain.cpu1.domain1.forkexec_idx = 0
kernel.sched_domain.cpu1.domain1.busy_factor = 64
kernel.sched_domain.cpu1.domain1.imbalance_pct = 125
kernel.sched_domain.cpu1.domain1.cache_nice_tries = 1
kernel.sched_domain.cpu1.domain1.flags = 4655
kernel.sched_domain.cpu1.domain1.name = MC
kernel.sched_domain.cpu2.domain0.min_interval = 1
kernel.sched_domain.cpu2.domain0.max_interval = 2
kernel.sched_domain.cpu2.domain0.busy_idx = 0
kernel.sched_domain.cpu2.domain0.idle_idx = 0
kernel.sched_domain.cpu2.domain0.newidle_idx = 0
kernel.sched_domain.cpu2.domain0.wake_idx = 0
kernel.sched_domain.cpu2.domain0.forkexec_idx = 0
kernel.sched_domain.cpu2.domain0.busy_factor = 64
kernel.sched_domain.cpu2.domain0.imbalance_pct = 110
kernel.sched_domain.cpu2.domain0.cache_nice_tries = 0
kernel.sched_domain.cpu2.domain0.flags = 687
kernel.sched_domain.cpu2.domain0.name = SIBLING
kernel.sched_domain.cpu2.domain1.min_interval = 1
kernel.sched_domain.cpu2.domain1.max_interval = 4
kernel.sched_domain.cpu2.domain1.busy_idx = 2
kernel.sched_domain.cpu2.domain1.idle_idx = 0
kernel.sched_domain.cpu2.domain1.newidle_idx = 0
kernel.sched_domain.cpu2.domain1.wake_idx = 0
kernel.sched_domain.cpu2.domain1.forkexec_idx = 0
kernel.sched_domain.cpu2.domain1.busy_factor = 64
kernel.sched_domain.cpu2.domain1.imbalance_pct = 125
kernel.sched_domain.cpu2.domain1.cache_nice_tries = 1
```

```
kernel.sched_domain.cpu2.domain1.flags = 4655
kernel.sched_domain.cpu2.domain1.name = MC
kernel.sched_domain.cpu3.domain0.min_interval = 1
kernel.sched_domain.cpu3.domain0.max_interval = 2
kernel.sched_domain.cpu3.domain0.busy_idx = 0
kernel.sched_domain.cpu3.domain0.idle_idx = 0
kernel.sched_domain.cpu3.domain0.newidle_idx = 0
kernel.sched_domain.cpu3.domain0.wake_idx = 0
kernel.sched_domain.cpu3.domain0.forkexec_idx = 0
kernel.sched_domain.cpu3.domain0.busy_factor = 64
kernel.sched_domain.cpu3.domain0.imbalance_pct = 110
kernel.sched_domain.cpu3.domain0.cache_nice_tries = 0
kernel.sched_domain.cpu3.domain0.flags = 687
kernel.sched_domain.cpu3.domain0.name = SIBLING
kernel.sched_domain.cpu3.domain1.min_interval = 1
kernel.sched_domain.cpu3.domain1.max_interval = 4
kernel.sched_domain.cpu3.domain1.busy_idx = 2
kernel.sched_domain.cpu3.domain1.idle_idx = 0
kernel.sched_domain.cpu3.domain1.newidle_idx = 0
kernel.sched_domain.cpu3.domain1.wake_idx = 0
kernel.sched_domain.cpu3.domain1.forkexec_idx = 0
kernel.sched_domain.cpu3.domain1.busy_factor = 64
kernel.sched_domain.cpu3.domain1.imbalance_pct = 125
kernel.sched_domain.cpu3.domain1.cache_nice_tries = 1
kernel.sched_domain.cpu3.domain1.flags = 4655
kernel.sched_domain.cpu3.domain1.name = MC
kernel.sched_domain.cpu4.domain0.min_interval = 1
kernel.sched_domain.cpu4.domain0.max_interval = 2
kernel.sched_domain.cpu4.domain0.busy_idx = 0
kernel.sched_domain.cpu4.domain0.idle_idx = 0
kernel.sched_domain.cpu4.domain0.newidle_idx = 0
kernel.sched_domain.cpu4.domain0.wake_idx = 0
kernel.sched_domain.cpu4.domain0.forkexec_idx = 0
kernel.sched_domain.cpu4.domain0.busy_factor = 64
kernel.sched_domain.cpu4.domain0.imbalance_pct = 110
kernel.sched_domain.cpu4.domain0.cache_nice_tries = 0
kernel.sched_domain.cpu4.domain0.flags = 687
kernel.sched_domain.cpu4.domain0.name = SIBLING
kernel.sched_domain.cpu4.domain1.min_interval = 1
kernel.sched_domain.cpu4.domain1.max_interval = 4
kernel.sched_domain.cpu4.domain1.busy_idx = 2
kernel.sched_domain.cpu4.domain1.idle_idx = 0
kernel.sched_domain.cpu4.domain1.newidle_idx = 0
kernel.sched_domain.cpu4.domain1.wake_idx = 0
kernel.sched_domain.cpu4.domain1.forkexec_idx = 0
kernel.sched_domain.cpu4.domain1.busy_factor = 64
kernel.sched_domain.cpu4.domain1.imbalance_pct = 125
kernel.sched_domain.cpu4.domain1.cache_nice_tries = 1
kernel.sched_domain.cpu4.domain1.flags = 4655
kernel.sched_domain.cpu4.domain1.name = MC
kernel.sched_domain.cpu5.domain0.min_interval = 1
kernel.sched_domain.cpu5.domain0.max_interval = 2
kernel.sched_domain.cpu5.domain0.busy_idx = 0
kernel.sched_domain.cpu5.domain0.idle_idx = 0
kernel.sched_domain.cpu5.domain0.newidle_idx = 0
kernel.sched_domain.cpu5.domain0.wake_idx = 0
kernel.sched_domain.cpu5.domain0.forkexec_idx = 0
kernel.sched_domain.cpu5.domain0.busy_factor = 64
kernel.sched_domain.cpu5.domain0.imbalance_pct = 110
kernel.sched_domain.cpu5.domain0.cache_nice_tries = 0
kernel.sched_domain.cpu5.domain0.flags = 687
kernel.sched_domain.cpu5.domain0.name = SIBLING
kernel.sched_domain.cpu5.domain1.min_interval = 1
kernel.sched_domain.cpu5.domain1.max_interval = 4
kernel.sched_domain.cpu5.domain1.busy_idx = 2
kernel.sched_domain.cpu5.domain1.idle_idx = 0
kernel.sched_domain.cpu5.domain1.newidle_idx = 0
kernel.sched_domain.cpu5.domain1.wake_idx = 0
kernel.sched_domain.cpu5.domain1.forkexec_idx = 0
kernel.sched_domain.cpu5.domain1.busy_factor = 64
kernel.sched_domain.cpu5.domain1.imbalance_pct = 125
kernel.sched_domain.cpu5.domain1.cache_nice_tries = 1
kernel.sched_domain.cpu5.domain1.flags = 4655
kernel.sched_domain.cpu5.domain1.name = MC
kernel.sched_domain.cpu6.domain0.min_interval = 1
kernel.sched_domain.cpu6.domain0.max_interval = 2
kernel.sched_domain.cpu6.domain0.busy_idx = 0
kernel.sched_domain.cpu6.domain0.idle_idx = 0
kernel.sched_domain.cpu6.domain0.newidle_idx = 0
kernel.sched_domain.cpu6.domain0.wake_idx = 0
kernel.sched_domain.cpu6.domain0.forkexec_idx = 0
kernel.sched_domain.cpu6.domain0.busy_factor = 64
kernel.sched_domain.cpu6.domain0.imbalance_pct = 110
```

```
kernel.sched_domain.cpu6.domain0.cache_nice_tries = 0
kernel.sched_domain.cpu6.domain0.flags = 687
kernel.sched_domain.cpu6.domain0.name = SIBLING
kernel.sched_domain.cpu6.domain1.min_interval = 1
kernel.sched_domain.cpu6.domain1.max_interval = 4
kernel.sched_domain.cpu6.domain1.busy_idx = 2
kernel.sched_domain.cpu6.domain1.idle_idx = 0
kernel.sched_domain.cpu6.domain1.newidle_idx = 0
kernel.sched_domain.cpu6.domain1.wake_idx = 0
kernel.sched_domain.cpu6.domain1.forkexec_idx = 0
kernel.sched_domain.cpu6.domain1.busy_factor = 64
kernel.sched_domain.cpu6.domain1.imbalance_pct = 125
kernel.sched_domain.cpu6.domain1.cache_nice_tries = 1
kernel.sched_domain.cpu6.domain1.flags = 4655
kernel.sched_domain.cpu6.domain1.name = MC
kernel.sched_domain.cpu7.domain0.min_interval = 1
kernel.sched_domain.cpu7.domain0.max_interval = 2
kernel.sched_domain.cpu7.domain0.busy_idx = 0
kernel.sched_domain.cpu7.domain0.idle_idx = 0
kernel.sched_domain.cpu7.domain0.newidle_idx = 0
kernel.sched_domain.cpu7.domain0.wake_idx = 0
kernel.sched_domain.cpu7.domain0.forkexec_idx = 0
kernel.sched_domain.cpu7.domain0.busy_factor = 64
kernel.sched_domain.cpu7.domain0.imbalance_pct = 110
kernel.sched_domain.cpu7.domain0.cache_nice_tries = 0
kernel.sched_domain.cpu7.domain0.flags = 687
kernel.sched_domain.cpu7.domain0.name = SIBLING
kernel.sched_domain.cpu7.domain1.min_interval = 1
kernel.sched_domain.cpu7.domain1.max_interval = 4
kernel.sched_domain.cpu7.domain1.busy_idx = 2
kernel.sched_domain.cpu7.domain1.idle_idx = 0
kernel.sched_domain.cpu7.domain1.newidle_idx = 0
kernel.sched_domain.cpu7.domain1.wake_idx = 0
kernel.sched_domain.cpu7.domain1.forkexec_idx = 0
kernel.sched_domain.cpu7.domain1.busy_factor = 64
kernel.sched_domain.cpu7.domain1.imbalance_pct = 125
kernel.sched_domain.cpu7.domain1.cache_nice_tries = 1
kernel.sched_domain.cpu7.domain1.flags = 4655
kernel.sched_domain.cpu7.domain1.name = MC
kernel.ostype = Linux
kernel.osrelease = 2.6.33.4-smp
kernel.version = #2 SMP Wed May 12 22:47:36 CDT 2010
kernel.hostname = papai
kernel.domainname = (none)
kernel.shmmax = 33554432
kernel.shmall = 2097152
kernel.shmmni = 4096
kernel.msgmax = 8192
kernel.msgmni = 1678
kernel.msgmnb = 16384
kernel.sem = 250 32000 32 128
kernel.auto_msgmni = 1
kernel.pty.max = 4096
kernel.pty.nr = 16
vm.overcommit_memory = 0
vm.panic_on_oom = 0
vm.oom_kill_allocating_task = 0
vm.oom_dump_tasks = 0
vm.overcommit_ratio = 50
vm.page-cluster = 3
vm.dirty_background_ratio = 10
vm.dirty_background_bytes = 0
vm.dirty_ratio = 20
vm.dirty_bytes = 0
vm.dirty_writeback_centisecs = 500
vm.dirty_expire_centisecs = 3000
vm.nr_pdflush_threads = 0
vm.swappiness = 60
vm.lowmem_reserve_ratio = 256 32 32
vm.drop_caches = 0
vm.min_free_kbytes = 3789
vm.percpu_pagelist_fraction = 0
vm.max_map_count = 65530
vm.laptop_mode = 0
vm.block_dump = 0
vm.vfs_cache_pressure = 100
vm.legacy_va_layout = 0
vm.stat_interval = 1
vm.mmap_min_addr = 4096
vm.vdso_enabled = 2
vm.highmem_is_dirtyable = 0
vm.scan_unevictable_pages = 0
fs.inode-nr = 183514 21755
```

```
fs.inode-state = 183514 21755 0 0 0 0 0
fs.file-nr = 3328 0 374104
fs.file-max = 374104
fs.nr_open = 1048576
fs.dentry-state = 164519 156162 45 0 0 0
fs.overflowuid = 65534
fs.overflowgid = 65534
fs.leases-enable = 1
fs.dir-notify-enable = 1
fs.lease-break-time = 45
fs.aio-nr = 0
fs.aio-max-nr = 65536
fs.inotify.max_user_instances = 128
fs.inotify.max_user_watches = 8192
fs.inotify.max_queued_events = 16384
fs.epoll.max_user_watches = 338510
fs.suid_dumpable = 0
fs.quota.lookups = 0
fs.quota.drops = 0
fs.quota.reads = 0
fs.quota.writes = 0
fs.quota.cache_hits = 0
fs.quota.allocated_dquots = 0
fs.quota.free_dquots = 0
fs.quota.syncs = 2
fs.nfs.nfs_callback_tcpport = 0
fs.nfs.idmap_cache_timeout = 600
fs.nfs.nfs_mountpoint_timeout = 500
fs.nfs.nfs_congestion_kb = 61888
fs.nfs.nlm_grace_period = 0
fs.nfs.nlm_timeout = 10
fs.nfs.nlm_udpport = 0
fs.nfs.nlm_tcpport = 0
fs.nfs.nsm_use_hostnames = 0
fs.nfs.nsm_local_state = 3
fs.xfs.irix_sgid_inherit = 0
fs.xfs.irix_symlink_mode = 0
fs.xfs.panic_mask = 0
fs.xfs.error_level = 3
fs.xfs.xfssyncd_centisecs = 3000
fs.xfs.inherit_sync = 1
fs.xfs.inherit_nodump = 1
fs.xfs.inherit_noatime = 1
fs.xfs.xfsbufd_centisecs = 100
fs.xfs.age_buffer_centisecs = 1500
fs.xfs.inherit_nosymlinks = 0
fs.xfs.rotorstep = 1
fs.xfs.inherit_nodefrag = 1
fs.xfs.filestream_centisecs = 3000
fs.xfs.stats_clear = 0
fs.ocfs2.nm.hb_ctl_path = /sbin/ocfs2_hb_ctl
fs.mqueue.queues_max = 256
fs.mqueue.msg_max = 10
fs.mqueue.msgsize_max = 8192
debug.exception-trace = 1
dev.scsi.logging_level = 0
dev.raid.speed_limit_min = 1000
dev.raid.speed_limit_max = 200000
dev.hpet.max-user-freq = 64
dev.mac_hid.mouse_button_emulation = 0
dev.mac_hid.mouse_button2_keycode = 97
dev.mac_hid.mouse_button3_keycode = 100
dev.cdrom.info = CD-ROM information, Id: cdrom.c 3.20 2003/12/17
dev.cdrom.info =
dev.cdrom.info = drive name:
dev.cdrom.info = drive speed:
dev.cdrom.info = drive # of slots:
dev.cdrom.info = Can close tray:
dev.cdrom.info = Can open tray:
dev.cdrom.info = Can lock tray:
dev.cdrom.info = Can change speed:
dev.cdrom.info = Can select disk:
dev.cdrom.info = Can read multisession:
dev.cdrom.info = Can read MCN:
dev.cdrom.info = Reports media changed:
dev.cdrom.info = Can play audio:
dev.cdrom.info = Can write CD-R:
dev.cdrom.info = Can write CD-RW:
dev.cdrom.info = Can read DVD:
dev.cdrom.info = Can write DVD-R:
dev.cdrom.info = Can write DVD-RAM:
dev.cdrom.info = Can read MRW:
dev.cdrom.info = Can write MRW:
```

```
dev.cdrom.info = Can write RAM:
dev.cdrom.info =
dev.cdrom.info =
dev.cdrom.autoclose = 1
dev.cdrom.autoeject = 0
dev.cdrom.debug = 0
dev.cdrom.lock = 1
dev.cdrom.check_media = 0
dev.parport.default.timeslice = 200
dev.parport.default.spintime = 500
net.netfilter.nf_log.0 = NONE
net.netfilter.nf_log.1 = NONE
net.netfilter.nf_log.2 = ipt_LOG
net.netfilter.nf_log.3 = NONE
net.netfilter.nf_log.4 = NONE
net.netfilter.nf_log.5 = NONE
net.netfilter.nf_log.6 = NONE
net.netfilter.nf_log.7 = NONE
net.netfilter.nf_log.8 = NONE
net.netfilter.nf_log.9 = NONE
net.netfilter.nf_log.10 = NONE
net.netfilter.nf_log.11 = NONE
net.netfilter.nf_log.12 = NONE
net.netfilter.nf_conntrack_generic_timeout = 600
net.netfilter.nf_conntrack_tcp_timeout_syn_sent = 120
net.netfilter.nf_conntrack_tcp_timeout_syn_recv = 60
net.netfilter.nf_conntrack_tcp_timeout_established = 432000
net.netfilter.nf_conntrack_tcp_timeout_fin_wait = 120
net.netfilter.nf_conntrack_tcp_timeout_close_wait = 60
net.netfilter.nf_conntrack_tcp_timeout_last_ack = 30
net.netfilter.nf_conntrack_tcp_timeout_time_wait = 120
net.netfilter.nf_conntrack_tcp_timeout_close = 10
net.netfilter.nf_conntrack_tcp_timeout_max_retrans = 300
net.netfilter.nf_conntrack_tcp_timeout_unacknowledged = 300
net.netfilter.nf_conntrack_tcp_loose = 1
net.netfilter.nf_conntrack_tcp_be_liberal = 0
net.netfilter.nf_conntrack_tcp_max_retrans = 3
net.netfilter.nf_conntrack_udp_timeout = 30
net.netfilter.nf_conntrack_udp_timeout_stream = 180
net.netfilter.nf_conntrack_icmp_timeout = 30
net.netfilter.nf_conntrack_acct = 1
net.netfilter.nf_conntrack_max = 65536
net.netfilter.nf_conntrack_count = 1
net.netfilter.nf_conntrack_buckets = 16384
net.netfilter.nf_conntrack_checksum = 1
net.netfilter.nf_conntrack_log_invalid = 0
net.netfilter.nf_conntrack_expect_max = 256
net.core.somaxconn = 128
net.core.xfrm_aevent_etime = 10
net.core.xfrm_aevent_rseqth = 2
net.core.xfrm_larval_drop = 1
net.core.xfrm_acq_expires = 30
net.core.wmem_max = 131071
net.core.rmem_max = 131071
net.core.wmem_default = 112640
net.core.rmem_default = 112640
net.core.dev_weight = 64
net.core.netdev_max_backlog = 1000
net.core.message_cost = 5
net.core.message_burst = 10
net.core.optmem_max = 10240
net.core.netdev_budget = 300
net.core.warnings = 1

error: permission denied on key 'net.ipv4.route.flush'
error: permission denied on key 'net.ipv6.route.flush'

net.ipv4.route.gc_thresh = 32768
net.ipv4.route.max_size = 524288
net.ipv4.route.gc_min_interval = 0
net.ipv4.route.gc_min_interval_ms = 500
net.ipv4.route.gc_timeout = 300
net.ipv4.route.gc_interval = 60
net.ipv4.route.redirect_load = 20
net.ipv4.route.redirect_number = 9
net.ipv4.route.redirect_silence = 20480
net.ipv4.route.error_cost = 1000
net.ipv4.route.error_burst = 5000
net.ipv4.route.gc_elasticity = 8
net.ipv4.route.mtu_expires = 600
net.ipv4.route.min_pmtu = 552
net.ipv4.route.min_adv_mss = 256
net.ipv4.route.secret_interval = 600
```

```
net.ipv4.neigh.default.mcast_solicit = 3
net.ipv4.neigh.default.ucast_solicit = 3
net.ipv4.neigh.default.app_solicit = 0
net.ipv4.neigh.default.retrans_time = 99
net.ipv4.neigh.default.base_reachable_time = 30
net.ipv4.neigh.default.delay_first_probe_time = 5
net.ipv4.neigh.default.gc_stale_time = 60
net.ipv4.neigh.default.unres_qlen = 3
net.ipv4.neigh.default.proxy_qlen = 64
net.ipv4.neigh.default.anycast_delay = 99
net.ipv4.neigh.default.proxy_delay = 79
net.ipv4.neigh.default.locktime = 99
net.ipv4.neigh.default.retrans_time_ms = 1000
net.ipv4.neigh.default.base_reachable_time_ms = 30000
net.ipv4.neigh.default.gc_interval = 30
net.ipv4.neigh.default.gc_thresh1 = 128
net.ipv4.neigh.default.gc_thresh2 = 512
net.ipv4.neigh.default.gc_thresh3 = 1024
net.ipv4.neigh.lo.mcast_solicit = 3
net.ipv4.neigh.lo.ucast_solicit = 3
net.ipv4.neigh.lo.app_solicit = 0
net.ipv4.neigh.lo.retrans_time = 99
net.ipv4.neigh.lo.base_reachable_time = 30
net.ipv4.neigh.lo.delay_first_probe_time = 5
net.ipv4.neigh.lo.gc_stale_time = 60
net.ipv4.neigh.lo.unres_qlen = 3
net.ipv4.neigh.lo.proxy_qlen = 64
net.ipv4.neigh.lo.anycast_delay = 99
net.ipv4.neigh.lo.proxy_delay = 79
net.ipv4.neigh.lo.locktime = 99
net.ipv4.neigh.lo.retrans_time_ms = 1000
net.ipv4.neigh.lo.base_reachable_time_ms = 30000
net.ipv4.neigh.eth0.mcast_solicit = 3
net.ipv4.neigh.eth0.ucast_solicit = 3
net.ipv4.neigh.eth0.app_solicit = 0
net.ipv4.neigh.eth0.retrans_time = 99
net.ipv4.neigh.eth0.base_reachable_time = 30
net.ipv4.neigh.eth0.delay_first_probe_time = 5
net.ipv4.neigh.eth0.gc_stale_time = 60
net.ipv4.neigh.eth0.unres_qlen = 3
net.ipv4.neigh.eth0.proxy_qlen = 64
net.ipv4.neigh.eth0.anycast_delay = 99
net.ipv4.neigh.eth0.proxy_delay = 79
net.ipv4.neigh.eth0.locktime = 99
net.ipv4.neigh.eth0.retrans_time_ms = 1000
net.ipv4.neigh.eth0.base_reachable_time_ms = 30000
net.ipv4.neigh.vboxnet0.mcast_solicit = 3
net.ipv4.neigh.vboxnet0.ucast_solicit = 3
net.ipv4.neigh.vboxnet0.app_solicit = 0
net.ipv4.neigh.vboxnet0.retrans_time = 99
net.ipv4.neigh.vboxnet0.base_reachable_time = 30
net.ipv4.neigh.vboxnet0.delay_first_probe_time = 5
net.ipv4.neigh.vboxnet0.gc_stale_time = 60
net.ipv4.neigh.vboxnet0.unres_qlen = 3
net.ipv4.neigh.vboxnet0.proxy_qlen = 64
net.ipv4.neigh.vboxnet0.anycast_delay = 99
net.ipv4.neigh.vboxnet0.proxy_delay = 79
net.ipv4.neigh.vboxnet0.locktime = 99
net.ipv4.neigh.vboxnet0.retrans_time_ms = 1000
net.ipv4.neigh.vboxnet0.base_reachable_time_ms = 30000
net.ipv4.tcp_timestamps = 1
net.ipv4.tcp_window_scaling = 1
net.ipv4.tcp_sack = 1
net.ipv4.tcp_retrans_collapse = 1
net.ipv4.ip_default_ttl = 64
net.ipv4.ip_no_pmtu_disc = 0
net.ipv4.ip_nonlocal_bind = 0
net.ipv4.tcp_syn_retries = 5
net.ipv4.tcp_synack_retries = 5
net.ipv4.tcp_max_orphans = 32768
net.ipv4.tcp_max_tw_buckets = 180000
net.ipv4.ip_dynaddr = 0
net.ipv4.tcp_keepalive_time = 7200
net.ipv4.tcp_keepalive_probes = 9
net.ipv4.tcp_keepalive_intvl = 75
net.ipv4.tcp_retries1 = 3
net.ipv4.tcp_retries2 = 15
net.ipv4.tcp_fin_timeout = 60
net.ipv4.tcp_syncookies = 1
net.ipv4.tcp_tw_recycle = 0
net.ipv4.tcp_abort_on_overflow = 0
net.ipv4.tcp_stdurg = 0
net.ipv4.tcp_rfc1337 = 0
```

```
net.ipv4.tcp_max_syn_backlog = 1024
net.ipv4.ip_local_port_range = 32768 61000
net.ipv4.igmp_max_memberships = 20
net.ipv4.igmp_max_msf = 10
net.ipv4.inet_peer_threshold = 65664
net.ipv4.inet_peer_minttl = 120
net.ipv4.inet_peer_maxttl = 600
net.ipv4.inet_peer_gc_mintime = 10
net.ipv4.inet_peer_gc_maxtime = 120
net.ipv4.tcp_orphan_retries = 0
net.ipv4.tcp_fack = 1
net.ipv4.tcp_reordering = 3
net.ipv4.tcp_ecn = 2
net.ipv4.tcp_dsack = 1
net.ipv4.tcp_mem = 80544 107392 161088
net.ipv4.tcp_wmem = 4096 16384 3436544
net.ipv4.tcp_rmem = 4096 87380 3436544
net.ipv4.tcp_app_win = 31
net.ipv4.tcp_adv_win_scale = 2
net.ipv4.tcp_tw_reuse = 0
net.ipv4.tcp_frto = 2
net.ipv4.tcp_frto_response = 0
net.ipv4.tcp_low_latency = 0
net.ipv4.tcp_no_metrics_save = 0
net.ipv4.tcp_moderate_rcvbuf = 1
net.ipv4.tcp_tso_win_divisor = 3
net.ipv4.tcp_congestion_control = cubic
net.ipv4.tcp_abc = 0
net.ipv4.tcp_mtu_probing = 0
net.ipv4.tcp_base_mss = 512
net.ipv4.tcp_workaround_signed_windows = 0
net.ipv4.tcp_dma_copybreak = 4096
net.ipv4.tcp_slow_start_after_idle = 1
net.ipv4.tcp_available_congestion_control = cubic reno
net.ipv4.tcp_allowed_congestion_control = cubic reno
net.ipv4.tcp_max_ssthresh = 0
net.ipv4.tcp_cookie_size = 0
net.ipv4.udp_mem = 80544 107392 161088
net.ipv4.udp_rmem_min = 4096
net.ipv4.udp_wmem_min = 4096
net.ipv4.netfilter.ip_conntrack_generic_timeout = 600
net.ipv4.netfilter.ip_conntrack_tcp_timeout_syn_sent = 120
net.ipv4.netfilter.ip_conntrack_tcp_timeout_syn_sent2 = 120
net.ipv4.netfilter.ip_conntrack_tcp_timeout_syn_recv = 60
net.ipv4.netfilter.ip_conntrack_tcp_timeout_established = 432000
net.ipv4.netfilter.ip_conntrack_tcp_timeout_fin_wait = 120
net.ipv4.netfilter.ip_conntrack_tcp_timeout_close_wait = 60
net.ipv4.netfilter.ip_conntrack_tcp_timeout_last_ack = 30
net.ipv4.netfilter.ip_conntrack_tcp_timeout_time_wait = 120
net.ipv4.netfilter.ip_conntrack_tcp_timeout_close = 10
net.ipv4.netfilter.ip_conntrack_tcp_timeout_max_retrans = 300
net.ipv4.netfilter.ip_conntrack_tcp_loose = 1
net.ipv4.netfilter.ip_conntrack_tcp_be_liberal = 0
net.ipv4.netfilter.ip_conntrack_tcp_max_retrans = 3
net.ipv4.netfilter.ip_conntrack_udp_timeout = 30
net.ipv4.netfilter.ip_conntrack_udp_timeout_stream = 180
net.ipv4.netfilter.ip_conntrack_icmp_timeout = 30
net.ipv4.netfilter.ip_conntrack_max = 65536
net.ipv4.netfilter.ip_conntrack_count = 1
net.ipv4.netfilter.ip_conntrack_buckets = 16384
net.ipv4.netfilter.ip_conntrack_checksum = 1
net.ipv4.netfilter.ip_conntrack_log_invalid = 0
net.ipv4.conf.all.forwarding = 0
net.ipv4.conf.all.mc_forwarding = 0
net.ipv4.conf.all.accept_redirects = 1
net.ipv4.conf.all.secure_redirects = 1
net.ipv4.conf.all.shared_media = 1
net.ipv4.conf.all.rp_filter = 0
net.ipv4.conf.all.send_redirects = 1
net.ipv4.conf.all.accept_source_route = 0
net.ipv4.conf.all.accept_local = 0
net.ipv4.conf.all.src_valid_mark = 0
net.ipv4.conf.all.proxy_arp = 0
net.ipv4.conf.all.medium_id = 0
net.ipv4.conf.all.bootp_relay = 0
net.ipv4.conf.all.log_martians = 0
net.ipv4.conf.all.tag = 0
net.ipv4.conf.all.arp_filter = 0
net.ipv4.conf.all.arp_announce = 0
net.ipv4.conf.all.arp_ignore = 0
net.ipv4.conf.all.arp_accept = 0
net.ipv4.conf.all.arp_notify = 0
net.ipv4.conf.all.disable_xfrm = 0
```

```
net.ipv4.conf.all.disable_policy = 0
net.ipv4.conf.all.force_igmp_version = 0
net.ipv4.conf.all.promote_secondaries = 0
net.ipv4.conf.default.forwarding = 0
net.ipv4.conf.default.mc_forwarding = 0
net.ipv4.conf.default.accept_redirects = 1
net.ipv4.conf.default.secure_redirects = 1
net.ipv4.conf.default.shared_media = 1
net.ipv4.conf.default.rp_filter = 0
net.ipv4.conf.default.send_redirects = 1
net.ipv4.conf.default.accept_source_route = 1
net.ipv4.conf.default.accept_local = 0
net.ipv4.conf.default.src_valid_mark = 0
net.ipv4.conf.default.proxy_arp = 0
net.ipv4.conf.default.medium_id = 0
net.ipv4.conf.default.bootp_relay = 0
net.ipv4.conf.default.log_martians = 0
net.ipv4.conf.default.tag = 0
net.ipv4.conf.default.arp_filter = 0
net.ipv4.conf.default.arp_announce = 0
net.ipv4.conf.default.arp_ignore = 0
net.ipv4.conf.default.arp_accept = 0
net.ipv4.conf.default.arp_notify = 0
net.ipv4.conf.default.disable_xfrm = 0
net.ipv4.conf.default.disable_policy = 0
net.ipv4.conf.default.force_igmp_version = 0
net.ipv4.conf.default.promote_secondaries = 0
net.ipv4.conf.lo.forwarding = 0
net.ipv4.conf.lo.mc_forwarding = 0
net.ipv4.conf.lo.accept_redirects = 1
net.ipv4.conf.lo.secure_redirects = 1
net.ipv4.conf.lo.shared_media = 1
net.ipv4.conf.lo.rp_filter = 0
net.ipv4.conf.lo.send_redirects = 1
net.ipv4.conf.lo.accept_source_route = 1
net.ipv4.conf.lo.accept_local = 0
net.ipv4.conf.lo.src_valid_mark = 0
net.ipv4.conf.lo.proxy_arp = 0
net.ipv4.conf.lo.medium_id = 0
net.ipv4.conf.lo.bootp_relay = 0
net.ipv4.conf.lo.log_martians = 0
net.ipv4.conf.lo.tag = 0
net.ipv4.conf.lo.arp_filter = 0
net.ipv4.conf.lo.arp_announce = 0
net.ipv4.conf.lo.arp_ignore = 0
net.ipv4.conf.lo.arp_accept = 0
net.ipv4.conf.lo.arp_notify = 0
net.ipv4.conf.lo.disable_xfrm = 1
net.ipv4.conf.lo.disable_policy = 1
net.ipv4.conf.lo.force_igmp_version = 0
net.ipv4.conf.lo.promote_secondaries = 0
net.ipv4.conf.eth0.forwarding = 0
net.ipv4.conf.eth0.mc_forwarding = 0
net.ipv4.conf.eth0.accept_redirects = 1
net.ipv4.conf.eth0.secure_redirects = 1
net.ipv4.conf.eth0.shared_media = 1
net.ipv4.conf.eth0.rp_filter = 0
net.ipv4.conf.eth0.send_redirects = 1
net.ipv4.conf.eth0.accept_source_route = 1
net.ipv4.conf.eth0.accept_local = 0
net.ipv4.conf.eth0.src_valid_mark = 0
net.ipv4.conf.eth0.proxy_arp = 0
net.ipv4.conf.eth0.medium_id = 0
net.ipv4.conf.eth0.bootp_relay = 0
net.ipv4.conf.eth0.log_martians = 0
net.ipv4.conf.eth0.tag = 0
net.ipv4.conf.eth0.arp_filter = 0
net.ipv4.conf.eth0.arp_announce = 0
net.ipv4.conf.eth0.arp_ignore = 0
net.ipv4.conf.eth0.arp_accept = 0
net.ipv4.conf.eth0.arp_notify = 0
net.ipv4.conf.eth0.disable_xfrm = 0
net.ipv4.conf.eth0.disable_policy = 0
net.ipv4.conf.eth0.force_igmp_version = 0
net.ipv4.conf.eth0.promote_secondaries = 1
net.ipv4.conf.vboxnet0.forwarding = 0
net.ipv4.conf.vboxnet0.mc_forwarding = 0
net.ipv4.conf.vboxnet0.accept_redirects = 1
net.ipv4.conf.vboxnet0.secure_redirects = 1
net.ipv4.conf.vboxnet0.shared_media = 1
net.ipv4.conf.vboxnet0.rp_filter = 0
net.ipv4.conf.vboxnet0.send_redirects = 1
net.ipv4.conf.vboxnet0.accept_source_route = 1
```

```
net.ipv4.conf.vboxnet0.accept_local = 0
net.ipv4.conf.vboxnet0.src_valid_mark = 0
net.ipv4.conf.vboxnet0.proxy_arp = 0
net.ipv4.conf.vboxnet0.medium_id = 0
net.ipv4.conf.vboxnet0.bootp_relay = 0
net.ipv4.conf.vboxnet0.log_martians = 0
net.ipv4.conf.vboxnet0.tag = 0
net.ipv4.conf.vboxnet0.arp_filter = 0
net.ipv4.conf.vboxnet0.arp_announce = 0
net.ipv4.conf.vboxnet0.arp_ignore = 0
net.ipv4.conf.vboxnet0.arp_accept = 0
net.ipv4.conf.vboxnet0.arp_notify = 0
net.ipv4.conf.vboxnet0.disable_xfrm = 0
net.ipv4.conf.vboxnet0.disable_policy = 0
net.ipv4.conf.vboxnet0.force_igmp_version = 0
net.ipv4.conf.vboxnet0.promote_secondaries = 0
net.ipv4.ip_forward = 0
net.ipv4.xfrm4_gc_thresh = 262144
net.ipv4.ipfrag_high_thresh = 262144
net.ipv4.ipfrag_low_thresh = 196608
net.ipv4.ipfrag_time = 30
net.ipv4.icmp_echo_ignore_all = 0
net.ipv4.icmp_echo_ignore_broadcasts = 1
net.ipv4.icmp_ignore_bogus_error_responses = 1
net.ipv4.icmp_errors_use_inbound_ifaddr = 0
net.ipv4.icmp_ratelimit = 1000
net.ipv4.icmp_ratemask = 6168
net.ipv4.rt_cache_rebuild_count = 4
net.ipv4.ipfrag_secret_interval = 600
net.ipv4.ipfrag_max_dist = 64
net.ipv6.neigh.default.mcast_solicit = 3
net.ipv6.neigh.default.ucast_solicit = 3
net.ipv6.neigh.default.app_solicit = 0
net.ipv6.neigh.default.retrans_time = 1000
net.ipv6.neigh.default.base_reachable_time = 30
net.ipv6.neigh.default.delay_first_probe_time = 5
net.ipv6.neigh.default.gc_stale_time = 60
net.ipv6.neigh.default.unres_qlen = 3
net.ipv6.neigh.default.proxy_qlen = 64
net.ipv6.neigh.default.anycast_delay = 99
net.ipv6.neigh.default.proxy_delay = 79
net.ipv6.neigh.default.locktime = 0
net.ipv6.neigh.default.retrans_time_ms = 1000
net.ipv6.neigh.default.base_reachable_time_ms = 30000
net.ipv6.neigh.default.gc_interval = 30
net.ipv6.neigh.default.gc_thresh1 = 128
net.ipv6.neigh.default.gc_thresh2 = 512
net.ipv6.neigh.default.gc_thresh3 = 1024
net.ipv6.neigh.lo.mcast_solicit = 3
net.ipv6.neigh.lo.ucast_solicit = 3
net.ipv6.neigh.lo.app_solicit = 0
net.ipv6.neigh.lo.retrans_time = 1000
net.ipv6.neigh.lo.base_reachable_time = 30
net.ipv6.neigh.lo.delay_first_probe_time = 5
net.ipv6.neigh.lo.gc_stale_time = 60
net.ipv6.neigh.lo.unres_qlen = 3
net.ipv6.neigh.lo.proxy_qlen = 64
net.ipv6.neigh.lo.anycast_delay = 99
net.ipv6.neigh.lo.proxy_delay = 79
net.ipv6.neigh.lo.locktime = 0
net.ipv6.neigh.lo.retrans_time_ms = 1000
net.ipv6.neigh.lo.base_reachable_time_ms = 30000
net.ipv6.neigh.eth0.mcast_solicit = 3
net.ipv6.neigh.eth0.ucast_solicit = 3
net.ipv6.neigh.eth0.app_solicit = 0
net.ipv6.neigh.eth0.retrans_time = 1000
net.ipv6.neigh.eth0.base_reachable_time = 30
net.ipv6.neigh.eth0.delay_first_probe_time = 5
net.ipv6.neigh.eth0.gc_stale_time = 60
net.ipv6.neigh.eth0.unres_qlen = 3
net.ipv6.neigh.eth0.proxy_qlen = 64
net.ipv6.neigh.eth0.anycast_delay = 99
net.ipv6.neigh.eth0.proxy_delay = 79
net.ipv6.neigh.eth0.locktime = 0
net.ipv6.neigh.eth0.retrans_time_ms = 1000
net.ipv6.neigh.eth0.base_reachable_time_ms = 30000
net.ipv6.neigh.vboxnet0.mcast_solicit = 3
net.ipv6.neigh.vboxnet0.ucast_solicit = 3
net.ipv6.neigh.vboxnet0.app_solicit = 0
net.ipv6.neigh.vboxnet0.retrans_time = 1000
net.ipv6.neigh.vboxnet0.base_reachable_time = 30
net.ipv6.neigh.vboxnet0.delay_first_probe_time = 5
net.ipv6.neigh.vboxnet0.gc_stale_time = 60
```

```
net.ipv6.neigh.vboxnet0.unres_qlen = 3
net.ipv6.neigh.vboxnet0.proxy_qlen = 64
net.ipv6.neigh.vboxnet0.anycast_delay = 99
net.ipv6.neigh.vboxnet0.proxy_delay = 79
net.ipv6.neigh.vboxnet0.locktime = 0
net.ipv6.neigh.vboxnet0.retrans_time_ms = 1000
net.ipv6.neigh.vboxnet0.base_reachable_time_ms = 30000
net.ipv6.xfrm6_gc_thresh = 1024
net.ipv6.conf.all.forwarding = 0
net.ipv6.conf.all.hop_limit = 64
net.ipv6.conf.all.mtu = 1280
net.ipv6.conf.all.accept_ra = 1
net.ipv6.conf.all.accept_redirects = 1
net.ipv6.conf.all.autoconf = 1
net.ipv6.conf.all.dad_transmits = 1
net.ipv6.conf.all.router_solicitations = 3
net.ipv6.conf.all.router_solicitation_interval = 4
net.ipv6.conf.all.router_solicitation_delay = 1
net.ipv6.conf.all.force_mld_version = 0
net.ipv6.conf.all.use_tempaddr = 0
net.ipv6.conf.all.temp_valid_lft = 604800
net.ipv6.conf.all.temp_prefered_lft = 86400
net.ipv6.conf.all.regen_max_retry = 5
net.ipv6.conf.all.max_desync_factor = 600
net.ipv6.conf.all.max_addresses = 16
net.ipv6.conf.all.accept_ra_defrtr = 1
net.ipv6.conf.all.accept_ra_pinfo = 1
net.ipv6.conf.all.proxy_ndp = 0
net.ipv6.conf.all.accept_source_route = 0
net.ipv6.conf.all.disable_ipv6 = 0
net.ipv6.conf.all.accept_dad = 1
net.ipv6.conf.all.force_tllao = 0
net.ipv6.conf.default.forwarding = 0
net.ipv6.conf.default.hop_limit = 64
net.ipv6.conf.default.mtu = 1280
net.ipv6.conf.default.accept_ra = 1
net.ipv6.conf.default.accept_redirects = 1
net.ipv6.conf.default.autoconf = 1
net.ipv6.conf.default.dad_transmits = 1
net.ipv6.conf.default.router_solicitations = 3
net.ipv6.conf.default.router_solicitation_interval = 4
net.ipv6.conf.default.router_solicitation_delay = 1
net.ipv6.conf.default.force_mld_version = 0
net.ipv6.conf.default.use_tempaddr = 0
net.ipv6.conf.default.temp_valid_lft = 604800
net.ipv6.conf.default.temp_prefered_lft = 86400
net.ipv6.conf.default.regen_max_retry = 5
net.ipv6.conf.default.max_desync_factor = 600
net.ipv6.conf.default.max_addresses = 16
net.ipv6.conf.default.accept_ra_defrtr = 1
net.ipv6.conf.default.accept_ra_pinfo = 1
net.ipv6.conf.default.proxy_ndp = 0
net.ipv6.conf.default.accept_source_route = 0
net.ipv6.conf.default.disable_ipv6 = 0
net.ipv6.conf.default.accept_dad = 1
net.ipv6.conf.default.force_tllao = 0
net.ipv6.conf.lo.forwarding = 0
net.ipv6.conf.lo.hop_limit = 64
net.ipv6.conf.lo.mtu = 16436
net.ipv6.conf.lo.accept_ra = 1
net.ipv6.conf.lo.accept_redirects = 1
net.ipv6.conf.lo.autoconf = 1
net.ipv6.conf.lo.dad_transmits = 1
net.ipv6.conf.lo.router_solicitations = 3
net.ipv6.conf.lo.router_solicitation_interval = 4
net.ipv6.conf.lo.router_solicitation_delay = 1
net.ipv6.conf.lo.force_mld_version = 0
net.ipv6.conf.lo.use_tempaddr = -1
net.ipv6.conf.lo.temp_valid_lft = 604800
net.ipv6.conf.lo.temp_prefered_lft = 86400
net.ipv6.conf.lo.regen_max_retry = 5
net.ipv6.conf.lo.max_desync_factor = 600
net.ipv6.conf.lo.max_addresses = 16
net.ipv6.conf.lo.accept_ra_defrtr = 1
net.ipv6.conf.lo.accept_ra_pinfo = 1
net.ipv6.conf.lo.proxy_ndp = 0
net.ipv6.conf.lo.accept_source_route = 0
net.ipv6.conf.lo.disable_ipv6 = 0
net.ipv6.conf.lo.accept_dad = -1
net.ipv6.conf.lo.force_tllao = 0
net.ipv6.conf.eth0.forwarding = 0
net.ipv6.conf.eth0.hop_limit = 64
net.ipv6.conf.eth0.mtu = 1500
```

```
net.ipv6.conf.eth0.accept_ra = 1
net.ipv6.conf.eth0.accept_redirects = 1
net.ipv6.conf.eth0.autoconf = 1
net.ipv6.conf.eth0.dad_transmits = 1
net.ipv6.conf.eth0.router_solicitations = 3
net.ipv6.conf.eth0.router_solicitation_interval = 4
net.ipv6.conf.eth0.router_solicitation_delay = 1
net.ipv6.conf.eth0.force_mld_version = 0
net.ipv6.conf.eth0.use_tempaddr = 0
net.ipv6.conf.eth0.temp_valid_lft = 604800
net.ipv6.conf.eth0.temp_prefered_lft = 86400
net.ipv6.conf.eth0.regen_max_retry = 5
net.ipv6.conf.eth0.max_desync_factor = 600
net.ipv6.conf.eth0.max_addresses = 16
net.ipv6.conf.eth0.accept_ra_defrtr = 1
net.ipv6.conf.eth0.accept_ra_pinfo = 1
net.ipv6.conf.eth0.proxy_ndp = 0
net.ipv6.conf.eth0.accept_source_route = 0
net.ipv6.conf.eth0.disable_ipv6 = 0
net.ipv6.conf.eth0.accept_dad = 1
net.ipv6.conf.eth0.force_tllao = 0
net.ipv6.conf.vboxnet0.forwarding = 0
net.ipv6.conf.vboxnet0.hop_limit = 64
net.ipv6.conf.vboxnet0.mtu = 1500
net.ipv6.conf.vboxnet0.accept_ra = 1
net.ipv6.conf.vboxnet0.accept_redirects = 1
net.ipv6.conf.vboxnet0.autoconf = 1
net.ipv6.conf.vboxnet0.dad_transmits = 1
net.ipv6.conf.vboxnet0.router_solicitations = 3
net.ipv6.conf.vboxnet0.router_solicitation_interval = 4
net.ipv6.conf.vboxnet0.router_solicitation_delay = 1
net.ipv6.conf.vboxnet0.force_mld_version = 0
net.ipv6.conf.vboxnet0.use_tempaddr = 0
net.ipv6.conf.vboxnet0.temp_valid_lft = 604800
net.ipv6.conf.vboxnet0.temp_prefered_lft = 86400
net.ipv6.conf.vboxnet0.regen_max_retry = 5
net.ipv6.conf.vboxnet0.max_desync_factor = 600
net.ipv6.conf.vboxnet0.max_addresses = 16
net.ipv6.conf.vboxnet0.accept_ra_defrtr = 1
net.ipv6.conf.vboxnet0.accept_ra_pinfo = 1
net.ipv6.conf.vboxnet0.proxy_ndp = 0
net.ipv6.conf.vboxnet0.accept_source_route = 0
net.ipv6.conf.vboxnet0.disable_ipv6 = 0
net.ipv6.conf.vboxnet0.accept_dad = 1
net.ipv6.conf.vboxnet0.force_tllao = 0
net.ipv6.ip6frag_high_thresh = 262144
net.ipv6.ip6frag_low_thresh = 196608
net.ipv6.ip6frag_time = 60
net.ipv6.route.gc_thresh = 1024
net.ipv6.route.max_size = 4096
net.ipv6.route.gc_min_interval = 0
net.ipv6.route.gc_timeout = 60
net.ipv6.route.gc_interval = 30
net.ipv6.route.gc_elasticity = 0
net.ipv6.route.mtu_expires = 600
net.ipv6.route.min_adv_mss = 1
net.ipv6.route.gc_min_interval_ms = 500
net.ipv6.icmp.ratelimit = 1000
net.ipv6.bindv6only = 0
net.ipv6.ip6frag_secret_interval = 600
net.ipv6.mld_max_msf = 64
net.nf_conntrack_max = 65536
net.unix.max_dgram_qlen = 10
sunrpc.rpc_debug = 0
sunrpc.nfs_debug = 0
sunrpc.nfsd_debug = 0
sunrpc.nlm_debug = 0
sunrpc.transports = tcp 1048576
sunrpc.transports = udp 32768
sunrpc.udp_slot_table_entries = 16
sunrpc.tcp_slot_table_entries = 16
sunrpc.min_resvport = 665
sunrpc.max_resvport = 1023
sunrpc.tcp_fin_timeout = 15
crypto.fips_enabled = 0
```

H.3 Ano de 1970

Fonte: http://www.casadovideo.com/fatosedatas/livro.htm

. Andre Agassi, tenista; nascimento, em 29-04-1970

. Argentina; presidente Ongania \'e derrubado por um golpe militar, em 08-06-1970
. Bertrand Russel; morte, em 02-02-1970
. Bol\'ivia; golpe militar leva ao poder o general nacionalista Juan Jos\'e Torres, em 07-10-1970
. Censura pr\'evia a livros e peri\'{o}dicos \'e institu\'ida por decreto-lei aprovado pela C\^amara, em 13-01-1970
. Charles De Gaulle; morte, em 09-11-1970
. Chile; Salvador Allende \'e eleito presidente e tenta implementar "via pac\'ifica para o socialismo", em 24-10-1970
. Elei\c{c}\~oes parlamentares; Arena faz 223 deputados, contra 87, do MDB, em 15-11-1970
. Futebol; Brasil conquista o tricampeonato mundial no M\'exico; a ditadura explora a vit\'{o}ria, em 21-06-1970
. Gabriela Sabatini, tenista argentina; nascimento, em 16-05-1970
. Guerrilha; avi\~ao da Cruzeiro \'e seq\"uestrado, em 01-01-1970
. Guerrilha; c\^onsul do Jap\~ao em SP \'e seq\"uestrado e depois trocado pela liberta\c{c}\~ao de 5 presos pol\'iticos, em 11-03-1970
. Guerrilha; noticiado o cerco a Lamarca no Vale do Ribeira, SP, por 5 mil soldados, com a fuga dos guerrilheiros escapam, em 18-04-1970
. Guerrilha; seq\"uestrado um boeing 737 da Vasp, em 25-04-1970
. Guerrilha; embaixador alem\~ao \'e seq\"uestrado no RJ e depois trocado pela liberta\c{c}\~ao de 40 presos pol\'iticos, em 11-06-1970
. Guerrilha; c\^onsul brasileiro, A. Gomide, \'e seq\"uestrado em Montevid\'eu por guerrilheiros Tupamaros, em 31-07-1970
. Guerrilha; Joaquim da C\^amara Ferreira, o "Velho", dirigente da ALN, \'e preso em SP e morto horas depois, em 23-10-1970
. Guerrilha; a VPR seq\"uestra o embaixador su\'i\c{c}o no RJ (depois, trocado pela liberta\c{c}\~ao de 70 presos pol\'iticos), em 07-12-1970
. Guerrilha; Eduardo Leite, o Bacuri, da VPR, morre sob tortura no Dops-SP, em 08-12-1970
. Incra - Instituto Nacional de Coloniza\c{c}\~ao e Reforma Agr\'aria; cria\c{c}\~ao, em 09-07-1970
. Jaime Oncins, tenista; nascimento, em 16-06-1970
. Janis Joplin, cantora norte-americana; morte, em 04-10-1970
. Jimi Hendrix (James Marshall Hendrix), guitarrista e cantor norte-americano; \'ultimo concerto, em 04-09-1970
. Jimi Hendrix; morte, em 18-09-1970
. M\'ario Alves, l\'ider do PCBR, \'e morto sob tortura no primeiro Batalh\~ao da Pol\'icia do Ex\'ercito, no RJ, em 16-01-1970
. M\'edici diz em Porto Alegre que "o homem n\~ao foi feito para a democracia", em 08-10-1970
. Moeda; cruzeiro novo volta a chamar-se cruzeiro, em 15-05-1970
. O Pasquim; governo M\'edici manda prender toda a "turma do Pasquim", em 04-11-1970
. OEA recebe den\'uncia de torturas no Brasil da Comiss\~ao Internacional de Juristas, de Genebra, em 24-07-1970
. Olavo Hansen, oper\'ario, \'e preso em SP e encontrado morto uma semana depois, em 01-05-1970
. Oscarito, um dos principais atores das chanchadas; morte, em 04-08-1970
. Paquist\~ao; enchentes deixam o saldo de 300 mil v\'itimas, em 13-11-1970
. Paulo Evaristo Arns assume a arquidiocese de S\~ao Paulo, em 22-10-1970
. Portugal; golpe derruba a Primeira Rep\'ublica e leva Salazar ao poder (at\'e a morte, em 1970), em 28-05-1926
. Recenseamento; o Brasil tem 93.139.037 habitantes, em 01-09-1970
. Rodovia Transamaz\^onica; in\'icio da constru\c{c}\~ao, em 09-10-1970
. Salazar (Ant\^onio Carlos de Oliveira), ditador portugu\^es; morte, em 27-07-1970
. Saques e invas\~oes no CE, RN, PE, PI e PB s\~ao iniciados por flagelados da seca, em 08-04-1970
. Televis\~ao; inaugura\c{c}\~ao da TV Gazeta, em 25-01-1970
. Televis\~ao; encerramento das transmiss\~oes da TV Excelsior, em 30-09-1970

Apêndice I

Assembly

Apêndice incluído pelo tradutor.

No slackware temos o as, o yasm e o nasm.

I.1 Alô Mundo

Usando:

```
gcc -O2 -S bit-pos-loop.c
```

Você consegue:

```
.file "bit-pos-loop.c"
.text
.p2align 4,,15
.globl main
.type main, @function
main:
.LFB18:
.cfi_startproc
subq $24, %rsp
.cfi_def_cfa_offset 32
movq 8(%rsi), %rdi
movl $10, %edx
xorl %esi, %esi
call strtol
movslq %eax, %rsi
xorl %edx, %edx
movl $1, %ecx
testq %rsi, %rsi
jle .L3
addq $1, %rcx
movl %edx, 12(%rsp)
cmpq %rcx, %rsi
jl .L3
.p2align 4,,10
.p2align 3
.L6:
movq %rcx, %rax
xorl %edx, %edx
sarq %rax
.p2align 4,,10
.p2align 3
.L4:
addl $1, %edx
sarq %rax
jne .L4
addq $1, %rcx
movl %edx, 12(%rsp)
cmpq %rcx, %rsi
jge .L6
.L3:
xorl %eax, %eax
addq $24, %rsp
```

```
.cfi_def_cfa_offset 8
ret
.cfi_endproc
.LFE18:
.size main, .-main
.ident "GCC: (GNU) 4.5.2"
.section .note.GNU-stack,"",@progbits
```

Seja o nosso tradicional programa *alo.c*:

```
#include <stdio.h>
main()
{
printf("Al\^o Mundo!\n");
}
```

Usando o comando:

```
gcc -S -masm=intel -mtune=native alo.c
```

Teremos a seguinte saída em *assembly x86*:

```
.file "alo.c"
.intel_syntax noprefix
.section .rodata
.LC0:
.string "Al\303\264 Mundo!"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
push rbp
.cfi_def_cfa_offset 16
mov rbp, rsp
.cfi_offset 6, -16
.cfi_def_cfa_register 6
mov edi, OFFSET FLAT:.LC0
call puts
leave
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (GNU) 4.5.2"
.section .note.GNU-stack,"",@progbits
```

Usando o comando abaixo gera-se o executável *a.out*:

```
gcc alo.s
```

I.2 bsrl

bsr/bsf - bit scan forward e bit scan reverse - bsrl - sintaxe AT & T

using as the GNU Assembler - versão 2.14 - Dean Elsner, Jay Fenlason & friends , p 128

Em http://sourceware.org/binutils/docs/as/i386_002dSyntax.html#i386_002dSyntax temos:

```
In AT&T syntax the size of memory operands is determined from the
last character of the instruction mnemonic. Mnemonic suffixes
of ‘b’, ‘w’, ‘l’ and ‘q’ specify byte (8-bit), word (16-bit),
long (32-bit) and quadruple word (64-bit) memory references.
Intel syntax accomplishes this by prefixing memory operands
(not the instruction mnemonics) with ‘byte ptr’, ‘word ptr’,
‘dword ptr’ and ‘qword ptr’. Thus, Intel ‘mov al, byte ptr foo’ is
‘movb foo, %al’ in AT&T syntax.
```

Na http://linuxgazette.net/94/ramankutty.html temos:

From C To Assembly Language
By Hiran Ramankutty

Apêndice J

Segurança

Apêndice incluído pelo tradutor.

J.1 Setuid no Debian 6.0.2

```
mostra setuid ativado em usu\'arios e grupos e outros
01 -rwsr-sr-x daemon daemon /usr/bin/at
02 -rwsr-sr-x root mail /usr/bin/procmail
03 -rwsr-sr-x root root /usr/bin/X

mostra setuid ativado em usu\'arios
01 -rwsr-xr-x root root /bin/mount
02 -rwsr-xr-x root root /bin/ping
03 -rwsr-xr-x root root /bin/ping6
04 -rwsr-xr-x root root /bin/su
05 -rwsr-xr-x root root /bin/umount
06 -rwsr-xr-x root root /sbin/mount.nfs
07 -rwsr-sr-x daemon daemon /usr/bin/at
08 -rwsr-xr-x root root /usr/bin/chfn
09 -rwsr-xr-x root root /usr/bin/chsh
10 -rwsr-xr-x root root /usr/bin/gpasswd
11 -rwsr-xr-x root lpadmin /usr/bin/lppasswd
12 -rwsr-xr-x root root /usr/bin/newgrp
13 -rwsr-xr-x root root /usr/bin/passwd
14 -rwsr-xr-x root root /usr/bin/pkexec
15 -rwsr-sr-x root mail /usr/bin/procmail
16 -rwsr-xr-x root root /usr/bin/sudo
17 -rwsr-xr-x root root /usr/bin/sudoedit
18 -rwsr-sr-x root root /usr/bin/X
19 -rwsr-xr-- root messagebus /usr/lib/dbus-1.0/dbus-daemon-launch-helper
20 -rwsr-xr-x root root /usr/lib/eject/dmcrypt-get-device
21 -rwsr-xr-x root root /usr/lib/openssh/ssh-keysign
22 -rwsr-xr-x root root /usr/lib/policykit-1/polkit-agent-helper-1
23 -rwsr-xr-x root root /usr/lib/pt_chown
24 -rwsr-xr-x root root /usr/sbin/exim4

mostra setuid ativado em grupos
01 -rwxr-sr-x root shadow /sbin/unix_chkpwd
02 -rwsr-sr-x daemon daemon /usr/bin/at
03 -rwxr-sr-x root tty /usr/bin/bsd-write
04 -rwxr-sr-x root shadow /usr/bin/chage
05 -rwxr-sr-x root crontab /usr/bin/crontab
06 -rwxr-sr-x root mail /usr/bin/dotlockfile
07 -rwxr-sr-x root shadow /usr/bin/expiry
08 -rwxr-sr-x root mail /usr/bin/lockfile
09 -rwxr-sr-x root mlocate /usr/bin/mlocate
10 -rwxr-sr-x root mail /usr/bin/mutt_dotlock
11 -rwsr-sr-x root mail /usr/bin/procmail
12 -rwxr-sr-x root ssh /usr/bin/ssh-agent
13 -rwxr-sr-x root tty /usr/bin/wall
14 -rwsr-sr-x root root /usr/bin/X
15 -rwxr-sr-x root mail /usr/lib/evolution/camel-lock-helper-1.2
16 -rwxr-sr-x root utmp /usr/lib/libvte9/gnome-pty-helper
17 -rwxr-sr-x root tty /usr/lib/mc/cons.saver
```

Apêndice K

Anexos aos Apêndices

Apêndice incluído pelo tradutor.

K.1 Signal.h

O trecho abaixo foi retirado do /usr/include/asm/signal.h de um Ubuntu 10.04 LTS 32 *bits*:

```
#define SIGHUP  1
#define SIGINT  2
#define SIGQUIT  3
#define SIGILL  4
#define SIGTRAP  5
#define SIGABRT  6
#define SIGIOT  6
#define SIGBUS  7
#define SIGFPE  8
#define SIGKILL  9
#define SIGUSR1 10
#define SIGSEGV 11
#define SIGUSR2 12
#define SIGPIPE 13
#define SIGALRM 14
#define SIGTERM 15
#define SIGSTKFLT 16
#define SIGCHLD 17
#define SIGCONT 18
#define SIGSTOP 19
#define SIGTSTP 20
#define SIGTTIN 21
#define SIGTTOU 22
#define SIGURG 23
#define SIGXCPU 24
#define SIGXFSZ 25
#define SIGVTALRM 26
#define SIGPROF 27
#define SIGWINCH 28
#define SIGIO 29
#define SIGPOLL SIGIO
/*
#define SIGLOST 29
*/
#define SIGPWR 30
#define SIGSYS 31
#define SIGUNUSED 31

/* These should not be considered constants from userland.  */
#define SIGRTMIN 32
#define SIGRTMAX _NSIG
```

K.2 Analizadores de Código

Retirado de http://www.ccppbrasil.org/wiki/Analisadores_de_C%C3%B3digo

C

* CQual A tool for adding type qualifiers in C.
* SNav Red Hat Source Navigator.
* Sparse a tool designed to find faults in the Linux kernel.
* Splint an open source evolved version of Lint (C language).
* Frama-C Frama-C is a suite of tools dedicated to the analysis of the source code of software written in C.
* Astr\'ee - A tool for proving the absence of runtime errors (overflows, failed assertions, etc.), taylored to critical embedded control code (was applied to Airbus A340 and A380 avionics code)
* Deputy - Deputy is a C compiler that is capable of preventing common C programming errors, including out-of-bounds memory accesses as well as many other common type-safety errors.
* CCured - CCured is a source-to-source translator for C. It analyzes the C program to determine the smallest number of run-time checks that must be inserted in the program to prevent all memory safety violations.
* RATS - RATS is a tool for scanning C, C++, Perl, PHP and Python source code and flagging common security related programming errors such as buffer overflows and TOCTOU (Time Of Check, Time Of Use) race conditions.
* LLVM/Clang Static Analyzer - standalone tool that find bugs in C and Objective-C programs.
* MOPS - MOPS is a tool for finding security bugs in C programs and for verifying conformance to rules of defensive programming.
* BOON - BOON is a tool for automatically finding buffer overrun vulnerabilities in C source code.
* BLAST - BLAST is a software model checker for C programs.

C/C++

* ABRAXAS Software codeCheck programmable C/C++ Standards Checking Tool.
* CHECKMARX CxSuite - a Source Code Analysis suite of products allowing developers and auditors identify software security vulnerabilities.
* CMT++ code metrics tool for C/C++ (also for Java).
* CP Miner, sold commercially as Pattern Miner detects copy-paste errors and provides refactoring support for C and C++ code.
* FlexeLint and PC-Lint Multi-platform static code analysis tools for C and C++ code.
* Green Hills Software DoubleCheck static analysis for C and C++ code.
* HP Code Advisor A static analysis tool for C and C++ programs
* LDRA Testbed A software analysis and testing tool suite for C & C++.
* QA-C (and QA-C++) deep static analysis of C for quality assurance and guideline enforcement.
* SPARROW Semantic-based static analysis tool for C/C++ which automatically detects buffer overruns, memory leaks, etc.
* Viva64 analyzes C, C++ code for detect 64-bit portability issues.

Apêndice L

Licença de Livre Publicação

Apêndice incluído pelo tradutor

1. Licença de Livre Publicação Esta é uma tradução não-oficial da Open Publication Licence versão 1.0, de 8 de junho de 1999, e não é substituto legal para a Licença original, disponível em http://www.opencontent.org/openpub. Entretanto, esta tradução poderá auxiliar pessoas que falem Português a entender melhor a licença Openpub.

É permitido a qualquer pessoa copiar e distribuir cópias desse documento de licença, desde que sem a implementação de qualquer mudança.

OPEN PUBLIC LICENCE

Draft v1.0, 8 june 1999 1.1. Requisitos comuns às versões não-modificada e modificada Os trabalhos protegidos pela Licença de Livre Publicação (Open Publication Licence) podem ser reproduzidos e distribuídos no todo ou em parte, em qualquer meio físico ou eletrônico, desde que os termos desta licença estejam incluídos, e que esta licença ou uma incorporação dela por referência (com quaisquer das opções escolhidas pelo autor ou editor) estejam presentes na reprodução.

A forma apropriada para uma incorporação por referência é:

Note: Copyright(c) (ano) (nome do autor ou proprietário da obra). Este material somente poderá ser distribuído se sujeito aos termos e condições firmados na Licença de Livre Publicação (Open Publication Licence), versão X.Y ou superior (a versão mais atual encontra-se disponível em http://www.opencontent.org/openpub/).

Esta referência, devidamente preenchida com os dados da publicação, deve ser seguida imediatamente com quaisquer opções escolhidas pelos autores ou editor do documento.

É permitida a redistribuição comercial de material licenciado pela Licença de Livre Publicação (Open Publication Licence).

Qualquer publicação no formato livro padrão (papel) requer obrigatoriamente a citação dos autores e editor originais. Os nomes dos autores e do editor devem aparecer em todas as superfícies externas do livro. Em todas as faces externas do livro, o nome do editor original deve estar impresso em tamanho tão grande quanto o título do trabalho, e citado como proprietário em relação àquele título.

Copyright

O copyright de todo trabalho protegido pela Licença de Livre Publicação (Open Publication Licence) pertence aos autores ou proprietários.

Escopo desta licença

Os termos de licença a seguir aplicam-se a todos os trabalhos protegidos pela Licença de Livre Publicação (Open Publication Licence), a não ser que explicitamente indicado no trabalho.

A mera adição de trabalhos protegidos pela Licença de Livre Publicação (Open Publication Licence) ou partes de trabalhos protegidos pela Licença de Livre Publicação (Open Publication Licence) em uma mesma mídia que contenha outros trabalhos ou programas não protegidos por essa licença não decorre em aplicação da Licença de Livre Publicação (Open Publication Licence) para esses outros trabalhos. O trabalho resultante deve explicitamente conter uma nota especificando a inclusão do material protegido pela Licença de Livre Publicação (Open Publication Licence) e o aviso de copyright apropriado.

APLICABILIDADE. Se alguma parte desta licença não puder ser aplicada em alguma jurisdição, as partes restantes deste documento continuam sendo aplicadas.

AUSÊNCIA DE GARANTIA. Os trabalhos protegidos pela Licença de Livre Publicação (Open Publication Licence) são fornecidos "como estão", sem garantias de qualquer tipo, explícita ou implícita, incluindo, mas não limitado a, as garantias implícitas de comercialização e conveniência para um propósito particular, ou garantia de não-infração.

Requisitos para trabalhos modificados

Todas as versões modificadas de documentos cobertos por esta licença, incluindo traduções, antologias, compilações e documentação parcial, deve seguir os requisitos abaixo:

A versão modificada deve ser indicada como tal.

As pessoas que fizerem as modificações e as datas de modificação devem ser identificadas.

O reconhecimento dos autores e editor originais (se aplicável) deve ser mantido de acordo com as práticas acadêmicas usuais de citação.

O local da versão não-modificada do documento deve ser indicado.

Os nomes originais dos autores não devem ser utilizados para indicar ou garantir seu endosso ao documento resultante sem a autorização expressa dos autores.

Práticas recomendadas

Em adição aos requisitos desta licença, é solicitado e extremamente recomendado aos redistribuidores que:

Se os trabalhos protegidos pela Licença de Livre Publicação (Open Publication Licence) estiverem sendo distribuídos em impressos ou CD-ROM, os autores sejam informados por email, ao menos trinta dias antes, para que os autores tenham tempo de providenciar documentação atualizada. Esta notificação deve descrever as modificaçoes introduzidas no documento, se existirem.

Todas as modificações substanciais (incluindo exclusões) devem ser marcadas claramente no documento, ou então descritas em um anexo ao documento.

Finalmente, mesmo não sendo obrigatório sob esta licença, é considerado de bom tom oferecer uma cópia sem ônus de todo o material modificado (impresso e CD-ROM) para os autores originais.

Termos opcionais

Os autores e editores de documentos protegidos pela Licença de Livre Publicação (Open Publication Licence) podem escolher certas opções de licença simplesmente incluindo alguns parágrafos após a cópia da licença ou sua referência. Estas opções são consideradas parte da licença e devem ser incluídas com ela (ou com a referência a ela) nos trabalhos derivados.

As opções que se aplicam a este trabalho são:

A: É vedada a distribuição de versões com modificações substanciais deste documento sem a expressa permissão dos proprietários do direito autoral".

B: É vedada a distribuição deste trabalho ou qualquer derivado seu em qualquer formato de livro padrão (papel) sem a prévia autorização dos proprietários do direito autoral.

Políticas de Publicações Livres

(O texto a seguir não é considerado parte da licença.)

Os trabalhos protegidos pela Licença de Livre Publicação (Open Publication Licence) estão disponíveis na home page da Open Publication.

Os autores de trabalhos protegidos pela Licença de Livre Publicação (Open Publication Licence) podem incluir suas próprias licenças nesses trabalhos, desde que os termos dessa licença não sejam mais restritrivos que os da Licença de Livre Publicação (Open Publication Licence).

Em caso de dúvidas sobre a Licença de Livre Publicação (Open Publication Licence), contactar David Wiley ou a lista de autores de publicações livres via email.

Para se inscrever na lista de autores de publicações livres (Open Publication Author's List), mande um email para opal-request@opencontent.org com a palavra subscribe no corpo da mensagem.

Para enviar mensagens para a lista de autores de publicações livres (Open Publication Author's List), mande um email para opal@opencontent.org ou simplesmente responda a uma mensagem postada.

Para se desinscrever na lista de autores de publicações livres (Open Publication Author's List), mande um email para opal-request@opencontent.org com a palavra unsubscribe no corpo da mensagem.

Apêndice M

A Licença Pública Geral do GNU - pt_BR

Apêndice incluído pelo tradutor.

Licença Pública Geral do GNU (GPL) [General Public License]
This is an unofficial translation of the GNU General Public License into Portuguese. It was not published by the Free Software Foundation, and does not legally state the distribution terms for software that uses the GNU GPL–only the original English text of the GNU GPL does that. However, we hope that this translation will help Portuguese speakers understand the GNU GPL better.
Esta é uma tradução não-oficial da GNU General Public License para o Português. Ela não é publicada pela Free Software Foundation e não traz os termos de distribuição legal do software que usa a GNU GPL – estes termos estão contidos apenas no texto da GNU GPL original em inglês. No entanto, esperamos que esta tradução ajudará no melhor entendimento da GNU GPL em Português.
Versão 2, Junho de 1991 Direitos Autorais Reservados 1989, 1991 Free Software Foundation, Inc. 59 Temple Place, Suite [conjunto] 330, Boston, MA [Massachusetts] 02111-1307 USA [Estados Unidos da América]
É permitido a qualquer pessoa copiar e distribuir cópias sem alterações deste documento de licença, sendo vedada, entretanto, qualquer modificação.
Introdução
As licenças da maioria dos softwares são elaboradas para suprimir sua liberdade de compartilhá-los e modificá-los. A Licença Pública Geral do GNU, ao contrário, visa garantir sua liberdade de compartilhar e modificar softwares livres para assegurar que o software seja livre para todos os seus usuários. Esta Licença Pública Geral é aplicável à maioria dos softwares da Free Software Foundation [Fundação do Software Livre] e a qualquer outro programa cujos autores se comprometerem a usá-la. (Em vez dela, alguns outros softwares da Free Software Foundation são cobertos pela Licença Pública Geral de Biblioteca do GNU). Você também poderá aplicá-la aos seus programas.
Quando falamos de software livre, estamos nos referindo à liberdade, não ao preço. Nossas Licenças Públicas Gerais visam garantir que você tenha a liberdade de distribuir cópias de software livre (e cobrar por isso se desejar), que receba código-fonte ou possa obtê-lo se desejar, que possa modificá-lo ou usar partes dele em novos programas livres; finalmente, que você tenha ciência de que pode fazer tudo isso.
Para proteger seus direitos, necessitamos fazer restrições que proíbem que alguém negue esses direitos a você ou que solicite que você renuncie a eles. Essas restrições se traduzem em determinadas responsabilidades que você deverá assumir, se for distribuir cópias do software ou modificá-lo.
Por exemplo, se você distribuir cópias de algum desses programas, tanto gratuitamente como mediante uma taxa, você terá de conceder aos receptores todos os direitos que você possui. Você terá de garantir que, também eles, recebam ou possam obter o código-fonte. E você terá a obrigação de exibir a eles esses termos, para que eles conheçam seus direitos.
Protegemos seus direitos através de dois passos: (1) estabelecendo direitos autorais sobre o software e (2) concedendo a você esta licença, que dá permissão legal para copiar, distribuir e/ou modificar o software.
Além disso, para a proteção de cada autor e a nossa, queremos ter certeza de que todos entendam que não há nenhuma garantia para este software livre. Se o software for modificado por alguém e passado adiante, queremos que seus receptores saibam que o que receberam não é o original, de forma que quaisquer problemas introduzidos por terceiros não afetem as reputações dos autores originais.
Finalmente, qualquer programa livre é constantemente ameaçado por patentes de software. Queremos evitar o risco de que redistribuidores de um programa livre obtenham individualmente licenças sob uma patente, tornando o programa, com efeito, proprietário. Para impedir isso, deixamos claro que qualquer patente deve ser licenciada para o uso livre por parte de qualquer pessoa ou, então, simplesmente não deve ser licenciada.
Os exatos termos e condições para cópia, distribuição e modificação seguem abaixo.
TERMOS E CONDIÇÕES PARA CPIA, DISTRIBUIÇÃO E MODIFICAÇÃO
0.Esta Licença se aplica a qualquer programa ou outra obra que contenha um aviso inserido pelo respectivo titular dos direitos autorais, informando que a referida obra pode ser distribuída em conformidade com os termos desta Licença Pública Geral. O termo "Programa", utilizado abaixo, refere-se a qualquer programa ou obra, e o termo "obras baseadas no Programa"significa tanto o Programa, como qualquer obra derivada nos termos da legislação de direitos autorais: isto é, uma obra contendo o Programa ou uma parte dele, tanto de forma idêntica como com modificações, e/ou traduzida para outra linguagem. (Doravante, o termo "modificação"inclui também, sem reservas, a tradução). Cada licenciado, doravante, será denominado "você".
Outras atividades que não a cópia, distribuição e modificação, não são cobertas por esta Licença; elas estão fora de seu escopo. O ato de executar o Programa não tem restrições e o resultado gerado a partir do Programa encontra-se coberto

somente se seu conteúdo constituir uma obra baseada no Programa (independente de ter sido produzida pela execução do Programa). Na verdade, isto dependerá daquilo que o Programa faz.

1.Você poderá fazer cópias idênticas do código-fonte do Programa ao recebê-lo e distribui-las, em qualquer mídia ou meio, desde que publique, de forma ostensiva e adequada, em cada cópia, um aviso de direitos autorais (ou copyright) apropriado e uma notificação sobre a exoneração de garantia; mantenha intactas as informações, avisos ou notificações referentes a esta Licença e à ausência de qualquer garantia; e forneça a quaisquer outros receptores do Programa uma cópia desta Licença junto com o Programa.

Você poderá cobrar um valor pelo ato físico de transferir uma cópia, e você pode oferecer, se quiser, a proteção de uma garantia em troca de um valor.

2.Você poderá modificar sua cópia ou cópias do Programa ou qualquer parte dele, formando, dessa forma, uma obra baseada no Programa, bem como copiar e distribuir essas modificações ou obra, de acordo com os termos da Cláusula 1 acima, desde que você também atenda a todas as seguintes condições:

a.Você deve fazer com que os arquivos modificados contenham avisos, em destaque, informando que você modificou os arquivos, bem como a data de qualquer modificação.

b.Você deve fazer com que qualquer obra que você distribuir ou publicar, que no todo ou em parte contenha o Programa ou seja dele derivada, ou derivada de qualquer parte dele, seja licenciada como um todo sem qualquer custo para todos terceiros nos termos desta licença.

c.Se o programa modificado normalmente lê comandos interativamente quando executado, você deverá fazer com que ele, ao começar a ser executado para esse uso interativo em sua forma mais simples, imprima ou exiba um aviso incluindo o aviso de direitos autorais (ou copyright) apropriado, além de uma notificação de que não há garantia (ou, então, informando que você oferece garantia) e informando que os usuários poderão redistribuir o programa de acordo com essas condições, esclarecendo ao usuário como visualizar uma cópia desta Licença. (Exceção: se o Programa em si for interativo mas não imprimir normalmente avisos como esses, não é obrigatório que a sua obra baseada no Programa imprima um aviso).

Essas exigências se aplicam à obra modificada como um todo. Se partes identificáveis dessa obra não forem derivadas do Programa e puderem ser consideradas razoavelmente como obras independentes e separadas por si próprias, nesse caso, esta Licença e seus termos não se aplicarão a essas partes quando você distribui-las como obras separadas. Todavia, quando você distribui-las como parte de um todo que constitui uma obra baseada no Programa, a distribuição deste todo terá de ser realizada em conformidade com esta Licença, cujas permissões para outros licenciados se estenderão à obra por completo e, conseqüentemente, a toda e qualquer parte, independentemente de quem a escreveu.

Portanto, esta cláusula não tem a intenção de afirmar direitos ou contestar os seus direitos sobre uma obra escrita inteiramente por você; a intenção é, antes, de exercer o direito de controlar a distribuição de obras derivadas ou obras coletivas baseadas no Programa.

Além do mais, a simples agregação de outra obra que não seja baseada no Programa a ele (ou a uma obra baseada no Programa) em um volume de mídia ou meio de armazenamento ou distribuição, não inclui esta outra obra no âmbito desta Licença.

3.Você poderá copiar e distribuir o Programa (ou uma obra baseada nele, de acordo com a Cláusula 2) em código-objeto ou formato executável de acordo com os termos das Cláusulas 1 e 2 acima, desde que você também tome uma das providências seguintes:

a.Incluir o código-fonte correspondente completo, passível de leitura pela máquina, o qual terá de ser distribuído de acordo com as Cláusulas 1 e 2 acima, em um meio ou mídia habitualmente usado para intercâmbio de software; ou,

b.Incluir uma oferta por escrito, válida por pelo menos três anos, para fornecer a qualquer terceiro, por um custo que não seja superior ao seu custo de fisicamente realizar a distribuição da fonte, uma cópia completa passível de leitura pela máquina, do código-fonte correspondente, a ser distribuído de acordo com as Cláusulas 1 e 2 acima, em um meio ou mídia habitualmente usado para intercâmbio de software; ou,

c.Incluir as informações recebidas por você, quanto à oferta para distribuir o código-fonte correspondente. (Esta alternativa é permitida somente para distribuição não-comercial e apenas se você tiver recebido o programa em código-objeto ou formato executável com essa oferta, de acordo com a letra b, acima).

O código-fonte de uma obra significa o formato preferencial da obra para que sejam feitas modificações na mesma. Para uma obra executável, o código-fonte completo significa o código-fonte inteiro de todos os módulos que ela contiver, mais quaisquer arquivos de definição de interface associados, além dos scripts usados para controlar a compilação e instalação do executável. Entretanto, como uma exceção especial, o código-fonte distribuído não precisa incluir nada que não seja normalmente distribuído (tanto no formato fonte como no binário) com os componentes principais (compilador, kernel e assim por diante) do sistema operacional no qual o executável é executado, a menos que este componente em si acompanhe o executável.

Se a distribuição do executável ou código-objeto for feita mediante a permissão de acesso para copiar, a partir de um local designado, então, a permissão de acesso equivalente para copiar o código-fonte a partir do mesmo local será considerada como distribuição do código-fonte, mesmo que os terceiros não sejam levados a copiar a fonte junto com o código-objeto.

4.Você não poderá copiar, modificar, sublicenciar ou distribuir o Programa, exceto conforme expressamente estabelecido nesta Licença. Qualquer tentativa de, de outro modo, copiar, modificar, sublicenciar ou distribuir o Programa será inválida, e automaticamente rescindirá seus direitos sob esta Licença. Entretanto, terceiros que tiverem recebido cópias ou direitos de você de acordo esta Licença não terão suas licenças rescindidas, enquanto estes terceiros mantiverem o seu pleno cumprimento.

5.Você não é obrigado a aceitar esta Licença, uma vez que você não a assinou. Porém, nada mais concede a você permissão para modificar ou distribuir o Programa ou respectivas obras derivativas. Tais atos são proibidos por lei se você não aceitar esta Licença. Conseqüentemente, ao modificar ou distribuir o Programa (ou qualquer obra baseada no Programa), você estará manifestando sua aceitação desta Licença para fazê-lo, bem como de todos os seus termos e condições para copiar, distribuir ou modificar o Programa ou obras nele baseadas.

6.Cada vez que você redistribuir o Programa (ou obra baseada no Programa), o receptor receberá, automaticamente, uma licença do licenciante original, para copiar, distribuir ou modificar o Programa, sujeito a estes termos e condições. Você não poderá impor quaisquer restrições adicionais ao exercício, pelos receptores, dos direitos concedidos por este instrumento. Você não tem responsabilidade de promover o cumprimento por parte de terceiros desta licença.

7.Se, como resultado de uma sentença judicial ou alegação de violação de patente, ou por qualquer outro motivo (não restrito às questões de patentes), forem impostas a você condições (tanto através de mandado judicial, contrato ou qualquer outra forma) que contradigam as condições desta Licença, você não estará desobrigado quanto às condições desta Licença. Se você não puder atuar como distribuidor de modo a satisfazer simultaneamente suas obrigações sob esta licença e quaisquer outras obrigações pertinentes, então, como conseqüência, você não poderá distribuir o Programa de nenhuma forma. Por exemplo, se uma licença sob uma patente não permite a redistribuição por parte de todos aqueles que tiverem recebido cópias, direta ou indiretamente de você, sem o pagamento de royalties, então, a única forma de cumprir tanto com esta exigência quanto com esta licença será deixar de distribuir, por completo, o Programa.

Se qualquer parte desta Cláusula for considerada inválida ou não executável, sob qualquer circunstância específica, o restante da cláusula deverá continuar a ser aplicado e a cláusula, como um todo, deverá ser aplicada em outras circunstâncias.
Esta cláusula não tem a finalidade de induzir você a infringir quaisquer patentes ou direitos de propriedade, nem de contestar a validade de quaisquer reivindicações deste tipo; a única finalidade desta cláusula é proteger a integridade do sistema de distribuição do software livre, o qual é implementado mediante práticas de licenças públicas. Muitas pessoas têm feito generosas contribuições à ampla gama de software distribuído através desse sistema, confiando na aplicação consistente deste sistema; cabe ao autor/doador decidir se deseja distribuir software através de qualquer outro sistema e um licenciado não pode impor esta escolha.
Esta cláusula visa deixar absolutamente claro o que se acredita ser uma conseqüência do restante desta Licença.
8.Se a distribuição e/ou uso do Programa for restrito em determinados países, tanto por patentes ou por interfaces protegidas por direito autoral, o titular original dos direitos autorais que colocar o Programa sob esta Licença poderá acrescentar uma limitação geográfica de distribuição explícita excluindo esses países, de modo que a distribuição seja permitida somente nos países ou entre os países que não foram excluídos dessa forma. Nesse caso, esta Licença passa a incorporar a limitação como se esta tivesse sido escrita no corpo desta Licença.
9.A Free Software Foundation poderá de tempos em tempos publicar novas versões e/ou versões revisadas da Licença Pública Geral. Essas novas versões serão semelhantes em espírito à presente versão, mas podem diferenciar-se, porém, em detalhe, para tratar de novos problemas ou preocupações.
Cada versão recebe um número de versão distinto. Se o Programa especificar um número de versão desta Licença que se aplique a ela e a "qualquer versão posterior", você terá a opção de seguir os termos e condições tanto daquela versão como de qualquer versão posterior publicada pela Free Software Foundation. Se o Programa não especificar um número de versão desta Licença, você poderá escolher qualquer versão já publicada pela Free Software Foundation.
10.Se você desejar incorporar partes do Programa em outros programas livres cujas condições de distribuição sejam diferentes, escreva ao autor solicitando a respectiva permissão. Para software cujos direitos autorais sejam da Free Software Foundation, escreva para ela; algumas vezes, abrimos exceções para isso. Nossa decisão será guiada pelos dois objetivos de preservar a condição livre de todos os derivados de nosso software livre e de promover o compartilhamento e reutilização de software, de modo geral.
EXCLUSÃO DE GARANTIA
11.COMO O PROGRAMA É LICENCIADO SEM CUSTO, NÃO HÁ NENHUMA GARANTIA PARA O PROGRAMA, NO LIMITE PERMITIDO PELA LEI APLICÁVEL. EXCETO QUANDO DE OUTRA FORMA ESTABELECIDO POR ESCRITO, OS TITULARES DOS DIREITOS AUTORAIS E/OU OUTRAS PARTES, FORNECEM O PROGRAMA "NO ESTADO EM QUE SE ENCONTRA", SEM NENHUMA GARANTIA DE QUALQUER TIPO, TANTO EXPRESSA COMO IMPLÍCITA, INCLUINDO, DENTRE OUTRAS, AS GARANTIAS IMPLÍCITAS DE COMERCIABILIDADE E ADEQUAÇÃO A UMA FINALIDADE ESPECÍFICA. O RISCO INTEGRAL QUANTO À QUALIDADE E DESEMPENHO DO PROGRAMA É ASSUMIDO POR VOCÊ. CASO O PROGRAMA CONTENHA DEFEITOS, VOCÊ ARCARÁ COM OS CUSTOS DE TODOS OS SERVIÇOS, REPAROS OU CORREÇÕES NECESSÁRIAS.
12.EM NENHUMA CIRCUNSTÂNCIA, A MENOS QUE EXIGIDO PELA LEI APLICÁVEL OU ACORDADO POR ESCRITO, QUALQUER TITULAR DE DIREITOS AUTORAIS OU QUALQUER OUTRA PARTE QUE POSSA MODIFICAR E/OU REDISTRIBUIR O PROGRAMA, CONFORME PERMITIDO ACIMA, SERÁ RESPONSÁVEL PARA COM VOCÊ POR DANOS, INCLUINDO ENTRE OUTROS, QUAISQUER DANOS GERAIS, ESPECIAIS, FORTUITOS OU EMERGENTES, ADVINDOS DO USO OU IMPOSSIBILIDADE DE USO DO PROGRAMA (INCLUINDO, ENTRE OUTROS, PERDAS DE DADOS OU DADOS SENDO GERADOS DE FORMA IMPRECISA, PERDAS SOFRIDAS POR VOCÊ OU TERCEIROS OU A IMPOSSIBILIDADE DO PROGRAMA DE OPERAR COM QUAISQUER OUTROS PROGRAMAS), MESMO QUE ESSE TITULAR, OU OUTRA PARTE, TENHA SIDO ALERTADA SOBRE A POSSIBILIDADE DE OCORRÊNCIA DESSES DANOS.
FINAL DOS TERMOS E CONDIÇÕES
Como Aplicar Estes Termos para Seus Novos Programas
Se você desenvolver um programa novo e quiser que ele seja da maior utilidade possível para o público, o melhor caminho para obter isto é fazer dele um software livre, o qual qualquer pessoa pode redistribuir e modificar sob os presentes termos.
Para fazer isto, anexe as notificações seguintes ao programa. É mais seguro anexá-las ao começo de cada arquivo-fonte, de modo a transmitir do modo mais eficiente a exclusão de garantia; e cada arquivo deve ter ao menos a linha de "direitos autorais reservados" e uma indicação de onde a notificação completa se encontra.
¡uma linha para informar o nome do programa e uma breve idéia do que ele faz.¿
Direitos Autorais Reservados (c) ¡ano¿ ¡nome do autor¿
Este programa é software livre; você pode redistribuí-lo e/ou modificá-lo sob os termos da Licença Pública Geral GNU conforme publicada pela Free Software Foundation; tanto a versão 2 da Licença, como (a seu critério) qualquer versão posterior.
Este programa é distribuído na expectativa de que seja útil, porém, SEM NENHUMA GARANTIA; nem mesmo a garantia implícita de COMERCIABILIDADE OU ADEQUAÇÃO A UMA FINALIDADE ESPECÍFICA. Consulte a Licença Pública Geral do GNU para mais detalhes.
Você deve ter recebido uma cópia da Licença Pública Geral do GNU junto com este programa; se não, escreva para a Free Software Foundation, Inc., no endereço 59 Temple Street, Suite 330, Boston, MA 02111-1307 USA.
Inclua também informações sobre como contatar você por correio eletrônico e por meio postal.
Se o programa for interativo, faça com que produza uma pequena notificação como esta, quando for iniciado em um modo interativo:
Versão 69 do Gnomovision, Direitos Autorais Reservados (c) ano nome do autor. O Gnomovision NÃO POSSUI QUALQUER TIPO DE GARANTIA; para detalhes, digite 'show w'. Este é um software livre e você é bem-vindo para redistribuí-lo sob certas condições; digite 'show c' para detalhes.
Os comandos hipotéticos 'show w' e 'show c' devem mostrar as partes apropriadas da Licença Pública Geral.
Naturalmente, os comandos que você utilizar poderão ter outras denominações que não 'show w' e 'show c'; eles poderão até ser cliques do mouse ou itens de um menu - o que for adequado ao seu programa.
Você também pode solicitar a seu empregador (se você for um programador) ou sua instituição acadêmica, se for o caso, para assinar uma "renúncia de direitos autorais" sobre o programa, se necessário. Segue um exemplo; altere os nomes:
A Yoyodyne Ltda., neste ato, renuncia a todos eventuais direitos autorais sobre o programa 'Gnomovision' (que realiza passagens em compiladores), escrito por James Hacker. ¡Assinatura de Ty Coon¿ 1 de abril de 1989, Ty Coon, Presidente
Esta Licença Pública Geral não permite a incorporação do seu programa a programas proprietários. Se seu programa é uma biblioteca de sub-rotinas, você poderá considerar ser mais útil permitir a ligação de aplicações proprietárias à sua biblioteca. Se isso é o que você deseja fazer, utilize a Licença Pública Geral de Biblioteca do GNU, ao invés desta Licença.

Referências Bibliográficas

[K & R (1989)] KERNIGHAN, B. W. & RITCHIE, D. M.; Tradução de Daniel Vieira. *C, A Linguagem De Programação: Padrao ANSI*. Rio de Janeiro: Editora Campus,1989.

[Gorman (2004)] Gorman, Mel. *Understanding the Linux Virtual Memory Manager*.p. cm.–(Bruce Perens' Open source series). ISBN 0-13-145348-3

[Djairo (1985)] Figueiredo, Djairo Guedes de. *Números Irracionais e Trancendentes*.Brasília, Sociedade Brasileira de Matemática, pref. 1980.

Índice Remissivo

/dev/full, 173
/dev/loop, dispositivos de loopback, 175
/dev/null, dispositivo nulo, 171
/dev/pts (PTYs), 179
/dev/random, 173, 266
/dev/urandom, 173
/dev/zero, 172, 175
/etc/services - arquivo, 155
/proc, 183

CFLAGS, 12, 14, 15

Descritores de arquivo, 30, 37, 41, 43, 45, 77, 133, 134, 138, 139, 141, 147, 148, 155, 167, 171, 180, 181, 186, 193, 201, 203, 214, 216, 218, 229, 266, 282, 291, 294, 295, 334, 337, 339, 341, 342, 344, 346, 349, 350
Dispositivo de números aleatórios, 266
dispositivo de números aleatórios, 173

GCC, 7–9, 11, 19, 47, 51, 198, 235, 238, 239, 241, 244

Internet, 32, 49, 55, 89, 145–147, 153, 154, 245, 263, 269, 276, 287, 291, 304, 318, 359

NULL, 16, 26, 32, 55, 63, 78, 82, 89, 92, 98, 100, 125, 133, 189, 221, 227, 351

sistema de arquivos, 183

Variável do make, 12, 14, 15

Colofão As ruínas das casas de banho Stabian em Pompéia, capturada pelo fotógrafo Mel Curtis, são ostradas na capa desse livro. Diz-se serem as maiores e mais antigas casas de banho, os banhos Stabian também oferecia mensagens e leituras poéticas. Os moradores de Pompéia visitaram esses banhos públicos diáriamente. Os banhos receberam esse nome pelo fato de estarem localizados na rua Stabian.
Esse livro foi escrito e editado em LaTeX, e então convertido para Microsoft Word pela New Riders e liberados na QuarkXPress. A fonte usada para o corpo do texto é a Bembo e a MCPdigital. O livro foi impresso em papel 50# Husky Offset Smooth na R.R. Donnelley & Sons em Crawfordsville, Indiana. A pré-impressão consistiu da tecnologia PostScript computer-to-plate (processo filmless). A capa foi impressa na Moore Langen Printing em Terre Haute, Indiana, na Carolina, somente em um lado da folha.